Agentic State

Context, Memory, and Control in Adaptive AI

A Technical Monograph on Conditioning, Tool Execution, Verification, and Self-Improvement

Technical Monograph

Literature cutoff: 29 October 2026
Latest source retrieval: 29 October 2026

Abstract

This monograph analyzes agentic computation as a versioned, partially observed control system. Across 34 chapters, it connects model architecture and learning to context construction, session and persistent memory, procedural skills, tool interfaces, orchestration, evaluation, security, alignment, self-improvement, and deployment. The unit of analysis is the coupled system: a model checkpoint alone does not identify the information available to a decision, the operation permitted to execute, or the evidence supporting its outcome.

The central synthesis separates proposal-law preservation, authority nonexpansion, and continuity of operation identities and effect status. These contracts distinguish a transformation that preserves state from a system that makes correct decisions. Conditional-information and change-of-measure arguments identify the assumptions needed for bounded decision and trajectory comparisons. Training likelihoods must distinguish the actual decoder and behavior-time inputs from a reconstructed transcript. Trusted authorization and effect state are distinguished from model-visible evidence; checkpoint restoration must not revive revoked grants, and a missing receipt does not prove that an external effect is absent. These distinctions connect context–memory–policy consistency to retrieval, cache validity, skill migration, and service-supported reconciliation.

Established results are distinguished from original synthesis, analytical deductions, implementation contracts, counterexamples, and proposed experiments. Author-reported findings retain their source and population limits; source inspection is not experimental reproduction. Model and harness comparisons specify adaptation, evaluator access, baselines, and complete resource boundaries. Resource analyses account for computation, parameter and optimizer memory, KV growth, bandwidth, latency, throughput, and cost under declared hardware and workload assumptions. No new experimental measurements are reported. Finite benchmark gains and improvement trajectories do not establish unrestricted AGI, arbitrary interface robustness, or indefinite recursive improvement.

Notation and evidence

The default symbols are H_t for recorded history, C_t for working context, M_t for persistent memory, E_t for environment state, S_t for a specified system state, θ for model-policy parameters, and δ for deployment decoding configuration. The compatibility manifest ν_t is distinct from the current authority epoch α_t; λ denotes an operation-effect ledger where specified. Local definitions take precedence where a chapter explicitly introduces another conventional meaning. A deployed proposal law includes its declared decoding, stopping, and parsing semantics; it need not equal a raw token likelihood or the law of authorized dispatches. Units and workload assumptions accompany resource calculations. A calculated example is not a measured benchmark result.

The literature cutoff and latest source-retrieval date are 29 October 2026 for the complete monograph. Foundational references support mathematics and mechanisms whose assumptions remain applicable; frontier claims are tied to identified papers, releases, and inspected revisions. An identifier, retrieval date, or repository inspection does not establish that a claim has been independently reproduced or remains valid under a different configuration.

Four evidence classes are distinguished: author-reported results, independently reproduced results, analytical deductions, and proposed experiments. This edition claims no independent experimental reproduction. Bibliographic identity checks, implementation inspection, and experimental reproduction are different evidential acts. Missing implementation details and unsupported inferences are identified rather than supplied by assumption. Proposed tests and mathematical bounds do not certify properties beyond their stated information, sampling, and enforcement premises.

Acknowledgments and computational assistance

OpenAI Codex agents assisted source analysis, technical drafting, mathematical and editorial checks, and LaTeX preparation throughout Chapters 1–34 and the front matter. Explanatory illustrations were generated through OpenAI's built-in image-generation tool. The interface did not expose the image model identifier; no exact image-model version is asserted. The illustrations communicate mechanisms and contain no experimental measurements. These tools are not authors, and their use does not establish human expert endorsement. Responsibility for a submitted edition, its authorship, source verification, and final disclosures remains with its named human authors.

Contents

1 Agentic Computation: System Boundaries, Generality, and Evidence **1**

1.1 The agent is the unit of analysis 1
1.2 A stateful formulation of execution 2
1.3 Generality as a profile of conditional competence 4
1.4 An ontology of models and connecting components . . . 7
1.5 What constitutes evidence in an evolving ecosystem . . 9
1.6 Three consistency contracts and their non-implication . 13
1.7 Cross-layer questions that organize the analysis 18

2 Probabilistic State, Decision Processes, and Statistical Identification **20**

2.1 Conditional prediction and observable information . . . 20
2.2 Information loss and decision-relevant state 22
2.3 Partial observability and beliefs 26
2.4 Optimization and policy-gradient identities 28
2.5 Risk calibration and constrained decisions 29
2.6 Statistical comparison and adaptive selection 31
2.7 From equations to falsifiable implementation claims . . . 33

3 Foundation Model Policies: Tokenization, Architectures, and Action Interfaces **35**

3.1 DeepSeek V4.1 and architecture level prefill reuse 35
3.2 The model distribution and the executed policy 36
3.3 Tokenization as a measurement and interface choice . . 38
3.4 Architectural state and access to history 39
3.5 A resource model at the policy boundary 44

3.6 From model output to a typed operation 45
3.7 Reproducible implementation and discriminating experiments 47
3.8 Implications for stateful agents 48

4 IO-Aware Attention Kernels and Computational State Allocation 51
4.1 FlashAttention 4 and asymmetric hardware scaling . . . 51
4.2 The computation being optimized 52
4.3 KV growth and the difference between storage and knowledge 56
4.4 Paged allocation and shared prefixes 57
4.5 Bandwidth limits and latency distributions 58
4.6 Approximate caches and a bounded perturbation argument 59
4.7 Implementation verification and workload design 62
4.8 When recomputation is mathematically necessary 64

5 Predictive Representation Learning and Multimodal State Abstraction 67
5.1 Dense predictive features and world action coupling in 2026 67
5.2 Representation quality is relative to a decision 69
5.3 Patch encoders and the cost of visual detail 74
5.4 Contrastive alignment and what it identifies 75
5.5 Joint embedding prediction and the collapse question . . 76
5.6 Multimodal fusion and executable grounding 77
5.7 Temporal sampling and observation freshness 78
5.8 Implementation and controlled comparison 79

6 Action-Conditioned World Models, Diffusion, and Model-Based Control 84
6.1 LeFlow and amortized latent trajectory planning 84
6.2 Prediction under intervention 85
6.3 Compounding error and a finite-horizon bound 88
6.4 Diffusion as a conditional generative mechanism 89
6.5 Flow matching and trajectory parameterization 91
6.6 Executable worlds for language-model agents 92
6.7 Experiments that separate prediction from control . . . 96

6.8 Practical limits of imagination 99

7 Pretraining Objectives, Scaling, and Distributed Optimization 101

7.1 DeepSeek V4 staged pretraining and distributed state . 101
7.2 What pretraining supplies to an agent 102
7.3 Scaling laws under an explicit compute model 104
7.4 Optimizer state and distributed memory accounting . . 105
7.5 Communication and parallel decomposition 106
7.6 Numerical stability and reproducibility 107
7.7 A worked training-budget calculation 111
7.8 Experiments linking infrastructure to behavior 113

8 Post-Training Objectives for Interactive and Tool-Using Policies 116

8.1 On policy distillation with precision matched rollout . . 116
8.2 Behavioral targets and trajectory supervision 119
8.3 Policy gradients and the likelihood-ratio contract 121
8.4 Group-relative advantages and sparse outcomes 122
8.5 Executable rewards and verifier error 123
8.6 Context edits and post-training consistency 124
8.7 Implementation paths and controlled training studies . . 125
8.8 Credit assignment across tools and memory 130

9 Inference-Time Search, Verification, and Metareasoning 132

9.1 Meta reasoning controllers and artifact selection in 2026 132
9.2 A conditional computation rather than a verbal certificate 133
9.3 Sampling coverage and repeated reliability 134
9.4 Self-consistency and answer aggregation 135
9.5 Selection error grows with the search space 136
9.6 Structured search and the granularity of a thought . . . 139
9.7 Value of computation and stopping 141
9.8 Experiments and failure attribution 143

10 Closed-Loop Planning with Partial Observability and Fallible Feedback 145

10.1 WorldEvolver and memory conditioned foresight 145

10.2 Goals must refer to external state 147
10.3 Belief states and information-gathering actions 148
10.4 Plan contracts and compositional verification 149
10.5 Receding-horizon execution and grounded feedback . . . 150
10.6 Self-correction can also damage correct work 151
10.7 Hierarchical plans and reusable skills 152
10.8 Partial effects and recovery planning 153
10.9 A reproducible planning evaluation 154

11 Long-Context Capability and Decision-Relevant Information 158
11.1 Capacity is a constraint on an inference procedure . . . 158
11.2 A sufficiency criterion for retained state 159
11.3 Position effects and compositional difficulty 161
11.4 Training distributions and length generalization 163
11.5 Computation and memory over an interaction horizon . 164
11.6 Experimental designs that isolate context use 166
11.7 A worked capacity and decision example 168
11.8 Effective context as controlled state 169

12 Context Transformation: Selection, Compaction, and Policy-Controlled Editing 171
12.1 The working view as a controlled variable 171
12.2 Selection before abstraction 172
12.3 Reversible offload and explicit context actions 174
12.4 Compaction triggers and delayed credit 176
12.5 Editing and cache validity 177
12.6 A worked decision-preservation analysis 179
12.7 Evidence comparisons and causal boundaries 180
12.8 Failure recovery and implementation discipline 184

13 Session Semantics, Event Histories, and Recoverable Execution State 186
13.1 Session identity is a semantic boundary 186
13.2 Events as evidence rather than prose 187
13.3 Reducers checkpoints and replay 188
13.4 External effects and the ambiguous timeout 190

13.5 Concurrency ownership and session forks 193
13.6 Retention deletion and provenance boundaries 195
13.7 A failure-oriented evaluation protocol 196
13.8 A reference contract for continuity 198

14 Persistent Memory: Writing, Consolidation, Validity, and Influence 200
14.1 Memory functions and physical substrates 200
14.2 Admission as a constrained decision 201
14.3 Episodic records and abstraction loss 204
14.4 Semantic memory and the status of a proposition 205
14.5 Hierarchy access and complete lifecycle cost 207
14.6 Temporal validity conflicts and forgetting 208
14.7 Retrieval does not imply appropriate use 210
14.8 Memory transactions and a reference test suite 213

15 Procedural Memory and the Lifecycle of Executable Skills 215
15.1 A procedure has preconditions and effects 215
15.2 Extracting procedures from experience 216
15.3 Hypergraphs preserve trajectory membership 218
15.4 Quality-guided maintenance and its assumptions 220
15.5 An inspectable execution trace 222
15.6 Reported results and unresolved attribution 223
15.7 Compatibility authority and the skill supply chain . . . 224
15.8 Consolidation as controlled hypothesis revision 226

16 Retrieval Policies, Evidence Assembly, and Grounded Execution 228
16.1 Retrieval selects evidence for an action 228
16.2 Index units and information boundaries 230
16.3 Query construction and adaptive access 232
16.4 State-conditioned procedural retrieval 235
16.5 Evidence packing attribution and influence 236
16.6 A worked multi-step evidence dependency 237
16.7 Evaluation contamination and uncertainty 238
16.8 An execution contract for grounded retrieval 240

17 Conditioning Consistency across Context, Memory, and Policy Updates 242
17.1 One decision depends on several versions 242
17.2 The conditioning record is part of the data 244
17.3 Likelihood ratios require support and the right history . 246
17.4 A bounded behavior-preservation proposition 248
17.5 Memory transactions and semantic validity 250
17.6 Model index and tool migration 251
17.7 Authority must survive every transformation 253
17.8 A consistency audit with causal counterexamples 254

18 Tool Discovery, Routing, and Typed Execution Contracts 256
18.1 A tool call is a typed state transition 256
18.2 Discovery changes the action space 257
18.3 Routing as constrained resource allocation 258
18.4 Argument construction and result validation 260
18.5 Execution granularity and local programs 261
18.6 Retry pruning and failure memory 263
18.7 Privilege control at the dispatch boundary 264
18.8 A tool-use experiment that can falsify the mechanism . 266

19 Versioned Interoperability: MCP, A2A, and Protocol Boundaries 268
19.1 Protocol compatibility is not agent competence 268
19.2 Modern MCP declares versions on every request 269
19.3 Tools resources prompts and skills remain distinct . . . 272
19.4 Transport statelessness and explicit durable work 273
19.5 Authorization is separate from declared identity 274
19.6 A2A tasks artifacts and delegated autonomy 275
19.7 A concrete cross-protocol execution trace 276
19.8 Conformance performance and residual uncertainty . . . 279

20 Agent Harnesses: Control Flow, Isolation, and Durable Effects 281
20.1 The effective policy includes the harness 281
20.2 Execution state and semantic commitments 283

20.3 Crash consistency and uncertain effects 284
20.4 Context construction is runtime scheduling 287
20.5 Budgets cancellation and backpressure 288
20.6 Implementation boundaries and evidence 290
20.7 A worked analysis of a recoverable repository workflow . 292

21 Multi-Agent Coordination, Information Routing, and Shared State 295
21.1 Task graphs and the value of decomposition 295
21.2 Independence diversity and correlated error 297
21.3 Messages as evidence-bearing interfaces 298
21.4 Shared memory and conflicting writes 302
21.5 Ownership cancellation and failure propagation 303
21.6 Experiments that isolate coordination mechanisms . . . 304
21.7 A bounded integration protocol 305
21.8 When isolation is the stronger coordination mechanism . 307

22 Agent Development Frameworks, SDKs, and Managed Runtimes 310
22.1 A semantic basis for framework comparison 310
22.2 Graph runtimes and opinionated harnesses 312
22.3 Typed SDKs and model-owned loops 313
22.4 Development kits interoperability and language boundaries 314
22.5 Role-specific framework and runtime inventory 315
22.6 Licensing lifecycle and migration evidence 319
22.7 A conformance suite before performance ranking 322
22.8 From public API to executable control path 323

23 Repository, Terminal, and Computer-Use Agents 326
23.1 Repository state as an executable environment 326
23.2 Specifications tests and evaluator blind spots 328
23.3 Terminal trajectories and learned execution policies . . . 330
23.4 Computer use grounding and hidden state 332
23.5 Coding product families and observable tests 333
23.6 A budget-matched evaluation protocol 337
23.7 Artifact authority and reviewable completion 338
23.8 A concrete grading failure and its discriminating controls 339

24 Research, Enterprise, and Domain-Specific Agent Systems 341
24.1 Research as evidence acquisition and revision 341
24.2 Enterprise grounding and permission propagation 346
24.3 Memory engines are distinct policy components 347
24.4 Work-product and vertical-system inventory 348
24.5 Causal evidence for organizational benefit 352
24.6 Professional benchmarks and economic interpretation . . 353
24.7 A reproducible evidence-review transaction 354
24.8 A changing-policy case in a shared knowledge workspace 355

25 Model Inspection, Executable Verification, and Evaluator Reliability 358
25.1 Properties observations and assurance boundaries 358
25.2 Causal interventions in model computation 360
25.3 Sparse features and their evaluation 361
25.4 Quality gates as fallible classifiers 364
25.5 Dynamic tests and requirement-grounded witnesses . . . 365
25.6 Judging long multimodal trajectories 366
25.7 Verification experiments and implementation traceability 367
25.8 A worked gate analysis with adaptive selection 368

26 Evaluation Harnesses, Contamination, and Statistical Validity 372
26.1 Define the estimand before the leaderboard 372
26.2 Harness construction and artifact identity 373
26.3 Attempts reliability and selection 375
26.4 Contamination information leakage and task lineage . . 376
26.5 Uncertainty paired designs and dependence 378
26.6 Ablations and causal mechanism claims 379
26.7 Evaluator audits and reproducible reporting 380
26.8 A concrete audit of a paired repository comparison . . . 381

27 Longitudinal Evaluation of Memory, Adaptation, and Recovery 385
27.1 Define horizon and the unit of continuity 385
27.2 Factor the memory pipeline 386

27.3 Implicit relevance and appropriate influence 387
27.4 Task order adaptation and hidden curricula 388
27.5 Temporal change deletion and migration 389
27.6 Horizon-dependent reliability and survival 390
27.7 A longitudinal implementation and reporting contract . 391
27.8 A counterfactual session-stream experiment 393

28 Stateful Agent Security: Red-Team and Blue-Team Mechanisms 398
28.1 Threat models and attacker capabilities 398
28.2 Injection across context tools and persistent memory . . 399
28.3 Least privilege and symbolic action controls 401
28.4 Controlled red-team platforms and judge validity 403
28.5 Cyber capability is not agent compromise 405
28.6 Blue-team detection containment and recovery 407
28.7 Security metrics and evidence interpretation 409
28.8 A bounded policy-expansion and memory-poisoning case 410

29 Alignment, Authorization, and Constrained Autonomy 414
29.1 Objectives proxies and constrained policies 414
29.2 Instruction hierarchy and authority-bearing records . . . 416
29.3 Constitutional and preference-based alignment 417
29.4 Human review and the semantics of approval 419
29.5 Privacy purpose and the lifecycle of memory 420
29.6 Operational constraints and residual risk 421
29.7 Evaluation of authority and aligned behavior 422
29.8 A governed model-and-harness update 423

30 Continual Adaptation and the Evidence for Recursive Self-Improvement 428
30.1 What changes when an agent improves 428
30.2 Selection pressure and finite-feedback overfitting 430
30.3 Task-local recursion and reusable improvement memory 433
30.4 Policy and harness co-adaptation 434
30.5 Continual memory maintenance as controlled adaptation 436
30.6 Testing the improver rather than its selected artifact . . 437
30.7 Bounds on claims and conditions for deployment 439

31 Agentic Inference: Scheduling, Speculation, and Tiered State **440**
31.1 The workload is a sequence of interrupted computations 440
31.2 A complete capacity calculation 442
31.3 Bandwidth arithmetic and the decode bottleneck 443
31.4 Exact reuse, partial reuse, and cache identity 444
31.5 Quantization speculation and structured action outputs 445
31.6 Memory-conditioned speculation and exact verification . 447
31.7 Scheduling disaggregation and queueing 448
31.8 A reproducible serving experiment 449

32 Operational Assurance: Monitoring, Migration, Rollback, and Recovery **451**
32.1 A release is a state-compatible system configuration . . 451
32.2 Observability must reconstruct decisions and effects . . 452
32.3 Canaries with persistent state and delayed outcomes . . 454
32.4 Rollback and compensation are different operations . . . 455
32.5 Failure detectors and bounded recovery policies 457
32.6 Service objectives and incident learning 459
32.7 A compatibility-first deployment procedure 460

33 Cross-Layer Causal Analyses and Compositional System Designs **463**
33.1 Composition changes the claim being tested 463
33.2 Case one: a repository repair with bounded side effects . 464
33.3 Parallel repair branches and cache-aware orchestration . 466
33.4 Case two: a persistent enterprise research assistant . . . 467
33.5 Testing relevance and influence rather than retrieval alone 469
33.6 Case three: an agent that modifies its own research harness 469
33.7 A unified acceptance record and discriminating interventions 471

34 Synthesis: Consistency Contracts and Discriminating Research Questions **474**
34.1 What the October 2026 evidence supports 474
34.2 Decision-preserving memory as an experimental target . 475
34.3 Conditioning under joint adaptation 476
34.4 Verification whose errors are measured independently . . 478

34.5 World models and representations must face action tests 479
34.6 A research program for reliable improvement 480
34.7 The remaining systems boundary 481

Chapter 1

Agentic Computation: System Boundaries, Generality, and Evidence

1.1 The agent is the unit of analysis

An agentic system is a coupled decision process whose behavior depends on a predictive model and on the machinery that determines what the model observes, which actions can execute, what persists, and how success is established. Treating the model endpoint as the whole system hides variables that can dominate an experiment. A model that proposes a valid shell command has not thereby completed a repository repair. The command must execute in the intended checkout, its effects must survive interruption, the resulting program must satisfy the specification, and the evidence must reach a decision maker. Each transition introduces an independently testable mechanism.

Let the operational configuration be

$$\Xi = (f_\theta, q, \delta, g_\eta, U, \mathcal{T}, \mathcal{R}, V, A), \tag{1.1}$$

where f_θ is the model, q the tokenizer and serialization interface, δ the decoding configuration, g_η the context constructor, U the persistent-state update rule, $\mathcal{T}$ the tool interface, $\mathcal{R}$ the execution runtime, V a collection of validators, and A the authorization mechanism. None of these components is assumed correct. The environment supplies observations and receives effects. A complete trajectory includes attempted actions, rejected actions, tool errors, state mutations, validator outputs, and termination, rather than only model messages. This boundary permits scientific attribution: changing a model and changing a harness are different interventions even if both increase a benchmark score.

The term *agentic* identifies controlled interaction, not a particular software package or level of intelligence. A fixed workflow can invoke a powerful model without delegating its control flow. Conversely, a modest model can choose among tools and adapt a plan, while remaining severely limited in competence. Useful architectural descriptions therefore state which

decisions are model-controlled, which are deterministic, and which require external authority. The ReAct formulation connects reasoning and environment actions, while SWE-agent demonstrates that the interaction interface around a model is itself an important engineering object. Neither establishes that every autonomous loop has useful generality [1, 2].

Three boundaries are especially consequential. The *epistemic boundary* separates available observations from latent environment state. The *authority boundary* separates a plausible model proposal from an executable permission. The *commit boundary* separates an attempted operation from an effect that has become durable or externally visible. A retrieved document may cross the epistemic boundary without crossing the authority boundary: observing an instruction does not grant permission to obey it. A tool response may report success without crossing a verified commit boundary: the operation may have been accepted asynchronously and later failed. Reliable autonomy requires all three boundaries to be represented explicitly.

This perspective explains why the ecosystem cannot be reduced to a list of models or SDK names. Tokenizers determine representational interfaces; context managers select current evidence; sessions preserve continuity; memory services retain and reinterpret information; skills package procedures; protocols transport capabilities and results; runtimes mediate side effects; evaluators define what is counted as success. Their interactions, rather than their isolated presence, are the main technical subject of this book.

October 2026 evidence makes this boundary experimentally consequential. TraceTrial-Bench separates task outcomes from harness-criterion satisfaction, using native traces and environment evidence rather than treating a successful deliverable as proof that every runtime obligation held [3]. ASAP evaluates frozen pretrained models through completion-based trajectory continuation, with reference prefixes and bounded recovery interventions [4]. Its observed competence therefore belongs to that elicitation protocol, not to an unaided model in an unspecified interface. These studies motivate recording both the system under test and the apparatus that measures it; neither a benchmark name nor a model name uniquely identifies the experiment.

1.2 A stateful formulation of execution

Let H_t be the ordered interaction records available at decision time t. History is not necessarily complete: a logging failure can omit an externally visible effect. Let M_t denote persistent memory, C_t the working context, E_t

the environment state, and ν_t the versioned compatibility manifest. Current authority is indexed separately by epoch α_t. The trusted observation map $\mathcal{O}_A$ reads authorization-relevant state; it need not reveal that privileged information to the model. With resource budget b_t, an agent step factors as

$$\begin{aligned} C_t &= g_\eta(H_t, M_t, b_t; \nu_t), \\ a_t &\sim \pi_{\theta,\delta}(\cdot \mid C_t, \mathcal{T}_t; \nu_t), \\ z_t^A &= \mathcal{O}_A(E_t; \alpha_t), \qquad d_t = A(a_t, z_t^A; \alpha_t), \\ (E_{t+1}, o_{t+1}) &\sim P(\cdot \mid E_t, a_t, d_t), \\ e_t &= (a_t, d_t, o_{t+1}, \nu_t, \alpha_t, \kappa_t), \\ H_{t+1} &= \mathcal{L}(H_t, e_t), \\ M_{t+1} &= U(M_t, H_{t+1}; \nu_t), \qquad H_{t+1} \neq \bot. \end{aligned} \tag{1.2}$$

The decision $d_t \in \{0, 1\}$ gates dispatch; denial does not prevent unrelated actors from changing E_t. The locator κ_t binds the actual model input and decoding configuration to retained evidence. A locator or digest alone does not recover unavailable content. The partial journal operator $\mathcal{L}$ returns the acknowledged append $H_t \oplus e_t$ or the unresolved marker $\bot$; the displayed memory update is defined only on its successful branch. An effect may precede a failed journal write, so recovery must reconcile it before claiming continuity. Current authority must remain valid at dispatch under the service's concurrency contract. Neither the notation nor a local append establishes an atomic transaction with an external service.

The parameters and component implementations in Ξ are fixed within the stated comparison epoch. Persistent-memory updates occur through U; changing model weights, prompt templates, executable skill implementations, or harness code changes the versioned configuration and requires an explicit outer transition. This distinction prevents a purported memory ablation from silently becoming a policy or runtime rewrite. Compatibility metadata and authority have different clocks: a restored model manifest cannot restore an expired external grant.

Editing a working context or selecting an already versioned skill can remain a state transition under a fixed implementation. The outer transition concerns a change to the governing component or artifact version, not every change to its runtime inputs. Whether a learned procedure is treated as memory data or executable code must be declared before comparing adaptation mechanisms.

The state visible to the policy is usually not Markov. A summarized conversation may omit a pending asynchronous job, an already consumed credential, or a revoked approval. The full execution system can nevertheless

maintain those facts in privileged state. Mixing privileged state into free-form prompts as the only enforcement mechanism sacrifices a useful separation: text is evidence to a stochastic policy, whereas authorization and commit status require explicit operational semantics. Chapter 17 develops compatibility across these representations.

A simple counterexample shows why context length alone cannot measure continuity. Consider two histories with identical retained text: both say that a report should be uploaded. In one history the upload has already committed; in the other, the request timed out before any effect. A policy conditioned only on retained text cannot distinguish the states. Retrying is appropriate in one case and may duplicate an external effect in the other. Increasing the context window helps only if the distinguishing event is recorded, retained, interpretable, and consulted. A durable operation identifier and a reconciliation endpoint can resolve the ambiguity without increasing the model window.

For scientific comparison, the intervention unit should therefore be the configuration Ξ and its state initialization, not merely the model identifier. An experiment that compares two agents while silently changing the tool registry, memory contents, retry policy, or execution environment cannot attribute its outcome to one named mechanism. The configuration should include release or commit identities, and the trajectory should preserve enough information to reconstruct the actual observations that conditioned each action. Reconstruction is not assumed possible when tool outputs or eviction histories were never stored.

1.3 Generality as a profile of conditional competence

Operational definitions of AGI differ in breadth, level of performance, adaptation requirements, and autonomy. A useful survey must expose these commitments rather than define AGI as whatever a preferred benchmark measures. Chollet emphasizes skill acquisition and generalization relative to priors and experience; the Levels of AGI framework separates breadth and performance while discussing autonomy as an additional axis. These are measurement frameworks, not demonstrations that unrestricted general intelligence has been achieved [5, 6].

Their foundational role must be separated from dated system examples. The inspected fifth revision of Levels of AGI retains classifications explicitly referring to September 2023; those examples are not an October 2026 assessment. Here AGI denotes a proposed operating claim over declared

task families, adaptation access, resource constraints, human assistance, and failure tolerances. No finite family below is asserted to exhaust general intelligence.

For a task distribution $\mathcal{D}_j$, adaptation protocol $\mathcal{A}$, and resource envelope B, let q denote a task and $P_\Xi(\cdot \mid q, B, \mathcal{A})$ the induced distribution of interaction trajectories. Define

$$G_j(\Xi; B, \mathcal{A}) = \mathbb{E}_{q\sim\mathcal{D}_j}\mathbb{E}_{\tau\sim P_\Xi(\cdot|q,B,\mathcal{A})}[s(\tau)], \qquad 0 \le s \le 1. \tag{1.3}$$

The index j ranges over declared families rather than an unspecified universe of possible tasks. The score is accompanied by failure severity, calibration, time, cost, and uncertainty. A system may improve in one coordinate and regress in another; a scalar average requires explicit weights and cannot erase those regressions. Autonomy is represented by the actions available under A, the human intervention protocol, and the permitted horizon. A system evaluated with unrestricted expert rescue is not being tested under the same autonomy conditions as one operating without rescue.

Adaptation must also be bounded. Allowing a system to acquire new tools, query demonstrations, run arbitrary experiments, or repeatedly inspect evaluation feedback changes the task. These capabilities may be entirely legitimate, but they belong in $\mathcal{A}$ and B. A method that solves a task after extensive environment-specific search is evidence of competence under that search budget. It does not establish that the initial policy already possessed the procedure or that transfer will persist when feedback is unavailable.

The human reference population matters whenever comparisons use the word human. Domain experts, general workers, and untrained participants have different knowledge, tools, and time budgets. Comparisons should align access to external resources and account for the costs of human supervision or correction. Claims of parity across a selected set of tasks remain claims about that set. They cannot establish competence over unobserved physical, social, or scientific environments.

Generality and reliability interact. A broad system that occasionally performs well but fails unpredictably under repeated attempts may be less useful than a narrower system with a verified operating envelope. Evaluation should preserve task-level distributions and repeated-run outcomes. A frontier score is an observation about a measurement process; it becomes evidence of generality only after its distribution, leakage controls, resource conditions, and failure modes have been examined.

1.3.1 Model–harness interaction and adaptation

Let $J(\theta, h)$ denote one coordinate of the competence profile with the model parameters and harness configuration exposed separately. Hold the task population, initialization, adaptation protocol, measurement apparatus, and complete resource envelope fixed. A two-model, two-harness comparison has interaction contrast

$$\begin{aligned} \Delta_h(\theta) &= J(\theta, h_1) - J(\theta, h_0), \\ I_{\theta,h} &= \Delta_h(\theta_1) - \Delta_h(\theta_0). \end{aligned} \tag{1.4}$$

A nonzero contrast means that the harness difference depends on the evaluated model. An estimate requires paired tasks, repeated executions where stochasticity matters, and uncertainty that respects task clustering. It is a causal effect only under a valid intervention design; a cross-product of leaderboard entries with different budgets is insufficient.

The 27 October paper on the decision geometry of agent interfaces distinguishes a harness's attainable regularized return from a deployed policy's mismatch to it [7]. Its exact identities assume finite states and actions, bounded rewards, discount below one, positive regularization, and positive reference support; stronger-model estimates in its coding study are proxies for the optimum. A fixed-policy harness comparison and a comparison after matched adaptation answer different questions. Harness Scaling likewise studies training over structured execution configurations and transfer to held-out harness combinations [8]. Neither result makes arbitrary interface changes harmless. Adaptation cost, training-harness exposure, and held-out combinations must remain explicit when interpreting transfer as generality.

The corresponding HRP source separates those evidential levels. In `environment/hrp_miniwob/oracle.py`, `solve` and `evaluate` operate on a finite control graph; `policy_metrics` in `environment/hrp_miniwob/measurements.py` separately computes the return gap and occupancy-weighted divergence. The training route `miniwob_v2_train.py` uses an Exact-PG update through a verl worker, whereas `StrongTeacherEstimator` supplies deployment proxies [9]. These paths were inspected without executing training. The repository's environment documentation also maps code harness labels to paper labels; treating the two label orders as identical would misidentify the experimental intervention.

Figure 1.1 distinguishes the controls for a frozen-policy comparison from those for an adaptation comparison. A favorable result in one protocol does not determine the ranking in the other.

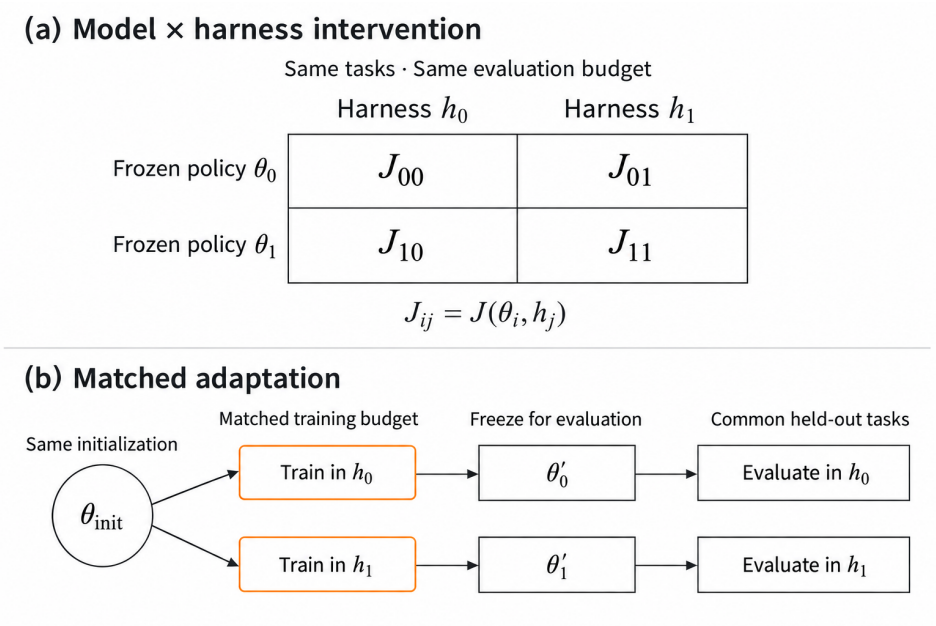

Figure 1.1: Two different harness-comparison protocols. (a) $J_{ij} = J(\theta_i, h_j)$ exposes all four fixed-policy cells needed for the interaction in Equation 1.4, under common tasks and evaluation limits. Entries are symbolic outcomes, not measurements. (b) Each θ'_i is obtained from the same θ_{init} with matched training budgets before evaluation in h_i on common held-out tasks. Primes denote adapted checkpoints frozen for evaluation, not certified optima. Compatible tasks, grading, and leakage checks remain necessary.

1.4 An ontology of models and connecting components

The distinction between component classes is functional. A tool is an invocation interface to an operation. A skill is procedural information, possibly including executable resources, that conditions how operations are chosen and composed. A protocol specifies exchanges and lifecycle semantics between participants. A session identifies and manages a sequence of interactions. A memory system determines how information is retained, revised, accessed, and forgotten. An SDK supplies construction interfaces; a managed platform operates selected mechanisms behind a service boundary. A product integrates them for a workload and user interface. One package may fulfill several roles with distinct contracts.

These distinctions prevent common category errors. A checkpoint can

Table 1.1: Component boundaries and independently testable mechanisms.

Component	Mechanism that requires independent evidence
Model and tokenizer	Conditional prediction, representational interface, precision, and actual accessible context.
Context constructor	Evidence selection, compaction, ordering, provenance, and behavior after editing.
Session service	Identity, checkpoint scope, concurrency, interruption, resumption, and retention.
Memory service	Write policy, temporal validity, retrieval, conflict resolution, deletion, and access control.
Skill mechanism	Applicability, procedural composition, resource loading, compatibility, and trusted execution.
Tool runtime	Argument validation, side effects, idempotency, timeouts, and execution isolation.
Protocol	Capability advertisement, message semantics, version compatibility, transport, and authorization hooks.
Evaluation harness	Environment identity, grading rules, missing outcomes, leakage controls, and trace integrity.

preserve a graph state without maintaining a correct semantic account of a user's changing preferences. A vector store can retain records without teaching a policy when those records should influence action. A protocol-compliant server can return false information. A skill can be syntactically valid but obsolete after a tool-schema change. A large model can interpret all of these artifacts while still acting on stale authority. The mechanism required for each property must be tested at its own boundary.

1.4.1 Current implementation and training boundaries

The inspected October 2026 source revisions instantiate different persistence objects. OpenAI Agents Python defines conversational item persistence through `Session`; `prepare_input_with_session` separately prepares the model input and the new items eligible for persistence. LangGraph separates

thread checkpoint/channel state from its cross-thread `BaseStore`. Google ADK separates a `Session`'s application, user, session identity, event history, and state from `BaseMemoryService` ingestion and search [10, 11, 12]. These are inspected interfaces, not interchangeable semantic-memory guarantees. The retained object, the actual model request, and the authority to reuse an item require separate evidence.

Agent Lightning v1.0 makes the training boundary equally concrete: a deployment harness supplies interaction while captured model calls become training data [13]. In the inspected source, `ProxyRouter.prepare_body` requests token identifiers and `forward_request` captures exchanges. In trajectory aggregation mode, `RolloutAdapter.get_train_data_batch` in `agentlightning/verl/rollout_adapter.py` merges compatible token prefixes and masks inserted observations. When enabled, `compute_rollout_level_advantage` groups captured samples by rollout identity and checks their shared task and reward [14]. This supplies an implementation path for examining conditioning and credit assignment, not proof that every supported configuration reproduces the paper. The source was inspected rather than trained. A session trace becomes suitable policy-training evidence only after its action tokens, behavior inputs, probability semantics, and grouping survive the conversion.

The brain analogy is useful only at this functional level. Working context resembles a limited active information workspace; episodic records preserve aspects of past interaction; semantic records organize claims; procedural memory stores reusable action guidance. Model weights, KV tensors, event logs, and databases implement these functions through very different update and access rules. A KV cache is a set of computation-dependent tensors, not a faithful semantic archive. External record deletion does not erase parametric associations. The analogy provides a vocabulary for timescales and access, not evidence of neurological equivalence. Memory surveys organize this landscape through different axes, including agent versus model memory, active versus inactive representations, and resource–distortion tradeoffs; the component ontology here complements those taxonomies rather than claiming to replace them [15, 16, 17].

1.5 What constitutes evidence in an evolving ecosystem

The literature and implementation review for all chapters extends through 29 October 2026; each source retains its reported revision and retrieval limits. Foundational methods are not excluded by age. Contemporary

papers, official technical reports, normative specifications, and versioned repositories are examined according to the mechanism being claimed. First public release, manuscript revision, code release, and product activity are distinct dates. A 2026 revision of a 2025 paper is not automatically a newly introduced 2026 method. A university evaluation of a proprietary model is not research authored by that model's vendor.

1.5.1 Search scope and inclusion rules

This is a mechanism-oriented critical survey, not a claim of exhaustive systematic identification. Candidate discovery combines a seed bibliography, citation following, and topic-specific searches for context editing, agent memory, procedural skills, tool protocols, harness optimization, agent evaluation, scientific agents, security, and serving. The present chapter additionally searches model–harness adaptation, pretraining capability elicitation, and harnessed reinforcement learning. Paper titles and identifiers are checked against original records; implementation claims use official repositories or documentation, with a commit, release, or protocol version when available. Retrieval occurred on 28–29 October 2026. The publication cutoff concerns artifact availability, not the calendar day on which an eligible artifact was inspected.

Inclusion requires a technical connection to an identified mechanism or evaluation boundary. Foundational references establish definitions, algorithms, or historical lineage; they do not substitute for current capability evidence. Frontier claims require an eligible primary artifact and retain its revision and experimental scope. A withdrawn manuscript does not support capability claims. Abstract-only or metadata-only inspection does not establish method details, and inaccessible implementation information remains an evidence gap. Organization-specific comparisons require verified paper affiliations rather than inferred ownership. Product catalogues provide ecosystem coverage, while feature and performance comparisons require additional mechanism-specific evidence.

For a mutable documentation page, the observation date and the available version have different meanings. A page read after the cutoff cannot establish that all of its present contents existed before the cutoff; a dated release or pinned artifact is needed for that historical claim. Consequently, the survey reports current mechanisms within the searched and inspected population rather than declaring any method universally best or all eligible work exhausted. Unresolved source leads are not counted as independently verified methods.

Four evidence classes are kept separate. An *author-reported result* is

attributed to an exact source version and experimental setting. An *independent reproduction* requires actually executed artifacts, data identity, environment, configurations, seeds, and recorded outcomes. An *analytical deduction* follows from stated assumptions and may provide a bound, counterexample, or resource calculation. A *proposed experiment* defines a hypothesis and a discriminating intervention but has no observed result. This book does not present independent reproductions where none were performed.

Verification of a title and identifier does not verify a result. Inspection of a repository does not show that its current head reproduces a paper. A benchmark improvement does not prove the proposed mechanism caused the improvement unless resources and relevant alternatives are controlled. In particular, additional tool calls, longer reasoning, more candidates, or a stronger evaluator can account for gains attributed to a memory representation or orchestration strategy. The evidential question is the effect of the mechanism under declared operating conditions.

Conflicting findings are compared through task populations, resource accounting, feedback access, model versions, memory initialization, task order, and grading. Where these differences do not resolve the disagreement, the conflict remains open. A method can simultaneously improve average accuracy and worsen tail risk, or improve a selection score while losing held-out transfer. Those outcomes require a multi-dimensional interpretation rather than an editorial ranking. Evaluation frameworks such as HELM motivate systematic scenario and metric coverage, while agent benchmarks expose additional dependence on tool and environment semantics [18, 19, 20].

1.5.2 Contemporary measurements with distinct estimands

Agents' Last Exam evaluates configured agents on an executed workflow population; an expanding advertised task collection is not that experiment's denominator. Its mixed model–harness comparisons and fixed-harness backbone comparisons must remain distinct [21]. AgentHorizon instead assesses judges on recorded instruction–trajectory pairs, including constructed near misses. It does not directly measure autonomous agent completion, and the constructed error distribution is not a natural deployment failure distribution [22]. Table 1.2 makes the different inference boundaries explicit. All empirical findings remain author-reported.

TraceTrial-Bench's selected archive is not a set of independent repetitions within every model–harness–task cell. Its semantic scoring also resolves some unexercised harness criteria using task outcomes, so the two evaluation

Table 1.2: Current evidence anchors and limits of inference, inspected 29 October 2026.

Artifact	Object measured or studied	Inference boundary
ALE, v4	Configured-agent workflow outcomes.	Executed population and backbone/harness grouping define the comparison.
AgentHorizon, v1	Judge detection of instruction violations in recorded trajectories.	Judge validity under the constructed distribution is distinct from agent reliability.
TraceTrial-Bench, v1	Task outcomes and operational harness criteria in a selected execution archive.	Candidate failure links require evidence review; criterion satisfaction is not a causal defect estimate.
ASAP, v1	Base-model execution under a completion scaffold, prefixes, and recovery.	Protocol-assisted elicitation does not identify unaided capability or later training gains.
Harness Scaling, v2	Policy training and transfer across execution configurations.	Held-out combinations and external harness tests remain bounded by training and evaluation access.

layers are not statistically independent. Automatically activated mechanism links are candidates, not validated causal propagation [3]. ASAP's reference-prefix interventions and Harness Scaling's training exposure similarly belong in the experimental configuration [4, 8]. These limitations change what a score estimates; they are not repaired by adding another decimal place or averaging unlike benchmarks.

1.5.3 Provenance and population boundaries

Identity is a relation among dated artifacts, rather than a string copied from a product list. A renamed project, acquired service, and rewritten SDK can share a commercial lineage while exposing different execution semantics. An alias record should therefore specify the entities, the relation, its effective interval, and the primary evidence. A merger does not prove API compatibility, and an organization named in an announcement does

not establish every author's affiliation. For an affiliation filter, the relevant observation is the author-to-institution statement in the inspected paper version; a repository owner or email domain is insufficient evidence of that paper's institutional authorship. These distinctions prevent organizational metadata from becoming an unsupported proxy for technical quality.

Release dates also have different meanings. The first preprint, a revised method, a public repository commit, a package release, and hosted availability are separate events. A claim about the earliest available mechanism must identify which event is relevant and retain the version that contained it. A later revision cannot be assigned the earlier version's date without checking that the mechanism was already present. Conversely, a preprint available before the literature cutoff remains eligible even when its stated conference occurs later. Dates establish availability, not independent validation or priority beyond the searched population.

Popularity measures need an equally explicit denominator. Repository stars, package downloads, public mentions, and service users measure different populations and have different duplication, access, and sampling effects. Record the platform, observation time, counting rule, entity identity, and missingness before comparing them. Such counts can describe visibility or adoption under that definition; they do not identify a capability ranking. A priority list can guide inspection without becoming evidence that its first entry is technically strongest.

An independent observatory supplies additional evidence only through a specified measurement protocol. Its task sample, evaluator, attempt budget, model endpoint, tool permissions, and observation date determine the estimand, just as they do for an author-run benchmark. Independence of organization does not guarantee independence of task data or judge errors. Agreement between two sites that reuse the same predictions is one result observed twice. Disagreement can instead arise from different configured systems or changing hosted versions. The appropriate synthesis preserves those boundaries and reports an unresolved comparison when they cannot be aligned.

1.6 Three consistency contracts and their non-implication

The organizing synthesis separates three obligations that are often collapsed into a statement that an agent retained its state. Let s denote a complete implementation state. Its projections include the model-visible context $c(s)$, tool interface $T(s)$, authoritative feasible-action set $F_\alpha(s)$ under current

authority epoch α, and an effect ledger λ_s. The ledger maps operation identifiers to typed states such as pending, reconciled success, reconciled failure, and unknown. These are semantic status classes, not unverified model assertions. Let $\Gamma : \mathcal{S} \to \mathcal{S}$ be a proposed representation or recovery transformation, with the model parameters, tokenizer, and decoding configuration fixed for the comparison.

For $\epsilon \in [0, 1]$, define proposal preservation by

$$\mathsf{P}_\epsilon(\Gamma, s) = \mathbf{1}\{\mathrm{TV}(\pi_{\theta,\delta}(\cdot \mid c(s), T(s)), \pi_{\theta,\delta}(\cdot \mid c(\Gamma s), T(\Gamma s))) \le \epsilon\}. \quad (1.5)$$

This compares proposal distributions on a common action alphabet before authorization, including an invalid-proposal symbol where required. It does not establish proposal correctness. Exact preservation is $\mathsf{P}_0 = 1$. For a fixed one-step loss $\ell(a) \in [0, L_{\max}]$, $\mathsf{P}_\epsilon = 1$ implies

$$|\mathbb{E}_{a\sim\pi'}\ell(a) - \mathbb{E}_{a\sim\pi}\ell(a)| \le L_{\max}\epsilon, \quad (1.6)$$

where π and π' are the two proposal laws in the predicate. This follows from the variational characterization of total variation. It is a one-step guarantee for the specified loss, not an episode-level reliability bound. Conversely, equal task utility need not imply proposal preservation: two distinct actions can have the same consequence. Approximate preservation is therefore a strong diagnostic contract whose necessity depends on the downstream equivalence relation. It cannot be inferred from lexical similarity or unchanged archive bytes. Training adds a distinct obligation: likelihood terms must refer to the information that actually conditioned behavior. Chapter 17 treats that likelihood-consistency problem explicitly.

Authority nonexpansion under a transformation that has no new external grant is

$$\mathsf{A}(\Gamma, s) = \mathbf{1}\{F_\alpha(\Gamma s) \subseteq F_\alpha(s)\}. \quad (1.7)$$

The authority epoch is held current, not restored from an old conversational snapshot. The subset condition permits conservative loss of permissions; such loss can reduce utility and must be evaluated separately. A legitimate new grant is a different operation with its own approval evidence. The predicate prevents a context rewrite, retrieved instruction, or recovery procedure from silently creating authority.

For a pure representation transformation, effect-state preservation is

$$\mathsf{X}(\Gamma, s) = \mathbf{1}\left\{\begin{array}{ll} \mathrm{dom}\,\lambda_{\Gamma s} = \mathrm{dom}\,\lambda_s, & \\ \lambda_{\Gamma s}(u) = \lambda_s(u) & \forall u \in \mathrm{dom}\,\lambda_s \end{array}\right\}. \quad (1.8)$$

The comparison holds external observations fixed. It requires preservation of operation identity and recorded status, including explicit uncertainty.

It does not validate an initially incorrect ledger or undo external effects. Reconciliation that acquires new service evidence is a separate transition and can legitimately change an unknown status to confirmed success or failure. A pending or unknown outcome cannot become a proven failure merely because a summary omitted the operation.

The exact contracts P_0, A, and X are logically independent. Consider a product state with separate context, permission, and ledger coordinates. Changing only a context bit that controls a deterministic proposal violates P_0 while preserving authority and the ledger. Changing only a permission coordinate from denied to allowed violates A while leaving the proposal distribution and ledger unchanged. Changing only a recorded committed outcome to unknown violates X while preserving context and permissions. Each construction is a witness that the other two predicates do not imply the third. The claim is not made for the vacuous tolerance $\epsilon = 1$. This is an analytical non-implication argument; it is not a claim of a new general theorem about cognition.

The distinction matters in actual mechanisms. Context editing and compaction primarily expose P_ϵ and behavior-conditioning obligations. Privilege monitors expose A. Checkpoints and effect reconciliation expose X. Memory updates and model migrations can affect all three. A verifier then tests a specified subset of the required properties; passing that verifier does not establish properties outside its scope. MemoryWalker addresses behavior-time conditioning and Progent privilege control [23, 24]. CyberGym-E2E's vulnerability, proof-of-concept, and patch checks and AgentHorizon's trajectory judgments test different outcome properties; neither demonstrates durable effect-ledger reconstruction [25, 22].

The originality of this survey lies in analyzing those boundaries jointly, connecting their assumptions to identifiable implementations, and constructing interventions that distinguish their failures. It does not relabel context editing, rate-distortion analysis, privilege control, or durable execution as newly discovered ideas. Contemporary memory and compression surveys already organize substantial parts of that landscape [15, 16, 17]. The added question is which contracts survive composition and co-adaptation, and what evidence can falsify that claim. Chapters 17, 29, 32, and 33 develop the resulting conditioning, authority, recovery, and integration analyses.

Figure 1.2 distinguishes the information, authority, and effect boundaries used by these predicates.

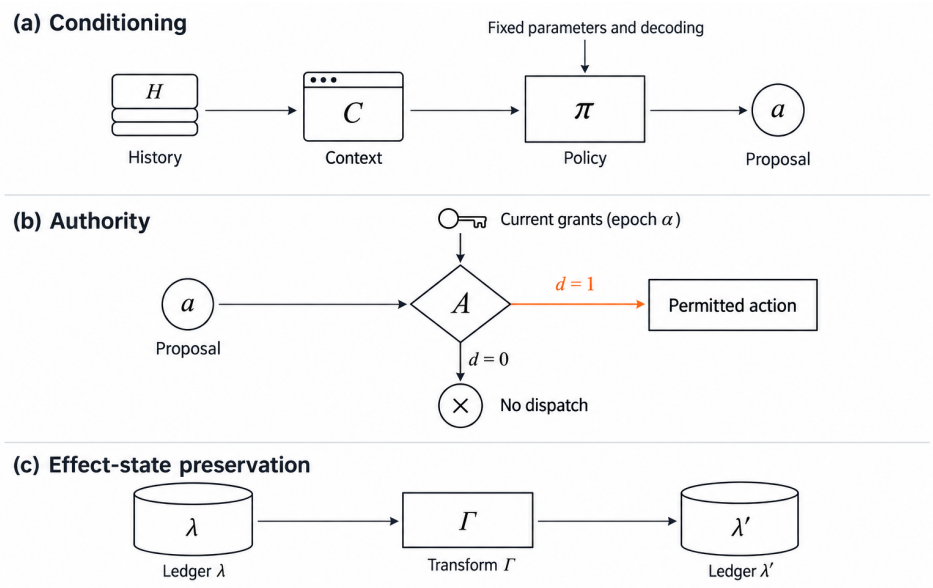

Figure 1.2: Three separate state contracts. (a) An illustrative history–context–policy path under fixed parameters and decoding; memory and other constructor inputs are omitted. (b) Current grants at authority epoch α gate permitted actions and no dispatch. (c) With $\lambda = \lambda_s$ and $\lambda' = \lambda_{\Gamma s}$, a pure representation transformation preserves operation identities and recorded status, including unknown outcomes. Reconciliation with new service evidence is a different transition. The schematic does not establish task correctness.

1.6.1 From preservation contracts to corrective evidence

A state transformation can preserve every proposal, permission, and ledger entry while preserving a wrong policy or an incorrect historical record. The three contracts therefore govern transformation integrity; task acceptance requires additional observed evidence. An operational implementation should return scoped verdicts and identify which evidence can resolve a failed or undecidable check. Merely adding a verifier whose output is always accepted would leave this distinction empty.

Let $\mathfrak{G} \subseteq \{\Gamma : \mathcal{S} \to \mathcal{S}\}$ be finite and $z \in \mathcal{Z}$ an evidence record binding source inputs, versions, current grants, and available receipts. With tolerance ϵ and authority epoch α fixed, for $i \in \{P, A, X\}$ let $v_i : \mathfrak{G} \times \mathcal{S} \times \mathcal{Z} \to \{0, 1, \bot\}$ return failed, passed, or unresolved under

a declared checking procedure. Define

$$\begin{aligned} \mathcal{D}_z(\Gamma, s) &= \{i \in \{P, A, X\} : v_i(\Gamma, s; z) \neq 1\}, \\ \mathfrak{S}_z(s) &= \{\Gamma s : \Gamma \in \mathfrak{G},\ \mathcal{D}_z(\Gamma, s) = \varnothing\}. \end{aligned} \tag{1.9}$$

Membership permits advancement under that procedure's scope; admission expires when a governing state version or authority epoch changes. It establishes the mathematical contract only when the checker and its premises justify that inference. In particular, sampled behavioral agreement does not prove equality of proposal distributions. The diagnostic set selects a corrective evidence operation, rather than another undirected reflection.

Table 1.3: Proposed corrective-evidence checks; no experiment is reported.

Check	Corrective operation	Required recheck
P	Restore source-bound behavior inputs and transformation lineage.	Declared proposal or decision-preservation property under the fixed configuration.
A	Refresh current grants; discard permission inferred from retrieved prose.	Nonexpansion and the exact operation's current authorization.
X	Recover operation identities; obtain service evidence when status is unknown.	Ledger continuity; unresolved outcomes remain explicit.

Effect reconstruction and task validation also need separate outputs. For an operation identifier $u \in \mathcal{U}$, let z_E and z_V be the corresponding effect-evidence and task-test projections of z. Define $\rho : \mathcal{U} \times \mathcal{Z}_E \to \{1, 0, \perp\}$ as confirmed effect, confirmed absence, or unresolved status under the service contract, and $w : \mathcal{U} \times \mathcal{Z}_V \to \{0, 1, \perp\}$ as the verdict for a specified task property. Retain

$$\mathcal{Y}(u, z) = (\rho(u, z_E), w(u, z_V)). \tag{1.10}$$

Here absence concerns the specified effect, not a claim that no partial effect exists elsewhere. A confirmed upload can contain the wrong artifact; a valid local artifact can coexist with an unknown upload outcome. These observations demand different corrections. Current dispatch authorization remains necessary even after a transformation passes. New service evidence may legitimately revise ρ; it must be recorded as reconciliation, not disguised as preservation of the old ledger.

A discriminating study injects one conditioning, grant, or operation-record fault at a time into otherwise matched states. It reports false admission, unresolved decisions, recovery cost, and verified task outcomes separately. No outcome is presumed here: the construction specifies how the chapter's three contracts become diagnosable engineering obligations.

1.7 Cross-layer questions that organize the analysis

The first question is whether retained information remains usable. A memory system can archive every event and still fail to retrieve a rare instruction before an irreversible action. Retrieval can succeed while the policy fails to recognize implicit relevance. A summary can preserve a fact while dropping its temporal qualifier. These are different failures, requiring separate measurements of retention, access, interpretation, influence, and execution. Treating all of them as recall produces misleading explanations.

The second question is whether the system acts under the history it is trained to assume. Context compaction changes conditioning. Training reconstructed trajectories that expose information unavailable during rollout can create leakage or support mismatch. MemoryWalker explicitly studies this problem, while Context Language Models make context editing part of the policy interface. Their relationship is a cross-layer issue: an editing policy, a learning objective, and a serving cache must agree about which history produced each action [23, 26].

The third question concerns improvement. A system that accumulates successful procedures may become better on recurring tasks while growing less robust to novel environments. An evolving harness can overfit a finite selection set. An evaluator can drift as the executor changes. Recursive improvement requires distinguishing the quality of the executor from the quality of the process that proposes, selects, and validates future changes. RRSI and Harness^2 provide concrete but bounded mechanisms to analyze; neither licenses a general claim of indefinite acceleration [27, 28]. The additional adaptation question is whether a revision improves its available control problem or merely fits the current policy; a frozen-policy ranking cannot settle that distinction. Section 1.3.1 specifies the corresponding comparison boundary.

The fourth question is operational: which guarantees survive interruption, concurrency, migration, and partial failure? A correct answer in a clean benchmark environment may rely on assumptions absent in deployment. Duplicate actions, inconsistent checkpoints, stale indexes, revoked

permissions, and unobserved external effects connect systems engineering directly to model behavior. Reliable autonomy is an end-to-end property of this coupled process. The subsequent chapters develop the mathematical, computational, and empirical machinery needed to test it.

Chapter 2

Probabilistic State, Decision Processes, and Statistical Identification

2.1 Conditional prediction and observable information

Agent mathematics distinguishes the world, the system's recorded information, and the inputs actually exposed to a decision rule. Let X_t be latent environment state, O_t an observation, and $(\mathcal{F}_t)$ the filtration of recorded system observations and preceding actions before action A_t. Let $\mathcal{G}_t \subseteq \mathcal{F}_t$ describe the information exposed to the proposal policy. Unlike the accumulated record, these exposed information sets need not be nested after compaction. A realizable stochastic policy is a $\mathcal{G}_t$-measurable probability kernel; sampling from it uses fresh randomization. Requiring the sampled action itself to be measurable before that randomization would exclude stochastic policies. Neither the kernel nor its input constructor may use tomorrow's observations or hidden evaluation answers. This distinction identifies leakage without pretending that the agent observes the latent world.

For an autoregressive model with serialized input c and output tokens $y_{1:n}$,

$$p_\theta(y_{1:n} \mid c) = \prod_{i=1}^{n} p_\theta(y_i \mid c, y_{<i}). \tag{2.1}$$

The factorization gives the likelihood of a specified finite token prefix under the raw model and conditioning set. It says nothing by itself about whether token probabilities are calibrated, whether a decoded tool call is valid, or whether the resulting action is authorized. If a harness deterministically parses output into an operation, the action probability is the probability mass of all token strings that parse to that operation, including any decoder constraints. It is generally not the probability of one canonical JSON string.

Let $q_{\theta,\delta}(v \mid c, y_{<i})$ be the normalized next-token law of the deployed decoder, including temperature, masks and truncation. This need not equal

the raw model law p_θ. Let $N \in \mathbb{N}_0 \cup \{\infty\}$ be the stopping time of one generation call and

$$q_{\theta,\delta}(y, n \mid c) = \Pr_{\theta,\delta}\{Y_{1:n} = y, N = n \mid c\}. \tag{2.2}$$

The event includes the stopping rule; a product for a finite prefix alone is not a normalized distribution on completed strings. Let ϕ map stopped records to actions or a distinguished invalid outcome a_{inv}. For $a \in \mathcal{A} \cup \{a_{\text{inv}}\}$, the finite-stop contribution is

$$\begin{aligned} \pi_{\theta,\delta}(a \mid c) &= \sum_{n \geq 0} \sum_{y:\phi(y,n)=a} q_{\theta,\delta}(y, n \mid c), \\ \pi_{\theta,\delta}(a_\infty \mid c) &= \Pr_{\theta,\delta}(N = \infty \mid c). \end{aligned} \tag{2.3}$$

The outcome a_∞ records nontermination and is distinct from an invalid parsed output. A hard token cap makes N finite in generation steps, but budget exhaustion still needs an explicit parser outcome. Wall-clock nonreturn requires a separately declared deadline outcome. Conditioning the reported law on successful parsing discards failure mass and describes a different policy.

For illustration only, suppose calls finish, $\sum_a p_a + q = 1$, and repeated calls have the same context, are independent, and yield a valid action a with probability p_a or an invalid output with probability q. With at most $R \geq 1$ calls,

$$\Pr(A = a) = p_a \sum_{j=0}^{R-1} q^j, \qquad \Pr(A = a_{\text{exhausted}}) = q^R. \tag{2.4}$$

An unbounded retry loop terminates almost surely only if $q < 1$ under these assumptions; then the valid-action law is $p_a/(1-q)$. When retries edit the context, this geometric calculation is inapplicable: the complete retry state and its transition law must be modeled. These are harness-level probability laws, not measurements.

These are proposal distributions. Authorization and dispatch introduce another transition, including denial and no-dispatch outcomes; successful parsing is not evidence of either permission or an external effect. If a mask leaves no legal next token, the implementation must return a specified failure outcome rather than silently divide by zero. Tool-use learning must identify which of these laws its objective estimates [29, 1].

OpenAI Agents Python's `prepare_input_with_session` in `src/agents/run_internal/session_persistence.py` retrieves history under

a configurable limit, merges history with new input, and returns prepared input separately from items eligible for persistence [10]. A stored conversation cannot substitute for the conditioning input in a likelihood calculation. Instrumentation must retain the final model request after subsequent filters; comparing the store with this preparation stage alone would miss later edits.

Bayesian conditioning provides a useful abstraction for uncertain observations. Given a prior $p(x)$ and observation likelihood $p(o \mid x)$,

$$p(x \mid o) = \frac{p(o \mid x)p(x)}{\sum_{x'} p(o \mid x')p(x')}. \tag{2.5}$$

The displayed sum is for discrete or countable states and requires positive predictive probability for the observation. A zero denominator identifies no posterior under this model; it calls for a declared model-recovery or conservative fallback rule, not a zero vector or an arbitrary normalization. Continuous states require the corresponding integrals and regular conditional distributions. A retrieved statement should not automatically be treated as a noiseless observation of x. Its reliability depends on source identity, timestamp, provenance, and the possibility of adversarial manipulation. Multiple retrieved passages may share a common origin and therefore fail to provide independent evidence. Multiplying their likelihoods as though they were independent can create extreme but unjustified confidence. The implementation analogue is to retain provenance links and deduplicate evidence lineages, rather than counting repeated text as corroboration.

A constructed timeout example makes the distinction operational. Let $X = 1$ mean that a submitted operation committed, and O denote a timeout. Under hypothetical assumptions $P(X = 1) = 0.9$, $P(O \mid X = 1) = 0.1$, and $P(O \mid X = 0) = 0.8$, Bayes' rule gives

$$P(X = 1 \mid O) = \frac{0.1 \cdot 0.9}{0.1 \cdot 0.9 + 0.8 \cdot 0.1} = \frac{9}{17}. \tag{2.6}$$

These values are analytical inputs, not measured service rates. The timeout does not identify the effect as absent; even a calibrated posterior does not itself authorize a retry. Reconciliation and the duplicate-effect loss belong to the subsequent decision problem. Chapters 13 and 32 connect this information restriction to recoverable execution.

2.2 Information loss and decision-relevant state

Let H denote interaction history and $Z = \varphi(H)$ a compressed representation. Compression is useful when decisions can be made from Z with

acceptable loss. Faithful text reconstruction is only one possible objective. A task may require preserving an authorization expiry time while allowing substantial paraphrase of everything else. Conversely, a summary with high lexical similarity may lose precisely the condition that changes the correct action.

For a discrete future target Y with finite relevant conditional entropies, the Bayes-optimal log-loss using history is $H(Y \mid H)$ and using the compressed representation is $H(Y \mid Z)$. For deterministic $Z = \varphi(H)$,

$$H(Y \mid Z) - H(Y \mid H) = I(Y; H \mid Z) \geq 0. \tag{2.7}$$

This follows by expanding conditional mutual information and using that conditioning on (H, Z) is equivalent to conditioning on H. Equality characterizes sufficiency for that target. It does not establish sufficiency for a different target, a changed objective, or a new tool interface. A representation optimized for next-token prediction may discard information needed for a rare future intervention.

The relevant target for control can be the optimal action, a value function, a constraint event, or a predictive distribution over consequences. These targets need not agree. Consider a binary historical fact B that has negligible effect on the average continuation text but determines whether a destructive operation is permitted. A compression objective dominated by common conversational tokens can tolerate losing B, while a risk-sensitive action objective cannot. This is a decision-theoretic explanation for evaluating compaction through subsequent behavior rather than summarization quality alone.

Define the Bayes decision risk for representation Z and loss ℓ by

$$R^*(Z) = \inf_d \mathbb{E}[\ell(d(Z), Y)]. \tag{2.8}$$

Because every decision rule on Z is also a decision rule on H, $R^*(H) \leq R^*(Z)$. The inequality is distribution-specific. It provides no numerical guarantee without assumptions on the lost information and the loss. A practical approximation is to train or evaluate a family of decision probes, including rare constraints, temporal updates, and adversarial distractors. Passing those probes constrains failure in the tested family; it does not prove universal sufficiency.

2.2.1 A bounded decision-loss consequence

Assume finite decision and target spaces, $Z = \varphi(H)$, and a fixed loss $0 \le \ell(a, y) \le L_{\max}$. Write $Q_h = P(Y \in \cdot \mid H = h)$ and $Q_z = P(Y \in \cdot \mid Z = z)$.

$$f(Q) = \min_a \mathbb{E}_Q \ell(a, Y), \qquad \mathrm{TV}(P, Q) = \sup_B |P(B) - Q(B)|. \tag{2.9}$$

The conditional Bayes risks satisfy $R^*(H) = \mathbb{E} f(Q_H)$ and $R^*(Z) = \mathbb{E} f(Q_Z)$. For any P, Q, using an optimizer for one distribution as a candidate for the other and then exchanging their roles gives

$$|f(P) - f(Q)| \le L_{\max} \mathrm{TV}(P, Q). \tag{2.10}$$

Consequently, with natural logarithms and finite conditional mutual information,

$$\begin{aligned} 0 \le R^*(Z) - R^*(H) &\le L_{\max} \mathbb{E}\, \mathrm{TV}(Q_H, Q_Z) \\ &\le L_{\max} \sqrt{\tfrac{1}{2} \mathbb{E}\, \mathrm{KL}(Q_H \| Q_Z)} \\ &= L_{\max} \sqrt{\tfrac{1}{2} I(Y; H \mid Z)}. \end{aligned} \tag{2.11}$$

The first inequality uses that a rule on Z is available to the decision maker observing H. The next steps use the bounded-loss variational inequality, Pinsker's inequality and Jensen's inequality [30]. The right side can be capped at $L_{\max}$. If information is measured in bits, replace I by $(\log 2) I$. This is a bound on the Bayes-optimal representation gap, not on the actual deployed policy. For a realized decision rule d_Z,

$$\mathbb{E}\ell(d_Z(Z), Y) - R^*(H) = [R^*(Z) - R^*(H)] + [\mathbb{E}\ell(d_Z(Z), Y) - R^*(Z)]. \tag{2.12}$$

The second term is the remaining decision-rule regret and is not controlled by the information bound. The target, loss, task distribution and decision space must remain fixed. A small average bound for one target does not certify a rare permission constraint or the truth of an effect ledger. Those remain distinct from the proposal-preservation, authority and effect contracts in Chapter 1.

Figure 2.1 exhibits the complementary pointwise obstruction. If two histories with distinct unique conditional optima collapse to one representation, a deterministic rule with only that representation cannot be optimal for both. A strict population risk increase additionally requires positive probability and positive excess loss on the affected histories or regions.

Rate-distortion language makes the resource tradeoff explicit:

$$\min_{p(z|h)} I(H; Z) \quad \text{subject to} \quad \mathbb{E}[\mathfrak{d}(H, Z)] \le \epsilon. \tag{2.13}$$

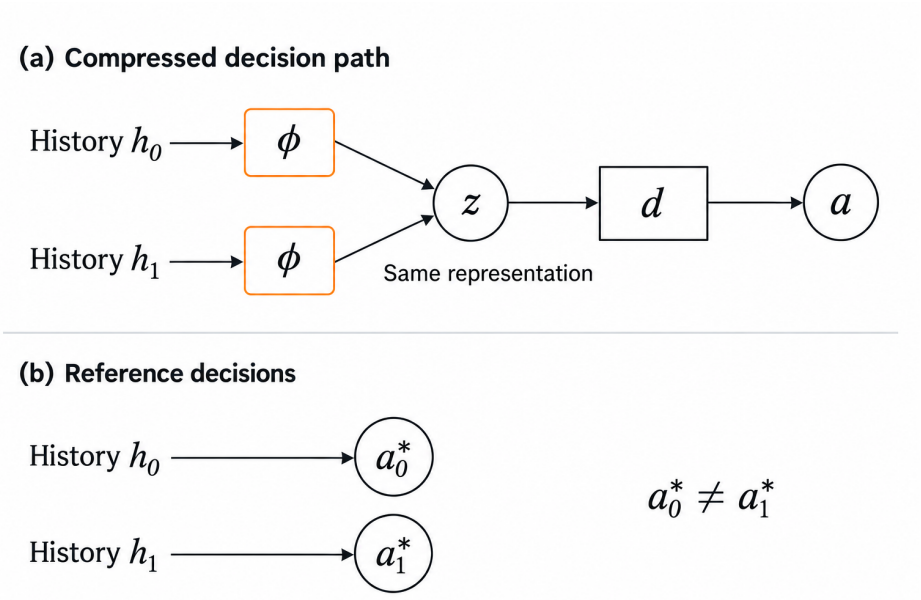

Figure 2.1: Representation aliasing is target-dependent. Distinct histories h_0, h_1 satisfy $\varphi(h_0) = \varphi(h_1) = z$, so a deterministic rule using only z returns the same $a = d(z)$. Under a declared loss with different unique conditional optimal actions a_0^*, a_1^*, that action cannot be optimal for both histories. This analytical construction contains no measurements; it does not quantify a population risk gap or exclude policies acquiring additional evidence.

For agents, a useful distortion $\mathfrak{d}$ should reflect decision loss or violated constraints. Token count is a proxy for representation cost, not mutual information itself. External memory also changes the problem: a compact working state can defer details to a store, but only if future retrieval is reliable and affordable. The combined representation is then the active state plus the retrieval mechanism, not the summary in isolation [31, 32].

If retrieval remains available, the decision experiment must specify its query policy, store version, observation channel, and cost. It then compares information-acquisition systems, rather than the one-shot risks of two representations under an unchanged information set.

2.3 Partial observability and beliefs

A finite POMDP consists of latent states $\mathcal{X}$, actions $\mathcal{A}$, observations $\mathcal{O}$, transition probabilities $P(x' \mid x, a)$, observation probabilities $Z(o \mid x', a)$, a reward function, and an initial distribution. A belief $b_t(x) = P(X_t = x \mid H_t)$ is a sufficient statistic for optimal control when the model is correct and the complete history is available. The Bayesian filter is

$$b_{t+1}(x') = \frac{Z(o_{t+1} \mid x', a_t) \sum_x P(x' \mid x, a_t) b_t(x)}{\sum_{\bar{x}} Z(o_{t+1} \mid \bar{x}, a_t) \sum_x P(\bar{x} \mid x, a_t) b_t(x)}. \tag{2.14}$$

The denominator must be positive. An observed event assigned zero predictive probability signals model or support mismatch, not a defined posterior update. Zero-probability observations contribute nothing to a model expectation but cannot be silently normalized when actually encountered. Belief sufficiency assumes a correct closed Markov model; changing tool versions, budgets, or permissions require an augmented state [33]. Tool environments rarely provide such a model explicitly. The belief-state formulation nevertheless clarifies what uncertainty remains after a tool response. An HTTP timeout can mean that a request never arrived, that it committed but its response was lost, or that it is still running. A belief over these states is more accurate than automatically labeling the action failed. A reconciliation query can have high value because it changes which retry decisions are safe.

The value of a belief under discount $0 \leq \gamma < 1$ satisfies

$$V^*(b) = \max_a \left[r(b, a) + \gamma \sum_o P(o \mid b, a) V^*(\tau(b, a, o)) \right], \tag{2.15}$$

where τ denotes belief updating. The maximization includes information-gathering actions. A policy that greedily chooses the action with the highest immediate completion probability can be inferior to one that first checks state. This is especially important when a cheap query prevents an irreversible mistake.

Actions have different durations. A fixed per-action discount can favor one hour-long operation over several short checks even when their physical-time consequences differ. Semi-Markov temporal abstraction discounts over holding time [34]; a continuous-time service model must also declare when rewards accrue. Let decision times be T_k, durations $\Delta_k = T_{k+1} - T_k$, utility rate $r_k(u)$, and completion reward g_k. For $\beta > 0$ in reciprocal time units, define

$$G_k^\beta = \int_0^{\Delta_k} e^{-\beta u} r_k(u)\, du + e^{-\beta \Delta_k} g_k. \tag{2.16}$$

Let ξ_k augment a sufficient belief with trusted authorization observations, governing versions, remaining budgets, and clock state whenever those affect the transition law. For nonterminal states with a nonempty feasible set, the corresponding equation is

$$V^*(\xi) = \sup_{a \in F_\alpha(\xi)} \mathbb{E}[G_k^\beta + e^{-\beta \Delta_k} V^*(\xi_{k+1}) \mid \xi_k = \xi, a_k = a]. \tag{2.17}$$

A state with no authorized continuation has a declared bounded terminal value; allowing further deliberation instead requires an explicit no-dispatch transition. The feasible set is supplied by current trusted authority, rather than inferred from a model's belief. Duration itself can be informative and belongs in the joint observation kernel. Bounded returns and nonexplosion conditions must be explicit: positive individual durations do not exclude infinitely many decisions in finite time. A continuing-time requirement is $\sum_k \Delta_k = \infty$ almost surely; on the bounded-value function space with uniformly bounded one-decision expected absolute returns, a uniform lower bound $\Delta_k \geq \Delta_{\min} > 0$ gives contraction factor $e^{-\beta \Delta_{\min}} < 1$. Finite horizons provide another well-defined setting. A linear latency penalty is a different modeling choice from exponential elapsed-time discount. Expected penalties also do not implement a hard authorization constraint.

For a tiny discrete environment, exact belief filtering and dynamic programming can provide an oracle baseline. In a real repository, browser, or enterprise system the state space is combinatorial and the transition model changes. Approximate filters, structured event records, and conservative uncertainty handling become implementation choices. The formal model should guide which approximations are tested rather than imply that exact POMDP planning is tractable.

The HRP source offers a bounded oracle baseline, not a general belief solver. In `environment/hrp_miniwob/oracle.py`, `solve` and `evaluate` operate on an ordered, fully observed deterministic state–action graph. The validator requires finite rewards, a strictly positive normalized reference policy, discount below one, and positive regularization temperature; dynamic programming computes regularized values and occupancies in floating-point arithmetic [9]. This does not implement the observation filter above. Partial observability requires a separate observation model and belief-state construction.

2.4 Optimization and policy-gradient identities

For supervised trajectories (c, y), the negative log-likelihood objective is

$$L_{\mathrm{SFT}}(\theta) = -\mathbb{E} \sum_i \log p_\theta(y_i \mid c, y_{<i}). \tag{2.18}$$

The student input and supervised target must define a declared learning task. Expert targets can be constructed using privileged information; this is not intrinsically invalid supervision, although the student may face irreducible uncertainty. A stronger requirement applies when the loss is interpreted as a rollout-conditioned likelihood or gradient: every scored decision must use the information that actually conditioned behavior. MemoryWalker makes that distinction concrete for context editing [23]. Offline relabeling may intentionally define another conditional task, but it cannot then be described as estimating the same behavior process.

For a differentiable deployed stochastic policy $\pi_{\theta,\delta}$, fixed decoding configuration δ, finite horizon and return $R(\tau)$, define $J(\theta) = \mathbb{E}_{\tau\sim p_\theta} R(\tau)$. Then

$$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau p_\theta(\tau) R(\tau) \tag{2.19}$$

$$= \mathbb{E}_{\tau\sim p_\theta} \left[R(\tau) \sum_t \nabla_\theta \log \pi_{\theta,\delta}(a_t \mid h_t) \right]. \tag{2.20}$$

This score-function identity [35] assumes that the environment dynamics and reward do not depend on θ through an unaccounted path, and that differentiation can pass through the expectation. If a learned reward or context policy shares parameters with the actor, the intended objective and gradient paths must be specified. Treating all feedback as a fixed scalar can silently omit dependencies. The probability law must be differentiable on the relevant support. A greedy decoder induces a Dirac action law; substituting raw softmax scores into its score-function identity generally estimates a different objective. Baseline fitting is detached from the actor score term unless the intended gradient explicitly includes that dependency.

A baseline $b(h_t)$ can reduce variance without changing the expected gradient because

$$\mathbb{E}_{a\sim\pi_{\theta,\delta}(\cdot|h)}[b(h)\nabla_\theta \log \pi_{\theta,\delta}(a \mid h)] = b(h)\nabla_\theta \sum_a \pi_{\theta,\delta}(a \mid h) = 0. \tag{2.21}$$

An action-dependent baseline requires a correction; otherwise it can bias the estimate. Advantage estimators, clipping, group normalization, and KL

penalties trade variance, bias, and stability under different assumptions. PPO and preference-optimization objectives are useful concrete examples, but they do not remove the need to preserve behavior conditioning and reward identity [36, 37].

In general, importance sampling requires the target trajectory law to be absolutely continuous with respect to the behavior law, and weights by their Radon–Nikodym derivative. The following action-product specialization applies only when all nonpolicy kernels and the recorded information semantics are shared. For data generated by behavior policy μ, it becomes

$$w(\tau) = \prod_t \frac{\pi_{\theta,\delta}(a_t \mid h_t)}{\mu(a_t \mid h_t)}. \tag{2.22}$$

The ratio is meaningful only when the behavior probability is known and the target distribution has support within the behavior distribution. A changed parser, retry controller, tool transition or authorization mechanism need not cancel from that trajectory ratio. A tool action made impossible by the old authorization policy cannot be recovered through a finite importance weight. Likewise, an unknown context-editing history can make the denominator undefined for the recorded action. Clipping large ratios can stabilize estimation but introduces a different estimator; it does not reconstruct missing information.

Agent Lightning makes this boundary inspectable. `RolloutAdapter.get_train_data_batch` in `agentlightning/verl/rollout_adapter.py` merges compatible token prefixes, masks inserted observations out of action loss, and omits batch `rollout_log_probs` if any row lacks usable captured values. `AgentLightningRayPPOTrainer._train_step` in `agentlightning/verl/trainer.py` requires finite captured log-probabilities in bypass mode; ordinary correction runs only when configured and the field is present [14]. These checks expose missing evidence. They do not establish equality between token probabilities and the action law after parsing, retries and authorization, or coverage of target-policy support.

2.5 Risk calibration and constrained decisions

An agent often needs to estimate whether an answer is correct, whether retrieved evidence remains valid, or whether an operation will violate a constraint. For a predicted probability $\hat{p}$ and binary outcome Y, calibration requires that events with predicted probability p occur at frequency p under the evaluated distribution. In conditional-expectation form, $\mathbb{E}[Y \mid \widehat{p}] = \widehat{p}$

almost surely under that law [38]. This score-level statement is weaker than identifying the full decision-information posterior $P(Y = 1 \mid \mathcal{G}_t)$. Calibration does not imply discrimination: a model that always predicts the base rate can be calibrated while failing to distinguish easy from hard cases. Ranking quality, probability calibration, and operational utility require separate evaluation.

Suppose executing an action yields known gain $g > 0$ if correct and known loss $\ell > 0$ if incorrect, while abstention has zero utility. If $p = P(Y = 1 \mid \mathcal{G}_t)$ is the correctness probability conditional on the decision information, execution is preferable when

$$pg - (1 - p)\ell > 0 \quad \Longleftrightarrow \quad p > \frac{\ell}{g + \ell}. \tag{2.23}$$

The threshold changes with consequence severity. Ordinary marginal calibration of a score is insufficient when errors or consequences vary with additional decision covariates; those covariates must be included or the utility evaluated within the appropriate strata. A confidence score calibrated on benign tool calls need not be calibrated on adversarially selected calls. Human escalation also has cost and delay, and may itself be fallible. Its value should be measured relative to a defined review process rather than treated as an infallible terminal action.

Chance constraints express a different objective. Let $Y_H \in \{0, 1\}$ indicate whether a declared violation occurs within the finite horizon H:

$$\max_{\pi} \mathbb{E}[R] \quad \text{subject to} \quad P_\pi(Y_H = 1) \leq \alpha. \tag{2.24}$$

Estimating a small α requires enough independent evidence about rare failures. Observing no violations in a small sample does not prove the probability is zero. Dependence across repeated sessions, shared memory, or common model failures reduces the information in nominal trial counts. A risk bound must identify both the sampling assumptions and the operational event being bounded.

Hard constraints can be enforced by a shield or validator that restricts the action set, but the guarantee is limited to the property that the validator actually checks. A schema validator can reject malformed arguments while accepting a well-formed unauthorized destination. A permission policy can restrict destinations while failing to detect false content. Composing controls requires examining whether their assumptions are compatible and whether one control changes the distribution seen by another.

2.6 Statistical comparison and adaptive selection

Let Y_{im} be the outcome for task i under method m. A paired comparison uses differences $D_i = Y_{i1} - Y_{i0}$ so that task difficulty is controlled. If multiple trials share a user, repository, or persistent memory trajectory, those trials are not independent samples merely because they have different random seeds. The resampling unit should preserve the dependence structure. A bootstrap over individual messages can produce narrow intervals that ignore the variation between whole task sequences.

For independent Bernoulli outcomes with probability p, finding at least one success in k attempts has probability $1 - (1-p)^k$, while all k attempts succeeding has probability p^k. At the hypothetical value $p = 0.8$ and $k = 3$, these are 0.992 and 0.512. These are analytical illustrations under independence, not benchmark measurements. Real attempts can be correlated through shared prompts, retrieved memories, or common tool failures. Reported pass metrics must use the exact benchmark definition rather than inherit this toy model.

Adaptive method selection introduces another source of optimism. If many harnesses are evaluated on the same finite task set and the best is retained, the retained score is selected partly for favorable noise. A held-out set used repeatedly to decide which candidate to keep has become part of the selection process. The required separation is operational: an evaluator cannot remain an untouched test merely because its file is named test. RRSI regularizes harness proposals and selection, but its cost/novelty shaping can admit candidates within a noise band; this is not simultaneous certification of promotions [27, 39].

Repeated looks at a fixed estimand and adaptive benchmark reuse are distinct. A confidence sequence supports optional stopping under its conditional stochastic assumptions [40]; it does not make candidates selected using the same task feedback independent of that feedback. REUSE instead proves simultaneous promotion control under an initially independent iid evaluation set, decision-only feedback, and error allocation over all possible promotion histories [41]. With t the round, p the number of earlier promotions, K tested candidates per history, and nonnegative weights predeclared before evaluation feedback and satisfying $\sum_t w_t \le 1$ and $\sum_p v_p \le 1$, its allocation is

$$\eta_{t,p} = \frac{\alpha w_t v_p}{\binom{t-1}{p} K^{p+1}}. \tag{2.25}$$

There are $\binom{t-1}{p} K^p$ possible promotion histories and K candidates at each, so the error budget summed over those tests is at most α. Revealing arbitrary scores or traces violates the decision-only feedback premise. The

reported experiment concerns classification-pipeline search on Covertype, not coding-agent or post-training validation. A theorem for that protocol is not a general guarantee for benchmark-guided self-improvement.

2.6.1 Simultaneous evidence for fixed candidates

Let $B_i \in \{0, 1\}$ mark a declared, observable contract failure in a fixed evaluation configuration. If the trials are independent and identically distributed with failure probability p, zero failures in a predeclared sample of size n gives the exact one-sided upper bound

$$p \leq 1 - \zeta^{1/n} \quad \text{at confidence } 1 - \zeta, \qquad 0 < \zeta < 1. \tag{2.26}$$

This follows by inverting $\Pr_p(B_1 = \cdots = B_n = 0) = (1 - p)^n$. It is neither a proof that $p = 0$ nor an authorization decision. If the checker can miss a true failure, the bound concerns detected failures unless its miss rate is independently bounded.

Repeated inspection and adaptive candidate selection need simultaneous coverage. One elementary construction uses fresh validation data for each candidate $j \geq 1$, fixes that candidate throughout its evaluation, and assigns

$$\begin{aligned} \zeta_{j,n} &= \frac{36\zeta}{\pi^4 j^2 n^2}, \qquad r_{j,n} = \sqrt{\frac{\log(2/\zeta_{j,n})}{2n}}, \\ C_{j,n} &= [\widehat{p}_{j,n} - r_{j,n}, \widehat{p}_{j,n} + r_{j,n}] \cap [0, 1]. \end{aligned} \tag{2.27}$$

Conditional on the information used to create candidate j, its validation trials must be independent Bernoulli observations with fixed mean p_j. Hoeffding's inequality [42] and $\sum_{j,n} \zeta_{j,n} = \zeta$ imply

$$\Pr\{p_j \in C_{j,n} \text{ for every } j, n\} \geq 1 - \zeta. \tag{2.28}$$

Thus choosing a candidate or stopping on these intervals preserves the stated coverage. The construction is conservative and is not a state-of-the-art confidence sequence. Reusing the same exposed holdout for adaptively constructed candidates violates the fresh-data premise. Continuing memory updates during validation also changes the estimand; treating the resulting sequence as trials of one fixed p_j is unjustified. An unresolved contract verdict must be reported separately or counted as an admission failure under a declared rule, not silently dropped.

For the proposal contract, paired sampled actions are not automatically a test of total variation. Under a known complete alphabet of size K,

predeclared sample sizes and independent draws within each law, a simple subset-union bound gives, with probability at least $1 - \zeta$,

$$\begin{aligned} \mathrm{TV}(p, q) &\le \min\{1, \mathrm{TV}(\widehat{p}_n, \widehat{q}_m) + r_n + r_m\}, \\ r_k &= \sqrt{\frac{(K+2)\log 2 + \log(1/\zeta)}{2k}}. \end{aligned} \tag{2.29}$$

Indeed, apply Hoeffding to every subset of the K outcomes, to both laws, and then use the triangle inequality. Large action alphabets make this bound weak. Coarsening an unbounded action space tests only the coarsened law and cannot upper-bound the original total variation without further assumptions. Authority and ledger equality can instead be checked exactly when their finite trusted sets and records are available; sampling their failures establishes only a distribution-scoped rate.

Table 2.1: Comparison questions and the controls needed to identify their effects.

Comparison question	Required control
Does memory help?	Matched policy and tools; declared persistence; task-order randomization; construction and retrieval costs included.
Does a team help?	Matched total inference and tool budget; independent-sample baseline; communication and judge access controlled.
Does compaction preserve decisions?	Same histories; rare constraints and changed facts; subsequent action and violation measurements.
Does an evolved harness transfer?	Independent environments and graders; frozen evaluation configuration; all candidate-search costs disclosed.
Does serving optimization help tasks?	Matched quality and offered load; warm/cold cache distinction; end-to-end tails and failures retained.

2.7 From equations to falsifiable implementation claims

The mathematical object in a paper and the operational object in code must be connected by an explicit correspondence. State variables need storage

locations and update semantics. Conditional probabilities need recorded conditioning inputs. An algorithm's action set must match the tool registry and authority available during execution. A complexity expression needs tensor shapes, precision, batch structure, and communication assumptions. A confidence interval needs a sampling unit and a treatment of missing outcomes. These are not documentation details added after the science; they determine which claim the implementation actually tests.

A useful validation procedure begins with a minimal environment whose latent state is fully known. Introduce one controlled departure at a time: omitted observations, delayed tool responses, a lossy context transform, a stale memory, an evolving schema, or a noisy verifier. Check that the observed failure matches the mechanism predicted by the formal model. This establishes a diagnostic connection before attempting large-scale benchmark comparisons. It does not substitute for realistic evaluation, because the minimal environment may omit the interactions that dominate deployment.

The most useful deductions are often impossibility or insufficiency results. If two histories are aliased by a representation but their required-action sets are disjoint, no downstream deterministic decoder with only that representation can satisfy both requirements. Figure 2.1 gives the unique-optimum case. If a behavior policy assigns zero probability to an action needed by the target policy, importance sampling cannot estimate that target outcome from the observed support. If two systems receive different evaluator information, an accuracy difference does not isolate the proposed learning mechanism. These observations limit overclaiming and identify the additional instrumentation or intervention required to make progress.

A proposed diagnostic reports checker errors, observable contract failures, unresolved verdicts, task outcomes, and recovery cost separately. It freezes the candidate during the stated validation regime or explicitly changes the estimand when policy or persistent state adapts. An unknown service outcome must not disappear merely because it is difficult to score.

The remaining chapters apply these principles to model interfaces, latent representations, context transformations, memory transactions, tool protocols, evaluation, and infrastructure. Mathematical precision is valuable when it changes the experiment or implementation decision. A symbolic objective without a realizable information set or a measurable failure event provides little assurance about an autonomous system.

Chapter 3

Foundation Model Policies: Tokenization, Architectures, and Action Interfaces

3.1 DeepSeek V4.1 and architecture level prefill reuse

The September 2026 DeepSeek-V4.1-Flash report makes prefill reuse an architectural choice. Its Section 2.2 derives upper-layer global KV projections from a middle-layer hidden state, while retaining layerwise sliding-window computation. Section 2.3 distinguishes full, reindex, and reuse modes for compressed sparse attention; the report also describes a four-bit main KV cache [43]. These mechanisms differ from merely allocating an ordinary dense cache more efficiently. Their correctness and quality depend on a model trained with those structural choices, so they cannot be applied as a transparent replacement for every earlier checkpoint.

In a schematic causal encoder-decoder with L layers, n prompt tokens, and bounded replay of w tokens in the upper half, a linear-in-length layer-cost model gives a prefill ratio of $(nL/2 + wL/2)/(nL) = (1 + w/n)/2$. This is an analytical model of the stated decomposition, not a measured speedup. If n is much larger than w, the ratio approaches one half. If $n = w$, the modeled reduction disappears. Actual projection, attention, index construction, and scheduling costs can move the result. The formula nevertheless identifies why a long-document cache miss and a short tool-response turn are not interchangeable workloads.

The same case qualifies a blanket invalidation rule while requiring a distinction between structural reuse and approximation. The report's Section 3.2.2 explicitly truncates sliding-window replay to the recent window and accepts approximate states; exact reconstruction would require a longer dependency history. Cached global projections and bounded sliding-window replay therefore have different equivalence claims [43]. In a conventional dense decoder, later representations may depend on the entire prefix. A trained sharing architecture changes that computation graph, but does not make every reconstruction exact. Bind each reuse invariant, approximation,

and quality test to the checkpoint and implementation. A cache produced for a different projection path, compression mode, or precision is not compatible merely because its token positions match.

A discriminating experiment compares cold prefill, exact eligible-prefix reuse, and repeated short tool turns while keeping the checkpoint, quality tasks, and external tool behavior fixed. Inspect physical cache bytes and actual recomputed tokens, not only the advertised context length. Include edits to an early permission constraint and questions requiring precise old identifiers. A model-specific compression method can preserve aggregate long-context scores while failing these rare distinctions. Conversely, a specialized architecture may achieve substantial resource savings without sacrificing the tested distinctions. The result must remain attached to the actual mechanism and workload rather than a generic claim of superior memory.

3.2 The model distribution and the executed policy

A foundation model supplies conditional token probabilities; the deployed decoder determines the distribution actually sampled. An agent system constructs the input, interprets sampled outputs, and controls their external effects. Let H_t denote recorded interaction history, M_t external memory, and $C_t = g(H_t, M_t)$ the context presented at decision time. Write $Q_{\theta,\delta}(y \mid C_t)$ for the sequence law induced by a checkpoint and decoding procedure δ, including temperature, grammar masks, truncation, and stopping. The declared output space includes length-limit and incomplete-generation outcomes; a finite-output probability law must not silently discard their mass. For a deterministic interpreter d, the proposal distribution is

$$\pi^{\mathrm{prop}}_{\theta,\delta}(a \mid C_t) = \sum_{y:d(y)=a} Q_{\theta,\delta}(y \mid C_t). \tag{3.1}$$

Ordinary ancestral sampling without extra transforms gives $Q_{\theta,\delta} = p_\theta$; greedy decoding generally does not. The sum includes multiple serializations of the same operation and a rejection outcome for malformed outputs. A randomized repair or interpreter instead requires a kernel $D(a \mid y, C_t)$ and the sum $\sum_y Q_{\theta,\delta}(y \mid C_t) D(a \mid y, C_t)$. Thus a repaired operation cannot be assigned the raw probability of one convenient serialization.

Authorization is a second transformation. For a gate that rejects an infeasible proposal without resampling, let $F_\alpha(S_t)$ be the feasible-action set under current authority epoch α and $\perp$ the no-execution outcome. Then,

for $a \neq \bot$,

$$\pi^{\text{exec}}(a \mid S_t) = \pi^{\text{prop}}_{\theta,\delta}(a \mid C_t)\mathbf{1}\{a \in F_\alpha(S_t)\}. \tag{3.2}$$

For a countable proposal space, the remaining mass is explicit:

$$\pi^{\text{exec}}(\bot \mid S_t) = \pi^{\text{prop}}(\bot \mid C_t) + \sum_{a \neq \bot:\, a \notin F_\alpha(S_t)} \pi^{\text{prop}}(a \mid C_t). \tag{3.3}$$

The proposal and execution distributions are normalized only when invalid, denied, and terminated-without-operation outcomes remain represented. This distribution is not obtained by renormalizing over feasible actions unless the executor actually resamples until acceptance under the corresponding assumptions. Proposal quality, refusal behavior, and executed-action quality therefore require separate denominators. This distinction connects the model boundary to the authority contract in Chapter 29.

The context constructor, decoding rule, and interpreter are therefore part of the deployed policy. Temperature changes the distribution before interpretation. A grammar mask removes sequences. A retry loop conditions the eventual output on previous failures. A tool router can remap a nominal operation to a different implementation. Comparing two checkpoints while changing these mechanisms does not isolate a model effect. Conversely, a harness can improve end-to-end reliability without changing any model parameter. The agent-computer interface studied in SWE-agent is a concrete demonstration that the interface deserves independent analysis, although its reported task results do not imply a universal interface advantage [2].

The state exposed to the model need not be the complete execution state. A scheduler may retain locks, transactions, budgets, or authorization decisions outside C_t. Denote complete system state by $S_t = (H_t, C_t, M_t, E_t, V_t)$, where E_t is environment state and V_t collects compatible versions. The implemented transition has the form $S_{t+1} = F(S_t, a_t, o_{t+1})$. A language model only parameterizes one conditional component. Statements that the model remembers, plans, or recovers should specify which information and transitions are supplied by the surrounding system. Otherwise a property of durable storage can be incorrectly attributed to model weights.

An important consequence concerns evaluation units. A successful sampled completion is evidence about the composition of model and decoder under one context. A successful completed task is evidence about a trajectory through a particular harness and environment. A successful resumed task additionally tests state reconstruction. These events have different denominators. A model that emits more valid JSON may still perform fewer useful operations if the schema encourages an incorrect decomposition. A smaller model can outperform a larger one when its tool arguments are

calibrated to the available interface. Neither observation establishes a general ranking without matched tasks and resources.

For implementation, the policy boundary should expose the exact serialized request, model and tokenizer identities, decoding settings, selected tool schema, raw output, parsed output, and execution outcome. Sensitive fields can be redacted in an audit representation, but their existence and transformations must remain identifiable. Logging only the final natural-language answer destroys evidence needed to distinguish a model error from parser repair or tool failure. This is particularly damaging during post-training: the gradient must be associated with the conditions under which an action was actually generated, rather than a convenient reconstruction assembled after the task.

3.3 Tokenization as a measurement and interface choice

Let b be a byte string and T a canonical encoding into token indices $(x_1, \ldots, x_n)$. Its teacher-forced token likelihood factors as $\prod_i p_\theta(x_i \mid x_{<i})$, with termination handled under a declared convention. A per-token loss depends on T and cannot directly rank byte-level predictive efficiency across tokenizers. For a fixed corpus of byte strings, a useful operational quantity is total negative log probability in base two divided by the total number of original bytes. Report normalization and boundary-token conventions with that code-length measure.

Canonical-encoding likelihood is also distinct from the probability of a decoded byte string when several token sequences decode to it: the latter sums their sequence probabilities. A lossy normalizer can additionally map distinct original byte strings to the same encoding. These distinctions matter when assigning probabilities to exact tool arguments. BPE merges frequent adjacent units; SentencePiece supplies tokenization with explicit normalization choices. Neither mechanism alone establishes the optimal vocabulary for every language, codebase, or tool protocol [44, 45].

An agent handles material with very different token statistics: prose, source code, URLs, paths, JSON, tables, OCR output, and arbitrary identifiers. A tokenizer that is economical on conversational English can fragment a long hexadecimal identifier into many pieces. That fragmentation increases context consumption and the number of predictions needed to reproduce an argument exactly. It can also alter where truncation occurs. A context limit expressed as a token count is consequently a capacity of a specific encoder, not a language-independent quantity of evidence. Cross-

model memory comparisons should report the underlying content and its encoded length for each tokenizer.

Normalization introduces a second issue. Unicode composition, whitespace treatment, casing, and byte fallback influence whether encoding and decoding preserve a tool argument exactly. Human-readable equivalence does not imply operational equivalence. A database key can distinguish two strings that render similarly; a shell interpreter can assign meaning to whitespace that a prose normalizer treats as redundant. The relevant invariant for an execution boundary is $D(T(b)) = b$ for the supported domain, with the domain explicitly stated. If preprocessing intentionally changes b, the system should retain the original bytes and the transformation rather than describing the result as lossless tokenization.

A tokenizer is not an authorization mechanism. Reserved role markers and tool delimiters can assist serialization, but the application must control which message fields can introduce them. An untrusted document containing the spelling of a privileged marker must remain document content. The parser and transport determine authority; the statistical model can still be influenced by a convincing imitation even when the actual privilege boundary is preserved. This distinction is central to prompt injection. Preventing a byte sequence from becoming a system message is necessary, while preventing every semantic influence from untrusted content is a separate learning and control problem.

Vocabulary size creates an engineering tradeoff. With hidden width d and vocabulary size v, an untied output projection has dv parameters and computes a distribution over v alternatives at each prediction. Tying input and output embeddings reduces stored parameters but does not remove the output computation. Increasing v can shorten sequences while increasing projection cost and making rare units harder to estimate. The optimum depends on corpus composition, architecture, batch shape, and hardware. It cannot be selected solely by minimizing tokens per character. A multilingual agent should also examine whether equal task budgets allocate systematically different usable text across languages.

3.4 Architectural state and access to history

The Transformer makes a position-dependent representation by mixing previous token representations through attention and applying learned nonlinear transformations. For one attention head, $Q = XW_Q$, $K = XW_K$, and $V = XW_V$, with $\mathrm{softmax}(QK^\top/\sqrt{d_h} + A)V$ defining the output under additive mask A. A causal mask prevents access to future positions. This is

Table 3.1: Tokenizer properties and agent-level consequences.

Property	Direct mechanism	Required check
Segmentation	Changes sequence length and prediction units	Compare encoded lengths for actual tools and languages.
Normalization	May alter bytes before inference	Test round trips on identifiers, code, and arguments.
Special tokens	Distinguishes serialized message roles	Reject role escalation at the application boundary.
Vocabulary size	Trades projection size against length	Measure total task compute, not token count alone.

a constraint on the computational graph, not a claim that the representation is a faithful account of everything in the past. Positional information, finite precision, training distribution, and learned attention patterns all affect what historical evidence influences a decision [46].

Dense attention offers direct content-dependent access to retained positions, with quadratic attention arithmetic during a full sequence pass. A recurrent architecture compresses past information into a bounded state. Transformer-XL introduces segment-level recurrence, while selective state-space models such as Mamba use input-dependent state updates designed for efficient sequence processing [47, 48]. The important comparison for agents is not whether an architecture has memory in an informal sense. It is what can be retained, accessed, modified, or invalidated under a fixed storage and computation envelope. A recurrent state can be compact while losing a rare constraint that an external event log still preserves.

Consider a deterministic recurrent update $z_{t+1} = f_\theta(z_t, x_t)$. If two distinct histories map to the same z_t, every subsequent decision based only on that state must treat them identically until new observations distinguish them. When the histories require different authorized actions, this state aliasing imposes an irreducible error for that interface. Attention can avoid that particular collision by retaining explicit positions, but it does not eliminate errors of selection or interpretation. External retrieval changes the accessible evidence again. Consequently, a fair comparison must hold the external memory service constant or explicitly attribute gains to the combined architecture.

Rotary position embeddings encode relative phase relationships in queries and keys. Their algebraic ability to evaluate larger positions does not establish useful reasoning at those positions [49]. Extending a context window can change frequency regimes, attention distributions, and numerical behavior. An agent with a long session also encounters a nonuniform evidence distribution: recent tool responses, early user constraints, and intermediate summaries have different roles. Tests that merely insert one recoverable string into a long input do not fully measure the ability to reconcile those roles. Nominal length and effective decision context therefore require separate reporting.

Mixture-of-experts architectures introduce conditional parameter activation. A router selects a small subset of experts for each token, so total stored parameters exceed the parameters active in one forward path [50]. This can improve the capacity-to-compute tradeoff, but expert weights still occupy memory and communication can dominate when routing crosses devices. An agent workload with small batches and heterogeneous contexts may exercise a different routing and communication regime from the large packed sequences used in training. Total parameter count is thus insufficient to predict latency, cost, or quality. Active FLOPs, resident memory, expert imbalance, and interconnect behavior belong in the comparison.

Grouped-query attention shares key and value heads across groups of query heads. It reduces the dimensionality of cached keys and values while retaining multiple query projections [51]. The reduction in KV storage is exact for a specified layout; the quality effect is empirical and depends on training or conversion. Calling grouped-query attention a larger context mechanism is incomplete: it makes a given capacity cheaper, but does not guarantee that the model uses additional historical information correctly. The distinction matters when a product increases the advertised window after an infrastructure change without changing its evidence-use evaluation.

Multi-head latent attention uses a different factorization. The DeepSeek-V3 report's Section 2.1.1 jointly compresses content keys and values into a latent vector and retains a separate rotary-position component [52]. In schematic column-vector notation, let $c_j = W_D h_j$, $k_j^C = W_K c_j$, and $v_j = W_V c_j$. The content score obeys $q_i^\top W_K c_j = (W_K^\top q_i)^\top c_j$, allowing a compatible implementation to score against the compressed state without storing every expanded content key. The separate positional contribution must still be handled; ignoring it does not reproduce the trained attention computation.

If the retained latent has dimension r and the extra positional key has dimension d_R, an ideal per-token cache stores $r + d_R$ elements instead of

$2hd_h$ for ordinary equal-width multi-head keys and values. This is a shape-based accounting identity, not a quality guarantee or an assertion that every implementation achieves the ideal storage. Up-projection arithmetic, query transformations, tensor layout, and kernel support affect performance. For an agent, this architecture changes the cost of preserving a long interaction history while leaving the separate question of decision-relevant recall unresolved. It should be compared with grouped-query and sparse-attention alternatives under matched quality and actual serving workloads.

3.4.1 Shared computational state in looped Transformers

The October 2026 Looped Prediction Transformer study trains repeated applications of a shared block stack to read the first recursion's full cache while retaining short local caches in later recursions. Its Section 3 specifies joint attention to shared and local entries in one softmax [53]. This is architectural recurrence within token prediction. It is neither a store of user sessions nor an ordinary deeper network with independently parameterized layers. The same parameters participate repeatedly, changing the computation and gradient paths without multiplying the stored block weights.

Let L be the shared stack depth, R its recursion count, T the retained token count, and w the local window for later recursions. Suppressing constant bytes and head dimensions, the two cache layouts have sizes

$$M_{\text{separate}} \propto LRT, \tag{3.4}$$

$$M_{\text{shared}} \propto L[T + (R-1)w]. \tag{3.5}$$

Their ratio is $[1 + (R-1)w/T]/R$. At $R = 5$, $T = 4096$, and $w = 64$, it is 0.2125. This calculated storage ratio does not imply a comparable reduction in computation: every recursion still evaluates the shared stack, and later queries read both shared and local evidence. Under the same simplified per-token read model, the extra local attention entries are a fraction $(R-1)w/(RT) = 0.0125$ of the separate-cache baseline.

Parameter sharing also constrains the function class. Repeating one transformation does not reproduce an arbitrary sequence of independently trained transformations, while explicit shared-cache access changes the available state at each recursion. A useful experiment separates weight tying, recursion count, local-window size, and shared-cache visibility at matched training exposure. Detaching the shared-cache gradient path tests a different mechanism from merely preventing later recursions from reading it. The source study analyzes these distinctions, but no universal benefit of increasing recursion depth follows from the architecture alone.

For agents, internal recursion spends computation without acquiring a new external observation. It can refine a prediction from existing evidence, but cannot resolve an unobserved transaction outcome by itself. Its cache remains tied to the model computation and does not establish durable cross-session memory. Compare extra internal recursions with external sampling, tool verification, or another observation under a declared cost envelope. These alternatives supply different information, so both their resource use and their evidence access must be reported when attributing gains to deeper reasoning.

The projection placement changes which quantities require expansion (Figure 3.1).

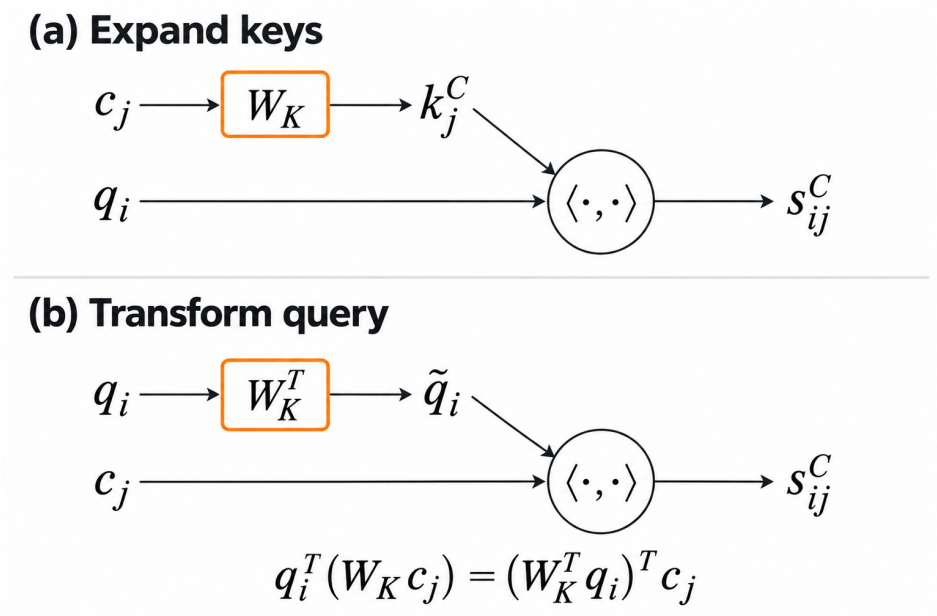

Figure 3.1: Two algebraically equivalent content-score paths, $s_{ij}^C = q_i^\top (W_K c_j) = (W_K^\top q_i)^\top c_j$, with transformed query $\tilde{q}_i = W_K^\top q_i$. Moving the linear projection permits a compatible implementation to score retained latents without storing expanded content keys. The positional score and value path are omitted; finite-precision implementations require numerical comparison. No measured performance result is depicted.

3.5 A resource model at the policy boundary

Let a decoder have L layers, model width d, feed-forward width m, and retained sequence length n. Ignoring biases, normalization parameters, and architectural variations, a dense layer with four d by d attention projections and two feed-forward matrices has approximately $4d^2 + 2dm$ parameters. A gated feed-forward layer generally adds another dm matrix. These expressions are accounting identities for the stated components, not universal model-size formulas. They help identify which assumptions change when comparing architectures. Embeddings, experts, vision modules, and output heads must be added rather than hidden in a single parameter multiplier.

For a full sequence pass, dense projection work scales approximately as nd^2 and ndm, whereas attention score and value products scale as n^2d. Constants depend on causal masking, multiply-add conventions, and implementation. During autoregressive decoding, the new query interacts with all retained keys, giving attention work proportional to nd per generated token and layer. Caching avoids recomputing old projections but introduces persistent tensors whose size grows with the session. The detailed memory and bandwidth analysis follows in Chapter 4; here the consequence is that an agent's historical input and output lengths jointly determine its resource footprint.

A hypothetical calculation makes tokenizer interaction explicit. Suppose two tokenizers encode the same complete interaction in 100,000 and 125,000 tokens. Under otherwise identical dense prefill assumptions, the quadratic attention term has ratio $(1.25)^2 = 1.5625$, while a linear projection term has ratio 1.25. The total compute ratio lies between these values when both components have positive cost and all other terms are unchanged. This is a deduction about a simplified accounting model, not a measured speedup or a realistic claim that two different models differ only in tokenization. Actual kernels, padding, caching, and data movement can change wall-clock behavior substantially.

At the action level, the expected cost also includes failed serializations. If one independent attempt costs c and has probability q of passing a syntactic validator, unbounded retries have expected attempt count $1/q$ when $q > 0$. This calculation says nothing about semantic correctness. A parser can accept a dangerous or irrelevant action. With at most k attempts, the success probability under the same independence assumption is $1 - (1 - q)^k$, and the expected number of attempts is $\sum_{i=0}^{k-1}(1 - q)^i$. Correlated errors, state changes between attempts, or adaptive repair invalidate the simple model and must be measured rather than assumed away.

An agent comparison should therefore report valid-action rate, task success, input and output bytes, model-specific token counts, actual calls, tool expenditure, and wall-clock distribution. A price per million output tokens is not a task cost when one system repeatedly replays a long history or performs more retries. A faster kernel can free a budget that the harness then spends on additional search. Whether that improves reliability is an empirical question. Resource savings should be measured at both the model-call boundary and the complete task boundary to avoid converting an implementation improvement into an unsupported capability claim.

Table 3.2: Calculated effects under deliberately simplified assumptions.

Change	Calculation	Interpretation
Length multiplied by 1.25	Linear term 1.25; quadratic term 1.5625	Compute components only; no latency claim.
Independent parse acceptance $q = 0.8$	Mean unbounded attempts 1.25	Syntax acceptance is not task success.
Three parse attempts, $q = 0.8$	Acceptance $1 - 0.2^3 = 0.992$	Assumes identical independent trials.

3.6 From model output to a typed operation

A typed action interface should distinguish prediction, validation, authorization, execution, and observation. Prediction selects a candidate operation and arguments. Validation checks syntax and local invariants. Authorization checks whether the candidate is permitted in the current state. Execution applies the operation. Observation reports what actually happened, including partial completion. Combining these stages into a single model-generated sentence removes useful control points. It also makes failures difficult to attribute: a statement that a file was updated may describe intent rather than an observed state transition.

Let $v(a, S_t)$ indicate structural validity and $u(a, S_t)$ indicate current authorization. A gated execution wrapper applies an operation only when both predicates hold. It should not assume that authorization inferred when constructing C_t remains valid when the operation executes. Permissions can be revoked, resources can change, and another agent can acquire a lock. The model's description of permission is evidence about its belief, not the permission itself. The authoritative check belongs to the execution

boundary. This remains true even if the model was trained extensively on compliant demonstrations.

Schema-constrained decoding removes token sequences that cannot be extended into valid outputs. This can improve syntactic validity but changes the sampling distribution. If the unmasked probabilities are p_i and the allowed set is A, the constrained distribution is $p_i / \sum_{j \in A} p_j$ for $i \in A$, assuming ordinary renormalization and positive allowed mass. If $\sum_{j \in A} p_j = 0$, this formula is undefined; the implementation must return an explicit generation failure or a separately specified fallback, not divide by zero. Likewise an empty feasible grammar continuation and a length-truncated valid prefix need distinct outcomes. A likelihood computed without the same mask is not the likelihood of the deployed action generator. The distinction becomes consequential in policy optimization, calibration, and off-policy analysis. Grammar versions and tool schemas should therefore be versioned alongside the checkpoint and tokenizer.

An abstract execution boundary can be specified as a composition with explicit rejection branches. Let Σ_t be the selected tool schemas, ν_t their configuration record, and σ the serialization map. With decoder δ and deterministic interpreter d as defined in Section 3.2,

$$\begin{aligned} \zeta_t &= \sigma(C_t, \Sigma_t, \nu_t), \\ Y_t &\sim Q_{\theta,\delta}(\cdot \mid \zeta_t), \\ a_t &= d(Y_t), \qquad b_t = v(a_t, S_t). \end{aligned} \tag{3.6}$$

The validity predicate assigns zero to the invalid symbol $\perp$ and to schema violations. It checks the selected interface rather than silently repairing its meaning. Only when $b_t = 1$ does the runtime obtain a current authoritative read ξ_t, refresh its execution state as $\widetilde{S}_t = \mathcal{R}(S_t, \xi_t)$, and evaluate $\beta_t = u(a_t, \widetilde{S}_t)$. The read supplies current service state and the applicable authority epoch. Let e_v and e_u be distinct non-execution outcomes for invalidity and denial. Let k_t identify the logical operation instance and its immutable arguments, with retries of that instance retaining the same identity. Let $\mathcal{X}_k$ execute an operation under identity k, with external randomness ω_t, and let $\mathcal{O}_{\nu_t}$ record the returned evidence and configuration. The boundary returns

$$e_t = \begin{cases} e_v, & b_t = 0, \\ e_u, & b_t = 1,\ \beta_t = 0, \\ \mathcal{O}_{\nu_t}\left(\mathcal{X}_{k_t}(a_t, \widetilde{S}_t; \omega_t)\right), & b_t = \beta_t = 1. \end{cases} \tag{3.7}$$

Execution appears only in the authorized branch. State refresh and authorization must be synchronized with dispatch under the service contract;

writing this composition does not prove distributed atomicity. The observation record can contain failure, partial completion, or an unknown outcome; it does not turn a returned message into proof of a durable effect. These maps specify implementation obligations, not properties automatically supplied by an SDK. Duplicate-effect prevention additionally requires service-appropriate idempotency semantics for k_t; a random request identifier alone is insufficient.

Tool descriptions are themselves inputs to the model. Their ordering, length, examples, and similarity can influence routing, even when every description is accurate. A model can also infer unavailable capabilities from familiar tool names. Robust interfaces return explicit unsupported-operation failures and avoid describing aspirational features as installed functionality. Toolformer studies learning to insert API calls under specific data and filtering conditions; its method is relevant to learning this interface, but it does not establish correctness for arbitrary tool inventories or changing enterprise permissions [29].

3.7 Reproducible implementation and discriminating experiments

An implementation record should separate architecture configuration from model weights, tokenizer artifacts, and policy-wrapper code. The configuration specifies head counts, sharing patterns, dimensions, positional treatment, context limits, and expert routing where applicable. The tokenizer record specifies vocabulary, merge rules or learned model, normalization, special tokens, and decoder behavior. The wrapper record specifies message serialization, grammar masks, parser handling, retries, and routing. A single model name rarely fixes all three. Reproduction requires immutable artifacts or hashes and a statement of any unavailable component.

The appropriate experiment is a factorial comparison rather than a universal leaderboard. Hold a checkpoint fixed while varying tokenizer-compatible serialization, schema ordering, constrained versus unconstrained decoding, and parser behavior. Separately compare model checkpoints under a common wrapper when their interfaces permit it. Include exact-copy tasks for identifiers, multilingual instructions, code editing, and multi-step tools with changing state. The outcome set should include useful task completion, rejected operations, invalid arguments, permission violations, and resource consumption. These measurements identify whether a change improves language modeling, interface compatibility, or the execution policy.

For architecture comparisons, define the resource axis before select-

ing models. Matching total parameters favors some sparse architectures; matching active FLOPs ignores resident memory; matching wall-clock time depends on implementation maturity. Report several axes when feasible and avoid declaring a single comparison intrinsically fair. A recurrent model with external retrieval should be compared with attention models using the same retrieval service, plus an ablation without retrieval. Otherwise memory infrastructure can conceal the architecture's actual failure mode. Token budgets should be translated into underlying bytes and evidence units as well as reported in native tokens.

Contamination controls extend to tools and interfaces. A benchmark can leak its expected API names or canonical action traces even if final answers are held out. Split by task template, environment instance, and repository lineage where those structures matter. Randomize harmless tool ordering to test sensitivity while retaining a fixed canonical order for the main result. Use paired task instances and repeated runs to distinguish stable gains from sampling variation. A changed model response format should not silently invalidate the baseline parser; both systems need a documented, equally capable integration.

The expected failure taxonomy is mechanistic. Segmentation failures corrupt exact strings; serialization failures cross message boundaries; contextual failures omit relevant constraints; routing failures select a wrong operation; authorization failures execute an impermissible one; semantic failures produce a valid but ineffective action. These categories can co-occur and should not be forced into a single label when evidence is ambiguous. A retained raw request and observed transition allow later reclassification. Without that evidence, a reported model mistake may actually be an execution race or a lossy context transformation.

3.8 Implications for stateful agents

The foundation model is a conditional computation embedded in a stateful control system. Its capabilities remain important: stronger representations, broader training coverage, and better action distributions can reduce the burden on the harness. Nevertheless, the useful unit of analysis is the composition of tokenization, conditioning, decoding, interpretation, and execution. A model can only act on information available through that composition. Persistent storage helps only when the evidence reaches a representation and interface through which it can affect the correct decision.

This observation constrains claims about memory and self-improvement. Replacing a tokenizer or changing special-message serialization can alter

the meaning of a stored token sequence even if the original text survives. Updating model weights can change how a procedural memory is interpreted. A context-editing harness can invalidate cached computational state. Compatibility is therefore more than a database schema version: it includes the model, tokenizer, prompt construction, action grammar, tool semantics, and authorization epoch. The detailed consistency analysis in Chapter 17 builds on these interfaces rather than treating memory as an isolated retrieval component.

The engineering implication is to maintain explicit boundaries while studying their interactions. Save canonical external records in a representation that preserves their intended semantics and provenance, reconstruct model-specific inputs under a declared policy, and evaluate behavior after every material interface change. Do not infer indefinite competence from a larger window, universal memory from a recurrent state, or safe execution from valid JSON. Each property has a measurable operational definition, a specific implementation boundary, and failure conditions that can be tested independently before assessing the complete agent.

A precise limitation of compressed architectural state. The state-aliasing argument can be strengthened into a decision bound. Suppose two histories h_0, h_1 occur with probabilities p and $1-p$, produce the same exposed state z, and require distinct unique correct actions a_0, a_1. Assume no additional observation arrives before the action. Any randomized policy conditioned only on z has success probability

$$p\pi(a_0 \mid z) + (1-p)\pi(a_1 \mid z) \leq \max\{p, 1-p\}. \tag{3.8}$$

The proof uses $\pi(a_0 \mid z) + \pi(a_1 \mid z) \leq 1$: the linear objective is maximized by allocating all probability to the more frequent required action. Hence the minimum error is at least $\min\{p, 1-p\}$. This is an elementary deduction under the stated observational restriction, not a new impossibility result for any particular architecture. External retrieval can remove the restriction by supplying evidence that distinguishes the histories. A permission check can prevent execution of a wrong action, but cannot reconstruct the missing task information by itself.

This bound suggests a targeted architectural evaluation. Construct paired histories that differ in one decision-critical fact, then measure whether the system's retained representation and available retrieval preserve that distinction. Keep the final query identical. Include an explicit-record baseline, a recurrent-state-only baseline where accessible, and an external-memory baseline with equal retrieval budgets. Balance the two histories

so that a default action cannot obtain a misleadingly high score through prior frequency. The test measures a particular lost distinction; it cannot establish that every important distinction survives. Nevertheless, it is more diagnostic than averaging many easy tasks whose correct action is unchanged by the omitted history.

Chapter 4

IO-Aware Attention Kernels and Computational State Allocation

4.1 FlashAttention 4 and asymmetric hardware scaling

FlashAttention-4 addresses Blackwell through a redesigned asynchronous pipeline, tensor-memory use, software-assisted exponentials, and a two-CTA backward strategy. Its Section 5 reports B200 BF16 results up to 1613 TFLOPs/s, with maxima of 1.3× against cuDNN 9.13 and 2.7× against its Triton comparator. The paper's Figure 4 also states that newer cuDNN versions incorporated related techniques and reached similar performance [54]. Those version-qualified observations rule out using the original maximum as a timeless vendor-library ranking.

The pinned forward class is `FlashAttentionForwardSm100` in `flash_attn/cute/flash_fwd_sm100.py`, with `mma_ping_pong_qk`, `softmax_loop`, and `correction_loop`; `FlashAttentionBackwardSm100` is in the corresponding backward module [55]. The important comparison with FA3 is the placement and overlap of work on hardware resources, not a different semantic attention objective. A faster matrix engine can leave exponentials or shared-memory traffic dominant. A correction stage removed from a critical path can help even when total arithmetic remains similar. This is why counting FLOPs alone can miss the mechanism of a kernel improvement.

An independent resource model makes the implication explicit. Let a tile require matrix time t_m, nonlinear time t_s, and movement time t_b on separately overlappable resources. An ideal pipeline cannot complete faster than $\max(t_m, t_s, t_b)$ per steady-state tile. Doubling only matrix throughput replaces t_m by $t_m/2$; when t_s already exceeds $t_m/2$, the full twofold gain is impossible without changing nonlinear work or overlap. This deduction does not require assigning an invented bandwidth to B200. It explains which measured counters could falsify a proposed bottleneck account.

Software approximations of elementary functions additionally need numerical auditing. The relevant comparison includes error before and after

conversion to the attention input precision, extremes of the logit distribution, and repeated long-sequence reductions. Approximation of an exponential within an implementation is different from removing attention positions. Both can alter output, but their error models differ. A code review should trace the selected architecture, dtype, head dimensions, and compile-time options to the actual path rather than assuming that installation of a library guarantees a particular kernel executes.

For agent serving, benchmark the selected FA4 path under both large prefills and the service's actual decode shapes, then inspect end-to-end task latency. Preserve a versioned cuDNN or alternative comparator, because a later library update can absorb the same optimization. Separate compilation time from execution time when evaluating a DSL-based implementation: faster iteration can be valuable even when steady-state kernel throughput is matched. None of these measurements establish better long-context reasoning without a quality test. The kernel contribution is reduced cost of a specified computation; the agent contribution is what useful decisions that computation enables under the released resource budget.

Figure 4.1 distinguishes evaluation of the same target attention operator from logical-to-physical cache allocation.

4.2 The computation being optimized

Attention kernels optimize a specific tensor operator; they do not determine what an agent should remember. For one head, let $Q \in \mathbb{R}^{n_q \times d_h}$, $K \in \mathbb{R}^{n_k \times d_h}$, and $V \in \mathbb{R}^{n_k \times d_v}$. With additive mask A, attention is $O = PV$, where $P = \mathrm{softmax}(QK^\top/\sqrt{d_h} + A)$ row by row. The mask can encode causality, padding, sliding windows, or a more specialized visibility relation. Changing the mask changes the operator. Changing its implementation while preserving the same mask and arithmetic target is an optimization of that operator. These cases require different correctness tests [46, 56].

For dense equal-length sequences and $d_v = d_h$, forming scores and multiplying probabilities by values requires approximately $4n^2 d_h$ floating-point operations when one multiply-add counts as two operations. Causal structure can reduce useful score work by roughly half, but actual savings depend on whether the kernel skips masked tiles. Projection and feed-forward costs are additional. A dense kernel that uses less temporary memory still evaluates a quadratic set of interactions. FlashAttention is exact attention in this algorithmic sense; it avoids materializing the full score and probability matrices in high-bandwidth memory, rather than replacing them with a low-rank or sparse approximation [56].

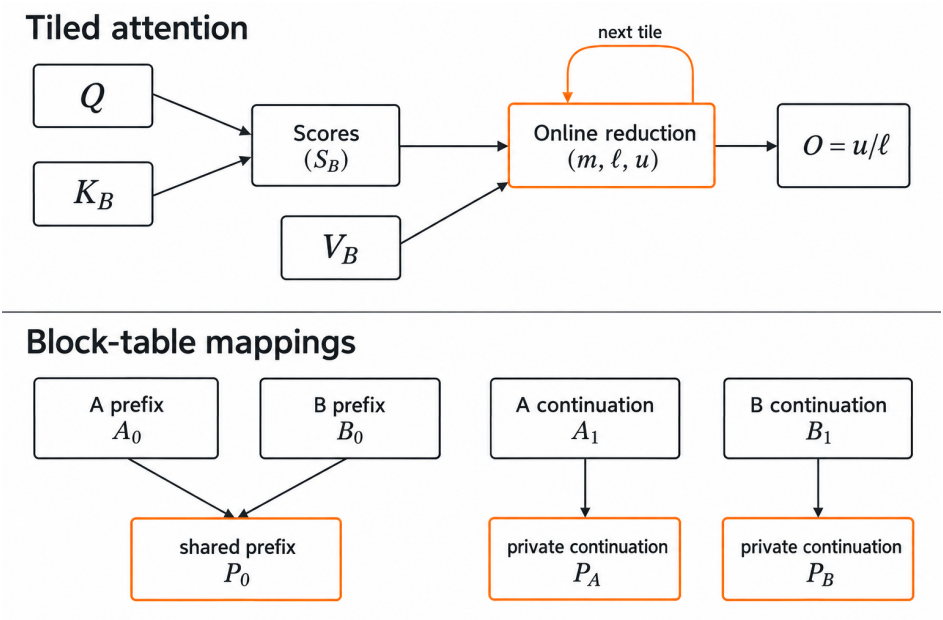

Figure 4.1: Tiled reduction and paged allocation preserve different objects. Top: query, key and value tiles update the running maximum m, normalizer ℓ and weighted-value sum u; normalization forms $O = u/\ell$ for a nonempty visible row. Bottom: block tables can share a complete eligible prefix while mapping divergent continuations to different physical storage. The lower arrows are allocation mappings, not attention edges. No performance measurements are depicted.

Two regimes matter for agents. Prefill processes a supplied context and commonly has many query positions. Autoregressive decode often has one new query position per sequence, interacting with a long retained cache. A technique that improves a large square matrix computation need not produce the same gain for a thin decode operation. During a long tool pause, the cache may occupy accelerator memory without generating tokens. The workload consequently includes active computation, idle retained state, resumption, branching, and cancellation. Kernel benchmarks that omit those phases do not fully characterize a long-running agent service.

Correctness also has several levels. Algebraic equality assumes real arithmetic. Floating-point implementations reorder reductions and can differ numerically while targeting the same function. Randomized dropout requires consistent random-number semantics in training and recomputation. Mask alignment, positional offsets, and sequence lengths must match exactly.

A kernel can pass average numerical tolerances yet fail at a rare boundary length or unsupported head dimension. For an agent, such a defect can appear as a semantic planning error many actions later. Operator-level verification should therefore precede task-level evaluation rather than relying on an aggregate task score to detect low-level mistakes.

4.2.1 Online normalization and tiled exact attention

Consider one query row with logits s_j. For a block B, define its maximum m_B, unnormalized mass $\ell_B = \sum_{j \in B} \exp(s_j - m_B)$, and weighted sum $u_B = \sum_{j \in B} \exp(s_j - m_B) v_j$. The normalized output is u_B / ℓ_B. Two disjoint blocks can be merged without storing their individual probabilities. Let $m = \max(m_B, m_D)$. Then

$$\begin{aligned} \ell &= e^{m_B - m} \ell_B + e^{m_D - m} \ell_D, \\ u &= e^{m_B - m} u_B + e^{m_D - m} u_D. \end{aligned} \tag{4.1}$$

The final output is u/ℓ. These equations follow by factoring the same global maximum out of the exponential sums. They explain how a streaming reduction can reproduce the dense softmax result in exact arithmetic while keeping only running statistics and an output accumulator.

The numerical role of the maximum is essential. Exponentiating arbitrary logits directly can overflow even when the normalized result is well defined. Subtracting the current maximum keeps exponentials at most one. When a later tile contains a larger maximum, previous accumulated sums are rescaled. The procedure must also handle rows whose positions are entirely masked: a naive subtraction of two negative infinities yields an undefined quantity. Implementations need explicit semantics for empty rows and must test them. A finite sentinel used in place of negative infinity can accidentally assign probability to forbidden positions if not treated consistently.

FlashAttention uses this reduction with tiles that fit in on-chip memory, minimizing transfers of large intermediate matrices between device memory and the processor. Its backward pass recomputes selected quantities rather than storing the full probability matrix [56]. Recomputation exchanges arithmetic for reduced memory traffic. That trade can be favorable because arithmetic capacity and memory bandwidth do not grow in the same proportion. It is nevertheless conditional on tile dimensions, head sizes, register pressure, shared-memory capacity, and compiler scheduling. The asymptotic I/O analysis explains a mechanism; it does not replace measurement on the deployed accelerator.

FlashAttention-2 changes work partitioning and reduces non-matrix-multiplication work. FlashAttention-3 uses Hopper-specific asynchronous

execution and low-precision techniques [57, 58]. Their author-reported speedups refer to specified GPUs and configurations. It is incorrect to transfer one of those multipliers to arbitrary inference traffic or to multiply it by an unrelated serving speedup as though the gains were independent. Both techniques may remove the same bottleneck, or one can expose another. End-to-end agent latency additionally includes communication, parsing, tools, queueing, and verification.

For one query, let $\mathcal{J}$ contain its unmasked positions, assume $s_j \in \mathbb{R}$ for every $j \in \mathcal{J}$, and partition $\mathcal{J}$ into ordered nonempty tiles $B_1, \dots, B_R$. Nonfinite unmasked logits violate this input contract rather than being silently discarded. For tile r, write $\mathbf{s}_r = (s_j)_{j \in B_r}$ and let $V_r \in \mathbb{R}^{|B_r| \times d_v}$ have rows $v_j^\top$. With componentwise exponentiation and a matching all-ones vector $\mathbf{1}$, the complete streaming recurrence is

$$\begin{aligned} (m_0, \ell_0, u_0) &= (-\infty, 0, \mathbf{0}), \\ b_r &= \max_{j \in B_r} s_j, \\ m_r &= \max\{m_{r-1}, b_r\}, \\ \alpha_r &= \exp(m_{r-1} - m_r), \\ \mathbf{w}_r &= \exp(\mathbf{s}_r - m_r \mathbf{1}), \\ \ell_r &= \alpha_r \ell_{r-1} + \mathbf{1}^\top \mathbf{w}_r, \\ u_r &= \alpha_r u_{r-1} + V_r^\top \mathbf{w}_r, \qquad r = 1, \dots, R. \end{aligned} \tag{4.2}$$

The nonempty-tile restriction makes every m_r finite, so the first rescaling uses $e^{-\infty} = 0$ without subtracting two negative infinities. Writing $\mathcal{J}_r = \bigcup_{i=1}^{r} B_i$, induction gives the invariant

$$\begin{aligned} m_r &= \max_{j \in \mathcal{J}_r} s_j, \\ \ell_r &= \sum_{j \in \mathcal{J}_r} e^{s_j - m_r}, \\ u_r &= \sum_{j \in \mathcal{J}_r} e^{s_j - m_r} v_j. \end{aligned} \tag{4.3}$$

Thus the normalized result has the partial specification

$$o = \begin{cases} u_R / \ell_R, & \mathcal{J} \neq \varnothing, \\ \bot, & \mathcal{J} = \varnothing. \end{cases} \tag{4.4}$$

Here $\bot$ marks an empty-row outcome requiring a declared implementation contract; it is not a numerical tensor or permission to divide by zero. In exact arithmetic, the nonempty-row result equals the dense masked-attention output. Vectorization, dropout, parallel reduction order, and floating-point error remain separate implementation obligations.

4.3 KV growth and the difference between storage and knowledge

In a decoder, previously computed keys and values can be retained so that each new token need not recompute every previous projection. For homogeneous layers with equal key and value width d_h, unsharded storage is

$$M_{\mathrm{KV}} = 2Lh_{\mathrm{KV}}d_h b \sum_{j=1}^{N} T_j \quad \text{bytes}, \tag{4.5}$$

where L is layer count, h_{KV} the number of KV heads, b bytes per element, and T_j retained tokens for sequence j. The factor two counts keys and values. This formula excludes allocation slack, indexing, quantization metadata, communication buffers, and replication. Tensor-parallel sharding changes local storage only when the implementation actually partitions the relevant heads or dimensions.

The following numbers are calculated examples for hypothetical layouts, not measurements of named models. With $L = 80$, $h_{\mathrm{KV}} = 8$, $d_h = 128$, $b = 2$, and $T = 131{,}072$, one sequence requires exactly 40 GiB before overhead. Eight such sequences require 320 GiB. If there are 64 KV heads instead, one sequence requires 320 GiB. Sharing KV heads therefore has a large and directly calculable storage effect. Whether the corresponding architecture retains equal task quality is a separate empirical question. Grouped-query attention explicitly studies this tradeoff rather than making storage reduction synonymous with intelligence [51].

Table 4.1: Calculated KV footprints for hypothetical two-byte caches.

Layers and KV heads	Retained tokens	Unsharded footprint
32 layers, 8 heads	32,768, one sequence	4 GiB
80 layers, 8 heads	131,072, one sequence	40 GiB
80 layers, 64 heads	131,072, one sequence	320 GiB
80 layers, 8 heads	4,096, one sequence	1.25 GiB

The cache is computational state tied to a particular model computation. It is not an independently meaningful semantic archive. A durable conversation log can survive a model upgrade while its old keys and values become unusable. A context summary can preserve the intended task while

changing every downstream hidden representation. Conversely, retaining an enormous cache does not ensure that the model will attend to a decisive constraint. The system should therefore distinguish three questions: whether source events are stored, whether their computation can be reused exactly, and whether their information affects the correct action.

Long sessions introduce an allocation problem even when the per-token formula is simple. Each tool result can increase retained length; branches can share a common prefix before diverging; abandoned runs can leave caches resident until cleanup. A serving system must define ownership and reclamation for each block. Cancellation is not complete until resources are released or transferred to an explicit resumable state. A session identifier alone does not prove that the cache is durable, and a durable cache does not prove that the external tool state can be replayed. These are independent recovery contracts.

4.4 Paged allocation and shared prefixes

PagedAttention separates logical token positions from physically contiguous storage. Fixed-size blocks hold keys and values, and a block table maps logical positions to physical blocks. This avoids reserving one large contiguous buffer for the maximum possible length of every request. It also permits sharing of completed prefix blocks across related sequences, with copy-on-write behavior when a shared mutable region would otherwise diverge. The original vLLM paper attributes throughput gains to improved memory use and associated batching opportunities under its evaluated workloads. Those are author-reported results for that implementation and its baselines [59].

Let a block hold B tokens. A sequence of length T occupies $B\lceil T/B \rceil$ token slots. Its internal slack is less than B slots; across N sequences the total slack is less than NB, before considering sharing or metadata. This bound explains one advantage over reserving each request's maximum length. It does not imply zero overhead. Smaller blocks reduce terminal slack but increase block-table size and indirection frequency. Larger blocks can improve locality while wasting more space near sequence ends. The best block size depends on the length distribution, attention kernel, and sharing patterns.

Prefix sharing is valid only for an identical computation prefix. Identical visible text is insufficient if tokenization, positions, masks, model weights, adapters, or relevant execution settings differ. Even an identical token sequence can yield different keys and values when an earlier hidden input

differs. A robust cache key therefore binds the complete computation identity required by the model implementation, together with tenant isolation rules. A semantic embedding match can suggest useful retrieval; it cannot justify substituting exact KV tensors. Confusing these mechanisms can create silent correctness failures rather than merely a lower cache hit rate.

Editing a prefix invalidates more than the edited tokens in ordinary causal attention. Representations at later positions depend on the earlier prefix through successive layers. Reusing their old keys and values after a deletion or replacement generally evaluates a different computational graph from a fresh forward pass. Safe reuse requires a proved invariant, a model architecture that supports the transformation, or recomputation of affected state. Context-editing systems therefore need explicit interaction with the cache manager. The compression service cannot be treated as a purely textual component whose output is transparently compatible with every retained tensor.

An agent often branches on uncertain plans or candidate patches. Shared prefixes can reduce physical memory duplication, but every divergent continuation still consumes computation and new blocks. A search controller that increases the branch count can exhaust capacity even while the cache manager is efficient. Admission control should reserve an envelope for likely continuation or support preemption with declared semantics. Recompute-on-resume and host offload are different policies with different latency and bandwidth costs. Neither can be ranked without the session length distribution, tool-pause distribution, and deadline requirements.

4.5 Bandwidth limits and latency distributions

A simple lower-bound model distinguishes arithmetic and movement. If an operation requires F floating-point operations and transfers D bytes through a bottleneck interface with usable rates P and W, then its duration is at least $\max(F/P, D/W)$ under ideal overlap. This is a lower bound, not a performance prediction. Kernel launches, synchronization, indexing, cache misses, and queueing add costs. The relevant bandwidth is the measured rate for the actual access pattern, not an advertised peak. Small batches and scattered page accesses may fail to approach that peak.

For long-context decode, repeatedly reading keys and values can dominate the attention component. Grouping more sequences can improve weight reuse and hardware utilization, but also increases active KV traffic and waiting time. An agent service should report time to first token, inter-token latency, complete action latency, and end-to-end task latency.

A tool call that takes seconds can hide a model optimization in the mean while the optimization still matters for the tail of interactive steps. Conversely, aggressive batching can increase the time before a safety-critical confirmation reaches the execution boundary.

Queueing creates another source of misleading comparisons. At low offered load, both implementations may have small queues. Near capacity, a modest service-rate difference can produce a large latency difference because requests accumulate. A benchmark that measures only completed-request throughput can conceal dropped or timed-out work. Report offered load, admitted load, completed load, error rate, and latency percentiles together. Open-loop arrivals are useful for exposing overload because the load generator does not automatically slow down when responses become late. Closed-loop tests answer a different question about a fixed client population.

Cache offload shifts the bottleneck. A host-memory cache can permit more paused sessions while introducing transfers on resumption. If D_{resume} bytes must cross a link with sustained bandwidth W_{link}, transfer time alone is at least $D_{\mathrm{resume}}/W_{\mathrm{link}}$ before overlap and scheduling effects. Compression may reduce transferred bytes while adding quantization or decompression work. A system that advertises more simultaneous sessions should disclose whether those sessions are immediately executable, offloaded, or represented only by a durable log requiring prefill. These states have different practical meanings for interactive agents.

4.6 Approximate caches and a bounded perturbation argument

Quantization and eviction change the attention input, whereas tiled exact kernels primarily change evaluation order. Consider a single query q, original keys k_j, and approximations $\widetilde{k}_j$ with $\|\widetilde{k}_j - k_j\|_2 \le \epsilon_K$. Cauchy–Schwarz gives a logit perturbation at most $\epsilon = \|q\|_2 \epsilon_K / \sqrt{d_h}$. If every logit changes by at most ϵ, then the ratio between corresponding softmax probabilities lies between $e^{-2\epsilon}$ and $e^{2\epsilon}$. The numerator changes by at most $e^{\pm\epsilon}$ and the normalization by the same factor, which proves the ratio bound directly.

Let $p, \widetilde{p}$ be the two probability vectors. The ratio bound yields the conservative estimate

$$\mathrm{TV}(p, \widetilde{p}) \le \min\{1, (e^{2\epsilon} - 1)/2\}. \tag{4.6}$$

If original values have norm at most B_V and every value perturbation has

norm at most ϵ_V, the output difference satisfies

$$\|\widetilde{O} - O\|_2 \leq 2B_V \,\mathrm{TV}(p, \widetilde{p}) + \epsilon_V. \tag{4.7}$$

The proof adds and subtracts $\sum_j \widetilde{p}_j v_j$, then applies the triangle inequality. These are elementary deductions under explicit per-vector error bounds. They do not assert a useful end-to-end task guarantee: query norms, downstream amplification, and proximity to action decision boundaries can make a small tensor error consequential.

Eviction requires a different argument because removed positions need not have small key perturbations. If an evicted set has total original attention mass $\alpha < 1$, renormalizing the nonempty retained set changes the output by at most $2\alpha B_V$ when all values have norm at most B_V. This follows by writing the original output as the mixture of retained and removed conditional outputs. When $\alpha = 1$, retained-set renormalization is undefined and an explicit fallback is required. The bound is informative only when α is known or controlled for the actual query. Historical average attention mass is not a guarantee for a future query about an old rare fact. An agent that suddenly revisits an early constraint can violate the heuristic's premise.

A local tensor bound reaches a discrete decision only through an additional stability condition. Let output logits ℓ change by at most η in maximum norm, and let v^* be their unique maximizer. Greedy selection is unchanged if

$$\ell_{v^*} - \max_{v \neq v^*} \ell_v > 2\eta. \tag{4.8}$$

The inequality follows by subtracting the worst perturbation from the leading logit and adding it to its strongest competitor. Extending it through an autoregressive action requires the margin condition at every shared-prefix step, not only at the first token. Stochastic sampling needs a distributional or coupling analysis instead. Consequently, an attention-output tolerance alone does not certify preservation of a tool name, an identifier, or a permission-sensitive argument. Numerical tests should retain action-margin failures alongside mean tensor error.

These distinctions define appropriate ablations. Compare exact attention, reduced precision without eviction, eviction without quantization, and their combination. Measure tensor discrepancies, next-token distribution changes, action validity, and complete task outcomes. Include queries that make an old low-frequency constraint suddenly relevant. A cache method can maintain language-model perplexity while damaging stateful decision reliability. The converse can also occur when a task never uses the affected information. The experiment should identify that dependence

rather than presenting one aggregate quality number as a universal cache-safety certificate.

4.6.1 Compression phase is a distinct positional variable

The September 2026 study of periodic weak spots reports retrieval sensitivity to a fact's position within chunked compression windows, including effects that persist without RoPE and with uniform compression weights [60]. This is a different mechanism from a generic average cache-error bound. Let windows begin at indices $b + kS$, with stride $S \geq 1$, width W, and origin b. The source position i has phase

$$\phi_b(i) = (i - b) \bmod S. \tag{4.9}$$

For compressed states $c_k = F(h_{b+kS}, \ldots, h_{b+kS+W-1})$, shifting content while keeping b fixed can change which vectors share a compression entry and their within-window roles. It need not preserve the information available to a future query, even when relative distances among the task's relevant tokens stay unchanged.

This should not be confused with exact rotary-position algebra. For fixed unrotated query and key vectors and an orthogonal rotation family satisfying $R_i^\top R_j = R_{j-i}$, a common position shift gives

$$(R_{i+\Delta} q)^\top (R_{j+\Delta} k) = q^\top R_{j-i} k. \tag{4.10}$$

The identity assumes unchanged vectors and visibility. Inserting an actual prefix can change hidden states and is not simply renumbering positions. Chunk compression additionally changes grouping when its boundary origin stays fixed. Therefore a phase-dependent retrieval failure cannot automatically be diagnosed as a faulty RoPE implementation, nor does exact RoPE algebra certify invariance of a complete compressed model.

Approximate arithmetic introduces another, separable interaction. A quantizer Q need not commute with rotation: $Q(R_\Delta k) \neq R_\Delta Q(k)$. As a calculated two-dimensional counterexample, take nearest-integer coordinate quantization, $k = (1, 0)$, and a rotation by 45 degrees. Quantizing after rotation gives $(1, 1)$, while rotating the quantized vector gives $(1/\sqrt{2}, 1/\sqrt{2})$. This is a mathematical counterexample, not a measured error of a particular cache. It shows why precision, scale grouping, and the order of rotation and quantization belong in a reproducible approximation specification.

A discriminating experiment crosses source phase, window origin, stride, width, and query location while holding the relevant fact and answer fixed. Use both pure positional renumbering on fixed tensors and actual

prefix edits, because they test different invariants. Include exact attention, an uncompressed reduced-precision control, and each compressed variant. Report the complete phase profile, its worst phase, and the difference between best and worst phases, not only the mean. Repeating the test without rotary embeddings can distinguish a compression-grouping effect from a rotary-specific one. A test with uniform compression weights can further separate learned gate specialization from information mixing at boundaries.

For agents, a changed tool description, a harmless comment, or a memory summary can shift a decisive old constraint to a different phase. The compression policy should therefore be evaluated under realistic serialization changes and rare action-changing facts. A fixed benchmark placement can accidentally favor a strong phase. Random placement helps estimate average behavior, but worst-phase evaluation remains necessary when a constraint must survive independently of incidental token offsets. The source paper's bounded retrieval evidence motivates this mechanism test; it does not establish a universal deployment failure rate.

4.7 Implementation verification and workload design

The FlashAttention and vLLM repositories provide identifiable implementations, with immutable snapshots recorded separately from their associated papers [55, 61]. A current repository snapshot is not assumed to be the code used for an older paper's measurements. The implementation audit should trace public interfaces to kernel dispatch, supported dtypes, head dimensions, mask conventions, block tables, and cache update logic. It should record fallback paths: an unsupported shape can silently invoke a slower implementation, changing the interpretation of a benchmark that purports to compare kernels.

Operator tests should compare forward outputs and gradients against a high-precision reference over a matrix of lengths, masks, offsets, and head dimensions. Include empty sequences, partially filled pages, shared-prefix divergence, and cancellation during allocation. For inference, compare fresh prefill with resumed-cache computation under identical inputs. For context edits, deliberately modify an early token and confirm that any claimed reuse matches a recomputed reference within declared tolerances. Numerical tolerance should reflect dtype and operation length, but a permissive tolerance must not conceal an incorrect visibility mask.

A concrete implementation chain connects Section 3.1 of the first FlashAt-

tention paper to the public `flash_attn/flash_attn_interface.py` interface and its `_flash_attn_forward` dispatch. The Hopper snapshot additionally exposes `flash_attn_func` through `hopper/flash_attn_interface.py`. These names were checked in the pinned repository; they identify interfaces, not a claim that the Python wrapper itself implements the tiled CUDA arithmetic. For a hypothetical single head with $n = 16{,}384$, $d_h = 128$, and two-byte inputs, each of Q, K, V occupies 4 MiB, whereas one fully materialized FP32 score matrix occupies 1 GiB. Two FP32 running statistics per row occupy only 128 KiB. This arithmetic illustrates the memory mechanism without assigning an invented device runtime.

A numerical implementation constraint is visible in the pinned Blackwell path itself. In `FlashAttentionForwardSm100.softmax_loop`, the probability scale offset a and permitted running-maximum lag r are checked against the largest finite probability-storage value $f_{\max}$:

$$a + r < \log_2 f_{\max}. \tag{4.11}$$

This is a finite-range condition for the scaled probabilities before the value product, not an attention-quality bound. The corresponding `correction_loop` restores the scale when forming the output [55]. A wide accumulator cannot repair a probability that already saturated during a narrower cast: the normalizer and the value product would then represent different masses. A useful stress test varies logit maxima, dtype and head dimension while checking the actual dispatched path against a reference; changing only an average tensor tolerance can conceal this failure. The inspected code establishes the guard and dataflow, not a new runtime or accuracy measurement.

At the allocation layer, Section 4.2 of the PagedAttention paper can be compared with the pinned vLLM module `vllm/v1/core/kv_cache_manager.py`, whose `KVCacheManager` exposes `get_computed_blocks`, `allocate_slots`, and `free`. This later V1 implementation is not the original paper's code version. A falsifiable integration check allocates two requests with the same eligible prefix, verifies their shared complete physical blocks, extends one request, and confirms that the other request's logical sequence remains unchanged. It then frees both requests and checks that the allocator's accounting returns to the initial state. This tests ownership and reuse independently of language-model quality.

The complete serving experiment should combine short and long prefills, varying output lengths, repeated prefixes, branching, tool pauses, and aborted sessions. Use a fixed workload trace for paired comparisons and a separate stress workload for overload. Warm and cold cache conditions need separate runs. Report accelerator count, available memory, precision,

Table 4.2: Attention optimizations require different correctness claims.

Mechanism	Preserved target	Distinct failure test
Tiled exact kernel	Same attention operator	Masks, numerical error, gradients.
Paged allocation	Same logical cache	Mapping, ownership, copy-on-write.
KV quantization	Approximate tensors	Distribution drift and rare constraints.
Cache eviction	Restricted evidence	Queries requiring removed positions.
Prefix reuse	Identical computation prefix	Version, position, and edit invalidation.

kernel versions, host-link characteristics, queue policy, and tenant isolation settings. Include allocation failures and recovery time in the outcome set. Throughput improvements are meaningful only when quality, admission, and latency conditions remain comparable.

The practical conclusion is a separation of obligations. Kernels must compute the intended operator; allocators must preserve logical state and ownership; approximation methods must justify their error; schedulers must satisfy service objectives; and the agent must use retained evidence correctly. Progress at one layer can enable progress at another, but cannot establish it by implication. This separation permits precise attribution while retaining the central interaction: a context policy that changes historical visibility also changes cache validity, resource demand, and the distribution on which the action policy operates.

4.8 When recomputation is mathematically necessary

The invalidation rule can be stated precisely for a deterministic causal decoder. Let $x_{1:n}$ be its input and let $z_i^{(\ell)}$ be the representation at position i after layer ℓ. Suppose every layer may attend to every earlier position and its key and value projections are functions of the preceding-layer representation. If position r changes, the original representations at positions $i \geq r$ are not generally reusable. At the input layer, $z_r^{(0)}$ changes. At the first attention layer, every later query may combine a changed value or a changed normalization term from position r. At subsequent layers these

changed representations produce changed keys and values. Induction across layers establishes a dependency path to every later cached position. This argument establishes possible dependence, not that every numerical value must change for every input.

Exact reuse therefore needs stronger evidence than a small edit distance. A special mask can remove the dependency path; an architecture can partition independent segments; a specific edit may leave a relevant representation unchanged. Each case requires a demonstrated invariant. Ordinary semantic similarity between the original and edited text supplies none of these conditions. The same reasoning applies to changing an adapter or a positional convention. Even if the visible token sequence is identical, the projection functions or their inputs differ. A cache manager should prefer a conservative invalidation rule to silently assuming equality that the model computation does not support.

There is a related distinction between prefix deletion and left truncation during generation. Removing old positions from the attention set while retaining representations computed when those positions were visible does not reproduce a fresh run on the shorter input. It evaluates a hybrid state: later representations already incorporate information from the deleted history, but future queries cannot access the deleted positions directly. Such a policy can be intentional, for example as an approximation that preserves some historical influence. It must be described and evaluated as that policy, rather than as exact execution of the shortened textual context. Training on a freshly shortened sequence would condition on a different computation again.

These observations motivate a three-way equivalence test. First run an uninterrupted reference trajectory. Second resume from a stored cache without editing; this tests serialization and restoration of computational state. Third apply the intended edit and compare with a fresh run under the edited inputs and mask. The second comparison should agree within the documented numerical contract. The third agrees only if the transformation claims exact equivalence; an approximate transformation should instead report its discrepancy and downstream effects. Holding the sampled continuation fixed through teacher forcing helps localize tensor drift before free-running outputs amplify it.

For an agent service, these tests should include a consequential old instruction that is later revoked, not merely irrelevant prose. If removed information survives indirectly in retained states, a textual audit may incorrectly conclude that it has been forgotten. If recomputation eliminates that influence, the cost can be larger than the nominal number of edited tokens suggests. This is a concrete interaction between computational

memory, policy conditioning, and deletion semantics. It also explains why the durable event log, the visible context, and the retained cache must be inspected as separate state objects when diagnosing long-session behavior.

Chapter 5

Predictive Representation Learning and Multimodal State Abstraction

5.1 Dense predictive features and world action coupling in 2026

V-JEPA 2.1 changes the training target rather than merely scaling the original video encoder. Its Section 2.3 applies prediction supervision to masked and visible tokens, introduces intermediate-layer supervision, and uses modality-specific image/video patch embeddings. The inspected 2026 implementation exposes `forward_target`, `forward_context`, and `loss_fn` in `app/vjepa_2_1/train.py` [62, 63]. This is a current, identifiable objective-to-code case; the original I-JEPA discussion supplies its foundation rather than a claim about the frontier's latest recipe.

The local-versus-global tradeoff has a useful optimization interpretation. Let L_m measure prediction of masked targets and L_c supervision of visible context features. Training minimizes $L_m + \lambda L_c$ in a simplified two-term view. A direction that reduces L_c can increase L_m when the gradient inner product is negative. Thus adding dense supervision is not guaranteed to improve every global metric. The scale and spatial weighting of the context term determine which distinctions receive pressure. This deduction explains why an implementation must retain the actual loss weights and scheduling, rather than treating the presence of an additional loss as a uniformly beneficial feature.

For agents, dense features are attractive because many operations depend on a localized affordance rather than a whole-scene label. The corresponding falsifiable hypothesis is that improved local representations reduce target-selection errors under a fixed policy and observation budget. Test that hypothesis with counterfactual layouts, small text, occlusion, and temporal changes. Retain a global-recognition diagnostic to detect whether the local gain trades away broader scene understanding. A classifier or robotics score from the source paper does not answer the computer-use question automatically; it identifies a mechanism worth testing.

Implementation inspection should follow the feature path across encoder layers, normalization, fusion, and predictor outputs. Multi-level features increase the amount of intermediate state that may be retained or recomputed. A frozen feature-extraction benchmark should include preprocessing and feature-transfer costs, not only encoder FLOPs. An agent that invokes the encoder on every observation may prefer a smaller distilled model, a crop policy, or event-triggered refresh even when a larger model has the best unconstrained representation score. Quality should therefore be plotted against complete decision latency and storage, with the action interface held constant.

The October 2026 revision of V-JEPA Policy couples a frozen V-JEPA 2.1 encoder with an instruction-conditioned future-latent predictor and a flow-matching action expert. Its Section 3 connects future-informed context keys and values to action generation, while the experiments compare visual foundations under a common downstream framework [64]. This is a more direct representation-to-action case than passive recognition alone. It still changes several trained components, so a result for the complete policy cannot be assigned entirely to the frozen encoder without the corresponding controlled comparison.

The actual coupling point matters for an ablation. The action expert reads the predictor's layerwise context keys and values, not the final future-latent readouts. In the paper's notation, its visual interface is

$$\mathcal{C}_\phi = \{(\mathcal{R}_{3\mathrm{D}}(K_c^{(j)}), V_c^{(j)})\}_{j=1}^{L}, \tag{5.1}$$

where $K_c^{(j)}, V_c^{(j)}$ are projections at context positions and only the keys receive the stated rotary transform [64]. Changing an unused final readout would therefore not test whether the action path uses future-conditioned context. A more diagnostic proposed study holds the visual encoder and action expert fixed and intervenes on future-query-to-context interactions, preserving observed tokens, positions, tensor shapes and comparable compute. A performance drop after that intervention can support functional use of the coupling, while leaving its calibration and interventional validity open. Shuffling the whole context interface would conflate loss of present evidence with loss of future conditioning. Out-of-distribution corruption remains an alternative explanation and requires matched controls; no such experiment is reported here.

Figure 5.1 separates latent prediction from the downstream control path whose validity must be tested.

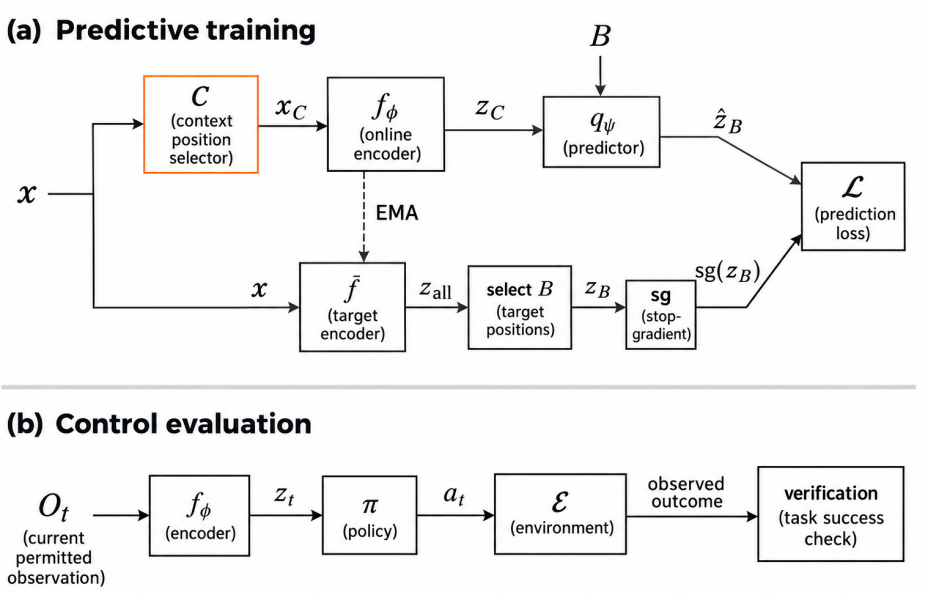

Figure 5.1: Predictive representation training and control test different properties. The context branch uses positions C; the target branch processes the complete observation, selects positions B, and blocks loss gradients through sg. An EMA parameter update is separate from gradient flow. The declared loss must match the selected JEPA variant. A downstream controller requires independent environment-outcome evaluation; small prediction loss does not establish action sufficiency.

5.2 Representation quality is relative to a decision

A representation is useful to an agent when it preserves distinctions that matter for future decisions under the available interaction policy. This criterion differs from visual similarity, image classification accuracy, or reconstructing every pixel. Let O_t be the current multimodal observation and $z_t = f_\phi(O_t, H_t)$ an encoded state. An image encoder may discard lighting variation while preserving object identity; a computer-use agent may additionally need exact text, focus state, disabled controls, and the difference between a preview and a committed operation. These requirements depend on the action space and objective, so no representation can be declared generally sufficient from a single transfer benchmark.

For a one-step decision with loss $\ell(a, Y)$, define the Bayes risk available from information X as $R^*(X) = \inf_\pi \mathbb{E}[\ell(\pi(X), Y)]$. If $z = f(X)$ is deter-

ministic, then $R^*(X) \leq R^*(z)$: every policy on z can be implemented on X by composition, while the reverse need not hold. Equality holds when some optimal decision is measurable with respect to z. This elementary proposition supplies the right direction of comparison. Compression cannot improve the ideal information-theoretic optimum, although it can improve finite-data learning, optimization, and computational feasibility. Those practical benefits should not be confused with creating information absent from the observation.

For sequential control, a sufficient state must support relevant prediction and reward decisions across future actions. A representation trained on passive video can preserve correlations that disappear when the agent intervenes. A representation trained on one action distribution can omit details needed by another. The complete state S_t includes external records, tool state, and permissions; a visual embedding is only one component. Session continuity therefore requires synchronizing encoded observations with event timestamps and action outcomes. An excellent screenshot representation paired with the wrong browser tab or stale environment state is still an incorrect decision input.

Evaluation should separate representation learning from policy learning. A frozen-encoder probe asks what a restricted decoder can recover. Fine-tuning asks whether the representation is a useful initialization under additional supervision. Closed-loop evaluation asks whether the integrated controller succeeds while its actions alter the observation distribution. These are different experimental interventions. A linear probe can fail even when information is present in a nonlinear code, and a powerful policy can compensate for a weak visual encoder through repeated tool queries. Reporting the intervention prevents either result from being overstated as a definitive property of the latent state.

5.2.1 A decision-preservation test for learned abstraction

An abstraction can be evaluated more directly than through a generic probe score. Let X be the complete permitted observation and let Y indicate a binary action requirement, with Y balanced. Construct two environments whose broad appearance is the same but whose required action differs because of a localized signal. If the encoder maps both observations to exactly the same representation and no additional evidence is available, the best downstream classifier has accuracy at most one half. This follows from the state-aliasing argument in Chapter 3. With approximate rather than exact collisions, the relevant object is the separability of the induced representation distributions under the actual decoder class.

For unrestricted binary testing with equal priors, the optimal success probability is $(1+\mathrm{TV}(P_0, P_1))/2$, where P_y is the representation distribution conditional on $Y = y$. The identity follows by choosing the decision region that maximizes $P_1(A) - P_0(A)$. This is a standard probability deduction, used here to specify a diagnostic rather than to claim a new representation theorem. It explains why a low average reconstruction error or a high semantic retrieval score cannot alone certify preservation of a rare action-critical signal: those metrics need not control the total variation of these conditional distributions.

In practice, total variation in high-dimensional learned representations is not directly known. A fixed classifier with population accuracy A_d gives $\mathrm{TV}(P_0, P_1) \geq \max\{0, 2A_d - 1\}$, not an upper bound on the information present. A finite held-out accuracy estimate needs a one-sided lower confidence bound A_L, valid for the sampling design, before reporting $\max\{0, 2A_L - 1\}$ as a statistical lower bound. Tuning on those test outcomes invalidates that independent-test interpretation. A failed probe can therefore motivate stronger probes or a direct action test, while a successful probe still leaves policy access and calibration unresolved. The experiment should report encoder stochasticity, preprocessing, sample size, decoder capacity, and the precise intervention defining Y. Those details determine what conclusion can be drawn from the observed accuracy.

The resulting design principle is to preserve a recoverable path from task-critical observations to execution, while allowing abstraction of genuine nuisances. Multiple channels can provide complementary evidence, but agreement between two models trained on similar data is not independence. A robust agent records which channel supplied a fact, checks freshness before action, and seeks another observation when the distinction remains unresolved. Representation learning contributes powerful statistical structure; the surrounding system must still connect that structure to the exact state and action semantics of the task.

5.2.2 Directed progress and finite-budget reachability

RC-aux changes two interfaces that one-step JEPA prediction leaves unspecified: recursive prediction over the planning horizon and a directed reachability estimate conditioned on remaining steps. Its Section 3 constructs positive and negative labels from observed trajectory offsets and also supplies predicted-latent inputs to the reachability head. Temporal-Distance JEPA's current second version instead calibrates a nonnegative asymmetric cost from demonstration step counts, with a metric-residual head and horizon-matched rollout supervision [65, 66]. A distance between

two embeddings is therefore no longer assumed to be the planning objective merely because the predictor was trained in that space.

The supervision has a precise epistemic limit. In deterministic state dynamics, let $D^*(s, g)$ be the minimum number of feasible actions from s to g, with value $+\infty$ when none exists. An observed valid segment from s_i to s_j of length $j - i$ proves $D^*(s_i, s_j) \leq j - i$. It does not prove equality. Consequently, labeling the pair unreachable at budget $h < j - i$ introduces possible false negatives whenever an unobserved shortcut exists. Cross-trajectory negatives are weaker still: two independently collected trajectories may pass through mutually reachable states. These are deductions from the labeling construction, rather than objections that invalidate every useful learned ranking.

Directionality is independently necessary when actions are irreversible. If a transition deletes an object and no action restores it, $D^*(s, g)$ can be finite while $D^*(g, s)$ is infinite. Every symmetric embedding distance assigns the same value in both directions. An asymmetric head removes that representational restriction, but does not establish calibrated reachability on unseen states. To distinguish the interventions, hold the encoder and rollout operator fixed while comparing symmetric and directed heads, then hold the scoring head fixed while changing recursive prediction training. Include observed loops and deliberately collected shortcut trajectories to measure label ambiguity.

For a stateful agent, the budget must count executable operations rather than an unspecified number of latent updates. A plan with three service calls and a required approval can be unreachable within three permitted actions even if its visual endpoint is nearby. Test changes in permission and resource availability while preserving the displayed scene. A reachability head that sees only pixels cannot resolve a hidden authorization change without another state channel. The strongest diagnostic therefore reports action feasibility, goal completion, and predicted reachability separately, including false acceptance of infeasible plans. Neither source's trajectory-based supervision supplies a general certificate for that broader execution contract.

5.2.3 Latent projection and the difference between predictability and control

Latent Projection for Latent Planning fits an action-free linear dynamics operator to frozen representations and changes the goal cost through a selected subspace. Its Sections 3–4 assume joint Gaussianity, stationary standard-normal marginals, Markov dynamics, and a normal transition

matrix [67]. Writing $z' = Az + \eta$ with independent zero-mean innovation, stationarity gives

$$\mathrm{Cov}(\eta) = I - AA^\top \succeq 0. \tag{5.2}$$

This follows by taking the covariance of the transition, not from an empirical assurance that a regularizer has made real embeddings exactly Gaussian. Normality further yields $A^h(A^h)^\top = (AA^\top)^h$, so one orthonormal basis diagonalizes the innovation covariance at every positive integer horizon. Without normality, the preferred subspace can change with horizon.

The pinned implementation makes several consequential choices visible. `NormalDrift` in `module.py` parameterizes contraction and rotation blocks; its evaluation mode zeros configured static line blocks. `LP2GoalMSE` in `cost.py` selects left singular vectors of the evaluation operator and compares terminal features after projection [68]. Thus the exact choice of dynamic planes, static lines, retained rank, and training versus evaluation mode belongs in the experimental configuration. A description that mentions only top singular vectors omits an implemented structural filter.

A counterexample limits any universal identification of predictable features with controllable goal progress. Let $z = (x, y) \in \mathbb{R}^4$, where $x, y \in \mathbb{R}^2$, and define hypothetical dynamics

$$x' = rRx + \sqrt{1 - r^2}\,\xi, \tag{5.3}$$

$$y' = a, \tag{5.4}$$

with $0 < r < 1$, orthogonal two-dimensional rotation R, and independent $\xi \sim \mathcal{N}(0, I_2)$. Under behavior actions $a \sim \mathcal{N}(0, I_2)$ independent of the current state, stationary marginals are standard Gaussian and the action-free fitted operator is $A = \mathrm{diag}(rR, 0_2)$. It is normal and its most predictable two-dimensional subspace is x. Yet actions directly set y, so a goal depending on y is reachable in one step while a projected cost retaining only x cannot distinguish candidate actions by that goal.

This constructed example is not a reported failure of the paper's evaluated models. It proves that the stated statistical assumptions alone do not guarantee decision sufficiency, even when the retained predictable features are dynamic planes rather than static lines. A falsifiable test introduces an autonomous visual distractor and a less behavior-predictable but controllable target, then compares full latent cost, fitted projection, random projection, and an action-conditioned relevance control at matched search budget. Vary the behavior policy used to fit the projection. A change in the selected subspace can reveal dependence on exploration statistics rather than an invariant property of environment controllability.

5.3 Patch encoders and the cost of visual detail

A Vision Transformer converts an image into patch embeddings and processes their sequence with attention [69]. For image dimensions $H \times W$ and nonoverlapping square patches of side p, the patch count is approximately $N = HW/p^2$ when dimensions divide evenly. Reducing p increases spatial sampling density and sequence length. In dense attention, the corresponding pairwise term grows as N^2, while projection and feed-forward terms grow linearly in N. The choice of patch size is consequently both a perception decision and a systems decision for agents that must read detailed interfaces.

As a calculated example, a 1024 by 1024 image yields 4096 patches at side 16 and 1024 patches at side 32. The patch count changes by four and the dense pairwise attention term by sixteen, before architectural details and implementation. These numbers do not predict a sixteen-fold latency difference: other operations, preprocessing, and memory effects remain. They do explain why uniformly increasing screenshot resolution can be expensive. An agent can instead request a local crop, use an accessibility tree, or extract text through a separate channel. Each strategy changes the evidence interface and can introduce its own omissions.

Small text illustrates a failure that semantic benchmarks can hide. A button label, decimal point, or negative sign may occupy a small fraction of the image but determine the correct action. Resizing can make that feature unrecoverable before the encoder sees it. No amount of later language reasoning can reconstruct the actual character reliably from an information-destroying preprocessing step. A model may guess correctly from context, which makes aggregate success deceptively reassuring. Tests should include counterfactual pairs whose large-scale layout is identical and whose correct action depends on the small feature.

Coordinates require an explicit transform. If the model receives a resized crop, a predicted point must be mapped back through scale, crop offset, display scaling, and possibly scrolling. A correct semantic target can become an incorrect click through a geometric conversion error. Record the image dimensions and transform alongside the observation, rather than reconstructing them from an assumed default. For browser agents, an element identifier from a structured interface can avoid some coordinate ambiguity, but the identifier must remain valid at execution time. A page mutation between observation and action can invalidate both geometric and structural references.

Table 5.1: Observation channels expose different information and failure modes.

Channel	Useful evidence	Distinct limitation
Screenshot patches	Layout, appearance, visual state	Resolution loss and coordinate transforms.
OCR transcript	Exact visible text when recognized	Reading order and recognition errors.
Accessibility tree	Roles, names, structured targets	Missing or inaccurate accessibility metadata.
DOM or API state	Programmatic properties	Hidden state may differ from visible affordances.
Video sequence	Temporal changes and motion	Frame sampling and stale observations.

5.4 Contrastive alignment and what it identifies

CLIP learns image and text representations through a contrastive objective on paired data [70]. Let normalized embeddings be u_i and v_j and temperature $\tau > 0$. A directional batch loss is

$$-\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(u_i^\top v_i/\tau)}{\sum_{j=1}^{B}\exp(u_i^\top v_j/\tau)}. \tag{5.5}$$

The symmetric objective also predicts images from texts. This trains compatibility between paired modalities relative to the negative set. It does not by itself supervise a state transition, a tool argument, or the consequences of an action. A representation that matches a screenshot to a caption can still confuse whether a visible operation has already occurred.

Negative construction affects the learned distinctions. Two captions can both accurately describe an image while the batch treats only one as its paired positive. Conversely, dataset artifacts can make a pair identifiable without learning the causal structure relevant to control. These issues do not invalidate contrastive learning; they identify its objective. An agent evaluation should therefore test exact distinctions such as enabled versus disabled, before versus after, or selected versus merely visible. Generic image-text retrieval can be included as a diagnostic, but should not substitute for the closed-loop decision task.

Temperature controls the sharpness of the softmax over similarities. A smaller temperature concentrates gradients on hard alternatives and can magnify calibration differences between domains. The similarity score is not

a calibrated probability that a proposed tool action is correct. Thresholding it as though it were one can yield unstable rejection behavior after a change in image style, prompt wording, or candidate set. Calibration should be measured against an explicit event, such as whether a retrieved frame contains the target control, under the deployment candidate distribution.

Cross-modal alignment also has a temporal dimension. A transcript can describe an earlier state while an image shows the current one. Combining them without timestamps can produce a fluent but internally inconsistent account. The representation pipeline should preserve modality, source, time, and transformation lineage. If an OCR engine and a vision-language model disagree, the agent needs a policy for seeking additional evidence rather than silently trusting whichever channel produces more confident language. Such disagreement is often an informative signal that the observation boundary needs another interaction.

5.5 Joint embedding prediction and the collapse question

I-JEPA predicts representations of target image regions from a context representation, using a predictor conditioned on target positions and a slowly updated target encoder [71]. In schematic notation, a context encoder gives $z_C = f_\phi(x_C)$, a target encoder gives $z_B = \overline{f}_{\bar{\phi}}(x)_B$, and a predictor produces $\widehat{z}_B = q_\psi(z_C, B)$. For illustration, a squared representation loss takes the form

$$\mathcal{L}_{\text{pred}} = \mathbb{E}_{x,C,B} \|\widehat{z}_B - \text{sg}(z_B)\|_2^2. \tag{5.6}$$

Here sg stops gradients through the target, and $\bar{\phi}$ is updated from the online encoder through an exponential moving average. The mask distribution determines which contextual and spatial relations must be learned. This squared loss is a schematic analysis objective: the inspected I-JEPA implementation uses smooth L_1, so reproducing that recipe requires its actual loss rather than substituting this expression.

Prediction in a learned latent space can avoid expending all capacity on unpredictable pixel detail. That is an attractive objective for agents whose actions depend on stable structure rather than every texture variation. However, which variation becomes irrelevant is learned from the data and objective. If a tiny visual indicator is weakly predictable yet action-critical, an abstraction optimized for broad semantic transfer may discard it. The appropriate question is whether the representation preserves task-relevant conditional distinctions, not whether it is more abstract in an unspecified sense.

The constant-representation solution clarifies a logical limitation. If both encoders output the same constant and the predictor outputs it, the schematic prediction loss is zero. Stop-gradient and a moving target change optimization dynamics but do not, by themselves, make this algebraic solution impossible. Actual JEPA training uses architectural, masking, normalization, and optimization choices that empirically avoid uninformative solutions in the reported settings. The existence of useful trained representations is an empirical result; a universal noncollapse guarantee should not be inferred from the abbreviated objective. This distinction is especially important when adapting the method to a new modality or action-conditioned domain.

V-JEPA extends feature prediction to video representations, learning temporal structure without reconstructing every observed pixel [72]. Temporal predictability and interventional identification remain different properties. Consider a binary latent variable $U \sim \text{Bernoulli}(1/2)$, an observational action $A = U$, and two possible transition mechanisms: $Y = A$ or $Y = U$. Both generate exactly the same observational pairs (A, Y). Under $\text{do}(A = 1)$, however, the first gives $P(Y = 1) = 1$ and the second gives $P(Y = 1) = 1/2$. This constructed counterexample shows that perfect passive prediction does not identify the effect of an intervention absent additional assumptions or action-grounded evidence.

The 2026 progression from dense V-JEPA features to action-conditioned policy components should therefore be evaluated at two distinct interfaces: preservation of observation distinctions and prediction of consequences under the deployed action distribution [62, 64]. A changed action schema can break the second while leaving the first intact. Relevant controls retain the encoder while changing the policy or action expert, then retain the action interface while changing the encoder. A frozen visual backbone can supply useful state features without constituting an identified transition model or a complete controller.

5.6 Multimodal fusion and executable grounding

A multimodal language model commonly maps visual features through a projection or adapter into representations consumed by a language decoder. Visual instruction tuning studies how such a system can answer or follow instructions involving images [73]. For an agent, the output must eventually identify an executable target and arguments. This adds a grounding obligation absent from many question-answering tasks: the referenced object must correspond to the current environment, and the selected action

must be valid there. Describing the correct button is not equivalent to clicking it, and producing a plausible screenshot description is not evidence that a state transition occurred.

Fusion strategies expose different bottlenecks. Early token fusion lets attention combine modalities directly but can consume a large context budget. A fixed-size visual summary controls cost while restricting recoverable detail. Cross-attention can separate language and visual lengths but adds its own compute and architectural assumptions. A tool-mediated approach leaves images outside the main model and retrieves structured descriptions or crops on demand. None dominates independently of task distribution. Exact document reading, spatial manipulation, and long video understanding demand different balances of detail, temporal coverage, and interaction latency.

The data pipeline must preserve the relation between natural-language instructions and observations. If a training example includes a final answer beside a screenshot taken after the action, the learner may exploit evidence unavailable when the action was chosen. A similar leak occurs when an evaluator supplies a full trajectory while judging a decision that should depend only on its prefix. Dataset construction should retain action timestamps and distinguish pre-action observations from post-action verification. The context-policy consistency issue is thus not restricted to text compression; it also applies to multimodal temporal alignment.

Untrusted visual content can contain instructions. A web page, document, or terminal screenshot can ask the agent to reveal secrets or change its objective. Converting that content to text through OCR does not change its authority. The system should retain provenance through every modality transformation and enforce authorization outside the model. A learned refusal policy can reduce harmful routing, but cannot replace an execution boundary that checks actual permissions. Evaluations should include task-relevant visual instructions and adversarial embedded instructions so that ignoring all image text is not rewarded as a successful defense.

5.7 Temporal sampling and observation freshness

Video and streaming interfaces require a choice of what to observe and when. Uniform frame sampling can miss short events; dense sampling can exhaust computation and context. For an event of duration d observed by periodic samples spaced Δ apart, a randomly phased event has detection probability $\min(1, d/\Delta)$ under ideal instantaneous sampling and visibility assumptions. This is a calculated sampling fact, not a benchmark result.

It shows why an agent that must notice transient confirmations or failures needs event-driven observations or a sampling rate matched to the event timescale.

Freshness is distinct from recognition accuracy. Let an observation be captured at time t_o and the action execute at t_a. Its age is $t_a - t_o$. A correct recognition at t_o can become wrong at t_a when another actor changes the environment. Systems should retain observation versions and use preconditions or refreshed observations before consequential operations. A multimodal model cannot infer an unobserved concurrent change reliably merely because it can describe the previous frame. The control policy should decide when the expected cost of stale evidence exceeds the cost of another observation.

A simple decision rule formalizes that trade. Suppose refreshing costs c_o and reduces expected action loss from L_{stale} to L_{fresh}. Refreshing is beneficial when $L_{\text{stale}} - L_{\text{fresh}} > c_o$, with all quantities measured in a common utility scale. Estimating the loss reduction is difficult and task-dependent; an arbitrary confidence score is not automatically that estimate. Nevertheless, the formulation separates uncertainty about recognition from uncertainty about environmental change. A high-confidence image classification can coexist with a high need for refresh.

An observation policy can itself be learned, but its training distribution must include the consequences of missed events and unnecessary observations. Rewarding only final task completion may obscure excessive sensing cost. Rewarding fewer frames without a reliability constraint can encourage omission of rare but decisive events. Evaluate sensing decisions with retained raw streams where permissions allow, so that missed information can be distinguished from reasoning failure. For privacy-sensitive environments, an appropriately scoped event record may be preferable to storing all video, but the resulting evidential limits should remain explicit.

5.8 Implementation and controlled comparison

The I-JEPA and V-JEPA repositories provide concrete encoders, predictors, masking pipelines, and training configurations [74, 75]. A reproduction should identify the exact snapshot and trace the target update, stop-gradient boundary, mask construction, positional embeddings, preprocessing, and downstream adaptation. A learned checkpoint can be incompatible with a changed resize or normalization policy even when tensor shapes match. The implementation record must bind preprocessing, weights, and evaluation.

An abstract training step separates observation visibility, gradient flow,

optimization, and target maintenance. Let $(x_t, p_t) \sim \mathcal{D}_A$ be a permitted training example with provenance p_t. Let $(C_t, B_t) \sim \mu_t(\cdot \mid x_t)$ select context and target positions from the example's index set. Their admissibility and possible overlap are determined by the chosen JEPA variant. The view x_{t,C_t} carries the selected observations together with their original position indices for the encoder's positional embeddings. Using the encoder and predictor notation of Section 5.5,

$$\begin{aligned} z_{C,t} &= f_{\phi_t}(x_{t,C_t}), \\ z_t^\star &= \mathrm{sg}\left[\overline{f}_{\bar{\phi}_t}(x_t)_{B_t}\right], \\ \widehat{z}_t &= q_{\psi_t}(z_{C,t}, B_t). \end{aligned} \tag{5.7}$$

Here ℓ_t is the declared prediction loss on the selected target representation. The operator sg preserves its argument's value while assigning zero derivative through the target branch. Write $w = (\phi, \psi)$, with current value $w_t = (\phi_t, \psi_t)$, and let s_t be optimizer state. A fresh gradient and the declared stateful optimizer update are

$$\begin{aligned} \mathcal{L}_t(w) &= \ell_t(q_\psi(f_\phi(x_{t,C_t}), B_t), z_t^\star)\,, \\ g_t &= \nabla_w \mathcal{L}_t(w_t), \\ (w_{t+1}, s_{t+1}) &= \mathcal{U}_t(w_t, s_t, g_t). \end{aligned} \tag{5.8}$$

The gradient contains no residual accumulation from the preceding step; optimizer state is retained according to $\mathcal{U}_t$. For the exponential-moving-average target update considered here,

$$\bar{\phi}_{t+1} = \tau_t \bar{\phi}_t + (1 - \tau_t)\phi_{t+1}, \qquad 0 \le \tau_t \le 1. \tag{5.9}$$

The order uses the updated online encoder, while the current loss uses the previous target. At $\tau_t = 1$ the target is unchanged for that update. Mask construction, loss, optimizer, and target schedule must match the inspected variant; these equations do not assert one universal JEPA recipe.

For I-JEPA, the paper's Section 3 prediction loss is implemented through `forward_target`, `forward_context`, and `loss_fn` inside `src/train.py`; spatial target construction is exposed by `MaskCollator` in `src/masks/multiblock.py`. The pinned V-JEPA repository has corresponding training functions in `app/vjepa/train.py` and a `VisionTransformerPredictor` in `src/models/predictor.py`. These checked code locations allow a reproduction to inspect where the target is detached, which positions are predicted, and whether a mask includes information that should have been hidden. A mask bug can make representation loss excellent while destroying the intended predictive task. The inspected I-JEPA

`loss_fn` calls `F.smooth_l1_loss`; the V-JEPA 2.1 path instead exposes an absolute-error power, per-target masking and optional distance weights. Its `forward_target` normalizes selected intermediate feature groups separately before concatenating them [74, 63]. The exponent, normalization axes, target positions and weighting schedule are therefore objective-defining variables, even when both implementations are called predictive representation learning.

The I-JEPA paper's Table 7 reports 66.9 percent versus 40.7 percent for target-encoder versus pixel targets in a ViT-L/16 low-shot linear evaluation using one percent of ImageNet-1K labels. The corresponding pretraining budgets are 500 and 800 epochs [71]. Those reported values support a useful target-space comparison in the authors' setup; the unequal training budgets preclude describing the difference as a perfectly compute-matched causal estimate. An agent-specific follow-up should hold preprocessing, training compute, policy, and observation access fixed, then test exact small-text distinctions and action success rather than assuming the classification advantage transfers unchanged.

For a controlled agent study, compare a frozen contrastive encoder, a frozen predictive encoder, a fine-tuned version of each, and a raw-observation or structured-state oracle where available. Keep the language policy, tool budget, and action interface fixed when attributing a representation effect. Include a separate end-to-end comparison that permits each method's best supported integration, but label it as a system comparison. Report training data exposure and adaptation compute: a method that uses action-labeled trajectories has a different information advantage from one trained only on passive images.

Task construction should include counterfactual visual pairs, temporal reorderings, altered themes, multilingual text, unseen layouts, and concurrent state changes. Split by application or environment lineage when template reuse could inflate transfer. Exact action outcomes should be verified through the environment rather than judged only from the model's explanation. Human or model judgments can still assess ambiguous semantics, but their uncertainty and disagreement need reporting. A visually plausible explanation should never substitute for evidence that the intended external effect occurred.

Multimodal resource accounting should identify where compression occurs. If v visual tokens are inserted into a language sequence of length n, the unmasked pair-count increment is $2nv + v^2$. Causal attention changes constants and visibility patterns but retains the interaction between old language context and new visual evidence. The vision encoder has a separate cost, and cross-attention architectures use a different accounting expression.

Table 5.2: Comparison axes for agent perception.

Axis	Controlled intervention	Diagnostic outcome
Spatial detail	Resolution and crop policy	Exact text and target localization.
Temporal coverage	Frame spacing and refresh	Missed transient events and staleness.
Representation objective	Contrastive or predictive	Transfer under action-relevant shifts.
Policy adaptation	Frozen or fine-tuned encoder	Benefit relative to added supervision.
Grounding channel	Pixels or structured state	Executed target correctness.

Reporting only the final language-token count can conceal an expensive image encoder or multiple high-resolution crops. Reporting only image resolution can conceal aggressive visual compression before the language policy sees the result.

Persistent multimodal memory adds another choice: retain raw observations, retain encoder features, or retain textual interpretations. Raw observations permit later re-encoding after a model update but consume storage and may carry sensitive information. Features are compact but tied to an encoder and preprocessing version. Textual interpretations can be searchable and cheap to condition on, while irreversibly losing visual details that were not described. A memory system should preserve the distinction between an observed fact and an interpretation derived from an image. When an agent later needs a detail absent from the interpretation, confidence in the summary cannot restore the missing evidence.

A useful ablation fixes storage bytes rather than the number of stored items. Compare raw thumbnails, cropped evidence regions, latent features, and textual summaries under that envelope. Measure retrieval of action-relevant facts, successful re-grounding after interface changes, and the cost of re-encoding or additional observation. Include an encoder-upgrade condition to expose representation-version dependence. The outcome may favor different storage forms for different timescales; there is no reason to expect one universal visual-memory format. Keeping the original evidence for selected high-value events can be worthwhile even when most routine observations are summarized.

Finally, visual confidence should be calibrated after fusion, not inherited

from one channel. A high image-text similarity, an OCR confidence, and a language-model token probability refer to different events and sample spaces. Multiplying them assumes a probabilistic model and dependence structure that usually has not been established. A decision-level calibrator can instead be trained and assessed against verified target selection or task outcomes, with separate evaluation under appearance and temporal shifts. Calibration then supports an explicit refresh or abstention policy rather than decorating an uncertain action with an unjustified numerical score.

Chapter 6

Action-Conditioned World Models, Diffusion, and Model-Based Control

6.1 LeFlow and amortized latent trajectory planning

LeFlow's August 2026 method generates a latent path with rectified flow, decodes latent transitions into action chunks through inverse dynamics, and reranks candidates by rollout through a frozen world model. Its Section 3 separates these components and includes consistency between generated and realizable trajectories [76]. This supplies a current counterpoint to solving an action-space optimization problem from scratch at every control step. The proposal distribution is learned offline, while verification still consumes online model computation.

An analytical cost comparison should keep that offline investment visible. If training the planner costs C_{train}, iterative planning costs c_{iter} per query, and amortized generation plus reranking costs c_{amort}, the latter has lower total compute after more than $C_{\text{train}}/(c_{\text{iter}} - c_{\text{amort}})$ queries when the denominator is positive and the compared quality constraint is satisfied. This is a break-even identity, not a claim about the paper's measured crossover. It fails as a deployment prediction when the learned prior becomes stale or must be retrained for changed dynamics.

The distinction between a generated endpoint and a realizable endpoint is equally important. Conditioning a latent generator on the desired goal can force its proposed path to end there by construction. That does not establish that decoded actions produce the path under the dynamics. Scoring the generated endpoint alone would reward a tautology. A rollout-based score instead checks what the decoded controls actually induce in the model, while retaining the real-model discrepancy analyzed below. A strong evaluation reports both generated-path error and rollout endpoint error, followed by actual environment success under receding-horizon execution.

6.2 Prediction under intervention

A world model for an agent predicts consequences conditional on actions and the information available before those actions. Let e_t be environment state, a_t an action, and o_t an observation. A fully observed model estimates $P(e_{t+1} \mid e_t, a_t)$ and a reward or task-progress function. Under partial observability it instead maintains a belief or learned latent state from H_t. Predicting a plausible next screenshot is not sufficient: the prediction must distinguish the consequences of the agent's available interventions, including failure, denial, and partial completion. A model that generates visually convincing but causally incorrect transitions can actively mislead a planner.

Observational data alone can leave the intervention effect unidentified. Suppose a training corpus shows a confirmation dialog followed by a completed purchase. The correlation does not establish which button was clicked, whether authorization was granted, or whether an unseen service completed the transaction. A controller using this correlation as an action model can infer success without performing the required operation. Action-labeled transitions, controlled interventions, and authoritative post-conditions reduce this ambiguity. They do not eliminate every hidden confound, especially when external services contain state not exposed to the agent.

The model's boundary should include failure semantics. A network timeout is not equivalent to a failed transaction: the remote operation may have committed while the response was lost. A useful transition model can represent this uncertainty and the value of a status query. A text generator trained only on successful demonstrations may instead fill in the expected success narrative. The distinction is operationally important for stateful agents because imagined certainty can cause duplicate execution, incorrect memory updates, or premature termination. The world model should support epistemic uncertainty about the actual external state, not merely stochastic variation in its rendered description.

World modeling also differs from a policy prior. A language model may know that tests usually follow a code edit, which helps propose an action sequence. That does not mean it predicts whether the particular patch passes the repository's tests. A learned policy can exploit regularities without an explicit simulator; a simulator can be useful even when the final policy executes without online lookahead. The comparative question is whether adding the model improves decisions under matched data and computation, and whether the gain comes from predictive accuracy, additional supervision, or extra search.

6.2.1 Latent dynamics and learning objectives

Let z_t be a stochastic latent variable and h_t a deterministic recurrent state. A generic latent dynamics model uses a prior $p_\phi(z_t \mid h_t)$, an observation-conditioned posterior $q_\phi(z_t \mid h_t, o_t)$, and an update $h_{t+1} = f_\phi(h_t, z_t, a_t)$. Observation, reward, and termination heads provide learning signals. A variational training objective combines reconstruction or prediction terms with a divergence between posterior and prior. Its purpose is to make a state inferable from real observations while allowing future latent trajectories to be sampled without access to those future observations.

The two directions of information flow matter. The posterior can use the new observation to correct its state estimate. The prior must predict before seeing that observation. A training loss that allows the prior to condition on future information is not a valid test of imagination. Similarly, a replay dataset must distinguish what was known at action time from what was later discovered. The same temporal leakage that damages agent policy training can make a world model appear more predictive than its deployable interface permits. Saving full trajectories is not enough unless their causal visibility is retained.

DreamerV3 provides a concrete latent-world-model approach in which the actor and critic learn from imagined latent trajectories. Its reported method uses separate prediction, dynamics, and representation losses, with stop-gradient placement and regularization controlling their interaction. The deployed actor in the cited version samples actions without online lookahead planning [77]. This distinction prevents a common category error: learning through imagined trajectories and searching a model at every decision are different uses of a world model. Both can improve control, but their computation and failure modes differ.

Representation pressure creates a tradeoff. Strong compression can make dynamics easier to predict while discarding task-critical detail. Weak compression can preserve observations while making the latent transition harder to learn. Reward and continuation predictions encourage retention of some control-relevant information, but their sufficiency depends on the training objectives and task family. A reward change can reveal distinctions the original representation ignored. A world model intended for reusable enterprise agents should therefore be evaluated under changed goals and tools, not only the reward function that shaped its latent state.

An uncertainty estimate should distinguish ambiguity in observations, stochastic environmental outcomes, and lack of model knowledge. Sampling multiple futures from one generative model does not automatically capture the third category. All samples can share the same systematic error.

Ensembles, held-out transition tests, and disagreement across independently informed predictors can provide diagnostics, but dependence and shared training data limit their interpretation. The controller needs a policy for falling back to real observations or conservative actions when model evidence is weak; otherwise uncertainty remains an unused output rather than a control mechanism.

6.2.2 Two temporal scales require an executable refinement interface

Dual-WM's September 2026 construction assigns primitive transitions and macro-action transitions to distinct latent spaces. Its Section 3 encodes windows of low-level features into a high-level state, learns a regularized macro-action interface, and refines high-level subgoals by projecting low-level rollouts back into the high-level space [78]. It does not require an inverse decoder from an arbitrary high-level state to one concrete primitive state. A final low-level goal phase restores distinctions that an abstract subgoal can omit.

The temporal units matter. If one low-level model action spans f environment operations and one macro transition spans k low-level steps, a high-level horizon H_H covers kfH_H operations. Comparing H_H directly with a primitive planning horizon confounds abstraction with temporal reach. Moreover, a high-level transition is usable only if some allowed primitive sequence realizes its proposed subgoal. A normal prior over macro-actions regularizes search but does not prove that every sampled point has an executable refinement.

For an agent experiment, fix an overall operation budget and compare primitive planning, two-scale planning without refinement validation, and two-scale planning with observed subgoal checks. Hold the number of environment operations covered constant before comparing prediction depth or latency. Include abstract states that alias different permissions or irreversible changes. If refinement reaches the high-level embedding but violates a required concrete predicate, score it as a failed refinement. This separates useful temporal abstraction from a planner that merely hides execution error inside a coarser representation.

6.3 Compounding error and a finite-horizon bound

The effect of model error grows with the horizon over which imagined outcomes influence selection. Consider a finite-horizon Markov system, a common initial-state distribution, a fixed policy, and true and modeled reward functions taking values in $[0, R]$. Suppose one-step transition total variation is uniformly at most ϵ_P and reward error at most ϵ_r on the union of state-action supports reachable under true and modeled dynamics. Couple true and modeled transitions so that, while their states agree, their actions agree and the next states disagree with probability at most ϵ_P. By a union bound, the probability of divergence before step t is at most $t\epsilon_P$, capped at one.

Before divergence, rewards differ by at most ϵ_r; after divergence their difference is at most R because both functions are bounded. Summing a conservative bound across H steps yields

$$|J_P(\pi) - J_{\widehat{P}}(\pi)| \leq H\epsilon_r + R\epsilon_P \frac{H(H-1)}{2}. \tag{6.1}$$

This is an elementary simulation argument under uniform error and bounded rewards, presented as a deduction rather than a new theorem. The quadratic term explains why a small one-step error need not justify long open-loop plans. The bound can be loose, and its assumptions are often harder to establish than a low average prediction loss.

The uniformity condition is the critical practical limit. A model can have excellent average error on logged behavior while being inaccurate on the unusual action sequences selected by optimization. Search actively seeks trajectories with high predicted value, which can concentrate on precisely those unsupported regions. This is model exploitation. Increasing search breadth can therefore reduce real performance even when it improves the planner's predicted objective. The relevant evaluation distribution includes states and actions induced by the planner, not only a random held-out subset of the original transition dataset.

If the return discrepancy is uniformly bounded by ϵ_J for every candidate policy and the model-based optimizer is at most η below its model optimum, its true-policy regret is at most $2\epsilon_J + \eta$. To prove this, add and subtract the modeled returns of the true optimum and selected policy. The two true-to-model discrepancies each contribute at most ϵ_J, and modeled suboptimality contributes at most η. This reduction makes the requirement explicit: improving search reduces η, but does not reduce model error. Once model error dominates, more precise optimization of the same inaccurate simulator

need not improve real performance. The result requires a common feasible policy class and a discrepancy bound over that class; a bound for one fixed logged policy is insufficient after selection.

Receding-horizon control reduces the interval over which the controller relies on uncorrected predictions. It plans for a horizon, executes a short prefix, observes the real outcome, and replans. This does not remove model error, but it can prevent an early mistake from propagating through an entire unobserved trajectory. The benefit must be balanced against observation cost, action latency, and irreversible effects. In a coding agent, running a cheap local test can correct a model of program behavior. In a payment workflow, an irreversible action may require stronger preconditions than replanning afterward can provide.

Table 6.1: World-model errors require distinct evaluations.

Error source	Misleading proxy	Better diagnostic
Representation aliasing	Low latent prediction loss	Paired states with different required actions.
Transition mismatch	Plausible next image	Verified action-conditioned state changes.
Reward exploitation	High imagined return	Real return after planner-selected actions.
Unseen interventions	Held-out logged transitions	Evaluation under changed action sequences.
Stale environment	Accurate past prediction	Freshness and precondition checks.

6.4 Diffusion as a conditional generative mechanism

Denoising diffusion models learn a reverse process associated with a prescribed noising process. For a declared schedule $0 < \beta_t < 1$, the standard Gaussian forward transition is $q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I)$. Writing $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ gives the directly sampled marginal

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{6.2}$$

A network can be trained to predict the injected noise under a timestep-conditioned squared-error objective. The reverse parameterization, variance choices, and sampling procedure determine how that prediction is used [79].

For an agent, conditioning can include previous observations, actions, goals, or structured state. The resulting generator models a distribution of possible future observations under the training data's conditional relationships. It does not become a causal simulator merely because actions appear in the prompt. The action labels must correspond to actual interventions, and the training data must identify the relevant effects. Otherwise the generator may produce what usually follows a textual description rather than what the external system would do. Visual realism can make this error harder to detect because the imagined outcome appears coherent.

Sampling cost is important when generation enters a planning loop. If a candidate trajectory requires K denoising evaluations and the planner evaluates B candidates over multiple rounds, model-call count can grow with both K and B. Batching can improve throughput while increasing peak memory. Reusing conditional features can save work but must respect changing actions and state. A comparison with a direct policy should include all generative sampling and scoring costs, not only the final chosen trajectory. A method that plans better with a much larger budget answers a different question from one that allocates equal computation more effectively.

Latent diffusion moves generation into a compressed representation produced by an encoder and decoded back into observations [80]. The compression reduces the dimensionality of the denoising computation, while introducing an information bottleneck and decoder distortion. If a safety-relevant indicator disappears in the latent representation, the diffusion process cannot reliably preserve it. A control-oriented latent generator should therefore be evaluated on exact state properties as well as perceptual quality. A small average image error can coexist with a wrong text label, object count, or action affordance.

Diffusion Transformers replace the denoising backbone with a Transformer-based architecture and study its scaling properties [81]. Architectural scaling results from image synthesis should not be promoted into evidence of reliable agent planning without a control experiment. The relevant bridge is whether action-conditioned generated trajectories improve verified decisions relative to matched alternatives. A generator can be useful for training diversity, counterfactual hypothesis generation, or visualization while remaining unsuitable as the authoritative source of environment state.

6.5 Flow matching and trajectory parameterization

Flow matching learns a time-dependent vector field that transports a simple distribution toward the data distribution [82]. For a simple illustrative path $x_t = (1 - t)x_0 + tx_1$, with endpoints drawn from a chosen coupling, the conditional velocity is $x_1 - x_0$. A regression objective trains $v_\theta(x_t, t)$ to predict that velocity. The marginally correct field is the conditional expectation of the target velocity given (x_t, t), obtained by the usual squared-loss projection property. This formulation separates a training path from the numerical solver used at sampling time.

The interpolation variable in this construction is not physical environment time. It parameterizes a transformation between distributions. Confusing the two can lead to an incorrect interpretation of an intermediate generated state as a physically possible step in an agent trajectory. A model can generate an entire action sequence or future video as one sample while its internal denoising or transport path passes through objects with no environmental meaning. Planning constraints must apply to the generated trajectory's action and state coordinates, not to arbitrary intermediate solver iterates.

Numerical integration introduces another approximation. A learned vector field can be accurate under its training distribution while a coarse solver accumulates error or evaluates off-distribution states. More solver steps may improve fidelity but cost latency. The solver, tolerance, precision, and conditioning policy belong to the reproducible method. A claimed model improvement that changes both the learned field and the solver cannot isolate which component produced the gain. For agents, the complete outcome also depends on the external verifier that accepts or rejects the generated plan.

Action constraints can be enforced through parameterization, constrained sampling, projection, or post-generation rejection. Each changes the distribution or feasible set. Projection may alter a generated trajectory so that its predicted consequences no longer correspond to its actions. Rejection can require many expensive samples when feasible plans are rare. A parameterization can guarantee local bounds, such as a control range, while leaving global requirements unproven. These mechanisms should be tested against exact environment constraints where possible, rather than assuming that a high-quality generator learns every hard rule from examples.

6.6 Executable worlds for language-model agents

Textual agents often operate in environments whose important state can be represented symbolically: files, database records, task statuses, permissions, and tool outputs. A world model can predict this structured state instead of rendering a visual future. Executable transition rules have the advantage that invariants can be checked directly. They have the disadvantage that a missing rule or hidden service behavior can invalidate the simulation. A generated tool response should be marked as imagined so that it cannot be mistaken for an actual observation and written into persistent memory as verified fact.

The separation between hypothetical and observed events should be structural. Maintain distinct event types or stores for simulation, proposals, and executed operations. A natural-language disclaimer embedded in the same untyped text is fragile: a later summarizer may omit it. If a planner imagines that a test passes and the memory writer records only the passing result, future sessions inherit fabricated evidence. The same problem appears when synthetic training trajectories are mixed with real traces without provenance. The system must preserve whether each transition was measured, simulated by executable rules, or sampled from a generative model.

Model validation should focus on transition invariants and counterexamples. For a file-editing environment, verify that reads reflect prior writes, path resolution is consistent, and failed operations do not silently mutate state. For a workflow environment, check allowed status transitions, authorization changes, and duplicate requests. A model that predicts common successful traces may still violate these invariants on rare branches. Such failures can be more consequential than an average next-observation loss suggests because planners deliberately explore unusual sequences.

The control loop separates model-generated futures from the evidence that updates the agent. Let H_t^o contain observed interaction records, $b_t = \mathcal{B}(H_t^o)$ be the inferred belief, and B_t the planning budget. A proposal operator $\mathcal{G}$ returns a finite set of nonempty action sequences,

$$\begin{aligned} \mathcal{P}_t &= \mathcal{G}(b_t, B_t) \subseteq \bigcup_{h=1}^{H_{\max}} \mathcal{A}^h, \\ \widehat{\tau}_t^{\,p} &\sim \widehat{P}_\phi(\cdot \mid b_t, p), \qquad p \in \mathcal{P}_t, \\ \widehat{J}_t(p) &= \mathcal{V}(\widehat{\tau}_t^{\,p}). \end{aligned} \tag{6.3}$$

The horizon limit $H_{\max}$ bounds sequence length, $\widehat{P}_\phi$ is the learned rollout law, and $\mathcal{V}$ is a declared scalar evaluation functional incorporating the

available uncertainty estimate. Neither that estimate nor the score is assumed calibrated. With a declared feasibility predicate χ,

$$\begin{aligned} \mathcal{P}_t^f &= \{p \in \mathcal{P}_t : \chi(p, b_t) = 1\}, \\ p_t^\star &\in \arg\max_{p \in \mathcal{P}_t^f} \widehat{J}_t(p), \qquad a_t = (p_t^\star)_1. \end{aligned} \tag{6.4}$$

This selection requires $\mathcal{P}_t^f \neq \varnothing$ and a declared tie rule. An empty set supplies no executable action; observation, replanning, or termination requires a separately specified policy. Predicted feasibility is not current execution authority.

Let $\widetilde{S}_t$ be refreshed execution state, $u(a_t, \widetilde{S}_t)$ the current authorization predicate, and $o_\perp$ a denial observation with no submitted operation. If K_E denotes the actual tool-response kernel, the observation law is

$$K_u(\mathrm{d}o \mid a_t, \widetilde{S}_t) = \begin{cases} \delta_{o_\perp}(\mathrm{d}o), & u(a_t, \widetilde{S}_t) = 0, \\ K_E(\mathrm{d}o \mid a_t, \widetilde{S}_t), & u(a_t, \widetilde{S}_t) = 1. \end{cases} \tag{6.5}$$

The symbol $\delta_{o_\perp}$ denotes a point mass, not a model-predicted response. Let $\oplus$ denote record append and let $\mathcal{U}$ implement the inference map $\mathcal{B}$ incrementally on the domain of observed interaction histories, including recorded denials. Then

$$\begin{aligned} o_{t+1} &\sim K_u(\cdot \mid a_t, \widetilde{S}_t), \\ H_{t+1}^o &= H_t^o \oplus (a_t, o_{t+1}), \\ b_{t+1} &= \mathcal{U}(b_t, a_t, o_{t+1}) = \mathcal{B}(H_{t+1}^o). \end{aligned} \tag{6.6}$$

The imagined trajectories $\widehat{\tau}_t^p$ never enter H_{t+1}^o as observations. A real tool response may itself report failure or an uncertain external outcome; recording it preserves that evidence rather than certifying the described effect. The update from this actual response is the control loop's anchor. Without it, the procedure can drift into optimizing its own narratives instead of the external task.

6.6.1 Current action conditioned implementations

The inspected V-JEPA 2 repository separates passive representation training from action-conditioned robot training. Its current `app/vjepa_droid/train.py` exposes `forward_target`, `forward_predictions`, and `_step_predictor`, while the V-JEPA 2.1 training module has a distinct representation-learning path [63]. This separation matters scientifically: a stronger visual representation and a better action-conditioned transition

model are related interventions, but their effects should not be conflated. The available code enables a reader to inspect where actions enter the predictor and which targets supply supervision.

The contemporary Dreamer implementation similarly exposes `Agent.policy`, `Agent.loss`, `imag_loss`, and `lambda_return` in `dreamerv3/agent.py` [83]. These checked function names identify the distinction between real interaction, model fitting, and imagined policy learning. A current repository snapshot may differ from the cited report's experimental release; reproducing its headline results requires the original configuration as well as code. For diffusion, the inspected Diffusers `DDPMScheduler.step` and `add_noise` in `scheduling_ddpm.py` make the forward-noise and reverse-update conventions explicit [84]. Swapping scheduler prediction types without changing the trained output interpretation is a concrete correctness error, not a harmless implementation variation. For example, writing f_θ for the raw model output and $\bar{\alpha}_t$ for the cumulative noise schedule, the scheduler's three interpretations recover different clean-sample estimates:

$$\begin{aligned} \widehat{x}_0^{(\epsilon)} &= \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, f_\theta}{\sqrt{\bar{\alpha}_t}}, \\ \widehat{x}_0^{(x)} &= f_\theta, \\ \widehat{x}_0^{(v)} &= \sqrt{\bar{\alpha}_t}\, x_t - \sqrt{1-\bar{\alpha}_t}\, f_\theta. \end{aligned} \tag{6.7}$$

These are alternative contracts for the trained output, not interchangeable decoders of one arbitrary checkpoint. Testing the same numeric tensor under all three modes would test a semantic mismatch rather than a solver improvement.

An integration test can hold the learned predictor fixed and replace imagined transitions with recorded real transitions for the same action sequence. The resulting oracle-transition control quantifies how much error arises in prediction versus downstream plan selection. A second ablation holds predicted transitions fixed while replacing the scorer with an independently verified objective. Neither oracle is deployable, and both must be labeled as diagnostic controls. Together they localize failures more sharply than a single final task score. They also expose a system whose representation predicts well but whose planner or reward interpretation remains wrong.

6.6.2 Trace reconstruction and the scope of a simulated fact

Trace2Env's Section 4 separates an immutable reconstructed worldbook, mutable episode state, and episodic interaction memory. The worldbook re-

tains schemas, observed transition evidence, induced rules, and provenance. An applicability gate distinguishes supporting evidence from uncertain or format-only evidence before a transition proposal reaches validation [85]. This addresses a concrete transfer error: an old trace can establish how a file-reading operation behaves without establishing that its particular file exists in a new episode. Reusable transition knowledge and current existence claims have different scopes.

In the pinned code, `RuntimeHarness.step` in `src/trace2env/runtime.py` validates candidate mutations, applies them to a copied state, checks types and verification, renders an observation, and commits through the session revision. `apply_mutations` and `check_invariants` reside in `engine.py`. An explicit `keep_observation` rejection policy can discard candidate effects while retaining a proposed observation [86]. In this branch, due events and the step increment have already been processed; the retained candidate is the resulting working state, not necessarily the original pre-step state. The returned verification object sets its acceptance flag while carrying an `effects_rejected` warning. An acceptance bit alone therefore cannot be interpreted as acceptance of the proposed mutations. Preserving a fluent observation while rejecting its associated effects creates a different semantic object from a verified matching transition, and both the policy and warning must enter the experiment record.

Let $I(s)$ be a checked state invariant and $C(s, o)$ an observation-consistency predicate. A transition gate establishing $I(s')$ does not thereby establish $C(s', o')$, and neither condition proves equality with the real environment transition. A proposed successful write can leave state unchanged after effect rejection while its observation still reports completion. The pertinent test checks the returned observation against the committed state and then replays held-out actions in the real environment. Track effect rejection, observation inconsistency, and real-transition mismatch as separate outcomes. This turns a general demand for simulator fidelity into independently falsifiable interfaces.

6.6.3 Generated executable worlds and learned response simulators

WildAWM synthesizes replayable computer states, cross-interface tasks, and executable verifiers. Its Section 3 includes repeated state replay, agent-based checking, and quality gates for initial noncompletion, solvability, difficulty, and judgment consistency [87]. These are environment-construction operations: once materialized, application code and stored state execute the interaction. Qwen-AgentWorld instead trains a language model to

produce environment responses through continual pretraining, supervised reasoning traces, and reinforcement learning. Its Section 3 retains masked low-information turns as context while reducing their contribution to the predictive loss [88]. Both support agent learning, but their transition semantics and error sources differ.

An executable synthetic environment can be internally deterministic yet implement the wrong task semantics. A learned simulator can return a plausible response while violating a prior state commitment. Agreement among a generated verifier and model judges is valuable evidence, but shared generation assumptions can make their errors correlated. Independent checks should therefore perturb a task's initial state, remove one required effect, and alter an irrelevant field to test verifier sensitivity and specificity. For response models, replay the same action sequence after earlier writes and denials; for executable worlds, inspect reset equivalence and dependencies spanning GUI, terminal, and service interfaces.

Cost comparisons require the same boundary. Charge world construction and verifier repair to the synthesized environment, and training plus sampled response generation to the learned simulator. Then report downstream real-environment performance separately from agreement with a synthetic judge. A gain in a simulated policy's return can reflect exploitation of the simulator or verifier, so a real held-out transfer set is indispensable. The comparison should identify which component provides reliable transitions, which supplies task diversity, and which determines reward; averaging those roles into one world-model score would conceal the mechanism.

Figure 6.1 isolates the mechanism and its stated information boundaries.

6.7 Experiments that separate prediction from control

A useful evaluation has three layers. First measure one-step prediction under a held-out transition distribution, with explicit action and observation timing. Second measure multi-step prediction under fixed action sequences, distinguishing teacher-forced correction from free rollout. Third measure closed-loop task outcomes when a planner or learned actor uses the model. Improvement at one layer need not imply improvement at the next. The study should retain cases where a model predicts more accurately but induces worse decisions; these cases can reveal reward mismatch, uncertainty miscalibration, or search exploitation.

Baselines should include a direct policy with the same action interface, a

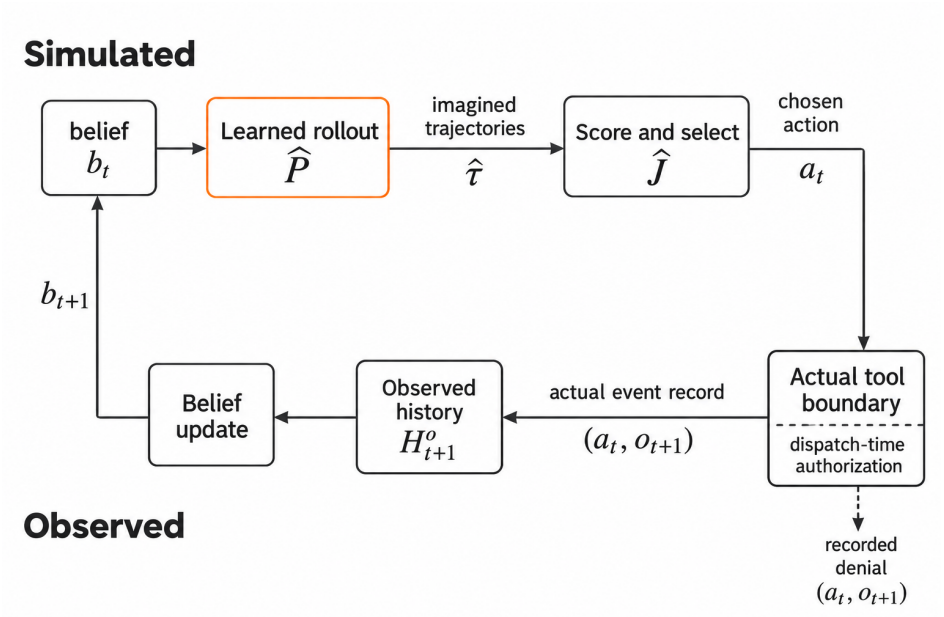

Figure 6.1: Model-generated futures and observed interaction have different evidential types. Candidate rollouts and scores guide selection; the actual attempted-action/response record updates H^o_{t+1} and the belief derived from it. The tool boundary checks current authority and can return denial without dispatch. An observed response can still leave an external effect uncertain. No experiment is reported. The denial note names one possible actual return, not a separate evidential history.

simple deterministic transition approximation where meaningful, a model-free memory of retrieved transitions, and an exact-transition simulator as a nondeployable diagnostic when available. An exact simulator does not make a fixed finite-budget planner optimal; removing model error can even remove a compensating bias in its heuristic. It is an upper bound only under a separately justified optimization and comparison contract. Match real environment interactions separately from model computation, and report both learning curves and resource curves.

The transition distribution selected by the planner is part of the evidence boundary. Let d_0 be the validation distribution over state-action pairs, d_π the planner-induced distribution, and $e \geq 0$ a specified transition loss. If $d_\pi \ll d_0$ and $\mathrm{d}d_\pi/\mathrm{d}d_0 \leq \kappa$ almost everywhere, then

$$\mathbb{E}_{d_\pi}[e] = \mathbb{E}_{d_0}\left[\frac{\mathrm{d}d_\pi}{\mathrm{d}d_0}e\right] \leq \kappa\,\mathbb{E}_{d_0}[e]. \tag{6.8}$$

This change-of-measure deduction requires a coverage bound; an ordinary held-out error estimate does not supply κ. When search selects transitions outside validation support, the displayed guarantee is unavailable. Evaluate the model on states selected by the actual planner, with selected examples excluded from fitting. Trace2Env's applicability gate and WorldEvolver's selective foresight address related exposure decisions through different mechanisms; neither should be interpreted as a proved density-ratio certificate [85, 89].

Table 6.2: Controlled interventions for imagined interaction.

Intervention	Fixed comparison element	Question answered
Model versus no model	Policy and real data	Does prediction add useful information?
Short versus long rollout	Sampling budget	When does compounding error dominate?
More candidate plans	World model checkpoint	Does search exploit model error?
Goal or tool change	Observation domain	Is the latent state reusable?
Real versus imagined memory	Storage policy	Does synthetic evidence contaminate later decisions?

Contamination controls should account for shared environments and generated data. Holding out task prompts while retaining the same transition templates can overstate generalization. Split by environment dynamics or tool implementation when the claim concerns new mechanisms. Evaluate rare failures and partial observations, not only successful demonstrations. Use paired seeds where appropriate and report uncertainty over environments as well as episodes. A large number of generated rollouts from one flawed world does not provide the same evidence as independent real-world episodes.

Text, graph, and latent representations require a common access contract before comparison. Give each method the same permitted observation history, action schema, real-transition training set, external memory access, and total planning budget. Charge parsing and graph construction to the graph system, language generation and rereading to the text system, and encoding plus rollout to the latent system. Evaluate the same goal

predicates, irreversible transitions, and permission changes. Textual plausibility, preservation of graph invariants, and latent prediction error are mechanism-specific diagnostics; verified reachability and execution success provide the shared behavioral comparison. If one representation receives hidden state, an oracle parser, or a larger simulator budget, report that advantage explicitly rather than attributing its effect to the representation format.

6.8 Practical limits of imagination

World models are valuable when their predictions preserve the distinctions that change a decision, their uncertainty influences control, and real observations correct their state. They are especially attractive when external trials are expensive or risky. Their principal failure is not merely an unattractive generated sample; it is a confidently wrong counterfactual that a planner selects and a memory service later treats as truth. Reliable use therefore requires typed provenance, bounded rollout horizons, real-state verification, and explicit treatment of irreversible effects.

The resource and evidence boundary should remain visible in every claim. A trained actor benefiting from imagined experience is different from online model-predictive control. A visually realistic generator is different from an executable simulator. A latent prediction objective is different from an action-sufficient representation. The strongest comparative studies connect these levels with ablations and verified outcomes instead of allowing one attractive metric to stand in for all of them. Under that standard, world modeling becomes a testable component of agentic intelligence rather than an undefined promise of internal simulation.

A quantitative horizon-selection example. The finite-horizon bound can guide an experiment without pretending that its unknown premises have been measured. Suppose, hypothetically, rewards lie in $[0, 1]$, uniform reward error is at most 0.01, and uniform transition total variation is at most 0.001. At horizon 10, the bound is $0.1 + 0.045 = 0.145$; at horizon 40 it is $0.4 + 0.78 = 1.18$. These are calculated worst-case return discrepancies under deliberately specified assumptions. They are not empirical errors of Dreamer or a language-model world model. The example shows how horizon changes can dominate a small improvement in average prediction loss. If the uniform transition premise is unavailable, these numbers cannot be used as a deployment certificate.

A more discriminating control experiment estimates errors on planner-

induced trajectories and compares short-horizon replanning against long open-loop execution at matched total model calls. Record which candidate states fall outside the training support according to a predeclared diagnostic, then execute a safe subset in the real environment to check that diagnostic. An uncertainty penalty is useful only if it predicts meaningful error on these selected states. Otherwise it can simply encode familiarity with superficial features. The uncertainty model should be calibrated on data held out from both world-model fitting and planner selection, with the remaining distribution-shift limitation made explicit.

Termination prediction deserves separate auditing. If a model incorrectly predicts successful completion, a planner can avoid further checks and inflate imagined return. If it incorrectly predicts failure, it can waste resources repeating an already completed operation. A continuation head is therefore part of the control semantics, not merely an auxiliary reconstruction objective. Test terminal success, terminal failure, recoverable interruption, and unknown remote status as distinct conditions. For long-running agents, the same task can cross multiple sessions, so a process ending is not necessarily an environment terminal state.

Finally, a world model can be evaluated as an evidence-selection mechanism rather than an authoritative simulator. It may predict which real query would resolve uncertainty, then ask the tool for that query. This use can tolerate some prediction error because the final fact comes from the external source. Compare it with random querying, fixed checklists, and a direct policy under the same query budget. The resulting outcome identifies whether imagined interaction improves information gathering, even when the model is insufficiently accurate for open-loop control. Separating these roles prevents an unnecessarily binary judgment that a world model is either universally reliable or useless.

Chapter 7

Pretraining Objectives, Scaling, and Distributed Optimization

7.1 DeepSeek V4 staged pretraining and distributed state

The 2026 DeepSeek-V4 report's Section 4 describes agentic data during mid-training, sample-level pretraining masks, and progressive context extension from 4K toward one million tokens. Its infrastructure discussion includes tensor-level checkpointing and communication overlap; its optimizer setup combines Muon for most parameters with AdamW for selected modules [90]. These are author-reported design choices, not independently reproduced implementation claims. They show why a contemporary account cannot reduce training to a single dense-model parameter count and a universal Adam memory multiplier.

Memory accounting must be applied component by component. Partition parameters into optimizer classes $\mathcal{P}_1, \ldots, \mathcal{P}_k$, with bytes per state and sharding rules for each. Persistent memory is the sum of their actual resident states; transient memory depends on overlapping gathers and updates. A matrix-specific optimizer can have different state shape and communication from a vector-wise optimizer. Consequently, replacing the word Adam in a configuration does not justify reusing the same bytes-per-parameter estimate without inspection. The computation and stability implications also depend on how matrix updates are distributed.

Progressive context training changes both the data distribution and the hardware workload. For a simplified dense-attention accounting model, let stage j process D_j tokens in causally independent segments of lengths n_{jr}, with $\sum_r n_{jr} = D_j$, L_j layers, width d_j, and mean activated parameter count N_j. Then

$$F_{\text{train}} \approx \sum_j \left[\alpha_j N_j D_j + \beta_j L_j d_j \sum_r n_{jr}^2 \right] + F_{\text{extra}}. \tag{7.1}$$

The declared coefficients encode the multiply-add and forward/backward conventions; F_{extra} includes separately counted recomputation and routing

work. This is an analytical model, not the report's measured training bill. Sparse or compressed attention requires its own pair-count expression. A mask that forbids cross-document attention does not guarantee that a selected kernel skips every masked arithmetic tile.

At fixed token count, the dense attention term depends on the token-weighted segment length, not merely the final maximum window. Charging all stages at that maximum can overestimate work; extrapolating early-stage throughput can underestimate it. Record stage token counts, actual sequence shapes, masks, precision, optimizer states, and kernel dispatch. This distinguishes long-context training from architectural compression even when both advertise the same maximum window, and connects training provenance to the inference accounting in Chapter 31.

7.2 What pretraining supplies to an agent

Autoregressive pretraining minimizes predictive loss on an encoded corpus, not task regret in an interactive environment. For sequences x drawn from a mixture $\mathcal{D}$, the objective is $\mathcal{L}(\theta) = -\mathbb{E}_{x\sim\mathcal{D}} \sum_i w_i \log p_\theta(x_i \mid x_{<i})$, where fixed nonnegative weights w_i specify which positions contribute. The model can acquire linguistic, procedural, and factual regularities useful to agents, but the loss does not distinguish an executed operation from a story about execution unless the data and interface preserve that distinction. A corpus containing fluent success narratives can teach the form of an answer without teaching reliable state verification.

Mixture design is therefore a policy-relevant intervention. Code, documentation, mathematical derivations, conversations, tool traces, and multimodal data contribute different conditional structures. Their sampling weights affect both information exposure and the optimization trajectory. A small collection of repeated tool traces can receive more effective training weight than a large rare domain. Counting unique documents is insufficient; report deduplication units, sampling probabilities, token exposure, and epoch repetition. For a stateful agent, training on histories with explicit failures, corrections, and permission boundaries may be more relevant than adding another large batch of generic prose.

Loss masks define the learning target in serialized interactions. A trace can contain user inputs, assistant actions, tool observations, and evaluator annotations. Training every position asks the model to predict all of them; training only assistant positions asks it to imitate the agent's behavior given the surrounding records. Neither is automatically correct for every objective. The decisive requirement is causal visibility. Future tool responses

or retrospective summaries must not appear in the conditioning context for an earlier action. Otherwise the learned policy is rewarded for using evidence unavailable at execution time.

Sequence packing adds a related implementation hazard. Several documents can share one physical tensor for utilization, while an attention mask prevents unwanted cross-document conditioning. Omitting the mask changes the data distribution by letting one document predict another. For independent text this may merely alter modeling assumptions; for agent traces it can leak answers, test outcomes, or private task metadata across examples. Position resets, end-of-document markers, loss masking, and attention masking should be checked together. A shape-correct batch is not necessarily a semantically correct training example.

7.2.1 Data integrity and causal trace construction

Deduplication is a statistical control as well as a storage optimization. Repeated content changes effective sampling weights and can leak benchmark material across splits. Exact hashing catches identical bytes; normalized or approximate matching catches additional overlap but can merge legitimately distinct examples. For code and agent traces, repository lineage, issue templates, and generated variants can carry the same underlying solution even when strings differ. The deduplication policy should state its unit and false-positive tradeoff. A claim of uncontaminated evaluation requires evidence about the held-out tasks, not merely that the training corpus passed a generic duplicate filter.

Agent trace construction needs stronger provenance than ordinary document collection. Each observation should identify its source and whether it was actually available before the action. A later grader's explanation is not an environment observation. A synthetic tool result is not a measured one. A trace modified after execution can be useful training data, but its conditioning semantics must be reconstructed deliberately. MemoryWalker directly studies training on contexts that differ from the live contexts of edited agent rollouts [23]. Its relevance extends to pretraining mixtures containing retrospective summaries or reconstructed tool histories, even when no reinforcement-learning update is being performed.

One practical audit reconstructs the visible prefix for every supervised assistant action and compares it with the serialized training input. Differences are classified as intentional transformations, unavailable evidence, or unresolved discrepancies. The audit should also check whether system instructions, tool schemas, and permissions match the recorded environment. A policy trained to imitate an action that was valid under an earlier

schema can emit an invalid operation after deployment changes. Data age alone does not identify this mismatch; compatibility metadata does.

Curriculum and filtering can introduce selection effects. Retaining only successful trajectories teaches useful behavior but removes evidence about failure recognition and recovery. Retaining only traces accepted by one learned judge can amplify that judge's blind spots. A mixture containing both successful and failed traces needs explicit targets: imitation of an erroneous action is not made desirable merely because the surrounding record is informative. Alternative objectives can predict outcomes, classify failure states, or train corrective continuations. The experiment should specify which information is learned from each trace category rather than relying on a vague label such as high-quality data.

7.3 Scaling laws under an explicit compute model

An empirical scaling fit can organize model and data allocation. Consider $L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$ with positive coefficients, where N is parameter count and D training tokens. Under the approximate constraint $C = \kappa ND$, substitute $D = C/(\kappa N)$ and differentiate with respect to N. The stationary condition is

$$N^{\alpha+\beta} = \frac{\alpha A}{\beta B}\left(\frac{C}{\kappa}\right)^{\beta}. \tag{7.2}$$

Thus $N \propto C^{\beta/(\alpha+\beta)}$ and $D \propto C^{\alpha/(\alpha+\beta)}$ within this model. Equal exponents imply equal power-law scaling of model size and tokens. The derivation is algebraic; whether the fitted loss form extrapolates is empirical.

The Chinchilla study compares allocation strategies under training compute and reports a 70-billion-parameter model trained on 1.4 trillion tokens, against Gopher's 280 billion parameters and 300 billion tokens in its comparison table [91]. These are author-reported configurations, not a universal prescription for later architectures. The fitted frontier depends on data quality, tokenizer, optimization, context length, and the compute accounting convention. Inference cost also changes the economic objective: a model served repeatedly can justify more training to reduce per-query cost, even when that choice is not optimal for one fixed pretraining-FLOP budget.

For agents, the downstream objective further complicates allocation. A lower average next-token loss need not improve rare tool syntax, permission handling, or multi-step recovery in the same proportion. A model can improve on common text while remaining poorly calibrated on long action histories. Scaling studies should therefore include held-out interactive

diagnostics whose exposure is controlled. These diagnostics should not be repeatedly tuned into the training mixture without retaining an untouched evaluation set. Otherwise the apparent relation between scale and agent competence can become a relation between scale and benchmark-specific data engineering.

The approximation $C \approx 6ND$ is useful for dense Transformer training under particular forward/backward accounting assumptions. It omits or simplifies attention's length-dependent contribution, embeddings, sparsity, recomputation, and communication. A long-context training run can depart materially from this estimate. Mixture-of-experts training must distinguish total stored parameters from activated parameters, while routing and all-to-all communication introduce additional costs. Report estimated mathematical FLOPs separately from hardware utilization and wall-clock time. Calling a run compute-matched because parameter count times tokens matches can conceal substantial differences in the actual computation.

7.4 Optimizer state and distributed memory accounting

Training memory contains model states, activations, temporary workspaces, and communication buffers. Under one explicit mixed-precision Adam convention, each parameter has a two-byte working weight, a two-byte gradient, a four-byte master weight, and two four-byte moment estimates, totaling 16 bytes before other allocations. Different implementations can use FP32 gradients, compressed states, or no separate master copy, so the convention must accompany the number. For a hypothetical seven-billion-parameter model, this layout requires 112 decimal GB, approximately 104.3 GiB, for these persistent states alone.

ZeRO partitions optimizer states, gradients, and parameters in progressively more extensive configurations [92]. Let the data-parallel degree be n and let the working-weight, gradient, and optimizer-state bytes per parameter be b_w, b_g, b_o. Idealized per-rank persistent storage is $(b_w + b_g + b_o/n)N$ for optimizer partitioning, $(b_w + (b_g + b_o)/n)N$ when gradients are also partitioned, and $(b_w + b_g + b_o)N/n$ when all three are partitioned. These expressions omit padding, replicated metadata, transient gathers, activations, and memory fragmentation. They are lower-level accounting expressions, not capacity guarantees for a real run.

The largest transient allocation can determine feasibility even when the persistent-state table looks comfortable. A parameter shard may be gathered before its layer executes, retained for backward, or prefetched

Table 7.1: Calculated persistent state per rank for a hypothetical 7B model and eight ranks.

Partitioning	Assumed layout	Decimal GB per rank
Replicated	$2 + 2 + 12$ bytes/parameter	112.0
Optimizer only	$2 + 2 + 12/8$	38.5
Optimizer and gradient	$2 + (2 + 12)/8$	26.25
All model states	$(2 + 2 + 12)/8$	14.0

while another layer runs. Larger buckets improve communication efficiency but increase peak memory. Activation checkpointing reduces retained intermediates by recomputing them during backward, adding arithmetic and potentially changing kernel schedules. Offloading moves state to host memory or storage, introducing link bandwidth and latency constraints. A complete capacity estimate must include the overlap policy, not merely sum ideal sharded tensors.

The implementation connection is concrete. The class `DeepSpeed ZeroOptimizer_Stage3` belongs to the pinned module `deepspeed/runtime/zero/stage3.py`, alongside gradient-partitioning logic and checkpoint-state methods [93]. Its later implementation has features beyond the original ZeRO paper, so reproduction should trace the configured path rather than attribute every current feature to that paper. A falsifiable memory experiment records peak allocated and reserved memory during forward, backward, optimizer step, and checkpoint save, then compares those observations with the state accounting. Unexplained peaks should be diagnosed before increasing the nominal model size.

7.5 Communication and parallel decomposition

Data parallelism replicates a model across examples and combines gradients. Tensor parallelism partitions individual matrix operations. Pipeline parallelism partitions layers across stages. Expert parallelism distributes conditional experts. Sequence or context parallelism partitions token-related work and associated state. These decompositions can be combined, but their communication patterns interact with the physical topology. An arrangement that minimizes memory can perform poorly when frequent communication crosses a slow link. The parallelism configuration is part of the scientific method because it changes feasible batch shapes, numerical

reduction order, and time available for training.

For a ring all-reduce of a tensor with D_g bytes over n ranks, an ideal algorithm transfers approximately $2(n-1)D_g/n$ bytes per rank, ignoring protocol overhead. With a hypothetical 14-GB gradient tensor and eight ranks, this is 24.5 GB per rank per all-reduce. The formula explains why gradient accumulation can amortize communication over several microbatches, provided the accumulation semantics are correct. It does not predict duration without topology, effective bandwidth, overlap, and contention. Multiple parallel groups sharing the same links can invalidate an isolated collective microbenchmark as a training-time predictor.

Megatron's tensor-parallel construction partitions complementary matrix dimensions so that communication can be placed between substantial blocks of computation [94]. The pinned `megatron/core/tensor_parallel/layers.py` exposes `ColumnParallelLinear`, `RowParallelLinear`, and `VocabParallelEmbedding` [95]. For $Y = XW$ with W split by output columns, each rank computes a subset of output features. A following compatible row partition can consume those features locally before a reduction combines partial outputs. The exact collective placement depends on the chosen layer composition and implementation options; a generic claim that tensor parallelism simply divides all compute by the rank count is false.

Pipeline schedules trade utilization against activation residency and stale work. A simple fill-and-drain pipeline has bubbles whose relative impact decreases with more microbatches, while more microbatches can increase activation memory or delay updates. Interleaving and other schedules change the tradeoff. For long agent traces, variable sequence lengths complicate balance: a stage can wait on an unusually expensive attention computation even if each stage has the same layer count. Measure stage-level time distributions and communication stalls rather than inferring balance from parameter allocation alone.

7.6 Numerical stability and reproducibility

Mixed precision changes the numerical contract. Lower-precision matrix multiplication can provide high throughput, while accumulation, normalization, optimizer updates, and reductions may require higher precision. Dynamic loss scaling can prevent small gradients from underflowing but does not solve every overflow or instability. Gradient clipping changes the update and should be reported with its norm convention and distributed aggregation. A local shard norm is not generally the global gradient norm. For

a disjoint partition $\mathcal{S}$ of logical gradient coordinates, $\|g\|_2^2 = \sum_{S \in \mathcal{S}} \sum_{i \in S} g_i^2$; replicated copies must not be counted as additional coordinates. The inspected Stage-3 `get_grad_norm_direct` uses parameter-ownership predicates and configured partition/model/expert reductions, rather than simply summing every resident tensor [93]. Testing the norm therefore requires the actual replication and process-group map as well as the tensor values. Incorrect clipping across parallel groups can create a different optimizer even when the learning-rate schedule is unchanged.

Randomness includes initialization, dropout, data order, sampling, and distributed execution choices. Saving a seed without generator state and data-loader position is insufficient for exact resumption. Some kernels are nondeterministic, and changing the number of ranks can change reduction order and sample assignment. Reproducibility should distinguish bitwise replay from statistical agreement of learning curves and downstream outcomes. The latter still requires enough repeated runs or a justified uncertainty analysis. A single expensive pretraining run provides limited evidence about sensitivity to optimization randomness.

Checkpoint integrity requires a consistent snapshot of weights, optimizer state, scheduler, random generators, data progress, and partition metadata. A partial write can leave shards individually readable but mutually inconsistent. Use a manifest that binds expected shards and checksums, with a commit marker written only after all required state is durable. Recovery should verify the manifest before constructing a model. This is a systems property that can be tested through fault injection: interrupt saves, remove a shard, alter a checksum, or resume under an incompatible parallel layout and confirm explicit failure or documented conversion.

Training-service monitoring should distinguish transient hardware errors from statistical divergence. Loss spikes, gradient norm changes, nonfinite values, data anomalies, and communication retries carry different evidence. Automatically discarding a spike can hide a genuine data-quality problem; automatically restarting from the same checkpoint can repeat the same deterministic failure. A recovery policy should retain the offending batch identity and numerical context where permitted. Otherwise the system can consume substantial compute without resolving the underlying issue, while the final report shows only the successful resumed segment.

Unequal sequence lengths can silently change the objective. Suppose rank j has B_j supervised tokens after masking and computes a local mean loss L_j. Averaging those local means across n ranks gives $n^{-1} \sum_j L_j$. The intended global token mean is instead $\sum_j B_j L_j / \sum_j B_j$. They agree

only under special conditions, such as equal supervised-token counts or identical local losses. The same discrepancy applies to gradients. Variable-length agent traces, tool-observation masks, and uneven packing make this more than a theoretical corner case: short traces can receive disproportionate weight when each rank normalizes locally before an unweighted average.

The correction is to aggregate the appropriate numerator and denominator or scale local contributions consistently with the collective's averaging convention. Gradient accumulation adds another level. If microbatch k on rank j contains B_{jk} supervised tokens, dividing every microbatch loss by a fixed accumulation count does not implement the global token mean when these counts differ. The correct relative contribution is proportional to B_{jk} for a token-weighted objective. A sequence-weighted objective is also legitimate, but must be intentional and implemented consistently. The distinction changes how long reasoning traces and short tool calls influence the learned policy.

A minimal verification uses a tiny deterministic model and two microbatches with deliberately unequal mask counts. Compute one reference gradient on their concatenation under the intended objective. Compare it with gradients from the distributed or accumulated path before the optimizer step. Repeat with a fully masked microbatch, which should have defined behavior rather than division by zero, and with uneven examples across ranks. This test is meaningful because it checks a mathematical objective independently of a large model's apparent convergence. A training curve can look normal while optimizing the wrong weighting of trace types.

Another useful check separates clipping from averaging. Clipping each microbatch gradient before accumulation generally differs from clipping the final accumulated gradient because clipping is nonlinear. Likewise, clipping each rank's local gradient differs from clipping the global combined gradient. Either policy can be chosen, but it changes the optimizer and its stability behavior. Record where loss scaling, unscaling, accumulation, reduction, and clipping occur. These details are particularly important when a post-training stage mixes long-horizon action trajectories with short preference examples on the same distributed infrastructure.

The numerical and causal contracts meet at the masked token. Its target, visibility, weight, and distributed contribution all determine the update. A trace that is causally correct but incorrectly normalized can bias learning toward an unintended task mixture. A numerically correct all-reduce cannot repair future-information leakage in the local batch. Auditing both layers produces a stronger interpretation of downstream agent gains than treating data preparation and infrastructure as independent implementation details.

For an averaging collective, the accumulation can be specified directly as a token-weighted gradient. Let $\theta \in \mathbb{R}^p$, with $p \geq 1$. On rank $j \in \{1, \ldots, n\}$, let $k \in \{1, \ldots, K_j\}$ index microbatches and $i \in I_{jk}$ their token positions. Let $a_{jki} \in \{0, 1\}$ be a fixed authoritative supervision mask and $\ell_{jki}(\theta)$ the corresponding differentiable token loss. The window defines

$$\begin{aligned} B &= \sum_{j=1}^{n} \sum_{k=1}^{K_j} \sum_{i \in I_{jk}} a_{jki}, \\ N_{jk}(\theta) &= \sum_{i \in I_{jk}} a_{jki} \ell_{jki}(\theta), \\ L(\theta) &= \frac{1}{B} \sum_{j=1}^{n} \sum_{k=1}^{K_j} N_{jk}(\theta), \qquad B > 0. \end{aligned} \tag{7.3}$$

All contributions are differentiated at the same parameter snapshot θ_t. With no additional microbatch normalization, the local and collective updates are

$$\begin{aligned} g_{j,0} &= \mathbf{0}, \\ g_{j,k} &= g_{j,k-1} + \frac{n}{B} \nabla_\theta N_{jk}(\theta_t), \\ g_t &= \frac{1}{n} \sum_{j=1}^{n} g_{j,K_j} = \nabla_\theta L(\theta_t). \end{aligned} \tag{7.4}$$

For a collective that sums rather than averages, replace n/B by $1/B$ and remove the outer $1/n$. A fully masked window has $B = 0$ and is rejected before this update is defined; it does not invoke the optimizer transition below. A fully masked microbatch within a valid window contributes the zero numerator without introducing a local division by zero.

Let $z_t = (\theta_t, \omega_t) \in \mathbb{R}^p \times \Omega$ include the declared optimizer state $\omega_t \in \Omega$. For finite $g \in \mathbb{R}^p$, a global norm limit $c \in (0, \infty]$ gives the optional clipping map and optimizer transition

$$\begin{aligned} \mathcal{K}_c(g) &= \begin{cases} g, & \|g\|_2 \leq c, \\ c\, g / \|g\|_2, & \|g\|_2 > c, \end{cases} \\ z_{t+1} &= \mathcal{U}_t(z_t, \mathcal{K}_c(g_t)) \,. \end{aligned} \tag{7.5}$$

The choice $c = \infty$ disables clipping, and $\mathcal{K}_c(\mathbf{0}) = \mathbf{0}$ without a division by zero. The operator $\mathcal{U}_t$ is the configured optimizer, not an assumption of plain gradient descent. A zero input gradient can still change weights or optimizer state through momentum, weight decay, or step counters; it differs from rejecting a window without invoking $\mathcal{U}_t$. The gradient identity

holds in exact arithmetic with fixed masks; loss scaling must be undone consistently, and rounding, reduction order, and backend accumulation behavior require the preceding implementation checks.

Figure 7.1 isolates the mechanism and its stated information boundaries.

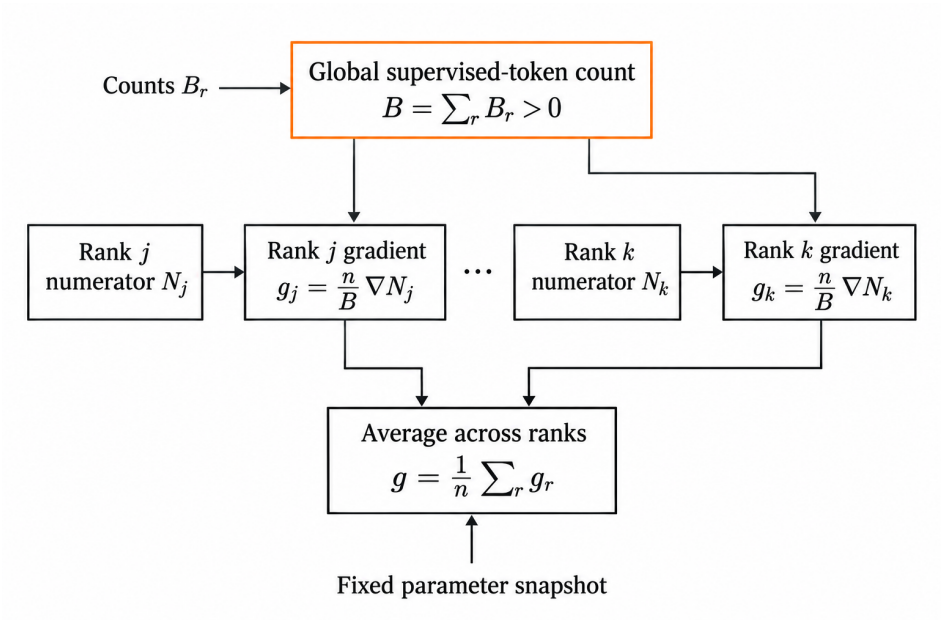

Figure 7.1: Token-weighted accumulation requires one global normalization. At a common parameter snapshot, rank j contributes token-loss numerator N_j and supervised-token count B_j. For $B = \sum_j B_j > 0$, scaling by n/B and an averaging collective yields $\nabla[(\sum_j N_j)/B]$. Counts are fixed batch metadata; the snapshot condition applies to every rank gradient. Locally normalized means generally define a different objective. No training measurement is depicted.

7.7 A worked training-budget calculation

Consider a hypothetical dense model with $N = 7\times10^9$ parameters trained on $D = 10^{11}$ tokens. Under the approximate $6ND$ convention, the arithmetic budget is 4.2×10^{21} FLOPs. Suppose 64 accelerators each sustain 10^{14} FLOPs per second continuously on the accounted work. The ideal duration is 656,250 seconds, or about 7.60 days. This is an arithmetic scenario with deliberately specified effective rates, not a measured training run. Data

loading, checkpointing, failures, utilization variation, attention corrections, and additional objectives can increase time or change the accounted FLOPs.

The example permits useful sensitivity analysis. Halving sustained compute doubles the ideal arithmetic time. Doubling the training-token budget doubles this approximation but may change the optimum model size under the scaling fit. Increasing context length at fixed token count can increase attention work even though ND is unchanged. Adding activation recomputation increases executed arithmetic without changing the nominal training-token count. A resource report should therefore retain both the analytic estimate and measured device or profiler evidence. Their discrepancy is a diagnostic, not a reason to silently redefine utilization after the run.

For an agent developer, training cost must be amortized over the intended use. A larger model can reduce retries or tool calls, while a smaller extensively trained model can reduce serving cost. The expected total cost is training plus the sum of inference, tools, storage, and operation costs over the deployment workload. The optimum depends on expected volume and quality constraints. A pretraining configuration cannot be selected from a public loss curve alone when its intended task requires expensive external actions and long-lived sessions. End-to-end cost should be compared at a specified success and risk level.

Table 7.2: Training comparisons require explicit accounting boundaries.

Claimed advantage	Necessary matched condition	Important confound
Better scaling	Data and compute convention	Tokenizer and data quality.
Lower memory	Precision and peak phase	Transient gathers and activations.
Faster training	Target loss and workload	Different kernels or sequence lengths.
Better agent behavior	Interface and task exposure	Trace leakage and benchmark tuning.
Reliable recovery	Complete state snapshot	Changed rank layout or data position.

7.8 Experiments linking infrastructure to behavior

A scientific training study should identify the intervention and preserve the other important determinants. To test data mixture, hold architecture, token budget, optimizer, and evaluation interface fixed while changing a documented mixture weight. To test parallelism, compare time to the same loss and downstream quality under matched numerical settings, while reporting any unavoidable batch or precision differences. To test long-context pretraining, include tasks requiring early constraints, distant evidence integration, and resumed execution rather than relying only on a nominal context-length setting.

Implementation ablations should have interpretable predictions. Enabling optimizer partitioning should reduce persistent-state residency according to the specified bytes-per-parameter model, while potentially increasing communication. Activation checkpointing should reduce retained activation memory and increase executed work. Packing should increase utilization without allowing cross-example visibility when independence is intended. These predictions can be checked with tensors and traces before a costly full run. A failed prediction indicates an accounting or implementation error that should be resolved before interpreting downstream benchmark changes.

The most useful negative results preserve their conditions. If a data addition lowers validation loss but worsens tool reliability, inspect whether the new examples changed serialization conventions or encouraged unsupported operations. If a distributed optimization speeds individual steps but worsens time to target quality, examine batch-size and optimizer effects. If resumption changes behavior, compare restored data position, random state, and parameter partitions before attributing the difference to an inherent instability of learning. Infrastructure and statistical learning are coupled, but their effects can still be separated through targeted measurements.

Pretraining supplies a reusable statistical substrate, not a complete agent operating contract. Its quality depends on the objective, evidence, optimization, and implementation all describing the same causal task. Distributed systems determine what training is feasible and whether its result can be reconstructed; data design determines which conditional behaviors are learned. The subsequent post-training stage can specialize this substrate for reasoning and tools, but cannot automatically repair evidence that was never recorded or an execution interface whose semantics remain undefined.

One final integration check should compare a restored checkpoint's logits with the saved run on a fixed, permission-cleared diagnostic batch before resumed optimization. This detects missing adapters, incompatible tokenizer artifacts, or incorrect parameter reconstruction before new updates obscure the difference. Agreement does not prove complete replay, but disagreement immediately falsifies it.

7.8.1 Predictive supervision inside policy optimization

The boundary between predictive training and interactive optimization is visible in PaW. Its current second version uses action-conditioned next-observation targets from policy rollouts, selects transitions by sampled action surprisal, and combines a confidence-clipped observation loss with the policy loss. Action and observation masks allow both terms in a causal forward pass; the deployed agent does not need an extra imagined-observation call [96]. This is auxiliary supervision during policy optimization, not evidence that ordinary corpus pretraining and environment interaction are interchangeable.

For a target-token probability p, the clipped auxiliary term is $(1-p)\mathbf{1}[p \leq \rho]$, with threshold $\rho \in [0, 1]$ and a detached indicator in the checked code. `compute_world_model_loss` in `verl/trainer/ppo/core_algos.py` shifts the token mask and keeps the full observation-mask denominator; `DataParallelPPOActor.update_policy` combines it with the policy loss [97]. Renormalizing only by surviving confidence-selected tokens would change this objective. One forward pass also does not imply zero extra training work: appended observation positions require activations and projections even when their eventual loss mask is sparse.

A first-order calculation exposes possible co-training conflict. Let $g_\pi = \nabla L_\pi$ and $g_w = \nabla L_w$ at the current parameters, treating selection and auxiliary weight $\lambda \geq 0$ as fixed. Under update $\Delta\theta = -\eta(g_\pi + \lambda g_w)$ and differentiable policy loss,

$$L_\pi(\theta + \Delta\theta) - L_\pi(\theta) = -\eta(\|g_\pi\|^2 + \lambda g_\pi^\top g_w) + o(\eta). \tag{7.6}$$

A sufficiently negative cross term can increase the policy loss despite decreasing the joint objective. This local deduction motivates measuring gradient interaction and task transfer rather than assuming that better observation prediction is always policy-improving. Its implication is conditional: a descent surrogate and verified environment return need not agree either.

DGS-WM makes selection policy-dependent in a different way. Section 3 scores candidate observation units by the likelihood they assign to the

recorded next policy response, retains units above a within-observation threshold, and applies detached confidence weighting to their predictive loss [98]. The full observation remains available to the policy; selection changes auxiliary targets. Response support is a correlational utility proxy. An already mistaken response can favor evidence consistent with that mistake, while facts needed only many steps later can receive weak support from the immediate response. Compare full-observation supervision, random units matched for target tokens, action-surprisal selection, and response-support selection under the same rollout data and complete compute budget.

The spectral composition study supplies a complementary intervention rather than another target-selection rule. It decomposes updates from a common pretrained base, projects policy-update matrices onto or away from left and right singular subspaces of world-model updates, and reevaluates behavior [99]. Preserving a subspace norm is not equivalent to preserving a function. A sound comparison includes random subspaces of equal rank and norm-matched perturbations, then measures both transition prediction and task return. These controls distinguish a consequential update direction from an attractive representation visualization, while retaining architecture and coordinate-system assumptions in every interpretation.

Chapter 8

Post-Training Objectives for Interactive and Tool-Using Policies

8.1 On policy distillation with precision matched rollout

The DeepSeek-V4 report's Section 5 separates specialist training from a later on-policy distillation merge, replacing its mixed-RL stage with that merge. It reports full-vocabulary logit distillation rather than only sampled token-level estimates and uses native quantized weights during rollout [90]. These current choices connect training objectives to serving numerics; they should not be summarized as a claim that one classical preference method describes the entire pipeline.

For a fixed prefix, teacher distribution q, and student distribution p_θ, reverse KL is $D(p_\theta \| q) = \sum_v p_\theta(v) \log[p_\theta(v)/q(v)]$. Its gradient is

$$\nabla_\theta D(p_\theta \| q) = \mathbb{E}_{v \sim p_\theta} \left[\log \frac{p_\theta(v)}{q(v)} \nabla_\theta \log p_\theta(v) \right], \tag{8.1}$$

assuming a finite vocabulary, differentiability, and $q(v) > 0$ wherever $p_\theta(v) > 0$. The constant score term vanishes because $\sum_v \nabla p_\theta(v) = 0$. Summing over the vocabulary evaluates this conditional expectation directly; a sampled estimator adds conditional sampling variance. Prefixes generated by the student remain random, so full-vocabulary evaluation does not make the entire training gradient deterministic. Sampling tokens from the teacher instead without correction estimates a different expectation.

The conditioning distribution also matters. Let a finite rollout horizon and a fixed, parameter-independent rule for selecting a prefix position define the normalized distribution $d_\theta(h)$ of its student-generated prefix h and let $K_\theta(h) = D(p_\theta(\cdot \mid h) \| q(\cdot \mid h))$. For $J(\theta) = \sum_h d_\theta(h) K_\theta(h)$, differentiating the finite sum gives

$$\nabla J = \mathbb{E}_{h \sim d_\theta} \left[\nabla K_\theta(h) + K_\theta(h) \nabla \log d_\theta(h) \right]. \tag{8.2}$$

Holding sampled prefixes fixed while differentiating the vocabulary loss estimates the first term. It is a conditional distillation update, rather than

an unbiased estimator of the displayed total derivative, unless the second term is included or vanishes under additional assumptions. This deduction identifies a reproducibility question; it does not assert an undocumented choice in the source implementation. Record whether rollout is detached, whether prefix score terms are used, and which objective the reported gradient is intended to optimize.

The memory cost can be substantial. In a hypothetical batch with eight sequences, 4096 positions, vocabulary size 128,000, and four-byte logits, one full logits tensor occupies 15.625 GiB. Holding both teacher and student tensors would occupy 31.25 GiB before activations or gradients. Chunked vocabulary reductions, fused loss kernels, and selective position processing can change the residency pattern. A study should identify which optimization is used and verify that it preserves the stated KL direction and normalization. Avoiding materialization is not the same as replacing the full objective with one sampled token.

Precision consistency creates a second falsifiable invariant. If rollout uses quantized weights and training recomputes probabilities with a different effective model, the recorded behavior distribution and optimized distribution can disagree before learning begins. Quantization-aware training can reduce that mismatch, but correctness still depends on scales, dequantization, kernel arithmetic, and sampling transforms. Compare logits and action probabilities under rollout and training paths on identical live contexts, stratified by sequence length and rare tool arguments. A small mean discrepancy does not rule out failures concentrated at an action boundary.

Distillation can also transfer the teacher's blind spots and erase useful diversity among specialists. Evaluate each domain separately before and after merging, including tool-use failures and memory conflicts, rather than accepting a higher aggregate score as evidence of uniform improvement. Keep a fixed external verifier and an untouched task set. An online student can visit prefixes absent from the original specialist data, which is one motivation for on-policy distillation, but teacher predictions there are not guaranteed correct. The appropriate evidence is verified downstream behavior and retained regressions, not the mere availability of dense teacher logits.

Preference-loss implementations also require an exact boundary between prompt and completion tokens. If the common prompt contributes to one candidate's summed log probability but not the other's, the optimized pair no longer represents the intended response comparison. Tokenization at the concatenation boundary can change the first completion token, so independently tokenizing prompt and answer is not always equivalent to tokenizing their combined serialization. Check these boundaries with actual tokenizer

artifacts, including whitespace and special messages. Compare summed conditional log probabilities against a direct teacher-forced reference on a small deterministic batch before interpreting any training gain.

Sequence probabilities and length normalization. The preference derivation developed below uses complete-response probabilities, hence sums of conditional token log probabilities. Replacing those sums with per-token averages changes the objective and generally loses the original reward reparameterization. A concrete hypothetical pair shows the difference. Suppose the preferred ten-token response has student probability 0.1 and reference probability 0.01, while a dispreferred two-token response has probabilities 0.2 and 0.1. The sequence log-ratio difference is $\log 10 - \log 2 = \log 5$, favoring the preferred response despite its lower absolute student probability. Dividing each log ratio by its response length gives $\log 10/10 - \log 2/2 < 0$, reversing the ordering.

These numbers are a calculated counterexample, not model measurements. They establish that length normalization is a substantive modeling choice. It can be useful for some objectives, but cannot be introduced as an innocuous numerical stabilization while retaining every theoretical claim of the original sequence objective. The same care applies to masking an end-of-sequence token or truncating long candidates. A completion that is cut off before its action or answer is not simply a shorter observation of the same preference event. The trainer should record truncation policy and the fraction of affected pairs.

An executable verification can construct a tiny vocabulary model with manually specified logits and enumerate every completion in a bounded response space. Compute the exact pair loss and gradient by direct probability products, then compare the trainer's token gathering, attention masks, reference scores, and reduction. Repeat with unequal response lengths, identical prefixes, padded examples, and an empty supervised suffix. This checks implementation against the chosen objective independently of whether a large training run happens to improve a benchmark. It is particularly valuable when the trainer supports multiple loss variants behind one public class name.

Tool trajectories introduce another reduction choice. A long episode can contain many observation tokens that are not policy actions. Including them in the policy ratio can assign optimization pressure to a distribution the agent does not control. Excluding them while accidentally retaining their padding or shifted positions can corrupt the action-token alignment. The trace should carry explicit action masks generated from the authoritative

serialization, not inferred later from role-like strings in untrusted tool text. Compare the masked sum with an independently assembled action list on a small trajectory containing multiple tools and failures.

Finally, report whether learning weights episodes, decisions, or action tokens. Equal episode weighting protects short tasks from being overwhelmed by long traces, while token weighting can emphasize detailed reasoning and extensive tool interactions. Neither choice is universally correct. The relevant question is which deployment objective the weighting approximates and whether the evaluation measures that objective. A gain on a benchmark dominated by short tasks can coexist with a regression on rare long sessions if the training reduction changes their relative influence. Retaining both stratified outcomes and the exact reduction makes that tradeoff scientifically interpretable.

8.2 Behavioral targets and trajectory supervision

Post-training changes a pretrained conditional model to support a more specific behavior distribution. For agents, the target includes selecting tools, constructing valid arguments, using observations, recognizing failure, and stopping under appropriate conditions. Supervised fine-tuning minimizes negative log likelihood on selected target actions or assistant tokens. Its statistical meaning is imitation of the recorded conditional distribution. A successful demonstration supplies one feasible behavior, not proof that every action in it is necessary or optimal. If a trace succeeds despite an unnecessary retry, direct imitation can preserve that inefficiency.

Let a trajectory contain contexts C_t and actions a_t. A sequence-level objective can weight each trajectory equally, while a token-level objective weights long outputs more heavily. These objectives differ when demonstrations have different reasoning lengths or numbers of tool calls. Report the weighting and loss mask, including whether tool observations contribute. A dataset that trains the model to predict authoritative-looking tool results can encourage simulation of results unless the output interface clearly distinguishes an action request from an observation. The model should not be rewarded for inventing the external evidence it is supposed to obtain.

Failure data require deliberate targets. A failed trajectory can teach recognition and correction without making its original erroneous actions desirable. One approach supervises only the corrective continuation given the failed prefix; another trains outcome or process evaluators. Simply concatenating failed and successful traces under the same imitation objective mixes incompatible behavioral targets. The data record should identify

which action was erroneous, what feedback was available, and whether the correction depended on later information. Retrospective reasoning can be useful as an explanation while still being invalid as the conditioning prefix for the original action.

Instruction following through human feedback provides an influential post-training baseline, but its preference judgments are specific to the task and annotator protocol [100]. An agent policy needs additional evaluation of external effects and long-run state. A response preferred by a human reader can still call the wrong tool or omit a necessary verification step. Consequently, preference quality and executable correctness should be recorded separately. The most convincing training signal combines task-relevant judgment with evidence from the actual environment, while preserving uncertainty when the specification itself is ambiguous.

8.2.1 KL-regularized control and preference optimization

Consider one context x and a response distribution π. With reference π_0, reward $r(x, y)$, and $\beta > 0$, maximize $\mathbb{E}_\pi r - \beta \,\mathrm{KL}(\pi \| \pi_0)$. Assuming the reference has support on feasible responses and the partition function is finite, a Lagrange-multiplier calculation gives

$$\pi^*(y \mid x) = \frac{\pi_0(y \mid x) e^{r(x,y)/\beta}}{Z(x)}. \tag{8.3}$$

Rearranging yields $r(x, y) = \beta \log[\pi^*(y \mid x)/\pi_0(y \mid x)] + \beta \log Z(x)$. Under a Bradley–Terry preference model, reward differences cancel the context-only partition term. This is the algebraic basis of direct preference optimization [37].

For a preferred completion y^+ and dispreferred completion y^-, define

$$d_\theta = \log \frac{\pi_\theta(y^+ \mid x)}{\pi_0(y^+ \mid x)} - \log \frac{\pi_\theta(y^- \mid x)}{\pi_0(y^- \mid x)}, \tag{8.4}$$

$$\mathcal{L}_{\mathrm{DPO}} = -\,\mathbb{E} \log \sigma(\beta d_\theta). \tag{8.5}$$

The per-example gradient is $-\beta\sigma(-\beta d_\theta)\nabla_\theta d_\theta$. Incorrectly ranked pairs receive stronger weight under this formulation. The derivation does not prove that observed preferences are generated by one stationary scalar reward, nor that the model class can realize the optimum. Cyclic preferences, label noise, and distribution shift remain outside the simple equivalence argument.

Agent preferences can compare trajectories with different external states, which complicates the meaning of a pair. A shorter trace may be preferable

only because it skipped an expensive but necessary check. A successful trace can violate a permission boundary that a failed compliant trace respected. A single binary preference can conflate task success, safety, cost, and presentation. The dataset should either encode the tradeoff deliberately or retain separate labels. Hard execution constraints should not be converted into a negotiable preference merely because the training method uses pairwise comparisons.

Support is another practical limit. If the reference assigns negligible probability to a useful action sequence, regularized optimization may struggle to discover it from static preference data. Offline pairs also provide little evidence about actions absent from the dataset. Online environment interaction can expose new failures, but introduces cost and safety requirements. DPO and reinforcement learning therefore optimize different data interfaces even when their idealized objectives are related. Comparing them requires matching not only model size but data exposure, online calls, reward access, and the opportunity to discover new trajectories.

8.3 Policy gradients and the likelihood-ratio contract

For a trajectory law $p_\theta(\tau)$ and a parameter-independent return $R(\tau)$, the score-function identity gives $\nabla_\theta J = \mathbb{E}[R(\tau)\nabla_\theta \log p_\theta(\tau)]$ under differentiation and integrability assumptions. Fixed environment-transition factors cancel from the score. All parameterized agent decisions remain, including memory writes, context edits, routing, or stopping when these are learned actions. Tool observations themselves are not policy-generated action tokens. A state-conditional baseline independent of the sampled action can reduce variance without changing the expected gradient; an action-dependent baseline requires correction.

For example, let a writer sample memory updates from $\omega_\psi(m_{t+1} \mid H_t, M_t)$ and let a separate actor use $\pi_\theta(a_t \mid C_t)$, with context a deterministic function of the realized history and memory. Assume ψ enters the trajectory law only through the writer. Then

$$\nabla_\psi J = \mathbb{E}\left[R(\tau) \sum_t \nabla_\psi \log \omega_\psi(m_{t+1} \mid H_t, M_t)\right]. \tag{8.6}$$

Training only the actor's action-token loss omits this writer gradient. It can still define a deliberate alternating or frozen-writer procedure, but is not joint optimization of both components. Shared parameters require adding every path through which they affect the sampling law; a differentiable

parameter-dependent reward adds its direct derivative. This decomposition gives a concrete audit target for harness-native training and the co-adaptation analysis in Chapter 17.

PPO uses a surrogate based on the ratio $r_t(\theta) = \pi_\theta(a_t \mid C_t)/\pi_{\text{old}}(a_t \mid C_t)$ and an advantage estimate $\widehat{A}_t$. Its clipped objective is

$$\mathbb{E}\min\{r_t\widehat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\widehat{A}_t\}. \tag{8.7}$$

The practical clipped method is not a universal monotonic-improvement guarantee. It relies on sampled data, estimated advantages, optimization choices, and an appropriate proximity regime [36]. In long-horizon agents, sparse rewards, variable action lengths, and tool delays make advantage estimation and sample accounting especially consequential.

The denominator must describe the behavior that generated the action under the same conditioning information. If a trace is later compacted and the numerator is evaluated on the final summary while the denominator comes from the live pre-compaction context, the ratio is not the intended conditional likelihood ratio. Likewise, a grammar mask or temperature used during sampling changes behavior probabilities. Storing base-model logits without the sampling transformation can produce an incorrect correction. The trace should retain the live context identity, sampled token probabilities after relevant transforms, and the exact action mask.

A trajectory-level importance ratio is the product of conditional ratios over its sampled actions. Its variance can grow severely with horizon. Token-level or per-decision approximations can reduce variance while changing the estimator and its assumptions. Clipping further introduces bias deliberately. These are not reasons to reject the methods; they are reasons to state which expectation is being estimated. An agent result should distinguish an exact identity, a biased practical estimator, and an empirical observation that the estimator worked in the reported setting.

8.4 Group-relative advantages and sparse outcomes

Group-relative policy optimization uses multiple responses for the same prompt to construct relative learning signals without a separate value model in its basic formulation. DeepSeekMath introduced an influential version of this approach [101]. If rewards are $R_1, \ldots, R_G$, a common normalized advantage is $(R_i - \bar{R})/(s_R + \delta)$, with $G \geq 2$, group mean $\bar{R}$, standard deviation s_R, and stabilizer $\delta > 0$. The normalization compares candidates within a prompt rather than estimating an absolute value function across

all states. Current implementations contain variants whose exact loss and normalization must be identified rather than collapsed into one method name.

There is a useful bias calculation even before standard-deviation normalization. Suppose G responses are independent given the prompt and $g_i = \nabla \log \pi(a_i \mid x)$. Then $\mathbb{E}[g_i] = 0$. With a group mean including the same sample,

$$\mathbb{E}[(R_i - \bar{R})g_i] = \left(1 - \frac{1}{G}\right) \mathbb{E}[R_i g_i]. \tag{8.8}$$

All other samples' reward terms vanish by independence; the self term removes $1/G$ of the signal. A leave-one-out baseline restores the unscaled identity under these assumptions. Dividing by a random group standard deviation creates additional dependence and generally changes the estimator again. This deduction explains why group size and normalization are algorithmic choices, not harmless reporting details.

Sparse binary rewards can create groups with no contrast. If every candidate fails, centered rewards vanish; if every candidate passes a weak validator, they also vanish. Increasing the number of candidates can help discover contrast while increasing generation cost. A reward that grades formatting more densely can improve syntax but dominate task correctness if its scale is poorly chosen. Record the fraction of zero-variance groups, reward components, response lengths, and successful trajectories. An average reward curve can hide that most prompts supply no useful comparative signal.

For agents, candidates can interact with a shared environment or memory unless isolated deliberately. Then conditional independence fails, and one candidate can change the state encountered by another. Resetting only the prompt does not reset external files or persistent memory. Evaluation and training environments should specify snapshot, reset, and tenant-isolation semantics. If a method intentionally shares discovered information across candidates, it should be analyzed as coordinated exploration with a different data distribution, not as independent response sampling.

8.5 Executable rewards and verifier error

Executable rewards connect training to external outcomes, but tests are partial specifications. A patch can pass a visible test suite while violating an untested requirement. A browser action can trigger a success banner without satisfying the user's actual objective. A reward model can learn superficial signs of success. Training pressure actively seeks these gaps. The

correct question is not whether the reward is automated, but what event it measures, which failures it misses, and whether the policy can manipulate its inputs or state.

Suppose the true success indicator is Y and a validator reports V. Its sensitivity and false-positive rate can be measured on an independently adjudicated sample. The observed pass rate is $P(V = 1) = P(V = 1 \mid Y = 1)P(Y = 1) + P(V = 1 \mid Y = 0)P(Y = 0)$. Without the conditional error rates or strong assumptions, validator pass rate does not identify true success. Moreover, those rates can change after optimization as the policy discovers adversarial cases. A validator calibrated on an early checkpoint should be re-audited on later policy outputs.

Table 8.1: Post-training signals and their inference boundaries.

Signal	What it directly supervises	What remains unproven
Demonstration likelihood	Imitation of recorded actions	Optimality and recovery off-distribution.
Pairwise preference	Relative judged desirability	Hard constraint satisfaction.
Executable test	Behavior covered by that test	Unspecified or untested requirements.
Outcome reward	Measured terminal criterion	Correct intermediate state and permissions.
Process judgment	Selected intermediate properties	Completeness and judge robustness.

Separate the reward service from the policy's writable environment. A policy should not be able to alter hidden tests, grader configuration, or the event records used as ground truth. Retain exact environment snapshots and reward versions. When a task permits editing tests as part of legitimate work, the acceptance checker needs an independent source of requirements. Otherwise the policy can improve its measured outcome by weakening the test. The same concern applies to memory: writing a claim of success into a store that the grader later reads creates a feedback channel that can bypass actual execution.

8.6 Context edits and post-training consistency

MemoryWalker identifies two distinct errors when training on edited rollouts: conditioning earlier actions on later compressed information, and

reconstructing a full context different from the live context the agent actually saw. Its exact tree-consistent methods and approximate distillation route address different implementation regimes [23]. The general lesson is a conditioning invariant. For every supervised or policy-gradient action, the training distribution must represent the information and visibility under which that action was generated, unless the method explicitly defines a counterfactual objective and corrects for the difference.

Consider an agent that queries a tool, observes a fact, and later writes a summary. Training its earlier query on the final summary leaks the tool result backward in time. Training a later answer on a reconstructed full history can instead expose information the live agent had already evicted. These errors have opposite structures but share the same root: a linear stored transcript is not always the computational history of an editing agent. Branch identity, masks, summaries, and action-time context versions are necessary to reconstruct the actual conditional distribution.

An implementation can materialize separate branches or use an attention mask that expresses their visibility. The latter is not merely a convenient packed sequence; its mask must reproduce the branch computation exactly. Specialized masks can conflict with optimized attention kernels, so correctness and performance must be measured together. An approximate consistency loss can be valuable when exact materialization is expensive, but residual divergence should be reported rather than described as equivalence. A lower training loss under a leaked context is not evidence that the deployable policy learned a better decision rule.

The corresponding audit is concrete. For a fixed checkpoint and retained rollout, recompute action logits under the captured live contexts. Compare them with the training pipeline's logits before any parameter update. Disagreement can reveal serialization, masking, position, or context-reconstruction errors independently of learning. Then repeat after enabling compaction and branch packing. This zero-update consistency check is particularly useful because it can falsify the data pipeline without requiring a large and noisy downstream training experiment.

8.7 Implementation paths and controlled training studies

The inspected TRL snapshot contains `DPOTrainer` in `trl/trainer/dpo_trainer.py`, with reference-log-probability preparation and loss computation, and `GRPOTrainer` in `trl/trainer/grpo_trainer.py`, with reward collection and per-token log-probability routines [102]. These code locations

identify a concrete implementation surface; they do not establish that every option matches the original papers. Record the selected loss type, reward normalization, reference policy, sampling backend, masking, clipping, and synchronization behavior. A repository version without configuration is not a reproducible algorithm.

The reductions are visible in `GRPOTrainer._compute_loss`. For a local batch of B completions with binary action mask m_{bt}, per-token surrogate contribution ℓ_{bt}, positive counts $n_b = \sum_t m_{bt}$, and configured maximum length $T_{\max}$, three inspected branches reduce as

$$\begin{aligned} L_{\text{grpo}} &= \frac{1}{B} \sum_b \frac{\sum_t m_{bt} \ell_{bt}}{n_b}, \\ L_{\text{bnpo}} &= \frac{\sum_{b,t} m_{bt} \ell_{bt}}{\sum_b n_b}, \\ L_{\text{dr}} &= \frac{\sum_{b,t} m_{bt} \ell_{bt}}{B T_{\max}}. \end{aligned} \tag{8.9}$$

These equations describe the positive-count case before the branch's accumulation scaling; the code clamps empty denominators and other loss types use different window-level normalization [102]. They demonstrate why the class name does not identify the objective. Varying completion length changes relative weights even with the same sampled rewards and token ratios.

For preference learning, `DPOTrainer.compute_ref_log_probs` and `_compute_loss` both construct completion masks and distinguish cached reference scores from a live reference forward. In the standard summed-log-probability branch, changing to a length-normalized variant is an explicit loss change. Cached reference scores should therefore be bound to the reference or adapter identity, tokenizer, serialization and exact completion mask; the cache's existence alone does not validate their compatibility with a later batch. This is a proposed integrity check on an inspected dataflow, not an executed trainer comparison.

Let $z_k = (\theta_k, \omega_k)$ be the policy and optimizer state at update k, and fix reference parameters $\bar{\theta} = \theta_0$. The manifest ν_k records the environment, context construction, sampling, and execution versions. For positive integer

batch size b_k and batch $X_k = (x_{ki})_{i=1}^{b_k}$, isolated trajectories satisfy

$$\begin{aligned} \bar{\theta}_k &= \bar{\theta}, \\ \boldsymbol{\tau}_k \mid X_k, \nu_k &\sim \bigotimes_{i=1}^{b_k} P_{\theta_k,\nu_k}(\,\cdot \mid x_{ki}), \\ r_{ki} &= V_\varphi(\mathcal{O}(\tau_{ki}), x_{ki})\,, \\ \boldsymbol{A}_k &= \mathcal{G}(\boldsymbol{r}_k, \boldsymbol{\tau}_k). \end{aligned} \tag{8.10}$$

Here $\mathcal{O}$ extracts recorded environment outcomes, V_φ is the separately specified outcome scorer, and $\mathcal{G}$ is the declared advantage estimator. The product law assumes independent initialization and randomization of isolated rollout environments; shared mutable state instead requires a joint trajectory law. Independence from the actor's update does not make the scorer an error-free oracle.

For trajectory i at trained model decision $t \in \mathcal{I}(\tau_{ki})$, let c^{b}_{kit} be the actual behavior input and $\widetilde{c}_{kit}$ the input used to recompute its probability during training. Let ν^{b}_{kit} and $\widetilde{\nu}_{kit}$ record the corresponding serialization and forward-computation contracts, apart from explicitly designated behavior and target parameter values. The admissibility indicator is

$$I_k = \mathbf{1}\begin{Bmatrix} \widetilde{c}_{kit} = c^{\mathrm{b}}_{kit}, & \\ \widetilde{\nu}_{kit} = \nu^{\mathrm{b}}_{kit} & \forall i,\ t \in \mathcal{I}(\tau_{ki}) \end{Bmatrix}. \tag{8.11}$$

Intentional off-policy updates need the declared behavior-probability correction rather than an assertion that policy versions are equal. The equality above concerns the inputs and forward semantics used to evaluate the required probabilities. It does not replace the earlier zero-update logit test.

On an admissible batch, a chosen policy-loss functional $\mathcal{L}$ and optimizer $\mathcal{U}_k$ give

$$\begin{aligned} g_k &= \nabla_\theta \mathcal{L}(\theta; \boldsymbol{\tau}_k, \boldsymbol{A}_k, \bar{\theta})\Big|_{\theta=\theta_k}, \\ z_{k+1} &= \mathcal{U}_k(z_k, g_k), \qquad I_k = 1, \\ \boldsymbol{e}_{k+1} &= \mathcal{E}(\theta_{k+1}; D_{\mathrm{a}}, \nu_{\mathrm{a}}). \end{aligned} \tag{8.12}$$

The sampled trajectories, rewards, and estimated advantages are held fixed in this derivative; it is the gradient of the chosen empirical surrogate, not an assertion about an unbiased total derivative through trajectory sampling. Normalization, clipping, and synchronization are part of the specified loss and optimizer. When $I_k = 0$, this update is not admissible until the conditioning discrepancy is resolved or a justified alternative estimator is supplied. The evaluator $\mathcal{E}$ uses a separate audit population D_{a} and manifest

ν_{a}. Its outputs are report-only for this protocol; feeding them into training or hyperparameter selection converts the audit into selection feedback and requires a separate confirmatory evaluation. These equations organize the implementation contract rather than identify a universal post-training objective.

A fair study includes supervised fine-tuning, an offline preference method, and online reinforcement learning where feasible, with matched initial checkpoints and clearly separated data budgets. Report both environment interactions and generated tokens. An online method's access to additional trajectories is an important advantage, not an incidental detail. Include a no-learning inference baseline that spends comparable compute on independent sampling and verification. Otherwise training gains can be confused with the benefit of simply allocating more attempts at evaluation time.

Table 8.2: Mechanistic ablations for agent post-training.

Ablation	Preserved controls	Diagnostic measurement
No outcome verifier	Same generated trajectories	Reward exploitation and false acceptance.
No context editing	Same policy and tasks	Conditioning mismatch contribution.
Fixed versus adaptive memory	Same storage budget	Credit assignment to memory actions.
Group size change	Same total generation budget	Contrast and estimator variance.
Reference strength change	Same preference data	Drift, support, and capability retention.

8.7.1 Harness-native reinforcement learning and segmented behavior policies

OpenForgeRL's current third version captures the deployment harness's actual model requests through an inference proxy and executes rollouts in remote containerized environments. Its Section 3 redistributes terminal reward to captured request-response samples, uses wall-clock timeouts across harnesses with incompatible turn definitions, and discards trajectories ending in infrastructure errors or timeout [103]. The intervention preserves the deployed context constructor and control flow during training rather than substituting a simpler chat loop. It does not by itself guarantee correct

token alignment or unbiased treatment of discarded episodes.

Molt's third version supplies complementary contracts. Sections 4.2–4.5 record sampled token identifiers, action ranges, behavior probabilities, and multimodal inputs; a rewritten conversation prefix starts a new segment while preserving the rollout identity. Partial generation across weight refits requires probability correction, and expert-routing replay is distinct from freezing an MoE router [104]. These are separate consistency conditions: using the same harness preserves the interaction procedure; exact token records preserve sampled actions; matching forward semantics preserves what their probabilities mean.

Let c_i be the actual rendered context for trained action token x_i, and let $\mu_i(x_i \mid c_i)$ be its recorded behavior probability, potentially from a different weight version at each token. Under identical fixed harness and environment kernels, and absolute continuity of the target policy relative to behavior, the full trajectory likelihood ratio is

$$W(\tau) = \prod_{i \in \mathcal{I}(\tau)} \frac{p_\theta(x_i \mid c_i)}{\mu_i(x_i \mid c_i)}, \tag{8.13}$$

where $\mathcal{I}(\tau)$ indexes all relevant policy-generated tokens and the nonpolicy transition factors cancel. This identity does not imply that the product is a practical low-variance estimator. Per-token clipping or rejection chooses a different bias–variance tradeoff. Reconstructing c_i from a final transcript after compaction instead changes the conditional probabilities themselves and cannot be repaired by a ratio computed on that altered history.

Discarding interrupted rollouts also changes the measured population. For bounded episode return R and retention indicator D, with $\Pr(D = 1) > 0$,

$$\mathbb{E}[R \mid D = 1] - \mathbb{E}[R] = \frac{\operatorname{Cov}(R, D)}{\Pr(D = 1)}. \tag{8.14}$$

Even a policy-independent network fault can correlate with task duration, and duration can correlate with difficulty. Therefore absence of a direct reward-based filter does not establish zero selection bias. Compare the native harness with a simplified training harness while holding weights, tasks, tools, sampling, and compute fixed; separately audit retained versus interrupted tasks by duration and domain. Keep tool observations out of action loss, verify segmentation after an actual prefix rewrite, and compare zero-update recomputed probabilities with the recorded behavior path before interpreting learning gains.

Figure 8.1 isolates the mechanism and its stated information boundaries.

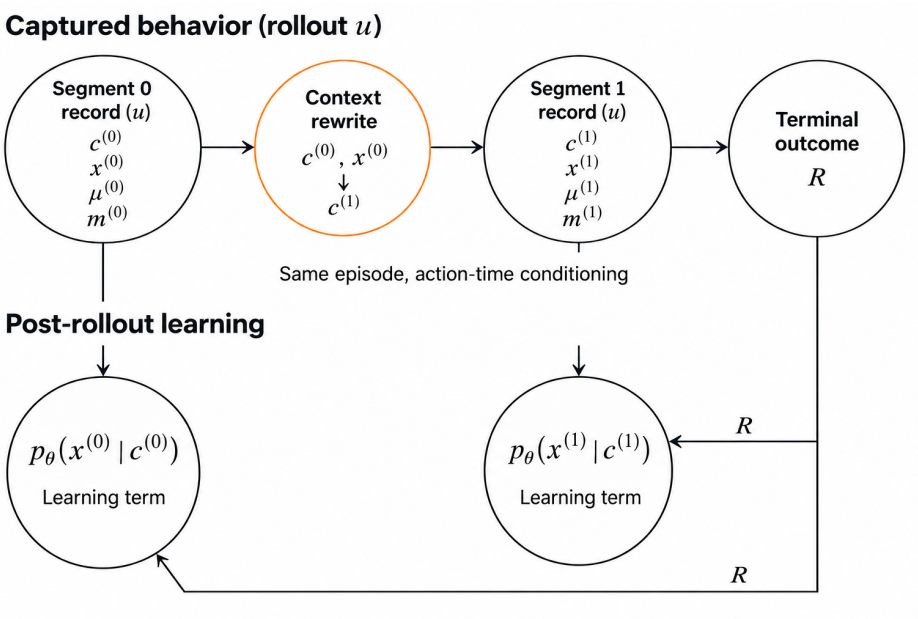

Figure 8.1: A single rollout u can contain multiple action-time contexts. Each segment retains context $c^{(i)}$, sampled tokens $x^{(i)}$, behavior probabilities $\mu^{(i)}$ and learning mask $m^{(i)}$. Terminal outcome R may weight later learning terms; it is not an additional conditioning argument of the displayed action likelihood. The lower circles denote learning computations, not behavior calls. This schematic omits the full post-training objective and correction factors.

8.8 Credit assignment across tools and memory

A terminal reward must assign credit to decisions separated by long tool delays and many intermediate tokens. Treating every token in a successful trajectory as equally good can reinforce redundant explanations and accidental workarounds. Tool selection, argument construction, context editing, and memory writing have different causal roles. A useful ablation removes or replaces one component while preserving resources and observes the outcome. Such interventions can inform training signals, but hindsight attribution remains uncertain when several actions jointly enable success.

Memory writing is particularly difficult because its value may appear only in a later session. Immediate task reward can favor a compact note that helps the current answer while discarding a rare future constraint. Conversely, rewarding every retrieved use can encourage excessive storage or irrelevant retrieval. The learning objective should declare the horizon and

cost of retention, retrieval, verification, and conflict handling. Evaluation should randomize task order and distinguish within-session adaptation from cross-session transfer. Otherwise improvements can depend on a favorable curriculum that deployment will not reproduce.

Model updates also change the policy that interprets existing memories and skills. A procedure distilled for one checkpoint may become redundant, misleading, or incompatible after post-training. Evaluate retained skill banks before and after a weight update, including stale assumptions and changed tool schemas. Updating memory and model weights together without separate ablations makes it difficult to identify where a gain or regression originated. Compatibility tests should therefore be part of the learning loop, with rollback capable of restoring the relevant combination of policy and external state.

Post-training can improve reasoning and tool use substantially, but the claim must remain attached to a measurable behavior and evidence source. Preference agreement is not universal alignment, a higher automated reward is not verified task success, and a finite sequence of improving checkpoints is not accelerating recursive self-improvement. The scientific standard is stronger: correct conditioning, identified estimators, independent acceptance criteria, resource-matched alternatives, and retained regressions. These requirements make the resulting policy improvement interpretable and useful for a stateful agent rather than merely attractive on its training dashboard.

Chapter 9

Inference-Time Search, Verification, and Metareasoning

9.1 Meta reasoning controllers and artifact selection in 2026

The September 2026 meta-reasoning study separates worker computation from a controller that assesses artifacts, proposes work, evaluates options, and dispatches context from persistent memory. Its analysis distinguishes whether a correct answer appeared, whether the controller recognized it, and whether it selected it. It reports gains alongside overhead that can hurt small-budget settings [105]. The paper's Appendix A explicitly limits component attribution and resource matching. Its shared budget allowance is not proof that every system consumed equal tokens, latency, or dollars. This is a current example of treating inference control as a mechanism rather than equating a larger reasoning transcript with better allocation.

An artifact graph supplies an observable account of the controller's allocation. Nodes represent partial solutions, evidence, checks, and selected outputs; edges record derivation or reuse. Repeated restarts and refinement of verified partial work can have different cost and failure behavior at equal accuracy. Distinguish a copied assertion from independently obtained evidence, and record whether a computation changed the final choice. A verifier invocation that merely confirms the already selected artifact has a different causal role from one that rejects the incumbent and causes a successful replacement. Log both the selection before the check and the selection after it; a final graph without these decision-time snapshots cannot identify that distinction.

An appropriate ablation keeps workers, tools, task instances, and total actual expenditure comparable while varying the control policy. Compare direct control, independent sampling, structured reuse without explicit evaluation, and the full controller. Separately remove persistent artifact retrieval to test whether compact controller context depends on external memory. The resulting experiment can identify which interaction matters; the combined method's headline score cannot assign causal credit to every

named stage. The accessible paper specifies the algorithm and prompts; an executable reproduction additionally requires independently established, versioned implementation artifacts.

For stateful deployment, the controller's compact state is itself a memory policy. Its summaries can omit constraints or misstate artifact validity. A worker should receive the evidence needed for its assigned subproblem, and its output should preserve unresolved assumptions. Version changes in workers or tools can invalidate earlier artifacts. The inference controller therefore needs the same compatibility and provenance contracts as other long-running agent components. A successful short benchmark run does not establish indefinite artifact validity or reliable cross-session reuse.

9.2 A conditional computation rather than a verbal certificate

Inference-time reasoning changes the computation used to choose an answer or action while leaving some or all model weights fixed. A model can produce intermediate text, sample alternatives, invoke tools, or delegate subproblems. These mechanisms have different information and resource effects. Intermediate text extends the model's conditional state; an external tool can supply new evidence; independent sampling explores alternative continuations; a verifier changes which continuation is selected. Calling all of them thinking obscures the intervention. Their contribution should be identified through the distribution they change and the evidence they add.

Let z denote an intermediate reasoning sequence and a the final action. A generative model defines $p_\theta(z, a \mid C)$, and its induced action distribution marginalizes over z. Choosing one greedy sequence, sampling a sequence, voting across answers, and searching a tree produce different decision rules from the same underlying model. They need not rank actions identically. A system that reports only the final action without the generation and selection procedure does not fully specify its inference policy. The selected reasoning trace also need not characterize all alternative computations the model could have produced.

Chain-of-thought prompting supplies examples or instructions that elicit intermediate language before an answer, with reported improvements in the original study's evaluated reasoning tasks [106]. This is an empirical prompting result, not a guarantee that every generated rationale is a faithful account of internal computation. A rationale can contain a correct derivation, a plausible explanation of a guessed answer, or an error later corrected. For an agent, the distinction becomes consequential when a trace

is stored as procedural memory. A fluent but invalid derivation should not become trusted guidance merely because the final answer happened to be correct.

The strongest reasoning evidence is an independently checkable object attached to a precise claim: an executable calculation, a proof accepted by a specified checker, a test demonstrating a behavior, or a trace of actual tool observations. Even these objects have scope limits. A proof checker validates a formal statement under its axioms; a test covers selected inputs; a calculator evaluates the supplied expression rather than determining whether it models the user's problem. Reasoning quality therefore includes translating the task into the right formal object and interpreting its result, not only generating a long sequence of intermediate tokens.

9.3 Sampling coverage and repeated reliability

Suppose independent attempts on one fixed task succeed with probability p. The probability of at least one success among k attempts is $1-(1-p)^k$, while the probability that all k succeed is p^k. These quantities answer different questions. With hypothetical $p = 0.8$ and $k = 3$, they are 0.992 and 0.512. The first describes solution coverage when an oracle can identify success; the second describes repeated-run consistency. Neither is the success probability of a practical selector unless the selector's behavior is specified. Reporting pass-at-k as dependable autonomous performance confuses these events.

Given n recorded attempts with c successes on a task, the conditional coverage of a uniformly chosen subset of $k \le n$ attempts is exactly $1-\binom{n-c}{k}/\binom{n}{k}$, with the numerator zero when fewer than k failures exist. Under independent identically distributed sampling, this statistic is also an unbiased estimator of population pass-at-k [107]. The first statement is combinatorial; the second additionally depends on the sampling model. It estimates a sampling property of the candidate set; it does not account for the cost of finding and verifying those candidates. Tasks with different p values should be aggregated at the task level rather than inserting a global average p into the formula, because the nonlinear transformation changes the result.

Independence is often unrealistic. Samples share the same model, prompt, retrieved evidence, and training-induced misconceptions. Let correctness indicators have common mean $0 < p < 1$, variance $p(1-p)$, and pairwise correlation ρ. Then

$$\operatorname{Var}\left(\frac{1}{k}\sum_i Y_i\right) = \frac{p(1-p)}{k}[1+(k-1)\rho]. \tag{9.1}$$

This follows by summing variances and covariances. For an admissible correlation with $1+(k-1)\rho > 0$, the corresponding variance-based effective sample size is $k/[1 + (k-1)\rho]$. At zero denominator the sample mean has zero variance under this model, rather than a defined finite effective count. It is a diagnostic under the stated exchangeability assumptions, not a universal replacement for the full joint distribution. A hundred correlated attempts can carry much less independent evidence than their count suggests.

Table 9.1: Calculated sampling quantities under explicit hypothetical assumptions.

Assumption	Calculation	Meaning
$p = 0.8$, three independent trials	$1 - 0.2^3 = 0.992$	At least one successful candidate.
Same trials	$0.8^3 = 0.512$	Every trial succeeds.
$k = 20$, pair correlation 0.1	$20/2.9 \approx 6.90$	Variance-based effective count.
$k = 100$, pair correlation 0.1	$100/10.9 \approx 9.17$	Correlation limits averaging benefit.

9.4 Self-consistency and answer aggregation

Self-consistency samples reasoning paths and aggregates their final answers, rather than trusting one path [108]. It approximates the answer mass induced by the model and decoding procedure. The modal answer can still be wrong. Under independent binary correctness with $p > 1/2$, a majority vote has an error probability bounded by $\exp[-2k(p - 1/2)^2]$ through Hoeffding's inequality, with tie handling specified separately. If the model's dominant misconception makes $p < 1/2$, increasing the sample count can make the wrong majority more stable. The bound's premises are stronger than merely using different random seeds.

Answer equivalence is itself a component. Exact string voting can split mathematically equivalent expressions or merge ambiguous natural-language answers incorrectly. A canonicalizer may simplify arithmetic expressions, normalize units, or compare structured actions. That canonicalizer needs a correctness contract. For tool use, two actions that have the same apparent intent can differ in permissions, resource identity, or irreversible side effects. Voting on a prose description before argument validation can select an action family whose actual executable instances

are all invalid. Aggregation should operate on a representation appropriate to the task's semantics.

Diversity interventions include changing prompts, sampling parameters, decompositions, tools, or model checkpoints. Each can reduce shared errors while introducing new confounds and costs. A heterogeneous team is not automatically independent, particularly when all members read the same flawed summary. A useful study measures error overlap and the selector's performance rather than relying on role names as evidence of diversity. When candidates have access to different evidence, retain that provenance so that disagreement can motivate another observation rather than being settled by a numerical majority alone.

In stateful tasks, candidates also need isolation. If the first attempt modifies a repository, the second is not a repeated trial from the same initial state unless the environment is restored. Shared persistent memory can make later attempts better informed, but the resulting process is adaptation rather than independent sampling. Both are legitimate experimental objects. They should be reported separately because their reliability, resource use, and deployment assumptions differ. A best-of-five outcome with accumulated memory does not estimate the same quantity as five fresh attempts.

9.5 Selection error grows with the search space

More candidates increase coverage but also expose the selector to more opportunities for error. Let candidate i have true utility u_i and estimated score $\widehat{u}_i = u_i + e_i$. Suppose each error has a sub-Gaussian tail $P(|e_i| > t) \leq 2\exp[-t^2/(2\sigma^2)]$. A union bound gives simultaneous error at most $t = \sigma\sqrt{2\log(2K/\delta)}$ with probability at least $1 - \delta$ across K candidates. Independence is not needed for this union bound, though the marginal tail assumption must hold for the evaluated candidates.

If $\widehat{i}$ maximizes estimated score and i^* maximizes true utility, simultaneous error control yields $u_{i^*} - u_{\widehat{i}} \leq 2t$. Add and subtract the two estimated scores; their difference is nonpositive by selection, leaving two bounded errors. This elementary deduction formalizes a selection effect: the score accuracy required for a fixed regret tolerance becomes stricter as the candidate set grows. A method can discover better candidates while selecting a worse final answer. The correct diagnosis separates coverage, recognition, and final selection rather than treating all failure as insufficient generation quality. For a selector restricted to the generated candidate set, let C be the event that the set contains a correct artifact and A the event that the

selected artifact is correct. Then $A \subseteq C$ and

$$\Pr(A) = \Pr(C)\Pr(A \mid C) \leq \Pr(C), \qquad \Pr(C) > 0. \tag{9.2}$$

This separates generator coverage from conditional selection success. A selector allowed to create a new answer changes the candidate set and must be evaluated under that expanded boundary; the inequality cannot be transferred silently to a different pipeline.

The assumptions deserve scrutiny in adaptive search. Candidates are proposed in response to earlier scores, so an error bound established on random unselected examples may not transfer. A reward model can be exploited by repeatedly searching for high-scoring artifacts. Calibration on ordinary completions is therefore insufficient evidence of robustness under optimization. Evaluate the judge on planner-selected candidates, preserve hidden tests, and record whether a correct candidate appeared but was rejected. These records explain whether additional compute should be spent on generation, verification, or a different scoring objective.

Binary validators have a related multiple-testing problem. If each invalid candidate has conditional false-acceptance probability at most ϵ given the prior search history, the probability of at least one false acceptance among at most K checks is at most $K\epsilon$. This conservative bound does not require independent checks. It does require the conditional error guarantee, which adversarially selected candidates can violate. A validator with a small average false-positive rate can become unreliable when an optimizer targets its blind spots. Increasing the sample budget without strengthening verification can therefore worsen accepted-output quality.

9.5.1 A calculated selector with an imperfect validator

Assume independent candidates are correct with probability p, and a validator accepts correct candidates with probability s and incorrect candidates with probability f. Its acceptance probability is $q = ps + (1-p)f$. Conditional on acceptance, correctness is ps/q when $q > 0$. If the system returns the first accepted candidate, unbounded retries have the same accepted-output correctness and mean attempt count $1/q$. More attempts increase the chance of returning something; they do not improve the precision of this stationary acceptance rule. This deduction separates availability from correctness and prevents a retry loop from being described as verification improvement.

For hypothetical $p = 0.3$, $s = 0.9$, and $f = 0.1$, acceptance is 0.34, accepted-output correctness is approximately 0.7941, and the mean number of unbounded attempts is approximately 2.9412. With four attempts, the

chance of any acceptance is $1 - 0.66^4 \approx 0.8103$. These are calculated examples, not measured characteristics of a model judge. They assume the validator's errors and the candidate distribution remain stationary, with fresh independent trials. Real adaptive search often violates precisely these assumptions because later candidates are chosen to exploit previous scores.

Table 9.2: Calculated behavior of the hypothetical imperfect selector.

Quantity	Value	Boundary
Per-attempt acceptance	0.34	Includes false acceptance.
Correctness given acceptance	0.7941	Does not improve with stationary retries.
Mean unbounded attempts	2.9412	No deadline or changing environment.
Acceptance within four attempts	0.8103	Availability, not verified success.

The example also identifies which intervention matters. Reducing the false-positive rate can improve accepted-output correctness substantially even if candidate generation does not improve. Increasing sensitivity can reduce wasted rejection of good candidates. Improving p helps both coverage and precision, but may cost a larger model or more computation per attempt. A scientific comparison should measure all three rather than crediting the generator for a better validator or describing a more permissive validator as improved reasoning. When true labels are expensive, audit a stratified sample of accepted and rejected candidates, with sampling weights that permit reconstruction of the desired rates.

Optional stopping requires care in evaluation. If the system repeatedly checks until a favorable score appears, reporting the final score without the number of checks introduces selection bias. The final artifact should be judged by a held-out acceptance procedure that was not used to guide the search. This does not require that every verifier be secret forever; it requires separation between optimization feedback and the evidence used to estimate generalization. For executable tasks, hidden tests and independent specification review can serve complementary roles. Neither resolves an ambiguous task specification automatically.

Stateful tasks make the stationary example especially fragile. A rejected candidate may have already changed files or consumed a one-time token. A repeated browser action can alter the environment even when the judge says it failed. The selector should either operate on isolated candidate

artifacts before execution or explicitly model these transitions. Retaining the complete operation history makes it possible to distinguish improved solution search from repeated interaction with a gradually changing task. This distinction is essential when comparing self-correction, sampling, and planning under the same nominal attempt budget.

Figure 9.1 isolates the mechanism and its stated information boundaries.

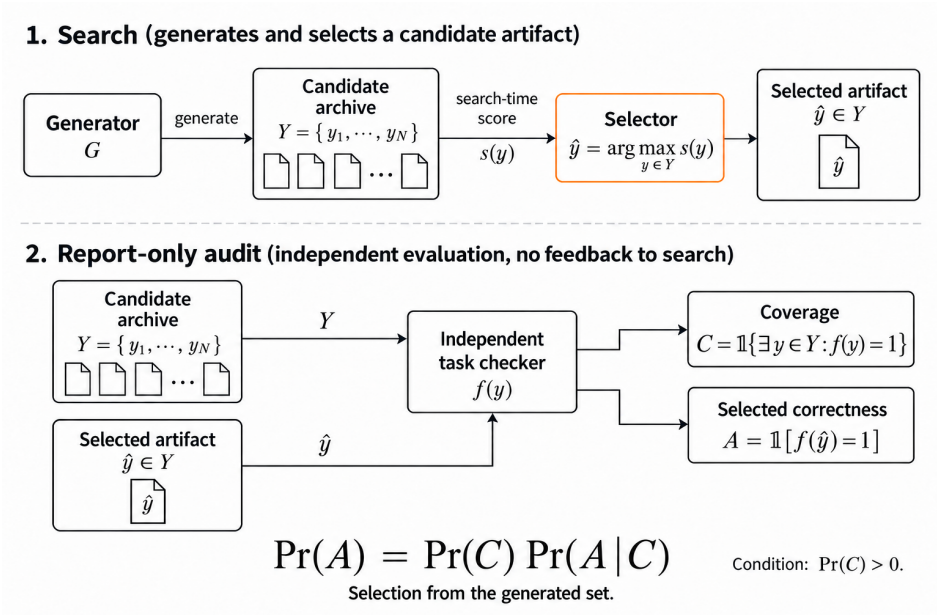

Figure 9.1: Search-time scores and report-only acceptance labels serve different roles. Under one fixed acceptance map f, the displayed indicators satisfy $A \leq C$ when $\widehat{y} \in \mathcal{Y}$. The probability notation abbreviates $\Pr(A = 1)$ and $\Pr(C = 1)$; the decomposition requires $\Pr(C = 1) > 0$. A fallible checker measures its declared acceptance criterion. Pruning attribution additionally requires stage-labelled candidate records; the selected artifact alone is insufficient.

9.6 Structured search and the granularity of a thought

Tree of Thoughts treats intermediate text units as search states and uses generation, evaluation, and traversal rules to explore alternatives. Its Game of 24 experiment reports 74 percent success for the evaluated method versus

4 percent for its chain-of-thought comparator [109]. Those author-reported results establish a task-specific system comparison with additional search; they do not isolate a universal architectural capability or equalize every inference cost. The inspected repository's `src/tot/methods/bfs.py` exposes `get_proposals`, `get_values`, `get_votes`, and `solve` [110]. This makes proposal, evaluation, and search policy identifiable implementation components. In this snapshot, `get_value` caches evaluations by the rendered value prompt, while `get_values` assigns zero to repeated candidate strings within one evaluation call. These are not independent repeated checks. The sampled-selection branch in `solve` normalizes scores by their sum, so its mathematical domain requires nonnegative weights with positive total; an all-zero set needs an explicit fallback in an integration. Record candidate entries, unique artifacts, fresh evaluator calls and cache hits separately. Otherwise repeated or cached work can be mistaken for additional search evidence.

Granularity determines branching and verifiability. Token-level search has a large shallow action space and weak semantic boundaries. A complete-program proposal permits executable testing but can waste work when one early assumption is wrong. Intermediate structured states can expose constraints, but the state representation must preserve all information needed to continue. Merging two reasoning nodes because their summaries look similar can destroy an important distinction. Search-state equivalence needs a task-specific invariant, just as cache reuse requires a computational invariant in Chapter 4.

With branching factor b and depth d, an unpruned tree has $(b^{d+1} - 1)/(b - 1)$ nodes for $b \neq 1$. Beam search with width w can reduce retained nodes while pruning a correct path based on an imperfect early score. Depth-first traversal reduces frontier memory but can spend a long budget on one unproductive branch. Search order therefore trades coverage, peak storage, latency, and susceptibility to evaluator error. Report the actual explored graph and pruning decisions. A nominal beam width alone does not identify how much model computation was used when proposals and evaluations have variable lengths.

For agents, branching can include real operations, not only text. A planner can safely explore hypothetical patches in isolated workspaces, but cannot assume every external action is reversible. A search method should distinguish simulated states from committed environment states and prevent one branch's side effects from contaminating another. Replaying a tool call can be expensive or non-idempotent. The search graph should retain operation identity and observed outcomes; executor permission checks remain independent of controller confidence.

9.7 Value of computation and stopping

Inference budget allocation is a sequential decision problem. Let b be the controller's information state and $V_0(b) = \max_{a \in \mathcal{A}(b)} \mathbb{E}[U(a) \mid b]$ the value of selecting now. Computation k costs c_k in utility units and returns evidence Z, producing state $b^{k,Z}$. Its one-step value is

$$\mathrm{VOC}_k(b) = \mathbb{E}_Z[V_0(b^{k,Z})] - V_0(b) - c_k. \tag{9.3}$$

Under calibrated Bayesian updating, unchanged external state, and retained access to all original actions, the unpriced information term is nonnegative by the expectation-of-maximum inequality. A learned judge or an irreversible external operation does not inherit these assumptions. Verification, retrieval, resampling, and delegation must be assessed by their effect on the selected artifact, including actual controller and worker costs.

Stopping when every one-step value is nonpositive is a myopic rule, not generally an optimal stopping theorem. In a constructed example, the correct binary answer is the exclusive-or of two independent fair bits. Either observation alone leaves optimal accuracy at $1/2$; both raise it to one. If each observation costs 0.1 in the same utility scale as correctness, each one-step net value is -0.1, while obtaining both has net value 0.3. Complementary computations can therefore justify continuing even when neither is individually valuable.

For a finite decision model with remaining resource budget B, deterministic resource cost $r_k > 0$, a known computation-transition law, and terminal selection utility, let W be the best continuation value. The recursion is

$$V(b, B) = \max\{V_0(b), W(b, B)\}, \tag{9.4}$$

$$W(b, B) = \max_{k : r_k \le B} \left[-c_k + \mathbb{E}_Z V(b^{k,Z}, B - r_k) \right]. \tag{9.5}$$

Set $W(b, B) = -\infty$ if no computation is feasible. These equations specify the ideal allocation problem; a deployed controller generally lacks its transition law and calibrated utilities. The 2026 meta-reasoning case in Section 9.1 supplies a concrete harness and artifact-level observations, rather than solving this recursion exactly [105]. Evaluate stopping against realized improvements and deadlines, including observation staleness induced by further deliberation. A model's statement that more thought would help is not itself an estimate of the displayed value.

Repeatedly checking the same candidate can have little value when checks share a failure mode. An independent executable test may be more informative than another paraphrased self-critique. Conversely, a cheap

local consistency check can eliminate an obvious error before an expensive environment call. The controller's action space should therefore expose cost and evidence type, not just a generic request to think harder. A budget measured in model calls can be misleading because one call may replay a huge context while another evaluates a short expression. Report tokens, external costs, and wall-clock time alongside the call count.

One operational realization is a myopic recurrence over a candidate graph G_n and remaining resource budget B_n. Decision epochs are measured after the controller's proposal and scoring costs have been charged. Let $\beta_v \geq 0$ reserve resources for the terminal check. For each proposed computation $k \in \mathcal{K}(G_n)$, $\widehat{\Delta}_n(k)$ estimates selection-utility gain, $\widehat{c}_n(k)$ its cost in the same utility units, and $\widehat{r}_n(k) \geq 0$ its total resource charge through the next decision epoch. Define

$$\begin{aligned} \mathcal{K}_n &= \{k \in \mathcal{K}(G_n) : \widehat{r}_n(k) \leq B_n - \beta_v\}, \\ k_n &\in \arg\max_{k \in \mathcal{K}_n} [\widehat{\Delta}_n(k) - \widehat{c}_n(k)]. \end{aligned} \tag{9.6}$$

The choice k_n is defined only for a nonempty finite candidate set; fix a tie rule. The continuation indicator and stopping index are

$$\begin{aligned} d_n &= \begin{cases} \mathbf{1}\{\widehat{\Delta}_n(k_n) > \widehat{c}_n(k_n)\}, & B_n > \beta_v,\ \mathcal{K}_n \neq \varnothing, \\ 0, & B_n \leq \beta_v \ \vee \ \mathcal{K}_n = \varnothing, \end{cases} \\ \tau &= \inf\{n \geq 0 : d_n = 0\}. \end{aligned} \tag{9.7}$$

For $n < \tau$, let a_n be the artifact actually returned by an execution operator $\mathcal{E}$ that applies the required isolation or current authorization. Its evidence record z_n retains source provenance σ_n, observed execution and recording cost $\rho_n \geq 0$, and the applicable configuration ν_n. Let $\gamma_{n+1} \geq 0$ charge preparation of the next decision epoch:

$$\begin{aligned} a_n &= \mathcal{E}(k_n), \\ z_n &= (a_n, \sigma_n, \rho_n, \nu_n), \\ G_{n+1} &= \Psi(G_n, z_n), \\ B_{n+1} &= B_n - \rho_n - \gamma_{n+1}. \end{aligned} \tag{9.8}$$

Here Ψ adds the observed artifact and its evidence links to the graph. For finite τ, a declared acceptance map $\mathcal{S}$ selects an artifact or returns the abstention marker $\perp$. The bounded terminal checker $\mathcal{V}$ returns a verdict v_τ and supporting record e_τ; its domain includes $\perp$, which produces an unresolved verdict and an abstention record:

$$\begin{aligned} a_\tau^* &= \mathcal{S}(G_\tau), \\ (v_\tau, e_\tau) &= \mathcal{V}(a_\tau^*; \beta_v). \end{aligned} \tag{9.9}$$

The gain estimate remains a hypothesis to validate, and $\mathcal{V}$ establishes only its specified property. Predicted resource feasibility does not guarantee $\rho_n + \gamma_{n+1} \leq B_n - \beta_v$; caps or justified bounds on both execution and controller work are needed to preserve the reserve. This formulation makes those admissibility obligations explicit without identifying the myopic controller with the ideal Bellman solution.

9.8 Experiments and failure attribution

Define comparison axes before allocating resources: single-attempt success, coverage at a candidate budget, selected-answer accuracy, repeated-run consistency, verified constraint violations, latency tails, and complete cost. Include a direct-answer baseline, ordinary intermediate reasoning, independent sampling, aggregation, and search with a common verifier where feasible. A stronger verifier is a meaningful system improvement, but should be named as the intervention. Oracle selection can estimate generation headroom and must be labeled nondeployable. It should not appear in the main autonomous-performance ranking.

Failure attribution should retain both candidate-level and task-level evidence. If no correct candidate exists, selection cannot solve the task. If a correct candidate exists but a weak judge rejects it, more generation may be wasteful. If an accepted program passes visible tests but fails a held-out requirement, the issue can lie in specification coverage. If a correct action is generated from a stale context, execution may still fail. These cases require different interventions. A single score labeled reasoning ability cannot identify them.

The practical role of inference-time computation is to convert available model competence and evidence into a better decision under an explicit resource envelope. Its limits arise from shared misconceptions, imperfect selection, stale state, and external effects as much as from insufficient token generation. The most informative studies expose those mechanisms with mathematical assumptions, implementation paths, and controlled outcomes. They establish where additional computation helps, where it merely repeats an error, and where another real observation is more valuable than another internal continuation.

An auditable search trace. The checked breadth-first search implementation offers a useful place to instrument these distinctions. Record every proposal before scoring, every value returned by the evaluator, the retained frontier, and the final selection. On a task with an exact checker,

label all generated candidates after the run without exposing those labels to search. This yields separate estimates of candidate coverage, evaluator discrimination, pruning loss, and final selection loss. A correct candidate pruned early is different evidence from a candidate that never appears. The instrumentation should include cache hits and repeated proposals so that duplicated work does not inflate apparent search diversity.

For a fixed set of candidates, compare the deployed selector with random choice, majority answer, shortest valid artifact, and oracle choice. These are selector ablations, not alternative generators. Then hold the selector fixed while varying proposal count and temperature. This two-stage design identifies whether a reported gain comes from finding more correct artifacts or recognizing them better. When the task requires external execution, candidate checks should occur in isolated environments or on immutable artifacts. Otherwise the checker itself can alter the state that later candidates encounter, invalidating the fixed-candidate interpretation.

The score-error bound earlier in this chapter also suggests a calibration diagnostic. Partition candidates by whether they were sampled randomly, selected for high score, or generated adaptively after feedback. Estimate judge error separately in each stratum. A much larger error in the selected stratum is direct evidence that ordinary calibration does not survive optimization pressure. Recalibrating only on the selected examples can help within that regime, but a new adaptive optimizer can create another shift. Keep an untouched evaluation sample and record each judge revision rather than treating the judge as a stationary oracle.

Memory reuse should be audited at the assertion level. If three workers all repeat a conclusion derived from one unverified artifact, their agreement is one evidence lineage, not three independent observations. A provenance graph can collapse these dependent claims before aggregation. Conversely, two independent executable checks on different relevant cases provide distinct evidence even when their natural-language summaries are identical. The controller should reason about that dependence explicitly. Otherwise increasing the number of agents can make a weak claim appear more certain through repetition alone.

These controls support practical allocation decisions. If generation coverage saturates but selection remains weak, spend budget on a stronger checker or a more informative test. If the checker is reliable but no correct candidate appears, improve decomposition, evidence retrieval, or the model. If both are adequate but state changes invalidate the chosen action, reduce observation age and strengthen execution preconditions. The resulting diagnosis connects inference-time reasoning to the surrounding agent system instead of using additional tokens as the default response to every failure.

Chapter 10

Closed-Loop Planning with Partial Observability and Fallible Feedback

10.1 WorldEvolver and memory conditioned foresight

WorldEvolver's September 2026 revision holds model parameters and the downstream agent fixed while revising world-model context through episodic transitions, semantic rules, and selective foresight. Its protocol permits five trials, with memory persistence affecting later attempts. Table 12 reports ScienceWorld ReAct results with Gemma-4-26B-A4B; the best-of-five and first-trial point estimates can move in opposite directions [89]. The overlapping task-level intervals are not a paired difference test, and the point estimates alone do not establish population-level superiority. These are author-reported results, not independent reproductions.

Table 10.1: WorldEvolver v2, Table 12: ScienceWorld ReAct success percentages. Brackets give the authors' task-level 95 percent bootstrap interval for best-of-five success.

Condition	Best of five [CI]	First trial
No world model	44.4 [34, 56]	28.9
WorldEvolver, unfiltered	46.7 [37, 57]	24.4
WorldEvolver, filtered	52.2 [42, 62]	36.7

The implementation chain is identifiable. The inspected `src/world_model/episodic_semantic.py` defines `WMEpisodicSemantic.predict`, `update`, and reset/checkpoint methods. The `SelectiveForesightGate.apply` path is in `src/world_model/modules/selective_foresight.py`, while `src/agents/wm_react_agent.py` registers `WMReactAgent` as a composition of the foresight mixin and the ReAct agent [111]. This permits a reproduction to inspect when a prediction is shown, when observed

mismatches update memory, and which state persists across trials. Those timings determine whether an evaluation measures fresh-task planning or adaptation from earlier attempts.

The gate uses the geometric mean token probability of the predicted observation when log probabilities are available; that quantity is not a calibrated probability of transition correctness [89]. In the pinned `SelectiveForesightGate.apply`, an enabled gate falls back to passing the forecast when mean log probability is unavailable, recording `open_no_logprobs` and a warning [111]. Enabling filtering in configuration therefore does not establish that filtering operated on every request; retain the per-call gate state and score availability. The source also selects thresholds on Word2World prediction-test confidence–quality curves before transferring them unchanged to AgentBoard [89]. This preserves a separate planning evaluation, but the prediction test used for threshold selection is no longer an untouched confirmatory sample for that selected gate. Test its discrimination and calibration against observed transitions, stratified by action type, output length, and environment shift. A useful gate can improve control by withholding harmful forecasts even if the underlying predictor is unchanged. Establish whether a forecast changes the action and whether the changed action improves the verified transition under a matched resource budget. Compare with a no-world-model baseline receiving equal opportunities to query or reflect.

Reset controls are essential when memory evolves. Clear state before every task to measure within-task adaptation; retain it across tasks to measure cross-task transfer; and separately reset between attempts if the objective is fresh-run reliability. Randomize task order and record the exact reset semantics. A best-of-five score under persistent memory is a valid outcome for a particular adaptive workflow, but it cannot be presented as the probability that one newly deployed run succeeds. The distinction matters directly for users who expect a system to complete a consequential task once.

For binary trial success Y_t, let $F_0 = \Omega$ and $F_t = \bigcap_{i=1}^{t}\{Y_i = 0\}$ under the specified reset and adaptation policy. On a positive-probability failure prefix, define $p_t = P(Y_t = 1 \mid F_{t-1})$; when $P(F_{t-1}) = 0$, set $p_t = 0$. Once a failure prefix is null, an earlier factor in the survival product is already zero, so this extension does not change the following chain-rule identity:

$$P\left(\bigcup_{t=1}^{k}\{Y_t = 1\}\right) = 1 - \prod_{t=1}^{k}(1 - p_t). \tag{10.1}$$

No independence assumption is required. Replacing every p_t by one common success rate requires an additional stationarity claim that persistent memory

can violate. Estimate trial-indexed hazards and paired task outcomes to distinguish improvement from prior attempts, repeated sampling, and task-mixture effects. This is a proposed diagnostic, not an explanation established for the reported point differences.

10.2 Goals must refer to external state

A plan is a policy for changing or learning about a task state, not merely an ordered explanation. Let $G(E)$ be a goal predicate on environment state, $I(S)$ an invariant on complete system state, and $c(a, S)$ an action cost. A useful planning problem specifies which terminal states satisfy the user objective, which intermediate states are prohibited, and what resources are available. An agent that produces a convincing account of G without establishing it in the environment has not completed that planning problem. The distinction is especially important when a language model can simulate the expected wording of a successful tool response.

Goal predicates can be partial or ambiguous. Passing tests may be an operational acceptance condition for a coding task without fully specifying intended behavior. A reservation can exist in a system while violating a requested time or budget. The planner should retain unresolved requirements and seek clarification or evidence when those distinctions affect action. Converting an ambiguous instruction into one confident internal goal silently changes the task. A practical goal representation therefore includes source instructions, interpreted constraints, acceptance checks, and unresolved assumptions, rather than only a short objective summary.

Multiobjective planning requires explicit priorities. Success, latency, monetary cost, privacy, and permission compliance cannot always be compressed into one unconstrained scalar reward. Some are hard feasibility conditions; others are tradeoffs within the feasible set. A planner should not spend an unauthorized action to gain a slightly higher task score. Let $Y_I(\tau) \in \{0, 1\}$ indicate whether trajectory τ contains a state violating the declared invariant I. A constrained objective can require $P_\pi(Y_I = 1) \leq \delta$, in addition to statewise authorization. These formulations are meaningful only when the probabilities and predicates have an evidential basis. A model-generated confidence statement does not establish the risk bound.

The plan's initial state is also a claim requiring evidence. A resumed session can contain stale memory, incomplete operations, or a tool schema different from the previous run. Before continuing, reconcile the durable event history with the current environment and permissions. The policy may need an information-gathering action rather than an execution action.

This makes planning inseparable from context and memory: the planner's apparent reasoning competence can be undermined by an incorrect state estimate before the first proposed step.

10.3 Belief states and information-gathering actions

Under partial observability, a belief $b_t(e) = P(E_t = e \mid H_t)$ summarizes uncertainty in an ideal known model. After action a and observation o, Bayes filtering gives $b_{t+1}(e') \propto P(o \mid e', a) \sum_e P(e' \mid e, a) b_t(e)$. Learned agents often use an approximate textual or latent state instead of an explicit distribution. That approximation should preserve which facts are observed, inferred, disputed, or unknown. A summary that converts uncertainty into an unqualified assertion changes the planning problem rather than merely compressing it.

Some actions primarily reduce uncertainty. Reading a file, querying transaction status, inspecting a browser element, or running a diagnostic can improve later decisions without immediately advancing the visible goal. Their value depends on what choices they can change. If two possible states require the same next action, distinguishing them has little immediate decision value; if they require opposite irreversible operations, the information can be decisive. This is the operational connection between representation sufficiency in Chapter 5 and planning. The planner needs distinctions relevant to its action set, not an exhaustive description of every environmental detail.

Information-gathering can itself have side effects. A diagnostic test can modify files or consume a rate-limited resource. A read can expose sensitive information to a model or external service. A browser refresh can discard an unsaved form. The planner should therefore model observation actions with costs and permissions rather than treating them as free oracle access. A common evaluation error gives a research agent a privileged state-inspection tool unavailable to the deployed system, then attributes the resulting gain entirely to better reasoning.

The belief update should incorporate negative evidence carefully. A missing response does not necessarily mean the operation failed; a missing memory result does not mean the underlying fact is false. The planner should distinguish absence of evidence from evidence of absence using the source's completeness contract. An authoritative database lookup can establish that no matching record exists under a defined query, while an approximate retrieval miss cannot. This distinction directly affects

whether the next action should retry, search differently, or proceed with a state-changing operation.

10.4 Plan contracts and compositional verification

For deterministic transitions, a local action contract can be expressed as $\{P_i\}\, a_i\, \{Q_i\}$: if precondition P_i holds and the action terminates according to its specification, postcondition Q_i holds. A sequence is compositionally valid when each postcondition establishes the next precondition and the invariant remains true. The proof is induction over steps. This is a precise reason to represent preconditions and observed postconditions explicitly rather than relying on a fluent plan whose steps merely sound compatible.

The weakest precondition for a deterministic action a and desired postcondition Q is the set of initial states from which executing a guarantees Q, under the action's semantics. Backward reasoning through a plan can reveal missing assumptions. If nontermination is possible, a weakest precondition for total correctness includes termination; a weakest liberal precondition describes partial correctness and does not require it. A planning contract must declare which claim is intended. For example, an update operation may require that a record exists, the version matches, and the user is authorized. A prior search that merely found a similar record does not establish those predicates. Natural-language planning can propose the chain, while executable checks verify the conditions at the actual boundary.

Stochastic actions require probabilistic contracts or conservative feasible sets. Suppose step i preserves the required conditions with conditional failure probability at most δ_i, given that previous steps have succeeded. A union-bound argument gives total failure probability at most $\sum_i \delta_i$, without requiring independent failures. The conditional qualification is essential. An average failure rate measured on unrelated states does not supply a bound for a planner-selected action. If authorization is a hard constraint rather than a statistical reliability target, it should be checked deterministically where the system has authoritative state.

Contracts remain partial specifications. A schema checker can establish argument shape, not task relevance. A unit test can establish behavior for selected cases, not every possible input. A proof can establish a formal statement while leaving the translation from user intent wrong. Compositional verification is strongest when those boundaries are explicit: combine local guarantees with independent checks of the overall acceptance condition, and retain uncertainty where the available specification does not decide the matter.

Table 10.2: Plan artifacts and the evidence they require.

Artifact	Required semantics	Typical failure
Goal predicate	Verified desired external state	Fluent completion claim without effect.
Action precondition	Current state and authority	Stale record version or revoked access.
Postcondition	Observed outcome after execution	Treating intended result as observed.
Invariant	Preserved constraint across steps	A locally useful but forbidden action.
Recovery branch	Defined partial-failure response	Blind retry of a committed operation.

10.5 Receding-horizon execution and grounded feedback

Open-loop planning selects a sequence before observing intermediate outcomes. Receding-horizon execution repeatedly plans, executes a prefix, observes the real result, and replans. The latter can correct state estimates and avoid compounding prediction errors, at the cost of additional decision and observation overhead. The correct horizon depends on action reversibility, environment volatility, model accuracy, and latency. A plan for a deterministic local calculation can be executed in a larger block than a sequence of irreversible operations against a changing external service.

ReAct interleaves reasoning and acting with environment observations, providing a concrete language-model-agent design for this feedback loop [1]. Its important boundary is that an action requests an external operation and an observation records a returned result. A model-generated continuation that imitates the observation is not equivalent. An execution harness should prevent these roles from being conflated in storage and training. A reasoning step can predict what a tool will return, but the prediction must remain distinguishable from the actual returned evidence.

Feedback has a source and scope. A compiler error identifies a problem relative to a language and configuration. A test failure identifies a violated assertion on an executed case. A human correction can clarify intent. A model critique supplies another statistical judgment. These signals should not all be assigned the same authority. A planner can use a weak critique to generate a hypothesis and then seek stronger evidence. Replacing an observed test result with a persuasive self-explanation reverses that

hierarchy and can cause the agent to reason away a real failure.

Feedback should update the right state. A transient service failure may justify retrying without revising a general skill. A consistent schema mismatch may require changing the tool adapter. An incorrect remembered assumption may require invalidating a memory record. A failed plan can expose an error in the goal interpretation, world model, action implementation, or acceptance checker. If every failure produces only a new natural-language reflection, the system may accumulate explanations while leaving the actual defect unchanged. Retain diagnostic evidence sufficient to choose among these updates.

10.6 Self-correction can also damage correct work

Self-correction is a transition from an existing candidate to a revised candidate based on feedback. Its value depends on both repair and corruption. Let the original candidate be correct with probability p, let r be the probability of repairing an incorrect candidate, and let c be the probability of corrupting a correct one. Under these aggregate conditional definitions, revised correctness is $p(1-c)+(1-p)r$, so the gain is $(1-p)r-pc$. Correction helps only when repair benefit exceeds damage to already correct work. A high repair rate alone is insufficient evidence of a useful correction system.

With hypothetical $p=0.9$, $r=0.3$, and $c=0.05$, revised correctness is 0.885, a decrease of 1.5 percentage points despite repairing 30 percent of initial errors. These are calculated probabilities, not experimental results. The example explains why evaluation should include initially correct cases and preserve their outputs before revision. A benchmark restricted to known failures can measure repair ability while missing the full system's regression rate. A correction gate that accepts only independently validated improvements can reduce corruption, but inherits the validator's own false-acceptance and coverage limits.

Reflexion uses verbal feedback and episodic reflection to influence later attempts without treating that textual update as a weight update [112]. This is a useful mechanism for memory-mediated adaptation. Its success depends on informative feedback, appropriate retrieval, and a policy that interprets the reflection correctly. A reflection can overgeneralize from one failure, blame the wrong component, or become stale after a tool change. Store the supporting case and applicability conditions rather than promoting every reflection into an unconditional rule.

Multiple correction rounds can amplify shared errors. If the same model

Table 10.3: Calculated correction behavior and required controls.

Quantity	Hypothetical value	Interpretation
Initial correctness p	0.90	Most candidates already correct.
Repair rate r	0.30	Conditional on an initial error.
Corruption rate c	0.05	Conditional on initial correctness.
Revised correctness	0.885	Net effect includes both transitions.

supplies the candidate, critique, and acceptance judgment, agreement need not indicate independent confirmation. A stronger experimental design crosses candidate generator, feedback source, and acceptance checker, retaining a common task set. Include no-feedback resampling to distinguish genuine feedback use from another chance to sample. Include misleading but plausible feedback to test whether the policy can reject unsupported corrections. The desired behavior is evidence-sensitive revision, not unconditional obedience to the latest criticism.

10.7 Hierarchical plans and reusable skills

Hierarchical planning replaces some primitive actions with temporally extended procedures. An option can be described by an initiation set, an internal policy, and a termination condition. For almost-surely finite duration $\tau \geq 1$, bounded rewards and discount $0 \leq \gamma < 1$, a semi-Markov value expression is $Q(s, o) = \mathbb{E}[\sum_{i=0}^{\tau-1} \gamma^i r_i + \gamma^\tau \max_{o'} Q(s', o')]$. The discount exponent records elapsed primitive time or steps under the chosen convention. Treating every skill as one equal-cost action ignores duration, tool expenditure, and failure exposure. A skill library can therefore make a plan shorter on paper while leaving its actual operational horizon large.

Reusable procedures need applicability conditions. A code-editing skill can depend on repository layout, language version, permissions, or a particular test command. A browser skill can depend on an interface that later changes. The planner should validate these conditions before invocation and interpret termination based on observed state. A natural-language skill description can help the model select a procedure, but it does not establish that the installed implementation satisfies the description. Versioned executable resources and tests provide a stronger connection between symbolic

planning and actual execution.

Abstraction can also conceal conflicts between subgoals. Two procedures may independently succeed while writing incompatible state, consuming the same budget, or invalidating each other's assumptions. A dependency graph should include shared resources and state versions, not merely an ordered list of textual tasks. Parallel execution is safe only when the relevant effects commute or are coordinated through an appropriate transaction or synchronization mechanism. Assigning different agent roles does not make their writes independent. This is a planning issue before it becomes a distributed-runtime issue.

A useful hierarchy experiment compares primitive control, fixed expert-authored skills, learned skill retrieval, and adaptive skill revision under the same external interfaces. Measure initiation errors, termination errors, task success, and complete cost. Hold out tool or repository variants to test transfer beyond memorized workarounds. A shorter generated plan is not by itself an improvement; it may simply hide more work inside a skill call. The scientific question is whether the abstraction improves decision quality and resource use while preserving the required constraints.

10.8 Partial effects and recovery planning

External operations can partially succeed. A file edit may commit before a test process crashes. A remote request may complete before a timeout. A multi-step workflow may create one resource and fail while linking another. Recovery begins by identifying the actual state, not by assuming the last returned error describes every effect. The planner should use operation identifiers, status queries, and recorded postconditions where supported. Blindly repeating a non-idempotent operation can turn an uncertain outcome into a definite duplicate.

Compensation is not the same as reversal. Deleting a newly created record may undo some state while leaving an email, audit event, charge, or downstream observation irreversible. A recovery plan should state which effects can be restored and which require disclosure or further action. This distinction constrains autonomous authority: a system may safely retry a read but need stronger authorization for an operation whose compensation is incomplete. A model's confidence that it can fix a mistake later is not evidence that the environment supports reversal.

The plan should reserve resources for verification and recovery. Spending the entire budget on candidate generation can leave no capacity to inspect the final outcome. A deadline-aware controller can allocate a minimum final-

check budget and stop exploration earlier. This is a constrained scheduling problem coupled to inference-time control in Chapter 9. Tool latency distributions matter: an operation with a short mean but a heavy tail can consume the recovery window. Report timeout and censored outcomes rather than treating them as if the agent had simply chosen to stop.

Fault injection can test recovery without inventing real failures in production. In a controlled environment, interrupt an operation after a known partial effect, drop a response, revoke permission between planning and execution, or replace a stored checkpoint with a stale one. Verify whether the agent reconstructs state, avoids duplicate effects, and preserves unresolved uncertainty. Keep these tests isolated and reversible. They diagnose mechanisms and do not estimate deployment incident frequencies unless the injected distribution is itself justified by operational data.

10.9 A reproducible planning evaluation

The evaluation record should contain the initial environment, interpreted goal, available tools, permissions, plan representation, observation policy, execution budget, and acceptance criterion. Baselines should include direct reactive control, a fixed decomposition, replanning without persistent reflection, and the proposed planning method. When a world model or verifier is added, hold the downstream policy fixed in a mechanistic ablation and also report a separately tuned system comparison. This separates component attribution from the practical performance of the complete integration.

Measure more than final success. Record invalid actions, violated preconditions, unnecessary operations, recovery success, corruption of initially correct artifacts, and the number of opportunities for irreversible effects. Use task-level paired comparisons and repeated runs where stochasticity is material. Preserve failures and censored runs in denominators. A planner that completes fewer tasks but avoids prohibited actions may satisfy a different operating objective from one that maximizes unconstrained success; the comparison must state that objective before ranking them.

The execution pattern can be stated as a guarded state recurrence. Let $s_0 = \mathcal{R}(H_0, o_0)$ reconcile recorded history with a current observation o_0, and let B_0 be the admitted resource budget. At iteration n, a plan-proposal law Π and prefix selector α produce

$$p_n \sim \Pi(\cdot \mid s_n, B_n), \qquad a_n = \alpha(p_n, s_n) \in \mathcal{A} \cup \{\bot\}. \tag{10.2}$$

The selector returns the next primitive operation of a prefix whose modeled preconditions hold, or $\bot$ when the next operation is unresolved. Let $\mathcal{D}$

acquire the required evidence and let $\mathcal{E}_{\nu_n}$ dispatch under the authority epoch current at execution. An observed-result update $\mathcal{U}$ retains actual evidence. The predicate $q(y) \in \{0,1\}$ identifies a partial or unknown outcome when it equals one; $\mathcal{H}$ augments that state with a proposed recovery plan:

$$\begin{aligned} y_n &= \begin{cases} \mathcal{D}(s_n), & a_n = \bot, \\ \mathcal{E}_{\nu_n}(a_n, s_n), & a_n \neq \bot, \end{cases} \\ s_n^+ &= \mathcal{U}(s_n, y_n), \\ s_{n+1} &= \begin{cases} \mathcal{H}(s_n^+, y_n), & a_n \neq \bot,\ q(y_n) = 1, \\ s_n^+, & a_n = \bot \ \vee \ q(y_n) = 0, \end{cases} \\ B_{n+1} &= B_n - \rho_n. \end{aligned} \tag{10.3}$$

The charge $\rho_n \geq 0$ includes the complete iteration, including evidence acquisition and recovery planning. The operators use realized observations; a predicted tool result does not substitute for y_n. A recovery proposal is retained work to evaluate, not evidence that compensation has executed.

Let $g(s) \in \{0,1\}$ be the specified goal-checker verdict and $b(s,B) \in \{0,1\}$ the resource-sufficiency verdict for safe continuation. With the latter checked after each completed transition, termination and reporting are

$$\begin{aligned} \tau_g &= \inf\{n \geq 0 : g(s_n) = 1\}, \\ \tau_b &= \inf\{n \geq 1 : b(s_n, B_n) = 0\}, \\ \tau &= \min\{\tau_g, \tau_b\}, \\ \mathcal{O}_\tau &= (\mathcal{C}^+(s_\tau), \mathcal{C}^?(s_\tau)), \qquad \tau < \infty. \end{aligned} \tag{10.4}$$

Here $\mathcal{C}^+$ collects verified outcomes with their evidence, including verified failures, and $\mathcal{C}^?$ retains unresolved outcomes. Initial admission and finite termination require their own implementation conditions. Neither the operator names nor a positive goal verdict establish properties beyond the checker's scope. Every predicate and update needs an implementation and tests against the actual environment.

Planning and self-correction improve agent behavior when they connect hypotheses to observed state and retain the evidence needed to revise them. Their failure modes are equally concrete: a wrong goal, an aliased belief, an invalid local contract, a misleading critic, a stale skill, or an unrecognized partial effect. Treating these as distinct mechanisms permits stronger analysis than attributing all outcomes to an undefined capacity for deliberation. It also connects planning to the rest of the system: context supplies the current evidence, memory supplies prior cases, the policy proposes actions, and execution determines what actually becomes true.

A temporal precondition counterexample. Consider a plan that reads a record at version seven, computes an update, and writes it after another actor has committed version eight. The read establishes the precondition only at observation time. If the write ignores the version, a locally well-formed plan can overwrite new information. An execution-time compare-and-swap condition changes the operation semantics: the write succeeds only if the version remains seven, otherwise the planner receives a conflict and must re-read or reconcile. This example shows why a correct textual deduction from a stale premise does not establish a valid external transition.

The same pattern appears with authorization. A permission observed during planning may be revoked before execution. A hard boundary checks the current authorization epoch, while the planner treats denial as new state information. Retrying the identical operation without refreshing that state is not self-correction. A learned policy can decide how to adapt after denial, but should not be allowed to reinterpret denial as permission because it believes the task is important. The separation of model inference and authoritative execution is what makes this behavior testable.

For multi-step plans, attach a state version or relevant precondition set to each consequential operation. A version mismatch does not always require discarding the entire plan: independent earlier conclusions may remain valid. The planner can maintain a dependency graph and invalidate only artifacts whose premises changed, provided that dependency relation is complete for the claimed reuse. Incomplete dependencies create the same kind of silent stale-state error as invalid KV reuse. Conservative full replanning costs more but supplies a useful reference baseline for evaluating selective invalidation.

A controlled test introduces concurrent updates at predetermined points while keeping the model and task fixed. Compare blind execution, full replanning after every change, and dependency-aware replanning. Measure lost updates, unauthorized attempts, useful work retained, observation cost, and complete task latency. These are proposed experiments; no result is presumed. The event schedule should be hidden from the agent but retained by the evaluator, and each run should start from a reproducible environment snapshot. Randomized schedules can test robustness after deterministic cases establish basic correctness.

This temporal view also improves self-correction evaluation. A revision that was correct when generated can become invalid before acceptance, while an apparent regression can reflect a changed task state rather than worse reasoning. Preserve the state against which each candidate and feedback item was judged. Without that association, a system can learn

contradictory lessons from the same artifact or report a false improvement by comparing outcomes under different environments. State-indexed evidence is therefore a prerequisite for interpreting planning progress across long sessions, concurrent agents, and tool upgrades.

Figure 10.1 isolates the mechanism and its stated information boundaries.

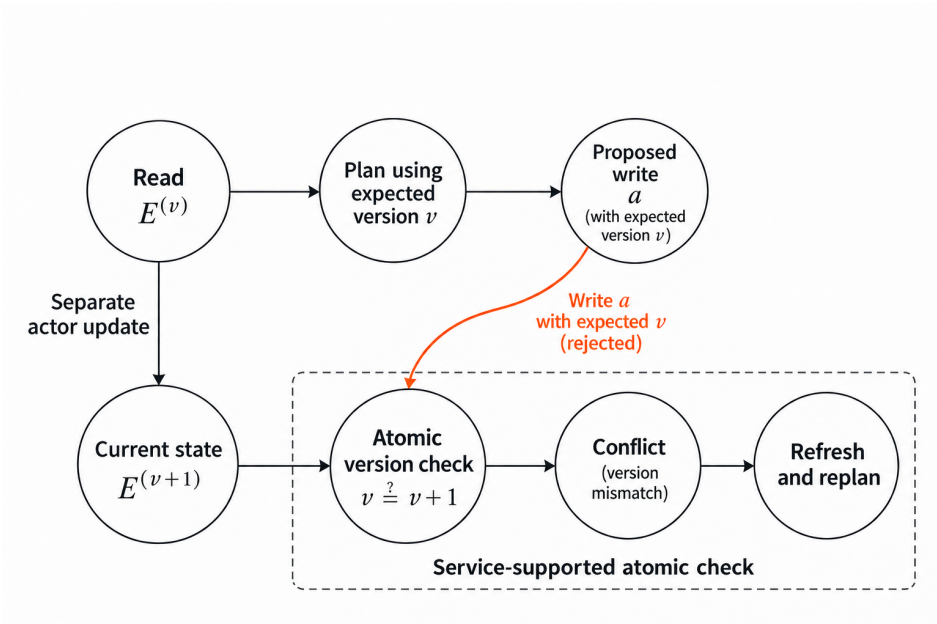

Figure 10.1: An observation-time precondition can become stale before execution. Here an intervening update changes version v to $v + 1$, so a service-supported atomic expected-version check rejects the proposed write. The planner must refresh or reconcile. This original mechanism example does not attribute atomicity to an ordinary client-side check.

Chapter 11

Long-Context Capability and Decision-Relevant Information

The 2026 Compaction Cliff study compares compression choices under task-dependent constraints rather than treating retained length as a sufficient quality measure [113]. Its conditional results motivate the central distinction developed here: an input can fit a window while losing a decision-critical relation. The mathematical target is preservation of relevant information, with access, reasoning, and computational cost measured separately.

11.1 Capacity is a constraint on an inference procedure

A context window is an admissible input length, whereas long-context intelligence is a property of a model interacting with a distribution of tasks. The difference becomes operational when an agent must combine an early instruction, a later correction, an external observation, and an unfinished commitment. A successful retrieval of an isolated sentence does not establish that the same model can reconcile those four objects. Conversely, an agent with a modest window may solve a long task by maintaining sufficient state outside that window. Input capacity, effective evidence use, and duration of operation therefore require different measurements.

Let the complete observable history be $H_t = (e_1, \dots, e_t)$, the query or current decision request be q_t, and the model input be $\mathcal{C}_t = \phi(H_t, q_t)$. The transformation ϕ includes serialization, retrieval, truncation, and any instruction or tool-description overhead. A model with token budget W requires $|\mathcal{C}_t| \le W$, but this inequality says nothing about whether $\mathcal{C}_t$ contains the evidence necessary for a correct action. Define a task-dependent evidence set $D_t \subseteq H_t$ and an action loss $\ell(a, Y_t)$. The relevant risk is

$$R(\phi, \pi) = \mathbb{E}[\ell\{\pi(\phi(H_t, q_t)), Y_t\}], \qquad |\phi(H_t, q_t)| \le W. \tag{11.1}$$

Here W is the admitted input budget. If the serving interface constrains input and generated tokens jointly, an output reserve must first be subtracted from that joint limit. The distribution in this expectation must identify

histories, decisions, environments, and evaluation times. Changing any of them can change the apparent effective context length without changing the architecture.

It is useful to distinguish a capacity failure from an accessibility failure. A capacity failure occurs when a chosen representation cannot fit a sufficient record into the available input. An accessibility failure occurs when a sufficient record is present but the model does not use it correctly. A third category, representation failure, occurs when information was transformed into an ambiguous or incorrect record before the model saw it. These categories imply different interventions: extend the budget, improve the model's use of evidence, or repair the state-construction procedure. Treating them as one phenomenon encourages expensive remedies that miss the responsible subsystem.

The transformer construction makes every permitted earlier token potentially available through attention, but potential connectivity is not reliable semantic access [46]. Position encodings, training distributions, intermediate representations, decoding, and instructions all mediate that access. A systems description should therefore specify the input length, the position of decisive evidence, the number of distracting or contradictory records, and the logical operations required to derive the action. The phrase "supports a long context" becomes scientifically informative only after those conditions are fixed.

11.2 A sufficiency criterion for retained state

The strongest justification for discarding history is that the retained representation preserves all information needed for the decisions being evaluated. Suppose Y is a finite-valued target and $Z = \phi(H)$ is a deterministic state representation. Under logarithmic loss, the optimal expected losses using Z and H are $H(Y \mid Z)$ and $H(Y \mid H)$, respectively. Their difference is

$$H(Y \mid Z) - H(Y \mid H) = I(Y; H \mid Z) \geq 0. \tag{11.2}$$

This is an information-theoretic deduction, not an empirical result about an existing agent. It assumes optimal conditional predictors and a fixed target distribution. Equality means that Z is sufficient for predicting Y relative to H. A fluent summary can have high semantic similarity to the original text while leaving this conditional mutual information strictly positive.

For sequential control, preserving the current answer is weaker than preserving future decisions. Let X_t be the environment state and let Z_t summarize observable history. A sufficient control state should support the

conditional distribution of future observations and rewards under relevant action sequences. One possible exact condition is

$$P(O_{t+1}, R_t \mid H_t, a) = P(O_{t+1}, R_t \mid Z_t, a) \tag{11.3}$$

for every action under consideration, together with a recursively valid update of Z_t. This condition is demanding because the environment is partially observed and the action set may change when new tools become available. A task-oriented summary is usually an approximation to this condition rather than a demonstrated sufficient statistic.

Consider a repository task whose earlier history contains a requirement to preserve an undocumented compatibility behavior. A summary that retains the current implementation plan but omits that requirement may support the next edit and still fail a later regression check. The target variable in Eq. (11.2) must then include that later check. If evaluation asks only for the next action, the compression can appear lossless even though it removes information necessary for task completion. Horizon and target definition are part of any claim about retained information.

An approximate decision bound can be stated without assuming optimal language modeling. Suppose two histories represented by the same z induce future-trajectory laws differing by at most $\epsilon \in [0, 1]$ in total variation under every evaluated policy. For the same return functional G taking values in $[-V_{\max}, V_{\max}]$, their expected returns differ by at most $2V_{\max}\epsilon$. If instead $G \in [0, V_{\max}]$, the bound is $V_{\max}\epsilon$. These follow from the range of a bounded function and the definition of total variation. The observation motivates evaluating behaviorally distinct histories that collapse to the same summary. It does not provide a practical certificate unless the required uniform bound can be estimated or established from environment structure.

The engineering consequence is to make discarded distinctions explicit. Preserve obligations, authorization, unresolved alternatives, evidence provenance, and invalidated assumptions as typed state rather than relying on a generic narrative summary. Exact sufficiency may be unattainable, but the system can identify which future decisions its representation is intended to support and which distinctions it knowingly loses.

Information availability and a fixed model's competence must be separated when interpreting a full-history baseline. For a finite action space and bounded loss, define the Bayes risk under information Z by

$$R^*(Z) = \mathbb{E}\Big[\min_a \mathbb{E}[\ell(a, Y) \mid Z]\Big]. \tag{11.4}$$

If $Z = \phi(H)$, a policy with access to H can imitate every Z-measurable decision rule, so $R^*(H) \le R^*(Z)$. An observed improvement after deleting

context therefore concerns the evaluated model, assembly procedure, and resource constraint; it does not show that removing information improves the unrestricted optimal decision. Compression can reduce distraction or make a finite computation feasible while still discarding information that another policy could use. This distinction is essential when attributing gains to a 2026 editor rather than to a weaker full-history implementation.

Figure 11.1 isolates the mechanism and its stated information boundaries.

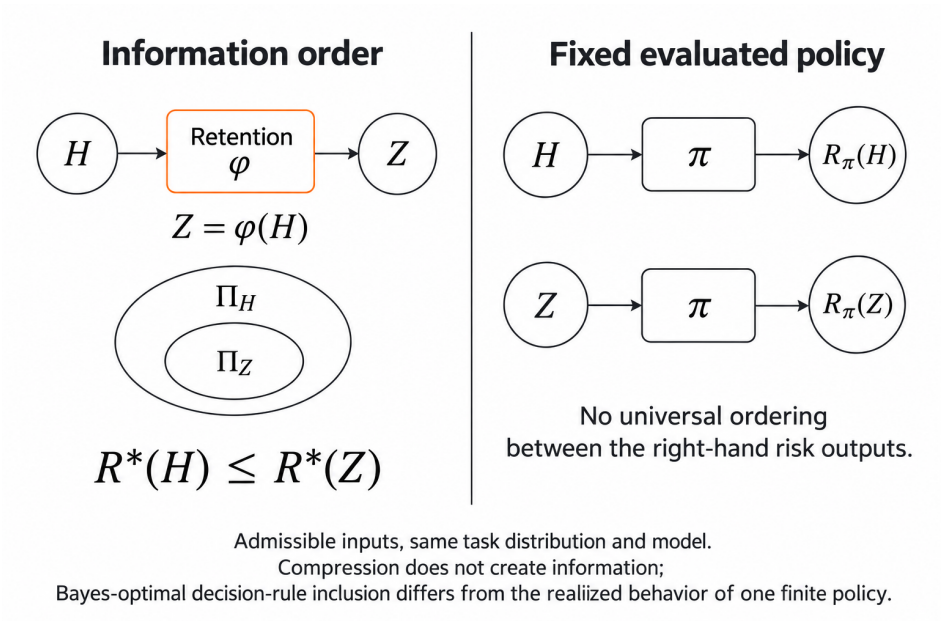

Figure 11.1: Deterministic retention restricts the available decision rules, so $R^*(H) \leq R^*(Z)$ under the stated Bayes-risk assumptions. A fixed model can nevertheless perform better on a compressed view because computation, distraction and serialization affect its realized behavior. The right-hand risks have no universal ordering. This original schematic contains no empirical comparison.

11.3 Position effects and compositional difficulty

Length is only one coordinate of task difficulty. A useful workload specification includes evidence distance, evidence density, distractor similarity, the number of required joins, contradiction depth, and the delay between observation and consequence. Evidence density is the fraction of input that

contributes to a correct answer. Low density stresses selection; high density with many dependencies stresses integration. Two inputs of equal token length can therefore create different computational demands.

Lost in the Middle provides evidence that information placement can affect model performance in its tested retrieval and question-answering settings [114]. RULER and LongBench broaden long-context evaluation through controlled and heterogeneous tasks [115, 116]. Their central methodological value here is to make context use observable under specified tasks. They do not establish a universal conversion from advertised token capacity to a reliable operating horizon. Synthetic retrieval, document question answering, and multi-step interactive execution expose overlapping but different failure modes.

The pinned RULER implementation makes several controls inspectable. In `scripts/data/synthetic/niah.py`, `generate_input_output` varies needle keys, values, queries, distractors, and insertion positions; `generate_samples` uses the selected tokenizer and reserves generated tokens and model-template overhead while fitting inputs. The standalone `scripts/pred/call_api.py` prediction path appends the stored answer prefix before dispatch. In `scripts/eval/evaluate.py`, `run_evaluation_per_task` applies the task metric and separately counts empty predictions [117]. Preserve the exact rendered input, tokenizer, seed, target position, and generation reserve when comparing lengths. The inspected prediction helper retries exceptions without a finite retry limit, and the evaluator skips absent task files after printing a warning. A reproducible adaptation should impose a declared deadline, reconcile expected and completed task identifiers, and retain failed requests in its accounting. These are implementation obligations; this static benchmark does not itself certify session continuity or correct external effects.

To separate these effects, construct a family of tasks with fixed decisive records and systematically permute their positions while keeping total length constant. Next, increase distractor similarity without changing the decisive records. Finally, change the number of relations needed to connect the records. The resulting response surface $s(L, d, k, \rho)$ records success as a function of length L, evidence distance d, join count k, and distractor parameter ρ. A length-only curve is a projection of this surface and can conceal strong interaction effects.

Repeated facts require special care. Repeating a constraint near the decision can improve accessibility while changing the workload's redundancy. If a long-context method benefits from explicit reminders, that result should be distinguished from recovery of a unique distant fact. Likewise, a task that includes both a stale fact and a corrected fact tests temporal interpretation

as well as retrieval. An answerer can retrieve both perfectly and still resolve their authority incorrectly. Position permutations must preserve timestamps and source roles or the experimental manipulation changes the intended semantics.

Compositional difficulty also interacts with output format. Choosing a cited answer from a small candidate set can be easier than synthesizing an executable action with multiple arguments. An agent must often retrieve an entity, infer its current state, select a tool, satisfy a schema, and respect an authorization boundary. Evaluation that terminates after entity retrieval measures only the first component. For expert comparison, the action-generation stage should be retained, and failures should be attributed to evidence access, reasoning, argument construction, or execution.

An informative negative control inserts irrelevant but lexically attractive records whose instructions conflict with the user task. The model should recognize these as data rather than authority. This extends context evaluation toward the security analysis in Chapter 28. Success under benign distractors does not imply robustness when retrieved content attempts to redefine the task or rewrite the memory policy.

11.4 Training distributions and length generalization

Length generalization depends on more than the numerical range of a position encoding. Training establishes distributions over token positions, document boundaries, dependency lengths, and tasks. Increasing an inference limit changes the region in which those learned mechanisms operate. Rotary or recurrent positional constructions address parts of this problem, but an architectural ability to represent a position does not by itself demonstrate competent reasoning at that position [49, 47].

For an agent, training must additionally expose the model to histories with interrupted plans, corrected observations, failed tool calls, and records that have become irrelevant. A long concatenation of independent documents is not equivalent to a coherent interaction trajectory. The latter contains dependencies between what the policy observed, what it attempted, what the environment changed, and what remains unresolved. Training data that omit these distinctions can reward answer extraction while failing to teach durable task-state maintenance.

Let $p_{\text{train}}(L, K, T)$ describe training examples by length, dependency complexity, and interaction type, and let p_{deploy} describe deployment. Reporting only a maximum training length suppresses shifts in the other

coordinates. A model may encounter deployment histories within the trained length range but outside the trained distribution of corrections or tool-output redundancy. The appropriate claim is then distribution-specific performance, not unrestricted length generalization.

A defensible extension experiment holds architecture and total training compute fixed where feasible, varies the long-sequence curriculum, and evaluates both short and long tasks. The short-task control detects capability tradeoffs, while the long-task suite distinguishes retrieval from integration and interactive continuity. If extra long-sequence training also increases the token budget or improves data quality, the comparison must identify those additional factors. A recovery phase after agentic post-training has a real compute cost and should not disappear from the accounting boundary.

Contamination controls operate at the level of both documents and tasks. Removing exact benchmark documents does not rule out exposure to paraphrased questions, released solutions, or public environment traces. For interactive tasks, the environment schema and common solution trajectory can leak even when the surface instruction is novel. Separate held-out entities, templates, environment states, and required action compositions. Record which dimensions were held out and avoid claiming general transfer from a single split.

Implementation should preserve document and event boundaries through packing. If truncation removes the role or timestamp of a message, the remaining content can acquire a different interpretation. Tokenization and serialization are therefore part of the learning distribution. Chapter 17 analyzes the stronger problem arising when training reconstructs a different context from the one used during rollout. Long sequences can be correctly packed in memory and still be conditionally wrong for learning.

11.5 Computation and memory over an interaction horizon

Assume a decoder with n_ℓ layers, hidden width d, h_{kv} key-value heads, equal key and value head dimension d_h, and b bytes per cached scalar. Assume the same cache shape at every layer and full residency of every visible token; latent caches and layer-specific or sliding-window caches require their own accounting. Ignoring allocator overhead, the cache for one sequence of L tokens occupies

$$M_{KV} = 2n_\ell L h_{kv} d_h b. \tag{11.5}$$

The factor two accounts for keys and values. This expression excludes model weights, activations, page metadata, workspace, and fragmentation.

Grouped-query attention changes h_{kv}; changing the total query-head count without specifying key-value sharing is insufficient for memory accounting [51]. Cache paging changes allocation behavior rather than the semantic amount of remembered evidence [59].

For dense attention, an incremental decode step reads a growing cache and performs attention against the visible prefix. A simplified traffic model is $B_t \simeq \kappa M_{KV}(L_t)$, with κ representing actual cache access, reuse, and implementation overhead. The bandwidth lower bound B_t/β assumes an effective bandwidth β and cannot predict observed latency without accounting for compute, synchronization, batching, and competing traffic. Peak device bandwidth is not sustained application bandwidth.

The interaction horizon introduces cumulative costs. If each tool turn contributes u input tokens and the system repeatedly reconstructs the full history, the total input volume over T turns scales as

$$\sum_{t=1}^{T} ut = uT(T+1)/2. \tag{11.6}$$

This is an analytic count under equal-sized turns and no reusable prefix accounting. Prefix caching can reduce repeated computation, while billing rules may count tokens differently. The expression therefore diagnoses a growth pattern rather than an invoice. A bounded working context of size W changes the repeated-input term toward $O(TW)$ but introduces storage, retrieval, editing, and error-recovery costs.

Long-context serving also creates capacity competition. A scheduler holding a large inactive session can reduce concurrency for short requests. Evicting that session's cache saves device memory but may incur a costly prefill when it resumes. The correct objective depends on the workload mixture and service-level constraints: minimize mean cost, maximize throughput, or control tail latency. A policy optimized for average token throughput can be undesirable when a few latency-sensitive tool decisions gate an entire workflow.

Latency should be decomposed into queueing, context construction, prefill, generation, tool execution, and validation. These components are not independent. Large contexts can increase queueing through resource contention; additional retrieval can reduce prefill but add a network round trip; compression can reduce the next call yet increase retries after information loss. Report distributions over complete tasks and retain timeouts. Measuring only successful completions introduces survivorship bias into both latency and cost estimates.

The practical design question is consequently when additional retained

context produces enough decision value to justify its full systems cost. That tradeoff depends on the probability that distant evidence becomes decisive, the cost of reacquiring it, and the consequences of forgetting it. Uniform retention and uniform eviction are both coarse approximations to this decision problem.

11.6 Experimental designs that isolate context use

A proposed long-context evaluation should begin with matched information access. Full-history, retrieval, and summary systems should receive the same underlying evidence, tools, model version, and authorization. The comparison then varies only how evidence reaches the model. A second experiment can permit learned context control, but its training and search costs should be reported separately. Otherwise, architectural selection effects become entangled with an additional optimization budget.

Recent work makes these distinctions concrete. The Compaction Cliff classifies knowledge by type and applies separate operators for compaction, decomposition, and retrieval, retaining recognized constraints under type-specific rules [113]. Its guarantees depend on constraint classification, applicable scope, feasible budgets, and verification; they are not unconditional preservation of every real safety requirement. Its behavioral retail comparison is not token-matched, and some reference labels are model-generated. Those qualifications prevent an observed advantage from being assigned solely to typing. The preprint was available before the cutoff even though its listed conference dates fall later.

The methodological conflict is useful. Uniform summarization assumes that one semantic compression objective is adequate across records, whereas typed retention treats some distinctions as constraints that should survive with stronger fidelity. The latter can preserve a rare prohibition even when a relevance score prefers repeated descriptive content. It can also fail if the classifier mistakes a declarative fact for ordinary background or assigns an inappropriate scope. A test of constraint preservation should therefore vary grammatical form and authority independently of content frequency.

Two 2026 surveys supply organizing lenses rather than independent experimental confirmation. The rate–distortion view groups several forms of context-derived compression under budget and downstream-utility tradeoffs [17]. The October selective-memory survey distinguishes information participating in computation from stored information that can later be activated [16]. These taxonomies help align accounting boundaries, but

they do not prove that cache eviction, semantic forgetting, and text summarization have identical guarantees. Their state transitions and recovery mechanisms remain different.

An original deduction follows from combining these distinctions with the sufficiency criterion: fidelity should be allocated according to the loss of the distinction being removed, while activation cost should be charged when stored evidence becomes computation. This does not yield a universal scalar salience score. It yields a falsifiable comparison: preserve the same constraint set, vary non-constraint evidence and activation policy, and measure both justified action and full resource use. A method that wins only by silently relaxing the constraint set has changed the task.

Use at least three baselines: a full-history baseline within the admissible budget, a recency window, and a simple retrieval or structured-state baseline. The full-history baseline tests whether the proposed transformation removes harmful distraction or merely compensates for capacity. The recency baseline measures whether most tasks are effectively short. The simple structured baseline tests whether elaborate learned control is necessary. An oracle evidence condition is useful as a diagnostic ceiling, provided it is clearly labeled and never compared as an executable production method.

For each task, instrument the first decision at which a required historical distinction matters. Record whether the distinction exists in storage, appears in the model input, is interpreted correctly, and leads to the correct tool action. This decomposition prevents a retrieval miss from being mistaken for weak reasoning and prevents a correct answer with an invalid side effect from being counted as success. The same decomposition supports targeted ablations: preserve the record but perturb its timestamp, preserve its content but remove its source role, or preserve both while moving its position.

Uncertainty should reflect the unit of generalization. Repeated decoding seeds on the same task measure stochastic policy variation, not variation across task families. Use task-level resampling for task uncertainty, environment-level separation where tasks share state, and independent training seeds when comparing learned context policies. Task order matters whenever memory or a learned skill bank persists. A single favorable sequence can behave as an implicit curriculum and should not be treated as evidence of stable continual improvement.

Contamination testing should include withheld corrections and contradictory updates rather than only unseen questions. A system that remembers the original fact but fails to adopt an authoritative revision has failed a different requirement from one that never stored the fact. Include negative

controls where an old record should be ignored and where no historical record is relevant. Without these controls, systems that overapply memory may appear strong on a benchmark constructed entirely from positive recall opportunities.

The proposed experiment is discriminating only if the result can falsify a mechanism. If a method claims to preserve decision-critical evidence, a loss of that evidence under matched storage and retrieval is directly informative. A small change in an aggregate score without an inspectable failure record may be insufficient to identify whether retention, attention, or action selection changed.

11.7 A worked capacity and decision example

Consider an explicitly hypothetical decoder with 32 layers, 8 key-value heads, head dimension 128, and two-byte cached scalars. Equation (11.5) gives 131072 bytes per cached token. A sequence of 32768 tokens therefore requires 4 GiB of raw keys and values, while 131072 tokens require 16 GiB. These are arithmetic consequences of the stated configuration, not measurements of a named model or service. They exclude every other allocation and do not establish how many sessions fit on a particular accelerator. Replication, quantization, cache paging, and attention implementation can change the actual residency.

Suppose a deployment must hold ten such long sequences concurrently. The raw cache term alone becomes 160 GiB before weights or workspace. Moving half the sessions to host memory reduces device residency, but a resumed session may need cache transfer or recomputation. A throughput analysis that treats those sessions as permanently active can differ sharply from a workload in which users pause for minutes between tool decisions. Concurrency is therefore a property of arrival and activity patterns, not merely a maximum batch-size argument.

Now consider two histories h_0 and h_1 that are identical except for a valid user correction: one permits publishing an artifact and the other permits only preparing a draft. A summary function that omits the correction maps both histories to the same state z. Suppose the task requires a binary publish-or-hold action, the two histories are equally likely, and the correct actions differ. Any deterministic policy conditioned only on z must be wrong for one history, giving error at least one half. Randomization cannot lower the average error because the optimal mixture still lacks the distinguishing bit. Unlimited additional reasoning on z cannot recover an observation the representation discarded.

If the correction remains available in raw storage, retrieval can repair the failure, but only if the retrieval policy requests the right evidence before the consequential action. The memory system has then preserved capacity while the working-state policy has failed accessibility. If the correction was deleted from all retained evidence, retrieval cannot repair it. If the correction is present but labeled as an untrusted web instruction, its authority is misrepresented. These three cases can yield the same final wrong action, yet their engineering remedies are different.

This example also exposes a limitation of average semantic fidelity. The omitted correction can occupy a tiny fraction of a very long history while determining the entire action loss. A representation that preserves almost every token may be worse for the task than one that retains only a small structured set of obligations and evidence pointers. Token retention is not a monotone proxy for decision sufficiency. Conversely, a compact obligation list can fail when a later task needs an unforeseen detail, which is why retaining reversible access to raw history remains useful.

The proposed diagnostic is to create matched history pairs differing in one consequential fact and test whether a context policy preserves that distinction through length extension, compaction, interruption, and resumption. Positive results would support a specified invariance over the tested transformations. They would not prove that every unknown future distinction is preserved. This makes a carefully delimited behavioral claim possible without pretending that a generic summary is a sufficient statistic for all future tasks.

11.8 Effective context as controlled state

Long-context systems expose an important asymmetry: storing more evidence is straightforward to specify, whereas identifying what will matter later requires assumptions about future tasks. A larger window relaxes a resource constraint, but it does not remove the need for organization, authority, or temporal interpretation. At sufficiently long horizons, unfiltered history becomes a record of abandoned plans, obsolete facts, and adversarial content as well as useful evidence. More history can increase both opportunity and ambiguity.

The original synthesis of this chapter is that effective context should be evaluated as a controlled state representation. Its quality depends jointly on information preserved, the policy's ability to use that information, and the operational cost of making it available. Equation (11.2) supplies a precise idealization for predictive loss, while the control-state condition

extends the concern to future action consequences. Neither equation turns present agents into certified sufficient-state estimators; both clarify which claims an experiment would need to support.

Several practical rules follow from that formulation. Preserve authoritative constraints independently of conversational salience. Retain references to raw evidence whenever summaries may be revised. Expose version and timestamp metadata at the decision point instead of expecting the model to infer them from prose. Measure complete-task cost rather than one-call context size. Treat cache state as an implementation of inference, not as a validated semantic memory. These are design deductions under explicit state and cost assumptions rather than reported superiority of a particular platform.

The limits of current evidence are substantial. Controlled context benchmarks are useful, but their target distributions need not match a changing enterprise workflow or a long scientific investigation. Public papers can establish gains under specific environments and budgets without establishing reliable indefinite operation. A context-management method may transfer across several tasks yet still depend on the same backbone, retriever, or evaluator. Independent reproduction and operational deployment answer additional questions.

For implementation, a useful boundary is to keep immutable interaction evidence, mutable task state, and the assembled model input distinct. This allows context construction to be inspected as a transformation rather than an invisible consequence of token overflow. It also permits counterfactual replay: reconstruct a prior decision with the original context, an edited context, or an oracle evidence set, while holding the environment snapshot fixed. Such replay is expensive, but it can reveal failure mechanisms that aggregate success rates conceal.

Chapter 12 develops the editing policy that chooses the model's working view. Chapter 13 treats the event and session substrate that makes those choices recoverable. The resulting picture is an agent whose finite context is one deliberately managed component of a larger stateful system. Its generality remains an empirical question about task transfer, adaptation, and reliable execution under bounded resources.

Chapter 12

Context Transformation: Selection, Compaction, and Policy-Controlled Editing

The October 2026 transparent-compaction implementation preserves conversational prose while masking recoverable blocks and exposing delegated lookup; Context Language Models instead permit programmatic edits to the live context [118, 26]. These mechanisms alter different information and execution boundaries. Comparing them requires a transformation contract, downstream continuation loss, restoration cost, and an exact record of which view conditioned each action.

12.1 The working view as a controlled variable

Context engineering is the construction of the actual conditional input on which a policy selects its next action. It includes the selection of instructions, evidence, tool descriptions, intermediate results, and unresolved commitments. Compaction is one intervention within that broader process. An agent can compact history while preserving raw storage, evict a cache without editing text, or edit its visible history without deleting a persistent record. These operations affect different state variables and should have different names in an implementation.

Let H_t denote the append-only interaction evidence, M_t the persistent memory, and $\mathcal{C}_t$ the model's working context. A context controller chooses an editing action u_t before the task policy chooses an environment action a_t:

$$\mathcal{C}_t = g(H_t, M_t, u_t), \tag{12.1}$$

$$a_t \sim \pi_\theta(\cdot \mid \mathcal{C}_t), \tag{12.2}$$

$$u_t \sim \mu_\psi(\cdot \mid H_t, M_t, B_t). \tag{12.3}$$

Here B_t is the remaining resource budget. The decomposition can be implemented by separate models, one model with distinct action types, or deterministic harness logic. It is a functional distinction, not a claim that

a separate context model is always beneficial. Access to H_t is not free: the total account must charge the controller's scans, retrieval, model calls, and storage operations as well as the task policy's input.

The context controller faces an unusual delayed-control problem. Removing a record can make the next request cheaper but create a failure many steps later. Its reward is mediated by another policy and by the environment. A controller trained only to reduce input size may learn aggressive deletion that looks efficient until a deferred constraint becomes decisive. Conversely, penalizing every retrieval call can encourage a policy to retain large amounts of irrelevant content. The objective must include task outcome and complete lifecycle cost.

A useful constrained formulation uses a total resource cost C_{total}, a terminal success indicator $Y_s \in \{0, 1\}$, and an indicator $Y_u \in \{0, 1\}$ that at least one unauthorized effect occurred. For fixed task and environment distributions, define

$$\min_{\mu,\pi} \mathbb{E}[C_{\text{total}}] \quad \text{subject to} \quad P_{\mu,\pi}(Y_s = 1) \geq \tau, \quad P_{\mu,\pi}(Y_u = 1) \leq \delta. \tag{12.4}$$

The thresholds $\tau, \delta \in [0, 1]$ are workload-specific requirements. They are not established constants, and success estimation must retain failures and timeouts. A Lagrangian objective can aid optimization, but it does not prove that the constraints hold after training. Authorization should generally be enforced outside a stochastic controller rather than represented only as a reward penalty.

The central design question is which distinctions the working view must preserve. A context assembled for code editing needs current files, interface constraints, test outcomes, and rejected hypotheses. A literature agent needs claims, supporting passages, conflicting evidence, and unresolved source identities. Applying the same generic summary template to both workloads can remove the information that makes their next actions valid. State fields should follow decision requirements, with prose used where a schema would erase necessary nuance.

12.2 Selection before abstraction

Selection retains some records verbatim; abstraction constructs a new record that represents several old ones. These operations have different error modes. Selection can omit a decisive source while faithfully preserving everything selected. Abstraction can include the right sources but misstate their contents, merge incompatible claims, or remove a qualification. A

practical pipeline should make the two stages separately inspectable instead of treating a single generated summary as an opaque transformation.

Suppose records r_i have token costs c_i, and $F(S)$ is the task-dependent utility of retaining the records indexed by S. A simple selection model is a budgeted set problem:

$$\max_{S \subseteq \{1,\ldots,n\}} F(S) \quad \text{subject to} \quad \sum_{i \in S} c_i \leq W. \tag{12.5}$$

If F is monotone submodular, marginal-gain heuristics have useful theoretical properties under suitable assumptions. Agent evidence does not generally satisfy those assumptions. Two individually useless records can jointly identify a contradiction, while one new correction can invalidate many older records. Utility may be complementary or nonmonotone. Using a greedy score can still be an engineering heuristic, but its approximation guarantees do not transfer without verifying the required structure.

Abstraction introduces a different optimization target. Let $z = s(R)$ be a summary of records R and let $\mathcal{D}_R$ be a specified conditional distribution over future requests q and targets Y given that source record. Define distortion through paired decision losses rather than textual similarity:

$$d(R, z) = \mathbb{E}_{(q,Y)\sim\mathcal{D}_R} \left[\ell\{\pi(q, z), Y\} - \ell\{\pi(q, R), Y\} \right]. \tag{12.6}$$

This is a proposed operational distortion measure. Both conditions receive the same request–target draw; a stochastic policy's internal sampling is also included in the expectation. The controller observes R and the current request, not the evaluation target Y. Distortion can be negative when abstraction removes distraction, and it depends on the evaluated policy. It is not a mathematical distance. Holding $\mathcal{D}_R$ fixed prevents a change in the future-question distribution from being misreported as better preservation.

Preserving exact text is especially valuable for identifiers, numeric requirements, authorization, and claims requiring attribution. A summary may safely paraphrase a search strategy while retaining an exact test failure or contract clause as a quoted evidence object. The representation can combine concise state with evidence pointers rather than forcing every item into the same compression format. Token allocation should reflect the cost of a mistaken abstraction, not only the record's length.

An implementation can maintain an immutable source layer, a typed decision-state layer, and a generated narrative layer. Each narrative assertion should link to one or more source objects or be marked as an inference. Contradictions remain separate objects until resolved by an explicit rule. This architecture makes it possible to regenerate a summary after a cor-

rection without pretending that the first summary was an authoritative replacement for the evidence.

12.3 Reversible offload and explicit context actions

ACM makes context management an explicit part of agent behavior through a summarization action and retrieval of earlier raw interaction history [119]. Its distinction between a bounded active view and preserved external history is technically important. Offloaded text can remain intact while the working context becomes lossy. The agent still has to recognize when it needs an omitted detail, issue a suitable retrieval request, and interpret the returned record correctly. Storage reversibility and behavioral reversibility are different properties.

The paper's mechanism can be understood as a state transition with a retained suffix, a compacted prefix, and access to raw evidence. The resulting policy is more flexible than mandatory truncation at a fixed threshold because it can choose when to summarize. That flexibility creates a larger action space and additional credit-assignment demands. The context-management decision can be correct locally but harmful to a later step, so evaluating only the immediate post-summary answer is insufficient.

Context Language Models go further by making the live context directly editable and studying serving support for such edits [26]. Their reported FLOP accounting should not be equated with measured latency or service expenditure. The generalizable distinction is between a fixed set of harness editing operations and a model-controlled editing program. The latter expands expressivity but also expands the space of invalid or semantically damaging edits.

A conservative editing interface can represent operations as typed transformations: retain, summarize, externalize, restore, annotate, and invalidate. Each operation has preconditions and a provenance record. "Invalidate" should preserve the reason a fact became obsolete; it should not silently erase the record needed to understand a past decision. "Restore" should recover source roles and timestamps together with text. A raw paragraph returned without its original authority can change its meaning.

The proposed construction below is an implementation contract, not a reproduction of ACM's code. Let $R = \mathcal{P}(S)$ pin obligations and authority-bearing records from state S, $D = \mathcal{L}(H, S)$ locate a finite set of currently readable candidate evidence in history H, and $N = \mathcal{N}(H, S)$ retain recent events. The token-cost map ℓ measures the variable serialization charge

against residual capacity W; fixed material already reserved in W is not charged again. When $\ell(R) \le W$, a declared selector σ chooses from

$$\begin{aligned} \mathfrak{E}_W &= \left\{ E \subseteq D : \ell(R) + \sum_{r \in E} \ell(r) \le W \right\}, \\ E_W &= \sigma(\mathfrak{E}_W; H, S), \\ z_W &= \mathcal{S}(E_W), \qquad c_W = \Sigma(R, z_W, N). \end{aligned} \tag{12.7}$$

Here $\mathcal{S}$ summarizes with source identifiers and Σ serializes the pinned records, summary, and recent events. No optimality of σ or semantic sufficiency of $\mathcal{S}$ is assumed. Let $v(R, c_W) \in \{0, 1\}$ test structural preservation of the pinned records in the completed serialization. A conservative admissibility rule returns either the completed context, an infeasible-budget marker $\perp_B$, or a failed-preservation marker $\perp_V$:

$$\widehat{C}_W = \begin{cases} \perp_V, & v(R, c_W) = 0, \\ \perp_B, & v(R, c_W) = 1,\ \ell(c_W) > W, \\ c_W, & v(R, c_W) = 1,\ \ell(c_W) \le W. \end{cases} \tag{12.8}$$

This inner construction is defined only when $\ell(R) \le W$. The complete budget guard is

$$C_W = \begin{cases} \perp_B, & \ell(R) > W, \\ \widehat{C}_W, & \ell(R) \le W. \end{cases} \tag{12.9}$$

An infeasible result requires replanning or a larger admissible budget, rather than deletion of a pinned obligation. The final length test is an explicit admissibility requirement: a raw-evidence selection bound alone does not constrain summary expansion or the cost of adding recent events. It is a conservative completion of the construction, not an assertion that the cited implementation already enforces this gate.

Let J be the edit journal, $\mu(R, E_W, z_W, N, c_W)$ the manifest linking the complete candidate view to its pinned records, selected sources, recent events, roles, timestamps, and transformation, and $\oplus$ append a durable record. Persistence is conditional on admission:

$$J^+ = \begin{cases} J \oplus \mu(R, E_W, z_W, N, c_W), & C_W \notin \{\perp_B, \perp_V\}, \\ J, & C_W \in \{\perp_B, \perp_V\}. \end{cases} \tag{12.10}$$

The preservation test is only as strong as its validator. A deterministic check can confirm that an identifier or typed obligation survives, but it cannot generally establish semantic sufficiency of a prose summary. Structural preservation and budget admission remain useful invariants, not a complete

proof that future decisions will remain correct. Returning C_W as an installed view additionally requires successful manifest persistence; a failed journal write is an unresolved transition, not a completed edit.

The budget W is residual capacity after fixed instructions, selected schemas, fixed serialization overhead, and any reserved output allowance. If pinned requirements exceed that residual, the feasible selection set is empty. Silently dropping a prohibition to make the request fit changes the task rather than solving the allocation problem. Replanning can reduce execution granularity, move validated control fields outside model-visible prose, or acquire a larger admissible input; it must preserve the dispatch monitor's current authority. A tool or protocol revision can change schema overhead and thus invalidate an otherwise unchanged compaction schedule.

12.4 Compaction triggers and delayed credit

A fixed threshold triggers compaction when the current input exceeds a fraction of the window. This is easy to implement and creates predictable resource bounds, but it ignores semantic phase boundaries. Compressing immediately before a decisive action can discard evidence that would have been useful for verification. Compressing after a stable subtask finishes may be less risky because its outcome can be represented as a validated artifact. A trigger should consider both token pressure and the task's transition structure.

For a fixed continuation policy and the same pre-edit history H_t, let $C^{(1)}_{t:T}$ and $C^{(0)}_{t:T}$ denote potential continuation costs under editing and retaining the current view, respectively. Both costs include the transformation itself and subsequent repair work. A controller may compare

$$\Delta_t = \mathbb{E}\Big[C^{(1)}_{t:T} - C^{(0)}_{t:T} \mid H_t\Big]. \tag{12.11}$$

Choosing an edit when $\Delta_t < 0$ minimizes this local two-action cost only if the estimates are correct, both actions satisfy the outcome and authority constraints, and the horizon captures relevant consequences. Observational averages over previously edited and unedited tasks do not identify this contrast when selection differs. In practice, these estimates are uncertain. A trigger can include a margin that reflects uncertainty, or it can defer compaction until a task checkpoint establishes a stable state. Neither device eliminates the need to test delayed failures.

Reward shaping can help assign local feedback to context actions, but it can also change the optimized behavior. Penalizing context length rewards short inputs even if later repair is expensive. Rewarding successful

retrieval can encourage unnecessary retrieval. A better diagnostic records downstream task outcome, semantic information loss, tool interactions, and cumulative cost separately. A scalar training objective may combine them, while the evaluation retains the separate quantities so tradeoffs remain visible.

Counterfactual continuation is particularly informative. Save the environment state before an edit and evaluate continuations with the original view, the edited view, and an edited view augmented with the omitted decisive record. If the third condition repairs a failure, the evidence supports a context-loss mechanism. If it does not, the failure may involve reasoning, environment stochasticity, or a changed action policy. Environment restoration must itself be validated; otherwise the counterfactual comparison changes more than the context.

Training introduces an additional requirement: each generated token must be associated with the context actually visible when it was produced. MemoryWalker identifies conditioning errors when edited rollouts are flattened into a final compressed trace or a full physical history [23]. Chapter 17 develops the corresponding likelihood contracts. The point for trigger design is that an edit is not only an inference-time event. It changes the data structure required to train correctly on the resulting trajectory.

Practical controllers should therefore emit an edit manifest containing the decision time, removed spans, replacement text, source identifiers, and view version. This manifest supports both failure analysis and correct rollout reconstruction. Recording only the final summary loses the temporal information needed to determine whether a token was trained under a future edit or under content that had already disappeared.

Figure 12.1 isolates the mechanism and its stated information boundaries.

12.5 Editing and cache validity

Text editing changes the input sequence, while cache reuse assumes particular computational dependencies remain valid. In a standard causal transformer, a cached hidden state for a later token can depend on earlier tokens through attention and residual computation. Deleting an earlier span does not generally leave later states unchanged, even if the surviving text is byte-identical. Reusing cached values therefore requires a justified dependency rule, not merely matching the visible suffix.

For a simple prefix-cache implementation, an edit at position j invalidates the reusable prefix from that position onward. If L is the new length, the

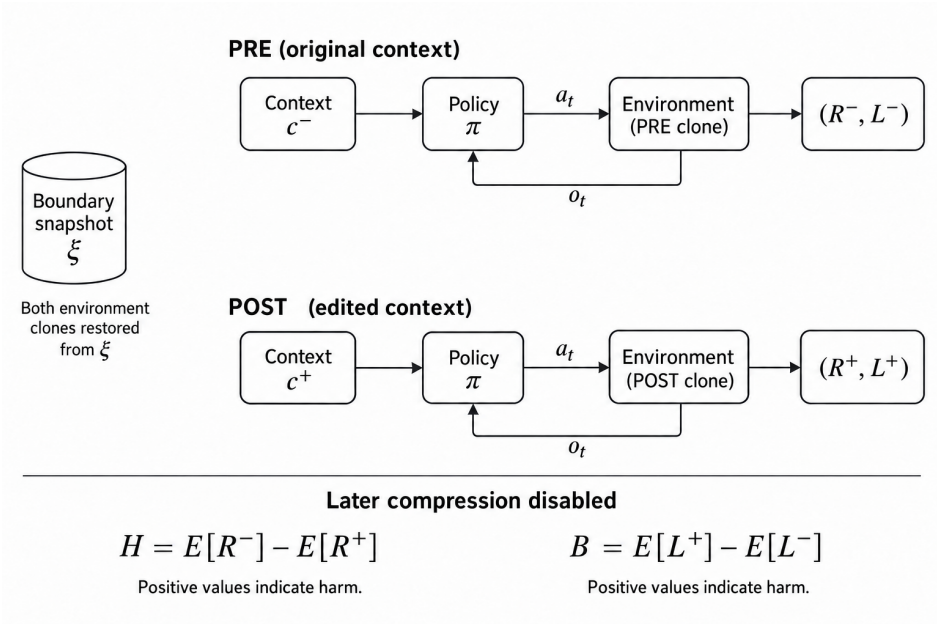

Figure 12.1: A paired continuation diagnostic changes the working view while restoring the same external boundary state and holding the continuation policy fixed. Disabling later compression isolates the current replacement. Outcome hazard $H = \mathbb{E}[R^-] - \mathbb{E}[R^+]$ and interaction burden $B = \mathbb{E}[L^+] - \mathbb{E}[L^-]$ quantify different consequences. Estimation requires validated restoration and repeated continuations; no outcomes are depicted. Original schematic informed by PAIR [120].

re-prefill work depends on $L - j$ as well as newly generated tokens. A context policy that repeatedly edits early content can reduce token count while destroying reuse. This explains why a representation optimized only for shortness can be expensive under an actual serving architecture. The relevant objective includes both retained size and edit location.

PagedAttention addresses logical and physical allocation and sharing of cached blocks [59]. It does not, by itself, establish that arbitrary semantic edits preserve the numerical validity of a cached block. A page allocator may retain bytes correctly while the model computation those bytes represent is no longer appropriate for the edited sequence. Allocation validity and inference validity must be checked separately.

CLM Suffix Cache Reuse is an explicit approximation: surviving suffix states retain dependencies on the pre-edit prefix [26]. Its full-attention path re-rotates relocated keys for their new positions, while the studied

hybrid path resumes pre-edit recurrent state instead of recomputing inserted content in linear-attention layers. Position adjustment does not remove stale content dependencies. Report this quality–compute tradeoff separately from exact prefix reuse and from measured end-to-end latency.

An edit-aware serving contract can attach a dependency fingerprint to every reusable region. The fingerprint identifies model version, tokenizer, position convention, attention mask, and relevant predecessor state. A reuse decision is valid only if the implementation's equivalence conditions hold. Exact fingerprints may be expensive; conservative invalidation sacrifices reuse to preserve correctness. Approximate reuse is possible, but its approximation should be evaluated as a quality tradeoff rather than described as lossless without proof.

Measure context-control latency end to end. Include the editing model call, storage writes, cache invalidation, re-prefill, extra retrieval, and any retries after a damaged summary. Report warm and cold behavior separately. A policy that looks strong when every prompt receives a fresh prefill can perform differently in a service with extensive prefix reuse. The comparison must reflect the intended deployment instead of selecting the serving regime most favorable to one method.

12.6 A worked decision-preservation analysis

Consider a task with three obligations: modify a parser, preserve a legacy behavior, and produce a reviewable patch without publishing it. The raw history includes a failed test, a revised hypothesis, and a user correction that narrows the authorized output. A token-oriented summarizer may retain the patch plan and the latest test output while dropping the legacy requirement and the authorization correction as old discussion. The resulting input is shorter but misrepresents the permitted output scope and success requirements in the model-visible view.

Represent the validated state as $z = (G, O, A, E, U)$: goals G, obligations O, authority A, evidence references E, and unresolved hypotheses U. A proposed compaction operator T should preserve O and A exactly, preserve enough of E to justify current conclusions, and mark unresolved hypotheses as unresolved. This is an application-specific contract. It does not claim that these five fields are sufficient for every task; it states which distinctions this parser workflow requires.

Suppose the original and compacted policies induce action distributions p and q at the next decision. For a bounded immediate loss $0 \le \ell \le 1$, their expected losses differ by at most $\mathrm{TV}(p, q)$. If a validator can establish

a bound on distribution change, that bound controls immediate risk under the stated loss. It does not automatically control the complete task, because a small action change can place the environment in a different region. Long-horizon guarantees require bounds on transition sensitivity or repeated coupling assumptions, which are rarely available for arbitrary tools.

For deterministic safety constraints, comparing distributions is unnecessary. Suppose every tool call passes the same monitor, whose allowed-action set is determined exclusively by the authoritative grant state and the current enforcement configuration. Holding those inputs fixed preserves the same syntactic boundary even when compaction changes the policy. Copying an old A into a summary does not satisfy this condition after a grant is revoked; the monitor must consult current authority at dispatch, independently of the edited prose. Semantic correctness remains open: the agent can take an allowed but wrong action. This separation is valuable because it prevents summary quality from being responsible for all enforcement. Authority should survive in a typed control layer even when task evidence is compressed.

The corresponding experiment compares four views: full history, generic summary, typed-state summary, and typed state plus targeted raw retrieval. Keep the model, tools, and environment snapshot fixed. Randomize whether the legacy constraint appears early or late and whether the latest correction expands or narrows authority. Score patch correctness, publication behavior, retrieval use, and cost separately. This is a proposed experiment. No numerical outcomes are implied by the formulation.

A successful typed-state condition would support the mechanism that explicit obligations survive compression in this workload. It would not establish that a manually designed schema generalizes to unknown scientific or legal tasks. A failure would still be useful if it identifies a missing state field or an incorrect validation rule. The scientific objective is to expose which information transformations preserve behavior, rather than to award a generic summary the label of memory.

12.7 Evidence comparisons and causal boundaries

ContextPilot separates the editing toolset from the learning rule. Its tools include structured memory, notes, planning, and reversible context offload. Training uses snapshots at editing boundaries, masks previously optimized tokens, and allocates continuation sampling around consequential editing actions [121]. Its action-level feedback addresses the fact that final task correctness is an imperfect label for every earlier edit. This does

not make the feedback causal ground truth: continuation outcomes still depend on the downstream policy and environment. A comparison must separate the expanded toolset, supervised initialization, branch allocation, and advantage estimator rather than attributing their joint effect to one component.

CompactionRL jointly trains execution and summary generation with token-level normalization. Its cross-segment credit correction is approximate, not exact full-trajectory GAE [122]. For segment s with $n_s \geq 1$ optimized tokens, discount $\gamma \in (0, 1]$ and trace parameter $\lambda \in (0, 1]$, the method scales its locally computed advantages by the number of later optimized tokens:

$$\widehat{A}_{s,i} = (\gamma\lambda)^{N_{>s}} A^{\mathrm{loc}}_{s,i}, \qquad N_{>s} = \sum_{j>s} n_j. \tag{12.12}$$

This restores the terminal-reward discount distance when the shared reward is placed at each independently optimized segment end. It does not recursively propagate all later TD residuals into earlier segments. The authors trade that reconstruction for parallel segment processing; a reproduction must distinguish reward-distance correction from full-trajectory GAE. Their coding evaluation uses a specified 200-task SWE-bench Verified subset, rather than the full benchmark. Successful joint optimization does not establish preservation of arbitrary future constraints.

TRACE v2 evaluates candidate summaries through paired closed-loop continuations from the same AppWorld state while keeping the agent and proposer weights frozen [123]. Its local verifier counts the union of blocked actions and canonicalized repeated tool calls over a fixed horizon; this does not detect every silent state corruption. Preferences between summaries reuse the same PRE continuations. The proposer receives the downstream system prompt, compressor input, and better–worse summary pairs, but not rollout traces or the verifier decomposition. Five candidate compression templates are then selected on development-set success in both repeated runs, not success in either run; the selected template is frozen for test evaluation. This separates boundary-local supervision, prompt selection, and held-out testing. The evidence remains concentrated in AppWorld, and restoration fidelity and verifier scope limit the causal interpretation.

PAIR likewise uses counterfactual continuations to identify damaging individual compressions and adapts sections of a structured prompt [120]. It distinguishes repeated-run reliability from finding one successful attempt. Its offline continuation search and restored-state access are part of the adaptation cost. These requirements can be reasonable in an experimental environment while unavailable for an external business tool with irreversible

side effects. TRACE and PAIR share research lineage, so their agreement is evidence of a developing mechanism rather than two independent replications.

These mechanisms expose a concrete disagreement with purely semantic compression objectives. A summary can retain entities and facts yet obscure whether an action already happened, whether a plan is obsolete, or whether the task is complete. In that case the failure is execution-state mislocalization, not necessarily absence of a fact. Evaluating summary similarity would miss the directional role of action-observation history. The appropriate response is to preserve validated progress and effect receipts, then test whether the agent stops, resumes, and avoids repeating completed work. This is a synthesis of the mechanisms above, not a new measured result.

ACM provides author-reported evidence that explicit context actions can improve its evaluated tasks while reducing peak active context [119]. The same study also illustrates why resource metrics must remain separate: in its reported BrowseComp-Plus comparison, tool calls increase from 19.5 to 46.2 per episode. This observation does not negate a context benefit; it limits the inference that smaller context necessarily means lower total latency or expense. These results have not been independently reproduced for this monograph.

Comparing learned compaction with deterministic summarization requires matching more than model identity. A learned controller may have seen environment-specific trajectories, received a larger training budget, or benefited from a retriever tuned to the evaluation interface. An ablation that removes the controller but also removes access to raw history changes two mechanisms simultaneously. Preserve the same storage and retrieval interfaces while varying the decision policy if the claim concerns control rather than memory availability.

Author-reported improvements across several benchmarks can support broader relevance, but shared infrastructure creates dependence between results. The same backbone, tool wrapper, answer extraction, and evaluator may appear across tasks. A cross-domain average does not turn those results into independent replications. Report task-specific outcomes and explain which components are shared. If one task improves and another regresses, the average should not hide the mechanism-specific disagreement.

The key comparison axes are behavior preservation, resource accounting, reversibility, training consistency, and implementation transparency. Full-history methods preserve observations but can suffer distraction and expense. Summaries reduce active input but can lose qualifications. Retrieval retains

scalable access but can miss implicit relevance. Editable contexts increase flexibility but complicate cache and rollout semantics. These are structural tradeoffs, not a ranking without workload assumptions.

Evidence should also identify what remains unavailable. If a paper describes prompts but releases no execution adapter, code-level reproduction is incomplete. If code exists but requires a private checkpoint or undisclosed environment, the visible repository does not close that gap. A reproducibility claim should identify model weights, data, tool fixtures, edit policy, and evaluation version. Successful compilation of a harness is not successful reproduction of its scientific results.

The practical implication is to choose the simplest context-control mechanism that satisfies measured task and authority constraints. A sophisticated learned editor can be appropriate when decision value changes rapidly and enough training feedback exists. A deterministic structured state can be preferable when obligations are explicit and failures are costly. The evidence needed to choose between them is a matched workload study, not the elegance of the editing interface.

An October 2026 alternative is transparent-compact for the Pi coding agent. The inspected paper evaluates version 0.8.0, retaining user messages and assistant prose verbatim while masking tool outputs and reasoning, explicitly marking removed material, and providing delegated recovery [118]. Its recovery action is `context_lookup`, with bounded fallback access. The guarantees are budget-limited: when retained prose reaches the configured cap, truncation begins. It therefore does not establish indefinite verbatim memory. Its next-turn probes concern commitments and planted facts with file and shell tools removed to isolate conversational memory. They do not establish complete software-engineering task success.

This method exposes a different comparison axis from learned abstraction. It preserves what was said while declining to treat that prose as verified truth. A retained assistant claim can still be wrong, and a removed tool result can require recovery before the claim is reused. The working view gains transparency about missing evidence at the cost of carrying more conversational text. Whether that tradeoff improves a task depends on the importance of exact commitments, the recoverability of observations, and the model's tendency to act confidently on an ambiguous summary.

The pinned CLM implementation supplies a concrete interface boundary. In `clm/clm_harness/context_env/env.py`, `ContextEnv._apply_edit` round-trips candidate messages through `parse_back`, counts tokens, and invokes `edit_gate.may_grow` before replacement [124]. The gate constrains growth; a shrinking edit that remains over budget can still be

accepted with a warning, so this helper alone is not an absolute budget-admission guard. In `clm/clm_harness/context_utils/context_string.py`, `parse_back` restores the protected prefix but maps every editable non-assistant role to user text and does not reconstruct structured tool calls. Legal serialization therefore does not preserve all original role and effect provenance. Evaluate those changes separately from semantic commitment preservation and from the stricter proposed admission contract in Eq. (12.8).

12.8 Failure recovery and implementation discipline

Context transformations should be recoverable operations. Persist the source view and its edit manifest before replacing the active context. Commit the replacement only after structural validation, and retain a rollback reference until the affected task stage has been verified. This does not require keeping every cache allocation forever; it requires retaining enough evidence and metadata to reconstruct the decision input. Storage retention and privacy constraints still apply.

Use separate error classes for failed summarization, invalid serialization, missing raw evidence, retrieval failure, and an edited view that violates a pinned obligation. A generic "context error" obscures whether the model, controller, or storage layer is responsible. Recovery can then be targeted: restore the prior view, fetch an omitted record, regenerate an abstraction, or stop before a consequential tool effect. Blindly retrying the same damaged summary can amplify the failure.

Concurrent editing requires ownership. If a worker updates a plan while a controller compacts an older version, a last-write-wins replacement can erase legitimate progress. An optimistic commit should compare the base view version with the current version and reject or merge a stale edit. Semantic merges deserve special treatment: two summaries can each be locally valid while their union contains contradictory commitments. A storage-level conflict resolution rule cannot determine which commitment the user currently authorizes.

Human-readable state remains useful for audit, but readability should not replace machine-checkable boundaries. Identifiers, timestamps, tool-effect receipts, authority, and completion predicates can be represented in typed fields. Generated prose can explain those fields without becoming their sole source of truth. The boundary between recorded fact and model inference should survive every compaction. This reduces the chance that

repeated summaries gradually promote a tentative hypothesis into an asserted observation.

The implementation path is incremental. First instrument the existing context assembly. Then introduce explicit source references and pinned obligations. Next add reversible offload with deterministic triggers. Only after those controls are measurable should a learned editing policy be compared under matched budgets. This sequence is a systems deduction about observability and attribution, not a claim that one particular product follows it or that it guarantees success.

The strongest outcome of context engineering is not a permanently small prompt. It is a working view whose contents, omissions, cost, and authority are deliberate and inspectable. Such a view supports the policy's current decisions while preserving a path back to evidence when assumptions change. Durable sessions in Chapter 13 provide the substrate for that path; persistent memory in Chapter 14 determines which evidence and abstractions remain useful across tasks.

Chapter 13

Session Semantics, Event Histories, and Recoverable Execution State

The 2026 ACM implementation exposes a consequential boundary between its live conversation, retained raw history, and checkpoint interface [125]. Continuity therefore depends on what a snapshot actually restores, rather than on the existence of a session identifier. That boundary connects model context directly to event ordering, external-effect reconciliation, concurrency, and retention semantics.

13.1 Session identity is a semantic boundary

A session is a bounded continuity relationship among an actor, an objective, a sequence of observations, and the authority under which actions occur. It is not identical to one model request, one browser tab, or one transcript. A task can span several model calls and tool processes; one user session can contain several tasks; a durable agent can serve many sessions while maintaining selected cross-session memory. Conflating these identities makes it difficult to determine what should survive interruption and what information may legitimately transfer to a new interaction.

Use separate identifiers for tenant, user or principal, session, task, run, model call, and external effect. A run is a particular attempt to advance a task under a chosen configuration. Retrying a run does not create a new user objective, and creating a new session does not automatically revoke or inherit every earlier authorization. These relationships should be explicit data rather than inferred from a conversational summary. Otherwise, a model may treat an old permission as current merely because both appear in the same retrieved context.

A useful state decomposition is

$$S_t = (I_t, G_t, A_t, P_t, E_t, V_t), \tag{13.1}$$

where I_t contains identities and scope, G_t the goal and completion predicate, A_t authorization, P_t progress and pending work, E_t evidence and effect receipts, and V_t version references. The decomposition is a proposed systems

specification. It does not claim that a tuple of these fields is a universally sufficient control state. Its purpose is to prevent model-visible history from being the only place in which operational commitments exist.

Session termination also needs a definition. A session can be paused, abandoned, completed, or closed for retention purposes. Those states have different consequences. Pausing should preserve pending work without implying that an external operation has been canceled. Completing a task should identify which success predicates were checked. Closing a session may remove active context while retaining legally or operationally required effect records. Memory consolidation can occur after closure, but only under the applicable retention and sharing policy.

Managed agent documentation and persistence frameworks expose concrete session and checkpoint mechanisms [126, 127]. These interfaces support implementation, but they do not establish the semantic correctness of a resumed task. A checkpoint can faithfully restore an obsolete plan. A session service can preserve every message while allowing a stale authorization to influence a new run. The application must define which state is authoritative and which transformations are permitted at each boundary.

13.2 Events as evidence rather than prose

An event record states that a particular observation or transition was recorded under specified conditions. It should distinguish a requested action from a completed effect, and a model's prediction from an environment observation. A proposed minimal envelope includes an event identifier, causal parents, producer, observed time, effective time where known, payload type, schema version, and integrity metadata. Sensitive payloads can be stored separately behind access controls while the envelope retains a reference and retention status.

The distinction between observed and effective time is essential for corrections. A user can report today that a preference changed last week. A system can discover a failed external operation after it previously recorded a timeout. Ordering by ingestion time alone does not establish which belief is currently valid. The event log should preserve both when information entered the system and what interval it describes. A query for the latest known fact and a query for what the agent knew at a past decision then become different operations.

Let $e_i \prec e_j$ denote a causal dependency. A tool result depends on its invocation, and a summary depends on the source events it abstracts. Causal order need not coincide with a single globally synchronized clock.

Lamport's treatment of event ordering supplies the foundational distinction between causal precedence and a compatible total ordering [128]. For agent logs, the practical consequence is to retain causal links rather than assuming that wall-clock sorting reconstructs the decision process.

Typed events enable different validation rules. A model proposal can be accepted as a proposal without being accepted as a fact. A tool receipt can require a validated tool identity and matching invocation identifier. A completion event can require specific tests. An authorization event can require an authorized principal and a scope. If all four are serialized as untyped text, later summarization can accidentally promote the first into the last. Data typing reduces this risk but cannot determine whether the content of a tool observation is itself truthful.

An append-only evidence log does not imply indefinite retention of all raw payloads. Deletion, redaction, and access revocation can be represented by policy-controlled transformations while preserving the fact that an earlier decision referenced an object that is no longer accessible. The system should avoid claiming that a deleted payload remains reconstructible if its retention policy intentionally removes it. Auditability and privacy create a real design tradeoff; an implementation must state which forms of evidence survive.

The resulting event history is different from a polished narrative. It can contain conflicting reports, partial failures, and superseded plans. Those are necessary evidence for diagnosing why the agent acted as it did. A model-facing summary is a derived view of that history, with its own version and provenance, rather than a replacement for the record of events.

13.3 Reducers checkpoints and replay

Define a reducer F_v that constructs application state from a prior state and an event under schema version v:

$$S_t = F_v(S_{t-1}, e_t), \qquad S_t = F_v^*(S_0, e_{1:t}). \tag{13.2}$$

If F_v is deterministic and all inputs are retained, replay can reconstruct the same application state. The assumptions matter. A reducer that silently consults the current clock, a mutable external service, or a newly upgraded language model is not deterministic under the recorded event stream. Nondeterministic results must be materialized as events if replay is expected to reproduce their consequences.

A checkpoint is a persisted state snapshot at a known event boundary. It can reduce replay cost from processing the complete history to processing the

suffix after that boundary. Its identity should include the reducer version, event position or causal frontier, relevant schema versions, and references to external artifacts. A checkpoint that stores only the visible conversation can omit pending tool operations, unpublished artifacts, and reservations on a resource budget. Restoring that conversation may reproduce the model input while failing to restore the actual task.

There are at least three replay objectives. State replay reconstructs the recorded application state. Decision replay reconstructs the exact model input used for a past action and, where possible, its sampled output. Environment replay reexecutes actions against an equivalent environment snapshot. State replay does not imply decision replay if tokenization or context assembly changed. Decision replay does not imply environment replay if tools read changing data or produce irreversible effects. Evaluation should identify which objective it achieved.

A proposed checkpoint protocol writes an event frontier, state snapshot, artifact manifest, and integrity digest as one committed checkpoint object. An interrupted write should remain uncommitted rather than becoming an apparently valid partial snapshot. Restore verifies that all required objects exist and satisfy their recorded schemas. Missing optional caches can be recomputed; missing effect receipts or authority records may require stopping or external reconciliation. The distinction should be encoded instead of handled through a generic best-effort restore.

Checkpoint frequency trades storage and synchronization work against recovery time. If failures occur with a workload-dependent rate, a more frequent checkpoint can reduce lost computation but increase steady-state overhead. Language-agent workloads complicate this tradeoff because expensive steps are not uniform: one model call may be cheap while one external operation changes the world. A checkpoint before a consequential effect can be more valuable than one after a fixed number of tokens. Recovery cost should therefore be measured in both computation and semantic work lost.

LangGraph distinguishes checkpoints at super-step boundaries from per-task pending writes. Its documented durability modes also differ: synchronous mode completes persistence before the next step, asynchronous mode overlaps it with execution, and exit mode delays persistence until graph exit. An in-memory saver does not survive process restart. Moreover, replay from an earlier checkpoint reexecutes subsequent nodes, including model calls and API requests [129]. These are distinct recovery contracts, not interchangeable settings. They do not make an external effect atomic with checkpoint storage. The application must state the selected backend, durability mode, replay frontier, and treatment of side effects.

A concrete distinction appears in ACM's inspected `HistoryManager`. Its live message list is separate from an append-only raw mirror; `compress_range` removes a half-open slice only from the live list. However, `snapshot` returns the live messages, and `load_snapshot` initializes both lists from that snapshot. A different constructor, `from_saved_history`, accepts a separate raw history [125]. Therefore, using the snapshot path alone after compression need not preserve the original raw archive. This is a source-level contract difference, not a measured failure in the paper's experiments. A durable wrapper should persist both views when its recovery guarantee includes original evidence. Testing only that the live conversation restores would miss the archive distinction.

13.4 External effects and the ambiguous timeout

The difficult interruption occurs between an external effect and recording its receipt. Suppose an agent submits a transaction, the remote service commits it, and the network response is lost. The local system observes a timeout. Retrying can duplicate the effect; assuming success can advance a task whose operation actually failed. A session transcript containing "tool timed out" does not distinguish these worlds. This is an information problem before it is a language-model problem.

Represent an effect by an operation identifier k, a payload digest d, a target resource, and a state from a declared effect state machine. A useful application-level sequence is proposed, authorized, dispatched, confirmed, and reconciled. "Unknown" is a legitimate state after a timeout. It should not be silently mapped to failure. If the remote service supports a stable idempotency key and records the corresponding result, retries with the same key can retrieve or complete the same logical operation under that service's contract. Without such support, exactly-once external execution cannot be inferred from local retries.

The ambiguity admits a minimal impossibility witness. Let w_0 be a world in which the first attempt did not commit and w_1 one in which it committed but its receipt was lost. Assume that both produce the same local observation and that an unkeyed retry commits once in either world. Let D indicate a duplicate effect and O an omitted required effect. If the recovery policy retries with probability r, then

$$P(D = 1 \mid w_1) = r, \tag{13.3}$$

$$P(O = 1 \mid w_0) = 1 - r, \tag{13.4}$$

$$\max\{r, 1 - r\} \geq \tfrac{1}{2}. \tag{13.5}$$

Omission here means failing to realize the required effect under a policy that terminates after this recovery choice. No policy based solely on the indistinguishable local record can guarantee both completion and absence of duplication. Indefinite abstention avoids a new effect but does not establish completion. A status query, a service-enforced operation key, or additional external evidence changes the information structure and can break the witness. This is an analytical construction, not an observed failure rate; its bound must not be applied to services whose retry contract differs.

The following proposed state transition separates validation, durable intent, external dispatch, and evidence-based reconciliation. Let u be a command instance including its logical intent identity, J its local journal, and α current authority. The predicate $v(u, \alpha) \in \{0, 1\}$ checks schema and access to the corresponding journal scope. For $v(u, \alpha) = 1$, let $k = \eta(u)$ identify this logical operation, $p(u)$ its payload, and $d = h(p(u))$ the payload digest. The stable identifier is reused on recovery; equal payloads alone do not identify the same user intent. Define $\gamma(u, J, \alpha) = 1$ exactly when $v(u, \alpha) = 1$ and either k is absent from J or its record binds the same logical intent identity and exact payload as u; otherwise $\gamma(u, J, \alpha) = 0$. A digest match alone does not establish that binding. Let $\sigma_J(k) \in \{\varnothing, \iota, \upsilon, \kappa_+, \kappa_-\}$ denote absence, persisted intent with no acknowledged dispatch marker, marked possible dispatch with unresolved outcome, confirmed effect, and confirmed absence of the specified effect. All callers must obey the marker-before-call ordering; otherwise an intent-only record does not establish absence of dispatch. For an absent or intent-only operation admitted by $\gamma = 1$, the fresh-dispatch branch Φ_0 is

$$\begin{aligned} J_1 &= \mathcal{I}(J; k, d, u, \alpha), \\ J_D &= \mathcal{M}(J_1; k), \\ r &\sim \mathcal{D}(\cdot \mid k, u), \qquad J_D \neq \bot_J, \quad a_D(u, \alpha) = 1, \\ J_2 &= \begin{cases} \mathcal{P}(J_D; k, r), & \chi(r) \neq \bot, \\ \mathcal{U}(J_D; k), & \chi(r) = \bot, \end{cases} \\ \Phi_0(u, J, \alpha) &= \sigma_{J_2}(k). \end{aligned} \tag{13.6}$$

Here $\mathcal{I}$ durably records the intent and $\mathcal{M}$ durably marks possible dispatch, changing ι to υ before the call. The local branch selection and marker transition assume a serialized dispatcher for each key or an equivalent atomic, fenced claim; separate lookup and unconditional writes do not enforce that assumption. The separate predicate $a_D(u, \alpha)$ checks current authority for new dispatch, including service-side concurrency conditions. If it is false, $\Phi_0 = \bot_A$ and none of the displayed fresh-dispatch operations is evaluated. Reading a terminal receipt is governed by journal access

and need not require the old write permission. The receipt classifier $\chi(r) \in \{1, 0, \bot\}$ denotes confirmed effect, confirmed absence of the specified effect, or unresolved status. The operator $\mathcal{P}$ persists either confirmed class; $\mathcal{U}$ records uncertainty. A transport acknowledgment alone is not effect confirmation. Every journal operator returns an acknowledged journal or $\bot_J$; the next line is evaluated only after acknowledgment. A storage failure returns $\bot_J$ and recovery begins from the last acknowledged journal. A lost receipt can therefore leave a marked, unresolved dispatch despite an external effect. A crash after the marker but before the call is also unresolved: this conservative false ambiguity prevents an unsafe automatic retry.

Let e be evidence acquired through a separately authorized reconciliation path; unavailable or inconclusive evidence leaves υ unchanged. With $\mathcal{R}$ restricted to evidence updates and never issuing the original effect, define

$$\Phi_R(u, J, \alpha; e) = \sigma_{\mathcal{R}(J;k,e)}(k),$$

$$\Phi(u, J, \alpha; e) = \begin{cases} \bot, & \gamma(u, J, \alpha) = 0, \\ \sigma_J(k), & \gamma = 1,\ \sigma_J(k) \in \{\kappa_+, \kappa_-\}, \\ \Phi_R(u, J, \alpha; e), & \gamma = 1,\ \sigma_J(k) = \upsilon, \\ \Phi_0(u, J, \alpha), & \gamma = 1,\ \sigma_J(k) \in \{\varnothing, \iota\}. \end{cases} \tag{13.7}$$

The lookup in Φ_R is evaluated only when $\mathcal{R}$ returns an acknowledged journal; otherwise it returns $\bot_J$. Terminal replay and unresolved reconciliation never call $\mathcal{D}$. In particular, current dispatch authority is necessary for a new effect but does not make an unresolved retry safe. A service-enforced retry contract can justify a different explicitly declared recovery branch; it is not assumed here. These maps specify application obligations, not an atomic transaction across the journal and remote service. Authority must be current at dispatch, even when no conversational state changed. Service-enforced key scope, retention time and payload-conflict behavior bound the idempotency guarantee; a key cached locally cannot impose that guarantee on the remote endpoint. The reconciliation path may query the remote service, inspect a resource version, or request a human decision when the outcome cannot be determined safely. It should not ask the model to guess what probably happened. A model can assist in interpreting receipts, but the effect state must remain tied to evidence.

Linearizability supplies a precise correctness condition for concurrent objects whose operations appear to take effect at a point between invocation and response [130]. It does not automatically extend across unrelated services or provide an application with a global transaction. A local database can atomically record an intent while an external email, payment,

or deployment remains outside that transaction. Calling the local workflow durable does not make the combined operation atomic.

Compensation is another distinct mechanism. A compensating action attempts to restore an acceptable business state after an effect has happened. It need not restore the original world exactly: a sent message cannot be unsent from every recipient's memory. The agent's plan should therefore distinguish reversible local edits, remotely idempotent operations, and effects requiring compensation or manual resolution. Session recovery should preserve those distinctions through summarization.

This analysis explains why successful restart tests can be misleading. If tests interrupt only before a tool call or after its confirmed receipt, they miss the ambiguous interval. Fault injection should include the boundary after remote commit but before local acknowledgment. That experiment directly tests whether the implementation's effect model matches the guarantee claimed for long-running execution.

The loss point in Figure 13.1 separates remote commitment from local knowledge; a missing receipt cannot establish absence of the effect.

13.5 Concurrency ownership and session forks

Parallel workers can advance the same objective while holding different views of its state. One worker may discover a requirement while another modifies the artifact that requirement constrains. If both publish free-form summaries into a shared record, the last writer can erase the first worker's evidence. A shared session therefore needs concurrency semantics, not merely a shared context window.

An optimistic transaction can read state version v, compute a proposal, and commit only if the relevant base version remains unchanged. This prevents a stale proposal from silently overwriting intervening updates. The relevant version need not be the entire session: an artifact, plan node, or authority record can have its own version. Finer granularity improves concurrency but requires an accurate dependency set. If a worker reads an unrecorded constraint, the commit validator cannot know that its proposal became stale.

For a precise local commit rule, let R be the recorded read set, v_r the version read for object r, and $v(r)$ its current version. Let Δ be the proposed update and $C(\Delta)$ a declared invariant check. A candidate can commit to the local transaction only if

$$\operatorname{admit}(\Delta) = \mathbf{1}\{\forall r \in R : v(r) = v_r\}\, \mathbf{1}\{C(\Delta) = 1\}. \tag{13.8}$$

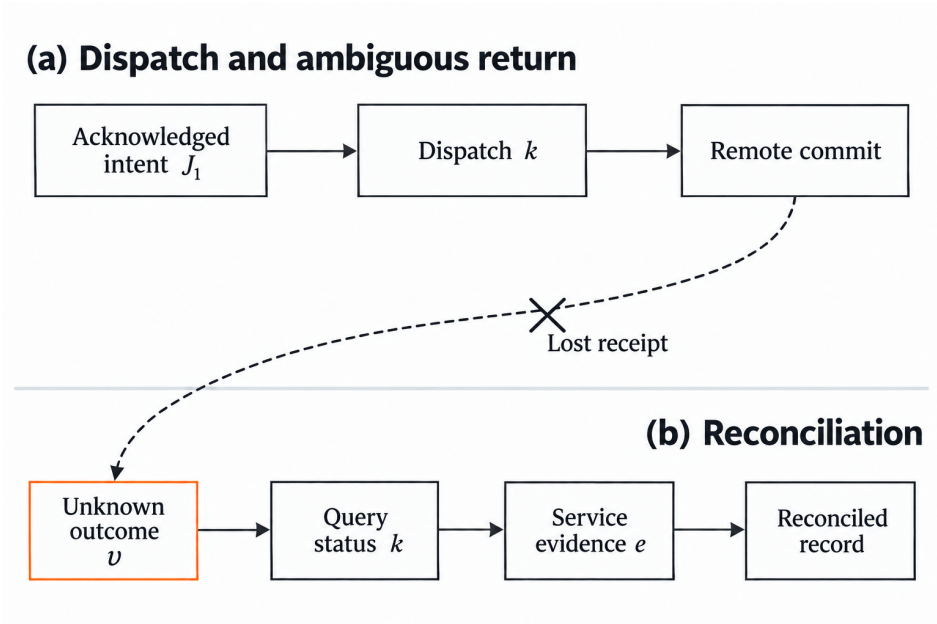

Figure 13.1: An illustrative committed-but-unacknowledged operation. Confirmed intent precedes dispatch; a lost receipt after remote commit leaves the local outcome unknown. A status query obtains additional service evidence for the local record. Without conclusive evidence, that record remains unknown. The pre-call possible-dispatch mark and other admission checks are omitted. Historical replay and fresh dispatch have different authorization requirements. No global transaction or exactly-once guarantee is implied.

This prevents stale-read acceptance only when R contains every relevant dependency and validation is atomic with the local write. It does not certify the semantic truth of the read values. A remote dispatcher additionally needs a service-recognized fencing or version condition if another worker can supersede ownership between local validation and the effect. Merely placing a monotonically increasing token in a prompt does not fence an old worker.

Not every concurrent write is a conflict. Appending independent observations can be safe when each retains provenance and source identity. Replacing a shared completion predicate or deleting a task obligation is more sensitive. Commutativity should be established for the operation's semantics rather than guessed from data format. Two JSON patches that touch different fields can still conflict if one changes the meaning of the

other. For example, changing a tool target while another update broadens a permission can create an unintended combination.

Forking a session creates a branch of reasoning and action history. The branch should record its common ancestor and any inherited authority. A branch used for simulation should not be mistaken for a branch whose actions affected the real environment. Merging branches requires comparing effects and evidence, not concatenating their final narratives. If two workers both report that a document was published, the merger must inspect operation identifiers to determine whether there was one shared effect, two effects, or merely two plans.

Authority also needs version-aware propagation. A user correction that narrows scope should invalidate pending proposals whose admissibility depended on the older scope. Canceling those workers does not prove that in-flight effects were prevented. The system must distinguish cancellation of local computation from revocation enforced at the effect boundary. An authoritative check immediately before dispatch is stronger than a permission check performed only when the worker started.

A proposed test uses two workers with controlled scheduling: one receives a correction, the other attempts a previously permitted action. Vary the point at which the correction becomes visible and inspect whether the monitor, context assembler, and effect dispatcher use the same authority version. This exposes races that ordinary task-success benchmarks rarely exercise. No empirical outcome is implied by the test design; its value is the explicit causal sequence it makes observable.

13.6 Retention deletion and provenance boundaries

Durability and retention answer different questions. Durability concerns surviving failures under a storage contract; retention concerns how long information should remain available and to whom. An agent can be durable while enforcing short retention, or retain data indefinitely while providing weak recovery guarantees. The design should state both instead of using "persistent" as a substitute for either.

A retention model should distinguish raw event payloads, derived summaries, embeddings, caches, effect receipts, and learned parameters. Deleting a source document does not necessarily delete the summary or embedding derived from it. Conversely, invalidating a derived record can prevent future use without physically erasing all copies. A truthful deletion claim must identify its scope and verification boundary. Cross-service backups

and model-training artifacts require different mechanisms from a local database deletion.

Let $D(x)$ denote the set of derived objects depending on source object x. A deletion or access-revocation event can traverse this dependency relation to invalidate downstream objects. The graph need not imply that each derivative can be perfectly unlearned from a model; it identifies objects for which the application has provenance. If provenance was not recorded, the system may not know which summaries contain the deleted fact. This is a concrete operational cost of treating generated summaries as untraceable text.

Access checks should occur when evidence is retrieved and again when it is exposed to a model or another agent. A cached result may have been authorized when created but unauthorized when reused. Partitioning by tenant and project reduces accidental mixing, but semantic records can still reveal information across boundaries if extraction merged facts from multiple principals. The memory writer must preserve ownership or refuse a merge that destroys the ability to enforce access.

Past-decision audit creates a difficult case. An organization may need to explain why an agent acted while also respecting deletion of the underlying sensitive content. A record can preserve that a decision used an object with a given identifier and policy state without retaining the full payload. Such an audit supports a narrower claim than exact reconstruction. The distinction should be reflected in the recovery interface: a deleted evidence object should produce an explicit unavailable state, not a fabricated reconstruction.

Session summaries should therefore carry retention and provenance metadata alongside their semantic content. Consolidation into cross-session memory is a new use of information, not an automatic entitlement granted by the existence of a transcript. The application must decide whether the information belongs to the user, the project, a shared team, or a short-lived task. Chapter 14 analyzes how those choices interact with semantic and episodic memory formation.

13.7 A failure-oriented evaluation protocol

A proposed session evaluation begins with a set of invariants rather than a single completion score. Examples include preserving the current goal, retaining unresolved obligations, keeping authority current, avoiding duplicate external effects, and making every confirmed effect traceable to an authorized intent. Each invariant should identify the component responsible for enforcement and the evidence required to check it. An invariant that

depends entirely on a model judge should be labeled accordingly.

Create interruption points before and after each event append, checkpoint commit, model response, tool dispatch, and effect receipt. Inject failures that lose local process memory, delay messages, duplicate deliveries, and reorder independent observations. Use deterministic tool fixtures where the objective is to isolate session mechanics. Then repeat selected cases against realistic external services or environment snapshots to test whether fixture assumptions hold. Both stages matter: the first improves diagnosis, while the second tests external validity.

A current managed-runtime boundary makes this distinction testable. OpenAI's Agents API documentation separates live events from saved items: reconnecting restores saved work rather than replaying every missed event. It also warns that an idle session or a completed turn does not prove every tool succeeded. Required input is read from the current session, not reconstructed solely from historical items [131]. A reconnect test should therefore reconcile item identifiers, current pending requests and effect evidence separately; retransmitting a prior action because a stream ended is not a valid recovery rule.

Metrics should separate restoration correctness, resumed-task success, duplicated effects, lost obligations, reconciliation frequency, and recovery latency. A fast recovery that repeats an irreversible action is not operationally successful. A correct recovery that waits for external reconciliation may be slower but necessary. Report the rate of indeterminate outcomes rather than removing them from the denominator. This prevents an implementation from appearing reliable by declining to classify its hardest failures.

The baseline cohort can include restart from the initial task, restore from a conversation-only checkpoint, restore from typed state plus events, and restore with effect reconciliation. Hold the policy and tools fixed where possible. The comparison asks which information and contracts are required for continuity. It should not be confounded by giving one baseline a stronger model or more generous retries. State exactly which external effects are mockable and which cannot be rolled back.

For concurrency, use a schedule matrix that forces stale reads, conflicting updates, and mid-flight authority changes. Verify that optimistic conflicts are detected and that accepted merges preserve the intended invariants. Storage-level correctness is necessary but insufficient: the semantic content of a memory update can be wrong even when its transaction commits correctly. Combine deterministic checks on identifiers and versions with targeted semantic judgments whose disagreement is reported.

These experiments are proposed and have not been run here. Existing persistence interfaces establish that checkpoints and durable execution can be implemented; they do not settle the workload-specific question of what must be restored. The scientific contribution of a session study is strongest when it connects a particular invariant, an injected failure, and an observed recovery mechanism. An average success score without that chain is weak evidence for durable agent behavior.

13.8 A reference contract for continuity

The following continuity contract is an original synthesis of the state and effect distinctions above. A resumed run should identify its session and task, load a validated event frontier, restore the authoritative goal and permissions, reconstruct the model-visible state through a versioned transformation, reconcile pending effects, and only then choose a new action. This ordering prevents the policy from treating uncertainty about external effects as if it were ordinary conversational ambiguity.

The contract does not require byte-identical future generation. A stochastic model can choose a different valid action after resumption. What should remain invariant is the set of obligations and the evidence about what has already happened. Reproducible sampling can help debugging, but operational continuity is not equivalent to deterministic decoding. A system that produces the same text while repeating an external action has reproduced the wrong aspect of the run.

Similarly, a session can migrate to a different model while retaining application continuity. That migration changes the policy interpreting the restored state and can alter how summaries, skills, or tool descriptions influence behavior. Version references make the change inspectable but do not prove compatibility. A migration gate should test representative obligations, stale-memory handling, and effect recovery under the new model. Chapter 17 analyzes this as a context-memory-policy compatibility problem.

For implementation, keep three stores logically distinct even if they share a database: immutable or policy-governed evidence, mutable task state, and derived model inputs. Evidence supports audit and reconstruction. Task state supports coordination and recovery. Derived inputs support efficient inference. Collapsing them into a single transcript makes every summarization operation responsible for persistence, authority, and learning provenance simultaneously. Separating the contracts makes failures attributable and allows different retention policies.

There is no implication that every agent application needs the strongest distributed guarantee. A read-only literature assistant can tolerate some duplicated searches, while a system dispatching externally consequential operations needs a more explicit effect model. The appropriate guarantee follows from the workload's failure costs and authority boundaries. The important scientific requirement is to state that guarantee precisely and test the intervals in which it could fail.

Durable state supplies continuity of evidence and obligations; it does not supply correct beliefs by itself. A faithfully restored session can still contain a mistaken inference or an obsolete semantic memory. The next chapter examines how memories are admitted, revised, and used, while Chapter 32 extends the operational analysis to deployment monitoring and rollback. Together these distinctions keep recovery claims bounded by the state that is actually preserved.

Chapter 14

Persistent Memory: Writing, Consolidation, Validity, and Influence

RecMem consolidates through recurrence-sensitive write stages, whereas October's Hy-Memory learns a memory writer from attributed feedback [132, 133]. InMind shows why stored or directly recallable information must still be tested for appropriate indirect influence [134]. The comparison unit is consequently the complete write–represent–retrieve–use lifecycle, including validity, delayed feedback, and the cost of maintenance.

14.1 Memory functions and physical substrates

Agent memory is a collection of mechanisms for preserving, transforming, locating, and applying information across decisions. Working memory describes the task-relevant information currently available for control. Episodic memory preserves contextualized experience, including what was attempted, observed, and concluded in a particular situation. Semantic memory represents more reusable propositions or abstractions. These are functional distinctions. A vector database can store episodes or semantic statements, and one text document can contain both. The storage technology does not determine the memory function.

Biological terminology can motivate questions about consolidation, salience, interference, and forgetting, but it does not establish a shared implementation. A transformer context is not a demonstrated model of human working memory, and a graph of extracted facts is not a biological semantic system. Parametric knowledge, key-value caches, external records, and application state have different update and access mechanisms. The memory surveys provide useful taxonomies of forms, functions, and dynamics while also revealing how heterogeneous the underlying mechanisms are [15, 135].

Represent a memory object as

$$m = (x, k, p, \tau_v, \tau_o, \sigma, \omega, v), \tag{14.1}$$

where x is content, k its functional type, p provenance, τ_v valid time, τ_o observation time, σ confidence or status, ω ownership and access scope, and v its version. This schema is proposed synthesis. It is deliberately richer than an embedding and text pair because future decisions can depend on whether a record is a direct observation, a tentative inference, or an obsolete preference. Embedding proximity alone does not supply those distinctions.

Working context should contain selected memory objects and the metadata needed to interpret them. A persistent store can retain a much larger set, but persistence does not ensure that the right item is retrieved or used. A useful evaluation separates admission, retention, retrieval, interpretation, and action. A failure at the final action can originate in any earlier stage. Treating all failures as "forgetting" obscures whether the information was absent, inaccessible, misclassified, or correctly retrieved but improperly applied.

Three changes should also remain separate: adding an external record, updating a reusable procedure, and changing model parameters. All can alter future behavior, but only the third changes the parameterized policy directly. A memory method around a frozen model can improve task outcomes without demonstrating continual weight learning. This distinction becomes important when a product or paper uses "learning" to describe a growing external store. The mechanism should be identified before its generality or long-term stability is assessed.

Figure 14.1 shows the distinct curation and selection operations between history, typed stores, and working context.

14.2 Admission as a constrained decision

Writing every observation is inexpensive only when storage, indexing, privacy, and later interference are ignored. Generated tool output can contain duplicate text, stale state, adversarial instructions, and transient failures. Admission decides which records enter a persistent store, at what level of abstraction, and with what provenance. It should distinguish retaining raw evidence for audit from promoting a proposition into reusable semantic memory.

Let $\mathcal{L}(M)$ be expected future decision loss when the evaluated reader and retrieval policy use bank M. Define its decision value as $U(M) = \mathcal{L}(\varnothing) - \mathcal{L}(M)$. With $c_s(m)$ denoting storage and maintenance cost, $c_p(m)$ privacy or policy cost, and $r(m)$ estimated harmful-reuse risk, a proposed

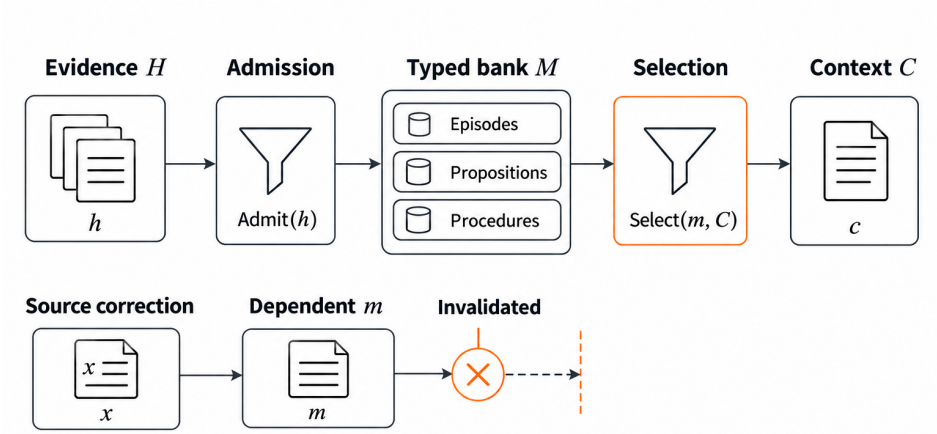

Figure 14.1: Typed retention and invalidation. Evidence h from history H is admitted into a bank M of episodic, propositional and procedural objects. Selection supplies working context c; an invalidated dependent object m is blocked pending its specified revalidation. Publication status and provenance remain part of each object record. Current authorization is a separate boundary omitted here. Arrows show lifecycle dependencies, not guaranteed truth or universal conversion between classes. Here C names the working-context component and c its constructed content.

admission objective is

$$\max_{M} \left\{ U(M) - \sum_{m \in M} c_\lambda(m) \right\}, \tag{14.2}$$

$$c_\lambda(m) = \lambda_s c_s(m) + \lambda_p c_p(m) + \lambda_r r(m), \tag{14.3}$$

subject to access and retention constraints. The nonnegative weights put the penalty terms on the decision-loss scale; their additive form is an accounting approximation, not an assumption that decision value is additive. The value of admitting m is conditional on the bank, $\Delta(m \mid M) = U(M \cup \{m\}) - U(M)$. An identifier and its disambiguating scope can each be useless alone and decisive together. A stale instruction can instead make adding a record harmful. Neither monotonicity nor diminishing returns follows from the memory label. These quantities depend on future tasks and the consumer

policy, so the objective specifies an estimand rather than a solved scorer. A one-time authority correction should remain a control constraint even when a learned utility estimate is low.

Recurrence is one practical proxy for consolidation value. RecMem buffers interactions with embeddings and triggers more expensive episodic and semantic extraction around recurring related content [132]. Its semantic refinement revisits associated raw interactions to recover details omitted by an episodic abstraction. This trades eager extraction for a recurrence-conditioned pipeline. The paper evaluates conversational-memory tasks, not indefinite autonomous execution. Its heuristic thresholds and the possibility of rare but important information limit generalization from average recall results.

The inspected implementation makes the gate concrete. In the `RecMem` class, `add_memory` first attempts an episodic merge. Otherwise it searches raw-memory neighbors, filters by a similarity threshold, and compares the resulting count plus the incoming item with a consolidation count. The non-triggered path inserts the raw message with a timestamp; the triggered path calls `_consolidate_memory` [136]. Thus the decision depends on both existing episodic coverage and raw recurrence. Describing it only as "three-layer memory" would omit the mechanism that determines its construction cost.

A deployment should add a separate pinning rule for information whose consequence is not well estimated by frequency. This is a design deduction, not a reported feature or result of RecMem. Examples include current authority, explicit user prohibitions, identifiers needed for reconciliation, and unresolved safety constraints. Such objects can remain in typed state even when ordinary conversational content follows recurrence-based consolidation. The resulting hybrid admits that one salience heuristic need not govern every memory class.

Personalized admission has a further feedback asymmetry. The October 2026 gating study formulates storage decisions with delayed, one-sided feedback: subsequent use can label an admitted fact, whereas the corresponding evidence for a discarded fact is unavailable. Its Warm-Start UCB method transfers a prior from other profiles and adapts through confidence-based exploration; the analysis assumes a specified linear utility model and a sufficiently accurate prior region [137]. Neither a missing retrieval hit nor a currently pending label proves uselessness. Moreover, retrieval-confirmed use is a proxy for utility, not an independent measurement of improved user outcomes. Logging the label deadline, retrieval policy, and admission propensity is necessary to distinguish feedback delay from selection bias and from genuine preference change.

The reported LoCoMo comparison with GPT-4.1-mini is instructive because the full-context reference remains stronger in accuracy while requiring no memory-construction stage. Table 14.1 separates accuracy from construction tokens; the latter exclude the complete query-time and infrastructure boundary. The values are author-reported and not independently reproduced.

Table 14.1: RecMem's reported LoCoMo configuration [132]. Construction tokens are in thousands.

Method	Accuracy (%)	Construction tokens
Full context	84.18	0.00
Mem0	62.92	1520.80
RecMem	81.10	193.20

14.3 Episodic records and abstraction loss

An episode should preserve the relationship between the situation, the policy's attempt, the observation, and the evaluated outcome. A useful record distinguishes a successful action from a plausible explanation of why it succeeded. If the environment returned a success signal, the agent can record that signal. Inferring a general strategy from the signal is a further abstraction. Keeping those layers separate prevents one lucky trajectory from becoming an apparently validated rule.

Let a trajectory be $\xi = (o_0, a_0, o_1, \ldots, a_{T-1}, o_T)$ and let $z = s(\xi)$ be its episodic abstraction. A downstream retrieval query may need an exact error message, a resource identifier, or a failed alternative that the summary omitted. The abstraction's quality depends on the future query distribution, as in Chapter 11. A narrative that captures the overall task can still be insufficient for a later repair. Store a pointer to the source trajectory when retention permits, and preserve exact evidence fields that support verification.

RecMem's code illustrates a nontrivial failure boundary. Its consolidation routine sorts existing raw neighbors by timestamp and then appends the incoming record, rather than inserting it into that chronological order. It constructs episodic memories and proceeds to semantic refinement. If episodic generation returns no records, it returns before deleting raw neighbors. If semantic extraction returns no facts, the inspected path can still proceed to remove consolidated raw items [136]. These are implementation observations at the cited revision, not measured paper failures. A backdated

incoming record provides a discriminating test of prompt order versus valid time; generation failure provides a separate test of which evidence survives each storage-layer transition.

There is a narrower ingestion gap on that early-return path: the incoming record is passed directly to consolidation, and the caller does not then insert it into the raw store. Preserving existing neighbors therefore does not prove that the newly arrived interaction remains recoverable. This is visible in the inspected control flow, not an independently reproduced loss event. A fault-injection test must check the incoming item separately from the old raw neighbors, with an upstream acknowledged event log as a distinct baseline.

The corresponding engineering question is atomicity of abstraction. Should raw evidence be removed only after every intended derivative is committed, or is the episodic artifact sufficient to justify removal? There is no universal answer. If the task requires fine-grained facts that episodic extraction can omit, deletion after partial refinement may reduce recoverability. If raw content has a strict retention limit, preserving it indefinitely can violate the application contract. The correct behavior requires an explicit transaction and retention policy.

Episode merging also risks losing exception structure. Two similar successes can differ in a precondition that later determines whether a procedure applies. Merging them into a generic account may hide that condition. An episode store should therefore preserve source membership and avoid claiming that a merged summary contains every distinction from its constituents. A proposed merge validator tests representative questions and action preconditions against both the originals and the candidate abstraction, with failures retained as counterexamples.

Outcome metadata needs comparable care. A task completed under oracle feedback, a permissive simulator, or an outdated tool version can be useful experience, but its outcome should not be treated as evidence for an unchanged production environment. The episode's environment and evaluator versions are part of its meaning. Without them, retrieval can faithfully surface experience whose apparent success rests on conditions that no longer hold.

14.4 Semantic memory and the status of a proposition

Semantic memory attempts to represent reusable information without carrying the complete episode each time. Its advantage is efficient access

to facts, preferences, and abstractions. Its risk is detaching a proposition from the conditions under which it was observed or inferred. A statement such as "the service accepts this argument" is incomplete without a service version, scope, and time. A preference such as "use the fast route" may apply to one task and not to every future decision.

Separate factual assertions, user preferences, procedures, hypotheses, and authority. They have different update rules. A new authoritative correction can supersede a preference; conflicting observations may require uncertainty rather than replacement; an explicit prohibition cannot be overridden by repeated assistant-generated statements. A semantic store that deduplicates solely by embedding similarity can merge records whose surface meanings overlap while their roles differ. Deduplication should preserve distinctions needed for future interpretation.

A proposed semantic update represents competing propositions as a set with status rather than forcing immediate collapse to one string. For entity x, predicate p, and value y, retain the source, valid interval, and confidence of each assertion. A resolver produces a current view according to explicit rules. For example, source authority can precede recency, and recency can be evaluated within the correct valid-time interval. This is a semantic policy, not merely a database ordering operation.

Generative Agents and MemGPT show different ways of organizing external information and making selected state available to an underlying model [138, 31]. Their architectures motivate the separation between stored experience and active context. They do not establish that extracted statements are universally correct, that biological memory mechanisms have been reproduced, or that an agent can retain all important information indefinitely. Those are stronger claims requiring different evidence.

Semantic extraction can also amplify certainty. A source says that a method "may help" under a particular setting; a first summary says it "helps"; a later consolidated memory says it is the preferred method. Each individual rewrite can look reasonable while the chain removes uncertainty and scope. Provenance should therefore be attached to assertions, and the system should preserve the original qualification where it affects decisions. Repeated generation is not independent corroboration.

An evaluation can construct paired records differing only in modality, scope, or authority: an observation versus a hypothesis, a project preference versus a global preference, and a user correction versus a retrieved recommendation. The memory system should preserve the distinction through extraction, merging, retrieval, and use. This proposed test directly measures whether semantic normalization erases operational meaning. Exact-match

fact recall alone does not expose these role errors.

14.5 Hierarchy access and complete lifecycle cost

A hierarchy can keep a small active state, a medium-sized abstraction layer, and a large evidence store. The hierarchy is useful when access frequencies and representation costs differ, but it introduces promotion, eviction, and synchronization policies. If a fact changes, every accessible derivative may need invalidation. If a summary is promoted to active state, it can displace another record whose future value was underestimated. The store layout and the control policy should be analyzed together.

Let N_r, N_e, N_s be the numbers of raw, episodic, and semantic records. A naive dense retrieval over all embeddings has arithmetic cost proportional to $(N_r + N_e + N_s)d$ for embedding dimension d, before ranking and model generation. Approximate indexes change the cost-quality tradeoff and add index-maintenance work. A graph hierarchy can reduce some searches while increasing construction and traversal overhead. No storage representation is automatically more efficient without a specified query distribution and update rate.

RecMem's `search_memories` exposes three retrieval branches with separately configured limits and thresholds, returning raw, episodic, and semantic results [136]. This matters for causal comparison: better answering can arise from more total evidence, different representation, or a different allocation across layers. A baseline that receives fewer retrieved tokens is not a clean test of consolidation structure. Match the complete evidence budget, and separately ablate which layer provides the decisive item.

Construction cost can dominate when many memories are written but few queries arrive. Query-time cost can dominate when a stable store serves many decisions. Let C_w be total writing and consolidation cost, C_q average retrieval-and-generation cost, and $Q \in \mathbb{N}_{>0}$ the number of useful queries. A simple amortized cost is $(C_w + QC_q)/Q$; it is undefined at $Q = 0$, when construction produced no served query. This arithmetic explains why reporting only retrieval tokens can reverse the practical ranking of memory designs. Storage, indexing, failed extractions, and maintenance should remain inside the chosen boundary.

Cache and semantic invalidation also differ. Evicting a vector-search cache can cause recomputation without changing knowledge. Invalidating a false semantic proposition changes what future decisions should use. A hierarchical system must not use the same "eviction" operation for both

without preserving the semantic reason. The working view should be able to tell whether an item is unavailable temporarily, deleted by retention, superseded by correction, or rejected as untrusted.

The practical implementation should record lifecycle counters by operation: admitted raw records, attempted extractions, successful episodes, semantic revisions, retrieval candidates, final injected items, invalidations, and deletions. These counters are diagnostic observables, not substitutes for task outcomes. They make it possible to explain why a method's cost or reliability changes with the number of sessions rather than reporting a single benchmark aggregate.

Hy-Memory-30B-A3B, introduced in an October 2026 technical report, instead targets a reusable writer inside several existing memory frameworks. It separates writer learning from the fixed reader used for reward, trains on policy-produced bank states, and combines per-chunk probes, attributed cross-chunk feedback, and a bank-growth penalty [133]. Its reported cross-harness comparison uses fixed 20% benchmark subsamples, including a held-out framework. That is evidence about a writer-reader-harness configuration, not a universal ranking of complete memory products. Its learned write protocol also differs from RecMem's training-free recurrence gate.

The comparison identifies a useful causal axis. RecMem changes when expensive abstraction is performed; Hy-Memory changes the policy performing framework-specific writes; HyperSkill changes the representation and maintenance of procedural experience. A result can improve because of any of these interventions, and combining their names does not establish a compatible composite system. A matched study would hold the reader and retrieval configuration fixed while varying admission timing, writer policy, and representation separately, then test interactions. In particular, a writer trained against one retriever can learn representations whose value disappears when the retrieval interface changes.

14.6 Temporal validity conflicts and forgetting

Prospective memory requires more than answering a question about the past: a future event must activate a previously established constraint. TriggerBench separates an early constraint from a later trigger using intervening dialogue, matched retrospective questions, overloaded positive triggers, and negative controls [139]. Its distinction motivates an implementation object (g, u, b): g is a trigger predicate over observed events, u is the intended intervention, and b is an applicability boundary including expiry and authority. At an event, evaluate the guard and current boundary before proposing

the intervention. This is a proposed representation, not a claim that the benchmark implements an event engine. Evaluate missed triggers and false activations separately; unconditional reminders can improve recall while failing negative cases. A reminder also cannot confer authority to execute the underlying action.

Memory usefulness depends on whether an assertion remains valid when it is used. A correct historical fact can be a wrong current instruction. A past failed attempt can be useful as an episode while its error cause has been fixed. Temporal validity should therefore be represented at the proposition or procedure level, not inferred from how recently a record was retrieved. Retrieval frequency measures access, not truth.

Use valid-time intervals $[t_b, t_e)$ where meaningful, and preserve observation time separately. If a user states at time t_2 that a preference changed at $t_1 < t_2$, the current resolver should incorporate the earlier effective change without pretending the agent knew it before t_2. This supports both present decisions and historical audit. A last-write-wins store ordered by ingestion can produce the wrong historical answer, while a store ordered only by effective time can lose what was actually known at a decision.

Conflicts can be factual, temporal, procedural, or authoritative. Factual conflict concerns incompatible descriptions of the same state. Temporal conflict can disappear once intervals are distinguished. Procedural conflict concerns strategies with different preconditions. Authority conflict concerns who may direct an action. A generic contradiction prompt can help identify conflicts, but the resolver's rule must reflect the class. Majority voting over repeated model-generated statements is especially weak evidence of authority or truth.

Forgetting has several meanings: preventing retrieval, removing a record, deleting derivatives, expiring an active preference, or modifying parameters. An external memory system should state which operation it implements. Removing a row from one index does not establish erasure from raw logs, summaries, backups, or downstream services. Conversely, revoking future use can be a meaningful operational guarantee even when some audit record remains. Claims should follow the actual deletion and retention contract.

A proposed invalidation protocol marks the source assertion obsolete, finds derived summaries and skills that depend on it, invalidates their reusable status, and rebuilds any current view that included them. This assumes a recorded dependency graph. Missing provenance limits completeness. A safe fallback can quarantine a larger region of memory at the cost of reduced utility. That tradeoff should be measured rather than hidden behind a claim of perfect forgetting.

Longitudinal evaluation should include corrected facts, changed preferences, expired permissions, and requests to forget. Score not only the desired new answer but also whether the agent stops using the old record in unrelated later tasks. Include negative controls where an update should not affect another project or user. These tests assess selective revision rather than indiscriminate amnesia. Their relevance increases as sessions, teams, and model versions share a growing store.

14.7 Retrieval does not imply appropriate use

Memory can fail through excessive influence as well as omission. MemSycoBench constructs decisions where relevant historical material must be rejected as factual evidence, bounded to its original scope, reconciled with current evidence, updated, or used for legitimate personalization [140]. These categories distinguish retrieval success from epistemically appropriate use. A stored preference is evidence about a user's preference; it does not become evidence that the preferred claim is objectively correct. A discriminating counterfactual preserves the current evidence and changes only the historical preference. An objective answer should remain invariant, while a preference-dependent recommendation may appropriately change. That conditional distinction is stronger than rewarding either universal obedience to memory or universal suppression.

Response concentration is a separate outcome. The October MemBubble benchmark derives preferences from real conversations and constructs topically related queries in distinct scenarios, then measures diversity using functional-equivalence classes of sampled answers [141]. Its query filtering and diversity pre-test define a selected evaluation population; the result cannot estimate all deployment queries without a sampling argument. An original evaluation recommendation is to report valid-personalization accuracy and response-distribution concentration jointly. Higher entropy can include irrelevant alternatives, while lower entropy can reflect a justified constraint. Hold the decoder and sampling budget fixed, inspect the equivalence-class judge, and separate memory-induced narrowing from changes already implied by the current request.

An agent can store a fact, retrieve it when asked directly, and fail to apply it when relevance is implicit. InMind isolates this distinction with paired storage and in-context controls and an answer-blind target-recall measurement [134]. Its 125-task diagnostic reports a large gap between direct recall and application under indirect queries. The result concerns the tested systems and selected implicit associations, not every possible

retrieval mechanism. Missing broad negative controls also matters when evaluating whether a remedy overapplies memory.

The mechanism is a sequencing problem. A query-conditioned retriever must decide what is relevant before the full policy sees the candidate fact. If relevance depends on a bridge supplied by world knowledge, surface similarity can miss it. Increasing embedding dimensionality does not necessarily solve the need to infer that bridge. A structured profile, proactive constraint store, query expansion, or model-assisted memory search can change the access interface, but each introduces a different cost and failure mode.

The official InMind artifact makes the evaluation boundary inspectable. Its timeline builder constructs the ordered conversation for a selected task, while its judge-payload builder separates measurements through metric-specific payloads [142]. These are benchmark construction and scoring paths rather than a released solution to the memory-use problem. A reproduction must preserve the paired controls, target-fact placement, answer-blind measurement, and system output schema. Omitting a control can turn a retrieval diagnosis into an ambiguous end-to-end failure.

InMind's reported in-context control reaches 84.0%, while the six evaluated memory systems reach at most 14.4% on indirect application and can reach 100% on direct recall [134]. The separately evaluated dense NaiveRAG baseline reaches 16.0% on indirect application; it is not included in that six-system maximum. These percentages have different experimental conditions and should not be subtracted as if they were a randomized treatment effect. The official timeline code also freezes the post-history state for direct and indirect queries, preventing one query's answer from teaching the subsequent query the target fact. This control is crucial: sequentially asking the direct question first would change the state and confound the diagnosis.

Appropriate use also requires authority calibration after retrieval. A memory can be relevant as background evidence without being a binding constraint. A tentative preference should not override a current explicit instruction, and an old successful procedure should not override a changed tool contract. One proposed interface labels each item with its role: observation, preference, candidate procedure, or current obligation. The model still interprets the item, while deterministic controls can prevent some role promotions at consequential boundaries.

Formally, let z denote the current task state and m a retrieved item. The useful question is the change in expected loss caused by making m visible, conditional on its status and provenance. Similarity is a retrieval

signal; causal usefulness is an outcome under an intervention. Estimating the latter requires matched continuations or randomized memory exposure where appropriate. Observational correlation between retrieval and success is confounded because easier or familiar tasks can both retrieve better and succeed more often.

An evaluation should include four conditions: the fact absent, present in storage, explicitly retrieved, and permanently visible as a diagnostic control. Compare action correctness and inappropriate-use rates, not only answer recall. If explicit exposure repairs performance, the bottleneck is likely upstream of use under those conditions. If it does not, missing bridge knowledge or interpretation remains possible. This decomposition gives a concrete experimental route from an aggregate memory score to a mechanism-specific conclusion.

14.7.1 Identifying memory influence

A controlled exposure study can make the causal estimand explicit. Let X_i bind a frozen task, memory-bank snapshot and model configuration. Let $B_i \in \{0, 1\}$ indicate whether one eligible memory item is exposed, with known assignment probability $e_i = P(B_i = 1 \mid X_i) \in (0, 1)$. Write $L_i(1)$ and $L_i(0)$ for the two potential decision losses and observe $L_i = B_i L_i(1) + (1 - B_i)L_i(0)$. Under randomized assignment conditional on X_i, consistent exposure, and no cross-trial state interference,

$$\widehat{\Delta}_n = \frac{1}{n} \sum_{i=1}^{n} \left[\frac{B_i L_i}{e_i} - \frac{(1 - B_i)L_i}{1 - e_i} \right] \tag{14.4}$$

has expectation $n^{-1} \sum_i \mathbb{E}[L_i(1) - L_i(0)]$, the average exposure effect for the sampled task population. The identity follows by conditioning each term on X_i and its potential losses. Negative Δ means beneficial exposure on this evaluated distribution. Very small propensities can make the estimator unstable, so uncertainty and the assignment range must be reported.

This construction is a proposed experiment, not a measured memory benefit. Use sandboxed or read-only continuations with reset state where withholding evidence is permissible; current authorization and mandatory obligations are not randomized away. A shared persistent bank updated by earlier trials breaks the no-interference premise unless the trajectory itself is the randomized unit. Delayed or missing labels require an additional observation model: logging admission propensity alone does not identify outcomes for permanently censored facts. The experiment also distinguishes the influence of an item from its factual correctness and from the quality of the writer that produced it.

14.8 Memory transactions and a reference test suite

Memory construction often spans several writes: raw evidence, embeddings, an episode, semantic propositions, and provenance links. A partial failure can leave these layers inconsistent. The central transaction question is which combinations constitute a valid published memory state. If the semantic record references an episode that never committed, retrieval can return an unsupported assertion. If the raw item is removed before the episode becomes durable, recovery can lose both evidence and abstraction.

A proposed two-stage publication process first writes candidate derivatives with explicit source references and pending status. Validation checks schemas, provenance existence, ownership, and any required exact fields. A commit then makes the derivatives retrievable and records whether raw evidence may be removed under retention rules. Failed candidates remain unavailable to the answer policy. This is an implementation pattern, not an established universal memory algorithm. Semantic validity still requires stronger checks than database atomicity.

For a conservative publication gate, let $D(m)$ be the recorded transitive dependencies of candidate m, $c(m)$ its committed-status bit, and $a_{\alpha,t}(m)$ its current access-and-validity check. A strictly source-bound view can expose

$$M_{\alpha,t}^{\text{vis}} = \{m : c(m) = 1,\ a_{\alpha,t}(m) = 1,\ a_{\alpha,t}(x) = 1\ \forall x \in D(m)\}. \quad (14.5)$$

This is a proposed policy, not a necessary rule for every legitimate aggregate. It assumes complete dependency recording and current checks; missing lineage requires unresolved status or a broader quarantine. It certifies publication structure and scope, not the semantic truth of m. A correction can revoke reusable status without falsifying the historical event that an old version was used. Effect receipts and current grants therefore retain their typed control semantics instead of becoming ordinary similarity-ranked facts.

The state can maintain separate read and write versions so that an ongoing task sees a coherent memory snapshot. A new correction can trigger explicit refresh rather than silently changing the evidence beneath a multi-step plan. For high-consequence obligations, refresh may need to occur immediately before tool execution. For ordinary background facts, snapshot consistency may be adequate. The choice follows workload consequences and should be specified per memory class.

The proposed test suite includes extraction failures, duplicate writes, interrupted consolidation, stale embeddings after a correction, conflicting

user scopes, and a summary whose source has expired. Each case has a deterministic structural oracle where possible and a separately evaluated semantic outcome. Compare a raw-log baseline, a flat fact store, a layered store, and a structured obligation profile under the same backbone and complete cost boundary. Report task order and repeat runs because the store evolves during evaluation.

For RecMem-like recurrence policies, vary the frequency of decisive facts independently of their importance. This tests whether recurrence is a useful cost proxy without making frequency and utility identical by benchmark construction. For semantic merging, create near-duplicate records with different temporal scope or authority. For retrieval-use failures, retain the InMind-style paired controls. For deletion, test derived artifacts and cross-session behavior rather than only a missing database row. These are proposed experiments, not new results.

The resulting engineering standard is precise but bounded. A memory system should say what it stores, when it consolidates, how it retrieves, which authority its records carry, how it handles corrections, and what its transaction protects. It should not claim lifelong continuity from successful direct recall, or parametric learning from external storage growth. Procedural consolidation in Chapter 15 extends this analysis to reusable skills whose errors can influence many later tasks.

Chapter 15

Procedural Memory and the Lifecycle of Executable Skills

HyperSkill's 2026 implementation makes procedural memory an inspectable graph lifecycle: trajectory membership guides retrieval, skills and mistakes enter the execution view, and utility feeds later pruning and merging [143, 144]. The relevant scientific questions concern applicability, attribution, maintenance cost, and compatibility; neither a successful source trajectory nor a compact graph proves that its distilled procedure transfers.

15.1 A procedure has preconditions and effects

A skill is reusable procedural knowledge that changes how an agent approaches a class of tasks. It can be a natural-language instruction, executable code, a workflow, or a package combining these forms. Its defining feature is a claim about applicability and action, rather than a claim about a fact alone. "The repository uses a particular test framework" is semantic knowledge; "run this fixture before changing the parser" is procedural guidance. A deployed skill can contain both, but the distinction matters when its environment changes.

Represent a candidate skill as $s = (P, \rho, Q, E, V)$, where P is an applicability predicate, ρ the procedure or policy fragment, Q an intended postcondition, E the evidence supporting it, and V compatibility information. This is a proposed contract, not a requirement that every skill system exposes these fields. Natural-language skills often leave P and Q implicit. That makes them flexible but creates a risk that the policy applies a procedure beyond the conditions under which its evidence was collected.

Procedural memory is external to the model when it is stored and retrieved as an artifact. Activating the skill changes the model's conditional input; it does not necessarily change model parameters. A skill bank can improve performance with a frozen backbone, but that result should not be described as weight learning. Conversely, a procedure can later be distilled into parameters, changing its update, deletion, and verification boundaries. Chapter 30 treats the difference between these improvement targets.

The Agent Skills specification describes an artifact interface for instructions and resources [145]. It is an interoperability and packaging reference, not empirical evidence that an agent chooses or executes skills correctly. A tool schema, a skill package, and a model parameter update are therefore distinct objects. A tool exposes an executable capability; a skill can tell the agent when or how to use capabilities; learning changes the policy that interprets both. Treating them as interchangeable makes attribution and authority ambiguous.

The institutional-knowledge perspective emphasizes that reusable procedures can encode local practices and constraints [146]. Its case-study evidence should not be mistaken for a controlled estimate of productivity. The more general engineering implication is to attach scope and provenance to a skill rather than treating every successful local workaround as universally reusable knowledge. A workaround for one model's failure may become harmful after the model or tool interface is upgraded.

15.2 Extracting procedures from experience

Repository-derived skills require a different evidential path from lessons extracted after an execution. A README describes an intended interface; a dependency manifest identifies a configured environment; an example demonstrates only the path it actually exercises. A proposed compiler should preserve those source roles, resolve referenced APIs at a pinned revision, and validate a minimal invocation in an isolated fixture before promoting the text into reusable guidance. Its output should state which preconditions were checked and which remain documentation claims. Successful package import does not establish that a remote operation is authorized or that an example's destructive defaults are appropriate. Repository instructions remain untrusted inputs during extraction. They cannot grant additional capabilities merely because the resulting package is called a skill. This separates source acquisition, executable validation, and activation authority.

An experience-to-skill pipeline takes a trajectory and constructs a reusable abstraction. The trajectory contains observations, attempted actions, failures, and an outcome; the abstraction identifies a pattern presumed to transfer. This is a causal inference problem under severe confounding. A successful episode does not identify which action caused success, and a failed episode does not prove that every included procedure was wrong. Outcome-conditioned extraction can help focus attention, but it cannot manufacture counterfactual evidence.

Let Y be task success and S indicate that a particular skill was made available. The difference $\mathbb{E}[Y \mid S = 1] - \mathbb{E}[Y \mid S = 0]$ is observational unless assignment is controlled or suitable identification assumptions hold. Familiar tasks can both retrieve a skill and be easier. A high success rate among retrieved uses can therefore overestimate causal value. The extraction system should retain the original situation and outcome rather than attaching an unconditional "effective" label to the resulting prose.

Positive strategies and negative lessons are also different memory objects. A negative lesson can specify a known failure pattern without prescribing a complete alternative. An agent should not convert "this search query failed" into "never use this search tool." The abstraction must preserve the scope of the failure. Including environment version, task subtype, and the relevant precondition can prevent a local observation from becoming a global prohibition.

HyperSkill explicitly extracts subtask structure, reusable skills, and lessons from experience, organizing them through trajectory-level hyperedges [143]. Its retrieved memory is assembled for a task and then used during execution. This makes the mechanism different from a store that continually rewrites the prompt at every step. The task-level assembly boundary is important: a newly discovered fact or a changed tool contract during execution may require an additional update mechanism outside that fixed retrieved guidance.

A robust extraction implementation should separate parsing from admission. First construct a candidate object. Then validate required fields, source references, compatibility, and prohibited authority changes. Finally decide whether the candidate should be active, experimental, quarantined, or discarded. A malformed extraction should not silently fall back to a broad generic rule that is indistinguishable from a validated skill. If a fallback is used, record its origin so later evaluation can distinguish generated knowledge from successful parsing.

The proposed acceptance test for a procedure includes a positive task where it should apply, a near-neighbor task where its precondition fails, and a changed-environment task where the procedure must be revised or ignored. This tests selective transfer rather than simple memorization. A skill that improves the positive case while damaging the negative control has not established broad utility, even if average benchmark success rises.

15.3 Hypergraphs preserve trajectory membership

HyperSkill represents subtask and skill nodes together with a hyperedge for each experience [143]. The hyperedge retains membership in a common trajectory and carries a task description, distilled lesson, and quality information. This is more specific than a claim that graphs improve memory. The representation keeps an association among several elements that were used together, whereas a flat vector store retrieves each element independently. The benefit must still be evaluated against matched retrieval and evidence budgets.

Let $V = V_u \cup V_s$ be subtask and skill nodes and let E be trajectory hyperedges. A task query q produces a decomposition $P(q)$. One retrieval route matches this decomposition to subtask nodes, then expands to incident hyperedges. A second route matches the task description to trajectory representations. Their union is

$$E^* = E_{\text{sub}} \cup E_{\text{task}}. \tag{15.1}$$

Candidate skills are members of those retrieved hyperedges. HyperSkill ranks them primarily through retrieved co-occurrence,

$$\kappa(s) = |\{e \in E^* : s \in e\}|, \tag{15.2}$$

with semantic similarity used for ties. These equations describe the inspected method, not a newly proposed retrieval result.

The representation should not be confused with a magical source of higher-order semantics. A hypergraph can be encoded as a bipartite incidence graph with explicit hyperedge nodes. What matters is retaining trajectory identity and membership, not the use of one graph library. A pairwise projection that discards hyperedge identity can lose which groups co-occurred together, while a representation that preserves the incidence relation can express the same membership. This is a structural deduction that sharpens the comparison axis.

The decomposition route can retrieve procedural analogies that differ lexically at the whole-task level. It can also amplify decomposition errors: if the generated plan misidentifies the task, it steers retrieval toward the wrong experiences. The task-level route provides a complementary signal but does not guarantee correction. A meaningful ablation holds total retrieved evidence fixed and compares task-only, subtask-only, and fused retrieval, with the same downstream model and prompt budget.

Co-occurrence is evidence of repeated use within retrieved experiences, not proof of causal usefulness. A generic skill can appear frequently because it was extracted from many tasks, while a rare exception-handling skill can be decisive. Frequency-based ranking and quality-based maintenance address different concerns. Their interaction should be inspected: pruning a rarely tested item, changing the set of retrieved edges, or merging broad skills can alter the candidate pool before the co-occurrence ranker runs.

Figure 15.1 isolates the mechanism and its stated information boundaries.

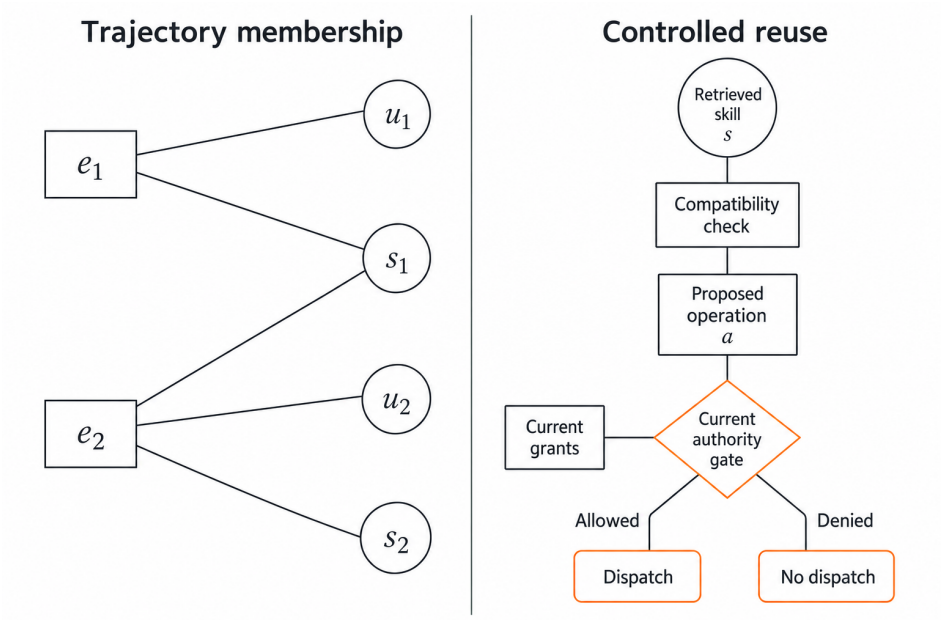

Figure 15.1: An illustrative incidence representation: e_i denote episodes, u_i subtasks, and s_i reusable skills. Undirected lines retain membership, not execution order. A retrieved skill remains a procedural hypothesis; compatibility and the proposed operation's current dispatch authority are separate checks. Neither membership nor compatible metadata proves procedural correctness. Maintenance uses additional evidence outside this schematic. Right-hand connectors show dependencies among checks and labelled conditional outcomes.

15.4 Quality-guided maintenance and its assumptions

An accumulating skill bank can become redundant, stale, and internally inconsistent. HyperSkill periodically prunes low-utility knowledge and merges structurally related skills [143]. Its inspected code computes a utility score combining observed success rate with normalized step efficiency. For retrieval count ν, success count σ, mean steps $\bar{T}$, and weight β, the form is

$$\gamma = \beta\frac{\sigma}{\nu} + (1-\beta)\left(1 - \frac{\bar{T} - T_{\min}}{T_{\max} - T_{\min}}\right), \tag{15.3}$$

for $\nu > 0$, $\beta \in [0,1]$, $0 < \bar{T}$, and $T_{\min} \leq \bar{T} \leq T_{\max}$ with $T_{\max} > T_{\min}$, where the last two quantities delimit the step-count normalization range. The implementation handles untested nodes and degenerate ranges separately [144]. This score is a maintenance heuristic. It is not an unbiased estimate of a skill's individual treatment effect.

The attribution problem is concrete. When several skills are retrieved together and a task succeeds, updating every retrieved node credits them jointly. A harmful item can survive because other guidance compensates for it. A useful item can be penalized on intrinsically difficult tasks. The quality estimate is policy- and task-distribution-dependent. If the model improves or the task order changes, old scores need not remain calibrated. Maintenance should therefore retain exposure counts and evaluation conditions rather than only a scalar utility.

The inspected `update_utility` updates retrieved nodes and their incident hyperedges after an episode. Its comment specifies that the utility is consumed by maintenance. The ranking routine still uses co-occurrence and similarity, not utility as its primary ordering [144]. This separation is easy to miss in a high-level summary. Changing the utility formula affects which knowledge survives and merges; it does not simply replace the online ranker with a quality score.

The normalization has a time-dependent implementation boundary. The core maintains global historical extrema of observed mean step counts, but stores each node's utility when that node is updated; pruning later reads that stored value. Without a full rescore, nodes can therefore carry utilities computed under different ranges. This is a source-level observation, not a demonstrated benchmark error. A discriminating test holds one node's exposure and outcome counts fixed, changes the global range through an unrelated trajectory, and compares cached with recomputed utility before pruning. Versioning the scoring context or recomputing all eligible scores

are possible interventions.

Structure-informed merging uses quality-weighted co-occurrence and propagated embeddings to propose related abstractions. A merged node can reduce redundancy, but it can also erase incompatible preconditions. The merge model should not decide compatibility solely from a high cosine score. Two procedures can use similar vocabulary while applying to different authorization scopes or tool versions. Retain source nodes or reversible merge metadata when the cost of a bad abstraction is high.

Maintenance frequency creates another tradeoff. Frequent updates can respond quickly to drift but add embedding, graph, and language-model work. Infrequent updates let redundant or harmful items influence more tasks. A maintenance schedule chosen on one benchmark order can overfit that order. Evaluate shuffled sequences, abrupt environment changes, and mixtures of easy and difficult tasks while charging all extraction and merge costs. Without these controls, a smaller final graph can look efficient even if its construction consumed substantial resources.

The practical lesson is to treat consolidation as a state transition with evidence and rollback. Pruned knowledge may still be relevant to historical audit, while merged knowledge should preserve references to the experiences that justified it. A physically compact bank is not necessarily a semantically safer bank.

Complexity depends on implementation rather than the hypergraph label. The inspected core constructs a dense quality-weighted co-occurrence matrix for the eligible abstraction nodes [144]. For n such nodes, matrix storage alone is $O(n^2)$ before propagated embeddings and merge candidates. If each of m episodes contains at most r eligible nodes, forming all within-episode pairs costs up to $O(mr^2)$ under a direct construction. These are analytic upper bounds for the stated representation, not timings. Sparse incidence operations can change the practical tradeoff, but a sparse source graph does not guarantee that the derived co-occurrence matrix stays sparse.

This matters when a memory method claims online scalability. Retrieval can remain fast while periodic maintenance becomes the dominant cost. The maintenance interval amortizes that work across tasks but also delays removal of stale knowledge. A benchmark ending just before maintenance can undercount the eventual cost; one ending just after it can overstate per-query latency if the work is actually asynchronous. Report cumulative maintenance time and resource use over the same task horizon, alongside the critical-path delay experienced by each task.

15.5 An inspectable execution trace

The inspected implementation supplies a concrete read-write-maintain path. `HyperSkillProvider.provide_memory` handles a memory request, obtains or generates a plan, invokes the hypergraph retrieval, and formats the resulting context. `take_in_memory` accepts a completed trajectory and performs outcome-conditioned extraction. In the core, `HyperGraph.retrieve` implements the task and subtask routes, while `maintain_graph` performs pruning, structural merging, orphan cleanup, and persistence [144]. These are identifiable source functions at a pinned revision; they have been inspected, not executed here.

One feedback boundary deserves explicit validation. The provider reads the trajectory metadata field `is_correct` with a default of false. If a caller omits that field for an indeterminate outcome, the extraction path treats the episode as failure rather than as unknown [144]. A wrapper claiming evidence-based consolidation should require a typed outcome $y \in \{0, 1, \perp\}$, update outcome-dependent utility only for $y \neq \perp$, and retain unresolved trajectories separately. This is a proposed integration safeguard; it is not a claim that the paper's labeled evaluation omitted its outcomes.

Consider a constructed example with two past trajectories. One retrieved edge contains subtasks for locating a source, verifying its date, and comparing a claim, together with skills for checking primary evidence and preserving uncertainty. A second edge contains source-location and comparison subtasks plus a skill warning against treating a product announcement as a paper. A new research query matches both through its decomposition, even if the original task descriptions use different domain vocabulary. Co-occurrence ranks the shared evidence-checking skill above a skill appearing in only one edge, subject to the configured candidate pool.

This is an algorithmic illustration, not a benchmark observation. It makes two failure conditions explicit. First, if decomposition misses date verification, a relevant edge can fail to enter E_{sub}. Second, if a generic skill appears in many retrieved edges, frequency can dominate a rare warning. The task route can recover some missed edges, but no guarantee follows from taking the union. Testing this example with controlled distractor edges would isolate a representation and ranking effect.

In the inspected core, the retrieval budget is divided between task-hyperedge candidates and subtask candidates. The subtask search over-fetches candidates before filtering for the correct node type. Skills and mistakes are then ranked separately and limited separately. Consequently, the final guidance count and token count are not determined by a single nominal k [144]. A cost comparison should inspect the actual formatted con-

text rather than treating the retrieval parameter as the complete evidence budget.

The implementation also preserves instance-specific subtasks differently from abstract skills. Subtask insertion uses content-aware exact identity, while reusable abstractions can undergo semantic deduplication and structural merging. This distinction reflects the need to retain episode-specific planning context. It prevents one broad subtask label from automatically collapsing every occurrence, but it does not eliminate extraction or compatibility errors in the skill nodes.

A source-level reproduction should log the generated plan, retrieved subtask identifiers, task-route hyperedges, fused edge set, ranked skill and mistake identifiers, utility updates, and maintenance changes. These observables connect an outcome to the actual mechanism. Reporting only a final success rate and graph size would leave retrieval, extraction, and maintenance contributions difficult to distinguish.

15.6 Reported results and unresolved attribution

HyperSkill evaluates multiple backbones on xBench, GAIA, and WebWalkerQA [143]. Table 15.1 records selected author-reported success rates. The improvements differ across backbones and tasks, and some comparisons tie another method. The paper's table should therefore not be paraphrased as strict dominance in every configuration. These values are not independently reproduced in this monograph.

Table 15.1: Selected HyperSkill success rates (%), author-reported [143].

Configuration	xBench	GAIA	WebWalkerQA
Qwen3-30B-A3B, no memory	46.00	32.12	39.41
Qwen3-30B-A3B, HyperSkill	52.00	36.97	50.59
GPT-4o, no memory	54.00	32.73	46.47
GPT-4o, HyperSkill	62.00	44.24	51.18

Task success and step efficiency need separate interpretation. A skill can reduce steps by encouraging premature termination. It can increase tool calls while improving evidence coverage. The paper reports these quantities separately, and a reproduction should preserve them rather than declaring every shorter trajectory better. Extraction, planning, embedding, and

maintenance costs are additional lifecycle components. Lower execution-step counts do not establish lower end-to-end expense.

The main setting uses outcome feedback for memory updates, including gold outcome information in the relevant evaluation configuration. This is stronger feedback than many production agents receive. A method can learn useful procedural memory with such feedback while remaining limited when outcomes are ambiguous or delayed. If a model judge replaces gold labels, the judge's errors become part of the memory update loop and can persist into later tasks.

The fixed task-order seed is another qualification. An evolving bank learns from earlier tasks, so order changes both available experience and estimated utility. Repeating model decoding under one order does not measure order robustness. A shuffled-order evaluation and an explicitly held-out transfer set would help distinguish a favorable curriculum from stable procedural transfer. The result should report variation across orders and initial banks, not only the final average.

Component ablations can also be causally ambiguous. Removing a hypergraph component may simultaneously change candidate retrieval, context length, and abstraction content. A clean structure comparison preserves candidate text and token budget while varying only the membership information. A clean maintenance comparison preserves retrieval and evaluates before and after specific prune or merge operations. The paper provides useful mechanism evidence, while these further controls remain proposed experiments rather than additional reported results.

The evidence supports studying structured experiential memory as an implementable mechanism. It does not establish that higher-order storage universally outperforms flat retrieval, that the learned procedures remain valid after a model upgrade, or that recursive improvement accelerates indefinitely. Those stronger conclusions require new controls and different horizons.

15.7 Compatibility authority and the skill supply chain

A skill package can modify how an agent interprets tasks, chooses tools, and handles errors. It is therefore part of the effective policy surface. The package's text may be external data, but loading it into a privileged instruction position can give it operational influence. Compatibility and authority should be checked before activation, rather than inferred from a familiar name or a high retrieval score.

A proposed compatibility relation is $\mathrm{Compat}(s, \theta, h, \mathcal{T}, A)$, involving the skill, model, harness, tool set, and authorization. The relation can require specific tool schemas, artifact formats, and environmental preconditions. It can also forbid a skill from broadening authority. A procedure that was safe when a tool was read-only can become unsafe if the same name later exposes write effects. Version pinning makes this change detectable but does not prove semantic equivalence across versions.

The current Agent Skills format distinguishes required descriptive metadata from optional compatibility and tool declarations. In particular, its `allowed-tools` field is experimental and support varies by agent implementation; structural validation checks the artifact format, not every execution permission [145]. A consumer must bind package content and referenced resources to the activated version and declare which fields it actually enforces. A description that says a tool is approved cannot override the dispatcher's current policy. Likewise, loading instructions on demand is a context-state transition under a fixed interpreter; changing executable skill resources changes the governing artifact version.

Progent provides a complementary enforcement principle: tool admissibility is checked by a symbolic policy, with policy expansion distinguished from narrowing [24]. A retrieved skill should not silently expand that allowed action set. Even if the model interprets the skill as an instruction to use a new resource, the monitor can require an authorized scope change. This separates useful procedural guidance from the authority to produce effects.

Persistent procedural attacks exploit exactly this boundary. A malicious or mistaken lesson can survive many sessions and influence tasks whose original user never saw the source. Skill provenance should therefore include the generating trajectory, extractor, feedback source, validation results, and activation history. Repeated retrieval is not independent endorsement. A skill created by another agent is evidence from that agent, not a direct observation by the current one.

Consolidation can spread a defect. If a poisoned skill is merged into a broader procedure, simple deletion of the original node may not remove its influence. Record dependency links through merges and re-extractions. On revocation, invalidate affected descendants and test whether cached prompts or preloaded profiles still carry the procedure. The relevant guarantee is the actual scope of future-use prevention, not an unsupported claim of erasing every trace.

The proposed release gate for a skill includes schema checks, negative applicability cases, authority-preservation tests, representative task valida-

tion, and a rollback reference. These checks do not guarantee correctness for all future tasks. They make the acceptance conditions reviewable and reduce the chance that a single unverified episode becomes globally active procedural policy. Chapter 28 examines the broader attack and defense setting.

15.8 Consolidation as controlled hypothesis revision

The most useful interpretation of a skill bank is a collection of procedural hypotheses with evidence, rather than a set of permanently true rules. A new episode can support, narrow, contradict, or leave unchanged a procedure's applicability. Consolidation should preserve those possibilities. Automatically rewriting every new outcome into a confident lesson makes the bank vulnerable to noise, evaluator error, and shifting environments.

One proposed evaluation separates selection from revision. Freeze the initial bank and measure which skills are retrieved on held-out tasks. Then allow extraction while keeping retrieval fixed. Finally enable maintenance. This sequence identifies whether improvements arise from better access, new content, or pruning and merging. Charge the full cost at each stage and keep the same task order for the paired comparison, then repeat across randomized orders. This is a design for causal diagnosis, not a claim that every effect can be independently identified with a small experiment.

For a candidate merge, retain the original nodes as an experimental control. Evaluate the merged procedure on tasks where each original should apply and on tasks where their preconditions differ. If merging improves average performance but causes violations in the distinguishing cases, the abstraction has traded specificity for coverage. That tradeoff should be reported explicitly. A compact graph with fewer nodes can be easier to serve while preserving a less useful decision boundary.

Harness^2 illustrates a neighboring improvement target: task-level harness adaptation and reuse of experience about such adaptations [28]. Its bounded task experiments differ from HyperSkill's explicit procedural-memory organization. Both can alter future behavior around a model without establishing an improvement in the model's own learning algorithm. The relevant comparison is what gets updated, which feedback selects it, how much extra execution is used, and whether held-out gains survive. Labels such as "self-evolving" do not settle those questions.

An original cross-layer deduction follows. The lifetime of a skill should be constrained by the lifetime of its assumptions. Tool version, environment

behavior, model interpretation, and authorization can each invalidate a procedure without making its original episode false. This suggests expiring reusable status separately from deleting historical evidence. A later agent can still study why the skill once worked while being prevented from applying it automatically under changed conditions.

Trajectory multiplicity creates a distinct ranking sensitivity. Suppose a bank retains k additional, separately identified copies of a retrieved trajectory containing skill s, and those copies survive candidate selection. The count rule above becomes $\kappa'(s) = \kappa(s) + k$ even though no independent experience was added. A competing skill absent from that trajectory receives no corresponding increment. This construction is conditional on retaining and retrieving the copies; it does not assert that the inspected implementation necessarily admits a particular duplicate. It identifies a test for exposure concentration: duplicate an episode's representation without changing its factual content and inspect ranking and downstream behavior. Grouping evidence by source lineage or normalizing trajectory contributions are possible interventions, whose effects on rare useful experience would also need measurement. Co-occurrence, independent support, and causal utility remain different quantities.

Conditional skill evidence can compose only under retained preconditions. Suppose a workflow has K stages and F_j is failure at stage j. If, for every reachable preceding success history under the evaluated configuration,

$$P(F_j \mid F_1^c, \ldots, F_{j-1}^c) \leq \epsilon_j, \qquad 0 \leq \epsilon_j \leq 1, \tag{15.4}$$

then the chain rule gives

$$P\left(\bigcup_{j=1}^{K} F_j\right) \leq 1 - \prod_{j=1}^{K}(1 - \epsilon_j) \leq \sum_{j=1}^{K} \epsilon_j. \tag{15.5}$$

No independence assumption is required, but the conditional bounds must hold for the composed workflow, not merely for each skill on its original marginal task distribution. A tool migration, shared memory update, or changed predecessor can invalidate that premise. Finite test estimates need their own uncertainty bounds; a stored utility scalar is not such a certificate. Mandatory authorization remains a separate enforced precondition.

Procedural memory thus connects representation, retrieval, control, and governance. Its promise is selective reuse of experience; its central risk is selective reuse of an abstraction whose scope was never established. The rigorous engineering response is not to avoid skill banks, but to make their preconditions, evidence, revisions, costs, and failures part of the technical object being evaluated. Retrieval in Chapter 16 determines which of these hypotheses reaches the next decision.

Chapter 16

Retrieval Policies, Evidence Assembly, and Grounded Execution

The 2026 retrieval evidence spans ReFind's controlled search over raw chat, LGM's query-conditioned latent graph, and MATM's state-conditioned procedural continuations [147, 148, 149]. Their different representation and computation boundaries prevent a single structure-based ranking. A rigorous comparison follows evidence from the stored object through access and assembly to the decision it actually changes.

16.1 Retrieval selects evidence for an action

Retrieval in an agent system is the selection of information that can change a consequential decision. The retrieved object may be a document passage, a temporal fact, a prior trajectory, a skill, or a tool description. These objects have different relevance criteria. A passage can support an answer without authorizing an action. A successful trajectory can suggest a procedure without proving that its preconditions hold now. Retrieval quality should therefore be defined relative to the decision and the role of the retrieved object.

Retrieval-augmented generation established a model in which generation conditions on external retrieved evidence [32]. That foundational formulation is not a description of every current agent-memory system. By the October 2026 cutoff, active graph traversal, state-conditioned trajectory retrieval, and runtime context construction make memory access a multi-step control process. The retriever and the reasoning policy can interact repeatedly rather than following a fixed retrieve-once-then-answer pipeline.

Let S be the current task state, D the available evidence store, and Y the evaluation target. A retrieval policy ρ selects $R = \rho(S, D)$ without observing Y. Let $\mathfrak{R}_A(S, D)$ contain the evidence subsets allowed by the current access policy A. For a fixed joint distribution of (S, D, Y), a

proposed objective is

$$\begin{aligned} \min_{\rho,\pi} \quad & \mathbb{E}[\ell(a, Y) + \lambda C(\rho(S, D), S)] , \\ \text{subject to} \quad & \rho(S, D) \in \mathfrak{R}_A(S, D). \end{aligned} \tag{16.1}$$

Here $a \sim \pi(\cdot \mid S, \rho(S, D))$ follows the actual deployed decoding and parsing law, and the expectation includes its randomness. The nonnegative resource cost C includes search, reranking, context assembly, and generation; $\lambda \geq 0$ converts that cost to the scale of the decision loss ℓ. Missing-evidence consequences belong to the loss term. Relevance labels based on lexical overlap or passage entailment are proxies for this objective. They can be useful training targets without fully capturing whether the evidence changes the final action correctly.

The distinction matters for tool execution. An agent can retrieve the right account identifier but the wrong version of a permission policy. It can locate a valid example of an API call whose side effects differ from the present request. A retrieval metric that marks the identifier passage relevant can score the step as successful while the resulting tool call remains invalid. End-to-end evaluation should retain these roles and inspect the first decision at which the missing distinction matters.

Evidence also competes for a finite input budget. More retrieved items can increase recall while reducing the policy's ability to identify the authoritative ones. A retrieval system should report candidate recall, final evidence packing, and downstream use separately. A method that retrieves a decisive record but drops it during prompt construction has an assembly failure; a method that includes it but follows a stale conflict has an interpretation failure. Those distinctions guide different repairs.

For a specified decisive-evidence path, define nested events $E_1 \supseteq E_2 \supseteq E_3 \supseteq E_4$: the required source is available and authorized, reaches the candidate set, survives packing with its qualification, and is appropriately used. When the conditioning events have positive probability,

$$P(E_4) = P(E_1) \prod_{j=2}^{4} P(E_j \mid E_{j-1}). \tag{16.2}$$

This is the probability chain rule, with no stage-independence assumption. It measures this evidence path, not all task success: another source or a different valid plan can bypass it. Conditional stage metrics expose a bottleneck that an aggregate recall score cannot locate. They should be estimated on the same workload cohort; multiplying rates from unrelated benchmarks is invalid. MRAgent, JAM, MATM, and MemCalib intervene

at different factors, which is why their reported scores cannot be treated as interchangeable evidence of retrieval quality.

Figure 16.1 isolates the mechanism and its stated information boundaries.

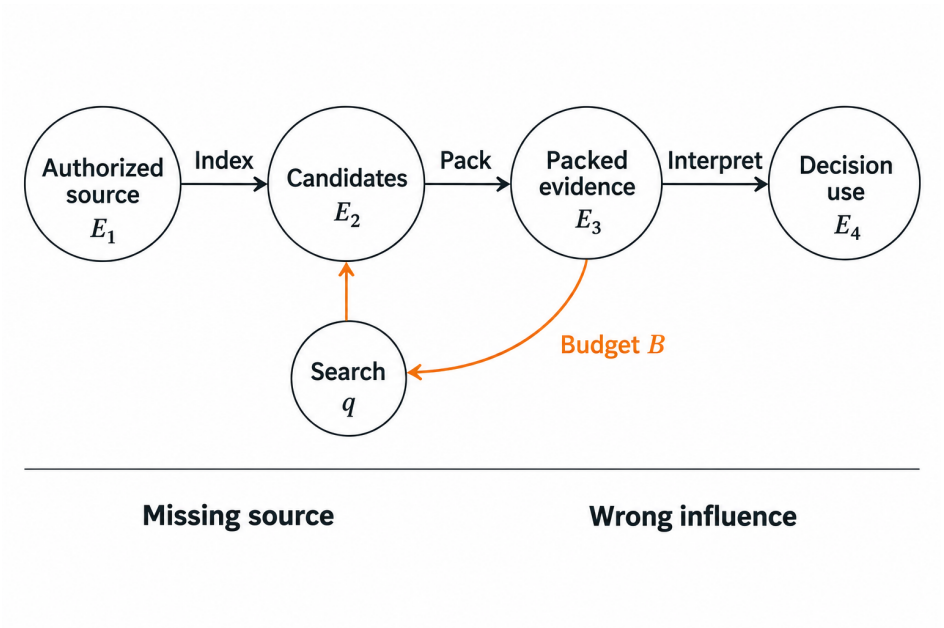

Figure 16.1: A decisive-evidence path with separate failure boundaries. Authorized source availability, candidate selection, qualified packing and appropriate use are nested events for this specified path. Conditional stage probabilities multiply without an independence assumption. Alternative valid evidence paths may still solve the task. A return to search is a policy choice with cost; it does not establish evidence sufficiency.

16.2 Index units and information boundaries

The indexing unit determines which relations retrieval can preserve. Fixed token chunks are simple but can separate an assertion from its qualification, a tool call from its result, or a correction from the statement it revises. Structural units such as sections, messages, episodes, and typed records can preserve those boundaries, but they vary in length and can contain multiple propositions with different authority. Chunking is part of the representation, not a neutral preprocessing detail.

A document index should retain source identity, version, location, timestamp, and access scope. An agent-memory index additionally needs event and session relationships. If a retrieved passage is regenerated from a summary, its identifier should distinguish the summary from the original source. Otherwise a citation can appear precise while pointing to an unsupported abstraction. A hierarchy can provide compact navigational text while keeping a path to exact evidence.

Dense retrieval uses learned representations to compare queries and candidates; lexical retrieval provides a complementary signal for exact strings, identifiers, and uncommon terms. Hybrid retrieval is an engineering response to different failure modes, not a universal guarantee. Dense similarity can miss a newly introduced identifier, while lexical overlap can retrieve a misleading near-duplicate. Query construction and metadata filtering should reflect the task rather than assuming one similarity function captures every relation.

Versioned indexing is necessary when documents change. A vector for an old passage can remain searchable after the displayed text has been replaced, creating an evidence mismatch. Store an index-generation identifier and source-content digest, and reject or refresh stale results according to the application contract. Re-embedding the entire corpus can be expensive; incremental updates require careful deletion and alias handling. The cost of these operations belongs in the lifecycle boundary.

Access filtering should occur before exposing content to an unauthorized model or worker. Filtering only the final answer does not undo the disclosure. Search ranking can itself reveal sensitive metadata if unauthorized records affect returned scores or counts. A strong application contract identifies which metadata may be visible, which store partitions are isolated, and how cached results are invalidated after access changes. Retrieval is an information-flow boundary as well as a ranking stage.

The September LGM method changes the representation boundary itself. It pools selected backbone hidden states, obtains sparse concept activations through an autoencoder, constructs query-dependent graph relations, and conditions generation through learned latent prefix tokens. Its training combines response generation, evidence reconstruction, and sparse reconstruction objectives [148]. These are coupled learned mechanisms, unlike an external text graph used only for lookup. The "neuro-symbolic" designation does not establish logical soundness or uniquely identifiable concepts: sparse activations and learned message passing do not by themselves prove either. A structure ablation must therefore control backbone adaptation, prefix capacity, and evidence-reconstruction supervision before assigning the full gain to graph topology. Reusing the latent bank after changing its

encoder or consumer also requires the compatibility tests of Chapter 17.

A proposed index audit samples records across document formats and update histories, checks that every returned item resolves to the recorded version, and verifies that qualification and role boundaries survive chunking. It should include deleted sources, renamed artifacts, repeated identifiers, and contradictory revisions. These checks do not prove semantic usefulness, but they establish that the retriever is searching the evidence it claims to search.

16.3 Query construction and adaptive access

A query is a hypothesis about which evidence will help. Query rewriting can add useful bridge concepts, but it can also encode an early mistaken interpretation and narrow the search prematurely. An adaptive agent should preserve uncertainty about its information need and revise the query when returned evidence contradicts its assumptions. The memory access policy is then part of reasoning rather than a fixed frontend.

MRAgent uses a Cue–Tag–Content graph and an active reconstruction loop [150]. Tags act as associative intermediates, allowing the model to select relations and then fetch content rather than expanding every neighboring node. The mechanism can use newly discovered cues to change later traversal. Its theoretical separation from restricted passive retrieval depends on the constructed task family and access assumptions; it should not be interpreted as a proof that every active agent outperforms every static retriever. Its published code provides a concrete object for implementation inspection.

JAM separates a hierarchical store retaining raw histories from a runtime Researcher that uses search, open, and browse actions to assemble query-specific context [151]. Its training combines verified-trajectory supervision with a subsequent policy-optimization stage. This changes the accounting boundary: less irreversible preprocessing can preserve details, while runtime exploration introduces model calls and latency. Query-conditioned construction does not remove the need for retention, access controls, or calibrated stopping when evidence remains incomplete.

The two mechanisms differ in what is navigated. MRAgent traverses explicit associative relations; JAM navigates a hierarchical workspace with raw pages and summaries. A clean comparison would hold raw evidence, model, maximum accessed content, and total compute fixed while varying the access representation. Without that control, an improvement can arise from more evidence, more reasoning calls, a different training distribution,

or a better interface. The architecture's name does not identify the cause.

Adaptive access requires a stopping rule. Let b be the current evidence state and let a be a possible retrieval action. An ideal controller would choose an action when its expected reduction in task loss exceeds its cost. In practice that value is uncertain, and self-assessed sufficiency can be overconfident. A useful implementation distinguishes stopping because evidence is sufficient from stopping because the budget is exhausted. Both can produce an answer, but only the first asserts that the information requirement was met.

A bounded decision model clarifies that stopping rule. Let $\xi \in \Xi$ be a sufficient controlled query state, including the target belief, query-use/history information, source/interface versions and permissions needed to determine available queries, costs, observation kernels and future state. Its target-belief projection is $b_\xi = \Pr(Y \in \cdot \mid \xi)$ for a fixed target Y. A posterior over Y alone need not be sufficient: identical target probabilities can coexist with different remaining queries or observation laws. For a fixed finite legitimate action set $\mathcal{A}$ and loss $\ell : \mathcal{A} \times \mathcal{Y} \to [0, L_{\max}]$ with finite $L_{\max}$, set $\mathcal{R}(b) = \min_{a \in \mathcal{A}} \mathbb{E}_b \ell(a, Y)$. A permitted read-only query q has observation kernel $K(\mathrm{d}o \mid \xi, q)$ and sufficient-state update $T(\xi, q, o)$. Assume Bayes-consistent target posteriors and that query selection uses only ξ and independent randomization, so selection itself supplies no hidden target information. Its one-step information value is

$$V_q(\xi) = \mathcal{R}(b_\xi) - \mathbb{E}_{o|\xi,q} \mathcal{R}(b_{T(\xi,q,o)}) \geq 0. \tag{16.3}$$

The inequality holds because the decision maker may ignore the new observation and retain its former rule. It does not describe a fallible deployed model, which may be distracted or misled by the returned text. It also does not compare different authority sets or changed task objectives.

Let $\mathcal{Q}(\xi)$ be a finite available query family, $B \geq 0$ a finite resource budget, $c(\xi, q) \geq 0$ the declared query cost, and $\lambda \geq 0$ a finite loss-scale multiplier. Write $\mathcal{Q}_B(\xi) = \{q \in \mathcal{Q}(\xi) : c(\xi, q) \leq B\}$. An explicit remaining-query horizon $n \in \mathbb{N}_0$ gives the finite-horizon controller

$$\begin{aligned} W_0(\xi, B) &= \mathcal{R}(b_\xi), \\ D_n(\xi, B, q) &= \lambda c(\xi, q) + \mathbb{E}_{o|\xi,q} W_{n-1}(T(\xi, q, o), B - c(\xi, q)), \\ W_n(\xi, B) &= \min\left\{ \mathcal{R}(b_\xi), \min_{q \in \mathcal{Q}_B(\xi)} D_n(\xi, B, q) \right\}, \quad n \geq 1. \end{aligned} \tag{16.4}$$

With no feasible query the inner minimum is $+\infty$. Unavailable results remain observations with incurred cost. The explicit horizon terminates the

recursion even when a permitted query has zero modeled cost. A budget alone bounds query count only under an additional positive lower cost bound. A myopic comparison $V_q(\xi) > \lambda c(\xi, q)$ can miss complementary multi-query evidence; the recursion exposes that limitation without claiming that a correct sufficient state or observation model is available in a production agent. Log budget or horizon exhaustion separately from a verified evidence-sufficiency decision.

The proposed diagnostic logs each information need, query, selected relation or page, returned evidence, and reason for stopping. It then replays failed cases with an oracle bridge cue or a missing page supplied. Repair under that intervention supports an access-policy bottleneck. Failure despite the evidence shifts attention to interpretation or action selection. This analysis is more informative than counting how many search calls an agent made.

The released code makes several boundaries more concrete. In the inspected MRAgent `Agent.answer_question` path, question-key extraction precedes graph-relation evaluation; temporal questions can take a specialized branch, and non-temporal evidence undergoes coarse selection, identifier-based deduplication, finer selection, and key-tag expansion before the tool-enabled answer stage [152]. These operations mean that a graph match is not identical to final evidence exposure. Deduplication by identifier prefix can reduce repeated material while also changing how much supporting detail survives. A reproduction should log both pre-selection candidates and the final answer input.

The repository linked by JAM includes a research implementation whose `ResearchAgent.research` refreshes retrievers, loads memory, plans a search, accumulates a temporary result, and reflects on sufficiency [153]. It records iterations and stops either when the reflection says enough or when the configured iteration bound ends. The returned integrated memory alone does not distinguish those stopping causes; the raw iteration record is needed to reconstruct them. The inspected March 2026 repository revision precedes the September JAM report. Its research and application variants therefore do not, by themselves, establish the deployed weights, training path, or every runtime interface of the paper's trained configuration. Public code and paper mechanism must be mapped at that narrower boundary.

MATM's inspected `UnifiedRerankerOrchestrator.rerank` supports staged reranking, normalizes candidates into one schema, and retains stage-level error metadata [154]. In the inspected error path, a failed stage can preserve its upstream candidate order instead of producing a new ranking. Thus a run configured for learned reranking need not actually apply that ranking on every query. Reporting configured mode without stage

application and error counters can misattribute a result to the reranker. These are source observations at pinned revisions, not measured error frequencies or reproduced benchmark outcomes.

16.4 State-conditioned procedural retrieval

Procedural retrieval should match the consumer's current situation rather than only the initial task description. Multi-Agent Transactive Memory indexes recent observation-action context as a key and a subsequent continuation as its value, enabling one agent to reuse another agent's experience [149]. A learned reranker estimates downstream utility beyond initial similarity. The method concerns shared artifacts across heterogeneous agents; it does not require that the agents jointly train or communicate during each task.

The essential indexing boundary is temporal. A key must contain information available before the continuation begins; the value can contain the later actions offered as guidance. If a key accidentally includes the successful future result, retrieval can exploit information unavailable to a real consumer. This is a procedural form of leakage. Dataset construction should validate the cutoff at each indexed step and preserve environment boundaries when sliding windows are formed.

The reported ALFWorld comparison improves from 47.08% without memory to 64.31% with retrieval and learned reranking, while WebArena gains are smaller [149]. These are author-reported population results under the paper's allocations and environments, not a universal memory-system ranking. Generating the experience repository and training rerankers also consume resources. Consumer-side savings should not be described as total lifecycle savings unless those production costs are included or explicitly amortized.

An executable continuation is not a command to replay blindly. A different consumer may have a different tool set, current object state, or authorization. The retrieved trajectory should be treated as evidence for a candidate plan whose preconditions must be checked. A replay mechanism that substitutes identifiers from the current task can still preserve a hidden assumption from the producer's environment. The interface should expose those assumptions where possible and allow abstention or fresh exploration.

HyperSkill's subtask and task routes provide a contrasting procedural index [143]. Rather than supplying only a raw continuation, they retrieve lessons and skills associated with relevant trajectories. This can improve abstraction and reduce raw context, while introducing extraction and

consolidation error. MATM and HyperSkill therefore compare along representation, timing, feedback, and cost axes. An aggregate success difference without matched consumer and producer budgets would not isolate whether raw or abstract experience is the better transferable object.

The proposed evaluation separates near-duplicate reuse, compositional transfer, and cross-environment transfer. Near-duplicate retrieval tests useful caching but is weak evidence for general procedural learning. Compositional transfer holds out combinations of known subtasks. Cross-environment transfer changes tools or state dynamics and tests compatibility. Report all three rather than pooling them into one "generalization" score.

16.5 Evidence packing attribution and influence

Retrieved content becomes useful only after it reaches the model in an interpretable form. Packing determines ordering, redundancy, source labels, and the separation between evidence and instruction. A strong ranker can still fail if the prompt truncates its best result or strips the qualification that made it relevant. The packed context should be treated as a versioned artifact that can be reconstructed for audit.

MemCalib studies a downstream distinction: a memory atom can be appropriately ignored, provide bounded support, or control a material conclusion [155]. Its benchmark supplies composite memory blocks and annotates intended influence at the atomic level. Its training method separates overuse and underuse signals and redistributes credit using counterfactual localization. These results evaluate use of supplied memory, not the quality of a complete storage-and-retrieval system. The distinction limits the claim while making the post-retrieval problem experimentally visible.

Influence should follow role and authority. A product page can provide a claimed feature, a paper can provide an author-reported result, and a user message can provide a task constraint. Packing all three as equally authoritative text invites category errors. The model-visible representation should preserve source type and scope. It should also mark deductions separately from observations so that later summarization does not promote a proposed explanation into cited evidence.

Attribution requires a mapping from an output claim or action to supporting evidence. For a text answer, this can be a claim-span-to-source relation. For tool execution, it can include the source of each argument and the authority permitting its effect. A citation to a long document is not sufficient when the cited passage does not entail the claim. Source

availability, source relevance, and claim support are different checks.

Conflicting evidence should remain visible until resolved. A packer that chooses only the highest-scoring passage can erase a known disagreement and create false certainty. A proposed packing objective reserves space for support, relevant contradiction, and unresolved uncertainty. This can reduce the number of distinct positive passages that fit, so its value should be evaluated through decision correctness and calibration rather than raw recall. The best evidence set can contain a passage that argues against the tentative answer.

The implementation should log both retrieved candidates and the final injected set. Record which records were excluded for budget, access, duplication, staleness, or low relevance. This supports targeted failure analysis and prevents a system from claiming that a fact was "retrieved" merely because it appeared in an internal candidate list. The final conditional input is the evidence boundary that matters for the policy.

16.6 A worked multi-step evidence dependency

Consider a research agent asked whether a newly announced system improves long-horizon reliability. One source is a product announcement, another is an earlier technical report, and a third is a revised paper whose evaluation changes the reported gain. A whole-query embedding may rank the announcement first because it repeats the system name and the phrase "long-horizon." The decisive evidence instead requires identifying the version lineage, the evaluation metric, and whether the tasks match the stated reliability claim.

A suitable evidence plan separates these needs. First resolve the canonical identity and versions. Next retrieve the method and evaluation protocol. Then locate the result table and limitations. Finally compare whether "reliability" means mean success, all-repeated-runs success, or a best-of-many metric. This is a constructed example, not a claim about a particular named system. It shows how retrieval targets can depend on earlier findings instead of being fully specified by the original query.

Represent the dependency as a directed acyclic graph of evidence requirements: identity precedes version-specific result selection; metric definition precedes interpretation; task distribution precedes transfer claims. The graph is a planning aid rather than a world model. A node can be unresolved, supported, contradicted, or unavailable. Reaching a final answer while an essential node remains unresolved should produce a qualified conclusion rather than a fabricated bridge.

Suppose the first retrieved result reports any-of-five success and the revised paper reports repeated-run reliability on a different split. Subtracting the two percentages would be invalid even if both were copied perfectly. Retrieval accuracy cannot repair a comparison-axis error. The synthesis stage must align attempts, denominators, splits, backbones, and budgets before ranking methods. A structured evidence table is often more reliable than a generated prose comparison because it makes missing fields explicit.

The proposed counterfactual test removes one requirement at a time. Hide the revision notice, hide the metric definition, or replace the primary report with a secondary summary. Observe which claim changes and whether the agent detects the missing dependency. A successful system should be able to revise its conclusion after new evidence arrives without discarding unrelated valid findings. This measures evidence integration and update behavior, not merely answer recall.

The same structure applies to execution. Before a tool action, the agent may need an object identifier, current resource version, relevant constraint, and authorization. These can come from different sources and have different freshness requirements. Grounded execution therefore extends retrieval beyond document question answering: it assembles a justified action under current state and authority, with the supporting evidence preserved in the run record.

16.7 Evaluation contamination and uncertainty

A retrieval evaluation should specify the unit of relevance and the unit of success. Passage relevance can be judged independently of answer generation, while task success includes retrieval, reasoning, and execution. Report both when possible. An oracle evidence condition identifies a useful ceiling, but it must be labeled as diagnostic because a real system does not know the decisive evidence in advance.

ReFind supplies a concrete counterweight to assuming that more elaborate memory structure is necessarily better. It retains raw chat turns, uses lexical indexing, and gives an iterative agent controls for session-aware rank fusion, neighborhood expansion, time filtering, and previously inspected sessions; a separate stage answers from saved evidence [147]. The absence of generated semantic structure does not mean the absence of organization: session boundaries and timestamps remain essential. Its reused leaderboard baselines should be distinguished from its matched control experiments. The scientific comparison is thus not “raw versus structured” in isolation, but offline abstraction versus query-time search under specified access controls,

backbone, final evidence, and complete cost. This baseline is especially relevant before attributing gains to a graph or to expensive LLM-based index construction.

Use retrieval ablations that preserve final context length. Comparing a graph method returning many rich records with a flat baseline returning fewer tokens confounds structure and evidence volume. Similarly, comparing an active retriever with a one-shot baseline under equal retrieval-call limits can conceal additional model computation. Match either complete resource budgets or report a Pareto surface over success, token use, tool interactions, latency, and construction cost. One scalar rank rarely captures all relevant tradeoffs.

Contamination can enter through the index, training data, task generator, and reranker labels. A trajectory index containing near-duplicate evaluation solutions can create a useful reuse system while weakening a claim of novel-task reasoning. Separate the producer and consumer splits, record artifact timestamps, and audit duplicate or templated trajectories. If synthetic queries are generated from known evidence, evaluate whether their wording makes the source unusually easy to retrieve.

Longitudinal stores introduce order dependence. A task can become easier because a previous evaluation task inserted its answer into memory. This may be an intended continual-learning setting, but it is not equivalent to an independent held-out test. Declare whether updates are allowed, reset stores between conditions where required, and report frozen-index and evolving-index results separately. Paired direct and indirect questions should read the same frozen state when the goal is to diagnose implicit relevance.

Uncertainty should include tasks, corpus samples, and stochastic access policies. Repeating decoding on a single fixed corpus does not estimate robustness to document composition. Rebuilding an index with a different embedding model can change candidate sets and calibration, not just vector quality. Treat index generation as part of the configuration. Judge-based relevance and citation scoring should be checked against an independently reviewed subset, with disagreement retained.

None of the proposed evaluations has been run for this monograph. The reviewed 2026 methods provide concrete mechanisms and author-reported evidence under their stated settings. A rigorous survey can compare those mechanisms, identify causal confounds, and derive discriminating tests without presenting new measurements. That boundary is especially important when active retrieval is advertised as a general solution to memory: its benefits remain conditional on what was stored, how it was

indexed, and what the policy can infer.

16.8 An execution contract for grounded retrieval

A proposed grounded-retrieval contract has six stages: define the information need, enforce access, retrieve candidates, verify source/version identity, assemble evidence with roles, and validate the resulting claim or action. Each stage emits inspectable artifacts. The contract is not a universal algorithm and does not guarantee that a model reasons correctly. It specifies the interfaces at which errors and unsupported transitions can be detected.

For execution, augment the evidence packet with freshness requirements and a precondition check. A retrieved resource state may have changed between search and tool dispatch. If the action depends on that state, revalidate it or use a conditional write supported by the remote service. Retrieval confidence cannot substitute for concurrency control. A perfect passage about an old state can justify the wrong present action.

Index compression and evidence compression should be audited separately. A fixed-dimensional document vector can discard the distinction needed to select a source, while a later passage packer can discard that distinction after the correct source was selected. Increasing the generation window repairs neither failure automatically. In a constructed diagnostic, first supply the correct document identifier while preserving the ordinary passage selector; then supply the decisive passage while preserving the ordinary packer. The two interventions locate losses at different boundaries. They are oracle diagnostic conditions, not deployable baselines. Account for the document-search, passage-scoring, and generation costs independently, and retain the final evidence spans so that the apparent benefit of a larger index cannot be confused with a larger model-visible evidence budget.

For scientific synthesis, retain the distinction among author-reported results, independently reproduced results, deductions, and proposed experiments. A retrieved repository can establish that code exists at a commit; it does not establish that its published experiments reproduce. An official product document can establish a documented interface; it does not reveal every proprietary implementation decision. The output should carry the same evidence boundary that the source supports.

Current 2026 retrieval mechanisms demonstrate several directions: query-conditioned runtime construction, associative traversal, shared procedural continuations, and calibrated influence of retrieved atoms. They solve different parts of the chain. JAM's preserved raw pages address pre-query information loss, MRAgent's active access addresses search adaptation,

MATM's state keys address procedural matching, and MemCalib addresses use after evidence is supplied. Combining them is a design hypothesis, not an established composite result.

The original synthesis is to view retrieval as a controlled transition from stored evidence to active decision state. The transition must preserve source identity, role, time, and access scope as well as content. Its success is measured by justified behavior under bounded cost, not by the number of records returned. Chapter 17 places this transition within a unified consistency model linking the stored history, assembled context, policy, tools, and environment.

Chapter 17

Conditioning Consistency across Context, Memory, and Policy Updates

MemoryWalker identifies training histories that disagree with actual edited rollout views; October's CODA study examines failures when skills and their consuming policy co-adapt [23, 156]. Together they expose two separate consistency questions: whether a likelihood is conditioned on the right information, and whether a changed component preserves intended behavior. Neither question is settled by preserving archive bytes or database transaction order.

17.1 One decision depends on several versions

An agent's behavior depends on more than its model weights. The input view, memory store, retrieval index, tool schemas, harness, authorization, and environment each contribute to the conditional decision. Two runs using the same named model can implement different policies if any of these components changes. Conversely, identical stored text can produce different behavior when a new policy interprets it. Consistency is therefore a relation among components and transitions, not a property conferred by one database.

Define the complete system configuration at a decision boundary as

$$Z_t = (H_t, \mathcal{C}_t, M_t, X_t, \theta_t, \delta_t, h_t, \mathcal{T}_t, A_t, \lambda_t, \nu_t, \alpha_t), \tag{17.1}$$

where H_t is recorded evidence, $\mathcal{C}_t$ the active view, M_t memory, X_t environment state, θ_t model parameters, δ_t deployment decoding, h_t harness behavior, $\mathcal{T}_t$ tool contracts, A_t trusted authority state, λ_t the effect ledger, ν_t compatibility metadata, and α_t the current authority epoch. Not every component is fully observable, particularly the external environment and proprietary model implementation. A consistency claim must state which components it constrains and which remain outside the guarantee.

Let $g_{\nu,h}$ be the versioned context-assembly map, including selection and serialization. Any random assembly choices are retained as decision

evidence or as a materialized view, rather than assumed recoverable from a seed alone. Let A_t^{vis} be the policy-safe projection of authority; the trusted monitor can retain more privileged information. The request is part of H_t. The deployed proposal law is

$$\mathcal{C}_t = g_{\nu_t,h_t}(H_t, M_t, \mathcal{T}_t, A_t^{\mathrm{vis}}), \qquad P(a_t \mid Z_t) = \Pi_{\theta_t,\delta_t,h_t}(a_t \mid \mathcal{C}_t, \mathcal{T}_t), \quad (17.2)$$

followed by a current-authority enforcement and execution transition. The law includes the parser and stopping semantics of Chapter 2. Governing implementations remain fixed within a comparison epoch; selecting a versioned skill or editing its runtime input can change state without changing its interpreter. A changed executable resource or model checkpoint requires an explicit configuration transition. Restoring ν_t cannot restore an expired grant or revise λ_t without new effect evidence. This formulation is original synthesis. It makes explicit that assembly and monitoring can change behavior without a weight update. It also identifies why a model-only benchmark cannot certify a complete deployed agent.

Consistency has several meanings. Conditioning consistency preserves what was visible when an action was sampled. Semantic consistency preserves the status and scope of assertions across memory transformations. Transactional consistency governs concurrent reads and writes. Computational consistency governs whether cached states correspond to the input being evaluated. Authority consistency preserves the current allowed-action boundary. These properties interact but are not equivalent; satisfying one should not be described as satisfying all.

The three contracts in Chapter 1 provide a common test across those mechanisms. Proposal preservation constrains the distribution produced from a transformed view. Authority nonexpansion constrains the admissible effects, even when proposals change. Effect-state preservation constrains the ledger needed to distinguish pending, committed, and reconciled operations. Conditioning consistency adds a training obligation: an unchanged terminal artifact cannot justify assigning an earlier action a history it never saw. Thus a context edit can preserve authority and effect records while changing proposals, and a perfect behavior replay can still be invalid under revoked authority. The component checks need to identify which contract they establish.

The 2026 context-management literature provides concrete instances. MemoryWalker examines rollout and training views [23]; CLM couples editable context with serving behavior [26]; HyperSkill changes reusable memory while retaining a task-level retrieved context [143]. Their mechanisms occupy different transitions in Z_t. A coherent architecture must

specify how those transitions compose rather than assuming that each component's local improvement remains beneficial after integration.

Figure 17.1 compares action laws under a fixed deployment and different views of common recorded history.

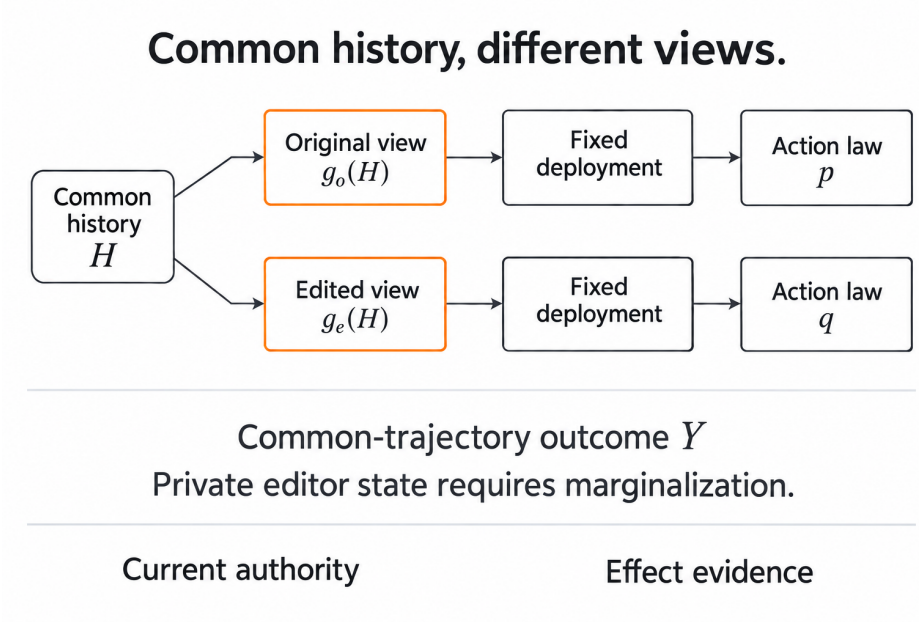

Figure 17.1: Different deterministic views of common permitted recorded history H can induce different action laws p, q under one fixed deployment. Model/tokenizer, decoder, parser and decision interface are held fixed. The behavioral bound concerns outcomes Y on the common trajectory; random private editor state requires conditional marginalization. Two sampled prompts do not establish that premise. Current authority and effect evidence remain independent checks.

17.2 The conditioning record is part of the data

For a sampled token or action y_t, let c_t^b be the actual behavior context and $q_t(y) = \pi_{\theta_b,\delta_b}(y \mid c_t^b)$ its behavior distribution. A likelihood term interpreted as behavior replay requires the actual decoded conditional law and relevant record, not merely the final transcript. Token laws and parsed-action laws are different sample spaces, as established in Chapter 2. Privileged teacher labels remain possible supervision, but are not exact replay probabilities. An edit at a later time can remove information that was present earlier; an

append-only physical log can retain information that a later decision no longer saw. Either reconstruction can therefore be wrong for a particular target.

MemoryWalker identifies these two flattening failures and gives exact tree-consistent reconstructions plus an approximate distillation approach [23]. The exact methods preserve the live conditioning path; their computational and masking requirements differ from ordinary single-sequence replay. Approximate alignment does not establish exact equivalence. The following analysis develops a general contract rather than reporting additional paper results.

Store a context-view identifier with each sampled span, together with its model, tokenizer, serialization, position treatment, and attention-mask version. The record should permit reconstruction of the exact visible prefix at that span's generation time. Recording only which raw messages existed is insufficient because an editor can select, reorder, summarize, or relabel them. Recording only the final visible messages is insufficient because later edits can change earlier conditioning retroactively in the reconstructed tape.

Consider an observation o that is visible when action a_1 is sampled and evicted before action a_2. Training both actions on a final compressed history removes o from a_1's conditioning. Training both on the full physical log restores o for a_2. The correct record has different views for the two actions. This example is a direct consequence of time-indexed conditioning; it does not depend on a particular model family or on whether the edited observation was useful.

An implementation can represent the views as immutable segments with references to shared prefixes, or as a tree with visibility masks. Storage deduplication can preserve shared content while retaining distinct conditioning paths. Numerical equivalence still requires an execution path that implements the intended positions and masks. A mask with the correct visible token set can be wrong if position identifiers differ from rollout, and a correct logical view can be wrong if a stale cache is reused.

The practical invariant is local and inspectable: every target interpreted as behavior replay should identify the conditional input and sampling law that generated it. Tests can compare replayed logits with recorded rollout logits under a frozen model, allowing for documented numerical differences. A small average discrepancy is a diagnostic, not a universal guarantee; rare large mismatches can dominate importance ratios or affect consequential actions.

17.3 Likelihood ratios require support and the right history

Suppose a target deployed policy $\pi_{\theta,\delta}$ is evaluated on actions sampled by q. At a fixed recorded information state c_t, a one-step importance ratio is

$$w_t = \frac{\pi_{\theta,\delta}(a_t \mid c_t)}{q(a_t \mid c_t)}. \tag{17.3}$$

The denominator is the probability under the actual sampling procedure, including temperature, truncation, and any constrained decoding. Recomputing a base-model softmax without those operations does not necessarily recover q. If behavior used a top-k support restriction, the target must be absolutely continuous with respect to that support for unbiased off-policy reweighting over the same state distribution.

Changing c_t is not an innocuous implementation shortcut. A denominator evaluated on a reconstructed context $\tilde{c}_t$ describes a different conditional experiment. Even if the model weights are identical, $\pi_{\theta_b,\delta_b}(a_t \mid \tilde{c}_t)$ need not equal the probability that generated a_t. A ratio formed with that value is not the original importance ratio. Clipping it can stabilize an optimizer while leaving the conditioning error conceptually unresolved.

For a full trajectory with fixed environment dynamics and compatible histories, the likelihood ratio includes a product of action-probability ratios. Long horizons can produce high variance because several modest deviations multiply. If memory or harness versions also differ, the trajectory law includes those transition policies as well. Reweighting only language-model tokens does not automatically correct a changed memory admission rule or a changed tool router. The estimated objective must identify which parts of the generative process are being corrected.

Make that boundary explicit by recording both a context-control action u_t and a task action a_t. Let s_t be the complete recorded information state before u_t; behavior uses $\mu_b(u_t \mid s_t)$ and $\pi_b(a_t \mid c_t)$, while the target uses $\mu(u_t \mid s_t)$ and $\pi(a_t \mid c_t)$. If both processes share the initial law, deterministic assembly $c_t = g(s_t, u_t)$, and environment transition kernel, then on common support

$$\frac{dP_{\mu,\pi}}{dP_{\mu_b,\pi_b}}(\tau) = \prod_{t=0}^{T-1} \frac{\mu(u_t \mid s_t)}{\mu_b(u_t \mid s_t)} \frac{\pi(a_t \mid c_t)}{\pi_b(a_t \mid c_t)}. \tag{17.4}$$

The common factors cancel; stochastic memory writes require additional factors unless their kernels are shared. An editor that deterministically retains a span in behavior but deterministically deletes it under the target can violate trajectory support. A token ratio computed after deletion

does not create the missing editing transition. If assembly itself changes, it must be represented in the state or transition law rather than silently treated as the same g. This identifies exactly when an LM-only correction is incomplete without implying that an empirical approximation is unusable.

The support condition is particularly important after tool-schema changes. A new tool action may have zero probability under the old behavior policy because the tool did not exist or its arguments were forbidden. No finite weight can recover samples of that action from an old dataset. Additional exploration or a different estimand is required. Calling the update "off-policy corrected" does not remove missing support.

Privileged training information creates another boundary. A teacher can condition on a successful solution or an oracle skill that the deployed policy does not receive. Distillation can transfer useful behavior, but the training conditional is intentionally different. That should be described as teacher-guided learning, not exact replay of the deployment process. The question becomes whether the student generalizes under its actual information state, with contamination and privilege leakage controlled.

These are mathematical deductions from conditional probability and change of measure. They do not imply that every approximate training pipeline fails in practice. They identify what an exact claim would require and why empirical success alone does not validate an incorrect likelihood interpretation. The appropriate experiment records behavior probabilities, reconstructs views, checks support, and reports the sensitivity of outcomes to the approximation.

A numerical construction illustrates why the wrong history cannot be repaired by a ratio computed more precisely. Suppose a binary observation o is equally likely to be zero or one. The behavior policy selects $a = 1$ with probability 0.9 when $o = 1$ and 0.1 when $o = 0$. After the observation is removed, the marginal probability of $a = 1$ is 0.5. For a sampled pair with $o = 1, a = 1$, the actual denominator is 0.9, not 0.5. Using the marginal denominator inflates a target probability of 0.9 from an exact ratio of one to a spurious ratio of 1.8. No floating-point improvement fixes that conditioning substitution. These values define a hypothetical probability model; they are not measured model outputs.

If the target policy genuinely does not observe o, its numerator should use the target's restricted information state. The trajectory estimator must still account for how the data were collected and what quantity is being estimated. One cannot silently replace both numerator and denominator with a new marginal experiment and claim equivalence to the original conditional objective. The distinction is especially consequential when

outcome or advantage estimates also depend on the missing observation.

The same construction can be implemented as a controlled rollout test with a visible bit, an edit, and a later tool decision. Record the generated action probability before editing, then compare replay under the live view, the final compacted view, and the full log. The expected ordering follows from the constructed distributions only if the model actually learns those distributions; an empirical study must measure rather than assume that condition. This separates a mathematical counterexample from a claim about how a particular language model behaves.

17.4 A bounded behavior-preservation proposition

Consider two finite-horizon agent variants on a common observation/action/environment trajectory space. Let h_t be its history immediately before action t, excluding variant-private context, memory and editor metadata. Both variants have the same initial law and the same nonpolicy observation/environment kernels conditional on this history and action, and use a common complete action alphabet including failure or termination outcomes. The horizon is $T \in \mathbb{N}_{>0}$; terminated runs are padded by the same absorbing outcome. The laws $p_t(\cdot \mid h_t)$ and $q_t(\cdot \mid h_t)$ are the actual action distributions conditional on this common history. For deterministic context constructors they are obtained by applying each constructor and the deployed decoder to h_t. With random private editor state, they must instead marginalize its conditional law; a model distribution at one sampled prompt does not establish this premise. At every matched reachable history, assume $\mathrm{TV}(p_t, q_t) \le \varepsilon_t$, where $0 \le \varepsilon_t \le 1$. Couple initial states identically and couple actions maximally while the common histories agree. Conditional on no earlier divergence, agreement at step t has probability at least $1 - \varepsilon_t$. Therefore, for the same $[0,1]$-valued functional Y measurable on this common trajectory space,

$$|\mathbb{E}Y_{\mathrm{orig}} - \mathbb{E}Y_{\mathrm{edit}}| \le 1 - \prod_{t=0}^{T-1}(1-\varepsilon_t) \le \min\left(1, \sum_{t=0}^{T-1}\varepsilon_t\right). \tag{17.5}$$

The first bound follows because identical trajectories have identical Y, while its difference is at most one after divergence. No independence of stepwise errors is assumed. For uniform $\varepsilon_t = \varepsilon$, the bound becomes $1 - (1-\varepsilon)^T \le \min(1, T\varepsilon)$. This is a standard coupling deduction under explicit hypotheses, not a measured guarantee for a context editor.

A KL premise permits a sharper horizon dependence than applying Pinsker separately at every step. Let P and Q be the laws of the common trajectory projection just defined, and require $P \ll Q$. With common nonpolicy factors, the chain rule gives

$$\mathrm{KL}(P\|Q) = \sum_{t=0}^{T-1} \mathbb{E}_{h_t\sim P}\, \mathrm{KL}\{p_t(\cdot \mid h_t)\|q_t(\cdot \mid h_t)\}. \tag{17.6}$$

Thus, if this sum is at most a finite $K_T \geq 0$, the bounded-functional variational inequality and Pinsker's inequality yield

$$|\mathbb{E}Y_{\mathrm{orig}} - \mathbb{E}Y_{\mathrm{edit}}| \leq \min(1, \sqrt{K_T/2}). \tag{17.7}$$

Natural logarithms and $\mathrm{TV} = \sup_B |P(B)-Q(B)|$ are used [30]. A uniform conditional KL bound η implies $K_T \leq T\eta$, giving $\min(1, \sqrt{T\eta/2})$. The stronger population premise can also hold as an expectation under the original trajectory distribution without being uniform. A finite average over unrelated ordinary prompts establishes neither premise. Estimating the relevant expectation still needs the sampling and uncertainty controls of Chapter 2; rare missing obligations can remain untested.

The histories in these comparisons are matched common histories, even though each variant constructs a different model-visible view from them. The result bounds observable task outcomes on this projection. It does not bound functionals of private memory, context strings or internal metadata: those can differ even when every action agrees. Enlarging the trajectory to include such variables requires their transition kernels in the likelihood ratio and coupling. Changing environment dynamics, tool effects or authorization kernels likewise invalidates cancellation unless those changes are modeled. Once coupled trajectories diverge, later inputs may differ arbitrarily; the coupling bound does not assume they remain close. A cumulative reward requires its own bounded range rather than silently reusing the terminal $[0, 1]$ scale.

There is a separate exact enforcement property. If a trusted monitor admits only actions in its current allowed set and mediates every consequential effect, a context edit cannot dispatch an outside action through that monitor. The property depends on correct interpretation, current authority and complete mediation; it still permits allowed but wrong actions. It does not establish effect-ledger truth or task correctness. This keeps the proposal, authority and recorded-effect contracts separate.

For a falsifiable experiment, freeze the decoder, parser, tools, environment fixture and current authority, then compare next-action laws at matched pre-edit states. Report the worst observed cases, missing support, and

the workload sampling distribution. Neither an empirical maximum nor a few repeated continuations establishes a global supremum over histories. The derivations identify what must be bounded; they do not supply the required evidence by themselves.

17.5 Memory transactions and semantic validity

Transactional consistency preserves relationships among reads and writes; semantic validity concerns whether the recorded assertions and procedures are appropriate. A database can provide atomic updates that commit a false summary perfectly. A language model can produce an accurate summary whose references become inconsistent because another writer changes the source before commit. An agent memory system needs both a storage contract and a semantic admission contract.

Let a proposed update read objects R at versions v_R and produce writes W. An optimistic commit validates that the relevant read versions still match. If not, the update should be recomputed, merged under a justified rule, or rejected. The hard part is identifying the relevant read set. Generated abstractions can depend on a source that was seen earlier but omitted from the recorded extraction input. Missing provenance defeats conflict detection even if the database transaction implementation is correct.

The RecMem consolidation path in Chapter 14 illustrates a multi-layer publication boundary: episodes, semantic facts, and raw-memory deletion are distinct operations [136]. HyperSkill maintenance similarly changes nodes, embeddings, and membership relations [144]. A partially persisted update can make one representation disagree with another. The required invariant is not simply "all writes completed," but that every published object has valid dependencies and can be interpreted under the same versioned schema.

Semantic validity needs richer conflict classes. A later observation can supersede a current fact without invalidating the truth of its historical episode. A skill can become incompatible with a new tool while remaining an accurate record of a past successful procedure. An authorization can expire without deleting the evidence that it once existed. These cases require status transitions, not indiscriminate overwrite or deletion.

Linearizable memory operations can support reasoning about a single shared object's update order [130]. They do not determine whether the chosen semantic resolver is correct. A last-write-wins fact register can be linearizable while choosing the wrong assertion because the later writer had lower authority or described an older valid-time interval. Storage

guarantees should therefore be stated using the actual abstract data type and application semantics.

A proposed semantic transaction writes candidate assertions with source links, validates role and authority, resolves temporal status, and atomically publishes a coherent view. If validation is model-based, retain its uncertainty and evidence rather than calling the transaction verified in the formal sense. For high-consequence obligations, deterministic fields can constrain what the model may revise. The goal is not to make every memory fact formally provable, but to prevent unexamined semantic changes from masquerading as ordinary storage maintenance.

17.6 Model index and tool migration

Memory portability depends on the consumer. A concise skill written for one model can rely on its implicit knowledge or compensate for its characteristic mistakes. A new model may interpret the same text differently. A new embedding model can change the geometry of queries and records. A new tool schema can invalidate a previously successful procedure. Version identifiers make these changes visible but do not establish compatibility.

An embedding example makes the issue precise. Suppose document vectors d and queries q originally share a space with score $q^\mathsf{T} d$. A replacement encoder applies an orthogonal rotation R to both, preserving within-space dot products. If only queries are updated, the score becomes $(Rq)^\mathsf{T} d$, which generally differs from $q^\mathsf{T} d$. Even an information-preserving coordinate change breaks mixed-version retrieval. Re-embedding both sides or using a validated alignment is required. This is a mathematical example, not a benchmark result about named embeddings.

For policy migration, preserve the old memory snapshot and compare the new model on matched tasks with no memory, original memory, and a revised memory representation. This separates backbone changes from memory compatibility. Include cases where the old memory should be ignored. A new model that applies every old instruction more aggressively can improve some recall tasks while violating current constraints. Positive transfer and overuse need separate outcomes.

Tool migration can be more discrete. Changing an argument's default, resource scope, or side-effect semantics can invalidate a skill even when its function name remains unchanged. Attach schema and behavior versions to the skill's compatibility record, and require fresh validation when relevant contracts change. A schema-compatible change can still alter semantics; syntactic validation is necessary but insufficient.

CLM's inspected environment shows a useful local boundary: `ContextEnv.step` writes the live mirror, executes a command, reads the mirror back, and applies a parsed edit through `_apply_edit`. The edit gate checks growth against a configured rule, and the parser retains a protected prefix [124]. This validates some structural and budget conditions, not the truth of every rewritten assertion. Migrating that environment to a new tokenizer or parser changes what the gate counts and which messages it reconstructs.

A successful edit is also not synonymous with fitting the token budget. In the inspected `_apply_edit` branch, growth is subject to the configured growth/fit rule, but a shrinking edit can be applied while the result remains over the limit; the readout then requests further compaction [124]. The synchronization counter likewise does not prove behavioral preservation. An integration should record applied versus rejected edits, final enforced token counts, and the actual next-decision input separately. Structural acceptance supplies no bound on P_ϵ.

A migration gate should therefore cover the entire path from stored record to dispatched effect. Record old and new versions, representative compatibility tests, unresolved failures, and rollback criteria. If a proprietary model version cannot be pinned exactly, state that reproducibility limit and use dated API identifiers and observed behavior. A rolling service name is not a reproducible policy identity.

The October 2026 skill–policy co-evolution study makes this dependence experimentally explicit through CODA. It perturbs the delivered skill bank and evaluates consistency under meaning-preserving rewrites, resistance to corrupted guidance, and worst-case success [156]. Its theory concerns candidate coverage and a specified cooperative acceptance rule, not every conceivable self-improvement objective. Its stress-test losses also do not estimate how frequently equivalent drift occurs in deployment. The result is a concrete warning against evaluating only the policy-bank pair that was selected together.

A corresponding original diagnosis separates three perturbations. Rewrite surface form while preserving procedural meaning, alter valid preconditions while retaining familiar wording, and replace the consumer policy while freezing the bank. The first tests interpretive invariance, the second tests applicability sensitivity, and the third tests portability. Pooling them into one robustness number can reward the wrong behavior: a policy should ignore irrelevant wording changes but respond to a changed precondition. Consistency is therefore invariance to specified transformations and sensitivity to others, not indiscriminate resistance to change.

17.7 Authority must survive every transformation

Authority is information about who may direct which effects under what conditions. It should not be inferred from memory frequency, semantic relevance, or the confidence of a generated summary. A retrieved page can be highly relevant to a task while being untrusted as an instruction source. A stored skill can be useful while lacking authority to change the task. A prior session can contain valid historical permission that no longer applies.

Progent represents allowed tool calls through symbolic policies and distinguishes narrowing from expansion [24]. Under complete mediation, an unapproved update can be restricted to a subset of the previous allowed set. Write this as $\mathcal{A}_{t+1} \subseteq \mathcal{A}_t$ for automatically accepted narrowing. The comparison must use the same current authority epoch; an allowed set restored from an old snapshot is not the current grant. The guarantee depends on correct policy interpretation, a trusted enforcement boundary, and every consequential call passing through it. It does not guarantee that all allowed calls are useful or semantically correct.

Compaction can preserve authority in two ways. It can retain the original authoritative statement verbatim, or it can preserve a validated typed representation whose semantics are enforced independently. A generated paraphrase without validation can weaken or broaden a condition. For example, "prepare a draft for review" can become "complete the publication task" after repeated summarization. The model may then believe a broader objective even if the external monitor still blocks publishing. Preserving both behavioral guidance and enforcement reduces this mismatch.

Concurrent updates create races. A worker may choose an action under authority version a while a user narrows scope to $a + 1$. Checking permission only at worker creation is insufficient if the effect occurs later. The dispatcher should evaluate the current applicable policy or require a validated authorization lease whose semantics cover that interval. Canceling the worker is not equivalent to preventing an already dispatched effect.

The proposed test records authority at task creation, context assembly, action proposal, and effect dispatch. Inject a narrowing correction between each pair of stages. Check whether stale proposals are rejected and whether the user-visible state accurately reports any in-flight uncertainty. This targets authority consistency directly instead of relying on a general prompt-injection benchmark to discover the race accidentally.

Authority preservation also constrains learning. A memory writer should not turn a successful prohibited action into a reusable recommendation merely because the environment rewarded it. Outcome feedback and

authorization are separate channels. A system can perform well on a task-success metric while learning procedures that violate its intended scope. The evaluation must retain both dimensions and report their interaction.

17.8 A consistency audit with causal counterexamples

A proposed audit begins with a versioned dependency manifest for each consequential decision. It records the active context, memory snapshot, retrieved evidence, policy configuration, tool schema, authority, and relevant environment observations. The manifest is not a claim of full observability. It states which dependencies are known and makes missing dependencies visible. A reconstruction that silently substitutes current versions defeats the purpose.

The first test class targets conditioning: restore the same frozen model and compare the recorded live input with training reconstruction. The second targets semantic updates: create near-identical records with different valid times or authority and inspect whether consolidation preserves the distinction. The third targets migration: update only the query encoder, tool schema, or model consumer and measure compatibility failures. The fourth targets concurrency and effect timing. These tests exercise different consistency properties and should retain separate results.

Counterexamples should be minimal enough to identify the missing contract. If a summary loses a publication restriction, the failure record includes the source restriction, summary, action proposal, and monitor outcome. If an index returns a stale passage, preserve the source version and embedding generation. If a skill overgeneralizes after a model upgrade, compare its applicability and execution under both consumers. A final wrong answer alone does not identify which transition failed.

Causal attribution requires controlled substitutions. Change one component at a time where feasible, but acknowledge interactions. A new model may need a different prompt format, making a literal prompt hold-fixed comparison unrepresentative. In that case report both a strict component substitution and an adapted configuration, charging the adaptation effort separately. This avoids treating either one comparison as the only legitimate view.

No new audit outcomes are claimed here. The propositions and protocols are deductions and proposed experiments grounded in the mechanisms reviewed. Their purpose is to turn vague continuity claims into explicit conditions that can fail. A coherent agent architecture should be able to

state which transitions are exact, which are approximate, which depend on model judgment, and which are unknown.

The cross-layer conclusion is that context, memory, and policy must evolve under compatible contracts. More storage does not repair wrong conditioning; correct transactions do not make a false abstraction true; a current model does not make old skills compatible; and a well-formed tool call does not establish authorization. Treating these distinctions together provides a technical basis for reliable stateful execution without claiming a general solution to intelligence.

Chapter 18

Tool Discovery, Routing, and Typed Execution Contracts

HyperTool's 2026 tool blocks change execution granularity, while selective delegation changes which worker acts and a privilege monitor constrains admissible effects [157, 158, 24]. These interventions address different boundaries. Their comparison must retain argument validity, dispatch authority, intermediate state, resource accounting, and the distinction between a retrying language-model trace and a repeated external operation.

18.1 A tool call is a typed state transition

A tool exposes a capability through an interface that an agent can invoke. Its scientific description should include input and output types, preconditions, side effects, failure states, authority requirements, and version identity. A natural-language name and argument schema provide only part of that contract. Two tools with identical JSON shapes can have different resource scopes or effect semantics. A model can produce a syntactically valid call that is operationally wrong.

For environment state space $\mathcal{X}$, argument space $\mathcal{U}_j$, observation space $\mathcal{O}_j$ and receipt space $\mathcal{E}_j$, represent tool j by a transition kernel

$$\mathcal{T}_j(dx', do, d\eta \mid x, u, A), \quad u \in \mathcal{U}_j. \tag{18.1}$$

Here A is current trusted authority, $o \in \mathcal{O}_j$ an observation and $\eta \in \mathcal{E}_j$ a typed receipt or uncertainty state. Rejection, timeout and partial completion need explicit outcomes; a partially observed kernel does not give the model access to x or to privileged authorization state. A successful HTTP response is not necessarily proof that the intended business effect occurred; a timeout is not proof that it did not. Chapter 13 analyzes the resulting recovery ambiguity.

The model's tool-use decision has at least three stages: select a capability, construct its arguments, and decide how to interpret its result. Toolformer and ReAct are foundational examples of learning or prompting tool-augmented behavior [29, 1]. They should not be treated as descriptions of every current runtime. Contemporary systems add discovery,

typed schemas, external enforcement, code-based composition, and durable execution. Each addition changes the effective policy and its failure surface.

A contract should identify which fields are exact. Resource identifiers, operation keys, and authority scopes should not be reconstructed from a vague summary when exact values are available. Free-text queries can remain flexible. Output records should separate authoritative data from tool-generated explanations and untrusted external content. Returning a page that contains instructions does not grant those instructions authority over the agent. The wrapper must preserve that distinction when the result enters active context.

For evaluation, define success at the tool boundary and at the task boundary. A schema-valid call can fail a tool precondition. A tool-successful call can be irrelevant to the task. A task can appear solved while the agent produces an unauthorized side effect. These outcomes need separate labels. A single "tool accuracy" number is ambiguous unless it specifies whether it measures tool selection, argument validity, execution, or complete justified behavior.

18.2 Discovery changes the action space

Discovery provides the agent or harness with the currently available capabilities. It can involve a static catalog, server-provided schemas, search over tool descriptions, or staged loading of detailed definitions. The catalog is part of the policy's information state. Adding or removing tools changes the represented proposal interface and can alter performance even when the model weights and task distribution remain fixed. Discovery does not itself grant authority: the dispatcher independently restricts which discovered operations may execute.

Large catalogs create both token and discrimination costs. Injecting every complete schema can consume context and expose near-duplicate actions. A two-stage mechanism can retrieve candidate tools using coarse descriptions and load exact schemas only before invocation. This reduces active input but can exclude the necessary tool at the discovery stage. Evaluation should report catalog recall separately from correct use among discovered candidates. An oracle candidate list is a diagnostic condition, not an executable baseline.

Tool descriptions are themselves data that can be stale, misleading, or adversarial. A server can expose a harmless-looking title while its implementation has broader effects. A retrieved description can include instructions unrelated to the capability. Treating the description as an

authoritative system instruction expands the trust boundary. Metadata can guide selection without being allowed to redefine the user's task or authority.

Current MCP separates discovery and invocation through its tool feature and schemas [159]. A capability catalog can vary with per-request authorization, so a cached list should be associated with the authorization context that produced it. The detailed protocol semantics are developed in Chapter 19. The application-level consequence is that catalog identity, credential scope, and schema version belong in the decision record. A tool visible to one principal need not be visible to another.

Discovery can also create a training-deployment mismatch. A policy trained with a fixed tool ordering can exploit names or positions rather than robust semantic selection. At deployment, reordered or expanded catalogs can degrade selection. Controlled tests should permute catalog order, introduce unrelated near-name tools, and hold schema content constant. A model that fails these tests may have learned a brittle interface shortcut rather than a transferable tool-selection mechanism.

The proposed implementation caches catalogs with explicit expiration and invalidation, stores a schema digest per selected tool, and validates the exact schema again before dispatch where required. It distinguishes a missing capability from an unavailable credential and from a temporary execution failure. Those errors lead to different next actions: revise the plan, request an authorized scope change, or retry under a bounded recovery policy.

18.3 Routing as constrained resource allocation

A router chooses among direct answering, tool execution, retrieval, specialized models, and delegated workers. Its objective should combine task value, latency, cost, and authority rather than maximizing a tool-use rate. Calling a tool unnecessarily can introduce expense and risk; refusing a necessary tool can create an unsupported answer. Tool abstention and tool invocation are both decisions requiring calibration.

Let j index a finite catalog of admissible routes and let $Q_j(s)$ estimate expected task utility in state s. Let $C_j(s)$ and $L_j(s)$ estimate resource cost and latency, with nonnegative conversion weights λ_c and λ_l. A proposed decision rule is

$$\widehat{j}(s) \in \arg\max_{j \in \mathcal{J}_A(s)} \{Q_j(s) - \lambda_c C_j(s) - \lambda_l L_j(s)\}, \tag{18.2}$$

where $\mathcal{J}_A(s)$ contains the authority-permitted routes and an explicit no-effect route $j = 0$. The latter permits abstention or a request for missing information, so the maximization remains defined when no effectful route is allowed. Ties require a declared selection rule. The estimates depend on model version, tool reliability, queueing, and remaining budget. Optimizing a router on static average prices can be inappropriate when latency or tool availability changes. Constraints should be enforced before ranking, rather than represented only as a large penalty that the model may trade away.

The estimated objective also needs an uncertainty boundary. Write v_j for the true expected net utility under one fixed configuration and $\widehat{v}_j$ for the estimate used by the router. If simultaneous bounds $|\widehat{v}_j - v_j| \le b_j$ hold for every feasible route, then, for $j^\star \in \arg\max_j v_j$ and $\widehat{j} \in \arg\max_j \widehat{v}_j$,

$$v_{j^\star} - v_{\widehat{j}} \le b_{j^\star} + b_{\widehat{j}} \le 2 \max_j b_j. \tag{18.3}$$

Insert and subtract the two estimates; the estimated-utility difference is nonpositive by selection. This conditional deduction needs simultaneous coverage, not a separate nominal interval interpreted after inspecting every route. It also assumes a correct current feasible set. Smaller utility uncertainty cannot repair a stale grant, an unmodeled side effect, or support missing from the route logs. Reusing estimates after a model, memory bank or worker interface changes requires a new compatibility argument.

If L_j is mean latency, this rule does not enforce a deadline guarantee. For random completion time D_j and deadline d, a tail-aware feasible set can additionally require

$$\mathcal{J}_{A,d,\alpha}(s) = \{j \in \mathcal{J}_A(s) : P(D_j > d \mid s) \le \alpha\}, \qquad \alpha \in [0, 1]. \tag{18.4}$$

The probability must include queueing, tool retries, and validation under the intended offered load. Two routes can have identical means and different deadline-miss probabilities. Estimating a low-probability tail from a small successful-call sample is insufficient, and timed-out calls remain in the distribution. If every useful route violates the constraint, the system must expose infeasibility or choose a declared degraded mode; replacing the empty set with an unqualified fast answer would change the requirement.

Uno-Orchestra jointly emits task decomposition and admissible model-primitive assignments rather than using an independent decomposition stage followed by a fixed dispatcher [158]. Its training uses verified worker interactions and a multi-turn objective with intermediate feedback. This is a learned orchestration policy, not evidence that every tool should be delegated. Its benchmark aggregation, worker pool, verifier, and cost boundary determine the meaning of its reported efficiency.

A router's confidence should be evaluated against route-specific outcomes. A model can be confident that a tool is relevant while being uncertain whether its argument values are correct. A worker can solve a subtask but return evidence in an unusable format. Treat route selection, execution, and integration as separate stages. A failed integrated result should not automatically label the route itself wrong if the downstream merger discarded a valid answer.

Training data for routing can be censored. Logs contain outcomes only for routes actually chosen; the unchosen alternative's success is unknown. Supervised imitation of historical routes can reproduce a prior system's biases. Counterfactual evaluation requires randomized exploration where appropriate, controlled replay, or justified off-policy assumptions. Chapter 17 explains why missing support and changed conditioning limit importance-weight interpretations.

The proposed baseline set includes a strong single route, a deterministic rule router, a cost-only router, and a learned utility router. Match complete budgets and retain unsuccessful calls. A router that improves average success by using more expensive workers has a useful capability result but has not established equal-budget efficiency. A Pareto comparison is often more informative than a single weighted score.

18.4 Argument construction and result validation

Argument generation translates a task and its evidence into an executable object. The model must identify entities, select current values, satisfy cross-field constraints, and preserve exact strings where required. JSON syntax is the easiest part of this problem. A call can satisfy a schema while naming the wrong account, using a stale resource version, or combining fields that are individually valid but jointly inconsistent.

Use layered validation. The parser checks serialization. The schema checks types and required fields. The application checks domain preconditions and cross-field constraints. The authority monitor checks whether the operation is permitted. The remote service checks its current state. These layers should return distinguishable errors so the agent can repair the responsible field without broad speculative retries. A generic failure message can induce repeated changes to already correct arguments.

Constrained decoding can reduce malformed outputs by restricting token sequences to an allowed language. It does not establish that the selected entity or action is correct. It can also alter the behavior distribution used for training and likelihood accounting. If a rollout applies schema

constraints, record them with the sampled output rather than recomputing probabilities under an unconstrained model. Syntactic reliability and statistical consistency interact at this interface.

Result validation likewise needs several levels. A parser can confirm that a returned object matches the declared type. A tool-specific validator can check a checksum, resource version, or test result. Semantic interpretation decides what the observation implies for the task. A language-model explanation generated by the tool is not automatically independent verification of the same model's action. Preserve raw evidence when it is needed to audit that inference.

One proposed argument-provenance record maps each consequential field to its source: direct user input, retrieved evidence, deterministic transformation, or model inference. This can be lightweight for read-only search queries and stricter for effects. If an identifier comes from an untrusted document rather than the authorized workspace, the dispatcher can detect the difference. The record does not make inferred values correct, but it makes their basis reviewable.

Evaluation should include near-valid errors. Test correct schemas with wrong identifiers, stale versions, units, boundary values, and conflicting fields. Include tools whose names are similar but whose effects differ. A benchmark containing only malformed JSON can overstate practical tool competence because it ignores the harder grounding problem. Conversely, a benchmark that scores only final task success can hide a substantial improvement in syntax while another subsystem remains the bottleneck.

Figure 18.1 isolates the mechanism and its stated information boundaries.

18.5 Execution granularity and local programs

One model decision per atomic tool call exposes every intermediate value to the main reasoning trace. This can be useful when each observation changes the plan, but wasteful when several operations form a locally deterministic subroutine. Code-based composition can keep temporary values inside an execution environment and return only the result needed by the outer policy. It changes execution granularity, not merely trace compression.

HyperTool uses an executable block that can call existing tools through their schemas and manipulate intermediate values locally [157]. Its method couples that interface with synthesized and verified training trajectories. The reported gains therefore involve both a new action abstraction and training for its use. A comparison that gives only one condition the

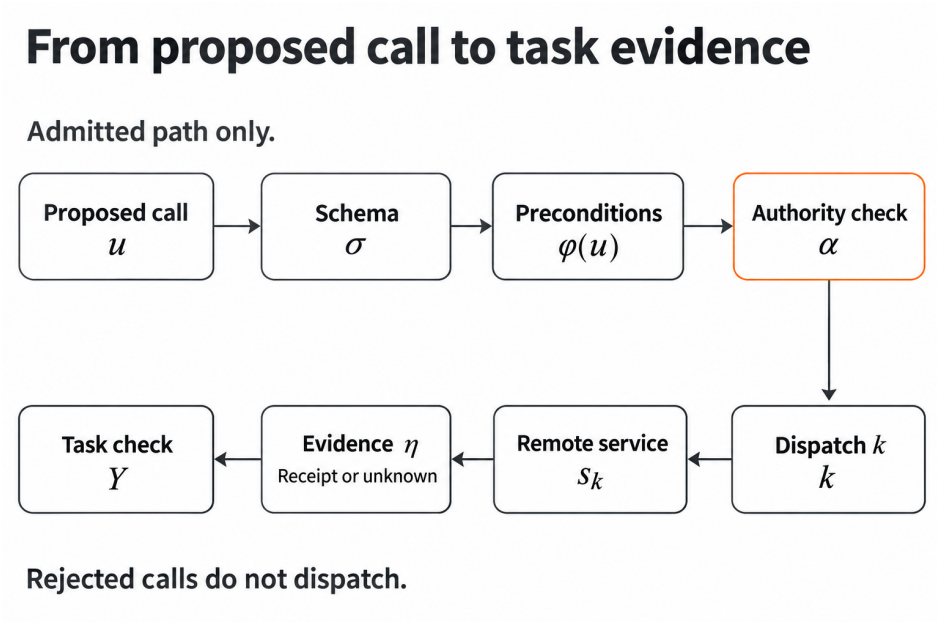

Figure 18.1: A normative admitted tool path: schema, domain and current-authority checks precede dispatch. Here α denotes the authority epoch. Actual service evidence η, including unresolved outcomes, is recorded separately from a task check Y. This schematic does not claim that framework defaults enforce every boundary, that admission and remote commit are atomic, or that a receipt proves task correctness. Figure-local σ names the schema, $\varphi(u)$ a domain predicate, k the logical operation identifier, and s_k its servicing endpoint.

specialized training cannot attribute the full result to block execution alone. The public paper now permits a method-level assessment beyond an abstract-only lead, while independent reproduction remains unestablished here.

The mathematical distinction is between choosing a sequence of atomic actions with model feedback at every step and choosing a program p whose internal transition sequence is executed under a runtime. A program can express loops, branches, and dataflow compactly. Its correctness requires both valid tool calls and correct control flow. It can reduce model-call latency while making intermediate effects less visible to the outer policy. The tradeoff depends on how often new observations should trigger reconsideration.

A constructed example retrieves two coordinates and computes a distance.

If geocoding and arithmetic are deterministic under the request, one block can pass values locally without asking the model to copy them between calls. If the first geocoding result is ambiguous, the block should expose that ambiguity or apply a declared rule rather than silently choosing a location. Grouping does not eliminate the need for checkpoints at semantically meaningful uncertainty boundaries.

Security enforcement must cover calls inside the block. Wrapping the outer executable tool while allowing unrestricted internal tool access can bypass per-tool policy. Every consequential internal invocation should preserve the principal, authority, operation identity, and audit trail. A returned concise result should retain references to the internal effect receipts. Context savings should not be purchased by deleting the evidence needed for recovery.

The proposed ablation compares atomic calls, an untrained block interface, and a block-trained policy under matched models and budgets. Measure model tokens, internal tool calls, critical-path latency, error recovery, and hidden effects. A reduction in outer calls is informative but incomplete. The relevant question is whether the new granularity preserves justified task behavior while reducing complete execution cost.

18.6 Retry pruning and failure memory

Tool failures create a sequence of attempted repairs that can dominate an agent's context. Some failures are transient; others reveal a wrong assumption or an invalid action. Repeating the same call can be appropriate for a rate limit and inappropriate for a permission denial. A retry policy should classify the failure before deciding whether to repeat, revise, replan, or stop.

PruneTIR distinguishes successful error resolution, a stuck repair trajectory, and repeated failures that justify temporary tool suspension [160]. It prunes resolved error traces, can resample from an earlier state, and can stop further tool use after retry conditions. Its primary reasoning evidence is concentrated in tool-integrated tasks such as mathematics. The observed correlation between erroneous calls and wrong answers does not by itself prove that deleting errors causes correctness; the interventions and ablations supply the relevant additional evidence.

Pruning has a semantic cost. A resolved error can contain a negative lesson needed to prevent recurrence. Removing the detailed trace from active context can be reasonable while preserving an explicit failure state or procedural warning outside it. Conversely, retaining every stack trace can

bias the model toward a failed approach. The controller should distinguish temporary execution noise from evidence that invalidates the current plan.

Resampling from an earlier state is safe only when the environment is also restored or when the removed actions had no relevant effects. Rewinding the conversation does not rewind an external service. If a failed-looking operation partially committed, retrying an alternative can create inconsistent state. A tool-pruning method evaluated in a controlled interpreter should not automatically be generalized to irreversible workflows. State restoration and effect reconciliation belong in the deployment contract.

A proposed failure memory stores error class, attempted action, relevant state version, repair outcome, and conditions under which retry became valid. It should avoid turning an environment-specific failure into a global tool prohibition. If the tool version changes, old failures may become obsolete. This connects retry control to procedural memory and temporal validity rather than treating it as a token-removal heuristic.

Testing should retain both benign transient failures and persistent semantic failures. Compare unbounded retries, fixed retry limits, typed retry rules, and learned failure-aware control. Measure completion, wasted calls, duplicated effects, recurrence of the same error, and premature abandonment. A method that reduces calls by refusing difficult tasks needs to be distinguished from one that reaches the same success rate more efficiently. All failures and abstentions remain in the denominator.

18.7 Privilege control at the dispatch boundary

Progent's concrete mechanism is a policy over tool names and argument values, checked before execution [24]. A model can propose an initial policy or an update, but deterministic analysis distinguishes a narrowing from an expansion. An expansion requires an authorized approval path under the paper's design. This is stronger than asking the same model to reconsider whether its own call is safe, because enforcement is external to the action-generating policy.

Let P denote an allowed-call set and let P' be a proposed update. A monotonic-confinement check asks whether $P' \subseteq P$. Under complete mediation and faithful policy interpretation, automatically accepting only such updates prevents unapproved expansion through that path. It does not establish that P was initially minimal, that an allowed call is semantically correct, or that all external effects pass through the monitor. Those assumptions determine the guarantee's scope.

Schema-based constraints can express useful restrictions on resource

identifiers, recipients, or operation types. They can also be too coarse. A call can satisfy a permitted path pattern while reaching a resource through an alias or redirection whose semantics the monitor does not model. A broad code-execution tool can hide many effects behind one allowed argument string. The policy language and tool wrapper must jointly model the effects being controlled; otherwise the apparent boundary can be incomplete.

Utility and security should be measured together. A policy that blocks every call achieves a trivial form of attack prevention while failing legitimate tasks. A permissive policy can complete tasks while leaving unnecessary authority open. Evaluate benign completion, prohibited-call rejection, false blocks, and unexercised excess privilege where the threat model requires it. Attack-success scores alone do not measure the entire allowed action surface.

Memory and discovery influence this boundary. A retrieved skill may propose a new tool or broader scope, and an updated catalog may expose a capability the initial policy did not include. The system should treat that as an explicit authority question rather than silently inheriting the new capability. The model can explain why an expansion would help; it should not be able to grant the expansion merely by rewriting its context.

The practical architecture places the check immediately before dispatch, with the current applicable policy and validated arguments. Earlier checks can support planning but do not cover intervening corrections. Record a receipt tying the executed call to the policy version that admitted it. This provides the causal link needed to audit whether a later undesirable effect came from policy design, stale authority, wrapper bypass, or incorrect tool behavior.

The inspected Progent implementation exposes this boundary through `check_tool_call` and wrappers in its `secagent.tool` module. Policy type checking is separate from containment analysis. In `security_policy_subset_check`, the generated-policy entries are compared through schema constraints, and an unproved containment result is rejected rather than assumed safe [161]. The source observation is narrower than a proof of the complete deployed system: it concerns the represented policy subset and the solver translation at that revision. Wrapper coverage, service effects, and manually configured policy semantics still require separate validation.

The configuration is part of this claim. At the inspected revision, `check_tool_call` warns and returns when no security policy is initialized; generated-policy containment is enabled only when `SECAGENT_ONLY_ALLOW_NARROW` is true, whose code default is false. The `tool.py` type-check wrapper does not call the separate checking helper, and that helper re-

turns warnings [161]. The existence of these functions therefore does not prove that every deployment enables confinement or rejects a malformed policy. A conformance test must bind initialization, configuration, wrapper coverage and rejection behavior before measuring benign utility or attack resistance. These source observations do not assert that the paper's evaluated configurations used the defaults.

The logical test can be expressed as searching for an argument value x satisfying $P_{\text{new}}(x) \wedge \neg P_{\text{old}}(x)$. A satisfying assignment is a counterexample to narrowing. If a represented old policy allows an amount at most 100 and the proposed policy allows an amount at most 200, $x = 150$ witnesses expansion. If the new bound is 50, the corresponding formula is unsatisfiable under ordinary numeric semantics. These numbers are an analytic illustration, not a benchmark result. The validity of the conclusion depends on whether the schema-to-solver translation faithfully captures the argument and tool semantics.

The type checker also matters because a policy restriction on an argument absent from the actual tool schema can be meaningless or malformed. Validation should reject that mismatch before enforcement, rather than allowing a generated policy to create the appearance of confinement. Conversely, a schema-valid restriction can still be inadequate if the tool accepts aliases, resolves paths, or performs downstream operations outside the modeled field. Syntactic alignment closes one failure mode without closing every semantic bypass.

For an implementation study, instrument policy generation, type-check outcome, containment outcome, admitted arguments, and the wrapped function that ultimately executes. Include solver-unknown or translation-failure cases and verify that they cannot silently broaden policy. Compare the monitor's decision with an independently specified small reference action set on synthetic tools whose semantics are known. Then test realistic wrappers to determine whether all consequential paths are mediated. This is a proposed test sequence; no additional security measurements are asserted here.

18.8 A tool-use experiment that can falsify the mechanism

A strong tool study defines comparison axes before ranking methods. For selection, measure whether the necessary capability is discovered and chosen. For arguments, measure structural and semantic validity. For execution, measure preconditions and effects. For integration, measure whether the

result changes the task state correctly. For authority, measure whether the call is permitted. Aggregating these into one score can be useful for a product decision, but the component outcomes are needed for scientific explanation.

The proposed baseline cohort keeps the underlying model and tools fixed while varying catalog presentation, routing policy, validation, and execution granularity. Include a direct single-agent baseline, a deterministic router, a learned router, and an oracle tool-selection condition for diagnosis. When specialized training is added, report it as a separate factor rather than silently comparing a trained interface against an untrained one. Count all controller, verifier, worker, and retry calls.

Task construction should include impossible requests, missing permissions, changed schemas, ambiguous entities, and tools that return plausible but stale data. A competent agent should sometimes abstain or request information. A benchmark in which every task is solvable with one known tool can reward overconfident invocation. Negative cases are necessary to measure whether the policy understands applicability and authority rather than simply matching names.

Contamination controls include tool-description overlap, released task trajectories, and environment fixtures. Training on one tool's examples can transfer useful syntax while leaking benchmark-specific workflows. Separate unseen schemas, unseen compositions, and unseen environment states. A model that handles a renamed familiar tool has achieved a different generalization target from one that composes new capabilities under a changed state distribution.

Latency and cost should include discovery, schema loading, validation, local program execution, remote calls, retries, and reconciliation. Report task-level distributions and overload behavior when concurrency matters. A block interface can reduce model transitions while increasing internal tool work; a router can improve success by choosing slower workers. Neither is intrinsically better without a workload and service objective.

The reviewed methods establish concrete design choices: learned selective delegation, inference-time repair pruning, executable local composition, and external privilege enforcement. Their mechanisms address different bottlenecks. A combined system is a hypothesis whose interactions must be evaluated, particularly where context editing changes routing evidence or where a composed block changes the enforcement surface. The next chapter treats the protocol layer that carries these tools and delegated tasks across process boundaries.

Chapter 19

Versioned Interoperability: MCP, A2A, and Protocol Boundaries

MCP's 2026-07-28 specification replaces the legacy initialization handshake with self-contained per-request version and capability declarations [162, 163]. A2A addresses a different boundary: delegated work represented through messages, tasks, and artifacts [164]. Neither protocol by itself establishes sufficient agent context, permission to execute an effect, or correctness of a returned artifact.

19.1 Protocol compatibility is not agent competence

An interoperability protocol standardizes messages and declared behavior across process or organizational boundaries. It can make tools discoverable, data addressable, and delegated work trackable. It does not determine whether an agent chooses the right capability, constructs a correct plan, or interprets evidence appropriately. A protocol-conformant system can execute a perfectly well-formed but irrelevant or unauthorized operation if the surrounding application grants it that authority.

MCP and A2A address different boundaries. MCP connects an application host and its clients to servers exposing context and capabilities. A2A represents interactions with a remote agent through messages, tasks, artifacts, and declared capabilities [162, 165]. A tool server can wrap sophisticated computation, and a remote agent can internally use many tools. The distinction is the contract exposed to the caller, not an ontological test of whether the remote implementation is intelligent.

The literature and implementation review for this chapter is current through 29 October 2026. The current MCP revision identified in the official versioning documentation is 2026-07-28 [166]. The versioning policy permits backwards-compatible changes while a revision is current, so a dated specification URL alone does not freeze every document; the inspected schema commit is recorded separately. Earlier handshake-based revisions remain historically important, but their session and initialization

assumptions must not be presented as the current core protocol. A2A's task lifecycle is anchored to the v1.0.1 repository snapshot [164]; ancillary documentation using a moving "latest" URL remains a reproducibility limitation unless archived or pinned independently.

The application must map protocol identities to its own semantic identities. A JSON-RPC request identifier correlates a request and response. It is not automatically a durable task identifier, a user session, or an idempotency key for an external effect. Similarly, a remote agent's declared name is not proof of its authority. Confusing these roles can produce duplicate work, cross-session state mixing, or misplaced trust while every message still validates against a schema.

For technical comparison, define interoperability at several levels: transport connectivity, message-schema compatibility, feature negotiation, semantic agreement about state transitions, authority propagation, and reliable recovery. Passing a connectivity test establishes only the first levels. A useful conformance report should identify which levels were exercised and which remain assumptions. The analysis below separates the current normative message facts from the systems deductions that follow when those messages carry long-running agent work.

Figure 19.1 distinguishes request correlation, task lifecycle, and conversational context; none of these identifiers establishes current dispatch authority.

19.2 Modern MCP declares versions on every request

MCP 2026-07-28 uses per-request version and capability metadata, without a negotiation handshake. The server accepts or rejects each request independently. Servers implement `server/discover`; clients may call it to obtain supported versions, capabilities, and identity, but may also send another request directly and handle an unsupported-version error [167, 168]. This is the current behavior, not a renamed form of the legacy initialization sequence.

The base protocol requires the protocol-version and client-capabilities fields in request `_meta`; client identity is recommended but not universally required [169]. The tool page's prefatory example note groups identity with required metadata, whereas the base requirements and `RequestMetaObject` make it optional. Requiredness here follows those explicit base and schema definitions [159, 163]. A discovery request can be described as a typed record without conflating its wire encoding with its semantic identities.

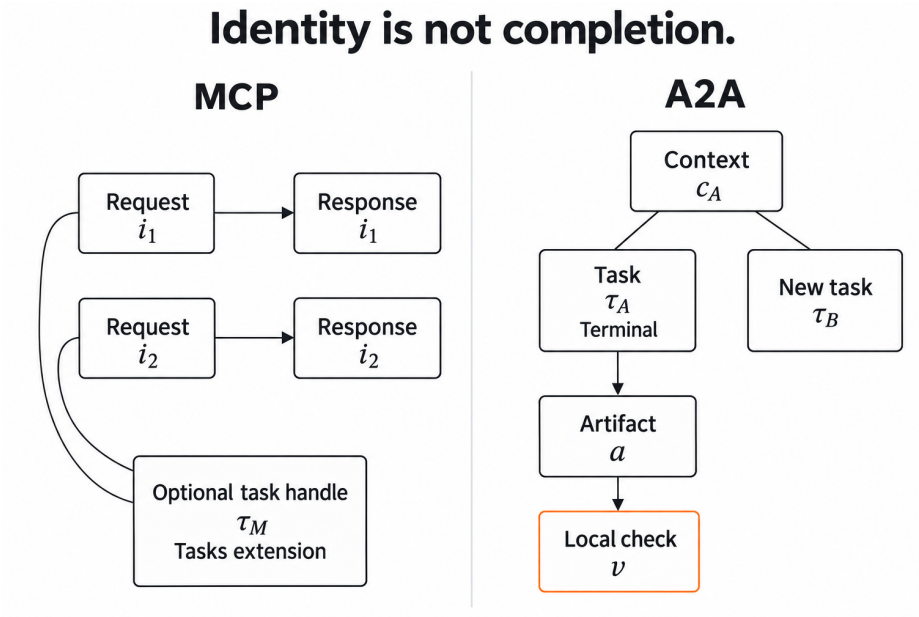

Figure 19.1: Protocol identifiers bind different objects. MCP RPC identifiers correlate individual exchanges; an optional Tasks-extension handle can relate requests to durable work. A2A context identifiers group interactions, while task identifiers bind units of work. A terminal task is not restarted; a later follow-up can create a new task in the same context. Returned artifacts still need local validation. No automatic cross-protocol identity or authority mapping is implied.

Let $\mathcal{Q}, \mathcal{I}, \mathcal{M}$, and $\mathcal{P}$ be the admissible RPC versions, correlation identifiers, methods, and parameter records. A constructed request is

$$\begin{aligned} r_d &= (\rho_2, i_d, m_d, p_d) \in \mathcal{Q} \times \mathcal{I} \times \mathcal{M} \times \mathcal{P}, \\ p_d &= \langle \mu_d \rangle, \qquad \mu_d = (v_\star, C_d), \qquad C_d = \varnothing. \end{aligned} \tag{19.1}$$

The first four coordinates encode the `jsonrpc`, `id`, `method`, and `params` fields. The constant ρ_2 denotes JSON-RPC 2.0, i_d an arbitrary request-correlation identifier, and m_d the `server/discover` method. Inside the parameter record, μ_d encodes `_meta`; $v_\star$ is the 2026-07-28 revision carried by `io.modelcontextprotocol/protocolVersion`, and C_d is the capability-declaration map carried by `io.modelcontextprotocol/clientCapabilities`. The empty map declares no client capabilities. These coordinates do not identify a durable task, conversational session, or external-effect key.

Equation 19.1 is a constructed schema-level record, not a trace from a tested server or a complete JSON serialization. On Streamable HTTP, the declared version is also represented in the protocol-version header under the transport's rules. A mismatch must be treated according to that binding rather than silently guessing which version the peer intended.

The same revision makes result interpretation explicit through `resultType`. Core values distinguish `complete` from `input_required`; a value unrecognized by both the core and agreed extensions is invalid, while an absent field from an earlier server is treated as complete for compatibility [169]. An input-required result is neither a completed tool effect nor a generic transport error. It can carry input requests, opaque continuation state, or both; continuation state alone need not mean that a user form is missing. A resubmission follows that continuation contract, uses a new RPC correlation identifier, and preserves the application operation identity where applicable. The continuation token is not itself a grant or proof of effect completion. This spelling and message role also differ from A2A's interrupted task state: translating between the protocols requires a semantic mapping, not replacement of one status string by another.

An unsupported-version response lists supported versions, allowing the client to retry with a mutually supported choice or surface incompatibility [167]. The retry should preserve the application operation's identity while respecting that a protocol retry and an effect retry are different concepts. A rejected version normally precedes application execution, but clients should not generalize that ordering to arbitrary transport failures or tool timeouts.

Self-contained requests reduce reliance on connection-local negotiation state. They do not mean that the server cannot maintain application data. Persistent state must be referenced explicitly rather than inferred from a shared socket or process. This supports multiplexing unrelated tasks while making the application responsible for its own session and authorization identifiers. Reusing a connection does not create a shared conversational memory contract.

Legacy revisions through 2025-11-25 establish a session through initialization. A dual-era implementation can support both modern and legacy behavior, using the transport-specific fallback rules [167]. A recognized modern error should not be mistaken for evidence that the peer is legacy. An indiscriminate fallback can hide a real version or capability error and accidentally select weaker semantics. Compatibility testing should therefore exercise explicit modern rejection, genuine legacy behavior, and a peer supporting both eras.

19.3 Tools resources prompts and skills remain distinct

MCP exposes several kinds of objects whose roles should remain separate. Tools are invocable capabilities with schemas; resources expose addressable context; prompts provide reusable interaction templates [162]. A host decides how those objects enter the model's working view. The protocol does not specify a universal context-selection, memory-consolidation, or reasoning policy. A shared transport therefore does not make every object an instruction or every server a memory system.

The current tool interface separates listing from invocation through `tools/list` and `tools/call`. Available tools can depend on request authorization, and descriptions and schemas guide the caller [159]. The host still needs grounding, argument validation, and an authority check before effects. A tool annotation is not a substitute for an enforced permission boundary. Chapter 18 analyzes these application-level contracts.

Resources have URI identities and can be listed, read, or described through templates [170]. A resource URI is a reference in the server's namespace, not necessarily a local filesystem path accessible to the client. The host should preserve the server and version context when storing or citing retrieved content. If a resource later changes, a previous citation requires a content snapshot or version reference to remain reproducible. A stable URI alone does not imply immutable evidence.

A skill is procedural knowledge packaged for selective use, as developed in Chapter 15. It may be delivered through a protocol extension or represented using resources, prompts, and tools, but those representations do not collapse its meaning into the underlying primitive. The Agent Skills specification concerns the skill artifact; MCP concerns exchange and invocation; the model policy decides applicability. A skill can instruct the use of a tool without having authority to grant access to it [145].

This separation has a concrete security consequence. A retrieved resource can contain adversarial text that asks the agent to change its goal. The host should label it as untrusted evidence rather than promote it to a privileged instruction. A prompt template can be selected by an authorized user while still containing external fields that require careful interpolation. A tool result can contain both structured data and explanatory prose with different trust properties. These distinctions must survive serialization and compaction.

The proposed implementation stores a typed provenance envelope for every imported object: server identity, object kind, object identifier, schema

or content version, access scope, and insertion role. A context assembler then decides what is visible without losing those labels. This is an application design, not a protocol guarantee. It makes cross-layer errors observable when the same bytes are later retrieved as memory or passed to another agent.

19.4 Transport statelessness and explicit durable work

The 2026-07-28 Streamable HTTP binding changed earlier behavior by removing the GET stream endpoint and protocol-level sessions [171]. A description of the old transport that assumes those features remain current is technically wrong. Streaming and long-lived work must be interpreted through the current binding and per-request declared features rather than inherited from a historical tutorial. Transport connections remain implementation resources, not semantic user sessions.

The base protocol's statelessness means that requests carry the information required to interpret them, with longer-lived application state referenced explicitly [169]. A server can still operate on a database, a file collection, or an ongoing job. The critical rule is that the server should not infer user or task continuity from connection reuse. A reconnect should not require the client to recreate an implicit conversational state hidden inside the old transport.

The Tasks extension supports long-running operations through explicit durable handles and subsequent status or result access [172]. Extension support is not assumed for every core endpoint. The returned task includes a retention lifetime and polling interval; a durable handle is not a promise of indefinite result availability. Cancellation is cooperative and can still end in a different terminal state [172]. The client must distinguish a synchronous complete result from a durable pending operation and preserve the handle in its task state. Dropping the handle during compaction can make the agent lose track of work that continues remotely. Creating a new call in response can duplicate the operation rather than resume it.

A proposed application state machine records pending, awaiting input, completed, failed, canceled, and outcome-unknown states as appropriate to the negotiated feature. The exact protocol states should follow the cited extension; the application can refine them to track external effects. Cancellation of a request, cancellation of remote work, and compensation of an already committed effect are distinct operations. A successful cancel response should be interpreted within the remote service's stated semantics.

Progress notifications are useful for liveness monitoring but do not prove successful task completion. A stream can stop because the network failed while the remote work continues. A host should retain explicit operation identifiers and use a status or reconciliation path when available. The same reasoning applies to retries after a timeout: a self-contained request improves routing and interpretation, but does not by itself make the requested effect idempotent.

The engineering benefit of the modern boundary is that session meaning becomes explicit. The cost is that applications cannot rely on an old initialization handshake to carry hidden task continuity. Durable agent state must record handles, versions, authority, and receipts independently of the socket. This aligns the protocol layer with the session contracts in Chapter 13 and makes failure recovery a testable application responsibility.

19.5 Authorization is separate from declared identity

MCP's HTTP authorization specification distinguishes client, protected resource server, resource owner, and authorization server roles. Authorization is optional at the protocol-implementation level; HTTP implementations that support it SHOULD conform to the cited framework, while stdio implementations SHOULD instead obtain credentials from their environment [173]. A server being reachable does not imply that every capability is authorized, and a client being authenticated does not imply that every requested effect is within the user's task scope.

The current specification uses protected-resource and authorization-server discovery mechanisms and defined registration options [173]. These are protocol and credential contracts, not a complete application policy. An access token can permit a broad API scope while the current user request authorizes only a narrow task. A local monitor may need to enforce the narrower boundary even when the remote service accepts the credential.

The protocol's client and server information fields are self-reported metadata rather than cryptographic proof of identity [168]. Applications should not treat a familiar display name or version string as authorization evidence. Trust should derive from the configured endpoint, transport authentication, credential validation, and applicable policy. This is particularly important for discovery catalogs that combine many servers with similar tool names.

Delegation adds another question: whose authority is being exercised? A host can authenticate to a tool server using its own credential while acting on behalf of a user. A remote agent can request a capability that the

original user never intended to delegate. The application should preserve the principal and scope through each hop, avoiding silent expansion when a worker or skill is loaded. Protocol compatibility makes the message exchange possible; it does not decide the appropriate delegation policy.

Credential exposure should not be treated as ordinary memory. The model generally needs a capability reference and allowed scope, not a plaintext token in its context. If a token or secret enters a tool result, subsequent summaries and indexes can spread it beyond its intended lifetime. A secure wrapper can keep credentials outside the model-visible evidence packet while recording which credential scope admitted an effect. The exact storage and redaction mechanism remains application-specific.

A proposed authorization test matrix includes correct credentials with excessive task scope, stale authorization after a user correction, catalog changes under different principals, and a reconnect that interleaves unrelated users. Measure both unauthorized acceptance and false rejection of legitimate work. A system that blocks every call is not a useful agent, while one that completes every task with broad credentials has not demonstrated least-privilege operation.

19.6 A2A tasks artifacts and delegated autonomy

A2A allows a remote agent to respond with a self-contained message or initiate stateful work represented as a task. Its lifecycle distinguishes interrupted states, such as `input-required` or `auth-required`, from terminal outcomes. A context identifier can group multiple interactions, while a task identifier refers to a particular unit of work [164]. These identifiers should not be mapped automatically to the model's active context window or to one transport connection.

Terminal tasks do not restart. A refinement creates new work within the related context and can reference the earlier task [164]. Consequently, client artifact acceptance and mutation lineage must survive beyond the producing task's terminal status. Reusing the old task identifier as a mutable revision slot would obscure which input produced which artifact. Continuity belongs to the explicit context and client-managed lineage, while terminal task identity preserves the completed unit of work.

A2A v1.0.1 also distinguishes idempotency by operation. Reads are naturally idempotent, message submission MAY use the message identifier for duplicate detection, and task cancellation is idempotent [165]. Thus a repeated message identifier is not a universal delivery guarantee, and idempotent cancellation does not establish rollback of a previously committed

effect. The client must record which remote deduplication contract was actually promised.

Discovery uses an Agent Card describing offered capabilities, endpoints, input and output modes, and security-related information [174]. The card supports selection of a suitable remote agent; it does not prove the agent's competence. A skill description in the card is an advertised capability, not necessarily a portable executable skill package. The caller still needs task validation, evidence handling, and a policy for trusting returned artifacts.

Delegating a task differs from invoking a narrow tool because the remote agent can choose internal methods and produce intermediate work. The caller may observe only status and artifacts rather than every internal action. This can be useful for modularity and privacy, but it changes verification. A completed task state is a protocol statement from the remote service, not independent proof that the original user's success predicate was met.

Artifacts should preserve provenance, version, and a relation to the task. A document returned by a remote researcher can contain supported claims, unsupported synthesis, or stale citations. An execution artifact can be syntactically valid while violating a local constraint unknown to the remote agent. The delegation request should therefore include the relevant obligations and authority, and the caller should validate the artifact against them before publication or further effects.

Parallel follow-ups create a state-ownership problem. Several tasks can share a context identifier while working on different branches of an objective. A late artifact from one branch should not silently overwrite a newer accepted artifact from another. The application needs version and acceptance rules beyond the protocol's ability to transport updates. Similarly, a request for more information should suspend or qualify dependent work rather than being ignored while the caller assumes progress.

A proposed delegation contract contains goal, completion predicate, budget, permitted effects, evidence requirements, and expected artifact schema. It allows the remote agent flexibility within those constraints. A callback or streaming update can report progress, while final acceptance remains a separate local decision. This distinguishes negotiated autonomy from unrestricted trust and connects protocol design to the orchestration analysis in Chapter 21.

19.7 A concrete cross-protocol execution trace

Artifact transfer needs an explicit trust boundary. An A2A artifact identifier belongs to the producing task's namespace; an MCP resource identifier

belongs to the serving endpoint's namespace. Resolving one into the other is an application mapping, not an equality implied by the identifier strings. A content digest can detect a changed payload but cannot establish the producer's authority or the truth of its claims. Preserve the producing task, source version, content type, and verification status when constructing the resource representation. The consumer should distinguish an agent's inference from evidence that the remote agent actually observed. Otherwise a technically correct transfer can launder a speculative conclusion into an apparently authoritative tool result.

An interrupted task also needs a scope-preserving continuation rule. Suppose a remote agent requests authorization for one operation and later resumes with changed arguments. Reusing the earlier approval solely because the task identifier is unchanged can authorize a different effect. A proposed application contract binds a grant to the relevant argument digest, resource scope, authority epoch, and expiry; a changed material precondition requires revalidation. This is an authority requirement derived from the execution model, not a claim that every A2A implementation enforces that binding. Protocol progress and user authorization are separate state machines, even when one user interface displays them together.

Consider a constructed workflow in which a coordinator delegates a research task to a remote agent and later uses a tool server to retrieve a source record. The A2A request identifies the objective, current constraints, and expected evidence artifact. The remote service returns a task handle or a direct message according to its supported interaction. The coordinator records the handle in durable task state rather than relying on an open stream to remember it.

When evidence requires a tool server, the coordinator's MCP client sends a request carrying the current protocol version and client capabilities. It may first use discovery, then list the authorized tools and obtain the selected schema. Before invoking the tool, the host validates arguments and task authority. The server's response becomes a typed observation with server and resource identity. It does not become a new user instruction merely because it was returned through a trusted transport.

Suppose the research task returns a completed artifact while one cited resource has changed. The coordinator should validate the cited version or preserve the captured passage, then decide whether the artifact satisfies the requested evidence standard. If the artifact proposes publication but the delegation allowed only preparation, local authority remains binding. A2A task completion and MCP tool success do not jointly imply permission to publish.

Now interrupt the network after a tool operation may have committed but before the coordinator receives the result. Reconnecting with the correct protocol metadata does not resolve the effect. The application needs an explicit operation identifier, remote status query, or reconciliation procedure. If no such mechanism exists, it should retain an outcome-unknown state. Reissuing the same semantic request with a new JSON-RPC identifier can duplicate work unless the tool supplies an independent idempotency contract.

This trace is original synthesis, not a tested integration or a claim that every SDK implements these steps. It identifies the records a correct implementation would need: delegation identifier, task/context relation, protocol versions, catalog/schema digest, authorization version, effect identifier, response receipt, and artifact acceptance decision. These records support causal diagnosis if the final output is wrong.

The most informative interoperability experiment would replay this workflow with independent client and server implementations, controlled delays, credential changes, and a mixed modern/legacy MCP environment. A successful result would support the tested semantic contract, not merely that two programs exchanged JSON. A failed result should identify whether the mismatch concerned version negotiation, capability declaration, effect semantics, or the meaning of task completion.

The normative implementation anchor is the versioned TypeScript schema. At the inspected July 28 commit, `RequestMetaObject` requires the protocol-version and client-capabilities members while declaring client information optional. `DiscoverRequest` fixes the discovery method, and `CallToolRequest` binds the tool-call method to its parameter structure [163]. These definitions are stronger implementation references than copying an old SDK example and assuming it speaks the current protocol. They describe message shape; they are not an executed conformance test of any client library.

A concrete negative test removes only the capabilities member from the illustrative request while preserving valid JSON and a recognized version. A second test keeps the capabilities member but changes the HTTP version header so it disagrees with the body. A third requests an unsupported protocol version while keeping body and header equal. These cases should not collapse into one generic retry: malformed metadata, inconsistent transport declaration, and unsupported version are different failures. Their distinction matters because only some errors justify selecting a different protocol version.

An application can generate clients from schema definitions and still

mishandle those outcomes. It might cache a successful discovery response under one credential and reuse the catalog under another. It might regard an optional identity string as a trusted principal. It might strip required metadata when forwarding a request to a worker. These are integration failures above the type definition. Test the complete forwarding path with different principals and concurrent tasks rather than validating only an isolated client serializer.

For long-running operations, retain two kinds of correlation. The protocol request identifier tracks a particular exchange, while an explicit application or extension handle tracks durable work. After a reconnect, a new exchange can refer to the same durable task. If the implementation uses request identity as the only operation identity, it can either duplicate work on retry or incorrectly suppress a legitimate new request. A reference fixture should model both identities and assert which one is stable across network retries.

The same distinction supports audit of delegated research. The coordinator can record that a remote task completed and that an artifact was received, while separately recording whether the artifact passed evidence validation. This avoids turning transport success into scientific acceptance. It also allows the coordinator to reject one artifact without falsely marking the remote operation as never completed. Protocol state, task state, and epistemic status remain separate even when the user interface displays one compact progress indicator.

19.8 Conformance performance and residual uncertainty

Protocol evaluation should separate schema conformance from application correctness. Conformance tests can validate required metadata, unsupported versions, capability errors, pagination, and response correlation. They can inspect whether the current modern request model is implemented rather than a historical initialization flow. They cannot establish that an agent's chosen action is useful or that a returned research artifact is scientifically sound.

Use a version matrix with modern-only, legacy-only, and dual-era MCP peers. Include missing required metadata, declared but unsupported capabilities, an unknown version, and a recognized modern rejection. Test whether fallback follows the binding's rules instead of treating every error as a reason to initialize a legacy session. Preserve exact specification and SDK revisions in the result. A package release number alone is insufficient when examples or generated schemas change independently.

For A2A, test direct messages, stateful tasks, interrupted work, terminal outcomes, and artifact updates. Include delayed and duplicated updates, failed streams, and concurrent follow-ups. Check that the application preserves task ownership and does not interpret a progress event as completion. Independently verify a sample of final artifacts against the requested predicate. Protocol-level terminal status and task-level validity should remain separate metrics.

Performance accounting includes discovery, schema transfer, serialization, credential flows, network latency, polling or streaming overhead, model decisions, and remote work. Compare warm and cold operation, and identify which metadata is cacheable under which principal. A cached catalog can reduce overhead while becoming stale after a permission or tool change. Latency gains should therefore be reported alongside freshness and correctness behavior.

The official MCP architecture documentation identifies an additional current-version change: sampling and some prior utilities are deprecated in the 2026-07-28 era, and change notifications are delivered through opt-in subscriptions [175]. Implementations and surveys should not infer present support from an older tutorial. Deprecation also differs from immediate removal; the exact feature lifecycle must be checked before asserting that a capability is unavailable.

No runtime conformance or cross-protocol benchmark has been executed for this chapter. The normative facts come from official specifications inspected at the cutoff; the execution trace, failure analysis, and test design are explicitly deductions and proposals. The main technical conclusion is bounded: interoperable messages make component composition possible, while durable state, authority, evidence quality, and verified outcomes remain contracts of the complete agent system. Those contracts determine whether interoperability improves reliable autonomy.

Chapter 20

Agent Harnesses: Control Flow, Isolation, and Durable Effects

Harness^2 and RRSI make a current experimental boundary explicit: a frozen backbone can be surrounded by a changing execution procedure, and the resulting gains belong to the configured system rather than automatically to the checkpoint [28, 27]. Harness^2 adds inference and model-based selection whose cost and evaluator dependence matter; RRSI limits repeated selection against a finite evolve set. These 2026 cases motivate the central question here: which state transitions changed, what evidence selected them, and can the claimed benefit survive independent tasks and runtime failures?

A model supplies a conditional distribution over candidate actions. A harness decides which observations enter that conditional distribution, which candidates become executable operations, how their effects are recorded, and what happens when execution stops unexpectedly. Consequently, harness behavior changes the effective policy even when model parameters remain fixed. An evaluation that reports only a model identifier omits part of the system being evaluated. Conversely, an improved harness does not demonstrate that the underlying model learned a new capability.

The engineering objective is controlled progress under incomplete observations and partial failures. Reliability requires separating three claims: the agent selected a sensible action, the runtime executed the intended operation, and the operation produced a state satisfying the task. A successful transport response proves none of the other two. This chapter derives an execution contract, examines its failure windows, and connects implementation choices to experiments. The discussion extends the state-consistency analysis of Chapter 17 and the interface constraints of Chapter 18.

20.1 The effective policy includes the harness

Let H_t denote the recorded interaction history, C_t the constructed model context, M_t persistent memory, and E_t environment state. A harness configuration h includes a context constructor g_h, action parser p_h, scheduler, authorization checks, and stopping policy. Let S_t be the recorded execution

state, defined in the next section. Let $\mathcal{Z}$ be the countable set of stopped output records and $\mathcal{A}$ the dispatch-action space augmented with a no-effect rejection outcome. Assume termination for this single-call model; otherwise nontermination needs its own mass as in Chapter 2. For $z_t \in \mathcal{Z}$ and $a_t \in \mathcal{A}$, the induced distribution is

$$\overline{\Pi}_{\theta,\delta,h}(a_t \mid S_t) = \sum_{z \in \mathcal{Z}} Q_{\theta,\delta}(z \mid C_t) K_h(a_t \mid p_h(z), S_t; \alpha_t). \tag{20.1}$$

Here $Q_{\theta,\delta}$ is the normalized stopped law of the actual decoder, $C_t = g_h(S_t)$ uses only the declared policy-visible projection, and K_h is the parser/enforcement kernel under current authority epoch α_t. Its output is a dispatched action or a no-effect outcome; dispatch is not remote commitment. It can reject malformed output or substitute a deterministic fallback. A request for model repair is instead a transition to another inference call, not a free operation inside this single-call expression. Its likelihood, latency and cost belong to the subsequent trajectory. This distinction prevents a repair-enabled harness from being analyzed as though it executed only the final valid response. A parser that silently normalizes an ambiguous filename makes a policy decision; it is not merely formatting infrastructure.

This decomposition explains why two products using the same checkpoint can differ substantially. They may expose different tools, truncate different observations, retry at different boundaries, or terminate after different evidence. The code-as-harness survey organizes these mechanisms through interfaces, execution feedback, memory, and coordination [176]. Its taxonomy is useful architectural synthesis, but does not establish that every additional mechanism improves performance. Additional machinery can increase failure opportunities and conceal the origin of a gain.

A clean causal comparison specifies the treatment as a configuration change. Suppose the treatment adds a planning file and a summarization tool. Its measured effect combines extra inference, altered context, persistent state, and a new action interface. Attributing the result to "planning" requires controls that preserve the other differences. One control allocates the same additional model calls to independent attempts; another retains the file interface but disables learned updates; a third supplies an oracle summary to estimate headroom. None is sufficient alone because the treatment can interact with task length and tool latency.

Harness adaptation creates a second policy. RRSI searches over harness changes while holding backbone weights fixed and constrains proposal and selection behavior [27]. The important lesson is not a universal numerical gain, but that repeated selection against a finite evolve set can optimize

quirks rather than transferable procedures. A runtime should therefore record the complete harness identity alongside the model and memory version. Reconstructing a trajectory with a later parser or prompt can change its interpretation even when every raw message is preserved.

For implementation, define the smallest state transition with an externally meaningful outcome. Avoid making an entire conversational turn the only transaction boundary: it can contain multiple tool effects and multiple opportunities for interruption. Also avoid recording every generated token as a committed action. Token streams are provisional inference products; an executable operation begins only after parsing, authorization, and allocation of a stable identity.

20.2 Execution state and semantic commitments

An execution record needs more than messages. A useful state is

$$S_t = (H_t, C_t, M_t, J_t, A_t, V_t, B_t), \tag{20.2}$$

where J_t is an operation journal, A_t authorization state, V_t the compatibility tuple, and B_t the remaining budget. The environment is observed through versioned evidence rather than assumed to be fully contained in S_t. An email service, remote repository, or browser session can change independently. Storing an observation is not locking the corresponding external object.

The journal distinguishes intent, dispatch, observation, and reconciliation. An intent says what the application authorized. Dispatch records that a request crossed a boundary. Observation records the response or the absence of one. Reconciliation establishes what external state now exists. These states must remain distinct because a timeout after dispatch is observational uncertainty, not proof of failure. Retrying an uncertain operation without examining its effect can duplicate it.

Three invariants make this distinction operational. First, every effectful request refers to an authorized intent whose argument hash and policy epoch match the request. Second, every committed result identifies the exact intent and the evidence used to accept it. Third, resumption does not infer completion from an optimistic natural-language summary. It consults the journal and, when necessary, the authoritative service. A summary may help the model reason about the result, but it cannot replace the result record.

Typed states also prevent a subtle category error: task completion and runtime completion are different predicates. Let $T(S)$ mean the user task is satisfied, $Q(S)$ mean required checks passed, and $R(S)$ mean no operation

remains unresolved. A defensible terminal condition is $T(S) \wedge Q(S) \wedge R(S)$ under the declared specification. A model-generated "done" token establishes only that the inference process proposed termination. A quality gate may still misclassify the output, so even this terminal condition is relative to the validators' coverage.

Application frameworks expose parts of these distinctions. The inspected LangGraph persistence documentation separates thread checkpoints from cross-thread stores [127]. That distinction should not be collapsed into a claim that a checkpoint already implements semantic memory. Checkpoints support restoration of declared graph state. Cross-thread stores retain application records. Neither feature independently establishes truth, relevance, authorization, or successful external-effect recovery.

The practical design choice is to make semantic commitments inspectable. A tool receipt should include object identifiers, version or timestamp, effective principal, argument digest, and verification status. Where a remote service exposes no stable receipt, record that limitation explicitly. It may justify requiring review before a consequential retry. The runtime must not manufacture transactional guarantees that the connected service does not provide.

20.3 Crash consistency and uncertain effects

Consider a request to create an external issue. A crash can occur before intent persistence, after persistence but before dispatch, after remote creation but before response receipt, or after receipt but before local commitment. These windows require different recovery actions. Before dispatch, replay can safely issue the request. After uncertain dispatch, recovery must first determine whether the remote creation occurred. After a committed receipt, replay should return the recorded outcome rather than create a second object.

An idempotency key helps only if the remote service gives it the required semantics. Reusing an application-generated string does not by itself make an API idempotent. The service must scope the key appropriately, retain its association long enough, reject incompatible reuse, and return a stable result. A short retention period can make an old replay unsafe. Idempotency may also apply only to one endpoint, leaving downstream notifications or billing outside the guarantee.

An execution protocol can be specified as a branch-dependent state transition. Let i be the intent, J the journal in S, α current authority, and ν the recorded release version. Define $v(i) \in \{0, 1\}$ by intent-schema

validation. Let $a_J(i, J, \alpha)$ govern access to the corresponding journal records and $a_D(i, \alpha)$ govern new external dispatch. Read access to a historical receipt is distinct from permission to repeat its effect; either denied check produces its own non-effect outcome. For a valid intent, $k = \eta(i)$ is its stable logical-operation identifier. The lookup $\sigma_J(k) \in \{0, 1, 2, 3\}$ distinguishes absence, persisted intent, marked dispatch without a terminal verdict, and committed terminal verdict. Define $\gamma(i, S) = 1$ exactly when $v(i) = 1$, $a_J(i, J, \alpha) = 1$, and either $\sigma_J(k) = 0$ or the recorded logical intent identity and exact payload match i; otherwise $\gamma(i, S) = 0$. This binding and record-access check precedes replay and reconciliation as well as fresh dispatch; revoking write authority need not revoke legitimate access to a past receipt. For a recorded operation, $\rho_J(k)$ returns its recorded outcome, or v when terminal evidence remains unresolved. The top-level transition is

$$\Phi(i, S) = \begin{cases} (J, \bot), & \gamma(i, S) = 0, \\ (J, \rho_J(k)), & \gamma(i, S) = 1,\ \sigma_J(k) = 3, \\ \Phi_R(i, S; k), & \gamma(i, S) = 1,\ \sigma_J(k) = 2, \\ \Phi_0(i, S; k), & \gamma(i, S) = 1,\ \sigma_J(k) \in \{0, 1\}. \end{cases} \tag{20.3}$$

For $a_D(i, \alpha) = 0$, $\Phi_0(i, S; k) = (J, \bot_A)$ without dispatch. Otherwise the following fresh-dispatch equations describe the acknowledged-write branch:

$$\begin{aligned} J_1 &= \mathcal{I}(J; k, i, \nu), \\ J_2 &= \mathcal{D}(J_1; k), \\ o &\sim \mathcal{T}(\cdot \mid i, k), \qquad J_2 \neq \bot_J, \\ J_3 &= \mathcal{O}(J_2; k, o), \qquad g = \mathcal{V}(i, o), \\ J_4 &= \mathcal{C}(J_3; k, o, g), \\ \Phi_0(i, S; k) &= (J_4, \rho_{J_4}(k)). \end{aligned} \tag{20.4}$$

Each journal operator returns an acknowledged journal or $\bot_J$; the next line is evaluated only on an acknowledged branch. Branch selection and the dispatch-marker transition require a serialized dispatcher per key or an equivalent atomic, fenced claim. A read followed by unconditional writes is insufficient under concurrent recovery. A storage failure returns the last acknowledged journal and a storage-error outcome, without inventing a new external status. In particular, intent or dispatch-marker failure prevents a new call, whereas receipt-write failure after a call can require reconciliation. Here $\mathcal{I}$ persists the versioned intent, $\mathcal{D}$ durably marks dispatch, $\mathcal{T}$ invokes the external tool under key k, $\mathcal{O}$ persists the returned observation, and $\mathcal{V}$ evaluates the effect evidence. The operator $\mathcal{C}$ persists its verdict $g \in \{-1, 0, 1\}$, denoting negative, unresolved, and positive. Only a

terminal verdict justified by adequate evidence yields $\sigma = 3$; $g = 0$ preserves $\sigma = 2$ and $\rho = \upsilon$. An unresolved first-call result therefore remains explicitly unresolved rather than acquiring certainty from the name of a commit routine. Reuse of an existing intent requires equality of its payload and declared identity; a key collision cannot authorize substitution.

For a previously marked dispatch, let $\widehat{o} = \mathcal{R}(i, J; k)$ denote evidence obtained by an independently authorized status/reconciliation operation, with υ denoting an unresolved observation. If no permitted reconciliation path exists, $\mathcal{R}$ returns υ without issuing a query. Define

$$\begin{aligned} \widehat{J} &= \mathcal{C}\big(J; k, \widehat{o}, \mathcal{V}(i, \widehat{o})\big), \\ \Phi_R(i, S; k) &= \begin{cases} (J, \upsilon), & \widehat{o} = \upsilon, \\ (J, \bot_J), & \widehat{o} \neq \upsilon,\ \widehat{J} = \bot_J, \\ (\widehat{J}, \rho_{\widehat{J}}(k)), & \widehat{o} \neq \upsilon,\ \widehat{J} \neq \bot_J. \end{cases} \end{aligned} \tag{20.5}$$

The definition of $\widehat{J}$ is evaluated only when $\widehat{o} \neq \upsilon$; a failed reconciliation write returns the last acknowledged journal and a storage error rather than applying a lookup to $\bot_J$. Neither the unknown branch nor replay of a terminal record invokes $\mathcal{T}$. The journal-access policy is checked before branch selection; a reconciliation query needs its own applicable permission. New dispatch always checks current effect authority under the remote service's concurrency contract. Neither reading a receipt nor reconciling status grants permission to repeat the original write. These relations specify required ordering and evidence semantics; they do not prove that a particular runtime or remote service implements them. Persisting a dispatch marker before calling introduces a conservative ambiguity: the marker may exist even if the request never left the process. Marking it after the call introduces the opposite ambiguity: an effect can exist without a marker. No local ordering removes both without cooperation from the remote system. This is the fundamental reason a journal cannot alone provide exactly-once external effects.

Compensation is a separate operation. Closing an erroneously created issue does not erase notifications already sent. Reverting a file does not retract information copied by another process. A compensating operation can restore an application invariant while leaving residual effects. Therefore recovery reports should distinguish restored local state, reconciled external state, and irrecoverable consequences. This classification supports meaningful incident analysis and prevents rollback metrics from becoming cosmetic.

Fault-injection experiments should target these exact windows. Kill the worker after remote acceptance but before local receipt persistence;

interrupt between verification and commit; corrupt one journal entry; delay a response beyond the retry threshold. Compare stateless retry, checkpoint replay, and journal-based reconciliation under identical fixtures. Measure duplicate effects, unresolved operations, recovery latency, and task completion. These are proposed experiments; no failure rate is inferred from the protocol's appearance.

20.4 Context construction is runtime scheduling

Context management determines which parts of execution state the policy can use. A journal may be complete while C_t omits the operation that explains a pending effect. Conversely, a summary may assert completion that the journal never committed. The harness therefore needs a projection contract between authoritative state and model-visible state. For a projection claiming effect-aware planning, the relevant unresolved operations and active constraints need a usable representation. Exact authority and sensitive effect evidence can remain privileged, with a policy-safe status projection; copying every grant or credential into the prompt is unnecessary.

A projection can be tested against invariants without requiring an exact semantic equivalence oracle. For example, inject a pending-operation record and verify that the constructed context includes its stable identifier and uncertainty state. Insert a revoked capability and check that the tool registry removes it regardless of whether the model remembers an earlier grant. These checks establish structural properties. Whether a compressed explanation preserves every decision-relevant fact remains a behavioral question and requires task-level evaluation.

Agentic Context Management supplies model-directed offloading and retrieval mechanisms [119]. Its live-run raw mirror can preserve access to discarded content, but archival retention does not guarantee that the agent retrieves the right item at the right time. Its reported benefits also depend on a capable backbone and reimplemented baselines. This distinction matters for runtime design: storing raw history is a recovery affordance; selecting useful context is an additional policy with cost, latency, and failure modes. The snapshot-only restore path discussed in Chapter 13 does not preserve a previously separate raw archive by itself [125].

An execution scheduler can reserve a protected context region for authoritative status and separately allocate a discretionary region for retrieved evidence. Protection should mean application-controlled serialization, not merely an instruction asking the model to respect a textual heading. The trusted region should be generated from current state on every call. If

a retrieved document can overwrite it through string concatenation or malformed role boundaries, the separation is nominal rather than enforced.

The resource constraint couples context and scheduling. Long tool outputs increase prefill cost and may delay other sessions. An aggressive truncation policy improves throughput but can remove the evidence needed for reconciliation. A token budget should therefore distinguish required status tokens, candidate evidence tokens, and speculative reasoning allocations. When the budget cannot accommodate required state, the runtime should reduce work, retrieve narrower evidence, or request a bounded continuation. Silent removal of constraints is not a legitimate optimization.

Evaluate these choices with task-length and output-size strata. Average success can hide a regime where compaction helps short tasks but fails after the first interruption. Include a baseline that keeps authoritative status fixed while varying only discretionary history. This separates failures of the projection contract from failures of the model to interpret otherwise available evidence. Report the cost of reconstructing omitted evidence, not only the reduction in visible prompt length.

20.5 Budgets cancellation and backpressure

A runtime executes under several budgets that are not interchangeable. Tokens constrain inference volume; wall-clock deadlines constrain responsiveness; tool quotas constrain access; monetary limits combine model and service costs; risk budgets constrain permitted effects. Converting all of them into one scalar can obscure hard requirements. A cheap operation may still be unauthorized, and a low-token plan may wait indefinitely on a remote tool.

Represent remaining resources by a vector B_t and each candidate action by a distribution of costs. Admission requires hard constraints to hold before expected utility is considered. For a deadline $D \geq 0$, current elapsed time $t \in [0, D]$, nonnegative uncertain duration $L(a)$, and tolerated miss probability $\epsilon \in (0, 1)$, a chance constraint $\Pr(L(a) \leq D - t) \geq 1 - \epsilon$ is meaningful only when the duration model is calibrated for the workload. A mean-latency estimate does not establish a tail guarantee. Timeouts truncate observations and can bias the estimated distribution if treated as ordinary successful durations.

A hard budget also needs reservations under concurrency. For a scalar budget B_0, let s_t be settled spend, P_t outstanding operations and r_i the

reserved upper cost of operation i. Require

$$s_t + \sum_{i \in P_t} r_i \leq B_0. \tag{20.6}$$

Admission checks the invariant and reserves r_i before dispatch in one linearizable budget-ledger transition. Settlement must use the same ledger and atomically add c_i to settled spend while removing r_i from outstanding reservations, with $0 \leq c_i \leq r_i$. It is idempotent under the logical-operation key: a repeated settlement returns the recorded result without charging or releasing again. Admission, settlement and release share one serialization order. Releasing a reservation before recording its charge would let another admission consume the apparent balance and invalidate the bound. Under these assumptions, each ledger transition preserves the invariant by induction. Vector budgets use componentwise inequalities. This guarantee requires valid cost bounds and linearizable, crash-consistent accounting; an expected-cost estimate or atomic reservation alone is insufficient. Cancellation does not release a reservation until the remaining charge or effect is bounded. An unbounded remote billing path cannot be turned into a hard cap by local accounting alone.

Cancellation propagates through an ownership tree. A parent may cancel its own inference yet leave a remote command, browser navigation, or delegated worker running. The runtime needs a record of outstanding operations, cancellation support for each tool, and a reconciliation strategy for tools that cannot stop. Returning control to the user does not prove that background effects have ceased. This distinction is particularly important when a later instruction changes authorization while an earlier operation remains in flight.

Backpressure prevents an overloaded system from amplifying its own failures. If a tool slows down, retries can increase offered load and make recovery less likely. A retry policy should distinguish transport failure, rate limiting, invalid arguments, and uncertain completion. Backoff addresses congestion; it does not repair a malformed request. A circuit breaker prevents repeated calls to a failing dependency but must preserve evidence about abandoned work. Otherwise the agent may reinterpret a dependency outage as evidence that a fact or object does not exist.

The scheduling objective must also include fairness. A single long-running agent can occupy scarce browser sessions or large KV allocations while shorter requests wait. Preemption can improve fairness but increases checkpoint and reconstruction cost. The appropriate policy depends on whether progress is checkpointable and whether external operations can be safely suspended. A runtime should expose these workload properties

rather than apply one universal quantum to every tool.

Benchmark admission and cancellation under open-loop arrival processes, not only fixed concurrency. Fixed concurrency reduces offered load when latency rises and can conceal instability. Report queueing delay separately from execution time, and include rejected or cancelled work in the relevant denominator. Tail latency, budget overshoot, orphaned operations, and completed useful work collectively describe whether the scheduler preserves service objectives under stress.

Figure 20.1 isolates the mechanism and its stated information boundaries.

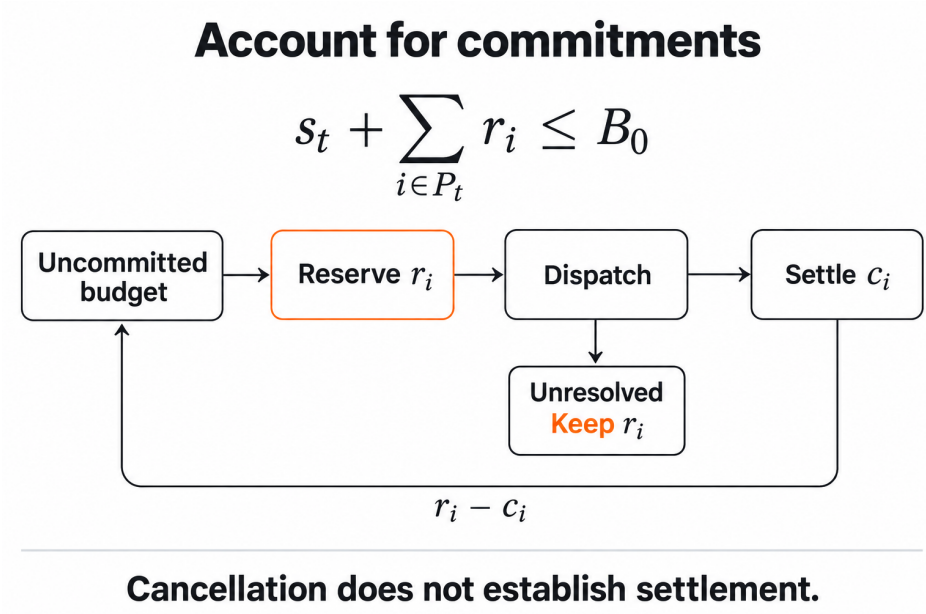

Figure 20.1: A hard-budget invariant accounts for settled spend and bounded outstanding commitments. Admission reserves an upper cost before dispatch. Keyed idempotent settlement atomically records the actual charge and releases only the unused reservation in the same linearizable ledger used for admission. Unresolved operations retain commitments until remaining charges are bounded; cancellation alone does not release them.

20.6 Implementation boundaries and evidence

An inspectable harness exposes action parsing, context construction, tool dispatch, state persistence, stopping, and error recovery as separable mod-

ules. Code location alone is insufficient: a module can delegate essential behavior to a proprietary endpoint. The implementation record should identify the local revision, external service version where available, configuration, and which semantics remain opaque. This is the minimum needed to connect an algorithm to an identifiable execution path.

The OpenAI Agents SDK documentation describes an application-owned deployment path, whereas a managed agent service supplies a different runtime boundary [126]. A thin SDK wrapper does not transfer ownership of application storage, tool semantics, or approvals to the provider. Conversely, a hosted runtime may maintain session state without exposing the underlying scheduler. These are role differences, not evidence that one architecture is universally more reliable. Chapter 22 compares their consequences without importing product priority rankings.

Harness^2 illustrates adaptation of execution and improvement harnesses using behavioral comparisons and experience reuse [28]. Without ground-truth task verifiers, such feedback remains vulnerable to shared model biases and execution variance. More editor sessions can improve the search over changes while also increasing cost and selection opportunities. A runtime supporting this mechanism needs candidate isolation, a record of rejected changes, and a rollback boundary that includes prompts, tools, and memory interpretation.

The current Harness^2 source distinguishes logical stage accounting from all invocation costs. Its `core/ledger.py` checks rollout and improvement-stage identities $R = 4k$ and $P = 3k$ for recursion budget k, rejects duplicate stage labels, and separately records retry attempts. Selection and reflection are outside this ledger's stated counting boundary [177]. These are source-level count identities, not measurements of runtime or total expense. A complete cost comparison must join those stages with selection, reflection and failed invocations. The ledger is an adaptation-budget record, not a transactional journal proving remote-effect recovery.

An implementation audit should begin with one complete request, not an abstract class diagram. Trace the request from input serialization to terminal artifact, identify every mutable dependency, and enumerate where errors are caught or suppressed. A broad exception handler that returns a plausible textual tool result can turn infrastructure failure into misleading model evidence. A successful deserialization test will not expose that defect; an injected dependency outage will.

The strongest verification combines local contract checks with end-to-end behavioral experiments. Contract tests can establish that duplicate operation identifiers are rejected or that schema changes invalidate cached

decisions. They cannot establish that a proposed patch satisfies an underspecified user task. Behavioral tests can estimate task reliability but may miss rare transactional windows. The two forms of evidence answer different questions and should remain separately reported.

The practical implication is a narrow but consequential design principle: authoritative execution state should be machine-checkable, recoverable, and distinct from model narration. The model remains essential for interpreting uncertain tasks and choosing useful actions. The runtime supplies the boundaries within which those choices can be evaluated and acted upon. Reliability emerges from their interaction; neither a fluent agent nor a durable database independently establishes it.

20.7 A worked analysis of a recoverable repository workflow

Consider a repository-maintenance agent asked to update a dependency, run tests, and prepare a reviewable change. Its environment has four independently mutable surfaces: the checkout, dependency registry, test infrastructure, and remote review service. The task is deliberately narrower than autonomous deployment. The relevant success predicate requires the intended version change, passing required tests under the declared environment, and a review artifact referring to the verified revision. A passing local command is not sufficient if the remote artifact contains another commit.

Start by assigning an immutable task identifier and recording the initial repository commit. The package resolution step produces a lockfile whose content depends on registry state as well as the requested version. Therefore the transition record must include the resolved artifact identity, not only the human-readable version string. A registry can republish metadata or a moving tag can resolve differently. Replaying the same command later may produce different bytes even when the agent's prompt is unchanged. Reproducibility requires controlling this dependency or recording that it remains uncontrolled.

Next, the agent edits files in a workspace branch. The harness can checkpoint a filesystem snapshot before execution, but the snapshot does not capture every external dependency. A test can consult network services, local caches, or environment variables. The verification record should bind the resulting verdict to the workspace digest, test selection, dependency manifest, and relevant environment identity. If the model changes a source file after tests finish, the prior verdict becomes stale. The runtime can

enforce this mechanically by invalidating verdicts whose dependency digests no longer match the current artifact.

This binding can be expressed as an acceptance rule. Let x be the proposed artifact, $d(x)$ its digest, v a verifier configuration, and r a recorded test outcome. The gate accepts only if r was produced for $(d(x), v)$ and the required checks passed. The rule establishes correspondence between artifact and evidence; it does not establish completeness of the tests. An independent specification defect remains possible. The separation is useful because it prevents a simple engineering mismatch from being obscured by philosophical uncertainty about correctness.

Now introduce a crash after the remote review service creates the artifact but before the harness stores its identifier. Replaying the create request can produce duplicates. If the service supports an idempotency key, recovery uses the original key. If it does not, the agent can query for an artifact marked with the task identifier and expected commit digest. That query may return zero, one, or several matches. Zero results are not conclusive under eventual consistency. Several results require reconciliation rather than arbitrary selection. A bounded waiting policy and an unresolved status are more accurate than asserting that the operation failed.

The cost of naive retries can be analyzed without inventing a measurement. Conditional on one dispatch, let A be the event that the request takes effect but its acknowledgment is lost, and let R mean recovery retries without reconciliation. For $q = \Pr(A) > 0$ and $r = \Pr(R \mid A)$, the chain rule gives $\Pr(A \cap R) = qr$. No independence assumption is needed with this conditional definition of r. If $q = 0$, the joint event has probability zero without defining r. Under the one-retry model this is a duplicate opportunity. The actual duplicate probability also depends on remote deduplication and whether the operation is intrinsically idempotent. This expression is not an estimate of any product. It identifies two distinct intervention targets: reduce acknowledgment ambiguity or eliminate blind retries. Improving model reasoning alone need not reduce either one.

Suppose authorization changes while the task is interrupted. The original grant allowed preparation of a review artifact, but the operator later revokes remote write access. Restoring the old checkpoint must not restore the old grant as if it were still current. Recovery loads execution progress from the checkpoint and authorization from the authoritative policy service. The resumed model may still see a historical permission statement in H_t; the tool boundary must reject a newly attempted write under the current epoch. This example shows why replay fidelity and present authorization can legitimately diverge.

Deletion introduces another dependency. If a user requests removal of sensitive task material, deleting the current context does not remove journal payloads, filesystem checkpoints, test logs, remote artifacts, or indexing copies. A retention service needs a dependency inventory and a distinction between records required for audit and content permitted for reuse. Where legal or operational retention prevents immediate physical deletion, the system should record the constraint and prevent subsequent model retrieval. It should not describe a tombstone as completed physical erasure. Chapter 29 examines the authority and governance implications.

A controlled evaluation of this workflow uses deterministic repository fixtures and instrumented remote-service simulators. Each fixture specifies the allowed edits, required checks, and permitted remote effects. Failures are injected at journal, network, verifier, and artifact-publication boundaries. The primary unit is a task instance with its complete failure schedule, not an individual model call. Paired schedules allow comparison of recovery strategies without confounding one strategy with easier failures. Model sampling remains a separate repeated-run factor, and uncertainty should preserve clustering by fixture.

Report artifact correctness, stale-verdict acceptance, duplicate creations, authorization violations, unresolved outcomes, and complete execution cost. A runtime can achieve fewer duplicates by refusing all remote work, so benign completion must be reported alongside containment. It can achieve low mean latency by abandoning difficult recoveries, so abandoned and timed-out runs remain visible. These paired metrics force the experiment to evaluate the desired tradeoff rather than reward a degenerate implementation. The worked example yields no universal winner, but it supplies a precise testable distinction between durable progress and merely persistent conversation.

Chapter 21

Multi-Agent Coordination, Information Routing, and Shared State

October 2026 LatComm replaces repeated text transfer with a reusable learned packet between frozen language models [178]. It makes coordination a representation-compatibility problem as well as a scheduling problem: a receiver must interpret what another model encoded, under a finite communication budget. This is evidence from bounded question-answering experiments, not a demonstration that arbitrary autonomous teams share a sufficient state. The chapter therefore begins with task dependencies and proceeds to evidence contracts, latent channels, concurrent memory, and causal team evaluation.

Delegation changes the distribution of computation across contexts, models, tools, and execution times. It does not create additional evidence merely by creating additional agents. A team can improve coverage through parallel exploration, specialist interfaces, and independent checking, yet it can also multiply a shared misconception. The relevant research question is therefore conditional: which decomposition preserves the information required for the task, and when does the resulting gain exceed communication, coordination, and verification costs?

This chapter treats a team as a distributed decision system with explicit ownership and mutable state. The discussion separates parallel sampling from communication, role assignment from capability diversity, and workflow topology from actual execution semantics. AutoGen and MetaGPT provide influential examples of conversational and role-organized coordination [179, 180]. Their architectural patterns motivate comparisons; they do not license treating every multi-agent deployment as a stronger policy.

21.1 Task graphs and the value of decomposition

A task decomposition is a directed graph whose nodes represent work products and whose edges represent prerequisites. A node should specify a deliverable, evidence requirements, permitted tools, budget, and acceptance

condition. “Research the topic” is too underspecified to support reliable integration. “Identify contradictory primary results under the declared cutoff, returning source identity, claim, experimental population, and limitation” defines an auditable information product. The distinction matters because integration quality depends on what the delegate was asked to preserve.

Let $G = (V, E)$ be an acyclic work graph, $w_v \geq 0$ the execution time of node v, and $m \in \mathbb{N}_{>0}$ the worker limit. Ignoring communication and scheduling overhead, the makespan T satisfies

$$T \geq \max \left\{ \frac{\sum_{v \in V} w_v}{m}, \max_{p \in \mathcal{P}} \sum_{v \in p} w_v \right\}, \tag{21.1}$$

where $\mathcal{P}$ is the set of dependency paths. The first term is a work bound; the second is the critical-path bound. Adding workers cannot overcome a serial evidence dependency. Communication, verifier queues, tool contention, and failed retries increase observed makespan above this idealized lower bound.

The graph itself may be uncertain. A search task can reveal a missing prerequisite, invalidate a planned branch, or require another tool. Dynamic orchestration therefore operates over both task actions and graph-editing actions. A manager that repeatedly replans can spend more time maintaining the graph than the workers spend executing it. Graph-edit operations should have budgets and stopping rules, particularly when a model is rewarded for producing elaborate plans rather than verified progress.

Decomposition also changes statistical difficulty. Breaking a coherent task into subtasks may remove context that made the original problem solvable. Conversely, it can reduce distraction by exposing each worker to a narrower evidence set. A controlled experiment should distinguish these effects by comparing the same decomposition executed serially, in parallel without communication, and in parallel with communication. If only the communicating condition improves, evidence exchange is implicated. If serial decomposition already captures the gain, parallelism may primarily affect latency.

An important operating condition is interface stability. Delegation works best when a subtask produces a bounded artifact that can be checked independently of the worker’s narrative. A parsed dataset, executable patch, or evidence table has a clearer acceptance surface than a general recommendation. This does not make artifact verification trivial, but it constrains where errors can enter integration. Tasks requiring continuous negotiation of a shared latent interpretation are harder to decompose safely than tasks with explicit data dependencies.

The graph also needs an invalidation relation. If node v consumed artifact

digest d from node u, a later replacement of that artifact must mark v and its affected descendants stale; graph reachability alone is insufficient when several artifact versions coexist. Reexecuting every descendant is conservative but potentially expensive. Reusing a descendant is justified only when its declared dependencies remain valid, or when an independently checked equivalence permits substitution. A model's statement that a revision is minor is not such an equivalence. This turns the critical path into a versioned evidence path: parallel work reduces elapsed computation only to the extent that its outputs survive integration without invalidating subsequent work. Chapter 32 applies the same dependency principle to deployed model, tool and memory updates.

21.2 Independence diversity and correlated error

Independent samples and collaborating agents answer different questions. If each independent attempt succeeds with probability p, then the chance that at least one of n attempts succeeds is $1 - (1 - p)^n$. This expression assumes independence and an oracle that identifies a valid attempt. A practical selector can reject the successful attempt or accept a plausible failure. Therefore the expression is headroom under a stated model, not an end-to-end reliability prediction.

For averaging, positive pairwise correlation inflates variance under a fixed marginal model. For $n \geq 2$ Bernoulli outcomes $X_i \in \{0, 1\}$ with common success probability $p \in (0, 1)$, sample mean $\bar{X} = n^{-1} \sum_{i=1}^{n} X_i$, and feasible common pairwise correlation ρ, the variance of their mean is

$$\operatorname{Var}(\bar{X}) = \frac{p(1-p)}{n} \left[1 + (n-1)\rho\right]. \tag{21.2}$$

The expression follows by expanding the covariance sum; it does not require full joint independence. When $1 + (n-1)\rho > 0$, the variance-equivalent effective sample size is $n/[1 + (n-1)\rho]$. A zero bracket is a degenerate zero-variance case for which that finite effective-size interpretation is undefined. It is only a variance equivalence, not a complete description of team success. Two joint distributions can share pairwise correlations while differing in the probability of all agents failing together.

Assigning different role names does not guarantee low correlation. Workers can share training data, model family, prompt assumptions, retrieval sources, and tool failures. A critic primed with the original answer may anchor on it even when told to be independent. A more meaningful diversity intervention varies information access, initial hypothesis, retrieval strategy,

or verification method, then measures whether these changes reduce harmful dependence. Diversity that increases disagreement without improving error detection is not automatically useful.

Communication can destroy independence. Once workers see a popular hypothesis, later agreement may reflect copying rather than independent confirmation. An evidence protocol can preserve initial private assessments before discussion. Record each worker's claim, confidence interpretation, supporting observations, and falsification test; then permit exchange. Comparing pre-discussion and post-discussion judgments reveals whether communication corrects errors or amplifies consensus. The process remains an experiment rather than a proof that private reasoning is always preferable.

The causal baseline should spend the same total budget on a single agent, independent samples, and communicating teams. Equal call counts are insufficient when prompts, tool actions, and verifier work differ. Also report wall-clock under a declared concurrency limit. A team may reduce elapsed time while increasing total cost, which can be appropriate for time-critical work. The conclusion should state that tradeoff rather than describe parallelism as cost reduction.

21.3 Messages as evidence-bearing interfaces

A delegation message is an interface contract. Its payload should separate instructions, observations, inferences, proposed actions, and unresolved questions. Flattening them into one natural-language paragraph forces the receiving model to reconstruct authority and provenance. This is especially dangerous when a worker quotes untrusted material that contains instructions. The manager must not treat quoted source content as a delegated command merely because it arrived through an internal agent channel.

An evidence-bearing result contains a claim identifier, source references, the relevant observation, the transformation applied, uncertainty, and applicability conditions. For a code change it includes the artifact digest, tested revision, test selection, and known failures. For a research claim it includes paper version, result location, population, baseline, and limitation. The representation can be structured JSON or another schema; the important property is that integration can validate required fields and inspect the source behind them.

Schema validity does not establish semantic validity. A worker can provide a well-formed citation that does not support the associated claim. Therefore validation must be layered. Structural validation checks field

presence and types. Referential validation resolves identifiers and versions. Evidential validation tests whether the cited observation supports the inference. Task validation checks whether the work product addresses the delegated objective. Conflating these stages makes a successful parser appear to be a scientific verifier.

Compression at handoff introduces an information bottleneck. Suppose a worker observes O, emits message Z, and the manager chooses an action from Z. If two possible observations require disjoint correct actions but induce the same Z and the same other manager information, no rule on that information can be correct on both. Prior knowledge or a follow-up observation can change the information set; the claim is not that all omitted text is unrecoverable. The data-processing principle motivates preserving sufficient evidence rather than only conclusions. In practice exact sufficient statistics are unknown, so interfaces should expose provenance links and permit targeted follow-up retrieval.

A useful failure case is a worker reporting that "the benchmark improved" while omitting that the improvement used additional attempts or a different evaluator. The manager integrates the result as a method gain, and the final synthesis becomes incorrect even though neither agent invented a number. A required comparison field would have exposed the mismatch. This example illustrates why evidence loss at handoff is a distinct error class from hallucination and deserves separate measurement.

Message cost also affects scaling. For a round in which each of n workers emits an L-bit message to every other worker, payload volume is $n(n-1)L$ before protocol overhead. A star that sends those reports once to a manager and broadcasts an L_b-bit summary uses $nL + nL_b$ bits, but holding L_b fixed can discard task-relevant distinctions. The bound is an accounting model, not a measured speedup or a guarantee that the summary is sufficient. Hierarchical summaries reduce volume but introduce another compression layer and a potential single point of epistemic failure. Topic-based routing and shared artifact stores can reduce duplication, provided access control and version identity remain explicit. The appropriate topology depends on the dependency graph; a universal full-mesh conversation is rarely a neutral baseline.

21.3.1 Learned latent channels and representation compatibility

Text is only one communication substrate. LatComm's October 2026 preprint encodes a context once, without a question, into fixed-size hidden-state slots; a different frozen receiver reads them through learned cross-

attention and prompt-dependent gates. Its two stages train the interface, then freeze the compressor while adapting reader-interface and gate parameters for question answering; both LLM backbones remain frozen [178]. Interlat instead studies direct last-layer-state communication and learned compression, including heterogeneous sender–receiver configurations [181]. These mechanisms require access below a typical text-only hosted API. They must not be inferred from ordinary agent handoff support.

To compare such systems, let $x \in \mathcal{X}$ be sender evidence, $q \in \mathcal{Q}$ a receiver request, and $y \in \mathcal{Y}$ the target. A general channel has an encoder $e_\phi : \mathcal{X} \to \mathbb{R}^{p \times s \times d}$, a transport or quantization map Q_b, and a receiver distribution $p_\psi(y \mid q, z)$ with $z = Q_b(e_\phi(x))$. Here $p, s, d \in \mathbb{N}_{>0}$ are channel-group count, slots per group and slot width (with $p = 1$ for a single group), and $b \in \mathbb{N}_{>0}$ is the stored bits per scalar. Under a fixed task distribution and nonnegative predictive-distribution loss ℓ (such as log loss), one possible joint design objective is

$$\min_{\phi,\psi} \mathbb{E}[\ell\{p_\psi(\cdot \mid q, z), y\} + \lambda_B B(z) + \lambda_T T(x, q, z)]\,. \tag{21.3}$$

The weights $\lambda_B, \lambda_T \geq 0$ convert transmitted bits B and end-to-end time T to the chosen objective units. This is a comparison formulation, not the objective attributed to either paper. Frozen-backbone methods restrict the trainable parameters to adapters, compressors, or readers. A reconstruction objective and a task objective impose different information preferences; successful reconstruction is not sufficient for correct action, while task-only optimization can discard distinctions needed by future requests outside the training distribution.

The channel's nominal payload is *psdb* bits before headers and error-control overhead, but token count is an inadequate measure of its cost. The sender may process every evidence token, the receiver attends to slots at selected layers, and each sequential agent can retain a growing cache. Count transmitted tensors, storage copies, dtype conversions, encoder FLOPs, additional receiver attention, and device transfers. A comparison that charges generated text tokens to a text team while excluding latent tensors and extra hidden-state computation gives the methods different accounting boundaries. Conversely, text may be repeatedly tokenized and prefetched when a packet is reusable; reuse count and cache policy must therefore be matched.

Compatibility is semantic as well as dimensional. A width-matching projection does not establish that a new receiver assigns the same meaning to an old packet. Replacing the sender checkpoint, changing selected layers, quantizing the packet, or retraining the decoder changes the channel.

Record an encoder–packet–reader compatibility identity and test migration with frozen packets as well as regenerated packets. If only regenerated packets work, the system has demonstrated a new channel, not backward-compatible memory. A context packet also needs source identity, validity interval, permissions, and deletion dependencies outside its latent values; a dense vector cannot be assumed to carry an enforceable authority label.

The inspected LatComm implementation makes this boundary identifiable: `LatentBridge.compress` constructs the packet, `LatCommModel.encode_context` obtains its encoder input, `LatentInjector` inserts receiver-side transformations, and `PacketCache` manages stored packets [178]. These callable definitions were read at a dated source revision; they were not executed here. Their existence establishes an implementation path, not numerical parity with the paper or robust compatibility after modification.

The packet shape and cache key matter in the released code. `LatentBridge.compress` returns one slot array per paired layer, so a context packet is $[p, s, d]$, not merely $[s, d]$. `PacketCache` selects that packet by a hash of context text and loads its stored dtype; this lookup does not itself validate an encoder–reader compatibility identity. The generation path computes prompt-conditioned gates and reuses them during decoding [182]. These observations bound the implementation claim: a wrapper must bind cache namespace and packet generation to the selected checkpoints and layer pairs. A matching text hash, shape or dtype alone does not establish semantic compatibility, and storage at 16 bits does not imply every consumer-side copy remains 16-bit. No mismatched-cache benchmark outcome is asserted.

A discriminating experiment compares raw text, equal-byte text compression, a single-vector representation, and a multi-slot channel under the same sender evidence, receiver, task stream, and total compute envelope. Include unseen questions about a fixed document and new document types; a query-independent packet should be tested on questions unavailable when it was encoded. Swap in mismatched packets as a negative control, remove the channel, and vary packet precision. Measure action correctness, contradiction handling, calibration, and effect-policy compliance alongside bandwidth and latency distributions. For model migration, cross old and new encoder–reader pairs and retain the off-diagonal failures. A large speed gain on repeated multiple-choice reading would remain useful, but would not by itself establish reliable sharing of open-ended plans, revocable authority, or concurrent agent state.

21.4 Shared memory and conflicting writes

Shared memory is a distributed database whose contents influence policy. Consequently, concurrency errors can become reasoning errors. Two agents may update a fact from different observations, overwrite each other's progress, or retrieve an obsolete skill after another worker changes its assumptions. The required consistency level depends on the record's semantics. Append-only evidence can often tolerate concurrent insertion; a current authorization grant cannot be resolved by choosing whichever natural-language summary sounds more persuasive.

Separate immutable observations from mutable interpretations. Observations carry source, time, object version, and principal. Interpretations cite the observations they depend on and declare a validity interval. A correction can then invalidate an interpretation without destroying the evidence explaining why it was previously adopted. This structure supports both auditing and policy migration. It also prevents a deduplication routine from merging contradictory evidence into a falsely certain consensus.

Optimistic concurrency control is useful when updates are infrequent relative to reads. A worker reads version v, computes a candidate update, and commits only if the stored version remains v. On conflict it must re-evaluate the update against current state. Automatically replaying the old text can reintroduce a revoked fact. The conflict handler is therefore part of the policy interface, not simply a database retry loop.

Different merge functions have different algebraic properties. Set union is associative and commutative for immutable evidence identifiers. Last-writer-wins is operationally simple but uses clock order as a surrogate for semantic authority. Confidence-weighted merging can be misleading when confidence estimates share a common source or when the older record is more authoritative. A memory service should expose the selected rule and its assumptions rather than claim that all conflicts are solved by embeddings.

Procedural memory adds compatibility constraints. HyperSkill organizes skills and subtasks through trajectory hyperedges and maintenance operations [143]. This offers a concrete mechanism for relational reuse, but a shared deployment must additionally define whether two workers may prune or merge the same skill concurrently. The paper's sequential evaluation does not establish those concurrency semantics. A production adaptation should isolate maintenance, version skill identities, and preserve references from trajectories whose interpretation would otherwise change.

Evaluation should inject conflicting writes with known authority order,

not merely test that both records appear in storage. Measure lost updates, stale reads that change decisions, unauthorized cross-user retrieval, and reproducibility of replay. Include a simple isolated-memory baseline: sharing is beneficial only when the transferred information improves the receiving task enough to justify contention, contamination, and governance complexity.

21.5 Ownership cancellation and failure propagation

Every delegated operation needs an owner responsible for its budget, authority, cancellation, and final integration. A manager that starts workers and then terminates can leave active processes with no accountable controller. Ownership should be represented in execution state, with child operations linked to parent task and policy epochs. A child cannot acquire permissions merely because its parent was authorized to delegate reasoning.

Delegation authority is narrower than tool authority. A worker may be allowed to inspect a repository but not publish a branch, or to retrieve enterprise documents but not export them. Passing an unrestricted service credential to every worker defeats role isolation. Capability-scoped credentials and application-enforced tool registries make the distinction executable. The language model's stated role is useful task guidance, but it is not an access-control boundary.

Cancellation has two meanings: stop producing additional work and stop all ongoing effects. The former may be achievable immediately; the latter can require tool-specific termination or reconciliation. If one worker completes the task, cancelling redundant workers should not discard evidence of requests already dispatched. The system needs an outstanding-operation set and a rule for handling late responses. Otherwise a result can arrive after the manager has finalized and silently mutate shared memory.

Failure propagation should distinguish dependent and independent branches. If a shared prerequisite is invalidated, every descendant that used it may require reevaluation. An unrelated worker can continue. This can be implemented with artifact dependency digests rather than broad restart signals. A global restart is simpler but increases cost and can repeat external effects. Overly narrow invalidation is cheaper but risks integrating work derived from obsolete premises.

The tail behavior of a team often depends on stragglers. Waiting for every worker maximizes coverage but can make latency dominated by one failed dependency. Quorum completion can reduce latency but changes

the evidence population and may systematically omit difficult or dissenting analyses. The stopping rule should specify whether missing workers count as failures, censored observations, or intentionally unused candidates. A post hoc choice of whichever stopping rule makes the run look best is evaluation leakage.

Fault-injection tests should combine worker loss, delayed messages, policy revocation, and shared-store outages. Independent single faults are easier to diagnose, but correlated failures are operationally important: one tool outage can affect all workers, while one poisoned shared memory can redirect the whole team. Report containment and benign completion together. A coordinator that avoids every incident by refusing delegation has not demonstrated useful orchestration.

21.6 Experiments that isolate coordination mechanisms

A factorial design can vary decomposition, communication, model diversity, and memory sharing. The basic conditions are a single agent, serial specialists, parallel independent specialists, and communicating specialists. Each uses the same task set, tool fixtures, evaluator access, and resource accounting. The design should declare whether it matches money, tokens, elapsed time, or more than one envelope. Different matches answer different questions; they should not be mixed into one headline ranking.

For a fixed task instance i, define the paired gain $\Delta_i = Y_i^{\text{team}} - Y_i^{\text{base}}$. Average gains should be accompanied by their distribution over task families and difficulty strata. A positive mean can hide large regressions in tasks requiring coherent global context. Repeated model samples are nested within task instances, and shared persistent memory can induce dependence across tasks. Resampling individual calls as if they were independent produces overly narrow uncertainty intervals.

Communication ablations should preserve available information when possible. Removing all messages can change both coordination and evidence access. A stronger control supplies the same evidence through a deterministic aggregator while removing agent discussion. Conversely, randomizing or delaying selected messages tests whether timing and ordering drive the result. Such interventions can distinguish helpful information exchange from social anchoring or redundant narration.

Verifier access must be matched. A team with a dedicated critic effectively has more inference and perhaps additional tools. Compare it with a single agent receiving the same executable feedback, and with independent

candidates selected by the same critic. If the gain comes from the critic rather than collaboration, the result still matters, but the mechanism should be named accurately. A critic's agreement is not oracle truth; its false acceptance and false rejection require independent audit.

The main failure taxonomy separates decomposition errors, worker errors, communication loss, shared-state conflict, integration errors, and verifier errors. A final wrong answer may involve several stages. Assigning all failures to the last agent obscures causal structure. Use seeded faults and counterfactual replay to test whether replacing a particular message or artifact changes the outcome. Replay is informative only when external state and model nondeterminism are controlled or explicitly accounted for.

These experiments are a proposed research program rather than results established by the cited frameworks. Their practical value is to make a claim such as "multi-agent reasoning helps" operational. The claim becomes a statement about a particular topology, information protocol, task distribution, and budget. That precision allows disagreement between studies to be investigated rather than averaged away.

The October 2026 coding-team study directly varies which workflow stage receives workers: planning, implementation or verification [183]. In its ProgramBench configurations, planning delegation captures a different reward–cost tradeoff from adding implementers or verifiers; inexpensive workers are not interchangeable across roles. This is a configured empirical result, not a general law that planning is always the best allocation. The study does not evaluate team safety. Its mechanism suggests a sharper ablation than team size alone: preserve the lead, budget and information access, vary the recipient stage, and measure integration work as well as worker output. A cheap planner can supply a useful alternative hypothesis while a cheap implementer creates edits the lead must repair. The relevant causal mediator is useful accepted work after integration, not the number of delegates or messages. Testing that mediator requires recording rejected plans, conflicting patches and redundant verification so hidden consolidation cost cannot be mistaken for free parallelism.

21.7 A bounded integration protocol

The integration step should be designed as carefully as worker execution. A manager receives candidate artifacts, validates their provenance, identifies disagreements, and chooses whether more work is justified. It should not simply concatenate worker summaries and ask another model to produce a fluent answer. Concatenation preserves text but does not resolve incompat-

ible assumptions, duplicated evidence, or different interpretations of the task.

One useful protocol maintains a claim graph. Nodes are claims or artifact predicates; edges identify supporting evidence, contradiction, or dependency. Workers submit proposed graph updates rather than final conclusions alone. The manager checks whether two claims concern the same population, version, and resource envelope before labeling them contradictory. Apparent disagreement can disappear when one worker reports author results and another reports an independent reproduction under a different environment. Genuine unresolved disagreement should remain visible.

Integration can stop when all required predicates have accepted evidence, unresolved contradictions have been represented, and the remaining expected value of additional work is below its cost under the declared objective. This stopping rule is conceptual because value estimates are uncertain. An implementation can approximate it with bounded rounds, explicit missing-evidence categories, and a limit on repeated requests for the same information. The approximation should be evaluated rather than presented as optimal metareasoning.

The protocol also needs a quarantine path. A worker result containing unverifiable citations, unexplained external changes, or suspicious instructions should be retained for diagnosis without entering trusted shared memory. Quarantine is not deletion and does not itself prove maliciousness. It prevents uncertainty about one artifact from silently becoming authority for subsequent actions. False quarantines reduce useful throughput, so their rate belongs in the evaluation alongside successful containment.

For a scientific review task, integration should retain disagreements over contamination, baseline strength, and statistical uncertainty even when the final narrative would be simpler without them. For a coding task, it should retain a failing regression test rather than allow a worker's confident explanation to supersede it. For enterprise actions, it should recheck authorization at the effect boundary. These examples share a principle: evidence and authority are properties of records and current policy, not consequences of a worker's role label.

The resulting team is best understood as a controlled distribution of computation and evidence. Parallelism supplies opportunities for coverage and latency reduction. Typed handoffs preserve what each opportunity discovered. Shared-state protocols prevent those discoveries from becoming incoherent or unauthorized. Independent evaluation determines whether the arrangement improves useful outcomes. None of these layers can be

omitted by appealing to the number of agents or the apparent sophistication of their conversation.

21.8 When isolation is the stronger coordination mechanism

Consider two workers preparing changes to different modules of one repository. Their edits appear independent, but both modify a shared interface specification. A shared writable checkout permits immediate coordination through the filesystem, yet it also permits one worker to test a transient mixture of changes. The resulting passing test verdict may correspond to no committed artifact. A later merge can then fail despite both workers reporting success. This is an evidence-identity failure rather than a lack of programming ability.

An isolated-workspace design gives each worker a base revision and a separate branch. The manager merges their artifacts and runs integration checks on the merged digest. This adds storage and validation cost but creates a clear causal boundary: worker tests concern branch artifacts; integration tests concern the combined artifact. If the interface changes are incompatible, the conflict is explicit. Isolation does not prevent semantic incompatibility, but it prevents accidental shared state from concealing it.

The same reasoning applies to research. Two investigators can independently inspect conflicting papers before receiving each other's conclusions. Their private notes preserve the evidence that motivated each interpretation. Discussion then operates on the disagreement rather than on an already homogenized summary. Complete isolation would prevent useful correction, so the design uses staged disclosure: independent extraction first, evidence exchange second, joint interpretation third. The stages allow an experiment to observe where agreement arises.

For memory systems, isolation can be temporary and selective. A worker may read a shared evidence snapshot while writing candidate interpretations to a private namespace. A promotion gate checks provenance, conflicts, and applicability before moving those interpretations into shared memory. This resembles a database transaction but adds semantic evaluation. The gate can still make mistakes, and its acceptance threshold determines the tradeoff between contamination and delayed learning. Promotion should therefore retain the original candidate and gate verdict for later audit.

The cost of this mechanism grows with candidate volume. If every worker proposes many redundant records, verification can dominate execution. Deduplication before promotion reduces load, but semantic deduplication

can incorrectly merge distinct conditions. A safer first stage deduplicates exact source identities and artifact hashes; a later stage considers whether two interpretations are equivalent under their stated scope. Keeping these stages separate makes it possible to attribute errors to retrieval, deduplication, or review.

An evaluation should compare immediate shared writes, snapshot reads with private writes, and fully isolated memory under the same task sequence. Inject one incorrect but plausible record and measure its propagation, subsequent correction, and effect on task outcomes. Also include a valid rare record: a system that blocks all unusual memories may contain contamination while preventing beneficial transfer. Measure both harmful propagation and useful adoption, along with the review work needed to distinguish them.

The implication is conditional rather than categorical. Shared state is valuable when its information benefits exceed its coordination and contamination costs. Isolation is valuable when it preserves evidence identity, authority boundaries, or independent judgment. A principal design decision is to choose the smallest shared surface required for the task. Increasing communication and memory visibility without identifying the dependency they serve is not a scientific explanation of why a team should improve.

Figure 21.1 isolates the mechanism and its stated information boundaries.

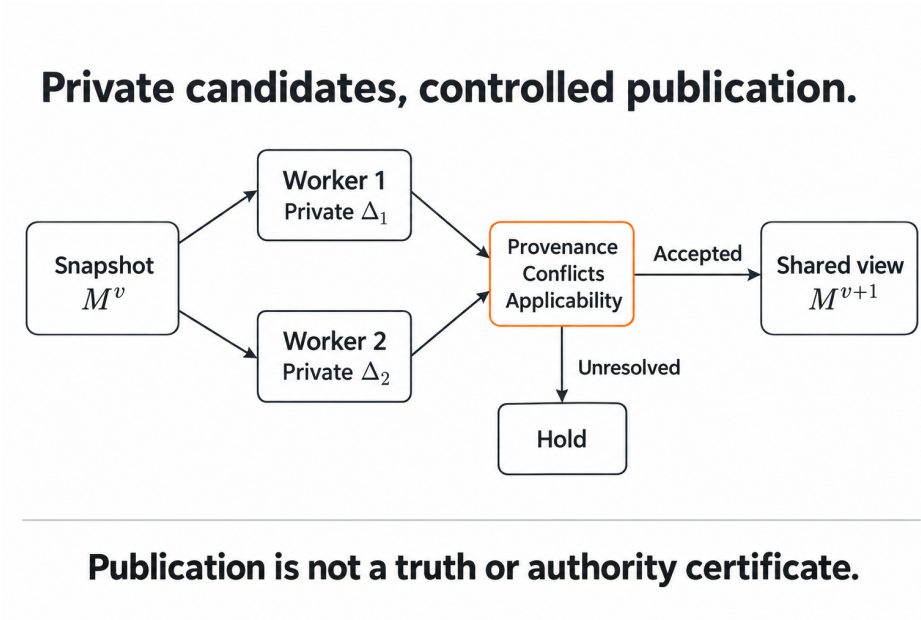

Figure 21.1: Workers can read one versioned snapshot while retaining candidate interpretations privately. Only the promotion path publishes accepted candidates into a new shared view; unresolved candidates remain separate. Provenance, conflict and applicability checks retain an auditable decision boundary. The gate does not establish semantic truth, current effect authority, or a performance improvement.

Chapter 22

Agent Development Frameworks, SDKs, and Managed Runtimes

The October 2026 SDK releases expose failures that a framework comparison must represent. OpenAI Agents Python v0.24.0 defers automatic compaction while approvals are pending and preserves completed tool outputs when a sibling tool fails; Google ADK v2.13.0 marks its MongoDB session and memory services experimental [184, 185]. These are version-specific documented changes, not guarantees inferred from a brand. They motivate comparing the exact interruption, publication, and compatibility semantics of a framework before counting its integrations.

An agent framework is a set of choices about where control resides. Its most consequential properties are not the number of model adapters or the expressiveness of a demonstration, but the semantics of state, effects, interruption, authorization, and observation. A framework can make those semantics explicit, hide them behind convenience interfaces, or delegate them to another service. These alternatives impose different experimental and operational obligations.

The comparison in this chapter begins with role rather than rank. A graph runtime, a typed model SDK, an opinionated harness, a workflow builder, and a managed execution platform occupy different boundaries. They can compose within one application and should not be treated as mutually exclusive competitors. The official-source inventory is interpreted as documentation of interfaces available for inspection on the literature cutoff, not as an independently stress-tested feature leaderboard.

22.1 A semantic basis for framework comparison

Define a framework profile by five sets: states it represents, transitions it controls, effects it mediates, observations it exports, and obligations it leaves to the application. This representation makes omissions visible. An SDK can validate a tool argument but leave external idempotency unspecified. A runtime can persist graph state but leave document authorization to a connector. A managed service can maintain sessions while exposing only

partial execution traces. None of these limitations is inherently disqualifying; they determine the boundary of a defensible claim.

A useful comparison matrix distinguishes documented support from verified behavior. “Persistence” should be decomposed into conversation continuity, process-restart recovery, durable external artifacts, semantic memory, and cross-version migration. “Human review” should identify where execution pauses, what the reviewer sees, and whether authority is rechecked after resumption. “Observability” should identify whether traces include full tool arguments, truncation indicators, source identity, and asynchronous work. A single Boolean column loses these distinctions.

The unit of versioning is similarly broader than a package number. An application may combine a pinned SDK, moving model alias, remote tool schemas, hosted runtime, and changing connector permissions. Recording the SDK release alone does not reproduce the system. Define a compatibility vector whose components include each mutable boundary and attach it to evaluation traces. Unknown service revisions should remain unknown rather than be inferred from a documentation date.

More precisely, a configured deployment has a manifest $\xi = (\nu, \theta, \delta, \tau, \beta, \alpha)$: application/framework revision ν, model identity θ, deployment decoder δ, tool-schema/adapter revision τ, persistence backend/schema β, and current authority epoch α. These components need not share a release cycle. A saved run additionally requires its realized context and external-effect records; ξ identifies configuration rather than reconstructing those data. Framework comparison holds the relevant components fixed or declares their joint change as the treatment. Otherwise a newer SDK coupled to a stronger model is not an estimate of an SDK effect.

For feature equivalence, require observable contracts. Two frameworks both exposing a method called resume are not equivalent if one restores a model-visible history and another resumes an event-sourced workflow. The first may repeat tool effects; the second may skip completed activities but depend on deterministic application code. A conformance workload should therefore include an interruption after external acceptance and before local commitment, rather than test only that a suspended conversation can receive another message.

The practical implication is to select a framework against the failure modes of the intended workload. A short extraction service may benefit most from typed output and dependency injection. A multi-hour engineering task needs workspace state, cancellation, and reconciliation. A regulated business action needs principal identity and authoritative permission checks.

Breadth of advertised features cannot replace the task-specific evidence needed to compare these roles.

22.2 Graph runtimes and opinionated harnesses

Graph runtimes make control flow an explicit application object. Nodes transform state, edges constrain possible continuations, and persistence attaches to defined execution boundaries. This can support inspection and fault recovery, but the graph is not automatically deterministic: nodes can contain model calls, time-dependent queries, or external effects. A graph's visual clarity therefore does not prove replay equivalence. The implementation must identify which values are recorded and which computations execute again.

LangGraph illustrates a runtime layer, while Deep Agents supplies a more opinionated harness over that ecosystem [186, 187]. The latter's documented filesystem, subagent, and context-management interfaces are mechanisms to examine. Their existence does not establish that summaries preserve every constraint or that delegated work is cheaper. A useful experiment holds the underlying runtime fixed and separately varies offloading, compaction, and delegation. Otherwise a comparison between the two layers confounds control defaults with the runtime itself.

An opinionated harness reduces application effort by choosing defaults for prompts, tools, memory, and stopping. The benefit is faster construction of a coherent system. The scientific cost is that a high-level configuration can activate several mechanisms at once. Researchers should expose these defaults in their experimental configuration and inspect the code path used by the evaluated task. Disabling a component should not silently activate a fallback whose cost or behavior is materially different.

Event-driven workflow systems provide another control model. Events describe completed work or new requirements, and handlers may schedule further tasks. This is useful when external processes, human decisions, or delayed data determine progress. However, event duplication and ordering become part of agent semantics. A workflow that processes the same approval event twice can execute an action twice unless it deduplicates by stable event identity. These concerns persist even if every model call is perfectly accurate.

The discriminating experiment is not whether a sample application runs. It is whether equivalent state transitions produce equivalent committed effects under interruptions, retries, and schema evolution. Use a small deterministic tool fixture first, then introduce model nondeterminism as a

separate factor. This ordering prevents a transport or persistence defect from being misdiagnosed as reasoning failure. It also establishes which guarantees survive when the framework delegates execution to a managed backend.

22.3 Typed SDKs and model-owned loops

Typed SDKs place schemas and application dependencies around model interactions. Type validation can reject malformed arguments, constrain output shape, and expose errors before application code executes. Its guarantee is syntactic and representational. A valid customer identifier can still refer to the wrong customer, and a well-typed monetary amount can still exceed authorization. Semantic preconditions must therefore remain explicit application checks rather than be assumed consequences of schema adherence.

The inspected OpenAI Agents SDK separates an application-owned SDK path from managed runtime options [126]. Handoffs, tool loops, session interfaces, and tracing provide useful abstractions, but applications still own tool effects and their storage semantics. Its SDK is provider-agnostic; the evaluation must identify the selected model adapter rather than infer it from the SDK name [188]. PydanticAI makes typed interaction an inspectable design surface and also supplies a configurable harness, while Pydantic Graph and Pydantic Evals are separate packages [189]. Restricting its inventory to output validation would omit its current context, tool and execution controls. A comparison should examine how validation failures are surfaced, whether repair attempts consume the same budget, and whether rejected calls enter the trajectory used for later training or evaluation.

The Claude Agent SDK occupies a different ownership boundary from a Messages API client. Current official documentation describes a Python/TypeScript library running the Claude Code binary in an application-operated process, with its agent loop, tool execution, permissions, sessions and hooks. A client SDK instead exposes model requests, while Claude Managed Agents supplies a hosted harness with a separately configured execution environment [190, 191]. These are three distinct experimental treatments. A common model identifier does not make their context constructors or permission mechanisms equivalent.

One failure mode is repair-induced drift. A model proposes an invalid tool call, the SDK returns an error, and the next attempt changes a meaningful argument while correcting the format. If evaluation records only the final valid call, it omits the information that caused the transition. For a

learned policy this can invalidate behavior-likelihood reconstruction. For a production service it can conceal repeated boundary probing. Validation and repair events should therefore be first-class trace records.

Model-owned loops choose their next action dynamically, whereas application-owned workflows constrain the available continuation structure. Neither is universally superior. Dynamic loops accommodate unforeseen subtasks but can cycle, invent unsupported tool names, or consume budgets without progress. Explicit workflows improve predictability but can fail when the task falls outside their branches. A hybrid design uses deterministic control for authority and commitment while permitting model-driven choice within a bounded action space.

The relevant baselines compare the same tool contract under direct function orchestration, a minimal agent loop, and the SDK's higher-level abstractions. Measure invalid calls, repair rounds, constraint violations, total cost, and completion. A framework's ergonomic advantage is real but requires a different study, such as developer implementation time under controlled tasks. It should not be inferred from task success or substituted for runtime reliability.

22.4 Development kits interoperability and language boundaries

An agent development kit typically bundles authoring conventions, tool integration, orchestration, testing, and deployment adapters. This breadth can improve consistency but also creates version coupling. A task object serialized by one language implementation may not preserve every field expected by another. A model adapter can normalize errors differently, changing retry behavior. Cross-language support is therefore a compatibility question rather than a guarantee that all implementations have identical semantics.

Google ADK's inspected repository describes graph-based workflows and structured delegation surfaces, while Microsoft Agent Framework describes multi-language agent and workflow construction [192, 193]. These are concrete interfaces for conformance tests. The experiment should pin the relevant implementation and compare equivalent task graphs, state updates, and failure injections. It should not compare unrelated examples and attribute the difference to the toolkit brand.

Protocol support is similarly conditional. A framework advertising MCP can expose tools, resources, or prompts through different transport and authorization configurations. An A2A integration can exchange task status

without sharing the same memory model or trust boundary. Chapter 19 treats these protocol distinctions directly. At the framework layer, the important question is which semantics are preserved when an adapter maps between local objects and external messages.

Serialization requires particular care for authority-bearing data. A human approval record should identify the precise operation, relevant arguments, principal, expiry, and policy epoch. Mapping it to an unstructured string in another framework weakens its meaning. The receiving application must validate the approval against its own action boundary. Successful message delivery cannot establish that the transferred object authorizes the requested effect.

Interoperability experiments should include a deliberate version mismatch, optional fields unknown to the receiver, partial tool failures, and revoked credentials. Report which failures are detected before dispatch and which emerge only after an effect. A graceful error is often preferable to a silent fallback that changes task meaning. The strongest interoperability result is a documented set of preserved invariants under declared adapters, not an assertion that any agent can collaborate safely with any other.

22.5 Role-specific framework and runtime inventory

Table 22.1 assigns each listed framework or managed runtime a concrete inspection target. The entries describe documentation surfaces, not measured capability or comparative quality. The final column is a discriminating test: it identifies an observation needed before an engineering conclusion is justified. Where a source redirects, the resolved resource defines the inspected family; historical names are lineage identifiers. Package and hosted-service parity remain unverified unless established by a version-specific test.

Table 22.1: Framework and managed-runtime surfaces with discriminating checks.

Product	Documented surface	Required test
LangGraph[186]	Graph checkpoints and stores	Crash after an effect; distinguish restored state from repeated dispatch.
OpenAI Agents SDK[188]	Agent and Runner loop interfaces	Preserve rejected calls, handoff ownership and application side-effect receipts.
Agent Development Kit — ADK[192]	Workflow and task delegation objects	Compare graph transitions under retry, human interruption and version mismatch.
PydanticAI[189]	Typed loop and configurable harness	Inject validly typed but unauthorized arguments; separate types from permission checks.
Deep Agents[187]	Filesystem offload and isolated subagents	Remove a rare constraint during compaction and test later recovery.
Microsoft Agent Framework[193]	Multi-language agents and workflows	Round-trip state across language adapters; retain cancellation and error semantics.
Agno / AgentOS[194]	Agent teams and runtime interfaces	Test whether shared memory commits conflict or silently overwrite one another.
LlamaAgents / Agent Workflows[195]	Event-driven workflow composition	Duplicate an input event and inspect whether downstream effects repeat.
Strands Agents / Harness SDK[196]	Composable harness control interfaces	Locate the actual agent-loop implementation after the source redirect; pin the adapter.
OpenClaw[197]	Cross-channel agent gateway	Revoke one channel principal and test isolation of historical context.
Mastra[198]	TypeScript agents and workflows	Interrupt a suspended workflow and verify that resumption preserves evidence identity.
Claude Agent SDK[199]	Claude-oriented SDK execution surfaces	Identify application versus provider ownership of hooks, session state and tool effects.

Product	Documented surface	Required test
Haystack[200]	Retrieval pipelines and agent components	Replace retrieval with oracle evidence while holding generation and tool budgets fixed.
NeMo Fabric[201]	Cross-harness execution and telemetry	Check whether adapter traces preserve truncation, delegated work and error provenance.
smolagents[202]	Code-action agent loop	Inspect the generated-code execution boundary and sandbox mediation of effects.
DeepSeek Harness[203]	Plugin-oriented harness interfaces	Disable one plugin and inspect transitive tool exposure and persisted assumptions.
AgentScope[204]	Agent application and team abstractions	Test cancellation propagation and whether conversation identifiers imply durable storage.
CrewAI[205]	Role-based crews and explicit flows	Compare identical role assignments with deterministic flow and model-selected delegation.
Pi[206]	Terminal harness and extension interfaces	Load a minimal extension and audit which credentials and context it receives.
DSPy[207]	Program optimization against evaluation	Keep an untouched holdout and count selection attempts as part of optimization cost.
OGX, formerly Llama Stack[208]	Portable server-side agent interfaces	Follow the OGX lineage and verify old client schema compatibility explicitly.
Qwen-Agent[209]	Qwen-oriented tools and retrieval	Match tool serialization and model settings before attributing benchmark differences.
n8n[210]	Workflow automation with agent steps	Separate durable workflow state from model-visible memory and connector permissions.
Langflow[211]	Visual Python agent composition	Export the executable graph and test whether visual and API paths agree.
Dify[212]	Visual workflows and retrieval pipelines	Check which runtime and connector components are actually inspectable and licensed.

Product	Documented surface	Required test
Elastic Agent Builder[213]	Search-grounded agent construction	Change document permissions after indexing and test retrieval-time enforcement.
CAMEL[214]	Collaborating agent framework	Compare communicating workers against equal-cost independent samples and one-agent controls.
Coze Studio[215]	Self-hostable visual agent workflows	Inspect credential scoping, deployment dependencies and duplicate-event recovery.
UiPath Maestro[216]	Business-process orchestration patterns	Interrupt a human approval chain; bind the resumed action to current authorization.
Agent Bricks[217]	Custom-agent hosting and data integration	Verify workspace persistence and trace identity across a container restart.
Bedrock AgentCore[218]	Composable runtime identity and memory services	Test service boundaries separately; a memory feature does not establish agent correctness.
Gemini Enterprise Agent Platform[219]	Managed agent lifecycle services	Distinguish low-code, managed API and custom ADK deployment semantics.
Agents API[220]	Managed sessions and execution harness	Compare saved turn history with actual external-effect reconciliation under failure.
Claude Managed Agents[191]	Managed long-running agent sessions	Inspect environment persistence, permission policy and incomplete-turn outcomes independently.
Agents & Conversations API[221]	Agent configuration and conversation entries	Test history continuity separately from tool execution and external transaction guarantees.

Several entries cross category boundaries. Pi can be used as a coding harness and as an extension substrate. NeMo Fabric concerns execution and telemetry across harnesses rather than a replacement for the model policy. DSPy concerns program and prompt optimization against evaluation objectives; it should be tested for selection overfitting as well as task quality. A visual builder such as Langflow, Dify, Coze Studio, or n8n can host model-driven steps inside deterministic workflows. Its graphical interface does not remove the need to examine stored state, credential scope, and

replay behavior.

Managed platforms change the observation boundary. AgentCore, Agent Bricks, and Google's enterprise platform document infrastructure surfaces that an application can compose with custom agents. Managed agent APIs supply a larger portion of the loop. These roles should be compared through migration effort, observability, failure recovery, and enforceable policy controls, rather than counted as additional reasoning architectures. A serverless runtime's availability commitment is not evidence that an agent's task output is correct.

22.6 Licensing lifecycle and migration evidence

Implementation visibility, artifact license, hosted-service terms, and operational inspectability are separate properties. A public repository can contain a client while the service remains proprietary. A source-visible license can restrict deployment or redistribution. A permissive core license may coexist with separately licensed connectors or enterprise features. The defensible statement identifies the inspected artifact and its license file at a revision; it does not attach an unqualified open-source label to an entire product family.

The inspected repository notices illustrate why this granularity matters. Mastra distinguishes its Apache-2.0 core from enterprise-licensed directories; n8n identifies Sustainable Use and Enterprise licenses; Dify identifies an Apache-2.0-derived license with additional conditions [198, 210, 212]. These notices establish artifact-specific terms, not a universal compatibility judgment between deployments. DeepSeek Harness explicitly declares developer-preview status and anticipated compatibility-breaking changes [203]. An active commit stream therefore does not establish a stable serialized-state or plugin ABI.

Lifecycle changes can invalidate comparisons even when names remain familiar. The inspected Llama Stack link resolves into the OGX lineage, and the Llama workflow link resolves to a broader agents repository. The Strands source link resolves to a harness SDK resource. These redirects are evidence of naming and packaging changes, not proof of complete backward compatibility. An application relying on old imports, storage schemas, or tool names needs a migration test rather than a renamed bibliography entry.

Historical products should be preserved where they explain architectural evolution, but they should not be described as active recommendations without current evidence. Archive status, maintenance activity, security-fix

policy, and availability are distinct. A repository can remain readable after service termination; a commercial service can remain available while a legacy SDK is deprecated. The operating consequence is to maintain a dependency inventory with replacement and rollback plans, particularly for long-lived memory formats.

A migration experiment starts from a saved run and its authoritative state. Restore under the target release, validate schemas, rebuild indexes when required, and replay deterministic fixtures before permitting new model decisions. Compare tool availability, parser behavior, serialization, and cancellation. A test that merely imports the new package misses the semantics most likely to affect agent behavior. If the provider does not expose the necessary version controls, report that reproducibility limitation explicitly.

For a deterministic fixture, write $F_\nu : \mathcal{S}_\nu \times \mathcal{U} \to \mathcal{S}_\nu$ for a release's state transition and $O_\nu : \mathcal{S}_\nu \times \mathcal{U} \to \mathcal{E}$ for its observable event output. Let $m : \mathcal{D} \subseteq \mathcal{S}_\nu \to \mathcal{S}_{\nu'}$ be a partial migration with the same fixture inputs $\mathcal{U}$. On a transition-closed migration domain, require

$$m(F_\nu(s,u)) = F_{\nu'}(m(s),u), \tag{22.1}$$

$$O_\nu(s,u) = O_{\nu'}(m(s),u). \tag{22.2}$$

If both identities hold for every admitted (s,u), induction on the finite input sequence gives identical fixture event traces and corresponding states after every step. This is a contract-preservation criterion, not a claim that all migrations satisfy it. The event space must include relevant tool intents, validation failures and recorded receipts; comparing only final chat strings discards precisely the events that can change effects. Real remote effects still require the service contract of Chapter 20. A migration may legitimately reject an unsupported saved state; silently coercing it into another agent's state violates the criterion. A finite test suite provides evidence on its reached states, not proof of the universal identities. Stochastic model transitions require distributional comparison or fixed recorded outputs, rather than assuming identical samples.

Ownership and acquisition claims are excluded from technical inference unless primary evidence establishes them and they matter to the system boundary. Even a verified corporate change does not imply that products share code, data, or runtime guarantees. The analysis follows inspectable mechanisms and contracts. This avoids turning a provider map into an unsupported architectural map.

22.6.1 Release-specific evidence at the October cutoff

The official release records rechecked on 29 October identify LangGraph 1.2.18 on 26 October 2026, OpenAI Agents Python SDK 0.24.0 on 15 October, Google ADK 2.13.0 published on 23 October with release-note heading dated 22 October, and PydanticAI 2.63.0 on 28 October[222, 184, 185, 223]. These identifiers delimit the evidence; a current main branch can contain later or unreleased changes. Microsoft's repository has distinct .NET 1.26.0 and Python 1.23.0 releases, dated 23 and 22 October respectively, rather than one universal cross-language release[224, 225]. CrewAI's inspected release was 1.15.32, while smolagents' latest returned release was 1.26.0 from May[226, 227]. Release recency alone is not a quality metric.

Several changes directly affect the consistency arguments in this book. LangGraph's notes change the default frequency of full delta snapshots from one thousand updates to one hundred, trading additional storage for faster reads on long threads. This is a concrete checkpoint-layout change, not evidence of better semantic memory. The Agents SDK notes defer automatic compaction during pending approvals, replace Redis compaction history atomically, and preserve completed tool outputs when a sibling fails. Those fixes identify distinct state boundaries: approval evidence, history replacement and partial parallel completion. They motivate targeted regression fixtures rather than a general claim that all consistency problems are solved[222, 184].

ADK's release notes mark MongoDB session and memory services experimental even while promoting other features to stable status. They also change the default telemetry schema. Consequently, neither maturity nor trace compatibility should be assigned at whole-framework granularity. An analytics pipeline that assumes an older span layout can silently lose attribution after an otherwise successful upgrade. PydanticAI's notes change the default of its `CodeMode` dynamic catalog so newly revealed tools append rather than rewrite cached prompts. That is a component-specific context-construction choice whose effect depends on tool-discovery workload and cache behavior; it is not a universal change to every tool path or a measured application-level speedup[185, 223].

Figure 22.1 isolates the mechanism and its stated information boundaries.

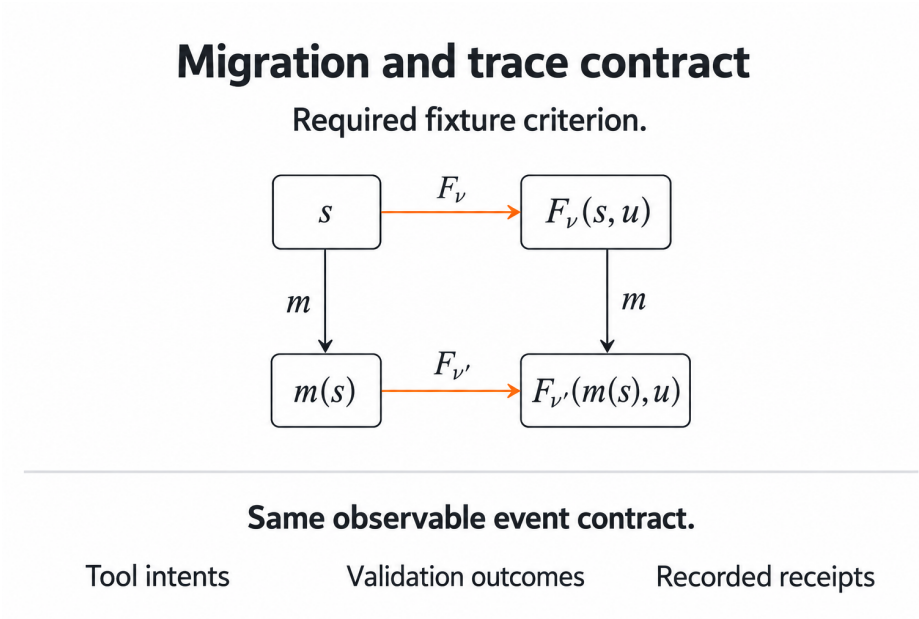

Figure 22.1: A deterministic migration criterion compares the two paths through corresponding fixture states. On an admitted transition-closed domain, migration must commute with the state transition and preserve the declared observable events. Finite fixtures test reached states; they do not prove the universal identities or establish remote-effect uniqueness.

22.7 A conformance suite before performance ranking

A framework conformance suite should exercise schema validation, state restoration, duplicate dispatch, cancellation, authorization revocation, and trace completeness before comparing task scores. Each fixture has an expected observable contract. For example, a revoked write capability must prevent a new effect even if an old checkpoint contains a permission statement. A committed terminal receipt should be replayed without a new effect; an unresolved dispatch should reconcile before any retry justified by the service's explicit retry contract. A truncated tool result should carry an explicit truncation indication or preserve a recoverable artifact reference.

Conformance is not a universal safety certificate. It establishes behavior for the tested contract and release. A framework can pass deterministic fixtures and still induce poor model decisions through context construction. Therefore the second stage evaluates task outcomes under matched models,

tools, and budgets. Separate the effect of the framework from the effect of its recommended prompt, default model, and hosted extensions. When those cannot be separated, the treatment is the complete configured system.

Component effects need not be additive. For two configurable mechanisms, let μ_{ij} be expected task outcome when mechanism one has state $i \in \{0, 1\}$ and mechanism two has state $j \in \{0, 1\}$, with the backbone, information access, resource envelope and task population fixed. The interaction is

$$I = \mu_{11} - \mu_{10} - \mu_{01} + \mu_{00}. \tag{22.3}$$

For example, compaction can change which tool descriptions remain visible, while dynamic discovery changes which descriptions need to survive compaction. A gain from enabling both does not identify either individual contribution; a nonzero I measures their departure from additivity on the declared outcome scale. If an SDK cannot expose all four configurations, the interaction is unidentified from that product comparison. This is an explicit observation limit, not a reason to assign the combined gain to the most prominently advertised feature.

Statistical comparisons should use paired tasks and report uncertainty at the task or session level. Framework adapters can fail only on particular tool schemas, so stratify by schema complexity and effect type. Count startup failures, missing outputs, and dependency outages rather than removing them as setup problems after observing results. A system that requires undocumented manual repair has a materially different operational profile from one that completes unattended under the same fixtures.

Finally, include an implementation-complexity analysis without mistaking it for benchmark quality. Count application-owned components, external dependencies, state stores, custom adapters, and recovery code. These counts do not form a universal simplicity score, but they explain where maintenance responsibility resides. A highly capable managed service may reduce local code while increasing opaque dependencies. A minimal SDK may expose more work but permit a stronger audit. The appropriate choice depends on which uncertainty the deployment can tolerate and which guarantees it must enforce.

22.8 From public API to executable control path

A concrete audit can start with three interfaces rather than an entire framework taxonomy. In the inspected LangGraph source tree, the state-graph implementation resides in the graph state module and exposes compilation into an executable graph. In the OpenAI Agents Python SDK, the run

module exposes the Runner control surface. The Google ADK source exposes runner infrastructure for application execution. These are identifiable inspection points, not assertions that the corresponding modules implement every guarantee discussed above. Their current source files were reachable at the cutoff; experiment revisions still require immutable pinning [186, 188, 192].

For the graph case, trace compilation into checkpoint use. Identify where thread identity enters configuration, which state updates are reduced together, and which node outputs are persisted before successors execute. Then place an effectful tool call inside one node and inject a crash immediately after the service accepts it. If resumption reexecutes the node, the application must supply idempotency or reconciliation. This test distinguishes graph-state durability from external-effect uniqueness without requiring a language model in the fixture.

For the Runner case, trace model output through tool-call decoding, schema validation, execution, and subsequent model input. Capture the exact error object returned for an invalid argument. Then inspect whether a handoff changes the active instructions, tools, model settings, or state ownership. The experiment should preserve the pre-handoff authority envelope and verify that delegation cannot expand it. A handoff that routes a conversation successfully can still be incorrect if it transfers more context or credentials than the receiving specialist requires.

Current source inspection supplies narrower testable claims. LangGraph's pinned `DeltaChannel` constructor sets `snapshot_frequency=100`; this supports the checkpoint-layout claim without asserting semantic preservation by compaction [228]. The Agents SDK's pinned `RunState` uses schema 1.21 and rejects ambiguous legacy agent identities; its documentation warns that neither context overrides nor strict context deserialization establishes snapshot integrity [229]. The required fixture therefore changes a duplicate-agent graph and verifies a safe rejection, rather than relabeling an old checkpoint until loading succeeds. Serialization compatibility and authenticity are separate checks.

For an ADK application, inspect how an event updates session state and how workflow execution consumes that state. Duplicate one event identifier and delay another. The expected behavior must come from the selected application's contract, because a generic toolkit may legitimately expose several scheduling modes. Evaluate whether the serialized event contains sufficient identity to diagnose the difference. A toolkit's flexibility increases the number of legal configurations; it does not eliminate the need to state which one produced a result.

For example, the pinned ADK `InMemorySessionService.append_event` skips partial events and suppresses an equal re-delivery only after matching its event identifier. Distinct event objects with the same identifier but different fields are retained. Its scope is an in-memory session implementation, not restart durability or universal event-ID uniqueness across backends [230]. A discriminating fixture must therefore test both identical re-delivery and conflicting reuse, and must inspect the selected persistent backend independently. An identifier's presence in a serialized object is insufficient evidence that every consumer enforces its intended uniqueness.

These inspections produce a claim-to-code ledger. Each row records the public claim, source revision, module and callable, relevant configuration, injected condition, expected invariant, and observed outcome. Until the fixture has run, the outcome field remains unmeasured. This restraint prevents source inspection from becoming an invented reproduction. When the experiment does run, a discrepancy should lead to a narrower conclusion: perhaps a configured checkpointer preserves local state, while the selected tool adapter repeats effects. Blaming or endorsing the entire framework would discard the evidence about where the behavior actually arose.

The ledger also identifies work needed for upgrades. If a release changes the callable responsible for event reduction or tool retries, rerun the fixtures attached to those claims. An unrelated documentation edit need not trigger all experiments. This dependency-based approach makes validation scalable while retaining scientific traceability. It is especially useful for agent systems because a small change to error serialization can alter the model's next action even when the application's visible API remains compatible.

Chapter 23

Repository, Terminal, and Computer-Use Agents

TestJack's October 2026 audit constructs prompt-grounded witness tests against candidates that passed existing coding evaluators; AgentHorizon separately examines computer-use evaluation with task instructions and execution trajectories [231, 22]. Both cases challenge the inference from a favorable grader response to completed work. They expose different observation gaps: a test suite may omit a stated requirement, while an interface snapshot may omit the action history needed to interpret the final state. Neither audit provides an infallible judge.

Coding agents operate on artifacts whose correctness is partly executable, but executable does not mean fully specified. A patch can satisfy the visible tests while violating a stated requirement that those tests omit, introducing a security regression, or modifying the tests themselves. An underspecified requirement instead needs adjudication before disagreement can refute the patch. Computer-use agents add another observation problem: a screenshot can show a plausible final state without proving that the requested object, recipient, or permission was selected. The shared objective is to connect task interpretation, environment interaction, and independently checkable evidence.

The central comparison is between complete configured systems. Model, harness, repository context, tool interface, execution environment, evaluator, and budget all influence results. Product names are insufficient experimental identifiers. This chapter analyzes these boundaries, gives concrete implementation and benchmark anchors, and distinguishes inspectable mechanisms from undocumented hosted behavior.

23.1 Repository state as an executable environment

A repository task is a partially observed transition problem. The agent observes selected files, search results, test outputs, and version-control metadata. It acts through edits, commands, dependency operations, and

sometimes remote services. The true environment includes build tools, generated files, network dependencies, and hidden tests that may not appear in its initial context. Treating the task as text completion discards the execution feedback that makes repository-level work distinctive.

The action interface should preserve artifact identity. A patch request names its base revision and intended file changes; command execution records working directory, environment, exit status, and captured output. The test verdict binds to the resulting artifact digest. If the agent edits code after testing, the verdict must be invalidated or rerun. This mechanical correspondence is separate from whether the tests capture the intended behavior.

An execution attestation can make that boundary explicit. Let A be the tested artifact, D its declared dependency/environment manifest, and $e = (a, d, q, x, \tau)$ record their digests, the exact test invocation q, exit status x, and execution trace τ. With $\kappa(\tau) = 1$ denoting a complete, attributable invocation record, define

$$v_e(A, D) = \mathbf{1}\{a = h(A)\}\mathbf{1}\{d = h(D)\}\mathbf{1}\{x = 0\}\mathbf{1}\{\kappa(\tau) = 1\}. \tag{23.1}$$

This predicate establishes provenance of a zero-exit execution for the declared inputs, assuming collision-resistant hashing and trustworthy records. It does not establish requirement coverage, authentic dependencies beyond the captured manifest, or absence of unrecorded network/environment changes. A failed command followed by a persuasive final message has $v_e = 0$; an edit after testing changes $h(A)$ and invalidates the old attestation. Test discovery failures and a command that executes no relevant tests still require separate validation even when its exit status is zero.

The 2024 SWE-agent study established an influential interface baseline and demonstrates the importance of the agent-computer interface rather than model capability alone [2]. Search, file navigation, editing, and error feedback change how effectively the model can act on repository state. The proper inference is interface sensitivity in the studied setting. It does not establish that one interface dominates across languages, repositories, models, and tool latency. A useful replication preserves the evaluator and workload while varying only the interaction surface.

The implementation anchor is an actual rollout through the published repository: model input construction, command interface, environment stepping, output truncation, and termination. A researcher should locate the callable that executes actions and the adapter that converts its results back into model-visible messages. Silent truncation or error normalization can materially change the policy. Comparing only prompt templates misses those mechanisms.

Repository isolation also matters. An agent should not see another benchmark task's generated artifacts or hidden test outcomes through a shared checkout or cache. The workspace, dependencies, and stateful tools need a reset policy. Reusing a build cache may be legitimate for cost analysis but must not transfer answer-bearing files across tasks. Separate a warm infrastructure cache from persistent semantic memory and report both explicitly.

Figure 23.1 isolates the mechanism and its stated information boundaries.

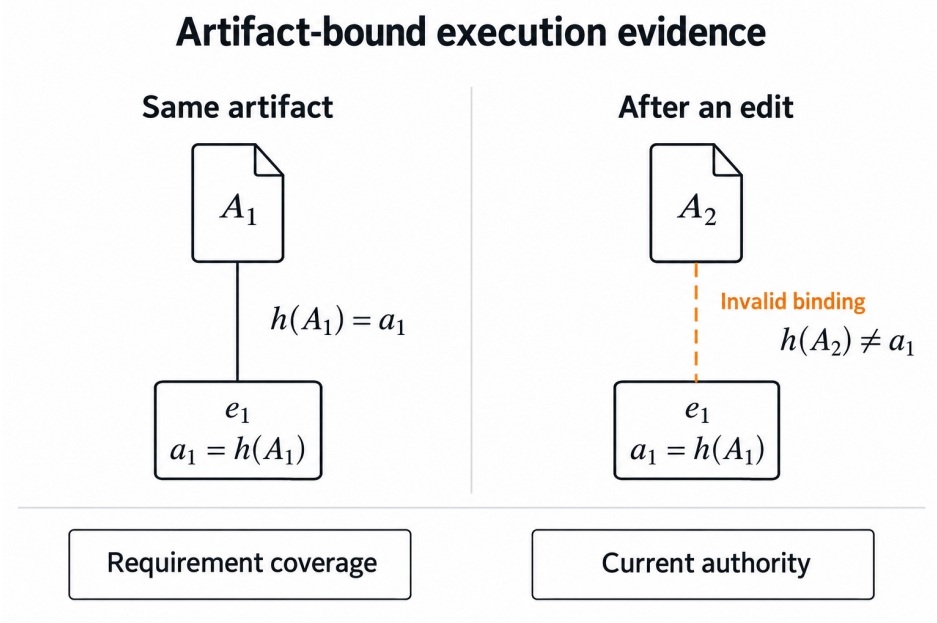

Figure 23.1: The same execution record is checked against two artifact revisions. It binds to the tested artifact in the left panel; the illustrated edit changes the digest and invalidates that binding in the right panel. Matching provenance identifies the object tested, while requirement coverage, the declared environment and current authority need their own checks.

23.2 Specifications tests and evaluator blind spots

Let $\mathcal{R}$ be the intended requirement set and $\mathcal{T}$ the executable test set. Passing $\mathcal{T}$ supports correctness only for the behaviors it covers under the tested environment. It does not imply satisfaction of every element of $\mathcal{R}$. The gap can arise from missing tests, ambiguous requirements,

faulty reference solutions, nondeterministic infrastructure, or a grader that observes the wrong artifact.

A verification gate therefore needs two error rates: false acceptance of a defective artifact and false rejection of a valid artifact. Both depend on the distribution of candidate patches. An evaluator calibrated on ordinary human patches may behave differently on agent-generated solutions that exploit unusual shortcuts. Adaptive optimization against the same visible tests increases this risk because the agent can learn the evaluator's blind spots rather than the task's intended semantics.

For a declared candidate population, let $C \in \{0,1\}$ denote requirement correctness after adjudication and $G \in \{0,1\}$ the grader's acceptance. Write $\pi = P(C = 1)$, $s = P(G = 1 \mid C = 1)$, $f = P(G = 1 \mid C = 0)$, and $q = P(G = 1)$, assuming both correctness classes have positive probability. The law of total probability gives

$$q = s\pi + f(1 - \pi), \qquad \pi = \frac{q - f}{s - f} \quad (s \neq f). \tag{23.2}$$

Thus accepted fraction is not identified with correct fraction. If $s = f$, this grader's acceptance carries no information about correctness prevalence in that population. The same decomposition yields $|q - \pi| \leq \max\{f, 1 - s\}$. These are population identities, not a correction obtained by importing a false-acceptance rate from a different benchmark. Estimation requires an independently adjudicated sample of the same candidate process, with uncertainty propagated jointly through q, s, f; a small $|s-f|$ makes inversion unstable. Changing agents, search budgets or visible-test exposure can change both conditional error rates. Stress fixtures establish sensitivity to their seeded defects and need not estimate the natural population rates.

SWE-Bench Pro increases task complexity and uses public, held-out, and commercial repository groups [232]. Its access structure is important evidence about reproducibility: independently auditing public tasks is not equivalent to reproducing results on unavailable proprietary tasks. A contamination-resistant design also does not prove absence of prior exposure. The survey should preserve those distinctions when comparing numbers across benchmark versions and scaffolds.

A concrete failure case is a patch that handles the sample input but fails on a boundary condition implied by the specification. Another modifies a serialization path correctly while breaking a related caller absent from the visible tests. A third passes locally but depends on an undeclared environment variable. These cases require different interventions: requirement-derived tests, integration coverage, and environment capture. More chain-of-thought tokens do not directly repair an evaluator that never

checks the failing behavior.

Reference implementations are useful but fallible. A differential test that distinguishes a candidate from a reference detects disagreement; it establishes a defect only if the reference behavior follows the requirement. Human or model interpretation of that requirement can still err. Report the adjudication procedure and unresolved ambiguity rather than treating reference agreement as mathematical truth. Chapter 25 develops this verification boundary further.

23.3 Terminal trajectories and learned execution policies

Terminal agents expose a broad action space through a compact interface. The same shell can inspect files, run tests, modify dependencies, and invoke network tools. This flexibility reduces the number of specialized APIs needed, but it makes semantic validation harder. A syntactically valid command can have effects far beyond the intended task. The sandbox and permission layer must mediate those effects independently of the model's stated intent.

Long terminal trajectories contain more than command examples. They include evidence gathering, hypothesis revision, failed attempts, recovery, and final artifact validation. TerminalHorizon explicitly studies synthesis of task environments that preserve the work needed for these behaviors [233]. Its environment-reconstruction mechanism is relevant because a synthetic setup can accidentally solve the difficult part before the agent begins. A training example then rewards superficial completion rather than the intended exploration.

The current v2 separates a task package into public instruction and environment and private reference solution, verifier and construction configuration. Its independent challenger is uninvolved in construction and solves from the public interface; trace-based diagnosis then guides reconstruction of work accidentally simplified during setup. Progressive expansion adds requirements under the same goal that depend on earlier results and feedback [233]. This is more specific than increasing a maximum turn count. Challenger success alone does not certify absence of every shortcut, and generated verifier agreement remains conditional on its task specification. The appropriate ablation changes reconstruction or expansion separately while preserving model, training tokens and evaluation access, and reports both task coverage and the work demonstrably left for the learner.

The causal comparison should distinguish richer trajectories from easier

environments. Keep initial information, required work, and evaluator access comparable across data-generation variants. Audit whether task descriptions leak a solution, whether setup scripts precompute key artifacts, and whether generated tests reveal hidden requirements. Supervised fine-tuning gains on multiple benchmarks can be informative while leaving these construction confounds unresolved.

Trajectory learning also depends on actual behavior conditioning. If a training pipeline reconstructs context after the fact and removes information the policy originally saw, logged actions no longer correspond to the assumed policy inputs. MemoryWalker addresses this class of mismatch [23]. For coding agents, record context edits, tool-output truncation, and environment version along with actions. A repository checkout and final patch are insufficient to reconstruct the decision process.

Evaluation should report progress through the full task rather than command validity alone. Measure artifact success, constraints, execution cost, recovery, and whether the final output identifies tested and untested behavior. A terminal agent that produces a plausible explanation after failing to run tests should not receive the same completion status as one with verified results. Conversely, an infrastructure outage should remain visible as an environmental failure rather than be silently removed from the denominator.

The inspected mini-swe-agent release 2.4.6, dated 23 July 2026, explicitly counts billed calls that fail parsing against the cost limit[234]. This small implementation detail materially changes budget-matched evaluation: excluding paid malformed responses would give a system additional effective resources while reporting the same nominal limit. It is a concrete example of why a minimal loop remains useful as a research baseline but still requires release-level inspection. The relevant experiment compares identical malformed-output schedules before and after the accounting change, measuring budget overshoot and completed work rather than inferring a reasoning improvement from the release note.

The pinned implementation confirms a further boundary. `DefaultAgent.query` checks already accumulated cost before the next model call; successful calls add returned cost, while the `FormatError` recovery branch adds reported billed cost from the exception record [235]. This is post-charge accounting, not the reservation protocol of Chapter 20. For one serial query loop, assume admission and cost updates are serialized, at most one charge remains unrecorded in flight, and each charge is recorded before admitting the next call. If $B > 0$ is the active threshold, the initial total is below B, and each newly admitted call costs at most $c_{\max}$, the pre-call check bounds the charged total by $B + c_{\max}$, rather than by B. Parallel

callers can all pass a stale total, so the one-call bound is not a global concurrency guarantee. Without a known call-cost ceiling there is no finite overshoot bound from that check alone. Tool fees and environment charges outside this counter require separate accounting. A reproducible budget comparison must therefore distinguish a declared stopping threshold from a guaranteed maximum expenditure.

23.4 Computer use grounding and hidden state

Computer-use agents act through visual observations and input events. The policy must connect screen coordinates or semantic targets to application state, often across scrolling, asynchronous updates, and multiple windows. A click can be delivered successfully to the wrong object. Therefore transport acknowledgment and task correctness are especially far apart. The environment should record both intended target and observed result where the interface permits it.

Coordinate systems require explicit transformations. Let $x_m \in \mathbb{R}^2$ be a point in the image presented to the model, p its letterbox offset, $w, h > 0$ the rendered content dimensions, and $w_c, h_c > 0$ the corresponding crop dimensions in physical screenshot pixels. If c is the crop offset inside a screenshot whose desktop origin is o, the adapter must apply

$$x_{\text{desktop}} = o + c + \operatorname{diag}(w_c/w, h_c/h)(x_m - p). \tag{23.3}$$

The expression assumes axis-aligned resizing without rotation and one physical-pixel coordinate system; CSS coordinates or device-independent units require an additional declared transform. Points outside the content rectangle should be rejected rather than clamped onto an unintended control. A deterministic mapping defect can dominate performance while being misdiagnosed as poor visual reasoning. Tests should perturb resolution, crop, letterboxing, viewport and display scaling independently. Even a correct map becomes stale if the application moves the target between observation and dispatch, so the observation identity and permitted delay remain part of the action contract.

There is a useful sufficient condition for geometric targeting, with deliberately limited scope. Let $A = \operatorname{diag}(w_c/w, h_c/h)$, suppose the model-space localization error is at most ϵ_m, and bound the remaining calibration residual on the relevant image region by ϵ_c. During delay Δt, assume the target region undergoes only a rigid translation of norm at most $v\Delta t$. If the intended desktop point has interior margin r to the target's complement at

observation time, then

$$\|A\|_2\epsilon_m + \epsilon_c + v\Delta t < r \tag{23.4}$$

ensures the dispatched point remains inside the translated target region: the triangle inequality bounds its displacement relative to that region by the left-hand side. This is an analytical interface condition, not an estimated guarantee for a vision model. Reflow, disappearance, occlusion, disabled controls, focus changes and a wrong semantic target invalidate or fall outside the hypotheses. Even an inside-region click still requires outcome verification. The condition explains why timestamped observations, calibration tests and a new screenshot after layout changes can address distinct failure mechanisms.

Screenshots are incomplete observations. They may omit hidden cells, collapsed panels, off-screen recipients, background downloads, or permission dialogs. A final screenshot showing a document title does not establish that the document contains the required data. Agents need application-specific verification or additional observations. When APIs are available, they can provide stronger state evidence, but the evaluation must declare whether the agent or only the evaluator can use them.

AgentHorizon evaluates judges on instruction-trajectory pairs, including closely related instructions whose trajectories are not interchangeable [22]. This paired design exposes a common failure: accepting a trajectory because it looks like successful work while missing a constraint that distinguishes the actual request. The result concerns judge reliability, not merely executor capability. It motivates storing sufficient history for verification while also testing whether a judge can locate the relevant evidence.

An experiment should separate perception, target selection, action delivery, and outcome verification. Oracle target coordinates quantify perception headroom; replayed actions quantify environment determinism; authoritative state checks audit the final judge. The oracle conditions are diagnostic controls, not deployable capabilities. Report how much each stage contributes to failure and how those contributions change with task horizon. A single end-to-end score cannot identify which interface should be improved.

23.5 Coding product families and observable tests

Table 23.1 places coding, terminal, editor, and application-building systems into one comparison program without treating them as equivalent. A product may span local and remote execution, and an exposed client does not make its entire service inspectable. The official surfaces were rechecked on

29 October 2026; experimental claims require the corresponding executable configuration and evidence. Retrieval failure on a product homepage is not a capability verdict: Lovable's official documentation establishes its application-building and code-export surfaces, while the resolved Antigravity 2.0 announcement distinguishes a standalone agent application from its separate IDE [236, 237].

Table 23.1: Coding and computer-work product surfaces and diagnostic tests.

Product	Inspected role	Diagnostic condition
Codex[238]	Open harness and SDK boundaries	Bind final artifacts to tested revisions; separate local execution from hosted service state.
OpenHands[239]	Self-hosted software-agent workflows	Inspect sandbox reset and replay after environment loss.
goose[240]	Local extensible tool automation	Audit extension authority and effects outside the workspace.
Warp[241]	Terminal and cloud development surfaces	Compare local state with remotely delegated job state at interruption.
Kilo[242]	Multi-model coding interfaces	Hold the selected model fixed before attributing gains to orchestration.
OpenCode[243]	Extensible coding runtime	Change tool schemas and test stored session compatibility.
Letta Code / Letta harness[244]	Stateful coding and editable memory	Revoke an old instruction and test its influence on resumed work.
Claude Code[245]	Terminal coding and developer integration	Verify permission enforcement independently of model-generated intent.
GitHub Copilot[246]	Repository editor and delegated coding surfaces	Distinguish IDE suggestions from autonomous remote task execution.
Cline[247]	Configurable local coding automation	Inject a failing command and inspect confirmation and retry behavior.
Zed Agent[248]	Editor with native and external agents	Test unsaved buffers, external-agent context and artifact identity.
Cursor[249]	Editor and cloud coding interfaces	Audit which project state reaches background workers and their final merge.

Product	Inspected role	Diagnostic condition
Sourcegraph Agentic Batch Changes[250]	Batch repository-change workflow	Measure migration coverage and per-repository regressions rather than one patch score.
Hermes Agent[251]	Persistent agent with memory and skills	Separate stored procedures from learned weight updates and test stale-skill effects.
Mistral Vibe[252]	Coding CLI and work interfaces	Identify local command mediation and managed conversation boundaries.
Devin[253]	Delegated engineering environments	Verify reproducible environment setup and independently test the delivered patch.
Factory Droids[254]	Local and cloud engineering work	Track workspace reuse, credential scope and review artifact provenance.
Amp[255]	Persistent coding threads	Interrupt work after tool dispatch and inspect repeated effects on resumption.
Antigravity[237]	Standalone agent app and separate IDE	Test project-scoped context, subagents and permissions under the selected product surface.
Qwen Code[256]	Coding runtime with selectable model adapters	Match model and tool serialization before comparing repository results.
mini-swe-agent[257]	Minimal research coding loop	Use as a low-complexity baseline with identical evaluator and budget.
Junie[258]	IDE, CLI and delegated coding workflow	Separate test execution evidence from generated explanations of correctness.
Harness Cosmos / Auggie[259]	Enterprise code-context tooling	Provider lineage does not prove shared implementation; test repository-context boundaries.
Kiro[260]	Specification-oriented development	Perturb a requirement after planning and test invalidation of dependent work.
Rovo Dev[261]	Issue-context engineering workflow	Verify that ticket permissions and repository write permissions remain distinct.
Replit Agent[262]	Application construction and deployment	Validate deployed behavior and environment configuration, not preview appearance alone.

Product	Inspected role	Diagnostic condition
MiniMax Code[263]	Persistent mini-app and task surfaces	Test background-task lifecycle and whether saved artifacts match verified output.
Tabnine Agent / Plugin for OpenCode[264, 265]	Enterprise coding distribution	Verify selected binary, provider lockdown and tool-governance configuration.
v0[266]	Generated full-stack web applications	Test backend access controls, deployment identity and post-generation regressions.
Lovable[236]	Full-stack app building and code export	Bind preview, exported code and published deployment to the same reviewed artifact.
CodeBuddy[267]	Subagent interface documentation	Test isolated contexts, inherited permissions and integration of conflicting edits.
TRAE / SOLO[268]	IDE and standalone task agent	Separate interactive assistance from unattended completion under the same task contract.
ZCode[269]	Desktop coding with tool interfaces	Audit memory persistence and browser authority separately from coding quality.

The important mechanisms cut across these families. Editor integration can reduce navigation cost and expose project context, but it can also hide unsaved state or privileged extensions. Cloud delegation can provide isolated environments and parallel work, but adds environment reconstruction and artifact-transfer boundaries. Application builders can integrate deployment tightly, but task success must then include the deployed artifact and its operational configuration rather than only a generated preview.

Codex's documented SDK and open harness interfaces provide concrete inspection boundaries, while its hosted application and services remain separate components [238]. Claude Code's public repository and SDK documentation likewise require artifact-specific interpretation [245, 199]. No claim of complete implementation visibility follows from a public issue tracker or client repository. For both families, experiments must identify what runs locally, what runs remotely, and where state and authority are enforced.

The current inventory also contains consequential context-lifecycle distinctions. Letta Code's pinned README describes memory, skills, prompts and harness modifications as adaptation surfaces; its cloud backend manages bundled default prompts on resume unless the documented preservation

option is selected. That is a configuration transition requiring capture, not evidence of a fixed policy or autonomous weight improvement [270]. Qwen Code exposes multiple provider protocols and SDK/desktop/browser interfaces, so treating every Qwen Code run as one Qwen model is incorrect [256]. Tabnine's OpenCode distribution supplies its own managed binary, Tabnine-served model selection and organization controls; sharing an underlying agent family does not establish equivalence to a separately configured OpenCode installation [265]. Provider lineage and product packaging must remain distinct from the manifest of an evaluated run.

In the pinned Codex TypeScript SDK, `CodexExec.run` spawns the CLI with a JSON event stream, and `Thread.run` reduces completed-item events into a buffered return value. A turn-failure event causes that buffered path to throw; completed items accumulated earlier are not returned as a successful `Turn`. The streaming path exposes events as they arrive [271]. An application needing partial-progress recovery should durably record relevant events and artifact identities while streaming, rather than infer that a thrown turn produced no work. Neither stream persistence nor a completed-item event by itself certifies the absence of unresolved remote effects. This is a source-level distinction between event delivery and task completion, not an independently measured failure rate.

23.6 A budget-matched evaluation protocol

The evaluation unit is a task plus initial environment, permitted resources, and scoring contract. Pin repository commit, dependency state, tool versions, model identity where available, and product configuration. If a hosted product does not expose a stable model or harness revision, record the retrieval and execution dates and restrict conclusions to the observed service. A nominally repeated experiment can change when the provider updates an opaque component.

Use task-level paired comparisons. Each system receives the same problem and a reset environment, with task order randomized where persistent memory is enabled. Declare whether memory may transfer between repositories or projects. A system with privileged workspace history has a different information budget from a fresh baseline. This difference may be a legitimate deployment advantage, but it must not be misrepresented as an intrinsic model gain.

Resource accounting includes input and output tokens, tool fees, environment startup, waiting, retries, verifier work, and human intervention. Report elapsed time and monetary cost separately. A cloud product can

exploit parallel resources unavailable to a local baseline; matching elapsed time alone then compares different total compute. Conversely, matching dollars may permit different wall-clock and environment availability. The comparison axes should precede any ranking.

Ablations should target interfaces as well as prompts. Compare full repository search with restricted retrieval, executable feedback with model-only critique, persistent workspace with fresh workspace, and independent samples with coordinated subagents. Preserve test access and scoring across conditions. Where a product does not permit a component ablation, report the complete configured treatment rather than inventing an internal causal explanation.

Failure analysis should retain invalid calls, unexecuted tests, timeouts, partial patches, and inaccessible tasks. Distinguish infrastructure failures from policy errors without removing either from overall operational reliability. Confidence intervals should cluster repeated attempts by task and repository where applicable. If the sample is too small to distinguish adjacent products, report that uncertainty rather than converting a noisy ordering into a purchasing recommendation.

23.7 Artifact authority and reviewable completion

A coding agent's final message should identify the artifact, the checks actually executed, their outcomes, and material limitations. This is not merely presentation quality. It gives the recipient the evidence needed to distinguish a proposed solution from a validated change. A confident statement that tests pass is not evidence if the trace records a skipped or failed command. The harness should generate factual execution status from authoritative records whenever possible.

Reviewable completion requires a stable artifact boundary. In version control, that can be a commit digest or patch against a known base. In a browser-created application, it can be an exported bundle and deployment identifier. In a computer-use task, it can be an authoritative application record with its permission state. The boundary varies, but it must allow the verifier to inspect the same object the agent claims to have completed.

Human review is an additional control, not an oracle. Reviewers have limited time, may anchor on the agent's explanation, and may not inspect every dependency or generated file. The agent should therefore expose high-risk changes and missing evidence rather than bury them in a long narrative. Evaluation can measure whether reviewers detect seeded defects

under a fixed review budget. It should not claim safety merely because a workflow contains an approval button.

The strongest engineering design binds authority to the transition being approved. Approval of a patch does not authorize later unrelated edits; approval to prepare a deployment does not necessarily authorize publication. If the artifact changes after review, the approval's dependency digest should fail validation. A resumed session must recheck current permissions even if its history contains an earlier grant. These conditions connect coding agents to the broader authorization analysis of Chapter 29.

The practical conclusion is that coding and computer-use competence is a property of an evidence-producing execution system. Better models expand what can be attempted. Better interfaces reduce avoidable friction. Stronger evaluators identify a larger subset of defects. Durable state and authority checks keep long tasks coherent. Each layer has an identifiable contribution and a residual failure surface. Scientific comparison requires preserving those layers rather than attributing every successful artifact to an undifferentiated agent product.

23.8 A concrete grading failure and its discriminating controls

Consider a task requiring a report to be saved in a particular project folder without changing its sharing settings. A computer-use trajectory creates a report with the correct title and visible content, but saves it in a similarly named personal folder. A judge that examines only the final screenshot may accept it. The error is not necessarily poor visual recognition: the relevant distinction may be absent from that screenshot. A stronger model given the same incomplete evidence cannot reliably infer the missing location.

The first control supplies an authoritative folder identifier to the judge while leaving the executor unchanged. If acceptance errors disappear, the observation interface is implicated. The second control provides the full trajectory, including the save dialog, but no authoritative identifier. This tests whether historical evidence is sufficient and whether the judge can retrieve it. The third swaps the instruction for a closely related request that would be satisfied by the observed trajectory. This tests whether the judge checks instruction-specific constraints rather than rewarding generic task plausibility. AgentHorizon's paired instruction design directly motivates this distinction [22].

An analogous repository failure occurs when an agent changes a public function's return format while preserving the sample command's output.

Unit tests focused on the sample pass, but a downstream caller breaks. Supplying more model-generated critique may or may not find the defect. A requirement-derived integration test provides a different kind of evidence because it executes the affected contract. Yet the test can still be wrong if it encodes an interpretation not required by the task. The evaluation must therefore retain both the executable failure and its requirement justification.

Causal diagnosis records the full execution and scoring chain. The task contains a constraint; the executor either overlooks it or lacks an adequate observation; the artifact violates it; the grader fails to inspect the decisive evidence; the final score reports success. Intervening only on the last step corrects the benchmark label but does not improve the executor. Intervening on the executor can improve artifacts while leaving the grader vulnerable to future failure classes. These are separate research contributions and should not share one undifferentiated reliability claim.

The implementation ledger for this case identifies the task adapter, initial state constructor, action executor, trace serializer, grader input builder, and acceptance function. At each boundary, record whether the decisive folder identifier or function contract remains available. A missing field is a concrete engineering finding. A present field ignored by the judge is a model or prompting finding. A correct judge verdict overwritten by a scoring adapter is an evaluation implementation finding. Locating the boundary prevents expensive model changes from being used to compensate for a deterministic data-loss defect.

The proposed experiment reports false acceptance, false rejection, evidence sufficiency, and evaluator cost under each control. Tasks are paired by instruction family, so uncertainty must preserve that pairing. Seeded defects help establish diagnostic sensitivity but do not estimate their natural deployment prevalence. Real trajectory audits complement them by identifying failures the fixture designer did not anticipate. The combination supplies a stronger basis for improvement than either a large leaderboard score or a handful of impressive demonstrations.

Deployment also requires a distributional statement. Folder-confusion fixtures test sensitivity to deliberately ambiguous names rather than overall product quality. A low frequency in a small natural sample does not exclude the failure. Combine stress-test sensitivity with an uncertain estimate of deployment exposure. When an error changes permissions or publishes an artifact, its tail consequence can dominate despite high average completion.

Chapter 24

Research, Enterprise, and Domain-Specific Agent Systems

The 2026 Knowledge Activation study and ALE locate two different evidence boundaries. The former reports engineers' perceived benefits in a particular organizational study; the latter constructs professional task evaluations whose measured outcomes do not directly identify enterprise productivity or economy-wide value [146, 21]. Treating either as proof that a named product improves every knowledge workflow would exceed its design. This chapter uses those boundaries to connect source-grounded research, permissioned business effects, memory services, and the deployment experiments required to establish value.

Research and enterprise agents share an evidence problem but differ in what their outputs authorize. A research agent must connect conclusions to sources and preserve unresolved disagreement. An enterprise agent may use those conclusions to change a customer record, create a legal work product, or trigger a business process. The latter adds identity, permissions, transactional effects, and organizational accountability. A fluent report is insufficient evidence for either role, and a well-grounded report does not automatically authorize the actions it recommends.

The objective of this chapter is a role-based account of these systems, including memory services that supply their context. Product documentation identifies integration and execution surfaces available for inspection; it does not establish scientific validity, deployment benefit, or universal capability. The analysis therefore connects each family to a concrete failure mechanism and a discriminating evaluation.

24.1 Research as evidence acquisition and revision

A research workflow contains query construction, source acquisition, identity resolution, claim extraction, comparison, synthesis, and revision. Each stage can fail independently. A source can be relevant but outdated, a claim can be extracted accurately but applied to the wrong population, or a synthesis can cite a paper that never measured the asserted outcome.

Counting citations or generated pages does not distinguish these errors.

Represent a claim as a tuple containing proposition, scope, evidence class, source version, result location, assumptions, and unresolved limitations. An evidence graph links claims to supporting or contradicting observations. This representation makes revision possible: if a paper changes its result, the system can identify dependent conclusions rather than rewrite every paragraph blindly. It also separates author-reported evidence from independent reproduction and analytical deduction.

The query policy should seek disconfirming evidence as well as support. A system rewarded for completing a coherent report can stop after finding sources that fit its initial hypothesis. A better evaluation includes questions where the decisive result is negative, where two studies use incompatible definitions, or where the available implementation cannot substantiate a headline claim. The correct output may be a narrower conclusion or an explicit unresolved state rather than a more confident answer.

Hosted research products and inspectable research harnesses occupy different reproducibility boundaries. DeerFlow exposes a research-oriented harness, while Asta's public resources distinguish research implementations from the hosted service [272, 273]. Consequently, a result obtained with a published baseline cannot be attributed to every hosted variant, and a hosted demonstration does not reproduce the baseline. Evaluation must name the actual execution path and available tools.

The stopping condition is evidence sufficiency under a declared question, not exhaustion of a token budget or attainment of a target report length. An implementation can maintain unresolved claim categories and a bounded search frontier. It should report whether searches failed because no qualifying evidence was found, because a source was inaccessible, or because the budget ended. These states have different scientific implications and should not collapse into a single statement that evidence is absent.

24.1.1 Biological agents: executable methods and empirical feedback

The October 2026 biological frontier spans several distinct control loops. Paper2Agent converts papers and associated code into MCP interfaces and checks their executable behavior; its September Nature article demonstrates genomic and transcriptomic cases while retaining human responsibility for open-ended interpretation [274]. WetRobo's September report instead studies coding-agent adaptation of a laboratory robot kit on three equipment-manipulation tasks, including bottle-cap transfer between two laboratories [275]. Neither result alone establishes autonomous experi-

mental discovery: method execution, equipment manipulation, scientific interpretation and selection of new experiments are different tasks with different ground truth.

This distinction determines what memory should retain. Reusing a numerical analysis method requires its data schema, software revision and assumptions. Reusing an equipment procedure additionally requires calibration, object geometry, admissible operating conditions and an execution monitor. Reusing a scientific conclusion requires evidence about the sampled biological system and measurement process. A skill that retains only successful command text can erase all three applicability boundaries. Model improvement does not repair a missing calibration record or turn an association into a validated mechanism.

The inspected Paper2Agent revision makes a limited verification boundary concrete. Its `validate_cases` routine can require positive acceptance cases for every expected tool; `check_subset` compares returned structures with declared expected values; `check_artifacts` checks returned artifact paths and identity conditions. Those functions support interface and smoke-case checks, not independent validation of a biological interpretation. The source was inspected at revision `8c2d059165ef`; no scientific analysis was executed here [276]. Generated tests inherit the assumptions of their expected outputs. An executable wrapper can therefore improve method accessibility while faithfully automating an invalid analysis; interface conformance and scientific validity must remain separate claims.

Figure 24.1 locates the evidence and execution boundaries in a conceptual scientific-agent loop.

24.1.2 An October closed-loop result and its attribution limits

Cutts and colleagues report three agent-guided aptamer design–test–improve rounds in October 2026 [277]. The myoglobin results in Table 24.1 are nonmonotonic across rounds. Human researchers performed the laboratory workflow, and the agent platform changed after the first round. The study therefore provides empirical evidence of iterative laboratory feedback and failure diagnosis, while its authors explicitly reserve matched-budget no-feedback and non-reasoning controls for future work. These observations do not isolate the causal contribution of agentic reasoning from harness changes, prior-round seeds or accumulated assay information.

The nonmonotonic middle round is analytically important. Optimizing an assay proxy can concentrate candidates in a region with favorable background interactions rather than the desired biological property. A

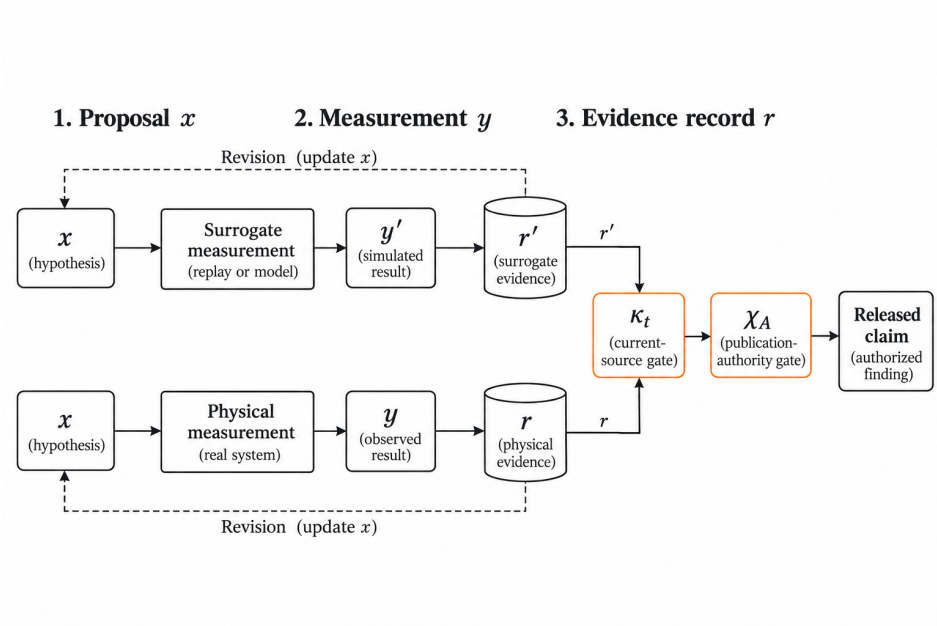

Figure 24.1: Only records already accepted by the declared verifier enter the current-source validity and publication-authority gates. A released surrogate or replay finding retains its backend and recording provenance; it is not fresh physical evidence. Closing either loop does not identify a causal improvement or establish unrestricted laboratory autonomy.

Table 24.1: Author-reported bead-corrected myoglobin hits among newly designed primary candidates. Candidate populations differ across rounds; these are campaign observations, not randomized treatment-effect estimates. Source: Results R.2 of Cutts et al. [277].

Round	Hits	Candidates	Hit rate (%)
1	20	196	10.2
2	9	244	3.7
3	184	549	33.5

memory writer that records only the apparent successful motif can promote the shortcut into a procedural prior. A writer that retains target and background evidence, candidate lineage, analysis revision and uncertainty makes the lesson revisable. This is the biological counterpart of a coding agent memorizing a test-specific patch: a higher proxy score can accompany weaker alignment with the actual objective.

For a transparent analysis, consider the illustrative measurement model

$$y_{rk} = f(x_{rk}, e_r) + b_{p(r,k)} + \varepsilon_{rk}, \tag{24.1}$$

$$\mathbb{E}[\varepsilon_{rk} \mid x_{rk}, e_r, p(r,k)] = 0. \tag{24.2}$$

Here x_{rk} is candidate k in round r, e_r the declared assay conditions, $p(r,k)$ its measurement batch, b_p a batch effect, and y_{rk} the observed scalar response; all additive terms have the response's units. This is an analytical model, not a fitted model of the cited experiment. If candidate strategy changes only between batches, differences in f and b_p are not separately identified without additional design or assumptions. Randomized allocation within batches, recurring controls, replicated candidates and independent confirmation can address different parts of that ambiguity. Repeated reads of one construct are measurement replication; they do not create independent evidence about unrelated molecular families.

An agent-design comparison must therefore randomize complete campaigns or sufficiently isolated candidate allocations, preserve assay conditions and measurement budgets, and predeclare which feedback is available before each proposal. Include conventional sequential-design and no-feedback controls, not only a weak language-model prompt. Hold out candidate families or experimental regimes when the claim concerns transfer. Evaluate the selected designs and the learned predictor separately: finding a promising candidate and predicting unseen candidates are different achievements. Costs include physical measurements, waiting, failed jobs, analyst review and computational selection. Token expenditure alone is an inadequate resource boundary when empirical feedback dominates elapsed time.

24.1.3 Replay laboratories, digital twins and live measurements

Autonomous Science Bench, released in October 2026, makes that discovery–learning distinction explicit through two acceptance gates [278]. Version 0.1 contains six campaigns: five return recorded measurements and one uses a hidden surrogate. A live backend and adaptive allocation scheduler

are planned, rather than evaluated results in that release. The laboratory service owns the measurement ledger independently of the agent, and grading combines measured-design quality with prediction on held-out designs. These design choices support controlled evaluation of sequential decisions; they do not constitute six live autonomous laboratory demonstrations.

The backend is part of the scientific claim. A replay can provide genuine historical measurements while excluding unrecorded designs. A digital twin permits more queries but tests optimization against its approximation and training coverage. A live assay adds drift, failed measurements, delayed returns and limits on reversibility. An interface that makes those backends look identical to the agent does not make their evidence interchangeable. Report backend provenance with every outcome and measure whether an agent exploits surrogate artifacts that fail under independent measurement.

Biological scope also constrains the autonomy claim. The aptamer study uses non-infectious protein targets under its stated laboratory conditions and does not study clinical efficacy, human subjects, or animal models [277]. An assay hit therefore remains an assay-specific observation. Extending a workflow to a different biological system requires its own material, measurement, containment, and approval conditions; preserving an API schema supplies none of those premises. The relevant system boundary includes the people or equipment that accept a proposed experiment and the independent evidence required before a scientific conclusion is promoted.

Delayed results create a further state boundary. Submitted, measured, quality-controlled and scientifically accepted experiments should have distinct identifiers and statuses. An agent must not treat an unreturned measurement as a negative label or spend the same physical budget twice after a timeout. When a result is corrected, dependent hypotheses and skills become stale without erasing the historical reason for their adoption. These requirements connect biological research to the session journal of Chapter 13, the conditioning analysis of Chapter 17 and the recovery contracts of Chapter 32. The cross-domain synthesis concerns preservation of evidence, authority and effect identity; it does not imply that software verifiers and biological assays have equivalent error models.

24.2 Enterprise grounding and permission propagation

Enterprise context is governed by principals, groups, purpose, retention, and changing access policy. Retrieval must enforce these constraints at use time. A document indexed while a user had access may become unauthorized later.

An embedding or extracted summary can preserve sensitive information after the original file is removed. Therefore permission checks cannot be limited to the connector's initial ingestion step.

Let a retrieved record carry source identity d, subject u, validity interval, and authorization epoch a. Before the record enters C_t, the application evaluates whether the current principal may use d for the requested purpose under the current epoch. A cached positive decision is reusable only within the policy's declared validity conditions. The record's textual content cannot grant its own authority. Instructions discovered inside a document remain data unless promoted through an authorized application channel.

Business workflow agents also depend on schema and process state. A customer-service action can be valid for one account tier and invalid for another. A procurement step may require approval after a threshold is crossed. These constraints should be represented in tool preconditions and authoritative business logic. Asking the language model to remember all exceptions from a policy manual creates an avoidable single point of failure.

Platforms such as Agentforce, ServiceNow's agent tooling, SAP Joule Studio, and Copilot Studio document domain-integrated development surfaces [279, 280, 281, 282]. The transferable technical question is how those surfaces preserve identity and process constraints across model decisions, connectors, and human review. Their domain integration does not establish that every generated action is correct, nor that all deployment configurations enforce the same controls.

A discriminating test changes access or process state midway through a session. The agent previously read a document or prepared an action, then the principal loses access or the business object changes. Evaluate whether retrieval, pending approvals, and queued actions are invalidated appropriately. A successful fresh-session test is insufficient because the failure lives in stale context and in-flight work. Report benign task completion alongside containment to expose overblocking as well as unauthorized use.

24.3 Memory engines are distinct policy components

A memory engine can store raw events, extract facts, build a graph, summarize experience, or expose reflection operations. These are distinct mechanisms. Storing more information can improve coverage while increasing retrieval noise and governance burden. A graph can encode temporal relationships without guaranteeing that the selected facts remain valid. A reflection operation can produce a useful abstraction or reinforce a mistaken

interpretation.

Mem0, Graphiti/Zep, Hindsight, MemOS, and OpenViking expose different storage and context-management surfaces [283, 284, 285, 286, 287]. The comparison should identify the unit written, retrieval policy, update semantics, provenance, deletion propagation, and compatibility with the consuming model. A memory product's use of the word learning does not establish changes to model parameters. The actual update target must be inspected.

Temporal memory requires both event time and system knowledge time. An event can occur on one date and be discovered later. A correction can invalidate a prior belief without making the historical observation disappear. A system that stores only the most recent text may lose the explanation for past decisions; one that stores every version without validity filtering may repeatedly retrieve obsolete facts. The application must define which temporal question it is answering.

Memory evaluation also needs a no-memory baseline and an oracle-retrieval control. The former reveals negative transfer; the latter separates storage and retrieval limitations from downstream interpretation. Construction cost belongs inside the accounting boundary. A memory engine can reduce per-task prompt size while spending substantial inference on extraction, consolidation, or maintenance. Amortization depends on reuse and should be reported as a function of workload, not assumed from one benchmark.

Licensing and inspectability remain artifact-specific. Where a repository's license and additional README restrictions appear inconsistent, record the conflict and avoid an unqualified legal classification. The technical consequence is uncertainty about reuse and deployment, not an inference about enforceability. More importantly for scientific comparison, source visibility should not be confused with an audited implementation. A public memory schema can still omit crucial behavior implemented by a remote service.

24.4 Work-product and vertical-system inventory

Table 24.2 completes the catalog with research, business, vertical, and memory systems. It describes the role established by the inspected resource and the evidence needed before accepting a stronger claim. Product families can overlap; the categories organize experimental questions rather than define exclusive market segments. Names and source redirects are preserved without importing prior rankings or unverified ownership assertions.

Table 24.2: Research enterprise and memory products with role-specific checks.

Product	Inspected role	Diagnostic condition
Mem0[283]	Fact-oriented memory layer	Test corrections, source provenance and deletion across derived records.
DeerFlow[272]	Inspectable research harness	Pin search tools and measure contradictory-evidence coverage at matched budget.
Graphiti / Zep[284]	Temporal graph context	Distinguish event time, knowledge time and invalidated facts during retrieval.
watsonx Orchestrate[288]	Enterprise orchestration	Verify connector principal propagation and business-process constraints at dispatch.
Hindsight[285]	Retention recall and reflection	Audit whether reflection adds unsupported inferences or preserves source links.
Agentforce[279]	CRM-grounded business agents	Change account state mid-run and test action preconditions and approval binding.
AI Agent Studio / Otto AI Agents[280]	Service-workflow agent tooling	Distinguish workflow completion from successful resolution and absence of adverse effects.
ChatGPT Work[289]	Research and connected work artifacts	Audit claim support and the exact saved artifact; hosted internals remain opaque.
Claude: work and Research[290]	Research and file-oriented work	Separate merged product naming from evidence about session and permission semantics.
AI by Zapier[291]	Cross-application adaptive automation	Inspect migration behavior and whether connector grants remain purpose scoped.
Asta[273]	Scientific search and analysis	Distinguish public research baselines from the hosted implementation and its tool access.
Cortex Agents[292]	Structured and unstructured data agents	Verify generated query authority and source-level access enforcement.

Product	Inspected role	Diagnostic condition
Gemini Deep Research / Deep Research Max[293]	Hosted multi-step research	Record preview or deployment tier and audit source coverage rather than report length.
Glean Agents[294]	Company-knowledge and action agents	Revoke document access after ingestion and test retrieval and generated summaries.
Copilot Studio[282]	Low-code business agents	Bind human approvals to exact tool arguments and current identity.
Manus[295]	Sandboxed general work	Test artifact persistence and unresolved external effects after an interrupted task.
Perplexity Computer[296]	Research-to-work interfaces	Measure transition from cited conclusions to correctly executed deliverables.
Rovo Agents[297]	Workplace knowledge and workflow agents	Check project boundaries and whether inherited permissions survive connector changes.
North / Agent Studio[298]	Enterprise agent construction	Treat private deployment and data isolation as configuration-specific claims requiring evidence.
Notion Custom Agents[299]	Scheduled workspace workflows	Inject duplicate triggers and inspect idempotency, current permissions and retention.
Joule Studio[281]	Business skills and SAP context	Test schema evolution and process-state constraints rather than conversational plausibility.
MemOS[286]	Composable memory interfaces	Identify actual update targets and distinguish memory writes from parameter learning.
OpenViking[287]	Context filesystem and retrieval	Inspect license artifact and additional restrictions separately; test deletion propagation.
Dust[300]	Company-context custom agents	Audit tool scope and evidence provenance across connected knowledge stores.
Kimi Agent[301]	Research analysis and work artifacts	Verify calculations and source support independently of presentation quality.

Product	Inspected role	Diagnostic condition
Harvey Agents / Agent Builder[302]	Legal workflow agents	Test jurisdiction, document versions, confidentiality and reviewer detection of seeded errors.
Sierra[303]	Customer-service agents	Measure policy compliance, resolution and escalation together with harmful-action rates.
WRITER Agent[304]	Enterprise playbooks and governance	Test whether reusable procedures remain valid after policy and model changes.
Grok Bot[305]	Persistent work-computer surface	Inspect identity, cloud-state lifetime and cancellation of background effects.
Copilot Researcher[306]	Web and enterprise research	Separate multi-model agreement from independent source verification.
Qianfan Agent Platform[307]	Regional agent-platform identity lead	Detailed mechanisms unresolved after source retrieval failure; no capability inference.
Genspark Super Agent[308]	Hosted research and deliverable creation	Check citation support, computation provenance and the final exported artifact.

The unresolved Qianfan source is included as an identity lead with unknown detailed behavior. This is more informative than silently omitting a regional product or guessing its controls from a neighboring offering. The same principle applies to feature parity across regions, deployment tiers, and private-cloud options: absence of inspected evidence constrains the claim. It does not establish absence of the feature.

Legal and customer-service systems require domain-specific outcomes. A legal workflow should be evaluated for source validity, jurisdiction, document consistency, review burden, and unauthorized disclosure. A support agent should be evaluated for policy compliance, resolution, escalation, and adverse actions, not only conversational satisfaction. A general benchmark can supply useful stress tests, but it cannot substitute for the domain's operational specification.

24.5 Causal evidence for organizational benefit

Organizational benefit is not directly measured by model benchmark accuracy. An agent can produce useful drafts while increasing review effort, or reduce time on routine work while creating rare costly errors. Deployment studies must define the outcome and the accounting boundary. Candidate outcomes include verified task completion, time to accepted artifact, reviewer effort, rework, error severity, and downstream process effects.

The Knowledge Activation paper proposes structured institutional knowledge units and reports an enterprise case study [146]. Its developer survey supports claims about perceived experience in that setting. Self-reported time savings are not instrumented productivity measurements, and concurrent model or tooling changes complicate causal attribution. The useful architectural contribution can therefore be discussed without promoting the case study into a universal magnitude claim.

A stronger study randomizes eligible work units or uses a carefully justified staggered rollout. The unit of assignment should minimize interference: engineers sharing skills and repositories can transfer treatment effects to nominal controls. Measure baseline workload, task difficulty, and prior tool exposure. Record adoption and noncompliance, because assigning an agent does not guarantee that workers use it. An intention-to-treat analysis and a usage-conditioned analysis answer different questions and should not be conflated.

Learning effects require longitudinal observation. Early users may spend extra time configuring context and tools; later users may benefit from accumulated skills. Conversely, initial novelty can inflate perceived benefit while maintenance costs appear later. Report trajectories rather than only a single post-deployment mean. Include unsuccessful and abandoned tasks, since excluding them can make a system appear faster by conditioning on completion.

External validity depends on organizational knowledge quality. A system grounded in accurate, maintained procedures may perform differently from one grounded in contradictory documentation. The treatment therefore includes knowledge engineering and governance, not merely an agent interface. An ablation can hold the model and harness fixed while varying institutional context quality. This identifies whether observed gains come from the policy, better information, or their interaction.

24.6 Professional benchmarks and economic interpretation

Professional-task benchmarks attempt to move evaluation beyond short questions into deliverables with multiple constraints. Their relevance is substantial, but the mapping to economic impact requires care. A benchmark task can resemble professional work while omitting coordination, accountability, access friction, and consequences of failure. A successful artifact is evidence about the tested task, not a direct estimate of organizational value or labor substitution.

Agents' Last Exam introduces long-horizon professional tasks with verifiable outcomes across occupational categories [21]. Its living-benchmark design requires exact task-pool and scoring snapshots for longitudinal comparison. An improved score can reflect a changed task mix or grader as well as a stronger system. Reports should retain the task version, evaluator identity, available tools, and artifact requirements rather than cite a moving leaderboard alone.

Economic interpretation also depends on selection. Benchmarks often choose tasks with accessible data and checkable outcomes. Work requiring tacit negotiation, physical access, or responsibility allocation may be underrepresented. This selection does not invalidate the benchmark; it limits the population to which results generalize. A transparent survey should state which professional activities the evaluation captures and which remain outside its observation boundary.

Task cost must include verification and remediation. Let gross production benefit be V, execution cost C_e, review cost C_v, and expected failure loss L. A deployment decision depends on $V - C_e - C_v - L$ under a specified workload. These quantities may be uncertain and correlated. Faster generation can increase review volume, while stronger checks can reduce failure loss at additional cost. No universal conversion from token throughput to economic value follows.

The proposed evaluation therefore combines benchmark tasks with deployment-specific shadow runs. Shadow execution observes what the agent would propose without granting consequential effects. It can reveal permission and evidence failures while preserving a human-controlled process. It does not fully measure autonomous deployment because humans and environments may behave differently when the agent's actions are real. The limitation should remain explicit when interpreting readiness.

24.7 A reproducible evidence-review transaction

A research agent can implement evidence review as a transaction over claim records. New sources enter a staging area, identity resolution determines lineage and version, extraction proposes claims, and validation checks support and conflicts. Only then are accepted claims available for synthesis. Rejected or unresolved claims remain auditable without becoming authoritative memory. This separates acquisition success from evidential acceptance.

Let s be a source and q the research question. Version resolution $\mathcal{I}$, location-preserving extraction $\mathcal{X}$, and claim proposal $\mathcal{G}$ define

$$\begin{aligned} \eta &= \mathcal{I}(s), \qquad \mathcal{E} = \mathcal{X}(s;\eta), \\ (c_1,\dots,c_m) &= \mathcal{G}(\mathcal{E},q). \end{aligned} \tag{24.3}$$

The identity η binds the source lineage and inspected version; every observation in $\mathcal{E}$ retains its location. Each c_i records a proposition, applicable population, evidence class, source version, locator, assumptions, and unresolved limitations. Proposal does not imply support. Let $\mathcal{A}(\mathcal{E},\eta)$ be the candidate domain satisfying the declared scope and evidence-class requirements. Let $\mathcal{B}_0$ be the declared prior staging ledger and $\mathcal{B}_i$ its append-only extension after candidate i. Let $\mathcal{K}$ be the declared, versioned evidence corpus accessible under current research authorization; it may include relevant sources beyond the current extraction and staged records. For $i = 1,\dots,m$,

$$\begin{aligned} N_i &= \mathcal{N}(c_i;\mathcal{K},\mathcal{E},\mathcal{B}_{i-1}), \\ (v_i,\zeta_i) &= \begin{cases} \mathcal{V}(c_i,N_i,\mathcal{E}), & c_i \in \mathcal{A}(\mathcal{E},\eta), \\ (-1,\zeta_i^{\perp}), & c_i \notin \mathcal{A}(\mathcal{E},\eta), \end{cases} \\ r_i &= (\eta,c_i,N_i,v_i,\zeta_i), \\ \mathcal{B}_i &= \mathcal{B}_{i-1} \parallel \langle r_i \rangle. \end{aligned} \tag{24.4}$$

The operator $\mathcal{N}$ retrieves relevant disagreements from $\mathcal{K}$, the current located evidence, and prior staged records, retaining source identities and locators. The audit $\mathcal{V}$ returns a verdict $v_i \in \{-1,0,1\}$ and its evidence ζ_i; the values denote rejected, unresolved, and accepted. The record $\zeta_i^{\perp}$ states which scope or evidence-class check failed. The append operator $\parallel$ preserves each review occurrence, including rejected and unresolved records, without silently deduplicating their audit history.

Acceptance is versioned evidence, not a permanent property of a proposition. Let $\kappa_t(r) \in \{0,1\}$ indicate that record r has no unresolved invalidation under the current source versions, declared research scope, and review procedure. A corrected source or a newly discovered contradiction can set

$\kappa_t(r) = 0$ without rewriting the historical verdict. For current publication authority A, let $\chi_A(r) \in \{0, 1\}$ indicate whether a staged record may be released. The available synthesis view is

$$\mathcal{P}_A(\mathcal{B}_m) = \{r \in \mathcal{B}_m : v(r) = 1 \ \wedge \ \kappa_t(r) = 1 \ \wedge \ \chi_A(r) = 1\}. \tag{24.5}$$

Here $v(r)$ extracts the recorded verdict, not an assertion of objective truth; the subscript t on κ_t makes the view time-dependent even when the append-only ledger is unchanged. The complete ledger remains available to authorized audit even when a record is absent from the synthesis view. A failed freshness check quarantines the dependent inference until re-review; it need not erase the observation that originally supported it. No candidate becomes authoritative merely because extraction or proposal succeeded. The audit function is not assumed infallible. It can combine deterministic checks, independent model review, and human adjudication, each with measured error characteristics. A deterministic check can verify that a cited page exists, but not that it supports an inference. A second model can detect some mismatches while sharing the first model's biases. Human review can resolve ambiguity but introduces cost and variation. The implementation should record which evidence supported acceptance.

This transaction is particularly valuable when the research output becomes enterprise memory. Without staging, a speculative conclusion can be consolidated into a reusable skill and later appear as established policy. Requiring an evidence class and validity scope makes that promotion visible. Subsequent corrections can invalidate dependent records and trigger reevaluation of artifacts that used them. The system still needs a retention policy for sensitive source material and an authorization check before reuse.

Evaluation should seed incorrect citations, outdated versions, incompatible populations, and plausible unsupported conclusions. Also include valid minority findings that conflict with a popular narrative. Measure support precision, coverage of decisive evidence, conflict retention, abstention, and revision correctness. These metrics prevent the system from optimizing citation count or agreement at the expense of scientific accuracy. They also provide a concrete basis for comparing research products whose internal architecture is unavailable.

24.8 A changing-policy case in a shared knowledge workspace

Consider an enterprise research agent preparing an internal recommendation about a vendor. It retrieves a pricing document, a legal assessment, and an

operational incident note. The documents have different owners and access scopes. During the task, the legal assessment is corrected and the requesting employee changes teams, losing access to part of the workspace. The agent's active context still contains the earlier excerpts. A fresh retrieval permission check alone cannot remove those excerpts from the current computation or from previously generated summaries.

The first design requirement is dependency tracking. Every extracted claim and generated summary should identify the source versions it used. When the legal assessment changes, the system marks dependent claims stale and prevents them from being promoted as current evidence. This does not imply that all prior artifacts must be automatically deleted: some may be retained as historical records under appropriate access controls. The key distinction is between historical provenance and current applicability.

The second requirement is authority revalidation. A pending action to share the recommendation must be checked under the employee's current principal and purpose. Historical access does not authorize a new disclosure. If the current context contains content no longer permitted for the intended action, the application can rebuild a restricted context or require a controlled review path. It should not ask the model to infer access from the prose, because the decisive policy state may be absent or deliberately untrusted.

The third requirement is correction propagation. Updating the original document and vector index does not update a separately stored summary, a procedural note, or another agent's copied context. A dependency graph can enumerate these derivatives, but actual propagation may be asynchronous. The service should expose whether invalidation is pending, complete, or impossible for a particular external copy. A claim of immediate deletion across all surfaces would require evidence about those surfaces, not simply successful deletion of one database row.

A useful experiment stages the policy change at several points: before retrieval, after retrieval but before synthesis, after synthesis but before sharing, and after a recipient has accessed the artifact. The expected control differs at each stage. Before sharing, the system can block or regenerate. After disclosure, it may revoke further access but cannot guarantee that the recipient forgot or did not copy the information. Recovery metrics should reflect this irreversibility instead of treating every revocation as full restoration.

The comparison conditions should include connector-only checks, retrieval-time checks, and dependency-aware checks spanning derived memory and artifact publication. Hold the model and documents fixed. Measure unauthorized disclosure, use of stale claims, valid recommendation

quality, review burden, and completion latency. A system that blocks every recommendation after any policy change has strong containment but poor utility; a system that always completes may have the opposite profile. Reporting both axes prevents a degenerate policy from appearing universally superior.

This case also clarifies why enterprise deployment evidence cannot be borrowed from a generic question-answering benchmark. The difficult behavior is not recalling a sentence but maintaining a coherent relationship among evidence version, principal, purpose, and action time. A memory engine can improve retrieval while leaving that relationship unenforced. A workflow platform can enforce approvals while presenting stale evidence to reviewers. A model can reason correctly about the visible documents while lacking the current policy state. The system must join these layers explicitly.

For implementation, represent each artifact with a content digest, source-version dependencies, audience, purpose, and approval record. Store access decisions separately from the artifact text so they can expire or be revoked. Generate the model-visible authority summary from current policy, and record the epoch used at each effect boundary. These are concrete engineering requirements that can be checked without claiming that the model has a complete understanding of organizational intent. The remaining semantic judgment concerns whether the recommendation follows from the authorized, current evidence; that judgment still needs a calibrated evaluator and appropriate review.

The evaluator should also inspect who could observe each intermediate artifact. A final recommendation may be properly restricted while a temporary spreadsheet or trace exposes the underlying assessment to a broader audience. Thus the privacy unit is the complete artifact lineage, not merely the final answer. Instrumented tests can enumerate created objects and compare their access policies with the task contract. Where an application does not expose intermediate-object visibility, the corresponding claim remains unverified. This is a concrete observation limit and should affect the deployment assurance case rather than be replaced by a general statement that the platform is secure.

Chapter 25

Model Inspection, Executable Verification, and Evaluator Reliability

An executable evaluator can be wrong about what it certifies. TestJack's October 2026 witness construction targets candidates that pass existing tests while violating prompt-grounded requirements [231]. The resulting conflict is between an operational gate and a task property; it is not resolved by declaring either a test or a language-model reviewer an oracle. This current case motivates the chapter's separation of internal interventions, executable verification, evaluator classification error, and release evidence.

Inspection asks what a model or execution system is doing. Verification asks whether a specified property holds under declared assumptions. The two can support one another, but neither subsumes the other. An interpretable activation pattern does not prove that a tool action is authorized. A passing test does not reveal the complete internal mechanism that produced the artifact. A reliable technical account must identify the property, observation boundary, intervention, and residual uncertainty for each claim.

This chapter connects internal model interventions to executable task validators and evidence-based quality gates. It emphasizes false acceptance, evaluator dependence, and specification gaps because these are the points where apparent assurance can exceed actual support. Every proposed test remains a proposed experiment unless execution evidence is available.

25.1 Properties observations and assurance boundaries

Start with a property $P(x, e)$ of artifact or action x in environment e. A verifier V observes a projection $o(x, e)$ and returns accept, reject, or unresolved. The projection can omit decisive state. For example, a screenshot may not show a file's location, and a test log may not identify the artifact revision. No verifier can reliably recover distinctions absent from its observation without additional assumptions or queries.

This leads to three separate questions. Is the property well defined? Does the observation contain evidence sufficient to judge it? Does the verifier use that evidence correctly? A weak result at the third stage can be improved by a stronger judge. A failure at the second stage requires a better observation interface. A failure at the first stage requires specification or adjudication, not merely more inference. Collapsing these cases into one accuracy score obscures the intervention needed.

Formal verification supplies strong conclusions when the model, specification, and proof obligations are appropriate. Its guarantee remains conditional on those objects representing the deployed system. A proof about a pure function does not establish the behavior of a surrounding agent that calls another version, supplies invalid inputs, or operates with changed permissions. The deployment boundary must bind the verified artifact and assumptions to actual execution.

Executable tests provide finite observations. They can falsify a candidate under the tested conditions and can estimate behavior over a sampled distribution. Passing them does not prove a universal property. Nonetheless, a reproducible failing test is stronger evidence than an unsupported critique because another investigator can replay the witness. The key is to preserve the relationship between the witness and the intended requirement rather than accept every disagreement with a reference implementation as a defect.

An assurance case organizes these distinctions. Its claims identify required properties; evidence records observations and proofs; arguments explain why the evidence supports the claims; defeaters identify conditions that invalidate the argument. This structure is especially valuable for agents because the system changes across sessions, memory updates, model releases, and tool schemas. Assurance must track those dependencies instead of treating one successful evaluation as permanent certification.

For a binary property Y and a fixed observation O, let $\eta(o) = \Pr(Y = 1 \mid O = o)$. Under zero–one loss, the minimum error over all measurable classifiers using only O is

$$R_O^* = \mathbb{E}\big[\min\{\eta(O), 1 - \eta(O)\}\big]. \tag{25.1}$$

At each observation, choosing the more probable class minimizes conditional error; averaging gives the expression. It is a population identity, not an estimate available without labeled data. Two equally probable states with identical observations and opposite property values contribute an irreducible error of one half on that ambiguous subset. More inference over the same evidence cannot remove the ambiguity. Additional authoritative queries can change O and therefore change the bound; a richer explanation generated

from the original O cannot add independent evidence. This supplies a formal reason to diagnose observation loss separately from judge weakness.

25.2 Causal interventions in model computation

Activation inspection begins by recording internal states associated with particular inputs and outputs. A probe can predict a property from an activation without showing that the model uses that property to decide. Correlation can arise because the activation reflects another variable or because the probe extracts information unavailable to the model's downstream computation. Causal claims require interventions and carefully stated counterfactuals.

Activation patching typically compares a clean input, a corrupted input, and a corrupted computation in which selected activations are restored from the clean run. The measured recovery depends on what was corrupted, which sites were patched, and which output metric was used. A patch can have a large effect because it repairs many entangled variables rather than a uniquely interpretable concept. The intervention distribution is therefore part of the claim.

As a methodological foundation, Meng and colleagues' 2022 work uses causal tracing to investigate factual associations and test rank-one model editing through ROME [309]. The important evidence connects localized interventions to measured changes in factual predictions and editing behavior. It does not establish that every fact occupies one isolated location or that editing a weight removes all related knowledge. The result is bounded by model family, prompts, corruption procedure, and evaluation of specificity and generalization.

Zhang and Nanda's 2023 methodological study shows that activation-patching conclusions can change with metrics and corruption methods [310]. This is a methodological constraint on interpretation: a heatmap is not a mechanism by itself. A convincing study repeats the analysis under justified alternatives, includes negative controls, and reports which conclusions remain stable. If localization shifts dramatically, the appropriate conclusion is sensitivity to the intervention setup, not a precise biological analogy.

The implementation ledger should identify the activation hooks, cached tensors, patch function, corrupted-input constructor, and metric computation. Verify tensor shapes and token positions; a shifted position can patch the wrong event while producing plausible visual output. Then compare no-patch, random-site patch, targeted patch, and relevant ablations. Model outputs should be evaluated on held-out prompts, including cases where

the proposed concept is absent or ambiguous. These controls constrain causal interpretation without pretending to reveal the entire model.

The 2026 Certified Interventional Fidelity framework makes adaptive sampling another explicit boundary [311]. For a fixed bounded score $Y(x, i) \in [0, 1]$, declare input and target-intervention laws $\mathcal{D}$ and Π:

$$\vartheta = \mathbb{E}_{\mathcal{D}\times\Pi}[Y(X, I)], \qquad q_t = (1 - \eta)\Pi + \eta\widetilde{q}_t, \quad 0 < \eta < 1. \tag{25.2}$$

If X_t is a fresh independent draw and q_t is selected from past observations before the current score, the importance weight is

$$w_t = \frac{\Pi(I_t)}{q_t(I_t)} \leq \frac{1}{1 - \eta}, \qquad \widehat{\vartheta}_n = \frac{1}{n}\sum_{t=1}^{n} w_t Y(X_t, I_t). \tag{25.3}$$

The mixture retains target support while concentrating some queries on suspected failures. A compatible confidence sequence can accommodate repeated looks; searching many explanations additionally requires multiplicity control. This estimates the declared intervention average. It does not identify a uniquely correct circuit or certify that the score captures mechanistic truth. Clipping an unbounded recovery score changes the estimand and must be disclosed.

The sampling path and the evidence-acquisition path impose different identification conditions (Figure 25.1).

25.3 Sparse features and their evaluation

Sparse autoencoders approximate an activation vector using a small number of learned dictionary features. A common objective combines reconstruction error with a sparsity constraint or penalty. Let x be a model activation, $f(x)$ sparse feature activations, and D a decoder dictionary. The reconstruction term $\|x - Df(x)\|^2$ measures approximation in activation space. It does not directly measure whether feature labels are correct or whether all behaviorally important structure is represented.

Anthropic's 2023 monosemanticity report applies dictionary learning to a small transformer and analyzes feature interpretation, intervention, splitting, and limitations [312]. OpenAI's 2024 sparse-autoencoder work studies scalable training and evaluation of reconstruction, sparsity, and feature quality [313]. These are primary research contributions with different scopes. Neither establishes that a sparse representation is a unique, complete ontology of the model's concepts.

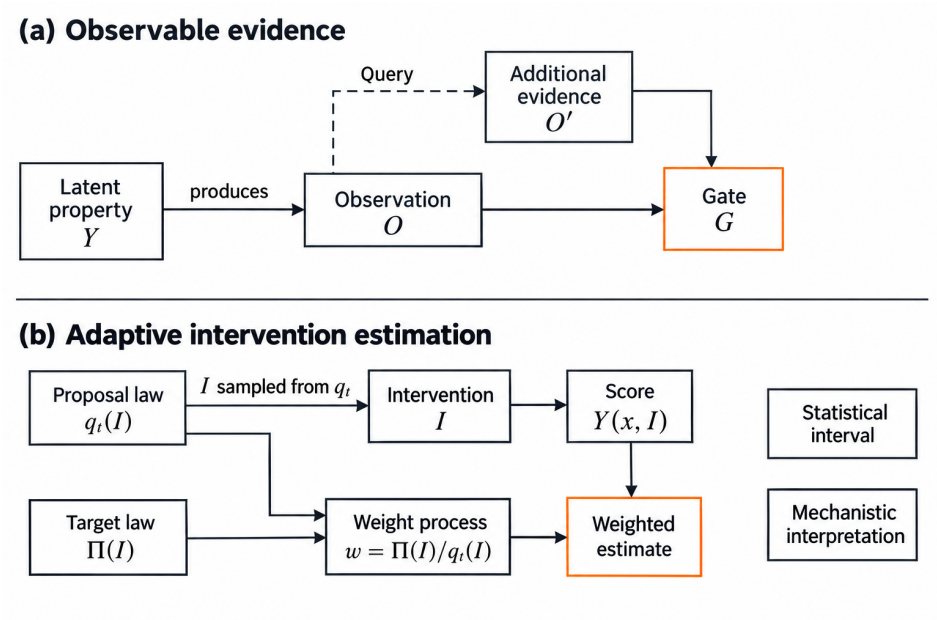

Figure 25.1: Verification depends on evidence and inference. The upper Y is a specified latent property; the lower $Y(x, I)$ is a declared intervention score. Additional queries can change observability. Only q_t samples I; both densities are evaluated at that sample to form $w = \Pi(I)/q_t(I)$. Target support and bounded predictable weights are assumptions. An interval for the resulting estimand does not establish mechanistic truth.

Feature interpretation must examine more than maximally activating examples. A feature can look clean at its strongest activations while responding to unrelated contexts at lower levels. Sampling only impressive examples introduces selection bias. Evaluation should include activation ranges, random examples, counterexamples, and downstream interventions. Automatic feature descriptions generated by another model require independent checking; a fluent label can rationalize a mixed feature.

Reconstruction and causal faithfulness can diverge. A dictionary can reconstruct common activation patterns well while missing rare behavior that matters for security or authorization. Conversely, a feature intervention can change behavior by moving computation off the model's natural activation manifold. The resulting effect is evidence about that intervention, not necessarily evidence that the feature normally causes the behavior in the same way.

For agent inspection, the relevant question is whether a feature-level

explanation improves a concrete decision or detects a failure under held-out conditions. Compare it with simpler probes, input-output monitors, and executable checks at matched access and cost. A feature-based monitor should be tested after model updates, context compaction, and tool changes because its calibration may depend on the original distribution. Internal access can provide useful evidence while remaining an additional system component with its own drift and false-alarm behavior.

25.3.1 The 2026 monitorability boundary

The preceding circuit and dictionary methods are methodological foundations, not descriptions of the entire October 2026 inspection frontier. Contemporary oversight also uses monitors of reasoning traces, actions and tool results. These observations are complementary to internal activations: a readable trace is neither a complete causal account nor a proof that an accepted action satisfies its specification.

The August implicit-influence study compares matched nudges delivered as incidental contextual details or as explicit instructions to use and conceal the nudge [314]. Across its tested task formats and extended-thinking models, behavioral influence can persist while trace-based detection weakens substantially in the implicit condition. Consequently, a monitor validated mainly on explicit concealment instructions can face a different observation problem in ordinary biased decision making. Its GPT-5-based judge does not establish that the influence is undetectable in principle; for Claude 4.5 runs the observed channel is an API summary rather than raw thinking. These are material access limits, not evidence that all reasoning traces are uninformative.

OpenAI's April 2026 release supplies datasets and metric code, but explicitly describes its public evaluation scaffold as illustrative and mocked, omitting restricted environments [315]. The inspected `gmean_minimal_criterion` routine computes an effect-normalized detection bound with eligibility conditions; `bootstrapped_gmean_metric` separates eligibility selection from estimation and resamples within instance–arm strata. Its reported quantity must not be relabeled ordinary classifier accuracy or a direct measurement of complete causal faithfulness. Report the eligible-instance fraction alongside the conditional metric; an undefined case is not evidence of zero monitor error. The source revision is `d14f181a0623`; it was inspected, not executed [316].

The September Astra system card supplies an additional current boundary: it distinguishes CoT-only, action-only and full-context monitors and reports reduced CoT monitorability relative to earlier models on its evalua-

tions [317]. These are provider-reported configured results. Full-context access contains evidence absent from a trace-only monitor, so a comparison must hold observation scope fixed before attributing a difference to the monitor model. The existence of a full-context deployment monitor does not establish the same visibility for an application consuming an ordinary API response or a summarized reasoning field.

A discriminating experiment crosses actor checkpoint, monitor checkpoint, observation scope, context transformation and adversarial awareness. Freeze complete trajectories for the monitor comparison; separately rerun actors when studying how awareness changes behavior. Compressing a trajectory after the fact measures evidence loss to the monitor, whereas letting the actor reason under compacted context changes both behavior and available evidence. Preserve that distinction when interpreting the interaction. Evaluate sensitivity to implicit influence, false alarms on benign reasoning, alert delay relative to effect commitment, and compute cost. A monitor that identifies an error after an irreversible effect provides diagnostic evidence but has not demonstrated preventive control. An internal signal should become a deployment gate only after its held-out calibration and intervention consequences have been tested at the actual action boundary.

25.4 Quality gates as fallible classifiers

Let $Y = 1$ mean a candidate satisfies the required property and $G = 1$ mean a gate accepts it. Define sensitivity $s = \Pr(G = 1 \mid Y = 1)$, false acceptance $f = \Pr(G = 1 \mid Y = 0)$, and prevalence $\pi = \Pr(Y = 1)$. Then

$$\Pr(Y = 1 \mid G = 1) = \frac{s\pi}{s\pi + f(1 - \pi)}. \tag{25.4}$$

Here $Y, G \in \{0, 1\}$, $s, f \in [0, 1]$, and $\pi \in (0, 1)$ so both class-conditional gate rates are defined; acceptance also requires $s\pi + f(1 - \pi) > 0$. Degenerate single-class populations need their own conditional interpretation. The identity follows from Bayes' rule. It shows why an apparently accurate gate can produce unreliable accepted outputs when valid candidates are rare. It also shows why calibration measured on one candidate population cannot be transferred uncritically to another.

Repeated search changes that population. If an optimizer generates many candidates and retains one that passes, it increases exposure to the gate's blind spots. Under an illustrative independent-invalid-candidate model, the probability of at least one false acceptance among n candidates

is $1 - (1 - f)^n$. Independence rarely holds exactly, but the expression identifies the selection pressure. A gate evaluated only on ordinary samples can fail after adaptive optimization.

Adding another gate does not automatically multiply reliability. If gates share a model, prompt assumptions, or evidence source, their errors can be correlated. The joint false-acceptance rate is

$$f_{12} = \Pr(G_1 = 1, G_2 = 1 \mid Y = 0). \tag{25.5}$$

It equals the product of the two class-conditional marginal rates only when conditional independence is justified. Diversity of implementation and observation can reduce shared failure modes, but that effect requires measurement.

An unresolved verdict is operationally useful. It allows the system to distinguish lack of evidence from detected failure. However, unresolved cases must remain in reporting denominators. A gate can achieve high precision by accepting almost nothing; whether that is useful depends on coverage and task consequences. Report accepted precision, valid-candidate recall, unresolved rate, cost, and residual failure severity together.

Gate evaluation should include adversarially selected candidates and naturally occurring failures. Seeded faults establish sensitivity to known defect classes; natural audits reveal unanticipated classes. Neither alone estimates deployment risk. The candidate generator, gate, and final adjudicator should have separate evaluation splits where possible. Otherwise the same held-out examples can leak into both optimization and assessment, creating an appearance of independent verification without actual independence.

25.5 Dynamic tests and requirement-grounded witnesses

TestJack audits benchmark-passing coding trials by generating tests tied to prompt requirements, checking them against a reference solution, and reviewing whether failures represent genuine requirement violations [231]. Its distinction between insufficient tests and underspecified prompts is essential: disagreement with the reference is not enough to refute a candidate. The paper reports substantial refutations in its audited passing-trial corpus; that proportion is not a prevalence estimate for all coding-agent executions.

The method's witness structure has three obligations. The test must correspond to a required behavior, pass on the reference, and fail on the candidate. The executable conditions are reproducible, but the requirement

correspondence remains interpretive. A faulty reference or ambiguous prompt can invalidate the inference. A reviewer agent reduces some mistakes but does not become an oracle merely because its role is called reviewer.

A stronger causal ablation keeps the candidate set and execution environment fixed while varying requirement grounding, reference validation, and review. Measure how many proposed witnesses survive independent adjudication and what defect classes they expose. Also examine false refutations: a method that discovers many failures by imposing extra requirements can reduce a leaderboard score while making the evaluation less valid.

Witness sharing across trials can amortize cost. If several candidates fail the same requirement, one validated test can inspect them all. This changes the unit of evidence: outcomes share a task and a discovered witness, so they are statistically dependent. Uncertainty should reflect that clustering. Randomly auditing trials can estimate properties of the declared audit population, but adaptive witness reuse and selective follow-up require a carefully described sampling scheme.

The implementation path should preserve the original repository, candidate patch, reference patch, generated witness, native test runner, and adjudication record. An artifact link or algorithm description does not establish that all pieces are available at an immutable revision. Until the exact implementation is inspected and run, report the method and author results separately from reproduction. This is particularly important for evaluator evolution, where changing a prompt or test-generation model can alter the measured defect population.

25.6 Judging long multimodal trajectories

Long computer-use trajectories challenge evaluators through observation volume, hidden state, and instruction-specific constraints. A judge may accept a visually convincing outcome while missing a wrong recipient, incorrect folder, or unrequested side effect. Conversely, it may reject a valid trajectory because the decisive evidence lies in an intermediate application state rather than the final image.

AgentHorizon uses paired instruction-trajectory relationships to probe these distinctions [22]. Its design motivates evaluating false acceptance and false rejection separately rather than relying on a single agreement rate. A judge that accepts almost every plausible workflow can appear competent on a positive-heavy dataset. Balanced metrics help, but they do not replace a clear definition of the underlying positive and negative populations.

Evidence access should be controlled experimentally. Compare final-

state observation, full trajectory, searchable artifact logs, and authoritative application queries. These conditions test different capabilities. Giving one judge an interactive tool harness and another only a fixed image sequence confounds model quality with evidence access. Tool-enabled judging also incurs latency and cost, and tools can fail or expose misleading state.

A useful implementation separates evidence retrieval from verdict generation. The judge identifies required predicates, retrieves supporting observations, and links each verdict to the corresponding evidence. This improves auditability but does not guarantee correctness. Evaluate whether cited observations actually establish the predicate and whether omitted constraints were checked. A verbose explanation can otherwise hide a shallow verdict behind extensive irrelevant evidence.

Trajectory compression is another treatment. Summaries can reduce evaluator cost while deleting the rare event that distinguishes success from failure. Compare compression methods on matched task families and include perturbations that move decisive evidence earlier or later in the history. Report the cost of reconstructing original observations. If the judge needs raw evidence after every difficult case, the summary's apparent savings may not survive end-to-end accounting.

25.7 Verification experiments and implementation traceability

A verification study should predeclare the property, candidate population, observation interface, and adjudication process. It should also distinguish discovery from confirmation. Cases used to design a new gate belong to development; untouched cases test whether the gate generalizes. Repeatedly inspecting failures and adjusting the gate against the same holdout turns the holdout into a training resource even if no model weights change.

The implementation ledger links a source claim to an equation or algorithm, a versioned code path, configuration, dataset split, and observed result. For ROME, inspect the intervention and editing implementation separately; a successful edit does not validate every localization claim. For sparse autoencoders, inspect training, feature extraction, and downstream metrics separately. For executable witnesses, preserve the native test environment and the requirement argument. These boundaries prevent one successful component from lending unwarranted credibility to another.

Negative controls are particularly valuable. Patch a random activation site, use an irrelevant feature description, supply a test that distinguishes an incidental reference choice, or give a judge a trajectory for a related but

incompatible instruction. A valid verification method should reject or limit these controls appropriately. Failing a negative control identifies a concrete weakness in causal interpretation or specification handling.

Uncertainty must follow the sampling unit. Multiple prompts for one fact, several features from one model layer, and many candidate patches for one task are not independent replications of the entire method. Use task-, fact-, model-, or repository-level grouping as appropriate. Report heterogeneity rather than only an aggregate mean, especially when rare errors have disproportionate consequences.

The practical implication is that verification is an evidence-producing system with its own model, tools, data, and failure modes. It should be versioned, monitored, and independently evaluated. Internal inspection can reveal useful mechanisms; executable checks can falsify artifacts; structured evidence can expose unsupported claims. Their combined strength comes from complementary observation and intervention, not from relabeling a second model as an infallible critic.

25.8 A worked gate analysis with adaptive selection

Consider a system that proposes a code change, obtains a test verdict, asks a model critic for approval, and publishes the accepted artifact for review. There are at least four distinct latent events: the patch satisfies the requirement, the test suite covers the relevant behavior, the critic interprets the evidence correctly, and the published artifact is the one that was checked. A failure at any stage can produce a misleading acceptance. Calling the arrangement a verification pipeline does not remove those events from the analysis.

Begin with the artifact-binding obligation. The test runner records digest d_1, while the publication tool is about to send digest d_2. If $d_1 \neq d_2$, the gate should reject or rerun regardless of the critic's confidence. This is a deterministic check over recorded identities. If the digests match, the system has established that evidence and artifact correspond, but it has not established that the tests cover the specification. The distinction allows an engineering invariant to be enforced without claiming an unavailable semantic oracle.

Next consider a candidate that passes all visible tests by special-casing the tested examples. A model critic may notice the shortcut, but it may also infer that the behavior is acceptable because the benchmark passes. A requirement-grounded witness can target an untested input implied by the

prompt. If the witness fails on the candidate and passes on the reference, the remaining question is whether its expectation actually follows from the task. An independent reviewer should be shown that question without being encouraged to treat the reference's incidental choices as requirements.

Now add adaptive search. The agent sees a rejection and revises the patch. The resulting candidates are not independent draws from a fixed distribution: each can be optimized against prior feedback. A false-acceptance rate measured before this interaction may no longer apply. To obtain a union bound over n attempts, one sufficient condition is a bound f_t on the conditional probability of false acceptance at attempt t given the entire previous search history. Then the probability of any false acceptance is at most $\min(1, \sum_t f_t)$. Marginal calibration on unrelated samples does not establish these conditional bounds.

The implementation consequence is to retain the search history and gate version. If a candidate generator is allowed to query the gate repeatedly, the evaluation must expose that access and include it in the budget. A final accepted artifact should identify how many candidates were considered and whether the gate's feedback was visible during revision. Otherwise two systems with the same final test suite can have different effective opportunities to exploit it.

A holdout gate can reduce direct adaptation, but it introduces its own governance problem. If researchers repeatedly inspect holdout failures to improve the system, the holdout gradually becomes development data. Rotating tasks or using a genuinely untouched confirmation set can help, provided the task distribution remains relevant. A smaller untouched sample may yield wider uncertainty than a large heavily reused benchmark. Scientific reporting should preserve that uncertainty rather than prefer the larger but compromised number.

Threshold selection also depends on consequence. A gate used to prioritize human review has a different loss function from a gate that authorizes external publication. Let $c_{FA} > 0$ be the cost of false acceptance and $c_{FR} > 0$ the cost of false rejection. For $q = \Pr(Y = 1 \mid O)$ conditional on the information O actually available to that decision, accepting has expected error cost $c_{FA}(1-q)$ and rejecting has expected error cost $c_{FR}q$ under a simplified two-action model. Acceptance is favored when $q \geq c_{FA}/(c_{FA} + c_{FR})$, with either decision permissible at equality. An estimated score requires validation in the corresponding decision population; marginal calibration on unrelated candidates does not supply this posterior when additional covariates change the costs or error distribution. The formula does not make either probability or cost available automatically.

An unresolved or review action changes that decision problem. Review can acquire additional evidence at cost, and the reviewer is also fallible. A rational policy compares expected residual loss after review with immediate acceptance or rejection. In practice these quantities are difficult to estimate, so applications often use hard rules for consequential effects and probabilistic gates for lower-impact filtering. The rules should be justified by the action boundary, not by an arbitrary global confidence threshold.

This analysis transfers to internal model monitors. Suppose a sparse feature is associated with a concerning behavior and triggers a warning. The feature's activation can be a useful signal without being a complete causal explanation. Calibration depends on model revision, context distribution, and intervention history. If the deployment policy changes behavior in response to the monitor, the observed data distribution changes too. A monitor evaluated only before intervention can therefore become miscalibrated after it is embedded in a control loop.

The discriminating experiment has separate discovery, calibration, adaptive-attack, and confirmation phases. Discovery identifies candidate signals or tests. Calibration estimates their behavior on a declared population. Adaptive evaluation allows a bounded optimizer to search for accepted failures. Confirmation evaluates the final fixed configuration on untouched cases. Report how candidates, tasks, and model families overlap between phases. This makes the independence claim inspectable rather than relying on the label held out.

Finally, preserve the distinction between a falsifying witness and a comprehensive safety claim. One replayable counterexample can refute a universal assertion about a gate. The absence of a counterexample in a finite search cannot establish that no accepted failure exists. It can support a bounded empirical statement about the search procedure, budget, and task population. That statement is valuable when accurately scoped. Inflating it into certification makes the evidence less useful precisely because it hides the operating conditions under which the gate was actually examined.

The ROME implementation makes this separation concrete. Its published causal-tracing experiment exposes a patching routine and hidden-flow calculation, while the model-editing module exposes the function that applies ROME updates[309]. These are distinct execution paths with distinct claims: restoring selected activations tests mediation under an intervention; applying an update tests whether a weight change produces the desired association and preserves specified neighboring behavior. A reproduction ledger should not mark both claims reproduced because only the editing demo ran. The sparse-autoencoder implementation likewise separates encoding and decoding in its Autoencoder class[313]. Reconstructing activations through

those functions establishes numerical behavior of the learned dictionary, while interpreting a feature and evaluating its downstream causal effect requires additional experiments. Record the model checkpoint, selected layer, activation normalization, feature selection rule, intervention magnitude and metric. These fields identify where two apparently contradictory analyses may actually differ. Code availability therefore strengthens inspectability without collapsing the distinction between a reachable function, an executed experiment and a supported scientific conclusion.

In particular, the inspected ROME source contains `trace_with_patch`, `calculate_hidden_flow`, `apply_rome_to_model` and `execute_rome`. Their presence was checked; no execution result is asserted here. Pinning these callables to a revision and running the associated controls is the implementation step required before an independent reproduction claim.

Chapter 26

Evaluation Harnesses, Contamination, and Statistical Validity

AgentHorizon and TestJack provide October 2026 examples of why evaluation logic belongs inside the scientific object being compared [22, 231]. A trajectory-aware computer-use judge and a coding witness-test auditor observe different evidence and target different mistakes. Their results should therefore alter the design of a comparison before they alter a leaderboard: define which property the instrument observes, audit its errors, and retain the configured execution that produced the artifact.

An evaluation harness is an experimental instrument. It constructs observations, constrains actions, allocates resources, records outcomes, and applies a scoring rule. Errors in any of those functions can change a measured capability claim. A benchmark name and a model name are therefore insufficient to identify an experiment. The evaluated object is the configured model-policy-harness-environment system under a particular sampling and scoring protocol.

This chapter develops the estimands, controls, implementation checks, and uncertainty analysis needed to interpret agent results. The aim is not to replace all benchmarks with one universal score. It is to make comparisons answer a precise question and to expose when apparently conflicting results concern different populations, information budgets, or evaluator behavior.

26.1 Define the estimand before the leaderboard

Let D be a task distribution, S a system configuration, B a resource envelope, and Y a declared outcome. The primary estimand can be written $\mu(S, B, D) = \mathbb{E}_{x \sim D}[Y(S, B, x)]$. Formally, D is a probability measure on an eligible task space $\mathcal{X}$, and Y is integrable under the joint task and execution randomness; a normalized success outcome has range $[0, 1]$. Every symbol carries a substantive choice. Tasks can be sampled by repository, user, domain, or time. Resources can constrain money, elapsed time, attempts, or tool access. Outcomes can measure final correctness, constraints, latency,

or utility. A numerical ranking is uninterpretable without these choices.

Micro- and macro-averages answer different questions. A micro-average weights each task equally, so a large task family dominates. A macro-average gives equal weight to families, potentially giving a small noisy group disproportionate influence. Neither is universally correct. Report the weighting rule and family-level results, and avoid changing weights after seeing which system benefits. Where a deployment distribution is known, a workload-weighted estimate can be relevant, but its sampling uncertainty and drift remain material.

The system boundary should include model checkpoint or service identity, prompt and harness revision, tools, memory policy, evaluator, and environment. A benchmark score obtained with a newer parser or different hidden-test version is a different observation even if the headline labels are unchanged. Closed service revisions may be unavailable; then the result is a dated service evaluation rather than a fully reproducible checkpoint comparison.

The foundational 2022 HELM framework emphasizes multidimensional evaluation across scenarios and metrics [18]. For agents, this perspective extends naturally to permission compliance, state consistency, external effects, and recovery. A system can improve success while worsening cost or security. A scalar utility can summarize those tradeoffs only after the weights and acceptable constraints are justified. It should not conceal a hard safety or authorization requirement inside a favorable average.

The practical first step is an evaluation contract that names the target population, eligibility rules, unit of analysis, allowed information, resource limits, stopping conditions, and outcome definitions. This contract supports later auditing because it makes deviations visible. It also prevents an experiment from drifting into a different question when difficult cases or infrastructure failures appear.

Figure 26.1 contrasts selection feedback with the one-way transfer of a frozen artifact to sealed evaluation.

26.2 Harness construction and artifact identity

The harness controls initial state and evaluator access. For repository tasks, it selects the checkout, installs dependencies, exposes the issue description, and later runs tests. For browser tasks, it resets application databases and authentication state. For memory tasks, it decides what persists across episodes. Each reset policy can leak information or remove difficulty if implemented incorrectly.

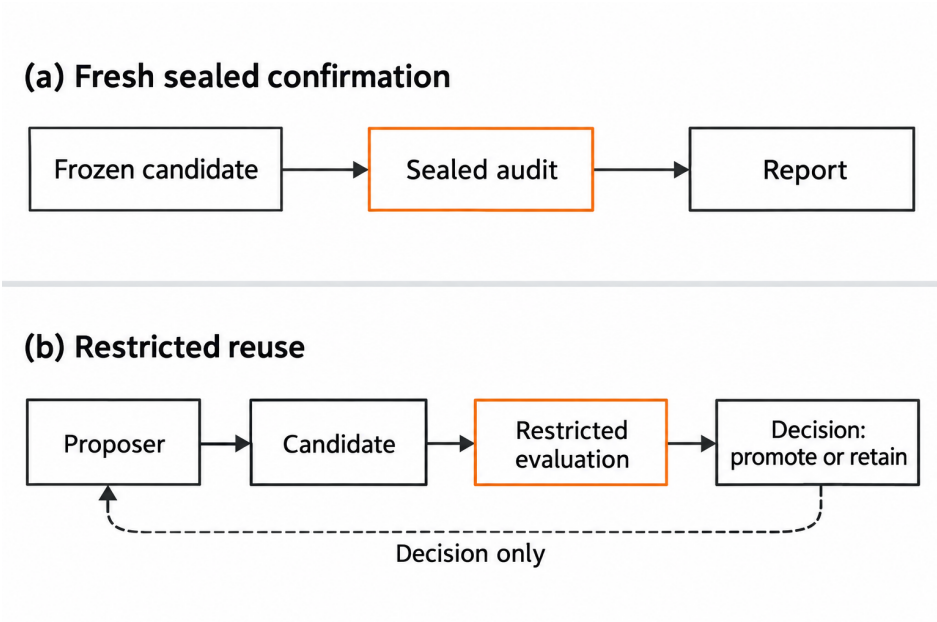

Figure 26.1: Two evaluation protocols with distinct feedback boundaries. Fresh sealed confirmation returns a report outside selection. Restricted reuse exposes only promotion decisions to the proposer; validity additionally requires the source protocol's initial evaluation independence and history-wide error allocation. The feedback restriction alone is not an error-control proof. No experimental outcomes are depicted.

The inspected language-model evaluation harness exposes evaluation entry points, while SWE-bench's harness exposes instance execution and aggregation paths [318, 319]. These are concrete implementation anchors, not evidence that every downstream adapter is valid. An audit should follow one task through construction, action execution, result serialization, and score aggregation. Check that the scorer reads the artifact generated by that task rather than an old file, cached response, or another worker's output.

Stable identifiers should connect task, attempt, environment, artifact, and verdict. A directory name alone is weak identity when workers reuse workspaces. Content digests and immutable manifests reduce ambiguity. The manifest should include test selection and evaluator configuration because a result can change when the same artifact is judged under a different rule. A successful container exit code does not establish that the intended tests ran.

Environment failures require explicit classification. Dependency installation can fail, a website can change, a tool can become unavailable, or the evaluator can crash. These failures affect operational reliability even when they do not diagnose model reasoning. Report both overall task success and conditional policy performance under valid infrastructure, with clear denominators. Removing failed setup cases after observing outcomes can bias comparisons if one system stresses infrastructure differently.

Sandbox isolation protects benchmark validity as well as deployment security. Hidden tests, reference artifacts, and scores must not be readable by the agent unless the protocol intentionally provides them. Shared caches and logs can create indirect leakage. A deterministic fixture should attempt to read forbidden paths and query evaluator state to verify the boundary. Such checks establish concrete isolation properties rather than relying on the phrase sandboxed environment.

26.3 Attempts reliability and selection

Finding one successful answer among several attempts differs from succeeding reliably on repeated execution. Under independent identical success probability p, the former is $1-(1-p)^k$ and the latter is p^k. These formulas are illustrative models, not general laws for agent systems. Attempts can share task difficulty, memory, environment state, or sampling behavior, making their joint distribution substantially different.

For n sampled candidates with c successes, the familiar combinatorial pass-at-k estimator is $1-\binom{n-c}{k}/\binom{n}{k}$ when the sampling and interpretation conditions apply [107]. The domain is $1 \le k \le n$ and $0 \le c \le n$, all integers, with $\binom{n-c}{k}=0$ when $k > n-c$. It estimates discovery of a success among selected candidates, not a deployable selector's performance. An actual system needs a gate that identifies a valid candidate without access to hidden labels. Selector false acceptance and false rejection belong inside end-to-end evaluation.

For a fixed realized candidate set, that expression is exactly the fraction of uniformly chosen size-k subsets containing a success. Its interpretation as an unbiased estimate of fresh k-attempt discovery additionally needs the corresponding exchangeable sampling model; adaptive revisions are not automatically such samples. Task heterogeneity introduces a separate distinction. If attempts are independent conditional on task q with success probability $p(q)$, then

$$\theta_{\mathrm{any},k} = \mathbb{E}_q[1-(1-p(q))^k], \qquad \theta_{\mathrm{all},k} = \mathbb{E}_q[p(q)^k]. \tag{26.1}$$

For integer $k \geq 1$ and $\bar{p} = \mathbb{E}_q[p(q)]$, Jensen's inequality gives $\theta_{\text{any},k} \leq 1 - (1 - \bar{p})^k$ and $\theta_{\text{all},k} \geq \bar{p}^k$. Thus even conditional independence does not justify raising a pooled mean to a power. These are analytical comparisons under the stated model, not estimates of any cited benchmark's reliability.

Attempt budgets should count failed parsing, tool errors, and abandoned generations where they consume resources. The mini-swe-agent accounting change discussed in Chapter 23 illustrates why this is implementation-sensitive. Equal nominal attempts can hide unequal token use or paid failed calls. Report measured cost and consumed resources rather than only configured maxima.

Repeated-run reliability should retain the benchmark's exact definition. Some protocols ask whether every repetition succeeds; others measure average success or consistency of a required constraint. A paper using one definition cannot be compared directly with another merely because both use a superscript or similar label. State the aggregation rule in ordinary language and provide the underlying per-task outcomes when possible.

Adaptive selection increases the effective search budget. A system that repeatedly revises prompts, harnesses, or memory using benchmark feedback has used the benchmark as an optimization resource. RRSI explicitly addresses overfitting in harness evolution [27]. The evaluation should preserve an untouched assessment set and record candidate counts, rejected changes, and evaluator access. Frozen weights do not imply absence of learning from test feedback.

An untouched confirmation set is one design; certified reuse requires a different protocol. REUSE permits feedback-adaptive proposals only through promotion decisions from an initially independent evaluation sample and allocates error across all possible promotion histories [41]. Returning detailed scores or failure traces to the proposer breaks that theorem's decision-only restriction unless the extra transcript is accounted for. A generic confidence sequence protects optional stopping under its assumptions; it does not erase information already leaked into candidate construction. The public report should therefore describe the feedback channel as precisely as the tested artifact.

26.4 Contamination information leakage and task lineage

Contamination is not a single binary property. A model may have seen the exact task, a near duplicate, the reference patch, a discussion of the solution, or a related repository version. A harness may leak hidden tests

at runtime. Persistent memory can transfer answer-bearing information from previous evaluation episodes. Each channel requires a different control and supports a different inference about generalization.

Use task lineage to track duplicates and revisions. A later benchmark release can contain transformed versions of earlier public tasks. Deduplication based only on exact text can miss equivalent code changes or renamed entities. Semantic similarity is useful for screening but can also flag legitimately related tasks. Report the screening procedure and unresolved cases rather than claim complete decontamination from one similarity threshold.

Temporal holdouts reduce some exposure but do not guarantee novelty. A task first published after model training may depend on a public issue or patch that predates the cutoff. Conversely, a benchmark revision date does not make an old task new. Record first public availability of the task and solution separately when evidence permits. For proprietary models, unknown training data limits what can be established.

Runtime leakage should be tested directly. Keep evaluator credentials outside the agent environment, separate reference artifacts, and inspect tool access. A browser agent can inadvertently observe a success marker or database field intended only for grading. A coding agent can discover test files copied into the wrong layer. These are implementation defects that can be fixed without speculative claims about model memorization.

Memory experiments require explicit contamination boundaries. If one arm carries experiences across tasks while another resets, the information difference is the treatment and must be reported. If a learned skill contains a benchmark-specific answer rather than a reusable procedure, transfer gains can reflect memorization. Holdouts should test both new instances and changed task structure, with source and experience provenance available for audit.

26.4.1 Interference through persistent state

A stateful benchmark generally does not admit independent per-task potential outcomes. Let $X_{1:t}$ be the task prefix, M_t^S the memory generated by system S on that prefix, and $Y_t(S, M_t^S, X_t)$ the next outcome. Changing the system at an early task changes later memory and therefore later outcomes even when the task list is fixed. A mean over tasks estimates performance of the complete stream policy under its order and feedback rules; it does not identify an isolated memory effect at every task.

Two experiments answer different questions. A frozen-prefix experiment initializes both arms from the same admissible state and varies the

next decision mechanism. A full-stream experiment initializes a common starting state and lets each arm construct its own subsequent memory. The former isolates a conditional local effect; the latter measures accumulated adaptation and error propagation. Resetting memory after every task removes the longitudinal treatment, while copying the stronger arm's evolved bank into both conditions changes the treatment again. Report the chosen intervention and cluster uncertainty by independently generated stream. Chapter 27 develops the corresponding counterfactual controls and contamination boundaries.

26.5 Uncertainty paired designs and dependence

An agent evaluation has several sources of variation: task sampling, model sampling, environment nondeterminism, evaluator error, and adaptive state. Treating every model call as an independent observation ignores most of this structure. The appropriate uncertainty analysis follows the unit to which the claim generalizes. Repository-level generalization needs variation across repositories, not merely many attempts on the same few issues.

Paired comparisons reduce nuisance variation by evaluating systems on the same task instances. Define $\Delta_i = Y_{i,A} - Y_{i,B}$ and estimate its mean over the target population. Confidence intervals should preserve pairing. If tasks cluster by user, repository, or persistent session, resample clusters or use an explicitly justified hierarchical model. The choice should be declared before inspecting which interval is narrower.

Repeated seeds estimate stochastic execution variation conditional on the sampled tasks. They do not substitute for broader task coverage. Conversely, one run on many tasks can estimate a task-population average while leaving repeated-run reliability uncertain. Report both dimensions when the deployment depends on consistency. If compute prevents sufficient replication, narrow the claim rather than inventing stability from a positive mean.

Multiplicity matters when researchers compare many models, prompts, memory mechanisms, and metrics. Selecting the best configuration and reporting an ordinary confidence interval as if it had been fixed in advance understates uncertainty. Separate exploration from confirmation, report the selection procedure, and use untouched data for final comparison. A universal correction is not required for every descriptive analysis, but the inferential claim must reflect how the configuration was chosen.

Missing and censored outcomes need visible treatment. A timeout may indicate excessive latency, infrastructure failure, or an unfinished but

promising trajectory. For a deadline-constrained task it is a failure to meet the deadline, regardless of possible eventual success. Additional analyses can examine completion-time distributions, but they should not erase the primary operational outcome. Report sensitivity to reasonable classification choices when the cause is ambiguous.

26.6 Ablations and causal mechanism claims

An ablation removes or changes a mechanism, but it can also change resources, information, and optimization opportunities. Removing a retrieval module may shorten the context and reduce cost; removing a critic may reduce model calls; replacing a hypergraph with flat retrieval may alter both representation and access policy. The observed difference identifies the combined treatment unless additional controls isolate the components.

HyperSkill provides a useful example. Its no-hypergraph condition replaces the structured retrieval pipeline with flat embedding retrieval, so the result does not isolate hyperedge representation alone [143]. A stronger comparison keeps decomposition, candidate pools, ranking budget, and maintenance cost comparable while varying the storage relation. This does not invalidate the system result; it narrows the causal explanation supported by that ablation.

Oracle controls estimate headroom. Oracle retrieval tests whether downstream reasoning can use sufficient evidence; oracle state tests whether partial observation is the bottleneck; an oracle selector tests candidate-generation potential. These controls are deliberately privileged. Their performance cannot be described as deployable capability, but the gaps help prioritize engineering work. A large oracle-retrieval gap suggests retrieval deserves attention before increasing reasoning depth.

Negative controls test whether a supposed mechanism is doing meaningful work. Shuffle memory, provide irrelevant skills, randomize delegation messages, or replace a feature description with a plausible wrong label. Some controls can damage the system in several ways, so interpret them carefully. The aim is not to produce a dramatic drop but to distinguish a specific explanation from alternatives such as extra context, extra compute, or evaluator bias.

A mechanism claim requires explicit operating conditions and failure regimes. It identifies where the mechanism helps, where a simpler baseline matches it, and where it causes regressions. This is more useful than a universal superiority claim because deployments occupy particular workloads. Method comparisons should therefore preserve heterogeneity across

horizon, task structure, evidence quality, and tool reliability rather than compress everything into one average.

26.7 Evaluator audits and reproducible reporting

Evaluators need their own validation set and error analysis. A benchmark can be well constructed while a particular grader accepts artifacts that violate requirements. AgentHorizon and TestJack expose different aspects of this problem: long-trajectory judgment and insufficient executable tests [22, 231]. The appropriate response is to audit the observation and scoring pipeline, not merely add another model judge and call the result verified.

October 2026 harness evaluations expose additional estimand boundaries. TraceTrial-Bench preserves raw operational states but resolves some unexercised criteria using the task outcome, so its semantic harness and task scores are coupled; its selected execution archive does not supply independent repetitions within every cell [3]. ASAP's frozen-base-model measurements include reference-prefix elicitation and bounded recovery, whereas Harness Scaling evaluates training exposure and transfer to withheld execution combinations [4, 8]. An audit must retain those interventions in its configuration ledger. Treating all three as ordinary backbone leaderboards would discard the variable each study is designed to examine.

Preserve raw verdicts, reason codes, evidence links, and subsequent adjudications. If an evaluator is corrected, publish both original and revised outcomes with the rule change. A moving grader can otherwise make historical comparisons impossible. Where legal or privacy constraints prevent releasing trajectories, provide a precise description of what remains inaccessible and what external investigators can still reproduce.

The reporting record should contain task manifest, environment construction, model and harness identities, prompts, tool schemas, memory policy, budgets, seed policy, evaluator revision, and complete outcomes. Logs should distinguish no response from empty response, unknown usage from zero usage, and incomplete work from completed failure. These distinctions affect both cost and capability estimates.

Reproducibility is graded rather than binary. Metadata verification establishes identity. Source inspection establishes an implementation path. Executed fixtures establish behavior under controlled conditions. Independent replication tests whether reported results recur. Cross-environment reproduction tests robustness to implementation and infrastructure differences. A survey should name the achieved level for each source rather than allow a verified title to imply verified experimental evidence.

The practical endpoint is a comparison whose conclusion can be reconstructed from its records. Another investigator should be able to identify what changed, which observations supported the score, how uncertainty was computed, and which failures remain unresolved. That standard does not require pretending that every proprietary component is transparent. It requires making opacity an explicit limit on inference rather than filling it with plausible assumptions.

26.8 A concrete audit of a paired repository comparison

Consider two coding systems evaluated on the same repository-task manifest. The manifest names the base commit, issue description, dependency environment, required tests, and allowed resource envelope. Each system produces one primary attempt and optional additional attempts under a separately declared budget. The experiment's primary question is deadline-constrained verified completion, not whether a successful patch can eventually be found with unlimited retries. Defining this question before execution determines how timeouts and infrastructure problems enter the analysis.

The harness first constructs an isolated environment and records its identity. The system receives the task and permitted tools, but not the reference patch or hidden evaluator. At termination, the harness freezes the produced artifact, records its digest, and runs the evaluator against that digest. If the artifact changes during evaluation, the run is invalid and must be diagnosed rather than scored against a moving target. This sequence is directly auditable in the task construction, execution and scoring code paths.

The inspected SWE-bench execution module exposes `run_instance` and `run_instances`; the language-model evaluation harness exposes `simple_evaluate` and `evaluate`. These names identify concrete entry points for examining how tasks become outcomes[319, 318]. Their presence does not establish that a custom adapter passes the intended arguments or that an experiment used the inspected revision. The ledger must connect the callable to actual configuration and recorded execution before calling the result reproduced.

Table 26.1 specifies the evidence needed at each boundary. The table contains no measured outcome. It is a testable contract for distinguishing model errors from deterministic evaluation defects.

Table 26.1: Evidence boundaries in a repository evaluation.

Boundary	Required identity or observation	Failure to expose
Task construction	Task ID, base commit, dependency manifest, reset policy.	A reused checkout contains a previous answer or a changed dependency.
Model context	Prompt digest, tool schemas, accessible history and memory scope.	One arm receives additional solution-bearing context.
Execution	Attempt ID, tool calls, costs, timeouts and environment events.	Failed paid calls disappear from the budget record.
Artifact freeze	Patch or commit digest and creation time.	Tests run on a different artifact from the submitted one.
Grading	Evaluator revision, selected tests, raw verdict and error status.	Evaluator failure is interpreted as an ordinary negative result or omitted.
Aggregation	Eligibility denominator, pairing and missing-outcome policy.	Difficult failed runs are excluded after their outcomes are known.
Confirmation	Untouched tasks and fixed final configuration.	Repeated holdout tuning is reported as independent assessment.

Suppose some outcomes remain unresolved because the evaluator failed after execution. A success rate computed only over resolved runs is a conditional quantity. Without assumptions about missingness, it cannot identify the overall task success rate. If there are N eligible runs, s confirmed successes and u unresolved outcomes, a simple identification interval is $[s/N, (s+u)/N]$ when all other outcomes are confirmed failures. The interval expresses missing information; it is not a confidence interval for task sampling.

This distinction matters when systems fail differently. One system may generate patches that stress the evaluator and cause more unresolved runs. Excluding them can make it appear better even if they are concentrated among difficult cases. Conversely, scoring every evaluator crash as a model failure mixes infrastructure and policy behavior. Report the overall operational outcome and a cause-stratified analysis, while retaining a sensitivity analysis for unresolved cases. Do not choose the classification after seeing which ordering it favors.

For paired comparison, preserve the outcome pair for each task. Tasks

where both systems succeed contribute differently from tasks where only one succeeds. A test based on discordant pairs can be useful under its assumptions, but repository clustering or persistent memory can violate independence between pairs. A cluster-aware bootstrap or hierarchical analysis should then follow the declared task-generation structure. More attempts on the same repository do not create independent evidence about new repositories.

Resource matching requires another audit. If system A retries malformed outputs while system B stops, equal configured call limits can conceal different paid usage. Record actual model and tool consumption. If one arm receives more wall-clock because its environment starts faster, decide whether startup is part of the deployment comparison or a nuisance variable to control. Both choices can be legitimate; silently switching between them cannot. A complete report shows configured ceilings and realized consumption.

The evaluator should also inspect null and reference conditions. The unmodified repository reveals whether the benchmark's tests already pass without a fix. The reference patch reveals whether the intended environment can represent the benchmark's declared solution. A null pass can indicate inadequate tests; a reference failure can indicate environment drift or benchmark defects. Neither result should be attributed to the evaluated agent. These controls establish whether the instrument can distinguish the states it is intended to score.

Finally, add an independent audit of accepted patches. Dynamic witness tests can expose behavior missing from the fixed suite, while requirement review prevents incidental reference choices from becoming new obligations. Keep the original and audited scores separate, identify the sampled passing-trial population, and cluster uncertainty by task. A revised acceptance criterion changes the estimand; documenting that change makes the evidence more useful rather than less impressive. The result is a comparison whose limitations are concrete enough to guide the next experiment.

An additional distinction concerns cost per successful task. Dividing total cost by confirmed successes can be operationally informative, but it is unstable when success is rare and can hide differences in task mix. Report total eligible tasks, confirmed successes, unresolved outcomes and aggregate cost alongside the ratio. If a system fails quickly on hard tasks, it may have low average cost per attempt without being efficient at useful work. If it spends heavily to rescue difficult cases, its average attempt cost can rise while its cost per accepted deliverable improves. The relevant conclusion depends on which work the deployment must complete, not on whichever denominator yields the smaller number.

The benchmark should preserve outcome severity as well as success. A task that returns no patch, a patch that fails tests, and a patch that passes tests while changing access controls all count differently in an operational loss model. The primary benchmark can retain a binary completion definition for comparability, but a secondary failure taxonomy should record those distinctions. Severity weights require domain justification and sensitivity analysis. They should not be tuned to favor the preferred system after the failures are known. This analysis is particularly important when moving from isolated coding containers to tools that can affect external users.

Releasing the comparison also requires attention to provenance and confidentiality. A trace can contain credentials, private code, personal data or hidden evaluator material. Redaction should preserve event identity, timing, error categories and the evidence needed to reconstruct the score while removing content that cannot be shared. If redaction removes decisive evidence, state the resulting reproduction limit explicitly. A public summary and a private complete audit are different evidence products. Neither should be represented as the other. The scientific requirement is an honest account of which claims an independent reader can actually check from the released artifacts.

A signed manifest can bind the released task list, artifact digests and scoring configuration, reducing ambiguity about which experimental objects a later reproduction attempts to reconstruct.

Chapter 27

Longitudinal Evaluation of Memory, Adaptation, and Recovery

InMind and MemCalib distinguish two failures that an explicit-recall score can conceal. InMind probes the recognition of indirectly relevant historical information, whereas MemCalib studies whether supplied memory influences an answer too little or too strongly [134, 155]. These 2026 studies do not establish indefinite retention or reliable autonomous operation. They motivate separate interventions on admission, retrieval, interpretation, and behavioral influence, followed by longitudinal tests of changing facts, policy, and consumer versions.

Long-horizon evaluation asks whether useful behavior persists as evidence accumulates, tasks change, memory evolves, and interruptions occur. A single successful episode cannot establish this property. A system may recall explicit facts while failing to apply them indirectly, improve on one task order while regressing on another, or preserve raw history while losing authorization and temporal validity. The evaluated object is therefore a trajectory of system states rather than a sequence of independent questions.

The objective is to distinguish retention, retrieval, interpretation, influence, adaptation, and operational continuity. These mechanisms interact, but they require different controls. This chapter develops a longitudinal experimental design and connects recent memory studies to the precise claims their evidence supports.

27.1 Define horizon and the unit of continuity

Horizon can mean model turns, tool actions, elapsed time, task count, or sessions. These quantities are correlated only under a workload model. A hundred short tool calls can be easier than a few actions separated by uncertain external events. An accelerated simulated clock can test temporal logic without establishing months-long deployment validity. Every study should declare which horizon is manipulated and which remains uncontrolled.

Continuity also has multiple units. A session can retain conversation history, a task can span several sessions, a user can have many tasks, and an organization can share a skill bank across users. The memory reset boundary determines the treatment. A fresh model call with a persistent external store is not a no-memory condition. Conversely, carrying raw conversation text without cross-session storage is not evidence of durable personal memory.

Represent a longitudinal unit by initial state, task sequence, environment-change schedule, model and harness versions, memory operations, and final outcomes. This record permits causal interventions on order, updates, and interruptions. It also exposes whether a claimed improvement is conditional on an implicit curriculum or on access to outcomes unavailable in deployment.

The two cited memory surveys provide complementary taxonomies of memory forms, functions, substrates, subjects, and dynamics [15, 135]. Their value here is conceptual organization and discovery of primary methods. A survey's broad coverage does not independently validate the experiments it summarizes. Quantitative conclusions should return to the original method, exact version, and evaluation protocol.

The practical implication is to choose the unit of generalization before collecting runs. If the claim concerns new users, many episodes from one user do not suffice. If it concerns memory evolution, repeated calls against one fixed bank do not estimate variation in bank construction. If it concerns robustness after upgrades, a fixed-policy study cannot answer it. These distinctions determine both experimental cost and statistical uncertainty.

27.2 Factor the memory pipeline

A memory pipeline writes records, organizes them, retrieves candidates, selects evidence, interprets it, and influences an action. Failure at one stage can look like failure at another. If a fact was never stored, improving the reader is irrelevant. If it is stored and retrieved but ignored, improving the embedding may not help. If it is applied too strongly after becoming stale, higher recall can worsen the result.

Use staged oracle controls. Supply the original record directly to test the reader's ability to use it. Supply an oracle retrieval set to test selection and interpretation. Replace learned writing with complete raw logs to estimate extraction loss. Preserve the same action task and resource accounting. Oracle controls are diagnostic and may exceed deployable budgets, so their scores should not be presented as operational performance.

The evaluation should include negative transfer. A no-memory baseline reveals whether irrelevant or incorrect records reduce task quality. A shuffled-memory condition tests whether apparent benefit comes from generic additional text or task-specific information. A stale-memory condition tests conflict resolution. A revoked-memory condition tests authorization. These are different interventions and should not be combined into one noisy-memory score without retaining their separate outcomes.

Storage and retrieval metrics remain useful intermediate observations. Measure record coverage, provenance completeness, retrieval recall, rank quality, temporal filtering, and evidence packing. But none alone establishes useful behavior. A perfectly retrieved false record can harm the task, and a low-ranked record may be irrelevant to the action. Relate intermediate metrics to task-level outcomes through explicit ablations rather than assuming monotonic improvement.

Construction and maintenance cost belong inside the experiment. Include extraction calls, embeddings, graph updates, consolidation, deduplication, deletion, and reindexing. Report amortized cost over different reuse counts. A system that is efficient after extensive preconstruction may be unsuitable for cold-start users; another may have low initial cost but unbounded storage growth. The appropriate operating point depends on the actual longitudinal workload.

27.3 Implicit relevance and appropriate influence

InMind tests a failure mode in which a memory is relevant through a knowledge bridge rather than lexical resemblance [134]. Its paired controls distinguish storage, bridging knowledge, and query-conditioned retrieval. This is more diagnostic than asking only whether the agent can repeat a fact on demand. The benchmark is selective, however, and its authors identify domain coverage, shared answerer-judge identity, and missing negative controls against over-warning as limitations.

The mechanism suggests a concrete comparison. Hold the decisive memory and task fixed, then vary explicit recall queries, indirect application queries, direct in-context exposure, and retrieval-mediated exposure. A gap between the latter two implicates the access interface rather than necessarily the model's knowledge. To test whether a proposed remedy overuses memory, add cases where a similar stored fact should not affect the answer. Without those negatives, more frequent warning or personalization can look like better judgment.

MemCalib isolates another stage: the amount of influence a supplied

memory should have [155]. It distinguishes over-use from under-use and studies training signals that balance the two. Because the relevant memory blocks are supplied, the result concerns memory-use behavior rather than complete storage and retrieval quality. Its counterfactual localization is a credit proxy with its own uncertainty, not a direct measurement of every token's true causal contribution.

An influence evaluation needs context-dependent targets. A memory can be evidence, a preference, a constraint, or an obsolete observation. The proper response depends on provenance, current authority, and task scope. A simple rule to always prioritize memory is therefore inadequate. Test contradictory updates, exceptions, and cases where a retrieved instruction lacks authority to modify the user's objective.

These studies jointly motivate separating recall from decision use. Better retrieval can expose more relevant evidence while a poorly calibrated reader overweights it. Better memory-use training can improve interpretation while retrieval still misses implicit relevance. A complete experiment crosses retrieval policy with reader policy and reports their interaction. This factorial design can reveal whether an improvement transfers across memory engines or depends on a particular writer-reader pairing.

27.4 Task order adaptation and hidden curricula

An evolving memory makes task order part of the treatment. Early episodes determine which procedures and mistakes enter the bank; later episodes determine whether those records are useful. A default dataset order may contain an implicit curriculum, grouping related tasks or increasing difficulty. Improvement along that sequence does not establish order-robust learning.

The fragility study reevaluates memory-based self-improving agents across multiple runs and shuffled orders, identifying substantial sensitivity to variance, order, and underspecification [320]. Its findings apply to the tested methods and environments, not every possible self-improvement architecture. The methodological implication is strong: one cumulative curve under one order is insufficient evidence for a stable improvement mechanism.

Use several independently constructed memory trajectories for each experimental condition. Within a trajectory, tasks share an evolving bank and are dependent. Across trajectories, randomize order and model sampling as declared. If a common initial bank is used, treat its construction as another factor rather than assume it is representative. Report variation across banks and orders, not only average task accuracy pooled over all

calls.

Curriculum effects can be studied directly. Compare random order, grouped order, reversed order, and an explicitly designed curriculum while holding the task multiset fixed. A curriculum that improves performance is a legitimate result if named as such. The error is presenting that conditional benefit as spontaneous general improvement. Also test new task families that do not share the procedures learned early in the sequence.

Underspecified tasks can contaminate memory with misleading lessons. If a failure arose from an ambiguous objective or unavailable tool, the agent may store an incorrect procedural correction. Later reuse then amplifies the original ambiguity. Memory construction should record failure attribution and uncertainty, and evaluations should distinguish environment failure from policy failure. Additional rubrics can improve specification while also giving the learner more supervision; that information difference belongs in the treatment description.

27.5 Temporal change deletion and migration

A persistent memory must cope with facts becoming false, permissions changing, and representations drifting. Static recall benchmarks cannot test these transitions. A useful longitudinal suite includes explicit corrections, contradictory sources of unequal authority, expired preferences, deletion requests, user separation, and model or embedding upgrades. The required behavior is not always to retain more: sometimes it is to stop using a previously useful record.

Define the validity interval of a fact separately from its observation timestamp. A record can remain historically true while becoming inapplicable to the current decision. Retrieval should expose this distinction, and the reader should not treat a recent ingestion time as evidence of recent event time. Inject delayed reports and backdated corrections to test whether the system confuses the two.

Deletion tests must follow derivatives. Raw events can generate summaries, embeddings, graph edges, skills, caches, and copied subagent context. Removing one row does not establish complete erasure. A test should enumerate the lineage, verify retrieval exclusion, and check whether reconstruction can reintroduce deleted content. Physical deletion, logical exclusion, and retention under restricted audit policy are different outcomes.

Migration changes memory meaning. A new embedding model can alter nearest neighbors; a new policy can interpret the same skill differently; a schema conversion can drop provenance or confidence. Compare pre- and

post-migration behavior on held-out tasks and known conflict cases. A successful database migration does not establish behavioral compatibility. Retain the old representation where permitted so failures can be traced and rollback can be assessed.

Authorization is particularly sensitive to restored state. A checkpoint can contain a historical grant, but a resumed session must consult current authority. The experiment should revoke access while the task is paused and then resume from the old checkpoint. Measure whether the forbidden record remains in model-visible context and whether new effects are blocked. This tests context-memory-policy consistency rather than storage persistence alone.

27.6 Horizon-dependent reliability and survival

Long tasks can be analyzed through time to a defined failure event. Let T be the first step at which a required invariant is violated and $S(h) = \Pr(T > h)$ the survival probability through horizon h. The hazard at step t is $\lambda_t = \Pr(T = t \mid T \geq t)$. Then $S(h) = \prod_{t=1}^{h}(1 - \lambda_t)$ under the corresponding discrete-time definitions. Here $T \in \mathbb{N}_{>0} \cup \{\infty\}$, $h \in \mathbb{N}_0$, the empty product is one, and λ_t is defined only while $\Pr(T \geq t) > 0$. The identity does not assume a constant hazard.

This representation distinguishes cumulative exposure from worsening policy. A system with constant small per-step risk can have declining long-horizon success without becoming less competent at each step. Conversely, memory contamination or context loss can make the hazard increase over time. Stratify failures by cause and inspect their relationship to compaction, retrieval, updates, and interruptions. A single final completion rate cannot distinguish these explanations.

Censoring requires care. If a run ends because the budget expires before failure or completion, the censoring mechanism may depend on task difficulty. Standard survival estimators require assumptions about that dependence. Report budget exhaustion explicitly and avoid treating censored trajectories as successful survivors indefinitely. Competing events, such as authorization failure, evaluator outage, and task completion, may require cause-specific analysis rather than one undifferentiated stopping time.

In particular, successful completion removes a run from exposure to further task actions. To distinguish this event from failure, let U be the first absorbing stopping time under the study protocol and J its cause. For each cause j, define $\lambda_j(t) = \Pr(U = t, J = j \mid U \geq t)$ while the risk set is

nonempty. Then

$$S_U(h) = \prod_{t=1}^{h} \left(1 - \sum_j \lambda_j(t)\right), \qquad F_j(h) = \sum_{t=1}^{h} S_U(t-1)\lambda_j(t). \tag{27.1}$$

Here the causes are mutually exclusive, $\sum_j \lambda_j(t) \le 1$, and $F_j(h) = \Pr(U \le h, J = j)$ is the cumulative incidence of that stopping cause. Treating completed tasks as ordinary independent censoring can instead estimate a hypothetical continued-exposure process. Faster completion and lower failure hazard are different interventions; report completion incidence, failure incidence, and unresolved budget stops separately. The identity is a probability decomposition, not a claim that a particular survival estimator is justified for the available logs.

Recovery changes the event definition. A transient tool failure that is correctly reconciled may not count as task failure, while an undetected duplicate effect may violate an invariant even if the final artifact looks correct. Declare whether the study measures first fault, unrecovered fault, constraint violation, or final failure. Each is useful, but they answer different operational questions.

Uncertainty should cluster by longitudinal unit, not by step. Thousands of steps from one evolving bank do not provide thousands of independent observations about population reliability. Report the number of independent users, sessions, task streams, and bank constructions. Where the data are limited, present the observed trajectories and conditional conclusions rather than fitting an elaborate hazard model whose parameters are weakly identified.

27.7 A longitudinal implementation and reporting contract

The evaluator needs a clock, task scheduler, environment-change injector, memory snapshotter, and artifact verifier. These components should be separate from the agent's own tools. Otherwise the agent can observe future tasks, change the evaluation clock, or inspect outcomes intended only for grading. A simulated environment can provide controlled interventions, but its simplifications must be identified when generalizing to deployment.

For every memory operation, record writer identity, source event, model and harness version, record type, validity scope, permissions, and affected derivatives. Read operations should record candidate sets and selected evidence where feasible. This enables failure attribution: a missing fact,

bad ranking, inappropriate influence, and stale authorization leave different traces. Privacy constraints may limit released content, but event structure and redaction policy can still be documented.

The baseline set should include no memory, raw-log retrieval, simple lexical or dense retrieval, curated factual records, procedural memory, and an oracle evidence condition. Match task streams and budgets, and include construction cost. A complex system should demonstrate benefit over a strong simple baseline, not only over an intentionally weak store. Report where simpler methods win and whether the difference survives shuffled orders.

The October 2026 audit of MemoryAgentBench's conflict-resolution split supplies a more specific baseline: a frozen parser and last-write resolver that directly executes the benchmark's newest-statement rule [321]. The parser was developed on one fact list and held fixed on three others. The audit separates parser or path-composition residuals from items whose released answer is unreachable in its frozen parser's final last-write graph. This operational classification does not itself prove that the released label is wrong: a fuller parser or a different grounding rule can change reachability. Its principal item-level model comparisons retain one scored outcome per item, without mechanism traces, and do not run learned or structured memory systems. Thus an apparent failure can reflect a label–rule conflict, rather than evidence that an agent failed selective forgetting; the audit does not establish how a particular memory mechanism reasons internally.

This case changes the experimental unit. Hundreds of questions sharing four fact lists do not constitute hundreds of independent histories. Answer-level uncertainty is conditional on those lists; generalization across history construction is limited by the small number of distinct streams. A proposed extension generates independent update streams from a versioned state machine, derives answers by replay, and verifies the gold graph independently of the agent's parser. Include first-write, last-write, valid-time, and authority-aware resolvers as distinct baselines only where the task semantics warrant them. Blindly applying last-write order to real organizational evidence would be incorrect when a later source has lower authority or reports an earlier valid-time interval. Benchmark-rule correctness and deployment conflict semantics must remain separate.

HyperSkill's sequential fixed-order evaluation, extra LLM maintenance calls, and gold-outcome main setting illustrate why these fields matter [143]. Its structured memory mechanism is substantive, while concurrency, order robustness, and deployment feedback availability require additional evidence. Similarly, context-management results that retain raw offloaded history should not be interpreted as indefinite usable memory without testing

retrieval and decision preservation [119].

The practical output is a trajectory-level evidence record that supports both mechanism analysis and operational decisions. It should show what the system retained, what it retrieved, how it used the evidence, what changed, and which constraints survived. This is a more demanding standard than a recall score, but it is necessary when memory is expected to influence long-running agents across users, tasks, and versions.

27.8 A counterfactual session-stream experiment

Consider a technical assistant supporting a software team over a sequence of repository tasks. Early sessions establish that one service uses a particular deployment command, that production changes require review, and that a diagnostic document belongs to a restricted project. Later sessions change the command, introduce an exception for a sandbox environment, revoke one user's access, and ask an indirect question whose answer depends on the old experience. The test is not whether the assistant can repeat each fact. It is whether it uses the correct record, scope and authority when choosing an action.

Construct independent task streams from templates with controlled entity substitutions, dependency structures and change schedules. The substitutions prevent exact-string memorization from explaining transfer, while the dependency structure preserves the reasoning required. Separate streams should not share answer-bearing memory unless cross-user sharing is explicitly the treatment. The evaluator retains the authoritative event sequence and can query the environment, but the agent sees only observations available at each step.

Table 27.1 lists interventions that distinguish the relevant mechanisms. Each row specifies what changes and what observation would diagnose the failure. The table is an experimental design, not a set of reported measurements.

Table 27.1: Counterfactual interventions in a session stream.

Intervention	Controlled change	Diagnostic observation
Write omission	Remove one source event from the writer input.	A missing record should be attributed to acquisition or writing, not retrieval.
Oracle record	Supply the decisive record directly in context.	Continued failure implicates interpretation or action policy under this control.

Intervention	Controlled change	Diagnostic observation
Implicit query	Replace explicit recall wording with a knowledge bridge.	A direct-versus-indirect gap identifies a relevance-routing limitation.
Contradiction	Add a newer authoritative correction.	The selected record and action should reflect validity and authority, not mere recency.
Scope exception	Make an old procedure valid only in a sandbox.	Blanket reuse or blanket rejection reveals inadequate applicability reasoning.
Revocation	Remove a principal's access while paused.	Resumption must not turn historical access into a new authorized effect.
Deletion	Remove source content and mark derivatives.	Retrieval and reconstruction should not silently reintroduce excluded information.
Migration	Change policy or embedding version at a boundary.	Differences should be linked to representation, retrieval or interpretation changes.

The baseline conditions should preserve information and budget as carefully as possible. A direct-context oracle can consume more prompt space than a retrieval system, so reserve equivalent context capacity or report the difference. A full-history baseline can become infeasible at long horizons; report the horizon at which it ceases to fit rather than silently switching its truncation policy. A simple raw-log retriever provides an important control because it avoids extraction errors while retaining the same source evidence.

Writer and reader policies should be crossed. A high-quality writer paired with a weak reader may produce accurate records that are misapplied. A strong reader paired with an aggressive summarizer may never receive the decisive constraint. Crossing these components identifies interaction effects and prevents a memory engine from receiving credit for a better backbone. The experiment should also vary whether outcome feedback is gold, self-judged, delayed or unavailable. A method that relies on immediate correct labels has a different deployment requirement from one operating without them.

Task order needs independent replication. Generate several permutations of the same multiset and several independently sampled streams. The first comparison isolates curriculum effects conditional on the tasks; the second

supports broader generalization. Preserve the entire evolving memory for each run so failures can be traced to prior updates. Reusing a bank built under one order across all conditions would remove an important source of variation and could favor methods compatible with that bank's organization.

The primary outcomes include verified task completion, constraint adherence, temporal correctness, authorized evidence use and recovery after interruption. Intermediate outcomes include write coverage, retrieval of decisive records, correct rejection of obsolete records and calibrated influence. These outcomes should be linked rather than pooled indiscriminately. For example, a retrieval improvement with no action improvement can reveal a reader bottleneck; an action gain accompanied by unauthorized context use is not an acceptable solution to the stated task.

Negative controls are essential. Include tasks where the remembered deployment command is irrelevant, where the restricted document is unnecessary, and where the sandbox exception should not generalize. Otherwise an agent that always emits warnings or always retrieves every memory can receive undeserved credit. Measure over-application and under-application separately, as motivated by MemCalib, while retaining the end-to-end storage and retrieval components that its supplied-memory setting intentionally isolates.

Failure adjudication should use the event history and artifact state, not the agent's explanation alone. A stored memory can claim that a tool failed because credentials were invalid when the actual cause was a temporary outage. If this explanation becomes a procedural rule, later tasks may avoid a useful tool unnecessarily. The evaluator should identify the original observation, the inferred lesson and the later consequence. This causal chain distinguishes bad experience attribution from simple forgetting.

Finally, report the maintenance workload as the bank grows. Track raw and derived storage, index updates, compaction, merge operations, deletion propagation and retrieval latency. A sublinear skill count does not imply bounded total memory if raw trajectories and subtask nodes continue to grow. Nor does a compact bank guarantee low decision cost if each query invokes extensive reasoning. The study should expose which resource grows with episodes, which is amortized across reuse, and which can be bounded only by discarding information. Those operating conditions determine whether a promising memory mechanism can support sustained agent execution.

The public HyperSkill implementation provides a concrete separation between the hypergraph core and the per-task provider interface[143].

A reproduction should instrument both: the core determines candidate reachability and maintenance, while the provider determines when retrieval and ingestion occur relative to task execution. Logging only final skill counts would miss a provider that ingests outcome information too early. Logging only model context would miss a maintenance operation that changes the interpretation of later records. Attach task and memory-version identifiers to each operation so the two traces can be joined.

The implementation audit should also verify that the evaluator's gold outcome is unavailable to deployment-style variants before they commit their own interpretation. A gold-feedback experiment is useful for estimating mechanism potential, but it should not be confused with a self-judged agent. When switching to self-judgment, measure the resulting memory-label errors and their downstream accumulation rather than reporting only final success. A modest immediate error rate can have a disproportionate longitudinal effect if the same mistaken lesson is reused repeatedly.

The fragility study's published code and InMind's released diagnostic resources offer additional primary anchors for this audit[320, 134]. Their existence improves inspectability but does not establish that the experiment described here has run. The appropriate next record is an immutable implementation revision, complete stream manifest and trace of the actual interventions. Until those observations exist, the proposed factorial and counterfactual tests remain a research design with explicit rejection criteria, not evidence of achieved lifelong reliability.

The inspected provider exposes `provide_memory` and `take_in_memory`; instrumentation at these boundaries can establish whether evidence was supplied before a decision or incorporated only after the task outcome.

27.8.1 What a counterfactual replay can identify

Replaying logged actions under a changed memory policy does not by itself estimate that policy's end-to-end performance. Once a changed context induces another action, the future tool results, environment and memory writes can diverge. Continuing to supply the original observations then evaluates a hybrid trajectory that neither policy would necessarily generate. The problem is particularly acute for recoverable effects: an original successful write cannot be assumed to exist after a counterfactual policy chose a read instead.

A frozen-prefix test can legitimately compare next-action distributions from the same recorded state, provided both contexts contain only information available at that boundary. This identifies local sensitivity, such as whether a revoked instruction changes a proposal, rather than full task

success. A full counterfactual test must rerun the environment from a valid checkpoint or use a justified transition model, exposing model error and replay limits. Importance weighting requires logged behavior probabilities and support for the alternative actions; deterministic tool logs do not supply those conditions automatically. Thus paired prefix diagnostics, complete reruns and model-based counterfactuals should be reported as distinct evidence products rather than interchangeable forms of replay.

Figure 27.1 isolates the mechanism and its stated information boundaries.

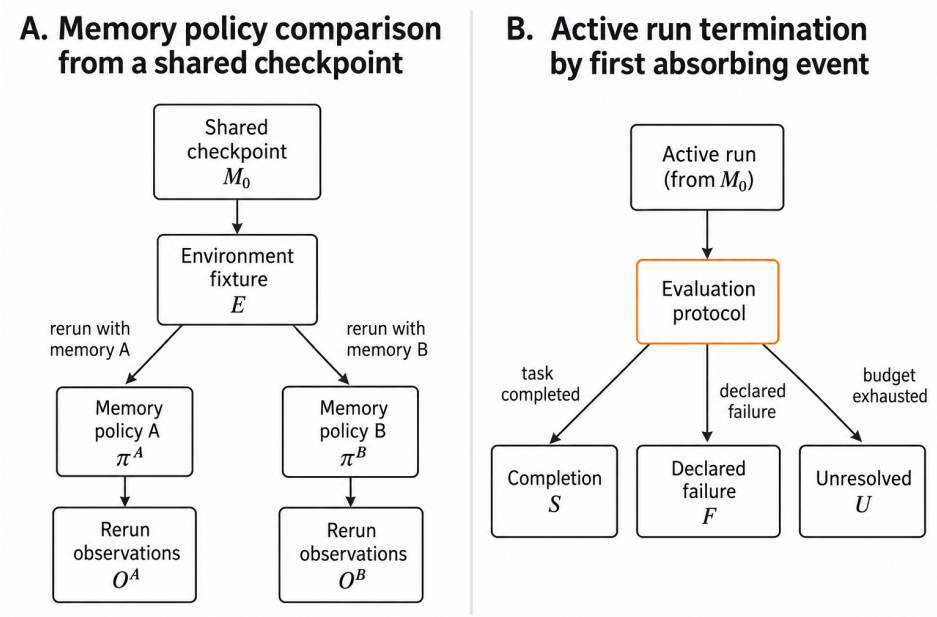

Figure 27.1: Memory policies π^A, π^B share an initial fixture (M_0, E) but can produce different rerun observations O^A, O^B. Observation-changing counterfactuals require valid reruns, not substituted answers on one fixed log. Completion S, declared failure F and unresolved budget stops U are distinct protocol outcomes; interpreting a stop as censoring requires additional assumptions. The figure-local S/F/U labels name outcome categories, not stopping times.

Chapter 28

Stateful Agent Security: Red-Team and Blue-Team Mechanisms

DTap, ExploitGym, and CyberGym E2E evaluate different 2026 adversarial objects: simulated tool-mediated interactions, agent-assisted exploitation tasks under deliberately permissive research conditions, and memory-safety vulnerability reproduction and repair [322, 323, 25]. A result in one object does not estimate attack prevalence or compromise probability in another. Progent's revised symbolic confinement mechanism addresses a different question again: which proposed tool effects can cross an enforced policy boundary [24]. These cases determine the threat model and observation boundary before any security comparison is meaningful.

Agent security concerns what a system can be induced to do across model, context, memory, tools and infrastructure boundaries. A harmful answer and an unauthorized external effect are different events. A model can resist a malicious instruction while a tool adapter leaks credentials, or produce an unsafe proposal that an application policy correctly blocks. Evaluation must observe the complete path from adversarial influence to actual effect.

This chapter treats red teaming as bounded adversarial evaluation and blue teaming as prevention, detection, containment and recovery. The focus is defensive research in controlled environments. Capability benchmarks, prompt-injection benchmarks and operational incident studies provide different kinds of evidence; their results should not be converted into one universal security score.

28.1 Threat models and attacker capabilities

A threat model specifies the attacker objective, controlled surfaces, knowledge, interaction budget, timing and permitted effects. An attacker who controls a retrieved document differs from one who controls a tool implementation, a skill package, or a shared memory store. The defender's trusted computing base also matters: a model-only defense cannot compensate for an attacker-controlled authorization service. Security claims must state

which components are assumed honest.

Represent the attacker policy by π_A and defender system by S_D. An attack-success event Z is evaluated under an environment distribution and attack budget. The quantity $\Pr(Z = 1 \mid \pi_A, S_D, D, B)$ is conditional on those choices. It is not the probability that an arbitrary deployment will be compromised. Adaptive attackers can search for failures, while benchmark attackers may have limited access and static payloads. Their results are not interchangeable.

The objective should be an observable violation. Examples include reading a forbidden synthetic record, modifying an out-of-scope fixture, or sending a test message to a controlled sink. A malicious-looking string in the model output is not necessarily a completed attack. Conversely, an unauthorized effect can occur even if the model's final message appears benign. Instrument the effect boundary and retain both attempted and completed violations.

The earlier AgentHarm benchmark measures harmfulness-related agent behavior under its benchmark protocol [324]. It supplies a useful evaluation axis but does not replace prompt-injection or access-control testing. The cited cybersecurity survey organizes broader offensive and defensive uses [325]; as secondary synthesis, it is a discovery and taxonomy source rather than independent validation of every included capability claim.

A strong threat model also identifies out-of-scope conditions. If the operating system or model-serving provider is trusted, state that assumption. If the attacker cannot observe defender feedback, do not describe the result as fully adaptive. If a test uses synthetic confidential data, describe the fixture rather than implying a real breach. These boundaries make the evidence interpretable and keep defensive evaluation reproducible.

Figure 28.1 locates the authority boundary after exposure to potentially persistent injected content.

28.2 Injection across context tools and persistent memory

Prompt injection exploits confusion between data and instructions across a trust boundary. An agent reads external content needed for a legitimate task, but that content attempts to alter goals, tool use, or disclosure. The vulnerability is architectural because the same language interface can contain both evidence and directives. A warning in the prompt may help behavior but does not create a hard separation of authority.

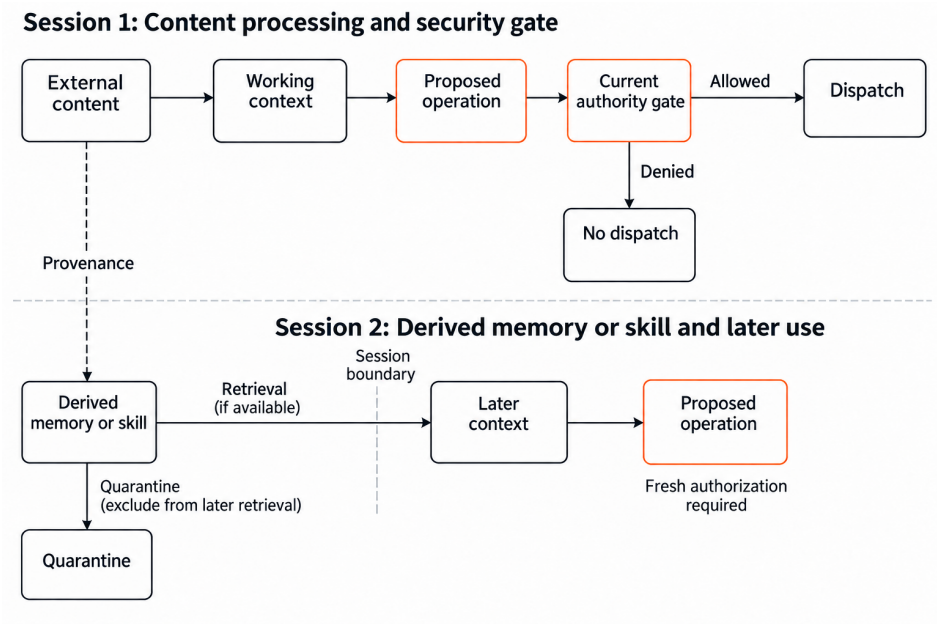

Figure 28.1: Untrusted content can persist through derived memory or skills after the original input disappears. Application-controlled quarantine excludes a retained record from later retrieval; current authorization constrains dispatch. The first-session decision does not authorize a later operation: fresh authorization is required before its dispatch, which is not depicted. An allowed operation may still be factually wrong, and these controls do not undo committed effects.

Tool outputs can carry several kinds of risk. They may contain untrusted prose, malformed structured fields, misleading status, or references to external artifacts. A tool can be honest while returning attacker-controlled content, so classifying the whole tool as trusted is too coarse. Preserve provenance and distinguish the tool's transport or schema guarantees from the authority of the data it retrieves.

Persistent memory extends the attack horizon. A malicious instruction can be extracted as a preference, consolidated into a summary, or promoted into a reusable skill. Later retrieval can make it appear to be established internal knowledge. The attacker no longer needs to be present in the current task. A memory writer therefore needs provenance, scope and promotion controls, and the reader must revalidate authority rather than trust the storage location.

Skill supply chains add executable and procedural risk. A skill can

contain instructions, scripts, dependencies and tool references. Reviewing only its short description misses transitive behavior. Loading a skill should not automatically grant the permissions needed by its embedded resources. Version pinning and application-enforced capabilities limit the effect boundary, while semantic review examines whether the procedure remains applicable to the current task.

MCP and other tool protocols provide structured interfaces, but protocol conformance does not establish tool honesty or policy compliance [162]. Discovery metadata, annotations and returned content have distinct trust roles. A client must enforce authorization and validate arguments independently of descriptive claims supplied by a server. Schema-valid invocation is a necessary interface property, not a universal security guarantee.

SkillJack makes cross-layer promotion concrete. Its attacker injects experiences without direct write access to the skill library; the unchanged extraction and routing pipeline can turn those records into separately stored procedural artifacts [326]. Deleting the original record can then leave the derived skill available. The report's scope is two systems with one model and a policy-violation proxy, not demonstrated live-service compromise. A diagnostic must therefore test source deletion and derivative invalidation separately, retain clean queries to detect unintended activation, and judge effects independently of whether a skill is routed. Changing extraction can reduce recognizable malicious wording without removing the harmful procedural implication.

MemEvoBench probes a different mechanism: repeated mixed correct and misleading memory, biased feedback, and noisy workflow returns can shift later behavior [327]. Its QA and workflow constructions use generated memory pools; their contamination rate and update protocol define an experimental exposure, not its prevalence in deployment. To identify accumulation rather than immediate persuasion, compare the same final visible evidence with and without intervening memory writes, freeze the reader policy, and counterbalance exposure order. A source-quarantine defense can prevent an initial write yet fail after an unsafe abstraction has been promoted elsewhere. These cases require the dependency-aware revocation and longitudinal units of Chapters 13 and 27.

28.3 Least privilege and symbolic action controls

Least privilege constrains the actions needed for the current task rather than relying on the model to avoid every tempting alternative. A permission policy should apply at the tool boundary, where names, arguments, principal

and effect scope are explicit. The model can propose actions and request policy changes, but an application-controlled procedure decides which proposals become executable.

Progent represents privileges using symbolic rules over tool names and arguments, checks calls deterministically, and distinguishes policy narrowing from expansion [24]. Its monotonic-confinement idea is precise: absent the required approval, the allowed action set should not grow. The guarantee depends on complete mediation and the semantics of the policy language. Effects outside the checked tool interface remain outside that guarantee.

Let $\mathcal{A}(P)$ be the actions permitted by policy P. A proposed update from P to P' is a narrowing when $\mathcal{A}(P') \subseteq \mathcal{A}(P)$. Establishing this relation requires reasoning over the policy's argument domain and predicates. A solver can decide it only within the supported formalization. Ambiguous natural-language intent, hidden tool effects and incomplete schemas are not solved merely by placing an SMT solver in the loop.

Approval of an expansion must bind to the proposed change and current context. A generic yes from an earlier session should not authorize a different argument set after a model or tool update. Record the principal, operation scope, expiry and policy epoch. Recheck the grant at dispatch, especially after interruptions. This prevents a stale permission record in memory from silently becoming current authority.

The utility-security tradeoff must be measured. An overly narrow policy can block legitimate work; an overly broad policy can preserve completion while failing containment. Compare policy generation, deterministic enforcement, and approval mechanisms separately. Include benign tasks and adversarial variants, and report blocked legitimate calls, completed violations, unresolved requests and review burden. A low attack-success rate achieved by refusing all tools is not sufficient evidence of a useful defense.

28.3.1 Open privilege can remain invisible to executed attacks

Ajar's September 2026 method probes defenses with candidate calls unnecessary for a benign task, at states reached by replaying its reference solution [328]. This measures permission left available even when an agent never proposes the corresponding call. It therefore adds an axis that observed attack success cannot identify. Its scope remains conditional: candidate generation samples environment-derived values, and most labels depend on the audit's own judgment rather than an existing benchmark predicate. A useful alternative plan involving a tool absent from the ref-

erence can be labeled excess. These limitations prevent the score from becoming a universal measure of least privilege.

Let $\mathcal{Q}$ be a declared finite set of state–call probes, $w_q \geq 0$ severity weights, $a(q) \in \{0, 1\}$ defense acceptance, and $e(q) \in \{0, 1\}$ an excess-call label. When the excess-weight denominator is positive, a diagnostic leakage rate is

$$\Lambda(\mathcal{Q}) = \frac{\sum_{q \in \mathcal{Q}} w_q e(q) a(q)}{\sum_{q \in \mathcal{Q}} w_q e(q)}. \tag{28.1}$$

This is an explicit sampled-set statistic, not the size of the entire reachable privilege space. Report sensitivity to labels, probe sampling and risk weights alongside false denial of entitled calls. Tightening a policy is useful only when the retained action set still supports legitimate work.

28.4 Controlled red-team platforms and judge validity

DTap supplies controllable simulated environments, adversarial exploration and policy-based evaluation for agent security [322]. The important contribution is an execution and measurement substrate that makes attack surfaces and outcomes inspectable. Simulations can support broad controlled comparisons, but their external validity depends on whether tools, policies and workflows capture the deployment conditions of interest.

A policy-based judge is still an evaluator. It can misclassify an action, omit a side effect, or accept a nominally blocked operation whose downstream consequences violate the intent. Judge validation should include positive, negative and ambiguous cases with authoritative fixture state. If an attack targets the judge or its observation path, the evaluation needs a separate trusted record of effects. Otherwise a successful deception can be scored as a failed attack.

28.4.1 Attacker–defender self-play changes the data distribution

GPT-Red's 2026 report trains an attacker against a population of concurrently improving defenders. It samples an environment and permitted insertion point, allows bounded defender queries, validates the submitted attack, then resumes the defender and updates both roles from their respective rewards. Invalid attacks receive zero reward. The defender reward includes legitimate-task completion, so the training is not generally a zero-sum refusal game [329]. The official July 15 announcement links the report;

the arXiv version was submitted July 28. These are separate publication dates, not evidence of an additional model revision.

For a general analysis, let D be a probability measure on a controlled environment space $\mathcal{E}$, $z \sim D$ an environment, $u \sim \alpha_\phi(\cdot \mid z)$ a proposed modification in a candidate space $\mathcal{U}$, and $\xi \sim P_\theta(\cdot \mid z, u)$ a defender trajectory in Ξ. Let $v(z,u) \in \{0,1\}$ denote validity under the declared attacker access, $L(z,u,\xi) \in [0,1]$ the prohibited-effect score, and $U(z,\xi) \in [0,1]$ legitimate utility. Distinct objectives are

$$J_A(\phi;\theta) = \mathbb{E}[v(z,u)L(z,u,\xi)], \tag{28.2}$$

$$J_D(\theta;\phi) = \mathbb{E}[U(z,\xi) - \lambda L(z,u,\xi)], \qquad \lambda \geq 0. \tag{28.3}$$

This formulation is an analytical comparison, not the proprietary training objective. It explains why attacker success alone cannot judge the defender: refusing every operation can suppress L while destroying U. A claimed equilibrium guarantee additionally needs a specified policy class, payoff, valid-action domain and convergence argument. Alternating stochastic updates in a partially observed agent environment do not automatically meet those conditions.

Co-adaptation makes the training curve ambiguous. Falling attack success can mean a stronger defender, a weaker or collapsed attacker, an easier sampled task mix, or a changed validity checker. Preserve frozen attacker and defender checkpoints and evaluate a cross-play matrix $R_{ij} = \mathbb{E}[L \mid \alpha_{\phi_i}, \pi_{\theta_j}, D_0]$ on a fixed held-out distribution D_0. Match query budgets and include benign utility, invalid-attack frequency, evaluator error and total generation cost. GPT-Red reports population diversity and held-out assessment; that evidence does not certify robustness to every unobserved tool, harness, or attacker [329]. Feedback validity is especially important when a learned judge supplies both training reward and reported success.

28.4.2 A modular adversarial interface is a measurement contract

The inspected SuperRed revision separates targets, optimizers, security claims, observable events and controllable surfaces. `Optimizer.on_event` receives routed events; `Controller.run` orchestrates interaction; hierarchical `SecurityDomainTag` scopes constrain access [330]. These are source-level interfaces, not proof that every wrapper exposes a complete trajectory. Its documented target lifecycle preserves durable per-task memory across ephemeral resets, and target-model spend is outside the optimizer's token budget. Consequently, a matched optimizer budget does not establish matched total experiment cost.

A defensive integration should first run a no-modification optimizer to check that wrapping preserves benign behavior, then verify access isolation using synthetic surfaces with known scope. Reset tests must distinguish new attempts against one evolving memory from independent clean tasks. A wrapper that inadvertently carries poisoned memory into a supposedly independent trial changes both the threat model and the uncertainty unit. Pin the controller and target adapter together: a revision to which events are observable can strengthen an attacker without changing its checkpoint. No SuperRed attack or training run is executed here.

Attack surfaces should be varied one at a time before studying combinations. Document injection, tool-result poisoning, skill modification and memory contamination can interact, but a combined successful attack does not identify which boundary failed. Use controlled ablations and counterfactual replay to test the causal chain. Preserve the benign task so the defender must continue useful work rather than simply classify a suspicious input in isolation.

Red-team artifacts should remain bounded and reproducible. Use isolated fixtures, synthetic confidential records and controlled destinations. The scientific record needs attacker access, permitted effects, payload category, budget, defender configuration and raw outcome evidence. It does not require exposing operational details that would enable misuse against live systems. A defensible survey can analyze the mechanism and controls without reproducing harmful deployment instructions.

28.5 Cyber capability is not agent compromise

Cybersecurity capability benchmarks evaluate whether an agent can perform technical security tasks. They differ from benchmarks asking whether an attacker can redirect a benign agent. Confusing the two can produce misleading safety conclusions. A capable vulnerability analyst can be well controlled, while a weaker model can still violate confidentiality through an overprivileged connector.

CyberGym evaluates vulnerability reproduction in realistic software projects [331]. A successful proof-of-concept reproduction demonstrates a particular capability under the benchmark's information and environment conditions. It does not establish full exploitation, remediation, or unrestricted offensive effectiveness. CyberGym-E2E broadens the evaluation toward discovery, reproduction and patch generation [25]. The stages remain distinct because success in one does not imply success in the others.

ExploitGym studies turning known vulnerability-triggering inputs into

concrete exploitation outcomes under controlled configurations [323]. Its trusted-access and safeguard conditions matter when interpreting frontier capability. Results obtained with safeguards disabled are capability-boundary evidence, not estimates of ordinary product behavior or real-world attack prevalence. Security protections and environment variants should remain part of the reported configuration.

Patch verification also needs dual criteria: removal of the vulnerability and preservation of required functionality. A patch can eliminate the triggering behavior by disabling the program or rejecting all inputs. That is not necessarily a useful remediation. Test security and regression behavior together, and retain uncertainty where the available tests do not exhaust either property. Reference patches can supply evidence without becoming infallible specifications.

For comparative studies, match initial information, tool access, time, and verifier feedback. An agent given a vulnerability description and triggering input has a different task from one asked to discover a defect from scratch. A benchmark using real historical vulnerabilities may also have contamination concerns. Report task lineage and exposure controls rather than infer novelty from the date of the benchmark paper alone.

28.5.1 Symbolic search, fuzzing, and static analysis have different oracles

The 2026 CyberGym and ExploitGym results belong to an older automated-security lineage, but do not supersede every mechanism in it. AEG, published at NDSS 2011, formulates control-flow exploit generation as a verification problem and uses preconditioned symbolic execution. MAYHEM, published at IEEE Security and Privacy 2012, works at the binary level and combines online and offline symbolic execution with symbolic memory-index reasoning [332, 333]. These are foundational designs with historical target and mitigation assumptions. Their feasibility results should not be presented as current success rates against modern software.

The methodological contrast is important. Symbolic execution constructs path conditions and asks a solver for witnesses within a model of program semantics. Fuzzing samples or mutates inputs and retains feedback that guides further search. An agent can orchestrate either process, interpret a failure, or propose a patch; its language-model reasoning does not replace the solver's satisfiability condition or the runtime's observed failure. A benchmark must identify which stage was automated and what evidence establishes the claimed outcome. A reproducible crash, a sanitizer diagnosis, control of an execution effect, and a validated repair are different predicates.

V8's inspected JS-Fuzzer source provides a concrete continuing implementation: it consumes a corpus, generates JavaScript cases, executes shells, and supports differential testing and failure minimization [334]. Its current documentation also describes a local crash runner. A differential mismatch need not be a security vulnerability: nondeterminism, unsupported flags, or permitted semantic variation can produce disagreement. Deduplicate by a justified root-cause criterion rather than counting every crashing input as a distinct defect. Compare repeated campaigns with equal CPU-time envelopes and include non-discoveries; a successful anecdote gives no estimate of search efficiency.

Fault-model alignment supplies a separate axis. SbxBrk's CCS 2025 study targets V8 software-based fault isolation by mutating data as trusted code loads it from the untrusted heap [335]. This deliberately assumes prior control inside the sandbox; ordinary front-end input fuzzing tests a different boundary. A correct conclusion is therefore conditional isolation failure under that fault model, not a complete end-to-end compromise probability. In an agent sandbox, the analogous research question is whether trusted wrappers remain safe when an inner computation is already adversarial. Prompt obedience cannot certify that boundary.

TYPEPULSE, in USENIX Security 2025, instead analyzes Rust type conversions, trait bounds and pointer aliases, constructs a property graph, and checks patterns for misalignment, inconsistent layout and mismatched scope [336]. It is a static-analysis mechanism, not an LLM agent benchmark or a guarantee for all unsafe Rust. Findings require confirmation under the analyzer's abstraction and affected program version; absence of a finding is not a completeness certificate. The cross-layer deduction is to retain distinct evidence channels: static candidate, dynamic reproduction, effect attribution, patch test, and containment result. An agent should advance a claim only as far as its strongest validated channel supports.

28.6 Blue-team detection containment and recovery

Blue-team controls operate at several boundaries. Input and retrieval controls preserve provenance and reduce untrusted instruction promotion. Tool controls enforce schemas, capabilities and principals. Runtime controls isolate processes, credentials and filesystems. Memory controls quarantine suspicious records and propagate corrections. Monitoring and recovery detect violations that prevention missed. These controls are complementary, but each has a cost and a specific failure surface.

Detection should be evaluated separately from prevention. A monitor can identify an unauthorized proposal that the tool layer already blocked, or miss an effect that occurred through an unmonitored channel. Report detection coverage, false alarms, latency and whether the intervention arrived before the effect. A high detection rate after irreversible disclosure is not equivalent to successful containment.

Quarantine protects future decisions while preserving evidence. A suspicious memory or skill can be excluded from active retrieval until reviewed. The system should retain provenance and dependencies so related records can be examined. Automatically deleting the evidence can prevent root-cause analysis; leaving it available to the model can continue the compromise. The quarantine namespace and review authority must therefore be application-controlled.

Recovery includes credential revocation, cancellation, state restoration and external-effect reconciliation. Restoring a clean checkpoint does not undo information already disclosed or actions already accepted by remote services. Report residual effects explicitly. A rollback that repairs local files but leaves a malicious scheduled task active has not restored the operational boundary.

The proposed experiment injects controlled failures at each layer and measures useful completion, violations, detection delay, containment and recovery. Include adaptive attacks against the defense configuration, not only payloads used to design it. Compare the full stack with single-control ablations under the same benign workload. This identifies where defense-in-depth supplies complementary protection and where several components merely repeat the same fallible classification.

28.6.1 Detection must precede the relevant commitment

Let T_A be the time a validated alert becomes actionable, $L_C \geq 0$ the containment delay, and T_E the counterfactual commitment time without that containment, under the declared fixture. A conservative timing requirement is $T_A + L_C < T_E$; an equality case depends on the service's ordering semantics. It is not sufficient: the containment action must control the actual effect channel and remain authorized. A prevented effect has no observed commitment timestamp, so its deadline must come from the fixture or a justified matched replay rather than be invented from the defender's narration. Reporting only alert recall discards these conditions. An asynchronous monitor can recognize every malicious trajectory and still prevent none if dispatch commits before the alert arrives.

Use service-side commitment receipts where available rather than the

defender's message timestamp. For delayed or multi-stage effects, define the relevant point separately for each consequence: revoking access can stop future reads while leaving an earlier disclosure irreversible. Evaluate attempted violations, completed effects, alerts, successful containment and residual consequences as separate events. The measured prevention fraction is conditional on the fixture's commit and cancellation semantics. It cannot be inferred by applying a monitor score to a different tool whose effects commit synchronously or whose cancellation behavior is unavailable. This connects blue-team evaluation to the effect journal of Chapter 20 and the operational recovery analysis of Chapter 32.

28.7 Security metrics and evidence interpretation

Attack success requires a clear denominator. Per-payload, per-task, per-attacker-budget and per-session rates answer different questions. A red team allowed many attempts can have a high probability of finding one failure even when most attempts fail. A defender that abandons the benign task can reduce observed attack success while destroying utility. Report both attacker outcomes and legitimate-task outcomes under the same protocol.

Deployment risk also includes exposure and non-adversarial failures. For a declared operational violation $V \in \{0, 1\}$ and encounter with a specified attack channel $E \in \{0, 1\}$, the same deployment law obeys

$$\Pr(V = 1) = \Pr(E = 1)\Pr(V = 1 \mid E = 1) + \Pr(E = 0)\Pr(V = 1 \mid E = 0). \tag{28.4}$$

Terms with zero encounter probability are interpreted through their zero joint contribution. A benchmark attack-success rate estimates neither the exposure probability nor, without transport assumptions, the deployment conditional rate. Moreover, faulty task interpretation or stale permissions can cause violations when $E = 0$. Report adversarial resistance, benign utility, and non-adversarial constraint failures separately instead of converting a stress-test rate directly into an operational breach forecast.

Severity and reach also matter. Attempted access to a synthetic forbidden record, actual controlled disclosure, persistent policy corruption and cross-tenant effects are different outcomes. Do not reduce them to one binary score without retaining categories. If severity weights are used, justify them and show sensitivity. A small average improvement can coexist with a serious regression in a rare but consequential class.

Statistical units follow the threat model. Multiple payloads generated from one template are not independent attack families. Several attacks against one workflow share environment and policy structure. Confidence

intervals and holdouts should preserve these dependencies. Transfer to new tools, tasks and model versions needs direct testing rather than extrapolation from the number of attempted strings.

An implementation ledger connects attack surface, defender boundary, policy decision, actual effect and judge verdict. A failure is especially informative when counterfactual replay shows that changing one boundary prevents the violation while preserving the task. This is stronger than a screenshot of a harmful output because it identifies a mechanism. The ledger also supports regression testing after SDK or tool updates.

The resulting conclusions should remain conditional. A passing suite establishes resistance to tested adversaries under the recorded budget and environment. A successful attack identifies at least one failure in that setting. Neither result alone determines deployment risk across all users and tasks. Security research becomes actionable when it identifies enforceable boundaries, concrete bypass conditions, and measured utility costs without inflating finite evidence into universal assurance.

28.8 A bounded policy-expansion and memory-poisoning case

Consider a controlled document-assistant fixture. The legitimate task is to summarize two approved project documents and save an internal report. The attacker can modify one retrieved document but cannot change the user's request, the authorization service, or the tool implementation. The fixture contains a synthetic restricted record and a controlled output sink. The prohibited event is access to that record or disclosure beyond the permitted audience. No real credentials, users or services are involved.

The attacker-controlled document may attempt to redefine the task or persuade the agent that broader access is needed. The defensive question is not whether the model recognizes the text as suspicious in isolation. It is whether the complete system can continue the legitimate task while preventing unauthorized effects and contamination of persistent memory. This preserves the utility requirement and the stateful attack surface that a simple refusal benchmark omits.

The initial policy permits reads of the two approved document identifiers and creation of one report in the authorized workspace. A proposed update P' is checked against the current policy P. A symbolic expansion test searches for an action a satisfying $P'(a) \wedge \neg P(a)$. A satisfying assignment exhibits a newly allowed call. Unsatisfiability supports narrowing only within the encoded domain and solver assumptions. A timeout or unknown

solver result is not evidence that the update is safe.

This formulation makes the policy boundary auditable. The application can record the candidate update, solver query, result, counterexample where available, and approval decision. The model may help propose the policy but does not decide whether the formal check succeeded. Progent's policy comparison supplies a primary mechanism for this style of confinement [24]. Its guarantee should remain tied to the mediated call set rather than be expanded into a claim about every possible downstream effect.

The fixture should include a wrapper whose apparent document-read interface can reach another resource. This tests whether top-level policy enforcement matches downstream effect semantics, rather than assuming the schema exhausts the tool's behavior.

Bind approval to an object version or revalidate it at dispatch. A revocation injected after proposal but before execution tests whether a paused worker can improperly restore an old authorization epoch.

Now extend the fixture across sessions. Suppose the memory writer extracts a procedure from the attacker-controlled document and stores it as a helpful workflow. The immediate task may complete without a violation, yet the persistent record can influence a later task. The attack outcome must therefore include delayed effects and attempted promotion into trusted memory. A one-turn security score can miss this route entirely.

Quarantine must apply to derivative summaries and skills as well as the injected source; subsequent semantic search should not bypass the promotion decision.

Table 28.1 identifies the observable chain in the fixture. Each row is independently testable and prevents a successful final artifact from hiding an earlier boundary violation.

Table 28.1: A defensive trace for a controlled stateful injection test.

Stage	Trusted observation	Required distinction
Acquisition	Source ID, version and attacker-controlled fields.	Useful document evidence versus task authority.
Context construction	Serialized roles, provenance and active policy epoch.	Quoted content versus application instructions.
Policy update	Candidate rule set, solver verdict and approval record.	Narrowing, expansion and unresolved comparison.
Tool dispatch	Principal, exact arguments and resulting fixture state.	Proposed violation, blocked attempt and completed effect.

Stage	Trusted observation	Required distinction
Memory write	Record type, source lineage and promotion status.	Untrusted observation versus reusable approved procedure.
Later retrieval	Retrieved records, validity and reader context.	Immediate containment versus delayed influence.
Recovery	Revocation, quarantine and derivative invalidation.	Local rollback versus residual external effects.

The ablation design compares model-only guidance, deterministic tool policy, memory provenance controls, and their combination. Keep the benign task, model, tools and attack budget fixed. A separate adaptive phase lets the attacker observe a declared subset of feedback and search within the same controlled scope. Hold out document styles, workflow structures or later-session tasks for transfer assessment. Otherwise a defense can learn the specific fixture while remaining vulnerable at the same semantic boundary elsewhere.

The primary outcomes are legitimate report completion, attempted and completed prohibited effects, unsafe memory promotion, delayed violations, false alarms, and review cost. A blocked malicious call is evidence of enforcement even if the model proposed it. A final benign answer is insufficient if an unauthorized read already occurred. A quarantined record is useful containment but may still require an incident response if it was previously shared. These distinctions make the score reflect actual effect semantics.

Validate the judge against authoritative fixture state and explicit positive, negative and unresolved controls. The defender's narration is not independent evidence of containment.

Recovery tests should inspect pending effects and downstream artifacts after revocation and clean-state restoration. A local rollback cannot retract a disclosure already received by the controlled sink.

The case supports a bounded scientific conclusion. If the policy layer prevents every tested expansion and the memory layer prevents delayed promotion under the declared attacks, those boundaries performed as specified in the fixture. The result does not establish immunity to attacker-controlled tools, compromised infrastructure, unseen action channels or a different policy language. The value of the experiment lies in identifying which assumptions were enforced, which were merely trusted, and which failures remain outside the observed boundary.

The capability benchmarks illustrate a parallel observation boundary.

CyberGym and CyberGym-E2E focus on memory-safety issues in C/C++ projects and use sanitizer-based evidence; that scope does not cover every vulnerability class [331, 25]. ExploitGym separates evidence of unauthorized code execution from a judge's assessment that the result targets the supplied vulnerability [323]. The first stage can be strongly instrumented while the second retains model-judgment uncertainty. Reporting both stages prevents a concrete runtime effect from lending unwarranted certainty to its causal attribution. A defensive evaluation should make the same separation between an observed prohibited effect and the claim about which adversarial channel caused it. Counterfactual replay and independent adjudication strengthen that attribution but remain conditional on the fixture and observation coverage.

Chapter 29

Alignment, Authorization, and Constrained Autonomy

OpenAI Agents Python v0.24.0's October change to defer automatic compaction during pending approvals exposes an alignment problem with a concrete runtime boundary [184]. A review request, the evidence it refers to, and the proposed effect must remain interpretable until a decision is applied. The release note establishes an implementation change, not universal approval correctness. Combined with Progent's distinction between policy narrowing and expansion, it motivates treating objectives, evidence, authority, and dispatched effects as separate versioned commitments [24].

Alignment concerns how system behavior relates to intended objectives, preferences and constraints. Authority concerns who may define or change those objectives and which effects the system may cause. The distinction is essential for agents. A model can correctly infer a user's preference while lacking permission to act on it; an application can enforce permissions while optimizing an inappropriate proxy. Reliable autonomy requires both a useful policy and a current, enforceable account of authority.

This chapter connects training objectives to operational control, human review, privacy, memory and deployment decisions. It treats approvals, monitors and evaluators as fallible components. No finite evaluation establishes universal alignment, and no role label or persistent identity establishes that a system understands organizational intent. The practical question is which constraints can be enforced, which judgments can be measured, and which uncertainties remain material to deployment.

29.1 Objectives proxies and constrained policies

A sequential objective can maximize expected discounted task reward, but the reward is a representation of intent rather than intent itself. Outcome labels may omit constraints, reward models may favor presentation, and benchmark graders may accept shortcuts. An agent optimized against such feedback can improve the measured objective while worsening the underlying task. This is a specification and evaluation problem as well as a

learning problem.

Let r_t denote task reward and $c_{j,t}$ a cost or constraint-violation signal. A constrained formulation is

$$\max_{\pi} \ \mathbb{E}_{\pi} \sum_{t} \gamma^t r_t, \qquad \mathbb{E}_{\pi} \sum_{t} \gamma^t c_{j,t} \leq d_j. \tag{29.1}$$

For this infinite-horizon expression, $t \in \mathbb{N}_0$, $\gamma \in [0, 1)$, $j \in \{1, \ldots, J\}$ indexes a finite set of constraints, $d_j \geq 0$ are thresholds, and rewards and nonnegative costs are bounded. A finite-horizon study instead truncates both sums at its declared stopping time. The constraints remain meaningful only if the signals and thresholds represent the desired properties. A positive expected-cost budget can permit rare severe violations. An exactly zero expectation of nonnegative costs would exclude positive costs almost surely at positively weighted times under the modeled law, but a finite sample of zero observed costs does not establish that premise; $\gamma = 0$ does not constrain later times at all. An authorization requirement may instead demand that every dispatched action belong to the current allowed set. That property should be enforced at the effect boundary rather than traded against reward through an arbitrary finite penalty.

Reward shaping and refusal training can reduce undesirable proposals, but they do not establish complete mediation. A model can encounter a novel representation or be placed inside a harness that grants broader tools. Conversely, a runtime policy can block unauthorized effects even when the model proposes them. Evaluate proposal behavior and executed behavior separately so a successful control layer is visible without misrepresenting the model's alignment.

Preference learning introduces population and context questions. Whose preferences are represented, under what task, and with which exceptions? A user preference stored months earlier may conflict with a current instruction or an organizational rule. Treating all preferences as timeless facts creates predictable errors. A policy should represent scope, provenance and update authority rather than simply weight the most frequently retrieved statement.

The practical implication is to separate hard constraints, uncertain judgments and optimization objectives. Hard constraints require application-enforced checks where possible. Uncertain judgments require calibrated evidence and review. Objectives guide useful behavior within those boundaries. Collapsing the three into a single textual instruction or scalar reward makes it harder to identify which mechanism failed.

29.2 Instruction hierarchy and authority-bearing records

An instruction hierarchy organizes competing directives by source and scope. Its implementation must distinguish an authorized user request, application policy, tool metadata, external evidence and model-generated plans. Text that describes a rule is not automatically an authoritative rule. A retrieved document can quote an instruction without being allowed to change the agent's task or permissions.

Authority-bearing records should identify the principal, action scope, affected objects, validity interval and policy epoch. An approval to edit a draft does not authorize sending it to an external recipient. Approval of one argument set does not authorize a later modified request. The runtime should bind approval to the exact operation or to a clearly specified capability envelope, then revalidate at dispatch.

Historical authority and current authority differ. Session logs preserve what was requested or granted earlier; the authorization service determines what is allowed now. Restoring a checkpoint should not resurrect expired or revoked grants. This separation can be enforced structurally even when the model's context contains the old instruction. The tool boundary should consult current authority independently of the model's interpretation.

The same principle applies to delegation. A manager may delegate reasoning without delegating every tool capability or every source document. Workers should receive the minimum context and authority needed for their subtask. A handoff must preserve restrictions and identify who remains responsible for the final effect. Internal agent messages should not become a privileged channel for promoting untrusted content.

Instruction-hierarchy evaluation should include conflicting but plausible directives, ambiguous provenance, changed permissions and resumed sessions. Measure whether the system identifies the governing source, requests clarification when necessary, and prevents unauthorized effects. A benchmark consisting only of obvious malicious strings tests a narrower capability. It cannot establish correct authority reasoning in ordinary multi-party workflows.

29.3 Constitutional and preference-based alignment

Post-training can shape how a model interprets requests, critiques outputs and handles uncertain or harmful tasks. The 2022 Constitutional AI procedure provides a foundational example of using stated principles and AI feedback in training [337]. The contribution concerns a training procedure and its evaluated behaviors. It does not establish that a deployed agent automatically follows every organizational policy or that a principle list resolves all contextual conflicts.

Preference optimization depends on the data-generation and judging process. If preferred responses are selected for fluency, completeness or agreement, a model can learn to present unsupported confidence. If refusals are rewarded too broadly, useful benign tasks may be blocked. Evaluate both harmful compliance and benign-task retention under the actual tool environment. A model-only conversation study and an effectful agent deployment have different action spaces.

AI-generated critique is useful feedback but can share the original model's blind spots. A critic may miss a constraint, infer a requirement not present in the task, or agree with a misleading rationale. Independent executable evidence can complement critique where available. For semantic or ethical judgments without a complete oracle, report the adjudication procedure and disagreement rather than treating the critic's role as proof of correctness.

Training can also change memory interpretation. A new policy may give stronger weight to persistent preferences or be more skeptical of retrieved instructions. Existing skill banks and summaries were written under earlier behavior. An alignment update can therefore improve one task family while producing compatibility regressions elsewhere. Model migration tests should include old memory, conflicting updates and changed authority, not only fresh prompts.

The relevant causal comparison holds the harness and information budget fixed while varying the trained policy, then separately tests interactions with context and memory mechanisms. If a post-training method changes output format or tool-call tendencies, the parser and evaluator may need compatible configuration. Report those changes rather than attribute every difference to an abstract quality called alignment.

29.3.1 Automated safety adaptation and independent confirmation

A3's March 2026 report supplies a concrete automated safety-finetuning loop: generate hypotheses around a failure, construct harmful and benign counterparts, tune LoRA and data mixtures, and retain experiment results for subsequent choices [338]. Its reported experiments primarily concern Qwen-2.5-7B Instruct, with one additional Llama setting, rather than the October frontier-model population. This is evidence for targeted adaptation under the tested feedback loop, not a result that arbitrary safety properties can be repaired automatically or transferred without regression.

The code reveals a material evaluation boundary. In the inspected `IterativeLoRASFTAgent`, `select_hyperparameters` consumes previous results; the selection prompt explicitly prioritizes constraints on both validation and OOD performance. The experiment log written by `update_experiment_log` includes OOD aggregate and per-hypothesis outcomes [339]. Therefore hypothesis-disjoint OOD data are not equivalent to an untouched final assessment when their performance can guide later iterations. This does not imply that OOD examples were inserted into training; the selection prompt separately restricts training weights to training hypotheses. The issue is adaptive evaluation feedback, which changes the independence claim even when example-level splits remain intact.

Let D_{dev}, D_{shift} and D_{confirm} denote development, shift-diagnostic and final-confirmation populations. An optimizer may legitimately use the first two to choose a candidate, but confirmation requires the selected artifact and procedure to be fixed before revealing the third population's outcomes. Repeated shift diagnostics help discover failure modes; they cease to be untouched evidence as soon as they influence selection. Record every query and rejected candidate, include benign refusal and capability regressions, and repeat with independent hypothesis generators and judges. The update should then be tested with old memory, pending approvals and existing tool schemas, because targeted output improvements do not establish compatibility with a stateful deployment.

Diffuse AI Control supplies a complementary 2026 warning about optimizing weak scorers on difficult-to-grade research proposals [340]. Its red team searches for proposals that a weak scorer rates highly while a stronger information-privileged proxy rates poorly. The stronger scorer is itself a proxy, and the improved blue-team prompts are found using access to that proxy. The result therefore demonstrates a configured oversight failure and a proof-of-concept mitigation under privileged search, rather than a

generally available guarantee against research sabotage. The appropriate synthesis is to improve the feedback channel, isolate final assessment and enforce authority boundaries; automating the optimizer does not remove the need to validate the criterion it optimizes.

29.4 Human review and the semantics of approval

Human review can supply judgment, authority or both. A reviewer may assess whether an artifact is technically sound, while a different principal authorizes publication. Combining these decisions into one button can create ambiguity. The interface should state what is being reviewed, which effect is being authorized, and what evidence remains uncertain. Approval should not be inferred from silence, elapsed time or a historical response to another request.

Review is a limited resource. Long agent-generated explanations can overload the reviewer or hide decisive facts among plausible detail. A useful review surface presents the exact proposed effect, artifact identity, scope change, evidence, failed checks and unresolved consequences. It should distinguish facts derived from execution records from claims generated by the model. This supports informed intervention without pretending that a human inspects every token.

The October Agents SDK release notes explicitly defer automatic compaction while approvals are pending [184]. This is a concrete interaction between context management and authority: changing the context during an unresolved approval can remove or alter the evidence needed to interpret it. The fix is version-specific evidence about one boundary, not a universal solution to approval correctness. Applications must still bind the approval to the action and current policy epoch.

A review experiment can seed defects and vary the presentation of evidence under a fixed time budget. Measure detection, false rejection, decision time, and whether approved actions match what reviewers believed they approved. Also test changes after approval: if the artifact digest or arguments differ at dispatch, the old approval should fail validation. This is a mechanical invariant independent of the reviewer's technical skill.

The check must also have a declared commitment point. Write a grant as $g = (p_g, \mathcal{C}_g, \alpha_g, t_g)$, binding principal, approved operation envelope, policy epoch, and expiry. For operation a and current principal p, a conservative commit-time admission predicate is

$$\mathbf{1}\{p = p_g,\ a \in \mathcal{C}_g \cap F_{\alpha(t)}(t),\ \alpha(t) = \alpha_g,\ t \le t_g\}. \tag{29.2}$$

Here an operation includes its arguments, object versions and purpose; $F_{\alpha(t)}(t)$ is the current authoritative feasible set. Requiring epoch equality intentionally invalidates old grants after any governing policy change; a service may instead support a formally specified revalidation rule. Checking this predicate before an unrelated remote write leaves a race between check and use. Enforcement needs a version-conditional or otherwise atomic service contract at the relevant effect boundary. If the service exposes no such primitive, report the residual race and reconciliation behavior rather than claim that an earlier approval proves current authorization. Cancellation cannot retract an effect already committed under the service's own ordering rules.

Escalation policies should distinguish missing preference, missing authority and missing evidence. A user can clarify desired behavior but may not have permission to override an organizational restriction. An administrator can grant a capability without establishing that the proposed action is technically correct. The system should route the question to the appropriate role rather than asking an undifferentiated human oracle to resolve every uncertainty.

29.5 Privacy purpose and the lifecycle of memory

Privacy depends on collection, use, sharing, retention and deletion, not only on whether storage is encrypted. An agent may retrieve a permitted document for one purpose and later reuse its summary in another context where that use is inappropriate. A memory record therefore needs subject, source, purpose, audience, validity and retention metadata in addition to content. These fields influence both retrieval and action authorization.

Purpose limitation is difficult to infer from natural language alone. A broad request to help with a project does not necessarily authorize exporting every related document. Application logic should define the available capability envelope, and the agent should request additional authority only when the task requires it. The request should identify the specific effect and evidence, not seek a blanket expansion that persists indefinitely.

Deletion has several meanings. Logical exclusion prevents future retrieval; physical removal deletes stored bytes; derivative invalidation addresses summaries, embeddings and skills; parameter unlearning concerns information absorbed into model weights. Completing one does not establish the others. A system should accurately report which action occurred and which copies or retention obligations remain. External disclosure cannot generally be undone by deleting the local source.

Shared memory raises multi-user conflicts. A procedure learned from one user's private task may be useful to another, but utility does not grant permission to share it. Abstraction can still reveal sensitive details or encode privileged assumptions. A promotion process should check whether the generalized skill contains source-specific information and whether its reuse is authorized. Evaluation should include cross-user leakage and benign transfer rather than reward sharing alone.

Privacy-aware traces require a similar balance. Detailed records support auditing and reproducibility but can expose the very content the system should protect. Separate access-controlled raw evidence from redacted operational metadata, define retention, and preserve stable identities so authorized auditors can join records. If redaction removes decisive evidence from public artifacts, state the resulting verification limit rather than imply full independent reproducibility.

29.6 Operational constraints and residual risk

Operational alignment includes budgets, deadlines, service objectives and recovery authority. A model can pursue a task too aggressively, continuing to consume resources after progress becomes unlikely or repeating actions after cancellation. Hard budget enforcement should reside outside the policy. The policy can allocate resources within the envelope, but it should not be able to redefine the envelope through persuasive narration.

Risk-sensitive objectives may emphasize tail losses rather than average success. A deployment can tolerate occasional formatting errors while requiring much stronger control of disclosure or destructive actions. Expected utility alone can hide that asymmetry. Constraints, review gates and effect-specific authority provide a more transparent mechanism than one global confidence threshold. Their effectiveness still needs empirical validation and explicit assumptions.

Monitoring detects drift in both behavior and environment. Changes in tool schemas, model versions, user populations or memory contents can invalidate earlier evaluations. A monitor should identify the version and state associated with an alert, support containment, and preserve evidence for diagnosis. A conceptual AgentOps framework organizes observation and automation needs [341]; operational benefit requires implementation and measured workload evidence beyond the framework's taxonomy.

Rollback must include compatible model, prompt, harness, tool and memory versions. Restoring only model weights can leave skills written for a later policy or an index built with another representation. External effects

require reconciliation and may remain irreversible. The authority to initiate rollback should itself be specified, especially when several organizational owners control different components.

Incident learning should avoid feeding unverified explanations directly into persistent memory. A model's account of why a failure occurred is a hypothesis. Validate it against logs, environment state and counterfactual tests before promoting it into a reusable rule. Otherwise an operational failure can become a misleading skill that causes future regressions. This is the same evidence-promotion problem encountered in research agents, now applied to deployment governance.

29.7 Evaluation of authority and aligned behavior

An authority evaluation needs tasks where the desired outcome and permitted action set are explicit. Include legitimate completion, conflicting instructions, expired grants, revoked access, ambiguous requests, delegated workers and pending effects. The evaluator should observe actual tool dispatch and environment changes, not only the final answer. A policy that verbally refuses but still causes an unauthorized effect has failed the operational constraint.

Separate model judgment from enforcement. Measure whether the model proposes appropriate actions, whether the runtime blocks disallowed ones, and whether useful work completes. This decomposition identifies complementary improvements. Better model behavior can reduce review and rejection load; stronger enforcement can contain residual mistakes. A combined score without these components cannot explain which layer supplies reliability.

Human-AI collaboration frameworks can clarify role expectations and degrees of agency [342]. Their conceptual categories should not be treated as empirical proof of trust, shared intentionality or appropriate delegation. Evaluate the actual workflow: what the human understands, what the agent can change, and whether authority remains legible after interruptions and handoffs. Anthropomorphic language can otherwise conceal concrete responsibility gaps.

The statistical design should preserve user and organization clusters, because policies and preferences are shared within them. Repeated prompts against one policy do not establish cross-organization generalization. Include heterogeneous task families and report failures by authority type and consequence. Adaptive tests should search for boundary violations under declared budgets while benign tests measure overblocking and review

burden.

The conclusion should be a bounded operational claim. A system can demonstrate useful behavior under specified objectives, constraints, principals and environments. It cannot infer unrestricted alignment from a high benchmark average, a constitutional prompt or a human-review stage. The scientific task is to make those conditions explicit, measure where they hold, and retain evidence about where they fail.

29.8 A governed model-and-harness update

Consider an agent authorized to improve an internal research workflow. It can propose prompts, context policies, tool adapters and reusable skills, then evaluate candidates on a development set. It cannot change the final assessment set, grant itself new data access, or deploy a candidate without the designated release authority. The desired outcome is better verified research quality within a resource envelope, not merely a larger development score. This case brings alignment, scientific validity and operational authority into the same concrete transition.

The first boundary is between proposal and commitment. Candidate changes are stored in an isolated workspace with a manifest identifying model parameters, harness revision, prompts, tools, memory schema and evaluator configuration. Running an experiment does not authorize promotion. A model can describe a candidate as successful, but the release record must refer to the exact artifact and evidence reviewed. If the candidate changes after review, the old decision no longer applies automatically.

The second boundary is between development and assessment. The agent may inspect development failures to form hypotheses, but an untouched assessment set must remain outside its ordinary tools and memory. If it sees the assessment outcomes repeatedly and modifies the workflow accordingly, that set becomes an optimization resource. This remains true when only prompts or skills change. Frozen model weights do not preserve statistical independence of the final evaluation.

A scalar reward cannot by itself guarantee this separation. Suppose the agent's objective is task benefit minus a penalty λ for a prohibited evaluation change. If a candidate can obtain benefit greater than the finite penalty, the penalized objective can prefer the prohibited change. Even if the intended benefit is bounded, the implementation may mismeasure it or expose an exploitable proxy. A hard application constraint on evaluator modification is therefore a different mechanism from discouraging such modification through reward.

This argument does not imply that all behavioral constraints must be expressed as low-level rules. Many aspects of research quality, honesty and relevance require judgment. It implies that constraints with clear effect boundaries should be enforced there, while uncertain semantic judgments are evaluated with evidence and review. Treating both as one preference score obscures whether a failure arose from missing authority checks or from an inadequate quality model.

The third boundary is memory compatibility. A candidate harness may interpret existing skills differently, or its context constructor may omit provenance that the previous policy relied on. Evaluating only fresh tasks from empty memory can miss these regressions. The release test should include saved longitudinal states, contradictory evidence, old procedural records and permission changes. Passing a schema migration establishes representational compatibility, while behavior under the migrated state requires separate tests.

Table 29.1 organizes the commitments. Each transition has an owner and an evidence obligation. The table is a governance design rather than a claim that any named product implements the entire process.

Table 29.1: Authority and evidence in a controlled agent update.

Transition	Authority boundary	Evidence obligation
Propose candidate	Agent may edit only declared development components.	Immutable diff and manifest identify every changed dependency.
Run development	Experiment budget and data grants remain external controls.	Complete outcomes include failures, costs and rejected candidates.
Assess candidate	Final evaluator and holdout remain independently controlled.	Verdict binds to the fixed candidate and declared test population.
Review release	Technical review and deployment authority are identified separately.	Reviewer sees regressions, unresolved evidence and scope changes.
Commit release	Approval binds to artifact digest and current policy epoch.	Dispatch checks that the reviewed object is still the deployed object.
Monitor and revert	Operational owner can contain or restore within a declared envelope.	Traces preserve affected tasks, state versions and residual effects.

An interruption can expose a subtle approval failure. The agent prepares a release, obtains a pending review request, compacts its history, and later

resumes after the reviewer responds. If compaction removed the exact candidate or scope, the response can be applied to the wrong operation. The release-specific SDK behavior discussed earlier provides evidence that pending approvals and context transformations interact in real implementations. The general invariant is stronger: approval identity and scope must be stored outside a lossy summary and revalidated against the current candidate.

The reviewer should not receive only the agent's preferred narrative. Present the full candidate manifest, comparison protocol, aggregate and stratified outcomes, negative results, changed permissions, and untested conditions. A model-generated recommendation can accompany these records but should be labeled as interpretation. This reduces the chance that persuasive prose becomes a substitute for evidence. It also makes disagreement between technical review and deployment authorization explicit rather than forcing both into one approval state.

The proposed experiment injects a candidate change after approval, an evaluator-version mismatch, a stale skill bank, a revoked data grant and a delayed review response. Compare text-only approval tracking with digest-bound records and current-epoch validation. Measure unauthorized promotion, stale-evidence acceptance, valid-release completion, review burden and recovery cost. The experiment should include benign updates so a system that blocks every release does not appear successful merely because no unsafe update occurs.

Failure attribution should follow the transition chain. If the assessor read the wrong artifact, the problem is evidence identity. If it read the correct artifact but missed a regression, the problem is evaluator coverage or judgment. If the release service ignored an expired approval, the problem is enforcement. If the agent optimized the wrong outcome while obeying all permissions, the objective or specification needs revision. These diagnoses require different interventions and should not collapse into an undifferentiated alignment failure.

The case also exposes responsibility for residual uncertainty. A release authority can decide that available evidence is sufficient for a bounded deployment, but that decision does not convert unknown behavior into known safety. Monitoring and rollback reduce some consequences while leaving irreversible effects and unobserved regressions possible. The assurance record should retain those limitations and identify which new observation would trigger containment or reevaluation.

Useful autonomy is therefore a relationship among a policy, its authorized action space, evidence-producing evaluators and accountable operators.

Training improves the policy's choices within that relationship. Context and memory preserve the information those choices require. Runtime controls prevent stale or unauthorized commitments. Scientific evaluation measures the resulting behavior under declared conditions. The research goal is to strengthen each connection while resisting the temptation to let success at one layer stand in for assurance about the others.

A release record should also identify the scope of any claimed guarantee. Passing a fixed regression suite supports behavior on that suite; a proof of a permission invariant supports the modeled action boundary; a reviewer decision records an authorized judgment. These records can coexist without being interchangeable. The release service can require all three for a particular effect while still preserving their separate failure modes. If one dependency changes, invalidate only the claims whose premises changed and rerun their associated checks. This dependency-aware assurance is more scalable than repeating every test after every edit, and more defensible than assuming that an unchanged product name implies unchanged behavior.

The approval-to-commit gap in Figure 29.1 persists unless the service binds validation to commitment.

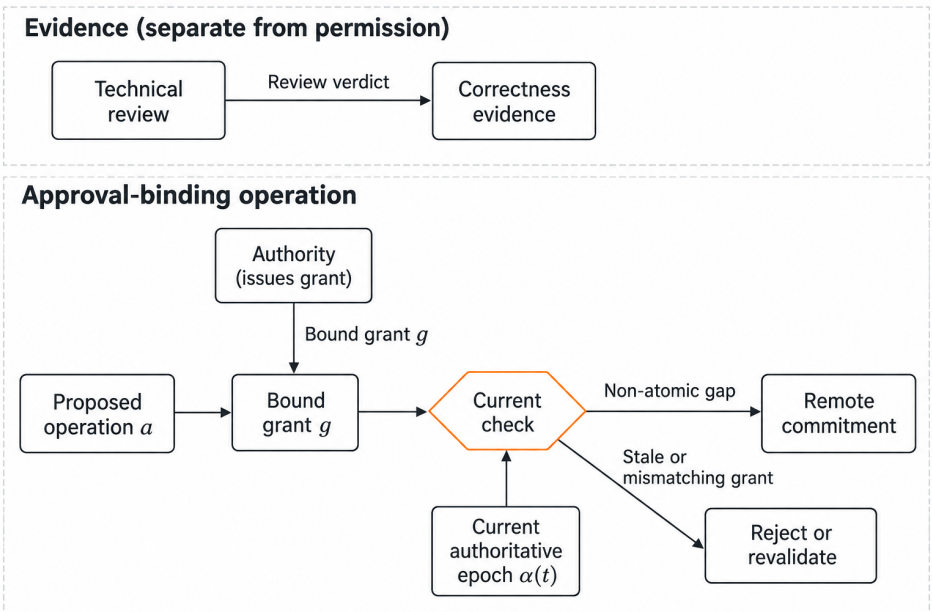

Figure 29.1: Technical review supplies evidence, while an authorized principal grants a specific operation envelope. A current check must bind the principal, arguments, object versions, epoch and expiry. If validation and remote commitment are separate, the service contract must address the remaining race; an earlier approval does not prove current authority or technical correctness.

Chapter 30

Continual Adaptation and the Evidence for Recursive Self-Improvement

30.1 What changes when an agent improves

An improvement claim is incomplete until the mutable object is identified. A system can change its active context, persistent records, procedural skills, execution harness, environment curriculum, verifier, or model parameters. These changes have different costs, failure modes, and transfer requirements. Updating a skill index changes which procedures are retrieved; updating a model changes how every retrieved procedure is interpreted. A harness search can improve a frozen policy without producing any new parametric capability. Distinguishing the update target is necessary for attributing a result and for deciding whether the resulting state can be safely reused.

Let $\xi = (\theta, h, m, v, e)$ collect model parameters, harness configuration, memory, verifier, and environment-generation rules. An improvement operator $\mathcal{I}_\omega$ proposes a new configuration from an evidence set D and resource budget B:

$$\xi' = \mathcal{I}_\omega(\xi, D; B). \tag{30.1}$$

An accepted update is a decision made by an acceptance rule, not a mathematical guarantee that quality increased. Let $\mathcal{U}_\omega(\xi, D; B)$ denote the complete proposal-and-acceptance procedure: it returns the selected configuration, or the unchanged incumbent when all proposals are rejected. Its budget includes the acceptance computation. The relevant quantity is held-out utility $J(\xi; \mathcal{D})$ under a declared task population and operational cost boundary. An update can improve its selection score while degrading J if it exploits grading weaknesses, changes the distribution, or consumes more unreported resources.

Recursive self-improvement adds a second question: does the change improve the process that will generate later changes? Define executor utility $J(\xi)$ and improver utility

$$K(\omega; \xi, B) = \mathbb{E}\left[J(\mathcal{U}_\omega(\xi, D; B)) - J(\xi)\right]. \tag{30.2}$$

The expectation includes task sampling, proposal randomness, evaluation noise, and the acceptance policy. A positive change in J does not imply a positive change in K. A patch can solve a particular recurring failure while making the next search process less diverse, more expensive, or more vulnerable to its evaluator. Evidence for improved execution and evidence for improved improvement must therefore be reported separately.

Table 30.1: Mutable adaptation targets and distinct validation obligations.

Update target	Observable mutation	Distinct validation obligation
Context	Tokens selected, compressed, or reconstructed	Decision preservation and correct behavior-policy conditioning.
Memory	Facts, episodes, links, or retrieval indexes	Validity, influence, task-order effects, and deletion propagation.
Skills	Procedures and their applicability conditions	Transfer, compatibility, and permission revalidation.
Harness	Prompts, control flow, tools, middleware	Resource-matched transfer and crash/recovery behavior.
Model	Weights, adapters, or learned routers	Held-out capability, regressions, and state migration.
Verifier	Tests, rubrics, judges, or calibration	Independent false-acceptance and false-rejection evidence.
Environment	Tasks, simulations, or curricula	Grounding, coverage, difficulty, and leakage.

The taxonomy is operational. A method may update several targets, but it should identify the order and dependency of those updates. Reflection-based text memories, for example, can alter subsequent actions without changing weights. Reflexion provides an early concrete mechanism for this form of adaptation, whereas parametric post-training changes the conditional model itself [112]. Calling both learning is reasonable only if the analysis retains their different persistence, compute, and generalization properties.

Figure 30.1 separates the executor, the improver, and the independent evaluation of a frozen executor.

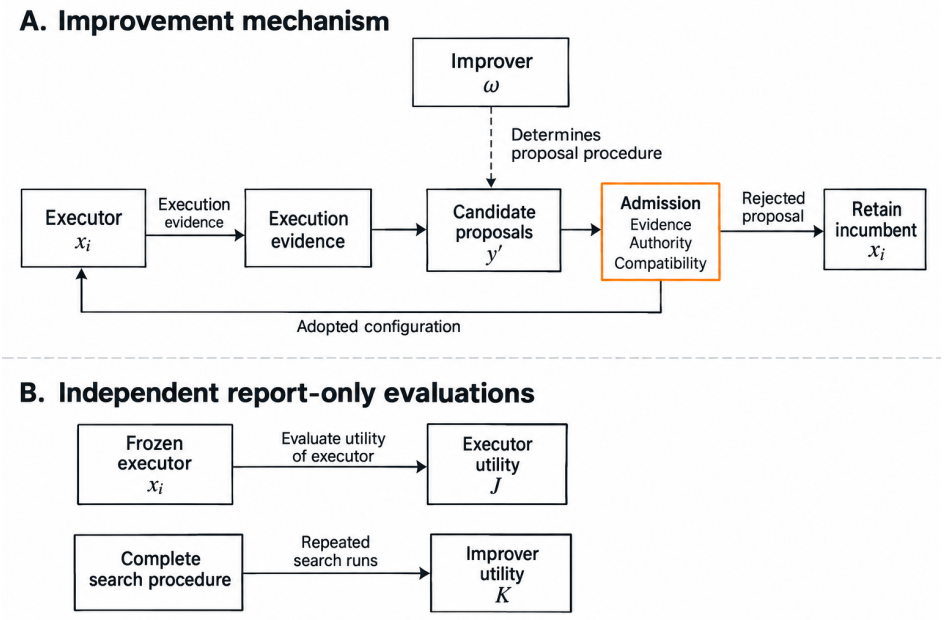

Figure 30.1: Executor utility J and the expected gain K of the complete improvement procedure are different estimands. Here x_i labels the incumbent executor configuration, y' a proposal, and ω the improver. Adoption also depends on evidence, authority and compatibility. Independent outer-loop runs and complete search costs are needed to assess improvement; a selected executor score cannot substitute for that assessment.

30.2 Selection pressure and finite-feedback overfitting

Repeated harness evaluation creates adaptive pressure on the evaluator. Let $\widehat{J}_n(h)$ be the average bounded score of harness h on n tasks. For a fixed finite candidate class $\mathcal{H}$, independent tasks, and scores in $[0,1]$, Hoeffding's inequality and a union bound give

$$P\left(\sup_{h\in\mathcal{H}} |\widehat{J}_n(h) - J(h)| > \epsilon\right) \leq 2|\mathcal{H}|e^{-2n\epsilon^2}. \tag{30.3}$$

Consequently, with probability at least $1 - \delta$, the empirical maximizer $\widehat{h}$ satisfies

$$J(h^*) - J(\widehat{h}) \leq 2\sqrt{\frac{\log(2|\mathcal{H}|/\delta)}{2n}}, \tag{30.4}$$

where h^* maximizes true utility within the fixed class. The proof applies the uniform deviation bound once to h^* and once to $\widehat{h}$, with the empirical optimization inequality between them. This is a standard finite-class deduction, not a guarantee for unrestricted harness evolution.

The fixed-class assumption is decisive. If proposals are generated after inspecting detailed errors on the same tasks, the effective hypothesis class is data-dependent. Replacing $|\mathcal{H}|$ by the number of evaluated proposals can be invalid: each proposal may encode arbitrary information extracted from the evaluation set. A code-writing agent can insert task signatures, evaluator-specific branches, or special-case instructions. More rounds increase both useful exploration and the opportunity to fit selection noise. Fresh evaluation tasks, restricted information release, complexity control, and independent final tests address different parts of this problem.

Regularization of proposals is not equivalent to regularization of outcomes. Limiting the number of edited components can improve inspectability and make causal attribution easier, but one short edit can still encode a benchmark-specific shortcut. A critic can reject obvious shortcuts but share the proposer's blind spots. Pruning a costly change can improve cost efficiency while removing a rare but important recovery mechanism. The scientific question is whether a regularizer changes transfer and failure distributions under matched resources, not whether it resembles a familiar statistical penalty.

RRSI makes this issue concrete. Its frozen backbone is surrounded by editable harness components; proposal capacity is annealed, underexplored evolution directions receive attention, and a critic/pruner constrains which edits become permanent. Version 3's Table 1 reports JobBench scores of 36.0 and 40.7 for the base and evolved harness, a 4.7-point difference; its engineering result uses the different Frontier-Eng Medal metric and reports a 4.3-point difference. The abstract's undifferentiated "up to 4.3" out-of-distribution summary is inconsistent with the larger JobBench difference in the same revision. These are author-reported bounded comparisons, not measurements reproduced here, and their units should not be pooled. The evolution set, task suites, model, selection procedure, and cost definition constrain their interpretation. The method does not demonstrate unbounded acceleration or improvements to the model weights [27].

The official repository makes this chain inspectable at a pinned revision

committed on 23 September 2026 and inspected in October. The schedule is implemented by `rrsi/schedule.py:edit_budget`; proposal construction by `rrsi/propose.py:propose`; screening by `rrsi/critic.py:precheck` and `review`; and acceptance by `rrsi/selection.py:judge` and `select_round`. The loop exposes separate round, held-out, and reevaluation methods [343]. Inspection of these functions establishes their code semantics, not reproduction of the paper's task measurements.

For $T > 0$, the edit-count schedule computes a ceiling of a cosine interpolation:

$$b_t = \left\lceil b_{\min} + \frac{b_{\max} - b_{\min}}{2} \left(1 + \cos \frac{\pi t}{T}\right) \right\rceil . \tag{30.5}$$

Here $t \in \{0, \ldots, T-1\}$ is the zero-based proposal round, and the configured bounds satisfy $0 \le b_{\min} \le b_{\max}$ with integer values. The quantity b_t bounds the number of declared independent edits in one candidate. It does not bound the number of mechanisms accumulated by the harness over its history. The inspected table generator uses rounds zero through $T-1$, so the final sampled budget need not equal the interpolation's endpoint at T. For hypothetical $T = 10$, minimum one, and maximum four, the final sampled value is two. This is calculated schedule behavior, not an observed evolution outcome. It is a useful boundary test for a reproduction that assumes the last round is necessarily a single-edit round.

The selection code has two cost regimes. Let $\Delta S = S' - S_t$, $\Delta C = (C' - C_t)/C_t$ for positive incumbent cost, ν the implementation's novelty count, and δ its configured score tolerance. Its cost-admissibility indicator is

$$a = \begin{cases} \mathbf{1}_{\{\Delta C \le \beta_0 + \beta_1 \Delta S\}}, & \Delta S > \delta, \\ \mathbf{1}_{\{w_s \Delta S - w_c \Delta C + w_n \nu > 0\}}, & \Delta S \le \delta. \end{cases} \tag{30.6}$$

The coefficients are configuration parameters; their values and the score/cost units determine the tradeoff. Additional admissibility requires $S' \ge S^\star - \delta$ and passing domain guards, where $S^\star$ is the best recorded score. The winner maximizes measured score among admissible candidates; no winner leaves the incumbent unchanged. These rules allow a bounded measured-score decrease relative to the best record and are not a monotonic guarantee for true held-out utility.

The deterministic critic precheck and model review also have distinct scopes. Pattern checks detect configured forbidden content, while model review interprets the diff and declared edits. Neither proves absence of benchmark specialization or runtime failure. A discriminating test changes only one gate at a time, preserves rejected proposals and their reasons, and

reruns the adopted artifact under the same evaluation manifest. The persistent artifact must identify its exact changed files and evidence. Otherwise a higher search log score can remain detached from the harness actually evaluated or released.

Restricted-feedback inference supplies a different guarantee. REUSE controls simultaneous false promotions by accounting for every possible promotion history under an initially independent evaluation sample and decision-only feedback [41]. Its guarantee does not apply when an improver also reads benchmark traces or detailed error scores. Thus proposal regularization, operational permission to adopt, and statistical evidence of improvement are three separate admission conditions. None can replace the others merely because all return an accept decision.

30.3 Task-local recursion and reusable improvement memory

Harness^2 separates refinement for the current task from adaptation of the mechanism that guides future refinement. The model weights remain frozen. The execution harness exposes eight component classes, including prompts, guardrails, memory, skills, tool guidance, scripts, subagents, and middleware. Each task begins from a common execution harness, while an improvement library supplies reusable components and evidence from earlier tasks [28].

Its Propose–Probe–Compose mechanism is technically important because it does not assume a ground-truth task verifier. A reference execution exposes behavior. Invariant and domain-specific edits are proposed, candidate harnesses execute under matched configurations, and a composer selects or combines changes based on behavioral contrasts. Those contrasts can reveal that a change caused a missing file to be read or a repeated error to be avoided. They do not by themselves establish that the final deliverable is correct. Shared model bias, stochastic executions, and incomplete observations remain.

The update can be represented as

$$(h'_t, z'_t, \zeta_t) = \Phi(q_t, h_0, r_t; B_t), \tag{30.7}$$

$$r_{t+1} = \Psi(r_t, h_t^{\mathrm{adopt}}, D_t), \tag{30.8}$$

where q_t is the current task, h_0 the common initial harness, r_t the improvement library, and B_t the allocated refinement budget. The operator Φ returns a candidate harness h'_t, execution record z'_t, and proposal/composition

evidence ζ_t. After adoption, Ψ maintains the library from the adopted harness and recorded evidence D_t, including unsuccessful probes where retained. This is an abstraction of the mechanism, not a claim that the implementation exposes these mathematical function names. The adopted harness is not denoted optimal: adoption is a decision under limited evidence. The library changes the distribution of future proposals and compositions, making it procedural meta-memory.

Two causal questions must be distinguished. Does additional task-local search improve the current result? Does experience from that search improve future refinement at a fixed future budget? The second cannot be inferred from the first. A library built with wider search contains more information and incurred more cost. A comparison that ignores library-construction cost or evaluates only familiar task patterns can overstate improvement of the improver. Appropriate controls include an empty library, a shuffled library, a library with the same entries but no evidence annotations, and a cost-matched independent-search baseline.

The mechanism also creates a compatibility problem. A successful edit may depend on a particular model's failure mode or an old tool schema. Reusing the edit after a model upgrade can add unnecessary instructions or conflicting procedures. Each entry should retain the applicable model and environment versions, the observed deficiency, the exact intervention, counterevidence, and an expiry or revalidation condition. A library of conclusions without the conditions under which they held can accumulate brittle folklore rather than transferable knowledge.

30.4 Policy and harness co-adaptation

Memory and harness updates change the trajectories on which a model learns. Model updates then change how those same memories and harnesses are interpreted. The coupled process is not generally equivalent to optimizing independent components. Let θ denote policy parameters and h the harness. A cross-component interaction can be measured by

$$\begin{aligned}\Delta_{\text{int}} = J(\theta_1, h_1) - J(\theta_1, h_0) \\ - J(\theta_0, h_1) + J(\theta_0, h_0).\end{aligned} \tag{30.9}$$

This is a difference of differences, not proof of a universal synergistic mechanism. Its interpretation requires the four configurations to be evaluated on the same task population under matched evaluation budgets. For already trained checkpoints it measures compatibility of those checkpoints and harnesses. Attributing a causal benefit to the training procedure additionally

requires controlled training interventions and their complete adaptation budgets. An interaction can change sign across environments. The harness may help the old policy precisely because it compensates for a weakness removed by training.

Self-HPO provides a concrete alternating design: the current policy proposes failure-conditioned harness candidates, then reinforcement learning updates the policy under the selected harness; unresolved tasks inform the next search set. The reported study uses one base policy family and two alternating rounds. Its authors evaluate harness-only, policy-only, and joint changes, including cross-pairings of optimized and base components. This supports studying co-adaptation, while the limited rounds and backbone leave sustained recursive improvement unresolved [344].

MASS alternates multi-agent workflow evolution and supervised internalization of generated trajectories, using the same underlying model in executor, optimizer, and evaluator roles. The reported experiment demonstrates two cycles. Its architecture raises a useful question: does a richer information topology produce training signals that a single execution would not expose? It does not eliminate evaluator dependence, nor does finite internalization establish that the resulting model can validate arbitrary research conclusions [345].

Its public research-benchmark analysis further separates delivery from quality. In the reported MLR-Bench runs, the higher aggregate score accompanies more delivered papers; conditional means among delivered papers concern different completed task sets. The appendix reports no delegation calls in those executions and notes that statistical conclusions change with the aggregation unit. Consequently, that transfer result does not itself identify delegation as the mechanism or establish consistent quality improvement across research tasks [345].

The training clock can also be wrong. Terminal-RSI's second revision, released on 25 October, distinguishes consequential environment turns from observation-token positions when assigning delayed credit. A verbose tool output should not automatically imply a longer decision horizon. Its turn-based discount changes the objective relative to token-time discounting; exact chunk continuation preserves both successor value and successor trace only for unchanged retained inputs. Summarization changes that conditioning. Although the revision describes TB-GAE utilization, its limitations still require an isolated estimator comparison, compute-matched alternation, online reward controls, and inherited-versus-reset critic controls before causal component attribution. Deriving an estimator identity is not evidence that a particular evaluated checkpoint used it [346].

These examples motivate a compatibility matrix for every co-evolving system. Model, tokenizer, context transform, memory schema, skill bank, harness, tool schema, reward, and verifier versions should be recorded together. Updating one component requires revalidation of its dependents. Otherwise the next training run may condition on procedures that the behavior policy never used, or a deployment policy may retrieve skills whose preconditions no longer hold.

30.5 Continual memory maintenance as controlled adaptation

Persistent memory creates adaptation even without gradient updates. A skill bank can change the prior over plans, the documents selected into context, and the interpretation of future observations. Its utility depends on the task stream. A procedure repeatedly useful in one order may interfere with a later distribution or conceal a rare exception. The relevant evaluation unit is therefore an evolving trajectory of tasks, not only isolated queries with a preconstructed memory.

HyperSkill illustrates the coupling between representation, retrieval, and maintenance. Its hypergraph connects subtask and skill nodes through trajectory-level hyperedges; dual-path retrieval and co-occurrence-based ranking use this structure, while periodic maintenance prunes or merges entries. The experiment does not isolate a pure graph-format effect when a no-hypergraph baseline also changes the access pipeline. A fixed task order and reliance on gold feedback in the main setting further constrain causal and deployment interpretations [143].

A minimal maintenance objective makes these tensions explicit. Let D_t be newly available, permission-filtered evidence and $M_{t+1} = U(M_t, D_t)$ the updated bank:

$$\max_{U} \; \mathbb{E}_{q \sim \mathcal{D}_{t+1}}[J(\pi, M_{t+1}; q)] - \lambda_s |M_{t+1}| - \lambda_u C(U), \tag{30.10}$$

subject to provenance, authorization, and deletion constraints. Here $|M_{t+1}|$ is declared storage size, $C(U)$ the update cost, and nonnegative coefficients λ_s, λ_u convert those quantities into the utility units of J. The next-task distribution $\mathcal{D}_{t+1}$ is unknown. Estimating it from recent tasks favors recency; estimating it from all history favors retention. Neither choice guarantees that a rare future instruction survives. A practical system can retain raw evidence under its retention policy while maintaining a smaller derived index, but archival retention and usable retrieval remain separate

properties. Immutability of an audit record is not permission to retain prohibited content indefinitely.

Forgetting also requires a distinction between utility pruning and revocation. A low-utility procedure may remain authorized but rarely relevant. A revoked credential or instruction must cease to influence execution even if it would improve task success. A memory optimizer that removes entries only according to observed reward cannot implement that requirement. Authorization metadata must constrain reads, derived summaries, cached contexts, and reconstructed skills across concurrent sessions.

Feedback provenance is equally important. A skill extracted from a trajectory judged successful may encode a workaround that bypassed the intended requirement. Store the tested outcome and its verifier scope, not an unqualified assertion that the procedure is correct. When later evidence contradicts the procedure, the update should preserve the conflict and the affected dependency closure rather than silently overwrite history. Continual adaptation needs a mechanism for learning that an earlier lesson was wrong.

30.6 Testing the improver rather than its selected artifact

An improver evaluation starts multiple independent improvement runs from the same initial executor. It varies only the improver, allocates the same search budget, and evaluates resulting executors on untouched external tasks. The outcomes include final held-out quality, quality as a function of consumed resources, regression severity, diversity of changes, and the frequency of invalid or unsafe proposals. A single successful search trajectory cannot estimate the distribution of improver performance.

AIDE^2 explicitly distinguishes this issue through its ignition test. Its discovered research agent is used as an outer-loop improver and compared with a human-engineered improver, holding the initial inner-loop agent fixed. The paper reports three seeds per arm and concludes that the evidence is inconclusive about superior self-improvement efficiency. This limitation is central: improved research execution does not by itself show that improvement will accelerate when the agent becomes its own optimizer [347].

An original discriminating protocol is the following. Pre-register two improvers, fixed executor initialization, a search task stream, independent validation tasks, and a sealed final evaluation population. Randomize search seeds and task orders. Count proposal generation, failed executions, verifier calls, environment setup, memory maintenance, and selection cost.

At predetermined resource checkpoints, freeze the incumbent executor and evaluate it without feeding results back into search. Compare complete cost-quality curves rather than only each run's selected maximum.

Let Ω be the finite predeclared set of improvers, let m, K be positive integers, let $s \in \{1, \dots, m\}$ index independent search replicates within each arm, and let $(b_k)_{k=0}^{K-1}$ be the fixed allocation schedule. The initializer ι_s creates the same executor and memory fixture for paired improvers. The manifest ν_k fixes the versioned sandbox and validation contract. For each (ω, s), suppressing these superscripts after initialization gives

$$\begin{aligned} \xi_0^{\omega,s} &= \iota_s(\xi_0), \qquad D_0^{\omega,s} = D_{\mathrm{s}}, \\ \mathcal{P}_k &\sim Q_\omega(\,\cdot \mid \xi_k, D_k; b_k), \\ (\mathcal{Q}_k, Z_k) &= \mathcal{V}(\mathcal{P}_k, D_{\mathrm{v}}; \nu_k), \\ \xi_{k+1} &= \mathcal{S}_\omega((\xi_k) \oplus \mathcal{Q}_k; D_k, Z_k), \\ D_{k+1} &= D_k \oplus Z_k, \qquad k = 0, \dots, K-1. \end{aligned} \tag{30.11}$$

The distribution Q_ω proposes a finite indexed family of candidate configurations $\mathcal{P}_k$. The sandbox and validation operator $\mathcal{V}$ returns its admissible subfamily $\mathcal{Q}_k$ and an evidence journal Z_k containing all probe outcomes, including rejection and failure. Trial identifiers distinguish repeated proposals with identical configurations. The selector $\mathcal{S}_\omega$ follows the recorded adoption rule and retains the incumbent when no candidate is adopted. The symbol $\oplus$ denotes ordered concatenation of indexed families or journal append, preserving multiplicity and identity. Search evidence D_{s} and selection-validation tasks D_{v} are distinct from the audit and final populations below.

An immutable snapshot map $\mathcal{F}$, report-only evaluator $\mathcal{E}_{\mathrm{a}}$, and componentwise resource accounting give

$$\begin{aligned} \bar{\xi}_{k+1} &= \mathcal{F}(\xi_{k+1}), \\ e_{k+1} &= \mathcal{E}_{\mathrm{a}}(\bar{\xi}_{k+1}; D_{\mathrm{a}}, \nu_{\mathrm{a}}), \\ \mathbf{c}_{k+1} &= \mathbf{c}_k + \mathbf{c}(\mathcal{P}_k, Z_k, e_{k+1}), \qquad \mathbf{c}_0 = \mathbf{0}. \end{aligned} \tag{30.12}$$

The snapshot includes model, harness, memory, and their versioned dependencies. Each coordinate of $\mathbf{c}$ uses one declared resource unit; proposal, failed execution, validation, maintenance, and audit costs remain visible. The origin $\mathbf{c}_0 = \mathbf{0}$ starts accounting after the common initialization; any separately measured initial-executor baseline must report its setup and evaluation costs. The allocation b_k constrains search only through an explicitly enforced budget policy. Audit outputs e_{k+1} enter a separate sealed journal and are absent from D_k, Q_ω, and $\mathcal{S}_\omega$; merely omitting them from an equation does not enforce that access boundary.

After all search runs are sealed, the complete snapshot collection and final evaluation are

$$\begin{aligned} \mathcal{X} &= (\bar{\xi}_k^{\omega,s})_{\omega\in\Omega,\ 1\le s\le m,\ 1\le k\le K}, \\ E_{\mathrm{f}} &= \mathcal{E}_{\mathrm{f}}(\mathcal{X}; D_{\mathrm{f}}, \nu_{\mathrm{f}}). \end{aligned} \tag{30.13}$$

This indexed family retains distinct run and checkpoint identities even when their executor snapshots are equal. The formulation separates selection feedback from assessment of the improver. In practice, evaluator isolation requires access control and logging. If an agent can inspect hidden tests, final benchmark files, or previous final-evaluation outputs through a shared workspace, the separation has failed. Improving a scoring script can be legitimate when the script is wrong, but that intervention must be reported and evaluated as a change in the measurement system.

30.7 Bounds on claims and conditions for deployment

Repeated improvement creates two kinds of accumulated state: executable changes and beliefs about why those changes help. Both need rollback. An acceptance record should include the hypothesis, modified artifacts, compatibility tuple, validation outcomes, held-out outcomes, resource cost, known regressions, and the exact conditions for reversal. Returning to old code while retaining a newly trained model or incompatible memory does not restore the old system.

Deployment should be gated on the property being changed. A context-saving edit needs tests for rare decision-critical information and conditioning consistency. A skill update needs applicability and permission checks. A learned router needs no-valid-tool and schema-drift tests. A verifier update needs independent adversarial cases. A model update needs behavioral and memory-interpretation regression tests. A global score can supplement these checks but cannot replace them, because many failures occupy a small probability mass with high consequence.

The strongest current evidence supports bounded adaptation mechanisms under specified task and feedback conditions. The harder open problem is maintaining trustworthy selection as executor capability, evaluator behavior, state representations, and environments co-evolve. A system that becomes increasingly effective at satisfying a fixed evaluator may be improving the intended task, exploiting the evaluator, or both. The decisive evidence comes from independent transfer, mechanism ablations, preserved failure records, and explicit evaluation of the improvement process itself.

Chapter 31

Agentic Inference: Scheduling, Speculation, and Tiered State

31.1 The workload is a sequence of interrupted computations

Agentic serving is a workload of correlated model calls separated by tools, waiting, branching, and state updates. The prompt of the next call often overlaps a prior prompt, but its useful overlap depends on serialization, context editing, and model compatibility. A tool can pause generation long enough for a cache to be evicted; several workers can share an initial prefix and then diverge; a resumed session can return to another serving replica. A benchmark that repeatedly decodes independent fixed-length prompts misses these interactions.

Represent a session by turns $j = 1, \ldots, J$, each with input length I_j, generated length O_j, reused prefix length R_j, tool duration Z_j, and state-transform identifier v_j. For serial calls whose tool waits do not overlap model work, elapsed time is

$$T_{\text{session}} = \sum_j \left[W_j + T_{\text{prefill}}(I_j, R_j, v_j) + T_{\text{decode}}(O_j, I_j) + Z_j \right], \qquad (31.1)$$

where W_j includes admission and scheduling delay. This equality is an accounting identity for that serial workload. Speculative tool execution, parallel workers, and overlapping serving stages require a dependency graph whose critical path replaces the sum; adding overlapping durations would double-count time. Reporting model time alone omits delays that often determine whether an agent finishes within its deadline.

The September 2026 vLLM AgentX engineering account explicitly treats multi-turn agent traffic, prefix reuse, scheduling, and prefill/decode disaggregation as a coupled serving problem. Its reported throughput and cost comparisons are configuration- and workload-dependent vendor evidence. They cannot be transferred to arbitrary task completion or substituted for quality-matched end-to-end measurements [348]. The analysis below therefore derives resource constraints under explicit hypothetical assumptions and identifies what a measurement must establish.

The serving policy itself can affect reasoning and memory behavior. Aggressive truncation changes observations. A timeout can cause the harness to retry and consume a different random continuation. Admission control can alter which parallel branch finishes first and therefore which evidence a selector sees. These are changes in the executed policy, even when the model weights are fixed. Performance optimization should preserve the declared behavioral contract or measure the consequences of changing it.

Figure 31.1 distinguishes cache placement from the computational identity required for reuse.

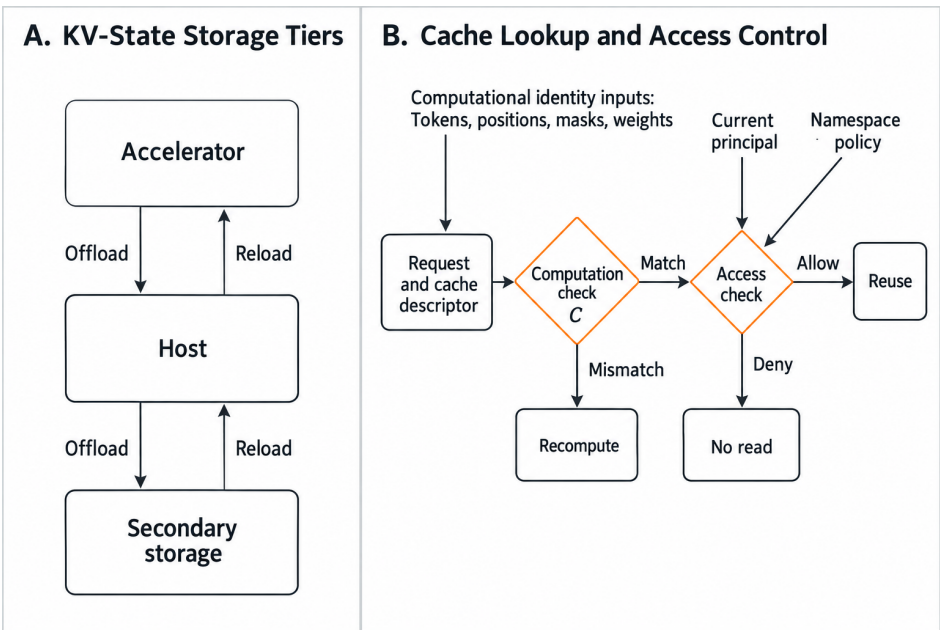

Figure 31.1: Tier placement, computational compatibility and permission to read are separate cache questions. The figure-local C is a computational-match predicate. The listed identity inputs are illustrative; a valid key covers every dependency in the declared forward pass. Descriptor lookup is namespace-scoped, and reuse requires current access authorization. A storage hit does not establish a latency benefit; mismatches need valid recomputation.

31.2 A complete capacity calculation

For a homogeneous attention model, a BF16 KV cache requires

$$M_{\mathrm{KV}} = 2Lh_{\mathrm{KV}}d_h b \sum_{i=1}^{B} T_i. \tag{31.2}$$

The dimensions are layers L, KV heads h_{KV}, head dimension d_h, element bytes b, and retained tokens T_i per active sequence. The factor two counts keys and values. GQA reduces h_{KV} without reducing the number of query heads by the same factor. Quantization adds scale and packing overhead. Replication, copy-on-write pages, speculative tokens, and allocator slack require additional terms. PagedAttention changes allocation and sharing; it does not remove linear growth in the tokens actually retained [59].

Consider a hypothetical dense model with $P = 8 \times 10^9$ parameters, $L = 32$, hidden size $d = 4096$, $h_{\mathrm{KV}} = 8$, $d_h = 128$, and two-byte weights and KV elements. Assume a device exposes 80 GiB usable memory and reserve 6 GiB for workspaces and other runtime allocations. This is an analytical configuration, not a claim about a named commercial accelerator. Weight storage is 16×10^9 bytes, approximately 14.90 GiB. KV storage per retained token is 131,072 bytes, or 128 KiB. A sequence retaining 32,768 tokens therefore consumes 4 GiB of KV state.

Ignoring page slack, fourteen such sequences require $56+14.90+6 = 76.90$ GiB and fit the assumed capacity. Fifteen require 80.90 GiB and do not. The result is a capacity bound under the stated dimensions, not a measured sustainable concurrency. Admission must leave headroom for new generated tokens and transient buffers. If each of fourteen sessions can generate another 4,096 tokens before releasing memory, their additional KV demand is 7 GiB, exceeding the remaining headroom. A scheduler that admits only on current allocation can fail midway through otherwise valid requests.

Hybrid models require a layer-group accounting model rather than multiplying one retention length by every layer. The inspected October 2026 vLLM snapshot distinguishes full-attention retention from sliding-window retention and groups compatible layers in a common allocation scheme. Its design document warns that some descriptions originate at an earlier implementation revision; current code and cache specifications must resolve discrepancies [61]. For analysis, write $M = \sum_g M_g$ with each group defined by its state shape, retention rule, precision, and checkpoint policy. A recurrent state may have constant active size per sequence while its reusable historical checkpoints still consume storage. Constant active state is not equivalent to cost-free prefix reuse.

Table 31.1: Calculated capacity under the hypothetical dense-model workload.

Calculated quantity	Hypothetical value	Interpretation
Weight bytes	16×10^9	BF16 parameter storage only.
KV bytes per token	131,072	Global unsharded homogeneous cache.
KV per 32,768-token sequence	4 GiB	Before allocator and metadata overhead.
Fourteen sequences plus reserve	76.90 GiB	Fits current-state capacity assumption.
Fourteen 4,096-token continuations	Additional 7 GiB	Requires admission reserve or eviction.

31.3 Bandwidth arithmetic and the decode bottleneck

A forward pass through dense parameter matrices costs roughly $2P$ FLOPs per generated token before attention and other corrections. For a batch of B sequences with retained length T, a useful first-order decode estimate is

$$F_{\text{step}} \simeq B(2P + 4LTd). \tag{31.3}$$

The attention term counts query-key products and weighted value accumulation for all query heads; KV-head sharing reduces storage but not that entire arithmetic term. Kernels, sparsity, precision, and architecture alter the actual executed work. MoE accounting must distinguish activated parameter work from total expert storage and communication.

If each weight is streamed once per batch step and all retained KV data is read once, an approximate HBM traffic demand is

$$D_{\text{step}} \simeq Pb_w + B\,2LTh_{\text{KV}}d_h b_{\text{KV}}. \tag{31.4}$$

This assumes reuse of weights across the batch and ignores additional writes and workspaces. It can underestimate traffic when kernels reload data, and overestimate it when relevant data remain in on-chip caches. Given

sustainable bandwidth $\mathcal{B}$ and sustainable compute $\mathcal{F}$,

$$T_{\text{step}} \geq \max\{D_{\text{step}}/\mathcal{B}, F_{\text{step}}/\mathcal{F}\}. \tag{31.5}$$

This roofline-style inequality is conditional on the stipulated work and streaming-traffic model, not a hardware-independent latency guarantee. If D_{step} overcounts traffic eliminated by on-chip reuse, the resulting quotient is not a lower bound for that implementation. Actual indispensable work and traffic would provide a lower bound; the approximations here provide a diagnostic floor for the declared streaming scenario. Launch overhead, synchronization, communication, imbalance, and queueing can increase measured time substantially.

For the hypothetical fourteen-sequence configuration above, assume 2.0 TB/s sustainable bandwidth and 120 TFLOP/s sustainable compute. Reading the 16 GB weights and fourteen 4-GiB caches demands approximately 76.13 GB per step. Its bandwidth lower bound is approximately 38.1 ms. The compute estimate is approximately 464.5 GFLOPs, giving a 3.87-ms compute lower bound. Under these assumptions the bandwidth bound dominates. Quantizing weights alone removes only part of the dominant traffic; reducing KV bytes or retained length can matter more at long contexts. These are deductions from a specified model, not empirical speedups.

Batching trades weight amortization against KV traffic and latency. Increasing B reduces weight bytes per output token but increases total state and can delay individual sequences. A scheduler optimized for aggregate tokens per second can worsen the time until a particular tool call is issued. For agents whose next action unlocks external work, the cost of that delay can propagate through an entire workflow. Performance should therefore include per-session interactivity and task-critical latency, not only engine utilization.

31.4 Exact reuse, partial reuse, and cache identity

Cache reuse requires a compatibility predicate. At minimum, relevant tokens, model and adapter weights, positions, attention masks, architecture mode, and numerical representation must agree with the cached computation. The causal dependency graph matters: editing an earlier token changes later hidden states, so suffix KV values are not generally reusable merely because their token strings are unchanged. A semantic embedding similarity score cannot certify equality of those tensors.

For a sequence of causal transformer layers, the invalidation rule follows by induction. A changed input embedding at position k changes that

position's hidden state. At the next attention layer, every later position whose receptive field includes k may change. Those changes propagate through residual and feed-forward transformations and into later-layer keys and values. Local or sliding-window attention can bound some dependency paths, but hybrid full-attention layers may restore long-range dependence. Exact invalidation must use the actual architecture, not the visual location of a changed text span.

The inspected vLLM V1 module `vllm/v1/core/kv_cache_manager.py` exposes distinct methods for computed-block lookup, slot allocation, cache reset, and computed-block truncation. Those control points make it possible to audit whether a context transform invalidates the dependent state before reuse. They are implementation interfaces, not a proof that a particular deployment invokes them correctly [61]. A cache test should alter one dependency at a time: adapter identity, position offset, prefix token, mask, or tool-result serialization, then compare with a fresh computation within declared precision tolerances.

A reusable key should identify computation, tenancy, and policy constraints. Hashing only token content risks collisions across incompatible adapters or security domains. Tenant isolation can require separate namespaces even when two prompts contain identical public text. A deletion request may also require removal from derived cache tiers, or at least prevention of future reads according to the declared policy. The mathematical possibility of reuse does not imply permission to reuse.

The September 2026 tiered-offloading design places host memory between accelerator storage and secondary tiers such as filesystems, object stores, and remote peers. Reload and offload have different allocation and synchronization costs [349]. A useful admission criterion compares reuse probability and avoided recomputation with transfer and occupancy costs:

$$p_{\text{reuse}}\, C_{\text{recompute}} > C_{\text{write}} + p_{\text{reuse}} C_{\text{read}} + C_{\text{occupancy}} + C_{\text{coordination}}. \quad (31.6)$$

Every term must use consistent units, such as dollars or time-weighted utility. A byte hit rate alone cannot establish net benefit when lookup delays block a critical request or when retained state displaces more valuable work.

31.5 Quantization speculation and structured action outputs

Weight quantization, activation quantization, and KV quantization change different error pathways. GPTQ targets post-training weight approximation,

while SmoothQuant redistributes activation difficulty to make low-precision execution more practical [350, 351]. Their historical methods remain useful foundations; the current implementation choice must be checked against the target architecture, kernels, and 2026 model release. A nominal bit width is not a complete specification: group size, scales, accumulation precision, outlier handling, and supported matrix shapes affect both numerical behavior and performance.

For an agent, a small logit perturbation can cause a discrete tool change. Average text perplexity is therefore an incomplete acceptance metric. Evaluate schema validity, tool selection, argument fidelity, termination, rare constraints, and task success. If the top two action probabilities are close, a modest numerical change can reverse their ordering even when most token probabilities are nearly unchanged. A quality-preserving quantization claim needs the decision distribution and workload that matter to the application.

Speculative decoding proposes tokens with a draft mechanism and verifies them with the target model. Under the appropriate acceptance rule, the goal is to preserve the target sampling distribution while reducing costly sequential target work [352]. Its benefit depends on acceptance, verification cost, draft cost, batch interaction, and available hardware parallelism. A high acceptance rate does not guarantee a speedup if the draft consumes the same bottleneck resource or increases memory pressure enough to reduce concurrency.

Let a draft propose k tokens, with expected accepted progress G_k, draft time D_k, and target verification time V_k. Ignoring overlap, the useful condition relative to target-only per-token time t is

$$\frac{D_k + V_k}{G_k} < t. \tag{31.7}$$

For independent identically distributed cycles with finite mean duration and positive finite mean progress, the ratio of those means is the long-run time per emitted token by renewal-reward accounting. It is neither the expectation of each cycle's time/progress ratio nor a latency percentile. Adaptive cycles, shared-resource contention, and changing context lengths require a justified stationary model or direct measurement. Structured tool outputs create additional boundaries. Drafted tokens cannot trigger side effects before the target output, parser, authorization policy, and commit logic accept the action. Streaming a plausible partial tool call into execution defeats the intended separation between speculative text and committed behavior.

31.6 Memory-conditioned speculation and exact verification

The 2026 Speculate with Memory method retains transition statistics, retrieved episodes, and recurring prediction errors in a speculator while leaving the actor's context unchanged. It predicts actions, observations, or chains during available computation windows and checks exact matches before reusing speculative work. Its prelaunch restriction excludes irreversible writes. The distinction from token-level speculative decoding is essential: a prediction can launch an environment query or an actor call, not merely propose the next token. Reported match accuracy and theoretical latency savings are not identical to end-to-end deployment speedups [353].

Let ℓ_A, ℓ_E, and ℓ_S be actor, environment, and speculator latency, and let $H \in \{0, 1\}$ indicate a correct, reusable prediction under the experiment's exact-match rule. With sufficient independent resources, an action prediction launched while the actor runs can hide at most

$$\Delta_1 = H \min\{\ell_E, (\ell_A - \ell_S)_+\}, \qquad (x)_+ = \max\{x, 0\}. \tag{31.8}$$

Predicting an observation while the environment runs can hide at most

$$\Delta_2 = H \min\{\ell_A, (\ell_E - \ell_S)_+\}. \tag{31.9}$$

These are overlap bounds, not measured speedups. Memory can increase $P(H = 1)$ while also increasing ℓ_S through lookup. Shared accelerator contention, extra API requests, and coordination can further reduce the realized benefit. A memory ablation must therefore measure both match quality and the critical-path cost of obtaining that quality.

Functional equivalence also requires more than identical action type and arguments. A read-only query issued early may observe a different version of a changing environment. Reusing its response later can alter the actor's context even though no irreversible write occurred. Exact trajectory preservation requires an appropriate snapshot, stable observation semantics, or freshness validation. Prelaunch can also consume rate limits or expose data to an additional service. The claim is consequently conditional on the workload's effect, time, authority, and resource contracts, not an unrestricted property of every operation labeled read-only.

For contrast, token-level rejection sampling supplies a distributional exactness argument. Let $p(y \mid C)$ be the target distribution and $q(y \mid C, M)$ a normalized memory-conditioned draft. Suppress the conditioning for notation. Accept a draft token y with probability $\alpha(y) = \min\{1, p(y)/q(y)\}$

wherever $q(y) > 0$. On rejection, sample from

$$r(y) = \frac{(p(y) - q(y))_+}{Z}, \qquad Z = \sum_y (p(y) - q(y))_+, \tag{31.10}$$

when $Z > 0$. Accepted mass is $\min\{p(y), q(y)\}$ and rejected replacement mass is $(p(y) - q(y))_+$, whose sum is $p(y)$. The mean acceptance is $1 - \mathrm{TV}(p, q)$. Thus memory may change draft efficiency without changing the target distribution if the correct probabilities and replacement rule are available. This standard rejection-sampling deduction does not show that an action-level prelaunch system implements the same guarantee, nor that a stale or unauthorized cache is usable. The boundaries of exact verification differ between tokens, model calls, observations, and external effects.

31.7 Scheduling disaggregation and queueing

Continuous batching admits and retires sequences between decode steps, reducing the waste of static batches with heterogeneous lengths. Prefill and decode can nevertheless interfere because they stress different resources and have different service times. Chunked prefill limits how much a long prompt delays interactive decode. Disaggregation places prefill and decode on different workers, trading interference against KV transfer, coordination, and potentially duplicated state.

For a request with prefill time P, decode time D, transfer time X, and scheduling delays W_p, W_d, disaggregation improves latency only if the reduction in interference and queueing exceeds X and added coordination. It can improve throughput while worsening a short request's latency. An experiment must compare offered load and quality, not merely the best isolated run on each architecture. SGLang's structured execution and prefix-sharing design provides another implementation perspective on serving programs with related prompts [354].

Little's law relates stable long-run averages: $N = \lambda T$ for average population N, admitted throughput λ, and mean residence time T under the corresponding boundary. It does not require exponential service, but it does require consistent accounting and a stable system. A more restrictive M/G/1 model with Poisson arrivals, identically distributed independent service times S independent of those arrivals, finite $\mathbb{E}[S^2]$, and first-come-first-served scheduling gives

$$\mathbb{E}[W] = \frac{\lambda \mathbb{E}[S^2]}{2(1 - \rho)}, \qquad \rho = \lambda \mathbb{E}[S] < 1. \tag{31.11}$$

The second moment makes service variability important. The model is a diagnostic approximation for isolated stages; continuous-batching GPU service does not literally satisfy its assumptions [355].

Retries can increase offered load precisely when service is degraded. If every attempt independently succeeds with probability $p \in (0, 1]$ and retries have no cap, expected attempts are $1/p$. With a cap of $K \geq 1$ attempts, the expected count is $[1 - (1 - p)^K]/p$; at $p = 0$ the capped count is K and an uncapped process never succeeds. These are counts, not latency formulas when attempt durations depend on outcomes. When timeouts depend on congestion, independence fails and the feedback can be worse: queueing causes timeouts, which cause retries, which increase queueing. Idempotency prevents duplicate effects but does not eliminate duplicate compute. Retry budgets, jitter, admission control, and reconciliation should be evaluated as part of the serving policy.

31.8 A reproducible serving experiment

The experimental unit is a trace-defined session with declared model, tokenizer, harness, and memory behavior. Record input and output lengths, reused prefix, cache tier, context edits, tool waits, retries, and termination. Run both open-loop offered-load tests and closed-loop concurrency tests, because closed-loop clients reduce arrivals when latency increases and can hide overload. Separate cold starts, warm kernels, cold caches, warm prefixes, and resumed sessions on different replicas.

Report time to first token, inter-token latency, request completion, session completion, throughput, rejection, timeout, and failure. Provide p50, p95, and p99 where sample size supports them, with the number of observations and a clear treatment of censored requests. A percentile from completed requests alone can look better when the slowest requests fail. Include tool time and memory maintenance in task-level results even when engine-only timing is also reported.

The ablation sequence should isolate one mechanism at a time: no prefix reuse versus reuse, uniform versus hybrid allocation, cache offload disabled versus enabled, fixed versus adaptive speculative budget, and monolithic versus disaggregated execution. Hold the action-output quality criterion fixed. If an optimization changes context policy or truncates evidence, report that as a behavioral intervention rather than an invisible serving improvement.

A negative result is informative. Tiered caching may lose on low-reuse traffic; speculation may lose with weak acceptance; a larger batch may im-

prove aggregate throughput while delaying critical tool actions; compressed KV may preserve average language quality while corrupting rare arguments. The deployment decision follows from the joint quality, latency, cost, and reliability envelope. Peak throughput is one coordinate of that envelope, not a sufficient description of an agent-serving system.

Chapter 32

Operational Assurance: Monitoring, Migration, Rollback, and Recovery

32.1 A release is a state-compatible system configuration

Deploying a new model endpoint does not by itself deploy a new agent system. The release also includes tokenizer and serialization rules, prompts, context transformations, memory schemas, embedding indexes, skills, tools, permissions, validators, and runtime behavior. A change can be compatible at the API level while incompatible at the level of meaning. A field that remains a string can change from an absolute path to a workspace-relative path; an old skill may then direct a valid call to the wrong location.

Define a release manifest

$$\nu = (v_\theta, v_q, v_\delta, v_g, v_M, v_I, v_h, v_T, v_A, v_V), \tag{32.1}$$

with versions for the model, tokenizer, decoder, context constructor, memory schema, index, harness, tools, authorization, and validators. The decoder reference binds sampling configuration and versioned constrained-decoding rules; actual per-call settings are retained alongside the model input when runtime-selectable. A comparison claiming fixed decoding holds that configuration fixed. Parsing, stopping, and retries remain part of the versioned harness. A session is pinned to a manifest or governed by an explicit migration rule. Compatibility is a predicate $\mathsf{C}(\nu, S)$ over configuration and retained state, not merely a comparison of version strings. It can require schema validation, representation conversion, regenerated embeddings, restored cache dependencies, reauthorized tools, and revalidation of procedures extracted under an older policy.

A deployment dependency graph has an edge $u \to v$ when the interpretation or safety of component v depends on the contract supplied by u. Changing a tokenizer can invalidate token offsets, cache keys, and learned interface adapters. Changing an embedding model can invalidate nearest-neighbor comparability. Changing authorization can invalidate queued

actions and retrieved procedural instructions. The affected closure is the set of reachable dependents. A release gate should examine that closure instead of treating each changed file as an independent patch.

This graph need not be acyclic in its full behavioral form. A policy writes memory that later conditions the policy; a verifier changes the training data that alters the executor. For deployment planning, use a versioned snapshot and a declared update order to break the operational cycle. Record which old artifacts can be read during transition and which new artifacts can be written. A system that writes new-schema records before all readers are compatible has introduced a mixed-version state that rollback may not be able to interpret.

Current framework documentation illustrates the need to distinguish persistence mechanisms. LangGraph's October 2026 persistence documentation separates thread-scoped checkpointers from cross-thread stores and warns that in-memory implementations do not survive process restarts [127]. Those documented roles do not establish an end-to-end durability guarantee. The application must select a persistent backend, define retention, verify recovery, and reconcile external effects independently.

32.2 Observability must reconstruct decisions and effects

A trace suitable for diagnosing an agent must connect a decision to the observations that conditioned it and to the effects it actually caused. A useful event envelope contains run, session, branch, model-call, tool-call, operation, and parent identifiers; timestamps with clock-domain information; the release manifest; context and memory versions; authorization decisions; result status; and links to evidence artifacts. Content may be redacted or retained separately, but the links must preserve the ability to distinguish absent evidence from inaccessible evidence.

Wall-clock order alone is insufficient in a distributed system. Two workers can observe different memory versions and log events with skewed clocks. Use causal identifiers and explicit read/write versions to reconstruct which state each worker consumed. A trace that records only the latest memory snapshot cannot establish whether a stale record influenced an earlier action. Similarly, a final tool success log cannot reveal whether the model first attempted several invalid operations that were blocked by policy.

Separate four event states: proposed, authorized, submitted, and reconciled. Proposed means the model or deterministic planner produced an action. Authorized means a policy accepted the operation under a spe-

cific authority epoch. Submitted means the request crossed the execution boundary. Reconciled means the system has evidence about the external outcome. A timeout between submission and reconciliation must remain an unknown outcome, rather than being relabeled as failure and retried blindly.

Table 32.1: Operational observations and the claims they can support.

Observation	What it can establish
Model completion	A token sequence was generated under recorded conditioning; no external effect is implied.
Parser acceptance	An output matches an invocation grammar; correctness and authority remain separate.
Authorization acceptance	The operation was allowed under one recorded policy state; later revocation may change executability.
Transport acknowledgment	A request reached a declared boundary; durable application effect depends on service semantics.
Reconciliation evidence	The operation's externally visible state was checked using its identity and the service contract.
Validator verdict	A specified property was tested; untested requirements remain unresolved.

Monitoring every prompt verbatim is neither necessary nor sufficient for observability. Raw content can contain credentials, personal information, proprietary documents, or malicious instructions. A design can retain hashes, typed state transitions, retrieval identifiers, and separately protected evidence while exposing only the minimum content needed for diagnosis. Redaction must preserve semantic distinctions important to a failure: removing every destination string, for example, can make an authorization bug impossible to reconstruct. The retention policy therefore belongs in the threat and debugging model rather than being an afterthought.

Sampling also changes what can be inferred. If traces are sampled only for successful requests, incident analysis will miss failure paths. If rare violations are oversampled for diagnosis, their raw frequency in the sample is not a deployment rate. Record selection probabilities or maintain a separate denominator stream so that operational metrics can be interpreted correctly. A dashboard without a defined observation process can produce

precise-looking but misleading numbers.

32.3 Canaries with persistent state and delayed outcomes

A canary compares a new release with a baseline on a declared traffic population. For stateless requests, random assignment can often occur per request. For agents with persistent memory, assignment should usually occur at a level that prevents state contamination, such as user, repository, or session lineage. Switching versions turn by turn can make the new policy read memories written by the old policy and vice versa, confounding both treatment and safety interpretation.

Let Z_i denote treatment assignment for independent cluster i, and $Y_i(z)$ its outcome under release z. The estimand can be mean task success, violation probability, completion time, or a vector of operational metrics. A release decision should specify acceptable changes before reading the results. A quality improvement cannot automatically compensate for an authorization regression. Delayed outcomes require waiting or explicitly handling censoring; early completion rates can favor the release that abandons difficult tasks quickly.

Shared retrieval indexes create interference even when sessions are assigned independently. If canary runs write new procedures into a global skill bank that baseline runs can retrieve, the baseline has been treated indirectly. Possible designs include isolated memory namespaces, frozen read-only shared evidence with separate write stores, or cluster-level assignment that includes the shared state. Each has a different cost and external-validity tradeoff. The experimental report must state which information crossed treatment boundaries.

Sequential monitoring creates repeated looks at the same data. Declaring success as soon as any metric crosses a conventional threshold inflates false positives. Use a predeclared stopping rule, a valid sequential procedure, or a fixed observation window appropriate to the estimand. Operational emergency stops are different: a serious violation can justify containment without first establishing statistical superiority or inferiority. The corresponding incident should be retained in the comparison rather than erased by restarting the experiment.

A proposed canary design uses four arms: baseline policy with baseline memory, new policy with baseline memory, new policy with migrated memory, and new policy with an isolated empty derived bank but the same raw evidence access. The first pair estimates a policy intervention at fixed

retained state; the second and third isolate migration under the new policy; the third and fourth assess dependence on inherited derived state. These are conditional contrasts within the specified configurations, not a complete decomposition of all interactions. If the new policy cannot consume the baseline representation, that arm requires a declared compatibility adapter or controlled replay and cannot isolate a pure policy effect. No outcome is asserted. Additional cross-pairings can diagnose incompatibility, while real-effect exposure requires the corresponding operating authority and isolation.

32.4 Rollback and compensation are different operations

Rollback restores a prior software or logical state. Compensation attempts to counteract an external effect using another operation. Neither necessarily reconstructs the world that existed before the original action. Deleting a published object may not undo its disclosure; reversing a financial transaction may not remove fees or downstream actions; restoring a database snapshot can erase legitimate concurrent writes. An agent runtime must classify which effects are reversible, compensatable, or irreversible under the actual service contract.

Consider a hypothetical enterprise agent that prepares a report, uploads it, then records the uploaded identifier in memory. The service accepts an idempotency key. The following failure sequence is possible: The correct recovery cannot be inferred from a natural-language summary that says upload failed. It requires operation identity and reconciliation semantics. If the service lacks idempotency and lookup, a global exactly-once claim is unsupported. The application may need conservative escalation, an outbox pattern with a specialized delivery service, or a bounded duplicate-risk policy. These are explicit changes to the operating contract.

Idempotency is a scoped service property. The same logical operation must retain its identity across retries, and reuse of an identifier with different arguments must follow a declared conflict rule. Deduplication retention, namespace, concurrent submission and crash behavior determine how long the protection holds. Expiry of a deduplication record can make an old delayed retry appear new. HTTP's idempotent-method semantics concern the intended effect of repeated identical requests, not a universal exactly-once execution or identical-response promise [356]. A client therefore needs the actual application's operation contract before retrying a non-idempotent effect. A local outbox can durably retain intent while delivery remains

Table 32.2: Constructed upload failure and effect-reconciliation sequence.

Step	Durable evidence	Recovery implication
1	Intent and operation key stored	The intended effect is identifiable after restart.
2	Upload request submitted	Submission alone does not prove the final state.
3	Service commits; response lost	Local timeout leaves outcome unknown.
4	Process crashes before memory write	Session history omits the completed effect.
5	Runtime resumes	Query by operation key before deciding whether to retry.
6	Committed object found	Record reconciled outcome; do not create a duplicate.

repeated; the receiving service still determines whether those deliveries become duplicate effects.

Memory rollback must preserve authority monotonicity. Suppose a credential was revoked after a checkpoint. Restoring that checkpoint must not restore the old credential's authority. Revocation information should be applied from an authoritative current source even when task state is rolled back. The same principle applies to deletion tombstones and legal retention boundaries: an old backup is not permission to resurrect information for use. Recovery is a compatibility-preserving reconstruction under current authority, not unconditional time travel.

The three contracts in Chapter 1 apply at different points in this reconstruction. A byte-faithful checkpoint may preserve old proposal conditioning while violating current authority. A permission-filtered restoration may preserve authority while changing proposal probabilities. Reconciliation may legitimately update an unknown effect status after obtaining new service evidence; that update must be distinguished from the preceding representation-only restoration. The recovery trace should therefore record restoration, current-authority filtering, and external reconciliation as separate typed transitions. Testing only equality with the old snapshot would reward the wrong behavior after revocation.

For a rollback from ν_1 to ν_0, the required condition is not simply that

old code starts. There must be an admissible conversion ψ such that

$$\mathsf{C}(\nu_0, \psi(S_1)) = 1, \tag{32.2}$$

while external effects and current authorization remain reconciled. A lossy forward migration may have no inverse. The release procedure must then retain a recoverable representation or declare the rollback boundary before deployment. Pretending every schema migration is reversible converts a known design limitation into an incident.

Figure 32.1 isolates the mechanism and its stated information boundaries.

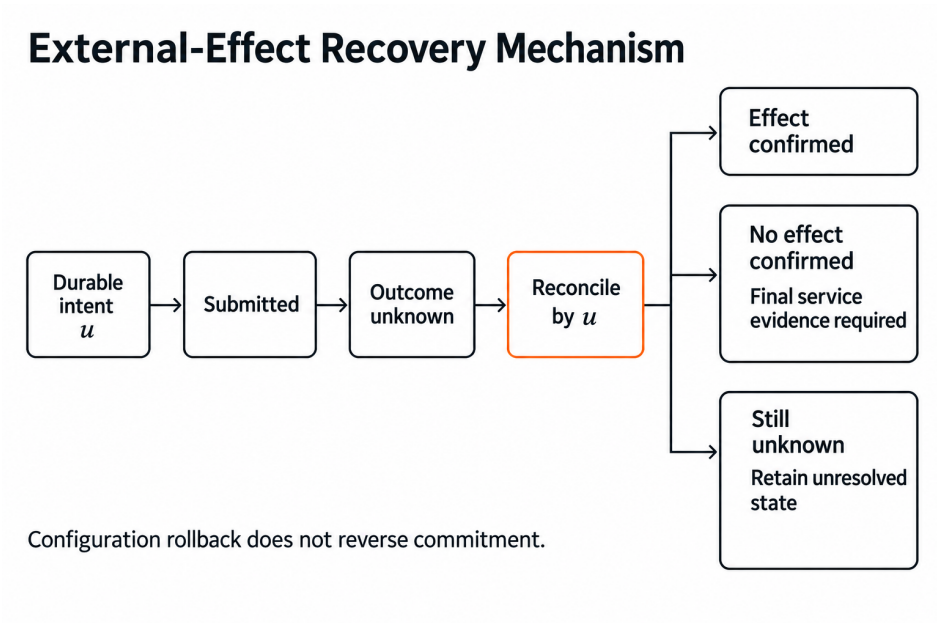

Figure 32.1: Recovery retains operation identity and distinguishes submission from commitment evidence. A missing receipt does not prove absence. Only the service contract can justify a confirmed effect or a final negative outcome; otherwise the ledger stays unresolved. Configuration rollback does not reverse an external effect.

32.5 Failure detectors and bounded recovery policies

Agent failures include model errors, invalid tool calls, stale observations, context loss, memory poisoning, authorization mistakes, service outages,

evaluator failures, and infrastructure faults. The same outward symptom can have several causes. A missing artifact may result from no attempted write, a permission rejection, a transport failure, a successful write in the wrong workspace, or a stale listing. Diagnosis should use discriminating observations rather than repeatedly asking the model to try harder.

A recovery policy is a decision rule over failure evidence and remaining budget. Let r be a recovery action with expected task value V_r, expected resource cost C_r, and specified risk Q_r. Define the admissible set

$$\mathcal{R}_q(S) = \{r \in \mathcal{R}(S) : Q_r \leq q_{\max}\}, \tag{32.3}$$

where $\mathcal{R}(S)$ excludes actions without current authority or required state, and $q_{\max}$ is a declared risk limit in the units of Q_r. For a nonempty finite set and $\lambda \geq 0$, one possible decision rule is

$$r^* \in \arg\max_{r \in \mathcal{R}_q(S)} [V_r - \lambda C_r]. \tag{32.4}$$

The coefficient λ converts resource cost to the utility units of V_r; ties require a declared selection rule. For general infinite action sets, attainment needs additional assumptions. An empty admissible set means that this optimization supplies no recovery action; cancellation or escalation is available only if separately admissible. Estimated values and risks remain uncertain. Conservative policies can prefer reconciliation when effect status is unknown. A retry count is not a substitute for a recovery model, because repeated identical attempts can be positively correlated and can worsen overload.

Budget exhaustion should produce a typed terminal state. Distinguish completed, failed, cancelled, waiting for external input, and outcome unknown. A model-generated success statement does not override the runtime's effect ledger. Conversely, a timed-out worker may have produced a valid artifact that can be recovered and verified. The termination protocol should specify which partial outputs are retained, who may resume them, and whether their authority or evidence has expired.

Bounded local recovery does not imply bounded external resolution. A timeout cannot by itself distinguish a failed remote service from a delayed one. If receipts remain unavailable, the runtime can stop spending its local budget and durably return an unresolved outcome, but cannot manufacture evidence that the effect is absent. An eventual-resolution claim requires explicit assumptions about service recovery, message delivery, identity retention and continued reconciliation access. Test those premises separately from restart success. In particular, expiring credentials can

make a previously available status query inadmissible; escalation must then obtain current authority rather than replay the old grant.

Failure injection tests these claims directly. Crash immediately before submission, after submission, after remote commit, after local reconciliation, and during derived-memory updates. Introduce delayed duplicate responses, stale index replicas, broken validators, schema changes, and worker partitions. Check invariants about duplicate effects, unauthorized operations, lost writes, and eventual terminal state. These tests should run against controlled fixtures or isolated services; a successful test supports the tested contract, not an unrestricted guarantee.

The proposed instrumentation should record recovery-induced changes in the policy distribution. Offloading a failed tool output, replacing a summary, or retrieving a different skill can change the model's next decision. A recovery that eventually succeeds may have introduced a hidden regression in a rare constraint. Evaluate the complete recovered trajectory rather than treating recovery as an infrastructure event independent of cognition.

32.6 Service objectives and incident learning

An operational objective needs a denominator, boundary, and observation window. Examples include the fraction of admitted tasks that complete under the specified verifier and deadline, the probability of a duplicate external effect per submitted operation, and the time from a detected compatibility failure to containment. Aggregate availability can hide a system that responds quickly but returns unusable artifacts. Report task success and effect correctness separately from endpoint availability.

Mean recovery time is sensitive to incident definition and censoring. A system that labels unresolved incidents as abandoned can appear faster. Time-to-recovery distributions should retain unresolved cases and distinguish restoration of service from reconciliation of residual effects. The latter may continue after the model endpoint is healthy. Cost accounting should include repeated model calls, tool requests, memory repair, reindexing, and human intervention according to the declared boundary.

Incident learning creates another feedback channel. A failure trace can reveal a benchmark answer, a confidential record, or an attacker payload. Automatically adding the trace to a shared memory or training corpus can contaminate future evaluation or persist the attack. Curate incident-derived records with provenance, scope, authorization, and a statement of the tested lesson. Preserve the raw restricted evidence separately when needed for audit. A procedure extracted from one incident should identify

the conditions that made it appropriate.

The AgentOps research proposal provides a vocabulary for observing and optimizing agent systems, but its historical publication date and conceptual role must be distinguished from current 2026 platform capabilities [341]. Current SDK and managed-agent documentation can establish available tracing, state, and recovery interfaces; they do not independently establish reliability in a particular workload [126]. An operational assurance claim requires evidence from that workload and the actual release manifest.

32.7 A compatibility-first deployment procedure

A defensible deployment sequence begins by freezing the candidate manifest and enumerating its dependency closure. Validate state migrations and authority handling in an isolated replay environment. Run deterministic contract tests and stochastic task evaluations with matched baselines. Include long histories, memory conflicts, interrupted sessions, concurrent writers, tool schema drift, and previously observed incidents. Record both improvements and regressions rather than selecting a favorable subset of tasks.

Next, deploy to an isolated canary population with declared state-sharing rules and stopping criteria. Monitor task outcomes, constraint violations, latency distributions, resource consumption, and unknown external effects. A rollback rehearsal must demonstrate that the retained state can actually be read by the previous release or converted safely. Rehearsing only a container restart does not test restoration of policy-compatible memory.

Let $\nu_\star$ be the frozen candidate manifest and $G = (V, E)$ its dependency graph. If V_Δ contains the changed components, the affected closure is $D_\star = V_\Delta \cup \mathrm{Reach}_G(V_\Delta)$, explicitly including the changed components themselves. Define Boolean migration, current-authority, and deletion predicates M, A, and D. For initial retained state S_0 and authority epoch α_0,

$$\begin{aligned} q_\star &= \mathsf{M}(\nu_\star, S_0)\,\mathsf{A}(\nu_\star, S_0; \alpha_0)\,\mathsf{D}(\nu_\star, S_0), \\ R_\star &= \begin{cases} \mathcal{T}(\nu_\star, S_0, D_\star), & q_\star = 1, \\ \bot, & q_\star = 0, \end{cases} \\ \chi_\star &= \begin{cases} \mathsf{J}(R_\star), & q_\star = 1, \\ 0, & q_\star = 0. \end{cases} \end{aligned} \tag{32.5}$$

The operator $\mathcal{T}$ runs the isolated regression and fault tests; $R_\star$ retains every outcome and known limit. The marker $\bot$ means that failed preconditions prevented this test stage. The predeclared release predicate J does

not inspect a selected favorable subset. The manifest, closure, predicate outcomes, results, and limits are appended to the release journal before any canary decision.

Only when $\chi_\star = 1$ is the candidate eligible for an isolated canary assignment Z. A migration predicate does not execute a migration: the canary initializer must construct $S_0^{\mathrm{c}} = \psi_{\mathrm{c}}(S_0)$ and check the compatibility relation $\mathsf{C}(\nu_\star, S_0^{\mathrm{c}}) = 1$ before admission. Here ψ_{c} is the tested conversion into the candidate's isolated state namespace; failure prevents admission. Let $K_{\nu_\star,Z}$ be the canary transition kernel and $\mathcal{H}_t$ the observations available to monitoring by time t. A predeclared adapted stopping rule B defines

$$\begin{aligned} S_{t+1}^{\mathrm{c}} &\sim K_{\nu_\star,Z}(\cdot \mid S_t^{\mathrm{c}}), \qquad t < \tau, \\ \tau &= \inf\{t \geq 0 : \mathsf{B}(\mathcal{H}_t) = 1\}, \\ \lambda_\tau^+ &= \mathcal{R}(\lambda_\tau, \mathcal{E}_\tau), \qquad \tau < \infty. \end{aligned} \tag{32.6}$$

The convention $\inf \varnothing = \infty$ leaves monitoring incomplete rather than inventing a terminal outcome. The reconciliation operator $\mathcal{R}$ uses external-service evidence $\mathcal{E}_\tau$ to update the release-relevant effect ledger λ_τ, including relevant production and canary operation identities; unsuccessful reconciliation preserves uncertainty. The kernel's namespace isolation and the stopping rule's information restrictions are implementation obligations, not properties established by their notation.

At a finite stopping time, let $d_\tau \in \{-1, 0, 1\}$ denote the recorded decision to roll back, hold, or promote, and let ν_0 be the rollback target. Obtain a fresh production snapshot $P_\tau = (\nu_\tau^{\mathrm{p}}, S_\tau^{\mathrm{p}}, \upsilon_\tau)$, containing its manifest, retained state, and concurrency version. With separately checked guards G_+ and G_-, the admissible transition request is

$$r_\tau = \begin{cases} (P_\tau, \nu_\star, \psi_+), & d_\tau = 1,\ G_+(P_\tau, \lambda_\tau^+; \alpha_\tau) = 1, \\ (P_\tau, \nu_0, \psi_-), & d_\tau = -1,\ G_-(P_\tau, \lambda_\tau^+; \alpha_\tau) = 1, \\ \bot, & \text{otherwise.} \end{cases} \tag{32.7}$$

The guards require current authority, deletion compliance, and reconciled effect status acceptable under the declared transition contract. The maps ψ_+ and ψ_- are declared retained-state conversions with compatible outputs, respectively $\mathsf{C}(\nu_\star, \psi_+(S_\tau^{\mathrm{p}})) = 1$ and $\mathsf{C}(\nu_0, \psi_-(S_\tau^{\mathrm{p}})) = 1$. The rollback conversion follows Section 32.4; a target that starts but cannot interpret the state is inadmissible. These guards inspect production state, not the isolated canary snapshot.

The request r_τ neither migrates state nor activates a manifest. Its implementation must revalidate the production concurrency version, authority

epoch, effect-ledger freshness, and transition guards when committing the state conversion and manifest activation. A changed precondition requires a hold and renewed validation. Atomicity within the deployment state store does not supply atomicity for external services. Failed admission, incomplete monitoring, or a rejected request supplies no promotion. The equations specify evidence order and admissibility, not a guarantee that a named deployment platform satisfies the contract.

The procedure is an original implementation contract, not a claim that any named product performs these steps automatically. Its purpose is to make deployment assertions testable. A system can expose excellent components and still fail at their boundaries. Monitoring, rollback, and recovery become part of agent intelligence when they determine what the policy knows, which effects are possible, and whether learned state remains meaningful after change.

Chapter 33

Cross-Layer Causal Analyses and Compositional System Designs

33.1 Composition changes the claim being tested

A component-level improvement can disappear when composed with the rest of an agent. A retriever can return more relevant material while increasing prompt length enough to force a harmful compaction. A model can generate more valid tool calls while increasing the fraction that target stale objects. A faster serving backend can change which parallel branch finishes first and therefore which artifact a selector adopts. Cross-layer analysis asks whether the composed system preserves the assumptions under which each mechanism was evaluated.

The cases in this chapter are worked analytical designs, not reported deployments or newly measured experiments. They combine mechanisms examined in earlier chapters into explicit state transitions, resource calculations, and falsification tests. Each case specifies the observation and authority boundaries, the information that must persist, the role of model inference, and the evidence required before an external effect or scientific conclusion is accepted. Their purpose is to make interaction failures concrete.

Let a workflow be a directed graph $G = (V, E)$ whose vertices are model decisions, retrievals, tool operations, state commits, or verification steps. Edges represent causal prerequisites, not merely visual order. Each vertex has an execution contract K_i, resource cost c_i, duration t_i, and possible failure events F_i. The global task requirement K_G follows from local contracts only if the contracts include all relevant coupling conditions:

$$\bigwedge_i K_i \ \wedge \ K_{\text{coupling}} \quad \Longrightarrow \quad K_G. \tag{33.1}$$

A collection of passing local tests does not establish the antecedent if K_{coupling} omits state-version consistency, authority, or external effects. This is the formal reason that integration testing cannot be replaced by component leaderboards.

If violation of K_G implies at least one enumerated failure event, a union

bound gives $P(\neg K_G) \leq \sum_i P(F_i)$. Independence is not required. The premise is strong: unenumerated failure modes and invalid coupling assumptions destroy the guarantee. Multiplying apparent component accuracies is even less defensible when all components rely on the same model-generated interpretation. Shared uncertainty does not become independent evidence because it passes through several boxes.

33.2 Case one: a repository repair with bounded side effects

Consider an agent asked to repair a regression in an unfamiliar repository and return a tested patch. The environment is a pinned checkout with isolated dependencies and a protected evaluator. The model may inspect files, search code, edit permitted paths, and run tests. It may not modify the protected evaluator, fetch hidden answers, or publish changes externally. This is an evaluation contract: whether a real benchmark enforces all these restrictions must be checked separately. SWE-agent and SWE-bench supply concrete historical interfaces and task structures, while the harder SWE-bench Pro material motivates examining repository diversity and contamination [2, 319, 232].

The privileged run state contains the repository commit, dependency-image digest, writable path set, tool schemas, remaining budget, and test-result identities. Working context contains the issue, relevant source excerpts, the current hypothesis, and selected tool evidence. Persistent procedural memory may supply a debugging strategy, but it cannot supply a hidden patch or override writable-path restrictions. A skill retrieved because it solved a similar error must be rechecked against the current repository and test environment.

Suppose the agent observes a failing parser test, inspects the parser and its callers, and proposes changing a branch condition. The decisive evidence is not the model's statement that the change is safe. A useful sequence is to preserve the pre-change failure, apply a minimal patch, run the targeted test, then run relevant regression tests under the same dependency image. If a dependency update is also made, its effect must be separated from the code change. Otherwise the experiment has changed two causal variables and cannot identify which one repaired the failure.

Context compaction can introduce a specific failure. The original issue says that an error must be raised before any output file is created. A summary preserves the expected error message but drops the ordering requirement. The agent produces a patch that raises the error after creating

Table 33.1: Required evidence in the constructed repository-repair workflow.

Stage	State that must be recorded	Failure exposed by the record
Initialize	Commit, image, permissions, schemas	A result obtained on the wrong checkout or environment.
Observe	Test command, output, exit status	A model treating truncated logs as a complete diagnosis.
Inspect	File revision and relevant code span	A patch based on stale or unrelated source.
Edit	Exact diff and permitted target	An unauthorized evaluator change or hidden broad mutation.
Test	Verifier identity and all outcomes	Selection of only favorable tests or an unobserved crash.
Compact	Transform lineage and preserved constraints	Loss of the issue requirement or writable-path boundary.
Finalize	Patch, artifact hash, test evidence	A success claim detached from the delivered artifact.

an empty file. A string-based test passes, while the specified side-effect condition fails. The appropriate counterexample test checks both the exception and the absence of the file. This case distinguishes answer retention, requirement retention, and behavioral verification.

A stored procedural lesson should therefore say more than check the error message. It should identify the applicability condition, the ordering requirement, and the test that distinguishes the correct implementation from a plausible shortcut. If later tasks use a different output contract, the lesson's preconditions prevent indiscriminate reuse. The memory record should retain its source trajectory and verifier scope, so a later failure can invalidate the procedure without erasing unrelated debugging knowledge.

An evaluation of the complete design would compare no procedural memory, unstructured reflections, typed procedures, and retrieval with applicability checks. Keep the model, repository set, token and tool budgets, and grading access fixed. Randomize task order if memory evolves, and include held-out repositories. Record successful repairs, incorrect accepted

patches, invalid edits, tool failures, and complete cost. This is a proposed experiment; the case supplies no fabricated outcome.

A contemporary reported case demonstrates the difference between tested and architectural contracts. Terminal-RSI's revised Rust–C evaluator case passes protected wrapper and output-equivalence checks in its final trials, yet one trial delegates expression evaluation to Python despite producing a statically linked Rust executable. The authors explicitly separate these observations from proof of the requested native evaluation path or memory safety [346]. This is author-reported trajectory evidence, not a reproduction here. It shows why an output oracle and an implementation requirement need separate witnesses: successful observable behavior does not entail that the specified internal mechanism produced it.

33.3 Parallel repair branches and cache-aware orchestration

Now allow four isolated workers to investigate different hypotheses. They share a read-only task prefix but have independent writable worktrees. A coordinator receives evidence and candidate patches rather than raw unbounded histories. The meaningful comparison is not simply four agents versus one. Compare against a single agent with the same total budget and four independent samples without communication. Otherwise gains from additional search can be misattributed to coordination.

A precise resource example uses the hypothetical homogeneous model from Chapter 31, with 128 KiB of KV state per retained token. Suppose each worker uses a common 32,768-token prefix followed by a private 8,192-token continuation. Without sharing, four sequences retain 163,840 token positions, consuming 20 GiB of KV state. With exact read-only prefix sharing and private suffix pages, the unique retained positions are $32{,}768 + 4 \times 8{,}192 = 65{,}536$, consuming 8 GiB before overhead. The 12-GiB difference is calculated storage sharing, not a measured speedup or a guarantee that an engine implements it.

Sharing is valid only while the common prefix computation remains compatible. If one worker's context constructor rewrites an early shared observation, its dependent state must diverge. Copy-on-write protects other workers' pages but does not make the edited worker's old suffix valid. A coordinator that changes the common instruction midway through execution creates another compatibility boundary; workers must receive a versioned update or remain explicitly pinned to the old task state.

The wall-clock lower bound is the duration of the longest dependency

path, while total resource consumption sums work across branches. A slow branch may contain the only evidence that exposes a constraint violation. Cancelling it solely because another branch produced a plausible patch trades latency against coverage. The selector should know which checks remain incomplete and whether its chosen candidate was examined by an independent validator. A vote among models that all missed the same requirement is not independent verification.

Selection also changes the outcome distribution. Let E_j indicate that worker j has produced an eligible candidate, T_j its completion time, and V_j a true contract violation by that candidate. Conditional on at least one eligible candidate, define $J = \arg\min_{j:E_j=1} T_j$ with a fixed tie rule. Then

$$P(V_J) = \sum_j P(J = j)P(V_j \mid J = j). \tag{33.2}$$

This identity does not allow the conditional risk to be replaced by each worker's marginal risk. Fast shortcuts, early termination, and validation cost can correlate quality with selection. A serving change that changes completion order can therefore change the delivered-artifact distribution even when every worker's policy is fixed. Compare selectors on identical saved candidate sets when isolating selection, and compare full executions when measuring the deployed system. Cancelled branches have unobserved counterfactual completions; counting them as successful or failed would invent outcomes.

The final merge is a separate operation. Two individually passing patches can conflict or invalidate each other's assumptions. Preserve each branch's base commit and diff, merge in an isolated worktree, and rerun integration checks on the merged artifact. If the coordinator adopts only one patch, it should retain the rejected branches' relevant counterevidence. Discarding all unsuccessful trajectories can create a memory bank that remembers only apparent successes and loses the information needed to avoid repeating failed hypotheses.

33.4 Case two: a persistent enterprise research assistant

Consider a system that answers questions and prepares reports from a changing collection of internal records. It has multiple sessions per user, shared organizational evidence, role-dependent permissions, and a persistent memory of prior work. Its central problem is not simply document retrieval.

The same claim can be true at one time, superseded later, restricted to one audience, or derived from a source whose access has been revoked.

Use a typed assertion record

$$m = (u, p, v_{\text{from}}, v_{\text{to}}, t_{\text{record}}, s, a, \rho), \tag{33.3}$$

where u identifies the subject, p the proposition, the v fields its asserted validity interval, t_{record} the recording time, s the evidence source, a its access scope, and ρ its derivation lineage. Valid time and record time differ: a correction recorded today can say that yesterday's claim was wrong. A retrieval system that sorts only by ingestion time can answer a historical question incorrectly, while one that ignores corrections can repeat superseded information.

The answer pipeline first determines the question's temporal and authority scope, then retrieves permitted source records, resolves relevant conflicts, constructs working context, generates an attributed answer, and verifies citations and requested output constraints. Derived memory may accelerate this process, but raw evidence remains necessary for contested or high-consequence claims. A retrieved summary is a claim about evidence, not a replacement for the evidence's provenance.

The following constructed sequence exposes a cross-session failure. Session A reads that a project owner is Maya and stores a semantic memory. Later an authoritative update names Ravi, and a deletion policy removes the old personnel record from active use. Session B retrieves an old summary that still names Maya. A semantically similar answer can be fluent and well cited to an obsolete derived artifact. Correct handling requires propagation of the update or revocation through summaries, indexes, caches, and procedural assumptions, together with an access-time validity check.

This is not solved by globally replacing every occurrence of Maya. The name may be correct in historical reports or unrelated projects. Deletion and correction semantics need subject and lineage identity. A tombstone can block future reads while physical cleanup proceeds, but the system should state whether backups, cached prompts, and downstream artifacts remain. A restored old checkpoint must not erase the authoritative revocation epoch. Chapter 32 explains why rollback and authority restoration must be separated.

33.5 Testing relevance and influence rather than retrieval alone

A memory can be retrieved and still have no appropriate effect on behavior. Suppose a user's prior instruction says that externally shared reports must omit a confidential appendix. A later request asks for a concise project update without explicitly mentioning confidentiality. A memory test that asks what the confidentiality instruction was measures explicit recall. The operational test asks whether the agent applies it when preparing the report. InMind's distinction between explicit recall and implicit application motivates this separation, while memory-writing and consolidation systems such as RecMem address different parts of the lifecycle [134, 132].

An influence experiment can intervene on a memory while holding the current task and evidence fixed. Compare a valid relevant instruction, an irrelevant record, a superseded instruction, and an adversarially injected instruction. The desired policy responds differently to these cases. Measuring only whether any memory was used rewards over-influence. Measuring only answer accuracy can hide an unauthorized disclosure that does not affect the nominal answer key.

The evaluation unit should be a user or organizational task stream with declared persistence. Freeze the post-history memory before testing explicit recall and implicit application so that the first test does not teach the answer to the second. Randomize task order or use independent cloned states. Preserve the exact accessible evidence and authority epoch. These controls distinguish a useful memory mechanism from leakage through the evaluation conversation.

The deployment architecture also needs a correction path. When a user disputes an answer, the system should identify whether the error came from source truth, retrieval, conflict resolution, context packing, model interpretation, or output attribution. Updating the answer text alone may leave the faulty derived memory intact. A correction should invalidate the appropriate descendants and record the reason, while avoiding a global overwrite that destroys legitimate historical information.

33.6 Case three: an agent that modifies its own research harness

The research agent performs experiments in a sandbox and proposes changes to its own harness. The mutable objects include prompt templates, search strategy, context management, memory access, and tool orchestration. The

evaluator contains protected tasks and graders. The candidate may inspect its search evidence but not final evaluation artifacts. This configuration resembles the bounded mechanisms studied by RRSI, Harness^2, and AIDE^2, with important differences in feedback and update targets [27, 28, 347].

Separate three artifact classes: the executor under test, the improver that proposes and selects changes, and the measurement system that scores them. Store immutable snapshots of all three at each accepted update. A proposal that changes a test must be classified as a measurement intervention, not quietly accepted as an executor improvement. A discovered evaluator bug can be real and worth repairing, but results before and after the repair need a consistent rescoring policy.

Suppose the current agent repeatedly overflows context and drops experiment configurations. A candidate harness offloads raw logs and retains a typed experiment record containing hypothesis, code revision, data split, random seed, metric definition, and result artifact. This may preserve scientific continuity with fewer active tokens. Its success should be tested on tasks where a later decision requires an earlier configuration, including changed metrics and failed runs. A summary that retains only the best score can create selection bias even if its memory use is efficient.

The candidate should be evaluated against a fixed-summary baseline, raw-log retrieval, and a no-change harness under matched total resources. Include the cost of creating and maintaining the structured records. If the candidate receives a stronger external critic, that critic's contribution must be separated. If the improver selects among many candidates, the final held-out evaluation should be independent of the selection stream. The relevant claim is transferable state management, not the ability to search longer on the same tasks.

The protected-evaluator design here should not be imputed to every cited method. Harness^2 uses execution contrasts and model judgment for its task-level edits rather than ground-truth verifier access; its limitation section explicitly allows shared model bias and failed transfer [28]. When repeated promotion reuses a benchmark, REUSE instead proves an admission guarantee under restricted decision-only feedback and an independent initial sample [41]. Revealing raw protected traces to the proposer violates that particular information restriction. Independent confirmation, adaptive reuse, and proxy-guided editing are distinct experimental contracts.

Now consider model post-training on the collected trajectories. If the retained record exposes information that the behavior policy did not see at a prior decision, training on a naively flattened history can change the conditioning problem. MemoryWalker identifies exact and approximate

ways to address compaction-induced inconsistencies [23]. The research system must record transform lineage and behavior inputs during collection; a later attempt to reconstruct them from a final summary may be impossible. The same memory design that helps execution can therefore create invalid training data if its information boundaries are ignored.

33.7 A unified acceptance record and discriminating interventions

Across the three cases, an accepted artifact should be associated with a claim record containing the objective, applicable population, configuration, intervention, baseline, evidence, resource boundary, failure observations, and limits. This is not a generic success label. A passing patch, a grounded report, and an evolved harness require different validators, but each needs a trace from the delivered artifact to the evidence that supports it.

Table 33.2: Cross-layer failure hypotheses and discriminating interventions.

Coupling failure	Targeted intervention	Discriminating observation
Context omits a constraint	Preserve or remove only the qualifier	Action changes while unrelated content is fixed.
Memory is stale	Swap valid and superseded records	Temporal resolution and authority behavior differ appropriately.
Cache reuse is invalid	Alter one prefix dependency	Fresh and reused computations diverge beyond declared tolerance.
Harness search overfits	Freeze candidate and change population	Selection gains fail to transfer under matched resources.
Verifier misses a requirement	Add a requirement-derived hidden check	Previously accepted artifacts reveal a specific false acceptance.
Rollback restores revoked state	Revoke after checkpoint, then recover	Recovery preserves current authority and deletion constraints.

The interventions are narrow enough to falsify mechanism explanations. They also expose where an observed improvement is underdetermined. If a structured-memory system changes storage, retrieval, prompting, and maintenance simultaneously, an end-to-end gain does not identify which component caused it. Factorial ablations or staged replacements can locate the contribution, while a complete system comparison remains useful for deployment decisions. Mechanistic attribution and engineering utility are different questions.

Cross-layer work should report the simplest baseline that survives these tests. An event log plus deterministic metadata can outperform an elaborate reflective memory on tasks dominated by exact state. A reactive policy with a strong tool interface can outperform search when observations are reliable and actions cheap. A single agent can outperform a team when coordination consumes the available context and latency budget. These are conditional possibilities to test, not universal rankings. The value of the system formulation is that it makes those conditions visible and turns a broad claim about autonomy into a set of specific, reviewable technical propositions.

Figure 33.1 isolates the mechanism and its stated information boundaries.

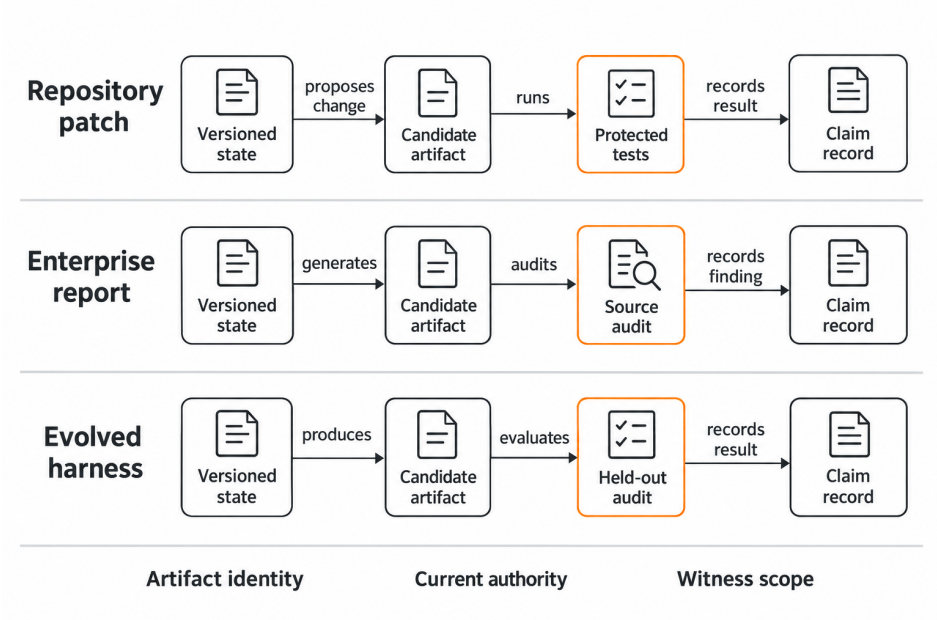

Figure 33.1: The constructed workflows require different witnesses: protected behavior checks, source-level review, or held-out evaluation. Each claim record binds the delivered artifact to its versioned state, current authority and actual witness scope. A passing local check does not certify the complete workflow.

Chapter 34

Synthesis: Consistency Contracts and Discriminating Research Questions

34.1 What the October 2026 evidence supports

The evidence supports a system-level account of agent competence. Model architecture and training determine a conditional policy, but context construction, tools, memory, runtime, and evaluation determine which policy is actually executed and which outcomes are credited. This is not an argument that model quality is secondary. It is an identification statement: when several layers change together, an outcome difference cannot be attributed to the model or to a named memory mechanism without additional controls.

The current frontier also changes which implementation assumptions are reasonable. FlashAttention-4, current hybrid cache managers, and low-precision architectures make hardware and model layout part of any performance claim. V-JEPA 2.1 and contemporary world-model work sharpen representation questions but do not collapse predictive quality into control validity. Context Language Models, MemoryWalker, RecMem, InMind, HyperSkill, and 2026 compaction studies expose different failure points in retained state: writing, consolidation, access, implicit relevance, influence, conditioning, and lifecycle maintenance. A survey that treats them all as larger memory misses the distinctions needed to design experiments [54, 62, 26, 23, 132, 134, 143].

The strongest practical conclusion is conditional. Reliable execution requires that decision-relevant evidence, procedural applicability, authorization, and effect status remain interpretable across the system's state transitions. Longer windows and larger stores can help, but neither establishes those properties. A persistent archive can coexist with failed retrieval. Correct retrieval can coexist with inappropriate influence. A valid procedure can coexist with revoked authority. An authorized call can coexist with an unknown external outcome. Each transition needs its own evidence.

No single benchmark or finite improvement trajectory establishes un-

restricted AGI. Generality remains a profile over declared task families, resources, adaptation, and risk. Recursive self-improvement remains a set of bounded mechanisms whose executor gains, improver gains, and evaluator dependence must be separated. Negative evidence, including task-order sensitivity, evaluator false acceptance, and limited transfer, is part of the scientific account rather than an inconvenience to remove from an aggregate score.

Fixed-policy interface changes and learning across interfaces also answer different questions. Harness decision geometry separates an interface's regularized optimum from the deployed policy's mismatch under finite-state assumptions; the coding experiments use a teacher proxy rather than access to the true optimum. Harness Scaling instead tests training across structured harness combinations and held-out compositions [7, 8]. Together they motivate measuring compatibility and adaptation separately. Neither a mismatch decomposition nor successful transfer among sampled interfaces establishes robustness to arbitrary future tool semantics, memory transformations, or authority changes.

34.2 Decision-preserving memory as an experimental target

A central research target is to minimize retained computation while preserving decisions that matter. The wrong objective is an unqualified maximize recall: some records should expire, some should never influence a given user, and some are irrelevant to the current task. The testable target is a constrained tradeoff between useful information, authority, cost, and loss.

Let U be a memory/context mechanism, $\mathcal{D}$ a declared task-stream distribution, $C(U)$ its complete cost, and $L(U)$ a vector of task and constraint losses. Let $\mathcal{U}_{\text{adm}}$ contain mechanisms satisfying the declared access and lifecycle constraints. Their feasible set is

$$\mathcal{Q} = \{(C(U), L(U)) : U \in \mathcal{U}_{\text{adm}}\}. \tag{34.1}$$

The Pareto frontier is the nondominated subset of $\mathcal{Q}$: no other feasible point has no greater cost and no greater loss on every coordinate, with a strict improvement on at least one coordinate. Empirical dominance requires uncertainty to be assessed jointly rather than inferred from favorable point estimates alone. A method that saves tokens while increasing violations is not unconditionally more efficient. A method that retains everything but requires unaffordable reconstruction is not a useful memory optimum.

One discriminating experiment uses paired histories that differ in a rare action-changing qualifier. Examples include a revoked permission, a temporal exception, a requirement about side-effect ordering, or a procedure valid only under one tool version. Compare raw history, fixed summarization, selective retrieval, typed memory, and learned editing at matched complete budgets. Hold the downstream policy fixed initially, then repeat after a controlled policy update. The estimands are the action-distribution change, constraint loss, task utility, and cost, not only a recall score.

The competing explanations are explicit. If a method succeeds only when the qualifier is repeated in the current prompt, it has not solved persistent retrieval or implicit relevance. If it retrieves the qualifier but fails to act on it, access is not the bottleneck. If it applies obsolete qualifiers, influence calibration or temporal validity is wrong. If success disappears after changing the model while retaining the same memory bank, the representation may encode policy-specific workarounds. These outcomes identify different next interventions.

An improved benchmark should expose the same latent facts through several surface forms and several future decision contexts. It should include irrelevant and contradictory memories, not only supportive ones. Evaluation queries must not update the frozen memory used by subsequent paired tests. The unit of uncertainty should include independent histories and task streams. This design can falsify broad claims of memory usefulness while preserving the cases in which a simple store or raw log is sufficient.

34.3 Conditioning under joint adaptation

Context editing, memory maintenance, and policy training can create incompatible conditional distributions. The failure is structural: an action generated under one information set is later trained as though it had been generated under another. Better final-answer supervision does not restore the missing behavior history. MemoryWalker supplies concrete exact and approximate corrections, while the broader system question is how to retain the evidence required to apply or validate those corrections [23].

A falsifiable hypothesis is that recording and respecting transform lineage improves deployment behavior after compaction beyond what can be explained by additional training compute. Test exact rollout-history conditioning where feasible, a naive flattened reconstruction, and an approximation with matched update budgets. Include a no-compaction control, vary the number and location of edits, and measure behavior log-probability diagnostics alongside held-out task outcomes. An approximation should

not be called equivalent merely because its average reward is close on one suite.

Joint updates require another factorial test. Train or evolve context policies and memory banks separately and together, then evaluate all cross-pairings with frozen policy checkpoints. Estimate interaction terms and migration failures under independent task streams. If joint gains disappear when component combinations are crossed, the system may have learned a useful but narrow compatibility relation. That is valuable engineering evidence, but it differs from a portable improvement to memory or reasoning.

Frozen cross-pairings measure compatibility of the resulting artifacts. They do not by themselves identify the causal benefit of joint training: separately trained checkpoints can differ in task exposure, optimization, or selection. That claim requires controlled training interventions with matched resources and independent training runs, followed by the same protected evaluation. More test trajectories for one trained checkpoint reduce evaluation uncertainty but do not create additional independent training runs.

Compatibility validation also has a resource constraint. Let A_t be the amount of new compatibility work introduced by releases and derived-state changes during interval t, and R_t the work actually validated. A simple backlog identity is

$$B_{t+1} = \max\{0, B_t + A_t - R_t\}. \tag{34.2}$$

If long-run required work exceeds validation capacity and no sound compositional shortcut is available, unresolved compatibility obligations accumulate. This is an accounting deduction, not a stochastic performance theorem. Possible responses include slower promotion, version-pinned sessions, narrower update scopes, or stronger contracts that reduce what must be retested. Silently assuming that old memories remain valid does not remove the obligation.

The research opportunity is to discover contracts that are both checkable and behaviorally meaningful. Syntactic schemas are easy to validate but weak about semantics. Full behavioral equivalence is generally intractable for open-ended stochastic agents. Intermediate invariants, such as provenance preservation, authority monotonicity, exact cache dependency identity, and restricted procedure applicability, can reduce risk without pretending to solve all semantic compatibility.

34.4 Verification whose errors are measured independently

Agent progress depends heavily on the quality of evaluators. A verifier can miss a requirement, reward a shortcut, crash selectively, or share the executor's mistaken interpretation. Increasing the number of judges does not guarantee independent evidence. The research target is to quantify the conditions under which a verifier's acceptance predicts the intended outcome and to detect when those conditions change.

TestJack's adversarial auditing and AgentHorizon-style evaluator inspection motivate a useful separation between an artifact's tested behavior, specification interpretation, and grading implementation [231, 22]. A witness test that fails a candidate and passes a reference is evidence of a difference; whether that difference violates the task still depends on the requirement. Reference agreement alone is not a complete oracle.

A discriminating verifier experiment inserts known fault classes into artifacts while preserving plausible surface quality. Vary the fault's visibility, execution path, and relationship to the written specification. Compare deterministic checks, model judges, adversarial test generation, and combinations. Report false acceptance, false rejection, abstention, cost, and fault-family transfer. Keep the final fault families hidden from verifier optimization. If the verifier is updated, independently reevaluate its errors before using it to select a stronger executor.

The unresolved problem is adaptive evaluator exploitation. A static false-acceptance estimate may not apply after an improver repeatedly searches for artifacts that the verifier accepts. Evaluation should therefore include an adaptive attacker constrained to the same information and resources available to the optimizer, followed by an independent audit. This does not produce a universal guarantee, but it tests the distribution shift induced by the very process the verifier is meant to supervise.

Statistical validity and semantic validity must be checked separately. REUSE controls erroneous promotions on its declared population under an independent initial sample and restricted decision-only feedback [41]. That guarantee concerns the supplied score; it cannot establish that the score measures the complete specification. Conversely, a semantically strong test set can still yield invalid statistical comparisons after unrestricted adaptive reuse. A credible improvement claim needs both a justified outcome definition and an evaluation protocol valid for the optimizer's actual information access.

34.5 World models and representations must face action tests

Predictive representations can improve perceptual and temporal structure while still omitting distinctions necessary for action. A visually coherent rollout can be wrong about a tool's side effects; a compact latent can predict typical observations while losing a rare precondition. The relevant research question is which representation preserves consequences under interventions, not merely which representation supports a strong passive probe.

Use controlled environments with known interventions to compare reconstructive, contrastive, JEPA-style, and action-conditioned representations at matched data and compute. Measure downstream planning or control with both frozen and adapted encoders. Include actions outside the behavior distribution and require uncertainty or abstention rather than extrapolation by default. Finally, validate selected plans through actual environment execution. A simulator-only gain can result from exploiting model error.

For tool environments, a world model can represent operation preconditions, possible effects, durations, and failure states rather than generate pixels. This typed model should be compared with direct execution, a learned unstructured predictor, and an oracle simulator in a controlled test bed. Replacing only the predictor with an oracle while holding the planner, representation interface, query budget, and evaluation population fixed estimates the effect of predictive error within that system; it is not a context-free decomposition of all errors. Comparing oracle-assisted planning with reactive control estimates a joint policy-and-information advantage under the declared budgets. Without these controls, a negative result cannot tell whether the model, planner, or task simply made planning unnecessary.

Generated imagery has no special evidential status. It can explain a mechanism or propose a possible future observation, but it cannot supply measured results. Diffusion and flow-based systems used as simulators need temporal, physical, and task-specific validation. Representation quality and generative realism are intermediate properties. Their practical importance depends on the decisions and failure events they change.

34.6 A research program for reliable improvement

Recursive self-improvement should be evaluated through independent outer-loop runs, held-out task populations, and complete resource accounting. RRSI, Harness^2, AIDE^2, MASS, Self-HPO, and skill-memory methods vary in update target, feedback, and persistence. Their results are informative precisely when those differences remain visible. A single label such as self-evolving agent does not define a comparable algorithmic class [27, 28, 347, 345, 344, 143].

The first hypothesis is that improvement of an executor transfers to improvement of an improver. Fix an initial executor, an outer-loop resource budget, and an independently measured task-utility functional Q. For improver I, define

$$g(I) = \mathbb{E}\Big[Q(A^I_{\text{final}}) - Q(A_0)\Big], \qquad \Delta = g(I_1) - g(I_0), \tag{34.3}$$

where the expectation includes independent search runs and their declared task sampling, and all proposal and selection costs fall within the fixed budget. Prespecify a meaningful advantage $\epsilon \geq 0$ and a confidence interval for Δ valid under that search and evaluation design. A lower endpoint above ϵ supports that advantage; an upper endpoint at or below ϵ rules it out at the interval's stated coverage. An interval crossing ϵ is inconclusive. Failure to demonstrate improvement is not proof of its absence. AIDE^2's inconclusive ignition test shows why executor transfer is insufficient. The second hypothesis is that jointly evolving policy and harness yields robust interaction gains. Test cross-pairings, multiple update rounds, task-order variation, and held-out environments; a gain tied to one checkpoint pair is a narrower result.

The third hypothesis is that co-evolving feedback improves alignment with the intended task rather than merely agreement between executor and evaluator. Evaluate the pair with an independent, fixed external audit and adversarial fault families. The fourth is that persistent procedural experience improves future adaptation without accumulating invalid assumptions. Test memory removal, shuffled evidence, stale schemas, model migration, and revocation. Each hypothesis has a different unit of intervention and should not be inferred from the same final leaderboard score.

Complete accounting includes failed proposals, discarded trajectories, evaluator calls, environment setup, memory construction, training, serving, and human review when used. Report the distribution across improvement runs and the regressions of selected artifacts. A method that produces one exceptional artifact after a large search can still be useful, but its scientific

claim is search efficiency under that budget, not reliable autonomous acceleration.

34.7 The remaining systems boundary

The October 2026 ecosystem makes persistence, tools, skills, and orchestration accessible through increasingly explicit protocol and runtime interfaces. The unresolved boundary is the evidence needed when these mechanisms are composed and updated. MCP conformance describes protocol behavior; it does not certify source truth or permission. Durable execution interfaces describe state management; they do not make every external effect reversible. Serving optimizations improve a resource envelope; they do not guarantee preserved task behavior after context or precision changes. The same distinction applies to scientific agents: an autonomous software loop does not establish autonomous physical experimentation, and a changed assay result does not isolate a changed reasoning policy.

Domain transfer also changes the admissible experiment. Cybersecurity capability in a controlled target environment does not grant authority over another system. Evidence from a noninfectious molecular-discovery sandbox does not establish clinical efficacy or the conditions for a different biological workflow. These boundaries belong to the task definition and evidence record; a larger model, longer session, or successful API call cannot discharge them.

The final research standard is therefore an auditable chain from problem formulation to delivered artifact: define the information available to each decision, identify mutable state and authority, derive the mechanism under explicit assumptions, connect it to an implementation, compare against controlled alternatives, and preserve failures and uncertainty. Where evidence is missing, the conclusion narrows. Where a simple baseline survives, complexity needs a measured justification. Where a current method improves only one layer, its cross-layer consequences remain part of the claim.

Generality will be better understood through experiments that discriminate these mechanisms than through broader labels. The technically productive questions are specific: which distinctions must a memory preserve, which histories a policy was trained on, which cache state remains valid, which operation committed, which verifier requirement was actually tested, and which improvement survives an independent task stream. Answering those questions is the path from a collection of model capabilities to reliable, adaptable agentic systems.

Figure 34.1 isolates the mechanism and its stated information boundaries.

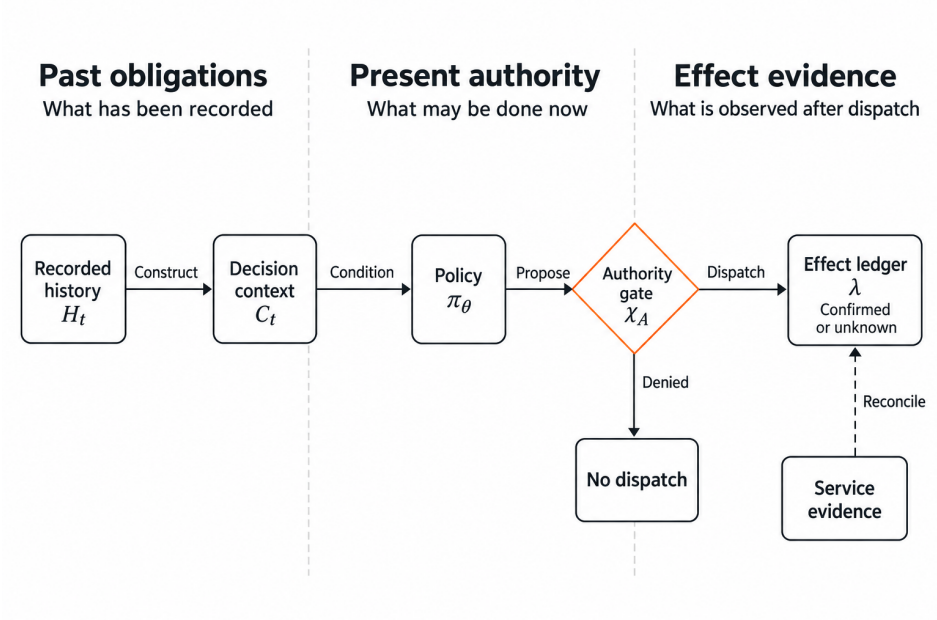

Figure 34.1: State continuity separates the information used by a policy, current dispatch authority and evidence of external effects. Permission does not establish source truth, and an authorized dispatch can leave the effect ledger unknown. Compatibility and independent evaluation are obligations for updates, rather than guarantees inherited from one component score.

Bibliography

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2210.03629v3

[2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2405.15793v3

[3] Z. Cao, H. Zhu, Y. Wang *et al.*, "Trial by Trace: Benchmarking AI Agent Harnesses for Reliability and Failure Attribution," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2610.32285v1

[4] J. Qin, J. Huang, Y. Xi, D. Yin, Y. Yu, W. Zhang, X. Sun, and W. Liu, "Evaluating Agentic Capabilities Before Post-Training: An End-to-End Approach," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2610.19425v1

[5] F. Chollet, "On the Measure of Intelligence," 2019, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1911.01547v2

[6] M. R. Morris, J. Sohl-Dickstein, N. Fiedel, T. Warkentin, A. Dafoe, A. Faust, C. Farabet, and S. Legg, "Levels of AGI for Operationalizing Progress on the Path to AGI," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2311.02462v5

[7] J. Mu, R. Shen, and Y. Shu, "Your Policy Is Secretly a Harness: The Decision Geometry of Agent Interfaces," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2610.37255v1

[8] F. Bai, H. Song, C. Hao, R. Tao, J. Yang, B. Dai, W. X. Zhao, and J.-R. Wen, "Harness Scaling for Generalizable Agentic Reinforcement Learning," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2610.29458v2

[9] J. Mu, R. Shen, and Y. Shu, "HRP: Harness realignment and policy training source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/2020-qqtcg/HRP/tree/c91d859c21f38f10e358091265eea6e571e23e47

[10] OpenAI, "OpenAI Agents Python: Session and memory source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/openai/openai-agents-python/tree/cd26bbbc52847845fde73b9952b51e62e2bcbdc0

[11] LangChain, "LangGraph: Checkpoint and store source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/langchain-ai/langgraph/tree/ffc3336367e8139668d11e5f0b0e6def57e7c762

[12] Google, "Agent Development Kit for Python: Session and memory services," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/google/adk-python/tree/3640e7d0c74bb2d377a58271ac6290d2e51ee60d

[13] Z. He, S. Zhang, Z. Zhou, Y. Yang, Y. Kang, Y. Zhang, L. K. Qiu, T. Y. Tsui, J. Xu, and C. Luo, "Agent Lightning v1.0: Towards Harnessed Agentic RL," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2608.17528v1

[14] Microsoft, "Agent Lightning: Harnessed agentic reinforcement learning source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/microsoft/agent-lightning/tree/05c7fe133175653e48c990546a3509e52dc56512

[15] Y. Hu, S. Liu, Y. Yue *et al.*, "Memory in the Age of AI Agents," 2025, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2512.13564v2

[16] C. Gou, "From Long Context to Selective Memory:A Survey of Memory-Centric Compression for Large Language Models," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.21165v1

[17] A. G. Colaco and N. Lahjouji, "What to Keep, What to Forget: A Rate–Distortion View of Memory Compaction in LLMs and Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.08032v1

[18] P. Liang, R. Bommasani, T. Lee *et al.*, "Holistic Evaluation of Language Models," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2211.09110v2

[19] X. Liu, H. Yu, H. Zhang *et al.*, "AgentBench: Evaluating LLMs as Agents," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2308.03688v3

[20] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2406.12045v1

[21] Y. Sun, X. Han, W. Zhang *et al.*, "Agents' Last Exam," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.05405v4

[22] X. H. Lù, D. Vattikonda, S. Hajimiri, F. P. Zadeh, P. BehnamGhader, G. Darwiche, A. Kazemnejad, C. Pal, A. Drouin, and S. Reddy, "AgentHorizon: Evaluating Agentic Judges for Long-Horizon Computer-Use Tasks," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.11050v1

[23] Z. J, X. Zhu, S. Huang, Z. Wang, P. Xie, and J. Ye, "MemoryWalker: Stop Training Agents on Contexts They Never Saw," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.00865v1

[24] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI Agents with Privilege Control," 2025, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2504.11703v3

[25] T. Shi, R. Rheem, D. Jiang *et al.*, "CyberGym-E2E: Scalable Real-World Benchmark for AI Agents' End-to-End Cybersecurity Capabilities," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.04460v2

[26] R. Shao, S. Z. Shen, J. O. Yin *et al.*, "Context Language Models," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.37725v1

[27] P. Xia, R. Han, Z. Wang *et al.*, "RRSI: Regularized Recursive Self-Improvement of Agent Harnesses," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.24972v3

[28] R. Xu, Y. Chen, Z. CuiZhu, B. D. Mishra, Y. Ming, H. Yu, R. Han, C.-Y. Lee, and T. Pfister, "Harness^2: Recursive Agent Harnessing for an Open World," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.14224v1

[29] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2302.04761v1

[30] M. D. Reid and R. C. Williamson, "Information, divergence and risk for binary experiments," *Journal of Machine Learning Research*, vol. 12, pp. 731–817, 2011, accessed 29 October 2026. [Online]. Available: https://www.jmlr.org/papers/volume12/reid11a/reid11a.pdf

[31] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2310.08560v2

[32] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2005.11401v4

[33] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998, accessed 29 October 2026. [Online]. Available: https://people.csail.mit.edu/lpk/papers/aij98-pomdp.pdf

[34] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999,

accessed 29 October 2026. [Online]. Available: http://incompleteideas.net/papers/SPS-aij.pdf

[35] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems*, vol. 12, 1999, pp. 1057–1063, accessed 29 October 2026. [Online]. Available: https://proceedings.neurips.cc/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf

[36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," 2017, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1707.06347v2

[37] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2305.18290v3

[38] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 1321–1330, accessed 29 October 2026. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[39] RRSI Authors, "RRSI: Regularized recursive self-improvement of agent harnesses, source code," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/google-research/rrsi/tree/be50316e1db05914068a973f322770ef08ed7ba1

[40] S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon, "Time-uniform, nonparametric, nonasymptotic confidence sequences," *The Annals of Statistics*, vol. 49, no. 2, pp. 1055–1080, 2021, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/1810.08240v9

[41] X. Sun, Y. Zeng, Z. She, and X. Wang, "Which self-improvements should we trust? reliable self-improvement when agents reuse their benchmarks," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2609.33180v2

[42] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *Journal of the American Statistical Association*, vol. 58,

no. 301, pp. 13–30, 1963, accessed 29 October 2026. [Online]. Available: https://www.cs.rpi.edu/academics/courses/spring06/random/hoefding.pdf

[43] DeepSeek-AI, "DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.19969v1

[44] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," 2015, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1508.07909v5

[45] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing," 2018, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1808.06226v1

[46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," 2017, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1706.03762v7

[47] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context," 2019, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1901.02860v3

[48] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2312.00752v2

[49] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2104.09864v5

[50] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," 2017, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1701.06538v1

[51] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer

Models from Multi-Head Checkpoints," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2305.13245v3

[52] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V3 Technical Report," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2412.19437v2

[53] G. Monea, K. Ramji, Y. El-Kurdi, L. A. Lastras, Y. Artzi, N. Godey, and R. F. Astudillo, "The Surprising Effectiveness of Shared Memory in Looped Transformers," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.02383v1

[54] T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, "FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2603.05451v1

[55] Dao-AILab, "Dao-AILab/flash-attention," Versioned software repository, accessed 28 October 2026. [Online]. Available: https://github.com/Dao-AILab/flash-attention/tree/98c1cee0ed59307833841cdbf567fe336ceec13a

[56] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2205.14135v2

[57] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2307.08691v1

[58] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2407.08608v2

[59] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2309.06180v1

[60] X. Zhu, P. L. Yi, Z. Cheng, A. Lv, J. Liu, L. Ying, Y. Ma, and X. Dong, "Periodic Weak Spots: Phase Sensitivity from Chunked

KV-Cache Compression," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.36322v1

[61] vllm-project, "vllm-project/vllm," Versioned software repository, accessed 28 October 2026. [Online]. Available: https://github.com/vllm-project/vllm/tree/c91ab83dced6f9a5b5227c6f83595abbf4868d6e

[62] L. Mur-Labadia, M. Muckley, A. Bar, M. Assran, K. Sinha, M. Rabbat, Y. LeCun, N. Ballas, and A. Bardes, "V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2603.14482v3

[63] Meta FAIR, "facebookresearch/vjepa2," accessed 28 October 2026. [Online]. Available: https://github.com/facebookresearch/vjepa2/tree/204698b45b3712590f06245fbfba32d3be539812

[64] Y. Zhang, J. Zhao, C. Fan, J. Hu, X. Yuan, C. Bai, and X. Li, "V-JEPA Policy: Building Effective World-Action Models on Predictive Visual Latents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.37250v2

[65] W. Li, G. Li, K. Maeda, T. Ogawa, and M. Haseyama, "Predictive but Not Plannable: RC-aux for Latent World Models," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2605.07278v1

[66] J. Bai and J. Xiong, "Temporal-Distance JEPA: Plan-Aware Representation Learning for Latent World Model Predictive Control," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.25337v2

[67] Y. S. Jang and M. Ryoo, "Latent Projection for Latent Planning," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.26979v1

[68] yjang43, "yjang43/lp2," accessed 28 October 2026. [Online]. Available: https://github.com/yjang43/lp2/tree/a05346a6d14dade575b08beef54136004bfc8eef

[69] A. Dosovitskiy, L. Beyer, A. Kolesnikov *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," 2020, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2010.11929v2

[70] A. Radford, J. W. Kim, C. Hallacy *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," 2021, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2103.00020v1

[71] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2301.08243v3

[72] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas, "Revisiting Feature Prediction for Learning Visual Representations from Video," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2404.08471v1

[73] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2304.08485v2

[74] facebookresearch, "facebookresearch/ijepa," Versioned software repository, accessed 28 October 2026. [Online]. Available: https://github.com/facebookresearch/ijepa/tree/52c1ae95d05f743e000e8f10a1f3a79b10cff048

[75] ——, "facebookresearch/jepa," Versioned software repository, accessed 28 October 2026. [Online]. Available: https://github.com/facebookresearch/jepa/tree/51c59d518fc63c08464af6de585f78ac0c7ed4d5

[76] H.-W. Huang, J. Shangguan, J. Lu, and J.-N. Hwang, "LeFlow: Generative Latent Flow Planning for World Models," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2608.24855v1

[77] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering Diverse Domains through World Models," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2301.04104v2

[78] D. Zhao, Z. Yue, S. Zhuang, J. He, X. Chen, Z. Wang, Y. Liu, L. Wang, and Y. Wang, "Beyond a single latent space: a dual-latent world model for long-horizon planning," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.37644v1

[79] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," 2020, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2006.11239v2

[80] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," 2021, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2112.10752v2

[81] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2212.09748v2

[82] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow Matching for Generative Modeling," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2210.02747v2

[83] danijar, "danijar/dreamerv3," accessed 28 October 2026. [Online]. Available: https://github.com/danijar/dreamerv3/tree/e01491fad6434b2245a3b8ca201dd7faedcc458c

[84] huggingface, "huggingface/diffusers," accessed 28 October 2026. [Online]. Available: https://github.com/huggingface/diffusers/tree/40e551a4380c0a48333f3eeb032ed7da1f361297

[85] Q. Long, X. Chen, J. Chen, H. Zhang, Q. Hu, J. Bao, and W. Wang, "From Traces to Agentic Worlds: Agentic Language World Models for Interactive Environment Simulation," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.06100v1

[86] ruyue0001, "ruyue0001/trace2env," accessed 28 October 2026. [Online]. Available: https://github.com/ruyue0001/trace2env/tree/52891471d8c3fbe883bbf99899aba875f0961bcd

[87] Z. Wang, Q. Wu, H. Wang *et al.*, "Agent World Model in the Wild: Computer-Use Agent Environment Synthesis," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.26828v1

[88] Y. Zuo, Z. Xiao, L. Sheng *et al.*, "Qwen-AgentWorld: Language World Models for General Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.24597v1

[89] X. Zhang, W. Zhang, S.-K. Ng, and Y. Deng, "Self-Evolving World Models for LLM Agent Planning," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.30639v2

[90] DeepSeek-AI, "DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.19348v1

[91] J. Hoffmann, S. Borgeaud, A. Mensch *et al.*, "Training Compute-Optimal Large Language Models," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2203.15556v1

[92] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," 2019, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1910.02054v3

[93] deepspeedai, "deepspeedai/DeepSpeed," Versioned software repository, accessed 28 October 2026. [Online]. Available: https://github.com/deepspeedai/DeepSpeed/tree/3dd1293fbfbc718496987dc2707a215e7a1abee2

[94] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/1909.08053v4

[95] NVIDIA, "NVIDIA/Megatron-LM," Versioned software repository, accessed 28 October 2026. [Online]. Available: https://github.com/NVIDIA/Megatron-LM/tree/f77f360c96b6a5ec032778811868196ec75daed9

[96] N. Lu, B. Lin, S. Liu *et al.*, "Policy and World Modeling Co-Training for Language Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.02388v2

[97] ColinLu50, "ColinLu50/Policy-WorldModel-Agent," accessed 28 October 2026. [Online]. Available: https://github.com/ColinLu50/Policy-WorldModel-Agent/tree/4feb96444ec220a7e36cbe1d739cdfa66bd70f1c

[98] M. Ran, M. Chen, D. Huang, W. Xiong, F. Huang, and W. Liu, "Model What Matters: Decision-Guided Selective World Modeling for Language Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.19642v1

[99] R. Xu, X. Yu, Y. Tang, C. Shang, and N. Singh, "How do World Models and Policies Compose in LLM Agents? A Joint Spectral and Behavioral Account," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2608.30067v1

[100] L. Ouyang, J. Wu, X. Jiang *et al.*, "Training language models to follow instructions with human feedback," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2203.02155v1

[101] Z. Shao, P. Wang, Q. Zhu *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2402.03300v3

[102] huggingface, "huggingface/trl," accessed 28 October 2026. [Online]. Available: https://github.com/huggingface/trl/tree/e29597afcf973b91b412662988cf7dc88dde4cd1

[103] X. Yu, B. Peng, R. Xu, H. Zou, Q. Wu, H. Cheng, W. Yao, N. Singh, Z. Yu, and J. Gao, "OpenForgeRL: Train Harness-native Agents in Any Environment," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.21557v3

[104] J. Hu, H. Li, H. Zhang *et al.*, "Molt: A Scalable PyTorch-Native Training Framework for Agentic Reinforcement Learning," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.21653v3

[105] P. Dahal, A. Bakhtin, T. Cohen *et al.*, "Thinking Before Thinking: Scaling Agentic Inference Through Meta-Reasoning," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.38147v1

[106] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2201.11903v6

[107] M. Chen, J. Tworek, H. Jun *et al.*, "Evaluating Large Language Models Trained on Code," 2021, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2107.03374v2

[108] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of

Thought Reasoning in Language Models," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2203.11171v4

[109] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2305.10601v2

[110] princeton-nlp, "princeton-nlp/tree-of-thought-llm," accessed 28 October 2026. [Online]. Available: https://github.com/princeton-nlp/tree-of-thought-llm/tree/8050e67d0e3a0fddc424d7fa5801538722a4c4cc

[111] WorldEvolver Authors, "magicgh/WorldEvolver," accessed 28 October 2026. [Online]. Available: https://github.com/magicgh/WorldEvolver/tree/f7013c63a0b012d3ba1f0fbbefab9218ba4d110e

[112] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2303.11366v4

[113] S. Zerhoudi, J. Mitrovic, and M. Granitzer, "The Compaction Cliff in Long-Running AI Agent Memory," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2608.22752v1

[114] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2307.03172v3

[115] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, "RULER: What's the Real Context Size of Your Long-Context Language Models?" 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2404.06654v3

[116] Y. Bai, X. Lv, J. Zhang *et al.*, "LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2308.14508v2

[117] NVIDIA, "RULER: Long-Context Language Model Evaluation Repository," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/NVIDIA/RULER/tree/c3f5e3b4f87f97e048793bb510a3a6b19a46bf3a

[118] F. Woolford, "Maximising Transparency in Context Compaction Improves Retention of Facts and Commitments for Lightweight Coding Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.20116v1

[119] X. Li, R. Ming, M. Chu, S. Shao, R. Jin, and C. Xiong, "ACM: Agentic Context Management for Long Horizon Tasks," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.23809v1

[120] G. Min, L. Wu, M. Shi, Y. He, M. Darbari, L. Hong, and C. Chen, "Adapting Context Compression for Long-Horizon Agents with Counterfactual Continuations," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.36526v1

[121] Z. Pan, Q. Pei, J. Lu, H. Lin, H. V. Zhao, D. Yin, and X. Sun, "ContextPilot: Teaching Agents for Proactive Context Management via Fine-grained RL," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2608.28476v1

[122] Y. Li, Z. Hou, Y. Jing, J. Tang, and Y. Dong, "CompactionRL: Reinforcement Learning with Context Compaction for Long-Horizon Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.05378v2

[123] G. Min, L. Wu, M. Darbari, C. Chen, and L. Hong, "Towards Reliable Context Compression for Long-Horizon Agents: An Empirical Study of Execution Instability," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2608.06503v2

[124] Repository contributors, "facebookresearch/context-language-models," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/facebookresearch/context-language-models/tree/483198229ab7b90c0eed4651306ab3b3505aaf50

[125] ——, "lixiaochuan2020/agentic-context-management," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/lixiaochuan2020/agentic-context-management/tree/f06f90e728af8580a4515812425c1620144145a2

[126] OpenAI, "Agents Documentation," 2026, accessed 28 October 2026. [Online]. Available: https://developers.openai.com/api/docs/guides/agents

[127] LangChain, "LangGraph Persistence," 2026, accessed 28 October 2026. [Online]. Available: https://docs.langchain.com/oss/python/langgraph/persistence

[128] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978. [Online]. Available: https://www.microsoft.com/en-us/research/publication/time-clocks-ordering-events-distributed-system/

[129] LangChain, "LangGraph Checkpointers," 2026, accessed 29 October 2026. [Online]. Available: https://docs.langchain.com/oss/python/langgraph/checkpointers

[130] M. P. Herlihy and J. M. Wing, "Linearizability: A Correctness Condition for Concurrent Objects," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990. [Online]. Available: https://www.cs.cmu.edu/~wing/publications/HerlihyWing90.pdf

[131] OpenAI, "Agents API: Events and Items," 2026, accessed 29 October 2026. [Online]. Available: https://developers.openai.com/api/docs/guides/agents-api/sessions/events

[132] Z. Dai, S. Deng, S. Guan, Y. Tian, X. Yao, X. Yan, and J. Cheng, "RecMem: Recurrence-based Memory Consolidation for Efficient and Effective Long-Running LLM Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2605.16045v1

[133] T. Fei, J. Kang, M. Zheng, X. He, and X. Yu, "Hy-Memory-30B-A3B: Attributed Reinforcement Learning for Long-Term Memory Writing," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.22196v1

[134] R. Li, M. Du, B. Xu, and Z. Mao, "Keep It InMind: Benchmarking the Implicit-Association Blind Spot in Agent Memory," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.24368v1

[135] W.-C. Huang, W. Zhang, Y. Liang *et al.*, "A Survey of Agent Memory in the Second Half: Towards Self-Evolving and Long-Horizon Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2602.06052v4

[136] Repository contributors, "CaiusDai/RecMem," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/CaiusDai/RecMem/tree/a84252f6e5587fd4a8caac03ec9f6c732b7a7f35

[137] K. Ji, Z. Chen, W. Zhao *et al.*, "Learning personalized memory gating from delayed one-sided feedback," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.23812v1

[138] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2304.03442v2

[139] T. Zhang, X. Wang, Q. Zhang, Q. Chen, K. Li, Y. Chen, D. Wang, H. Meng, and Y. Lu, "Triggerbench: Investigating prospective memory for large language models," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.23459v1

[140] Z. Xiang, Z. Chen, Y. Tang, Z. Wei, R. Ning, Y. Lin, Q. Zhang, and J. Su, "MemSyco-Bench: Benchmarking Sycophancy in Agent Memory," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2607.01071v3

[141] J. Tan, F. Sun, X. Ma, S. Zhang, D. Su, Q. Cao, D. Shi, H. Shen, and X. Cheng, "Membubble: Benchmarking how agent memory shapes llm response diversity," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.25127v1

[142] Repository contributors, "imlrz/InMind," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/imlrz/InMind/tree/5a2ab2686d3d5b832575f0288d41ecc93eab358b

[143] R. Xu, T. Yang, and W.-C. Huang, "HyperSkill: Self-Evolving LLM Agents via Hypergraph-Structured Skill Memory," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2608.16114v1

[144] Repository contributors, "rux001/HyperSkill," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/rux001/HyperSkill/tree/a34978aa7439d2eff7ccc3d2ad5372b34d285bd5

[145] Agent Skills, "Agent Skills Specification," 2026, accessed 28 October 2026. [Online]. Available: https://agentskills.io/specification

[146] G. Bakal, "Knowledge Activation: AI Skills as the Institutional Knowledge Primitive for Agentic Software Development," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2603.14805v2

[147] R. Li, L. Zhang, B. Xu, M. Du, Z. Fu, and W. Chen, "When Your Agent Opens the Chat App: Agent-Controlled Search over Raw Chat Logs Rivals Structured Memory," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2608.12888v2

[148] C. Ke, X. Chen, X. Shen, K. Chen, S. An, J. Dong, R. Xu, R. Qiao, and X. Sun, "Disentangling Long-Term Memory via Latent Neuro-Symbolic Reasoning," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2609.18461v2

[149] T. E. Kim, X. He, D. Jain, A. Agrawal, N. Arabzadeh, and F. Diaz, "Multi-Agent Transactive Memory," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.19911v2

[150] S. Ji, Y. Li, and B. Hooi, "Memory is Reconstructed, Not Retrieved: Graph Memory for LLM Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.06036v1

[151] B. Yan, C. Li, H. Qian, S. Lu, C. Li, and Z. Liu, "Just-In-Time Agent Memory with Runtime Agentic Research," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.34385v1

[152] Repository contributors, "Ji-shuo/MRAgent," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/Ji-shuo/MRAgent/tree/7441506db984b7c4da32e8dbeb2527f2e351270a

[153] ——, "VectorSpaceLab/general-agentic-memory," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/VectorSpaceLab/general-agentic-memory/tree/565db2cc2518d377e44389b82aecf3cc129d5fe5

[154] ——, "kimdanny/matm," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/kimdanny/matm/tree/2fb906b1a572f9ced0177741dee44be70ef1223c

[155] R. Cao, F. Zhao, F. Yao, L. Dong, J. Xu, Y. Zhao, H. Zhang, and L. Xiao, "MemCalib: Benchmarking and Optimizing Memory Use in LLM Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.24259v3

[156] Y. Feng, X. Wang, J. Yang, X. Shi, P. Xie, P. S. Yu, X. Wu, and J. Ye, "Evolving Together, Failing Apart: The Skill-Policy Co-evolution Trap in Self-Improving Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.20845v1

[157] Y. Du, Y. Zhou, Y. Ge, J. Wang, X. Pang, S. Tang, T. Zheng, B. Dai, J. Yang, and S. Chen, "HyperTool: Beyond Step-Wise Tool Calls for Tool-Augmented Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2606.13663v1

[158] Z. Cui, H. Xie, J. Yuan *et al.*, "Uno-Orchestra: Parsimonious Agent Routing via Selective Delegation," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2605.05007v1

[159] Model Context Protocol Contributors, "Tools," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[160] L. Zhang, D. Song, Z. Wu *et al.*, "PruneTIR: Inference-Time Tool Call Pruning for Effective yet Efficient Tool-Integrated Reasoning," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2605.09931v1

[161] Project contributors, "Progent implementation," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/sunblaze-ucb/progent/tree/b02783b896d4bf01a2adf24738015e258db8d35b

[162] Model Context Protocol Contributors, "Model Context Protocol 2026-07-28 Specification," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28

[163] Project contributors, "MCP 2026-07-28 TypeScript Schema," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/modelcontextprotocol/modelcontextprotocol/blob/271ecc9accafdd9b83a3c869fa67c22953b2af80/schema/2026-07-28/schema.ts

[164] A2A Project, "Life of a Task, A2A v1.0.1," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/a2aproject/A2A/blob/3303592588e388e62e0f69f701af531d2f4e3991/docs/topics/life-of-a-task.md

[165] ——, "A2A Protocol Specification, v1.0.1," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/a2aproject/A2A/blob/3303592588e388e62e0f69f701af531d2f4e3991/docs/specification.md

[166] Model Context Protocol, "Versioning," accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/docs/2026-07-28/learn/versioning

[167] Model Context Protocol Contributors, "Versioning and Compatibility," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/basic/versioning

[168] ——, "Discovery," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/discover

[169] ——, "Base Protocol Overview," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/basic/index

[170] ——, "Resources," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/resources

[171] ——, "Streamable HTTP," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/basic/transports/streamable-http

[172] ——, "Tasks Extension," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/extensions/tasks/overview

[173] ——, "Authorization," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization

[174] A2A Project, "A2A Agent Discovery, v1.0.1," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/a2aproject/A2A/blob/3303592588e388e62e0f69f701af531d2f4e3991/docs/topics/agent-discovery.md

[175] Project contributors, "Architecture Overview," 2026, accessed 28 October 2026. [Online]. Available: https://modelcontextprotocol.io/docs/2026-07-28/learn/architecture

[176] X. Ning, K. Tieu, D. Fu *et al.*, "Code as Agent Harness," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2605.18747v1

[177] Google Research Contributors, "Harness2 Source Code," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/google-research/harness2/tree/1cfe254e8d25b15f37048c3730134686abe60335

[178] H. Lin, J. Liu, T. Yu, and W. Zhao, "LatComm: Reusable context compression for latent communication between large language models," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.26840v1

[179] Q. Wu, G. Bansal, J. Zhang *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2308.08155v2

[180] S. Hong, M. Zhuge, J. Chen *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2308.00352v7

[181] Z. Du, R. Wang, H. Bai, Z. Cao, X. Zhu, Y. Cheng, B. Zheng, W. Chen, and H. Ying, "Enabling agents to communicate entirely in latent space," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Jul. 2026, pp. 27 106–27 129. [Online]. Available: https://aclanthology.org/2026.acl-long.1248/

[182] LatComm Contributors, "LatComm Source Code," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/huawei-lin/LatComm/tree/0aff932047eca0056bc3960e9921c70f9e5db033

[183] K. Samway, D. D'souza, O. D. Domingues, M. Fadaee, T. Yi-Chern, and N. Dziri, "Scaling inference-time compute with teams of coding agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.17471v1

[184] openai, "openai/openai-agents-python release v0.24.0," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/openai/openai-agents-python/releases/tag/v0.24.0

[185] google, "google/adk-python release v2.13.0," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/google/adk-python/releases/tag/v2.13.0

[186] LangGraph, "GitHub - langchain-ai/langgraph: Build resilient agents. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/langchain-ai/langgraph

[187] Deep Agents, "GitHub - langchain-ai/deepagents: The batteries-included agent harness. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/langchain-ai/deepagents

[188] OpenAI Agents SDK, "GitHub - openai/openai-agents-python: A lightweight, powerful framework for multi-agent workflows · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/openai/openai-agents-python

[189] PydanticAI, "GitHub - pydantic/pydantic-ai: How Python does AI. Agents, realtime voice, image generation, embeddings. Every model, every interface, typed end to end. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/pydantic/pydantic-ai

[190] Anthropic, "Agent SDK Overview," 2026, accessed 29 October 2026. [Online]. Available: https://code.claude.com/docs/en/agent-sdk/overview

[191] Claude Managed Agents, "Claude Managed Agents overview - Claude Platform Docs," 2026, accessed 28 October 2026. [Online]. Available: https://platform.claude.com/docs/en/managed-agents/overview

[192] Agent Development Kit — ADK, "GitHub - google/adk-python: An open-source, code-first Python toolkit for building, evaluating, and deploying sophisticated AI agents with flexibility and control. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/google/adk-python

[193] Microsoft Agent Framework, "GitHub - microsoft/agent-framework: A framework for building, orchestrating and deploying AI agents and

multi-agent workflows with support for Python and .NET. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/microsoft/agent-framework

[194] Agno / AgentOS, "GitHub - agno-agi/agno: The self-improving agent platform · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/agno-agi/agno

[195] LlamaAgents / Agent Workflows, "GitHub - run-llama/llama-agents: Llama Agents + Workflows are an event-driven, async-first, step-based way to control the execution flow of AI applications like agents. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/run-llama/llama-agents

[196] Strands Agents / Harness SDK, "GitHub - strands-agents/harness-sdk: Build an agent harness and control it end-to-end. Open-source SDK for production AI agents in Python & TypeScript - any model, any cloud. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/strands-agents/harness-sdk

[197] OpenClaw, "GitHub - openclaw/openclaw: The AI that really does things. Any OS. Any Platform. The lobster way. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/openclaw/openclaw

[198] Mastra, "GitHub - mastra-ai/mastra: Mastra is the modern TypeScript framework for AI-powered applications and agents. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/mastra-ai/mastra

[199] Claude Agent SDK, "SDKs, CLI, and libraries - Claude Platform Docs," 2026, accessed 28 October 2026. [Online]. Available: https://platform.claude.com/docs/en/cli-sdks-libraries/overview

[200] Haystack, "GitHub - deepset-ai/haystack: Open-source AI orchestration framework for building context-engineered, production-ready LLM applications. Design modular pipelines and agent workflows with explicit control over retrieval, routing, memory, and generation. Built for scalable agents, RAG, multimodal applications, semantic search, and conversational systems. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/deepset-ai/haystack

[201] NeMo Fabric, "NVIDIA NeMo Fabric Documentation | NVIDIA NeMo Fabric," 2026, accessed 28 October 2026. [Online]. Available: https://docs.nvidia.com/nemo/fabric/about-nemo-fabric/overview/

[202] smolagents, "GitHub - huggingface/smolagents: smolagents: a barebones library for agents that think in code. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/huggingface/smolagents

[203] DeepSeek Harness, "GitHub - deepseek-ai/deepseek-harness: DeepSeek Harness: Everything is a Plugin. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/deepseek-ai/deepseek-harness/

[204] AgentScope, "GitHub - agentscope-ai/agentscope: Build and run agents you can see, understand and trust. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/agentscope-ai/agentscope

[205] CrewAI, "GitHub - crewAIInc/crewAI: Framework for orchestrating role-playing, autonomous AI agents. By fostering collaborative intelligence, CrewAI empowers agents to work together seamlessly, tackling complex tasks. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/crewAIInc/crewAI

[206] Pi, "Pi," 2026, accessed 28 October 2026. [Online]. Available: https://pi.dev/

[207] DSPy, "GitHub - stanfordnlp/dspy: DSPy: The framework for programming—not prompting—language models · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/stanfordnlp/dspy

[208] OGX, formerly Llama Stack, "GitHub - ogx-ai/ogx: Open GenAI Stack · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/ogx-ai/ogx

[209] Qwen-Agent, "GitHub - QwenLM/Qwen-Agent: Agent framework and applications built upon Qwen>=3.0, featuring Function Calling, MCP, Code Interpreter, RAG, Chrome extension, etc. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/QwenLM/Qwen-Agent

[210] n8n, "GitHub - n8n-io/n8n: Fair-code workflow automation platform with native AI capabilities. Combine visual building with custom

code, self-host or cloud, 400+ integrations. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/n8n-io/n8n

[211] Langflow, "GitHub - langflow-ai/langflow: Langflow is a powerful tool for building and deploying AI-powered agents and workflows. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/langflow-ai/langflow

[212] Dify, "GitHub - langgenius/dify: Build Agentic workflows, RAG pipelines, with rich AI model and tool support on one collaborative workspace. Deploy on cloud, VPC, or self-hosted, so teams move from prototype to production without rebuilding the stack. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/langgenius/dify

[213] Elastic Agent Builder, "Elastic Agent Builder | Elastic Docs," 2026, accessed 28 October 2026. [Online]. Available: https://www.elastic.co/docs/explore-analyze/ai-features/elastic-agent-builder

[214] CAMEL, "GitHub - camel-ai/camel: CAMEL: The first and the best multi-agent framework. Finding the Scaling Law of Agents. https://www.camel-ai.org · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/camel-ai/camel

[215] Coze Studio, "GitHub - coze-dev/coze-studio: An AI agent development platform with all-in-one visual tools, simplifying agent creation, debugging, and deployment like never before. Coze your way to AI Agent creation. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/coze-dev/coze-studio

[216] UiPath Maestro, "Maestro Orchestrate - September 2026," 2026, accessed 28 October 2026. [Online]. Available: https://docs.uipath.com/maestro/automation-cloud/latest/release-notes/september-2026

[217] Agent Bricks, "What is Agent Bricks? | Databricks on AWS," 2026, accessed 28 October 2026. [Online]. Available: https://docs.databricks.com/aws/en/agents/concepts

[218] Bedrock AgentCore, "Amazon Bedrock AgentCore FAQs - AWS," 2026, accessed 28 October 2026. [Online]. Available: https://aws.amazon.com/bedrock/agentcore/faqs/

[219] Gemini Enterprise Agent Platform, "Agents overview | Gemini Enterprise Agent Platform | Google Cloud Documentation," 2026, accessed 28 October 2026. [Online]. Available: https://docs.cloud.google.com/gemini-enterprise-agent-platform/agents

[220] Agents API, "Agents | OpenAI API," 2026, accessed 28 October 2026. [Online]. Available: https://developers.openai.com/api/docs/guides/agents

[221] Agents & Conversations API, "Agents & Conversations | Mistral Docs," 2026, accessed 28 October 2026. [Online]. Available: https://docs.mistral.ai/studio/agents/agents-api

[222] langchain-ai, "langchain-ai/langgraph release 1.2.18," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/langchain-ai/langgraph/releases/tag/1.2.18

[223] pydantic, "pydantic/pydantic-ai release v2.63.0," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/pydantic/pydantic-ai/releases/tag/v2.63.0

[224] microsoft, "microsoft/agent-framework release dotnet-1.26.0," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/microsoft/agent-framework/releases/tag/dotnet-1.26.0

[225] Microsoft, "Microsoft Agent Framework Python 1.23.0 Release," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/microsoft/agent-framework/releases/tag/python-1.23.0

[226] crewAIInc, "crewAIInc/crewAI release 1.15.32," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/crewAIInc/crewAI/releases/tag/1.15.32

[227] huggingface, "huggingface/smolagents release v1.26.0," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/huggingface/smolagents/releases/tag/v1.26.0

[228] LangChain Contributors, "LangGraph DeltaChannel Source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/langchain-ai/langgraph/blob/ffc3336367e8139668d11e5f0b0e6def57e7c762/libs/langgraph/langgraph/channels/delta.py

[229] OpenAI, "OpenAI Agents SDK RunState Source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/openai/openai-agents-python/blob/cd26bbbc52847845fde73b9952b51e62e2bcbdc0/src/agents/run_state.py

[230] Google ADK Contributors, "ADK InMemorySessionService Source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/google/adk-python/blob/3640e7d0c74bb2d377a58271ac6290d2e51ee60d/src/google/adk/sessions/in_memory_session_service.py

[231] S. Yao, H. Wang, K. Sen, S. Chen, B. Ray, and D. Song, "TestJack: Should you trust the results in coding benchmarks? Agentic Coding Benchmarks Auditing via Evaluator Evolution," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.10619v1

[232] X. Deng, J. Da, E. Pan *et al.*, "SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?" 2025, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2509.16941v2

[233] S. Zou, Z. Ma, Y. Zou, Y. Wang, L. Chen, Z. Chen, G. Chen, X. Chu, and F. Zhao, "Scaling Agentic Data for Long-Horizon Terminal Intelligence," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.13166v2

[234] SWE-agent, "SWE-agent/mini-swe-agent release v2.4.6," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/SWE-agent/mini-swe-agent/releases/tag/v2.4.6

[235] SWE-agent Contributors, "mini-swe-agent DefaultAgent Source," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/SWE-agent/mini-swe-agent/blob/04d809ceab9df28f9adaed044884180159172930/src/minisweagent/agents/default.py

[236] Lovable, "Welcome to Lovable," 2026, accessed 29 October 2026. [Online]. Available: https://docs.lovable.dev/introduction/welcome

[237] The Antigravity Team, "Introducing Google Antigravity 2.0," 2026, accessed 29 October 2026. [Online]. Available: https://antigravity.google/blog/introducing-google-antigravity-2

[238] Codex, "Codex as a platform: build on the open agent harness | OpenAI Developers," 2026, accessed 28 October 2026. [Online]. Available: https://developers.openai.com/blog/codex-as-a-platform

[239] OpenHands, "OpenHands | Open Source AI Coding Agent Platform," 2026, accessed 28 October 2026. [Online]. Available: https://www.openhands.dev/

[240] goose, "goose | Your open source AI agent," 2026, accessed 28 October 2026. [Online]. Available: https://goose-docs.ai/

[241] Warp, "Warp — Open Infrastructure for Cloud Software Factories," 2026, accessed 28 October 2026. [Online]. Available: https://www.warp.dev/

[242] Kilo, "Kilo – Open Source AI Coding Agent in IDE, CLI and Cloud," 2026, accessed 28 October 2026. [Online]. Available: https://kilo.ai/

[243] OpenCode, "OpenCode | The open source AI coding agent," 2026, accessed 28 October 2026. [Online]. Available: https://opencode.ai/

[244] Letta Code / Letta harness, "GitHub - letta-ai/letta-code: Stateful agents that are like people, with memory, identity, and the ability to learn and adapt · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/letta-ai/letta-code

[245] Claude Code, "GitHub - anthropics/claude-code: Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/anthropics/claude-code

[246] GitHub Copilot, "GitHub Copilot weekly releases — October 5 - GitHub Changelog," 2026, accessed 28 October 2026. [Online]. Available: https://github.blog/changelog/2026-10-09-github-copilot-weekly-releases-october-5/

[247] Cline, "Cline - AI Coding, Open Source and Open Choice," 2026, accessed 28 October 2026. [Online]. Available: https://cline.bot/

[248] Zed Agent, "Zed — Your last next editor," 2026, accessed 28 October 2026. [Online]. Available: https://zed.dev/

[249] Cursor, "AI Coding Agent for Building Ambitious Software | Cursor," 2026, accessed 28 October 2026. [Online]. Available: https://cursor.com/

[250] Sourcegraph Agentic Batch Changes, "Agentic Batch Changes - Sourcegraph docs," 2026, accessed 28 October 2026. [Online]. Available: https://sourcegraph.com/docs/agentic-batch-changes

[251] Hermes Agent, "GitHub - NousResearch/hermes-agent: The agent that grows with you · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/NousResearch/hermes-agent

[252] Mistral Vibe, "Vibe | Mistral Docs," 2026, accessed 28 October 2026. [Online]. Available: https://docs.mistral.ai/vibe

[253] Devin, "Devin: the autonomous software engineer," 2026, accessed 28 October 2026. [Online]. Available: https://devin.ai/

[254] Factory Droids, "Factory | Agent-Native Software Development," 2026, accessed 28 October 2026. [Online]. Available: https://factory.com/

[255] Amp, "Amp: Coding agent and dev environment built for the frontier," 2026, accessed 28 October 2026. [Online]. Available: https://ampcode.com/

[256] Qwen Code Contributors, "Qwen Code Source and Interfaces," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/QwenLM/qwen-code/tree/66d2eae154d0640aac45b1236d911ee8eb8ce84a

[257] mini-swe-agent, "GitHub - SWE-agent/mini-swe-agent: The 100 line AI agent that solves GitHub issues or helps you in your command line. Radically simple, no huge configs, no giant monorepo—but scores >74% on SWE-bench verified! · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/SWE-agent/mini-swe-agent

[258] Junie, "Junie," 2026, accessed 28 October 2026. [Online]. Available: https://junie.jetbrains.com/

[259] Harness Cosmos / Auggie, "Augment Code: Agentic software development at organizational scale," 2026, accessed 28 October 2026. [Online]. Available: https://www.augmentcode.com/

[260] Kiro, "How Kiro works - Features - Docs - Kiro," 2026, accessed 28 October 2026. [Online]. Available: https://kiro.dev/docs/how-kiro-works/

[261] Rovo Dev, "Rovo Dev | AI Coding Agent for Software Teams | Atlassian," 2026, accessed 28 October 2026. [Online]. Available: https://www.atlassian.com/software/rovo-dev

[262] Replit Agent, "AI App and Website Builder | Replit," 2026, accessed 28 October 2026. [Online]. Available: https://replit.com/

[263] MiniMax Code, "Introducing MiniMax Code Mini App | MiniMax," 2026, accessed 28 October 2026. [Online]. Available: https://www.minimax.io/blog/minimax-code-mini-app-1792650729

[264] Tabnine Agent / Plugin for OpenCode, "Tabnine Agent | Tabnine Docs," 2026, accessed 28 October 2026. [Online]. Available: https://docs.tabnine.com/main/getting-started/tabnine-agent

[265] Tabnine, "Tabnine Plugin for OpenCode," 2026, accessed 29 October 2026. [Online]. Available: https://docs.tabnine.com/main/getting-started/tabnine-plugin-for-opencode

[266] v0, "v0 by Vercel - Build Full-Stack Web Apps with AI," 2026, accessed 28 October 2026. [Online]. Available: https://v0.app/

[267] CodeBuddy, "Subagent," 2026, accessed 28 October 2026. [Online]. Available: https://intl.cloud.tencent.com/ind/document/product/1316/84444

[268] TRAE / SOLO, "Introducing The New SOLO | TRAE - Collaborate with Intelligence," 2026, accessed 28 October 2026. [Online]. Available: https://www.trae.ai/blog/new_solo_beta_0331

[269] ZCode, "ZCode Docs | GLM-5.3 Agentic Coding Guide," 2026, accessed 28 October 2026. [Online]. Available: https://zcode.z.ai/en/docs/welcome

[270] Letta Contributors, "Letta Code Source and Backend Documentation," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/letta-ai/letta-code/tree/2cc372d61a7ea7363f48229dc3c52752347cfd6d

[271] OpenAI, "Codex TypeScript SDK Source," 2026, accessed 29 October 2026. [Online]. Available:

https://github.com/openai/codex/tree/23934d31b15b1127c83535f84854e5009f7cd51d/sdk/typescript/src

[272] DeerFlow, "GitHub - bytedance/deer-flow: An open-source long-horizon SuperAgent harness that researches, codes, and creates. With the help of sandboxes, memories, tools, skill, subagents and message gateway, it handles different levels of tasks that could take minutes to hours. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/bytedance/deer-flow

[273] Asta, "Asta Agents: AI Tools for Scientific Research," 2026, accessed 28 October 2026. [Online]. Available: https://allenai.org/asta/agents

[274] J. Miao, J. R. Davis, Y. Zhang, J. K. Pritchard, and J. Zou, "Reimagining research papers as interactive and reliable AI agents," *Nature*, vol. 658, pp. 1272–1280, 2026, accessed 29 October 2026. [Online]. Available: https://www.nature.com/articles/s41586-026-11044-y

[275] Y. Oikawa, K. Endo, T. Uzawa, Y. Zhang, M. Anjaria, L. Pinto, S. Yang, and K. Tsuda, "WetRobo: A reproducible robot kit for coding agents in biological laboratories," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2609.18435v1

[276] Paper2Agent contributors, "Paper2Agent verification interfaces," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/jmiao24/Paper2Agent/tree/8c2d059165ef8cdcb70dbea76655b9c2b55b38e6

[277] Z. Cutts, J. Coller, R. Chen *et al.*, "Aptamers as a sandbox for lab-in-the-loop agentic molecular discovery," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2610.24806v1

[278] Y. Wen and H. Liu, "Autonomous science bench: Evaluating AI agents for autonomous scientific discovery," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2610.18449v1

[279] Agentforce, "Agentforce: The AI Agent Platform | Salesforce IN," 2026, accessed 28 October 2026. [Online]. Available: https://www.salesforce.com/in/agentforce/

[280] AI Agent Studio / Otto AI Agents, "Docs | ServiceNow," 2026, accessed 28 October 2026. [Online]. Available:

https://www.servicenow.com/docs/r/intelligent-experiences/ai-agent-studio.html

[281] Joule Studio, "AI Release Highlights Q3 2026 | SAP News Center," 2026, accessed 28 October 2026. [Online]. Available: https://news.sap.com/2026/10/ai-release-highlights-q3-2026/

[282] Copilot Studio, "What's new in Copilot Studio - Microsoft Copilot Studio | Microsoft Learn," 2026, accessed 28 October 2026. [Online]. Available: https://learn.microsoft.com/en-us/microsoft-copilot-studio/whats-new

[283] Mem0, "Mem0 - AI Memory Layer for your Agents & Apps | Persistent Context," 2026, accessed 28 October 2026. [Online]. Available: https://mem0.ai/

[284] Graphiti / Zep, "Graphiti | Zep," 2026, accessed 28 October 2026. [Online]. Available: https://www.getzep.com/platform/graphiti/

[285] Hindsight, "Overview | Hindsight," 2026, accessed 28 October 2026. [Online]. Available: https://hindsight.vectorize.io/

[286] MemOS, "INTELLIGENCE BEGINS WITH MEMORY," 2026, accessed 28 October 2026. [Online]. Available: https://memos.openmem.net/

[287] OpenViking, "GitHub - volcengine/OpenViking: Self-evolving Context Database for AI Agents. Unify Agent Memory, Knowledge RAG and Skills. · GitHub," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/volcengine/OpenViking

[288] watsonx Orchestrate, "IBM watsonx Orchestrate," 2026, accessed 28 October 2026. [Online]. Available: https://www.ibm.com/products/watsonx-orchestrate

[289] ChatGPT Work, "ChatGPT is now a partner for your most ambitious work | OpenAI," 2026, accessed 28 October 2026. [Online]. Available: https://openai.com/index/chatgpt-for-your-most-ambitious-work/

[290] Claude: work and Research, "Claude Cowork and chat are now one Claude | Claude by Anthropic," 2026, accessed 28 October 2026. [Online]. Available: https://claude.com/resources/articles/cowork-is-now-claude

[291] AI by Zapier, "Migrating from Agents to AI by Zapier – Zapier," 2026, accessed 28 October 2026. [Online]. Available: https://help.zapier.com/hc/en-us/articles/47402591569805-Migrating-from-Agents-to-AI-by-Zapier

[292] Cortex Agents, "Cortex Agents | Snowflake Documentation," 2026, accessed 28 October 2026. [Online]. Available: https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-agents

[293] Gemini Deep Research / Deep Research Max, "Introducing Deep Research and Deep Research Max," 2026, accessed 28 October 2026. [Online]. Available: https://blog.google/innovation-and-ai/models-and-research/gemini-models/next-generation-gemini-deep-research/

[294] Glean Agents, "Building Agents with Glean | Glean Developer," 2026, accessed 28 October 2026. [Online]. Available: https://developers.glean.com/guides/agents/overview

[295] Manus, "Introducing Manus 2.0," 2026, accessed 28 October 2026. [Online]. Available: https://manus.im/en/blog/introducing-manus-2-0

[296] Perplexity Computer, "Deep Research, command panel, and enterprise controls," 2026, accessed 28 October 2026. [Online]. Available: https://www.perplexity.ai/changelog/deep-research-command-panel-forking-inline-actions-and-enterprise-controls

[297] Rovo Agents, "Agents | Rovo | Atlassian Support," 2026, accessed 28 October 2026. [Online]. Available: https://support.atlassian.com/rovo/docs/agents/

[298] North / Agent Studio, "Automate and Accelerate Work with AI Agents | Cohere," 2026, accessed 28 October 2026. [Online]. Available: https://cohere.com/north/agent-studio

[299] Notion Custom Agents, "Custom Agents in Notion | Notion Help," 2026, accessed 28 October 2026. [Online]. Available: https://www.notion.com/help/custom-agents

[300] Dust, "Dust AI Platform: Build Custom AI Agents for Your Organization," 2026, accessed 28 October 2026. [Online]. Available: https://dust.tt/home/solutions/dust-platform

[301] Kimi Agent, "Kimi Agent Overview - Kimi Help Center," 2026, accessed 28 October 2026. [Online]. Available: https://www.kimi.ai/help/agent/agent-overview

[302] Harvey Agents / Agent Builder, "Harvey Agents | Legal AI for Workflow Automation," 2026, accessed 28 October 2026. [Online]. Available: https://www.harvey.ai/platform/agents

[303] Sierra, "Better customer experiences | Sierra," 2026, accessed 28 October 2026. [Online]. Available: https://sierra.ai/

[304] WRITER Agent, "What's new at WRITER | WRITER Knowledge Base," 2026, accessed 28 October 2026. [Online]. Available: https://support.writer.com/articles/1313908954-what-s-new-at-writer

[305] Grok Bot, "Grok Bot | SpaceXSI Docs," 2026, accessed 28 October 2026. [Online]. Available: https://docs.x.ai/grok-bot/overview

[306] Copilot Researcher, "Use Model Council with Researcher in Microsoft Copilot | Microsoft Support," 2026, accessed 28 October 2026. [Online]. Available: https://support.microsoft.com/en-us/microsoft-365-copilot/use-model-council-with-researcher-in-microsoft-365-copilot

[307] Qianfan Agent Platform, "Qianfan Agent Platform official resource (retrieval unresolved)," 2026, accessed 28 October 2026. [Online]. Available: https://aca.bce.baidu.com/doc/qianfan/index.html

[308] Genspark Super Agent, "Super Agent | Genspark Help Center," 2026, accessed 28 October 2026. [Online]. Available: https://genspark.prod.sspark.ai/helpcenter/super-agent

[309] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and Editing Factual Associations in GPT," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2202.05262v5

[310] F. Zhang and N. Nanda, "Towards Best Practices of Activation Patching in Language Models: Metrics and Methods," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2309.16042v2

[311] A. Asiaee, "Certified interventional fidelity: Anytime-valid, adaptive evaluation of causal claims in mechanistic interpretability," in *Proceedings of the 42nd Conference on Uncertainty in Artificial Intelligence*, ser. Proceedings of Machine Learning Research, vol. 337,

2026, pp. 210–227, accessed 29 October 2026. [Online]. Available: https://proceedings.mlr.press/v337/asiaee26d.html

[312] T. Bricken, A. Templeton, J. Batson *et al.*, "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning," *Transformer Circuits Thread*, 2023, accessed 28 October 2026. [Online]. Available: https://transformer-circuits.pub/2023/monosemantic-features/index.html

[313] L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu, "Scaling and evaluating sparse autoencoders," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2406.04093v1

[314] A. Duzan and A. C. Stickland, "Chain-of-thought monitoring can be unreliable in implicit-influence settings," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2608.04735v1

[315] OpenAI, "Open sourcing monitorability evaluations," 2026, accessed 29 October 2026. [Online]. Available: https://alignment.openai.com/monitorability-evals

[316] ——, "Monitorability evaluations: Intervention metric implementation," 2026, accessed 29 October 2026. [Online]. Available: https://github.com/openai/monitorability-evals/blob/d14f181a0623f7949fdb2d8c3f15874115fa44ba/metric/intervention_gmean_metric.py

[317] ——, "GPT-6 Astra system card: Monitorability," 2026, accessed 29 October 2026. [Online]. Available: https://deploymentsafety.openai.com/gpt-6-astra/monitorability

[318] EleutherAI, "EleutherAI/lm-evaluation-harness," Versioned software repository, accessed 28 October 2026. [Online]. Available: https://github.com/EleutherAI/lm-evaluation-harness/tree/d6de81643928d653435c431bae19945d41d32520

[319] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2310.06770v3

[320] Q. Ye, Y. Li, Y. Pruksachatkun, J. Zhang, and C.-S. Wu, "On the Fragility of Self-Improving Agents: Variance, Task Order, and

Underspecification," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2608.18066v2

[321] E. Pakhomov and E. Nijkamp, "Bookkeeping, composition, or unreachable gold? reading MemoryAgentBench's conflict-resolution scores against a frozen last-write resolver," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.09193v1

[322] Z. Chen, X. Liu, H. Tong *et al.*, "DecodingTrust-Agent Platform (DTap): A Controllable and Interactive Red-Teaming Platform for AI Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2605.04808v1

[323] Z. Wang, N. Schiller, H. Li *et al.*, "ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?" 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2605.11086v1

[324] M. Andriushchenko, A. Souly, M. Dziemian *et al.*, "AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents," 2024, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2410.09024v3

[325] S. J. Lazer, K. Aryal, M. Gupta, and E. Bertino, "A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-case Prototypes," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2601.05293v1

[326] Z. Ying, X. Wu, H. Wu, X. Zheng, H. Cheng, X. Shi, and J. Guo, "SkillJack: Persistent skill backdoors in self-evolving agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2608.03509v2

[327] W. Xie, S. Guo, F. Zhang, T. Xia, X. Yang, L. Ma, J. Yan, and Q. Ren, "MemEvoBench: Benchmarking safety risks from memory misevolution in LLM agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2604.15774v2

[328] R. K. Sharma, L. Jiang, S. Chen, and Z. Lin, "Ajar: Measuring open privilege in agent defenses," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.26900v1

[329] E. Wallace, C. A. Choquette-Choo, N. Kandpal *et al.*, "GPT-Red: Automated red teaming via self-play at scale," 2026, accessed 28

October 2026. [Online]. Available: https://arxiv.org/abs/2607.26115v1

[330] SuperRed contributors, "SuperRed: Modular red-teaming framework," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/sunblaze-ucb/superred/tree/3a92348e7ff0a4c6a1a898a98bdbd470b952dc62

[331] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song, "CyberGym: Evaluating AI Agents' Real-World Cybersecurity Capabilities at Scale," 2025, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2506.02548v3

[332] T. Avgerinos, S. K. Cha, B. L. T. Hao, and D. Brumley, "AEG: Automatic exploit generation," in *Network and Distributed System Security Symposium*, 2011, accessed 28 October 2026. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2017/09/Avg.pdf

[333] S. K. Cha, T. Avgerinos, A. Rebert, and D. Brumley, "Unleashing Mayhem on binary code," in *2012 IEEE Symposium on Security and Privacy*, 2012, pp. 380–394. [Online]. Available: https://doi.org/10.1109/SP.2012.31

[334] V8 project contributors and O. Chang, "JS-Fuzzer: Javascript fuzzer for stand-alone shells," 2026, accessed 28 October 2026. [Online]. Available: https://chromium.googlesource.com/v8/v8/+/b4a4e29b030dde71ecd55a633ec4b77466bfdf6c/tools/clusterfuzz/js_fuzzer/README.md

[335] N. Bars, L. Bernhard, M. Schloegel, and T. Holz, "Empirical security analysis of software-based fault isolation through controlled fault injection," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 2639–2652. [Online]. Available: https://sefcom.asu.edu/publications/sbxbrk-ccs2025.pdf

[336] H.-M. Chen, X. He, S. Wang, X. Zhang, and K. Sun, "TYPEPULSE: Detecting type confusion bugs in Rust programs," in *34th USENIX Security Symposium*. USENIX Association, Aug. 2025, pp. 6957–6976. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/chen-hung-mao

[337] Y. Bai, S. Kadavath, S. Kundu *et al.*, "Constitutional AI: Harmlessness from AI Feedback," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2212.08073v1

[338] J. Zhang, H. Sleight, and J. Benton, "A3: An automated alignment agent for safety finetuning," 2026, accessed 29 October 2026. [Online]. Available: https://alignment.anthropic.com/2026/automated-alignment-agent/

[339] A3 contributors, "A3: Iterative safety finetuning and experiment log," 2025, accessed 29 October 2026. [Online]. Available: https://github.com/safety-research/A3/blob/c5c79e776fe6b8fc62865c55428b557be1435ef7/agent/iterative_sft_lora_agent.py

[340] M. Terekhov, C. Gulcehre, V. Hebbar, and J. Benton, "Diffuse AI control on fuzzy tasks," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2606.08892v2

[341] D. Moshkovich and S. Zeltyn, "Taming Uncertainty via Automation: Observing, Analyzing, and Optimizing Agentic AI Systems," 2025, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2507.11277v1

[342] L. Yan, "From Passive Tool to Socio-cognitive Teammate: A Conceptual Framework for Agentic AI in Human-AI Collaborative Learning," 2025, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2508.14825v1

[343] Google Research, "google-research/rrsi," accessed 28 October 2026. [Online]. Available: https://github.com/google-research/rrsi/tree/be50316e1db05914068a973f322770ef08ed7ba1

[344] Z. Wei, W. Yao, B. Peng, D. Wang, Q. Wu, H. Cheng, Y. Meng, and J. Gao, "Self-Evolving Coding Agents via Harness-Policy Optimization," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.22791v1

[345] H. Lee, J. Xu, J. Seely, D. Lee, S. Sojoudi, M. Zaharia, and Y. Tang, "Recursive Self-Improvement through Multi-Agent Self-Supervision," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2610.12176v1

[346] J. Yang, Z. Li, Y. Shi, Z. Li, R. Wang, R. Zhou, H. Mi, S. Yan, and L. Liang, "Terminal-RSI: Recursive self-improving terminal agents

via recursive self-rewrite and policy-refine," 2026, accessed 29 October 2026. [Online]. Available: https://arxiv.org/abs/2610.22192v2

[347] D. Srikanth, B. Zhao, D. Xu, Y. Wu, and Z. Jiang, "Recursive self-improvement of AI research agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2609.26457v1

[348] vLLM Team and Inferact, "vLLM x AgentX: Optimizing for Real-World Agentic Serving," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/vllm-project/vllm-project.github.io/blob/main/_posts/2026-09-08-vllm-agentx.md

[349] O. Ozeri, D. Harnik, R. Schaffer, I. Etelis, and V. S. Rabindranath, "Tiered KV Cache Offloading in vLLM," 2026, accessed 28 October 2026. [Online]. Available: https://github.com/vllm-project/vllm-project.github.io/blob/main/_posts/2026-09-10-tiered-kv-offloading.md

[350] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2210.17323v2

[351] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2211.10438v7

[352] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," 2022, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2211.17192v2

[353] Y. Li, Q. Ye, P. K. Choubey, J. Zhang, and C.-S. Wu, "Speculate with Memory: Lossless Acceleration for LLM Agents," 2026, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2607.12236v1

[354] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient Execution of Structured Language Model Programs," 2023, accessed 28 October 2026. [Online]. Available: https://arxiv.org/abs/2312.07104v2

[355] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action.* Cambridge University Press,

2013. [Online]. Available: https://www.cs.cmu.edu/~harchol/PerformanceModeling/book.html

[356] R. Fielding, M. Nottingham, and J. Reschke, "HTTP semantics," Internet Engineering Task Force, RFC 9110, Jun. 2022, accessed 29 October 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2